初稿A1（更新）

科学计算前四章 • G20、G8、G3、G14

整理日期：2026-11-06。主分支版本：`230873df074c`。本稿保留各组原有讲解、实验说明、习题与参考解答，以及尚未完成内容。仅整理标题层级、原稿目录、公式标记及相对路径。

目录

- 第1章 • G20
- 第2章 • G8
- 第3章 • G3
- 第4章 • G14

第01章　误差的代价

> 编写组：A1（G2O）；来源版本：`230873df074c`，main已合并稿。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

数值分析与科学计算引论

一个本应大于零的差额，计算机却算成了 0。取 $x = 10^{16}$，计算

$$d(x) = \sqrt{x+1} - \sqrt{x}, \qquad x > 0. \tag{1.1}$$

因为 $x + 1 > x$，这个差额在数学上严格大于零。然而，在本章的 `float64` 环境中，直接求值却得到 0。换用数学上等价的公式

$$d(x) = \frac{1}{\sqrt{x+1} + \sqrt{x}}, \tag{1.2}$$

则得到约 5×10^{-9} 的正数。

下面是本机 Python 交互解释器的实际运行记录（Windows CPU，Python 3.12.14；`float` 为 binary64）：

```
>>> import math
>>> x = float(10**16)
>>> math.sqrt(x + 1.0) - math.sqrt(x)
0.0
>>> 1.0 / (math.sqrt(x + 1.0) + math.sqrt(x))
5e-09
```

为什么一个正数会被算成 0？为什么换一个等价公式，结果就改善了？要回答这两个问题，需要了解计算机怎样表示数，误差怎样产生，以及误差在算法中怎样传播。本章围绕浮点数、条件数和稳定性展开，用差分求导、积分递推和分类损失计算逐步建立判断计算结果的依据。

本章的核心问题是：当计算机告诉你答案是 3.141592653589793 时，你凭什么相信它？屏幕上的位数只说明怎样显示结果；结果有多少准确位，还取决于输入数据、问题本身的敏感性与所用算法。

误差可能来自不同环节。测量误差和模型简化会使输入数据或数学模型偏离实际对象；把无限过程截成有限步骤会产生截断误差，用有限精度表示和运算会产生舍入误差。本章主要讨论：数学问题确定后，近似计算会引入多大误差，这些误差如何传播，以及怎样控制它们。

表 1.1　本章内容与学习目标

小节	主要问题	学习后应能完成的判断
1.1	浮点数与舍入误差	区分表示精度、显示位数与结果精度，选择合适的误差尺度
1.2	截断误差与灾难性抵消	推导误差项，解释步长与舍入的竞争，选择可靠的等价公式
1.3	条件数与稳定性	分清输入扰动的固有放大与算法造成的额外放大

小节	主要问题	学习后应能完成的判断
1.4	AI关联：训练中的数值问题	改写 Softmax 与交叉熵，解释低精度存储与梯度缩放

本章用到实数运算、导数、Taylor 展开与向量范数。四节各两学时，共八学时；标为“选读”的推导与附录供进一步学习，不计入八学时的最低完成要求。实验提供可直接运行的配套 Notebook；首次使用 Python 的读者可先观察输出，再学习修改输入。关键数组操作会随代码解释，无须预先学习机器学习。

1.1　浮点数与舍入误差（2学时）

计算机处理的是有限集合中的机器数，数学分析处理的通常是实数。输入十进制小数、计算函数值或累加数组时，都可能产生表示误差或舍入误差。本节从表示方式出发，说明怎样根据数值大小选择合适的误差衡量方法。

实数怎样成为机器数

一个非零的规格化二进制浮点数可写为

$$x = (-1)^s (1.b_1 b_2 \cdots b_{p-1})_2 \, 2^e. \tag{1.3}$$

这里，$s \in \{0,1\}$ 决定符号，指数 e 决定数量级，p 是有效二进制位数，括号内的有效数决定该数量级内能区分的相对位置。规格化数的首位 1 通常不需要显式存储。因此，`float64` 的 52 个显式小数位对应 53 位有效二进制精度，而非“64 位有效数字”。

在固定指数区间 $[2^e, 2^{e+1})$ 中，相邻的规格化浮点数的间距为

$$\Delta_e = 2^{e-p+1}. \tag{1.4}$$

指数增加 1，间距也扩大 2 倍。这意味着浮点数的分布并非均匀：在 1 附近能分辨的绝对变化，与在 10^{16} 附近能分辨的绝对变化不同。

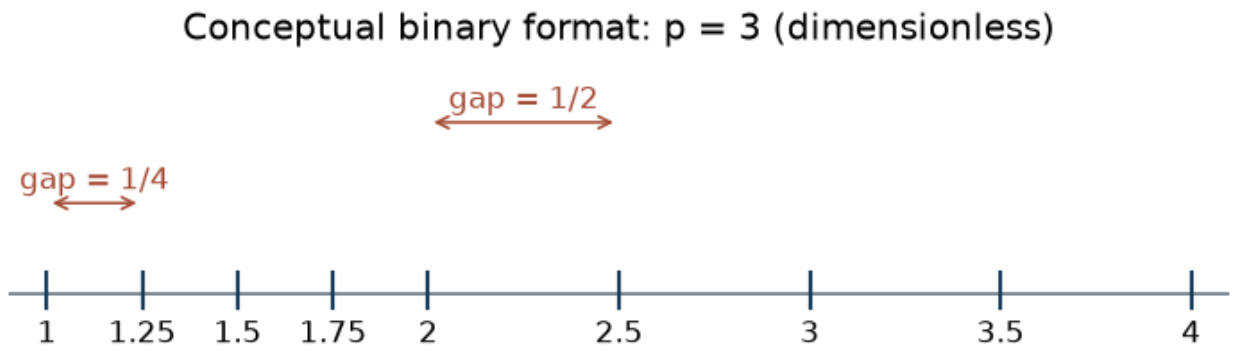


图 1.1　用 $p = 3$ 的假想二进制格式说明间距随指数变化。图为概念示意，不是 `float32` 或 `float64` 的完整数轴。

舍入后的数与准确数相差多少，需要选定衡量尺度。例如，用 101 近似 100，绝对误差为 1，相对误差为 $1/100 = 1\%$。若同样相差 1，但准确值是 10000，相对误差就只有 0.01%。下面正式定义这两个量。

定义 1.1　误差与误差限

准确值为 x，近似值为 $\hat{x}$ 时，定义带符号的误差

$$e = \widehat{x} - x.$$

其大小为 $E_{\mathrm{abs}} = |e|$。若已知 $|e| \le \varepsilon$，则 $\varepsilon \ge 0$ 是绝对误差限。对于 $x \neq 0$，相对误差的大小为

$$E_{\mathrm{rel}} = \frac{|\widehat{x} - x|}{|x|}.$$

部分教材把带符号的 e 称为“绝对误差”，另一些资料直接以 $|e|$ 定义绝对误差。本章在代数推导中用 e 表示带符号误差，在数值表中报告非负的 E_{abs} 与 E_{rel}，以免混淆。

例 1.1　从十进制 0.1 到二进制舍入

约分后的有理数能用有限二进制小数精确表示，当且仅当其分母是 2 的幂。由于 $1/10$ 的分母含因子 5，其二进制表示无限循环：

$$0.1 = (0.0001100110011\cdots)_2 = (1.100110011\cdots)_2\, 2^{-4}.$$

在只有 $p = 5$ 位有效精度的假想格式中，两个相邻候选数是

$$(1.1001)_2\, 2^{-4} = 0.09765625, \qquad (1.1010)_2\, 2^{-4} = 0.1015625.$$

舍入到最近数时应选择后者，其绝对误差为 0.0015625，相对误差为 0.015625。这里的位数特意取小，以便手算；实际的 `float64` 使用更多有效位，但无法改变 0.1 不能被有限二进制精确表示这一事实。

一个十进制输出也可能只是机器数的简洁表示。Python 显示 `0.1`，不意味着存储值恰好等于 $1/10$。例如，可用 `as_integer_ratio()` 读取准确的有理表示；也可用 `Decimal.from_float()` 查看存储值的十进制展开。屏幕上打印了多少位，不能用来判断结果的实际误差6。

机器精度与舍入模型

现在把相对误差用于一次舍入。若 $t \neq 0$，记 $\delta = (\mathrm{fl}(t) - t)/t$，就有 $\mathrm{fl}(t) = t(1+\delta)$；$|\delta|$ 正是这次舍入的相对误差。单位舍入误差 u 将为它提供统一上界。

本章采用以下舍入规则：舍入到最近的浮点数；恰处于中点时，选择有效数最低位为 0 的数，即舍入到偶数。对有效精度为 p 的二进制格式，定义单位舍入误差

$$u = 2^{-p}. \tag{1.5}$$

而软件中常称为机器 epsilon 的量

$$\epsilon_{\mathrm{mach}} = \mathrm{nextafter}(1, +\infty) - 1 = 2^{1-p} = 2u \tag{1.6}$$

是 1 与其右侧相邻浮点数的间距。不同资料对“机器精度”的命名可能不同，因此应先明确这里指 u 还是 ϵ_{mach}。NumPy 的 `finfo(dtype).eps` 采用式 (1.6) 的约定5。

表 1.2　常用格式的有效精度与数值范围

格式	p	u	最小规格化正数	最大有限正数
binary32	24	5.9605×10^{-8}	1.1755×10^{-38}	3.4028×10^{38}
binary64	53	1.1102×10^{-16}	2.2251×10^{-308}	1.7977×10^{308}

binary32 与 binary64 分别使用 1 位符号、8 或 11 位指数、23 或 52 位显式小数。有效精度约对应 7 位与 16 位十进制数字，但这是表示能力的量级，不能直接当作任意算法结果的有效位数。

命题 1.1　舍入到最近数的局部相对误差模型

若实数 $t \neq 0$ 落在可用规格化数正常舍入的范围内，且没有上溢，则

$$\mathrm{fl}(t) = t(1+\delta), \qquad |\delta| \le u. \tag{1.7}$$

对按上述规则正确舍入的基本运算，可相应写为

$$\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta), \qquad |\delta| \le u, \quad \circ \in \{+, -, \times, \div\}, \tag{1.8}$$

其中除法要求 $b \neq 0$，精确结果须处于该模型适用的数值范围。

证明。 在 $[2^e, 2^{e+1})$ 内，相邻数间距是 Δ_e。最近舍入的绝对误差不超过 $\Delta_e/2$，而 $|t| \ge 2^e$，故

$$\frac{|\,\mathrm{fl}(t) - t|}{|t|} \le \frac{2^{e-p}}{2^e} = 2^{-p} = u.$$

基本运算的模型是把运算的精确实数结果作为 t 再舍入；该结论描述这一次运算引入的新误差，不包含操作数此前已有的误差。

为了表示更接近零的数，浮点格式还设置了非规格化数，填补零与最小规格化正数之间的空隙。它们不再以式 (1.3) 中的首位 1 规格化，且相邻数的绝对间距固定。在这个区域内，应改用绝对误差界，不能直接沿用命题的统一相对上界。结果超出最大有限数时可能上溢为 `inf`；`inf`、`NaN` 与未定义运算也不属于式 (1.8) 的范围。

格式说明：binary32 与 binary64 的最小正非规格化数分别为 2^{-149} 与 2^{-1074}；在渐进下溢和最近舍入下，绝对误差至多为相应间距的一半。

对 $\exp$、$\sin$ 等初等函数，不能直接把式 (1.8) 当作函数库的精度保证。若已知某次求值满足 $|\widehat{f}(t) - f(t)| \le \varepsilon_f$，应把这个绝对误差界代入后续传播分析；当 $f(t) \neq 0$ 且存在适当相对界时，也可写成 $\widehat{f}(t) = f(t)(1+\delta_f)$。这里的 δ_f 由具体函数实现决定，不自动等于基本运算模型中的 u。上述范围限制及误差模型的来源为文献 2、3。

误差限与有效数字

真值未知时，常以 $|\widehat{x}|$ 代替 $|x|$ 作为相对误差分母。这个替换只有在误差较小且近似值远离零时才合理。如果 $|\widehat{x} - x| \le \varepsilon < |\widehat{x}|$，则严格地有

$$\frac{|\widehat{x} - x|}{|x|} \le \frac{\varepsilon}{|\widehat{x}| - \varepsilon}. \tag{1.9}$$

当 $\varepsilon/|\widehat{x}| \ll 1$ 时，右侧与 $\varepsilon/|\widehat{x}|$ 的差才是高阶量。真值为零时，通常应报告绝对误差；也可根据问题选择固定尺度 $S > 0$，报告 $|\widehat{x} - x|/S$，但须明确 S 的含义。

例如，$x = 10 \pm 1$ 与 $y = 1000 \pm 5$ 中，后者的绝对误差限较大，相对于其数值尺度却更精确。比较精度时，既要看绝对误差，也要看它相对于数值本身有多大。长度误差有长度量纲，相对误差则无量纲。

若把一个准确数按 10^{-4} 为单位四舍五入，得到 8.0000，则舍入误差不超过 0.00005，该近似数按下文定义至少具有五位有效数字。若 8.0000 是测量记录，则还需要计入测量本身的误差。设真实值为 x，舍入前的测量值为 v，记录值为 $\hat{v}$，则

$$|\hat{v} - x| \le |\hat{v} - v| + |v - x|.$$

记录的四舍五入只控制第一项，第二项需要根据仪器精度等测量信息估计；实际有效数字应由总误差判断。删去末尾的零、写成 8，会丢失原记录所表达的精度信息；仅将计算结果格式化为 8.0000，也不能确定它的实际误差。

定义 1.2　有效数字

把非零近似数写为 $\hat{x} = \pm a\,10^{m}$，其中 $1 \le a < 10$。若

$$|\hat{x} - x| \le \tfrac{1}{2}\,10^{m-n+1}, \tag{1.10}$$

就称该近似数至少具有 n 位有效数字。

有效位数从第一个非零数字起计，与小数点后的位数不同；它必须有误差界支持，不能只由显示格式决定。

用 3.14 近似 π，误差约为 0.00159265。此时 $m = 0$；第三位有效数字在百分位，其半个单位是 0.005，大于当前误差，所以按定义至少有三位有效数字。第四位在千分位，其半个单位是 0.0005，小于当前误差，因此不能判为至少四位。判定时要比较准确误差或可信误差限与“末位半个单位”，而非只数打印了几位。

选读：相对误差限怎样保证有效数字

设 $\hat{x}$ 的首个非零十进制数字为 a_1。由

$$a_1\,10^{m} \le |\hat{x}| < (a_1+1)\,10^{m}$$

可得，以近似值为分母时，式（1.10）蕴含

$$\frac{E_{\mathrm{abs}}}{|\hat{x}|} \le \frac{1}{2a_1}\,10^{1-n}. \tag{1.11}$$

反过来，一个保证至少 n 位有效数字的充分条件是

$$\frac{\varepsilon}{|\hat{x}|} \le \frac{1}{2(a_1+1)}\,10^{1-n}. \tag{1.12}$$

式（1.11）是由有效数字定义推出的必要条件；式（1.12）则用已知误差限保证有效数字。两式都以 $|\hat{x}|$ 为分母，右端的 a_1 与 a_1+1 分别来自 $|\hat{x}|$ 的下界与上界，因此不能直接互换。有效数字的判定最终应回到末位半个单位的定义1。

从输入误差估计输出误差

设函数 f 在包含 x 与 $\hat{x}$ 的区间上连续可微，由中值定理，

$$|f(\hat{x}) - f(x)| \le M_1 |\hat{x} - x|, \qquad M_1 = \sup_{\xi} |f'(\xi)|. \tag{1.13}$$

这是严格的上界。如果 f 具有连续二阶导数，则在近似值处展开可写为

$$f(x) - f(\hat{x}) = f'(\hat{x})(x - \hat{x}) + \tfrac{1}{2} f''(\xi)(x - \hat{x})^2. \tag{1.14}$$

扰动足够小且余项可忽略时，得到一阶估计

$$\varepsilon_f \approx |f'(\hat{x})|\,\varepsilon_x.$$

若 $f'(\hat{x}) = 0$，一阶项消失，此时应考察二阶项及更高阶项。

例 1.2　长方形面积的误差估计

长方形测量值为 $\hat{l} = 110\,\mathrm{m}$、$\hat{d} = 80\,\mathrm{m}$，误差限分别为 $0.2\,\mathrm{m}$ 与 $0.1\,\mathrm{m}$。求面积误差的一阶估计与严格上界。

解：近似面积为 $8800\,\mathrm{m}^2$，一阶估计为

$$\varepsilon_s \approx 80(0.2) + 110(0.1) = 27\,\mathrm{m}^2.$$

长度误差对面积的影响约为 80×0.2，宽度误差的影响约为 110×0.1。写 $l = \hat{l} + \Delta l$、$d = \hat{d} + \Delta d$，展开后还会出现乘积项 $\Delta l \Delta d$，其绝对值不超过 $0.02\,\mathrm{m}^2$。因此严格界为

$$|ld - \hat{l}\hat{d}| \le 27 + 0.2(0.1) = 27.02\,\mathrm{m}^2.$$

一阶估计 $27\,\mathrm{m}^2$ 省略了这一较小的乘积项，而 $27.02\,\mathrm{m}^2$ 把它也计入了上界。

例题出处：测量数据源自文献 1 §1.2.3 例1.4；本例同时给出一阶估计与严格界。

这个例子中的两个系数 80 和 110，分别是面积对长度和宽度的偏导数。对一般函数 $A = f(x_1, \dots, x_n)$，记 $\Delta A = f(\boldsymbol{x}) - f(\hat{\boldsymbol{x}})$，各输入误差限为 $|x_j - \hat{x}_j| \le \varepsilon_j$。若 f 在近似输入附近具有连续二阶偏导数，输入扰动足够小时，省略高阶项得到

$$|\Delta A| \lesssim \sum_{j=1}^{n} \left| \frac{\partial f}{\partial x_j}(\hat{\boldsymbol{x}}) \right| \varepsilon_j. \tag{1.15}$$

每个偏导数衡量对应输入的单位变化会使输出改变多少；取绝对值再求和，估计各项误差共同作用的最坏情形。符号 $\lesssim$ 在这里表示忽略高阶项的一阶估计。若要得到严格界，可在连接准确输入与近似输入的线段上，对每个偏导数取绝对值上确界。式 (1.15) 估计输入误差的传播，运算舍入另行计入。

实验一：表示误差与求和次序

完整环境和运行方法见章末实验说明，本实验见配套 Notebook §1.1。下面分别检查十进制输入与求和次序。

（1）十进制输入与加法。先预测：准确十进制数 0.3 与 `float32`、`float64` 计算的 $0.1 + 0.2$ 会完全相等吗？

`dtype` 是本次实验选定的浮点类型，`dtype(value)` 把值转换成该类型；下面用显式转换控制输入与加法的工作精度。程序输出两种类型的存储结果，对应表 1.3。

误差表以准确十进制 0.3 为参考。先将输出转为 Python 的 `float`，再用 `Decimal.from_float` 读取其准确存储值 v；参考值 r 由 `Decimal("0.3")` 建立。在足够高的十进制精度下计算 $E_{\mathrm{abs}} = |v - r|$ 和 $E_{\mathrm{rel}} = |v - r|/|r|$。对 binary32，转为 Python 的 binary64 不改变其存储值。`Decimal.from_float` 读取实际存储的机器数，`Decimal("0.3")` 从字符串建立准确十进制参考；两种构造不能互换。Notebook 保存这一参考计算的可运行版本。

```
import numpy as np

for dtype in [np.float32, np.float64]:
    y = dtype(dtype("0.1") + dtype("0.2"))
    print(dtype.__name__, float(y))
```

表 1.3 $0.1 + 0.2$ 的实际存储结果及其误差（参考为准确十进制 0.3）

类型	存储值的17位表示	E_{abs}	E_{rel}
float32	0.30000001192092896	1.192×10^{-8}	3.974×10^{-8}

类型	存储值的17位表示		
float64	0.30000000000000004	4.441×10^{-17}	1.480×10^{-16}

表 1.3 的差异来自输入表示与加法舍入的共同作用。这里 float32 的结果并非“只显示了几位”，而是其真实存储值与 0.3 有可测的差距。另一方面，float64 的误差虽很小，也不是严格的零。

（2）求和次序。再预测：在 2^p 上连续加 128 个 1，与先把这些 1 相加再加到 2^p，会产生相同的增量吗？

下面程序每行依次输出“大量在前”“小量在前”和 fsum 的增量，对应表 1.4。先比较前两列；a[::-1] 将列表次序倒转，使 128 个小量先相加。函数 sequential 在每次加法后都转换到指定类型。

```
import math
import numpy as np

def sequential(values, dtype):
    total = dtype(0)
    for value in values:
        total = dtype(total + dtype(value))
    return total

for dtype, p in [(np.float32, 24), (np.float64, 53)]:
    big = 2**p
    a = [big] + [1] * 128
    print(dtype.__name__,
          float(sequential(a, dtype)) - big,
          float(sequential(a[::-1], dtype)) - big,
          math.fsum(a) - big)
```

表 1.4　固定精度逐项累加的实际增量（准确增量为128）

类型	起始大量	大量在前	小量在前	fsum (binary64)
float32	2^{24}	0	128	128
float64	2^{53}	0	128	128

在 2^p 的右侧，间距恰为 2。加 1 后的精确结果位于两个机器数的中点；按舍入到偶数的规则，当前累加器回到 2^p。先累加 128 个 1，再与大量相加时，128 本身可精确表示，加到 2^p 后也仍是机器数。因此，逐项加入的 1 被舍入丢掉，先累加则能保留 128 的增量。

表 1.4 中前两种累加使用当前行指定的精度；math.fsum 使用 Python 的 binary64 浮点数。实验中 1 右侧的实际间距分别为 $1.1920928955 \times 10^{-7}$ 与 $2.2204460493 \times 10^{-16}$，与表 1.2 的 $2u$ 一致。

math.fsum 不只保存一个不断舍入的总和，而是跟踪多个中间部分和，保留普通累加容易丢失的低位信息，最后再合并6。这解释了它为何能保留本例的增量；它求的是输入浮点数的和，无法恢复输入转换时已经丢失的信息。

讨论与自检

在本章的 binary64 环境中，将可精确表示的整数 10^{16} 与 1 相加，结果仍为 10^{16}。用相邻浮点数的间距解释这一结果，并以准确整数之和为参考，分别计算这一次加法的绝对误差与相对误差。再手算 $(0.1101)_2$ 的十进制值，说明它为何能被两种精度准确表示。

迁移自检：假想二进制格式取 $p=8$，采用最近舍入、遇中点取偶数。对 $x=2^7$ 与 $x=2^8$，分别手算 $\mathrm{fl}(x+1)$；再判断 $\mathrm{fl}(2^8+3)$。先标出相邻机器数及中点，再解释舍入的选择，不能只比较增量与 u。

本节讨论的提示见章末“讨论与自检提示”。只读主线可直接转至 § 1.2；下面两项推导均为选读。

选读：逐项求和的误差界

逐项累加的误差还可以定量估计。记 $\gamma_r = ru/(1-ru)$（$ru<1$），并约定 $\gamma_0=0$。单步误差因子的连乘可用这个量控制，证明见本节末的选读。对已准确存储的 $a_1,\dots,a_n$，从 $\hat{s}_1=a_1$ 开始执行 $\hat{s}_k=\mathrm{fl}(\hat{s}_{k-1}+a_k)$。在每步都符合舍入模型且 $(n-1)u<1$ 时，展开递推，每个输入前的系数都是至多 $n-1$ 个 $1+\delta_j$ 的乘积，故

$$|\hat{s}_n-\sum_{i=1}^{n}a_i|\le\gamma_{n-1}\sum_{i=1}^{n}|a_i|.$$

若准确和非零，相对误差界还要乘以 $\sum_i|a_i|/|\sum_i a_i|$；正负项强烈抵消时，这个比值可能很大。输入本身的表示误差不包含在上述界内，需要另行计入。求和误差分析的出处为文献 2 第4章。

选读：多次舍入的连乘界

连续执行多次运算时，单步误差会通过乘积累积。设 $|\delta_j|\le u$、$ru<1$，记

$$\prod_{j=1}^{r}(1+\delta_j)=1+\theta_r,\qquad \gamma_r=\frac{ru}{1-ru}.$$

则 $|\theta_r|\le\gamma_r$。这一界是文献 2 § 3.1 引理3.1的一个特例。事实上，上侧可由 $(1+u)^r\le(1-u)^{-r}\le(1-ru)^{-1}$ 控制，下侧由 $(1-u)^r\ge1-ru$ 控制，其中后一不等式是 Bernoulli 不等式。当 $ru\ll1$ 时，$\gamma_r\approx ru$；它说明误差界还取决于运算次数，不能把一次运算的 u 直接当作整个算法的误差界。

1.2　截断误差与灾难性抵消（2学时）

舍入误差来自有限精度，截断误差来自有限步骤。二者常在同一算法中相互竞争：提高离散化精度可能增加运算成本，也可能把舍入误差放大。理解这种竞争，需要既推导连续数学中的余项，也分析计算机实际执行的操作。

截断误差是怎样产生的

设 f 在展开区间上具有所需阶数的连续导数。用有限 Taylor 多项式代替 f 时，

$$f(x)=\sum_{k=0}^{n}\frac{f^{(k)}(0)}{k!}x^k+\frac{f^{(n+1)}(\xi)}{(n+1)!}x^{n+1}. \tag{1.16}$$

舍去最后一项所产生的误差就是截断误差。它在精确实数运算中已经存在，与是否使用计算机无关。实际计算该多项式时，还会叠加各次运算的舍入。

另一种更能体现误差竞争的例子是数值求导。对于 $h>0$，以前向差分

$$D_h f(x)=\frac{f(x+h)-f(x)}{h} \tag{1.17}$$

近似 $f'(x)$。若 $f\in C^2$，Taylor 公式给出

$$D_h f(x)-f'(x)=\frac{h}{2}f''(\xi_h),\qquad \xi_h\in(x,x+h). \tag{1.18}$$

因此当 $|f''|\le M_2$ 时，截断误差不超过 $M_2h/2$；若 f 再具有连续三阶导数，

$$D_h f(x)=f'(x)+\tfrac{1}{2}hf''(x)+O(h^2).$$

这里 $O(h^2)$ 表示当 h 足够小时，余项的绝对值可由某个与 h 无关的常数乘 h^2 控制；它给出量级，不给出某次计算的精确误差。后文 $o(|\Delta x|)$ 表示余项除以 $|\Delta x|$ 后趋于零，$a\sim b$ 表示在指明的极限下 $a/b\to 1$。这些记号各有含义，不能都读成“约等于”。

在精确运算中，前向差分的截断误差在 h 趋于零时趋于零；在上述导数界下，其上界随 h 线性减小。实际误差不必在任意两个步长间单调下降。然而，实际算法还需计算两个接近的函数值再相减。

相减为何会暴露已有误差

设要计算 $a-b$，却只掌握 $\hat a=a+\Delta a$ 与 $\hat b=b+\Delta b$。先暂时忽略最后一次减法自身的舍入，则

$$(\hat a-\hat b)-(a-b)=\Delta a-\Delta b.$$

若 $|\Delta a|\le\eta|a|$、$|\Delta b|\le\eta|b|$，且 $a\neq b$，有

$$\frac{|(\hat a-\hat b)-(a-b)|}{|a-b|}\le\eta\,\frac{|a|+|b|}{|a-b|}. \tag{1.19}$$

当 a 与 b 同号且十分接近时，分母很小。即使它们各自的相对误差很小，差值的相对误差也可能很大。灾难性抵消的关键是：差值很小，而差值要承受操作数此前已有的绝对误差。

抵消的影响取决于操作数已有的误差。例如，$1.5-1=0.5$ 中，两个操作数及其差都能被本章使用的二进制格式精确表示，这次减法就是准确的。分析抵消时，应沿求值过程追溯误差从何处进入[3]。

回到式 (1.1)。当 x 很大时，

$$\sqrt{x+1}+\sqrt{x}\sim 2\sqrt{x},\qquad d(x)\sim\frac{1}{2\sqrt{x}}.$$

若两次平方根的相对求值误差为 $O(u)$，那么它们的绝对误差约为 $O(u\sqrt{x})$。相减后除以差值尺度，得到的相对误差量级可能达到

$$O(ux). \tag{1.20}$$

这里 $\sim$ 指 $x\to\infty$ 时的渐近等价，$O(u)$、$O(ux)$ 描述上述误差模型中的量级。它们本身不确定某次运行的误差常数。

这一估计说明了两次平方根求值误差的影响；$x+1$ 的舍入也可能使差额消失。采用式 (1.2) 后，分母是两个同号正数之和，避免了相减造成的额外放大。在平方根、加法和除法正常计算且误差为 $O(u)$ 的条件下，最终相对误差也可保持在 $O(u)$ 的量级。

例 1.3　小根的可靠计算

求方程 $x^2-16x+1=0$ 的小正根。若开方结果只保留为 $\sqrt{63}\approx 7.94$，怎样更好地利用这个近似值？

解：方程的两个根为 $8\pm\sqrt{63}$。若仅知道 $\sqrt{63}\approx 7.94$，直接计算小根得到 0.06。利用两根乘积为 1，

$$x_{\text{small}}=\frac{1}{8+\sqrt{63}}\approx\frac{1}{15.94}\approx 0.0627.$$

后一个公式避免两个接近的数相减，更好地利用了给定开方值的精度。准确小根约为 0.062746067；以它为参考，直接式 0.06 的相对误差约为 4.38%，改写式 $1/15.94$ 的相对误差约为 0.0172%。后一误差按未进一步缩写的 $1/15.94$ 计算。两种方法使用同一个开方近似，改善来自不同的误差传播。

例题出处：方程与开方近似源自文献 1 §1.3.3 例1.7。

类似的改写包括 $1-\cos t=2\sin^2(t/2)$。本章 $\log$ 均表示自然对数，与 $\ln$ 同义。若已知小增量 t，计算 $\log(1+t)$ 可选用专门的 `log1p(t)`，以避免先形成 $1+t$ 时丢失小量。改写时应留意新的中间量：例如把 $\log x-\log y$ 写成 $\log(x/y)$，仍需分析比值的舍入和对数求值的误差。

差分步长的两种代价

设计算 $f(x)$ 与 $f(x+h)$ 时引入的绝对求值误差分别为 r_0、r_1。忽略更高阶项时，差分误差中出现

$$\frac{r_1-r_0}{h}.$$

即使 $|r_i|$ 很小，除以过小的 h 也会放大它们。现在固定 x，假设其附近二阶导数有界，函数求值与输入舍入造成的分子误差具有 $O(u)$ 量级，且运算处于正常浮点范围内。计入减法和除法的误差，忽略高阶项后，可把绝对误差估计为

$$|\widehat{D}_h f(x)-f'(x)|\lesssim C_t h+C_r\frac{u}{h},\tag{1.21}$$

其中 C_t 与二阶导数有关，C_r 取决于函数值、输入尺度与实际求值过程。截断项随 h 减小而降低，舍入项则因除以 h 而增大。

若 $C_t,C_r>0$ 且此模型适用，最小化右侧得到

$$h_{\text{bal}}=\sqrt{\frac{C_r}{C_t}}\,u.\tag{1.22}$$

当函数与输入尺度固定时，平衡步长与 $\sqrt{u}$ 同量级。因此，步长需要兼顾截断误差与舍入误差；实验中的误差曲线将展示这种竞争。

注：式 (1.21) 是忽略高阶项的估计。C_t 与 C_r 带有使两项量纲一致的单位，选择步长时应保留它们所反映的尺度；若 $x+h$ 已舍回 x，应直接检查实际分子。

实验二：从差额消失到误差曲线

实验设计：第一个实验取无量纲输入 $x\in\{1,10^4,10^8,10^{12},10^{16}\}$，分别以两种精度计算直接公式与有理化公式。设输出为 $\widehat{d}$，程序实际存储的输入为 $\widehat{x}=\mathrm{fl}(x)$。总误差与求值误差的带符号形式分别为

$$\underbrace{\widehat{d}-d(x)}_{\text{总误差}},\qquad\underbrace{\widehat{d}-d(\widehat{x})}_{\text{求值误差}}.$$

阅读下面代码时，重点比较同一行的直接式与有理化式；两条路径共用同一对开方结果。

总误差包括输入的表示误差。比较求值误差时，两种算法使用同一个已存入计算机的 $\widehat{x}$，因此可以比较计算过程本身造成的误差。后面的数值表对这些差取绝对值，再按相应参考值归一化。两个参考函数值均由高精度十进制运算得到。完整结果见配套 Notebook §1.2。

```
import numpy as np

for dtype in [np.float32, np.float64]:
    for sx in ["1", "1e4", "1e8", "1e12", "1e16"]:
        x = dtype(sx)
        a = dtype(np.sqrt(dtype(x + dtype(1))))
        b = dtype(np.sqrt(x))
        direct = dtype(a - b)
        rational = dtype(dtype(1) / dtype(a + b))
        print(sx, dtype.__name__, float(direct), float(rational))
```

表 1.5　相对于原定十进制输入的总相对误差：含输入表示误差

x	float32：直接相减	float32：有理化	float64：直接相减	float64：有理化
1	5.843×10^{-8}	5.843×10^{-8}	2.334×10^{-16}	9.937×10^{-17}
10^{4}	5.243×10^{-4}	1.695×10^{-8}	2.130×10^{-13}	1.726×10^{-17}
10^{8}	1	2.276×10^{-8}	1.362×10^{-8}	9.011×10^{-17}
10^{12}	1	2.525×10^{-9}	7.614×10^{-6}	1.317×10^{-16}
10^{16}	1	6.077×10^{-9}	1	4.592×10^{-17}

表 1.6　float32 输入表示不精确时，有理化式的两种相对误差

原定 x	实际存储的 $\hat{x}$	总相对误差	求值相对误差
10^{12}	999999995904	2.525×10^{-9}	4.573×10^{-9}
10^{16}	10000000272564224	6.077×10^{-9}	7.551×10^{-9}

表 1.5 给出总相对误差；表 1.6 仅列出本组测试中输入不能精确存储的两种情形，比较有理化式的两种误差口径。其余输入可精确存储，两个参考值相同。例如，float32 将 10^{16} 存为 10000000272564224，对应的总相对误差约 6.077×10^{-9}，求值相对误差约 7.551×10^{-9}。

对 float64，10^{16} 本身可精确存储，但其右侧间距为 2。$x+1$ 恰位于中点并舍回 x，两次平方根因而得到同一机器数，直接公式输出 0。有理化公式输出约 5×10^{-9}，总相对误差约 4.592×10^{-17}。开场的零结果由此得到解释：小增量在加法时已被吸收，直接求差再也看不到它。

第二个实验：取 $f(x) = \mathrm{e}^{x}$、$x = 1$，解析导数为 e。在 10^{-16} 至 10^{-1} 之间按对数间隔取 151 个 h，计算前向差分。每个 h 先转换成相应机器数，函数值与分子、分母均保持所选类型。

```
from decimal import Decimal, localcontext
import numpy as np

with localcontext() as ctx:
    ctx.prec = 100
    ref = Decimal(1).exp()
    for dtype in [np.float32, np.float64]:
        rows = []
        for exponent in np.linspace(-16, -1, 151):
            h = dtype(10.0 ** float(exponent))
            xp = dtype(dtype(1) + h)
```

```
            num = dtype(dtype(np.exp(xp)) - dtype(np.exp(dtype(1))))
            estimate = dtype(num / h)
            total = abs(Decimal.from_float(float(estimate)) - ref)
            rows.append((float(h), float(total)))
        print(dtype.__name__, min(rows, key=lambda r: r[1]))
```

这段代码的阅读目标是观察最小误差出现在哪个步长，而非尽量缩小 h。

`np.linspace` 在本例中从 -16 至 -1 生成 151 个等间距指数，随后 10^{exponent} 给出按对数分布的步长。`xp = dtype(dtype(1) + h)` 使 $1+h$ 按指定精度舍入；最后的 `min` 按误差列选出最小的一行。

Notebook 还计算高精度理想差分 D_h^{ref}，输入为实际存储的 h，而非显示出来的十进制缩写。这样可以把误差拆为

$$\widehat{D}_h - \mathrm{e} = \underbrace{(D_h^{\text{ref}} - \mathrm{e})}_{\text{截断部分}} + \underbrace{(\widehat{D}_h - D_h^{\text{ref}})}_{\text{有限精度部分}}. \tag{1.23}$$

有限精度部分包含 $x+h$ 的舍入、函数库求值与算术舍入。图中分别画出三项的绝对值；总误差并不等于两部分绝对值的和，因为两部分的带符号误差有时会抵消。

读图时，从右向左表示步长逐渐减小：先看右侧截断误差主导的下降趋势，再看较好步长附近，最后看左侧有限精度影响主导的区域。三个区域的边界依赖精度与实现，不要求曲线是平滑的U形。

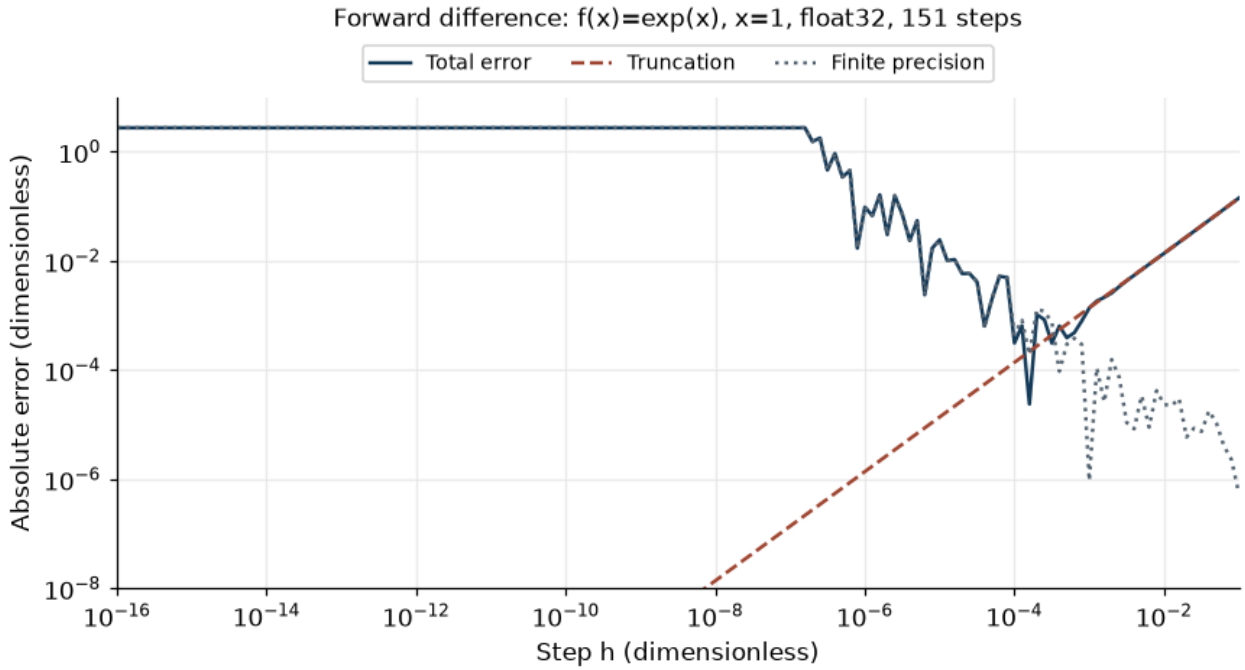


图 1.2　前向差分的 `float32` 实验：$f(x)=\mathrm{e}^{x}$、$x=1$，151 个步长；曲线为式 (1.23) 各项的绝对值；从右向左依次经过截断主导、较好步长附近和有限精度影响主导区域。

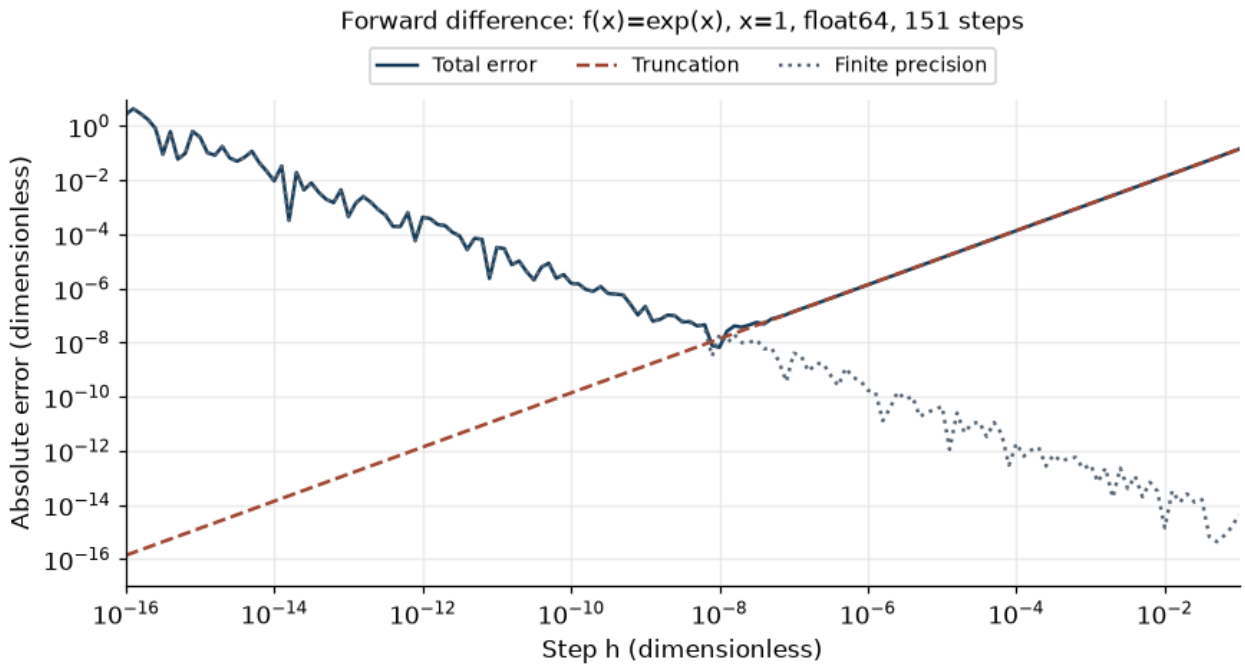


图 1.3　前向差分的 `float64` 实验：$f(x)=\mathrm{e}^{x}$、$x=1$，151 个步长；曲线为式 (1.23) 各项的绝对值；从右向左依次经过截断主导、较好步长附近和有限精度影响主导区域。

表 1.7　步长扫描中的代表性绝对误差

h 的目标值	`float32`	`float64`
10^{-1}	1.406×10^{-1}	1.406×10^{-1}
10^{-4}	3.100×10^{-4}	1.359×10^{-4}
10^{-8}	2.718×10^{0}	6.603×10^{-9}
10^{-12}	2.718×10^{0}	4.323×10^{-4}
10^{-16}	2.718×10^{0}	2.718×10^{0}

本次扫描中，`float32` 的最小观测绝对误差约为 2.352×10^{-5}，对应 $h\approx1.585\times10^{-4}$；`float64` 的最小观测值约为 6.603×10^{-9}，对应 $h=10^{-8}$。局部谷底的两部分误差有时会抵消，所以实测最优点会偏离式 (1.22) 的平衡估计。理解曲线的整体趋势，比记住某一个步长更有用。

当 `float32` 的 h 小到使 $1+h$ 舍回 1 时，分子成为零，导数估计也成为零，其绝对误差达到 e；本次扫描中最大这样一个步长约为 5.012×10^{-8}。同一现象在 `float64` 的扫描端点 $h=10^{-16}$ 也出现。两幅图显示：步长太大时截断误差主导，步长太小时有限精度误差主导。

讨论与自检

不使用高精度开方，能否利用有理化公式证明 $1/(2\sqrt{x+1})<d(x)<1/(2\sqrt{x})$？这个范围能否用来排除直接式的零输出？对差分实验，再依据 1 右侧的浮点间距，预测两种精度中 $1+h$ 开始舍回 1 的步长范围，并与扫描结果比较。

迁移自检：对 $0<|t|\ll 1$，比较 $1-\cos t$ 与 $2\sin^2(t/2)$。利用 Taylor 展开估计准确结果的尺度，再分析两种求值过程怎样传播函数库的误差。即使恒等式避免了抵消，为什么仍须检查 $t/2$ 的表示、正弦求值与平方运算？

本节讨论的提示见章末“讨论与自检提示”。只读主线可转至 § 1.3；下面的中心差分推导为选读。

选读：中心差分改变了什么

对于中心差分，

$$C_h f(x)=\frac{f(x+h)-f(x-h)}{2h},$$

在 $[x-h,x+h]$ 上作两次 Taylor 展开，二阶项在相减时抵消。若 f 具有连续三阶导数且 $|f'''|\le M_3$，则

$$C_h f(x)-f'(x)=\frac{h^2}{12}\bigl[f'''(\xi_+)+f'''(\xi_-)\bigr],\quad \xi_+\in(x,x+h),\quad \xi_-\in(x-h,x),$$

因此 $|C_h f(x)-f'(x)|\le M_3h^2/6$。与前向差分相比，截断误差界由 $O(h)$ 改为 $O(h^2)$；若舍入项仍可估计为 Bu/h，则最小化 Ah^2+Bu/h 得到 $h_{\mathrm{bal}}=(Bu/(2A))^{1/3}$，其中 $A,B>0$。提高差分阶数改变了两种误差的平衡，仍需检查输入舍入与函数求值。

1.3　条件数与稳定性（2学时）

数值问题规定输入与所求输出之间的数学关系；算法规定怎样用可执行的步骤近似这个输出。例如，开场的两种公式求的是同一个差额，却采用了不同的计算步骤。

结果不准确时，先问两个问题：输入数据的微小变化，是否已经使数学问题的准确结果发生较大变化？计算步骤是否又额外放大了误差？前者由问题的条件性描述，后者由算法的数值稳定性描述。本节分别考察输入怎样变化，以及同一输入怎样计算。

用导数定义局部敏感性

如果输入有约万分之一（10^{-4}）的相对误差，输出误差可能有多大？在小扰动范围内，若条件数约为 10，输出相对误差的一阶估计就是千分之一（10^{-3}）。若条件数约为 10000，同样的数据误差就可能使输出精度无法达标。但当预测值已达到 1 的量级时，不能只凭一阶项断言实际误差，需要检查余项或直接分析准确变化。条件数要回答的就是“输入误差会被问题本身放大多少”。

设 $y=f(x)$，f 在 x 附近可微。对输入扰动 Δx，

$$f(x+\Delta x)-f(x)=f'(x)\Delta x+o(|\Delta x|).$$

这先给出了局部绝对条件数 $|f'(x)|$：它描述绝对输入误差向绝对输出误差的放大，通常带有由输入、输出单位决定的量纲。

当 $x\neq 0$ 且 $f(x)\neq 0$ 时，可以把输入与输出都按自身尺度归一化：

$$\begin{aligned}\frac{|f(x+\Delta x)-f(x)|}{|f(x)|}&\approx\frac{|f'(x)||\Delta x|}{|f(x)|}\\&=\left|\frac{xf'(x)}{f(x)}\right|\frac{|\Delta x|}{|x|}.\end{aligned}\tag{1.24}$$

定义 1.3　标量函数的局部相对条件数

对上述非零输入和输出，定义

$$\kappa_f(x) = \left| \frac{x f'(x)}{f(x)} \right|. \tag{1.25}$$

等价地，它是相对扰动趋于零时，输出相对变化与输入相对变化之比的极限：

$$\kappa_f(x) = \lim_{\eta \to 0} \frac{|f(x(1+\eta)) - f(x)|}{|\eta|\,|f(x)|}.$$

条件数由问题、输入点和误差尺度决定；同一函数在不同输入点也可能有不同条件数。κ_f 大意味着小的相对输入误差可能被显著放大。判断一个问题是否病态，需要结合数据精度与目标误差。

若期望输出相对误差不超过 τ，输入相对误差约为 η，则一阶关系 $\kappa_f \eta \lesssim \tau$ 可以帮助判断数据是否够用。严格误差界需要进一步控制余项，或使用扰动区间上的导数上界。输入或输出为零时，改用绝对或混合尺度。

例 1.4　同一函数在不同位置的条件性

对于 $f(x) = x^n$，n 为正整数，且 $x \neq 0$，有 $\kappa_f(x) = n$。若输入相对改变 2%，$n = 10$ 时一阶预测输出相对改变约 20%，实际改变为 $1.02^{10} - 1 \approx 21.8994\%$。差异来自没有被一阶项描述的高阶部分。

对于 $f(x) = \log x$，$x > 0$、$x \neq 1$，

$$\kappa_{\log}(x) = \frac{1}{|\log x|}.$$

当 x 接近 1 时，输出本身接近零，其相对变化可能很大；但绝对条件数 $1/x$ 仍接近 1。因此，“对数在 1 附近病态”是在以 x 和 $\log x$ 自身为尺度的相对意义下所说，不能据此断定对数的绝对误差也一定巨大。

如果小量 $t = x - 1$ 本身可以作为准确输入，把输入改为 t 并计算 $\log(1+t)$，其关于 t 的相对条件数为

$$\left| \frac{t}{(1+t)\log(1+t)} \right| \longrightarrow 1 \quad (t \to 0).$$

这涉及输入参数与误差模型的改变。若 t 是由一个已经不精确的 x 相减得到，不能把新表达式的条件数解释为原始数据精度已经改善。

开场问题良态吗

对 $d(x) = \sqrt{x+1} - \sqrt{x}$，直接求导可得

$$d'(x) = \frac{1}{2\sqrt{x+1}} - \frac{1}{2\sqrt{x}} = -\frac{d(x)}{2\sqrt{x(x+1)}}.$$

因此

$$\kappa_d(x) = \left| \frac{x d'(x)}{d(x)} \right| = \frac{1}{2}\sqrt{\frac{x}{x+1}} < \frac{1}{2}, \qquad x > 0. \tag{1.26}$$

随着 $x \to \infty$，条件数趋近于 $1/2$。也就是说，在一阶近似下，输入 x 的相对变化传到 $d(x)$ 时还会缩小。开场问题关于这个输入是良态的，零输出的原因应到计算路径中寻找。

这也解释了看似矛盾的两种描述：把减法单独看成两个独立输入 a, b 的问题时，条件数可能含 $(|a| + |b|)/|a - b|$；把整个表达式看成单个输入 x 的函数时，条件数却小于 $1/2$。前一种分析允许两个操作数独立变化，后一种分析只改变同一个输入 x。两次平方根分别舍入后，误差未必相互抵消；再求差时，这些误差可能变得显著。

算法误差应与什么比较

开场例子的相对条件数小于 $1/2$，因此问题本身对输入扰动并不敏感。但直接式的相对误差可能达到 $O(ux)$，随 x 增大而被放大；在 §1.2 所述正常运算条件下，有理化式的相对误差可保持在 $O(u)$ 量级。

判断算法时，要把实际计算误差与这个问题本来会放大的误差相比。算法额外引入的误差若远超这一尺度，就需要检查计算步骤。表 tab:conditionstability 概括了两种判断，下面把这一比较写成定量要求。

对 $x \neq 0$、$f(x) \neq 0$ 的标量函数求值，若算法误差在所讨论输入范围内满足形如

$$\frac{|\widehat{f}_u(x) - f(x)|}{|f(x)|} \lesssim C\,[1 + \kappa_f(x)]u \tag{1.27}$$

的估计，可据此判断其前向稳定性。C 与算法步骤有关，在所讨论输入范围内应不依赖 u，也不随输入产生额外的巨大放大。式中的 u 表示最终输出舍入的尺度；$\kappa_f u$ 表示输入若受到 $O(u)$ 的相对扰动，准确结果会有多大相对变化。有理化式在开场问题上符合这一误差尺度。

对于较长的计算过程，可以建立误差递推，分别追踪已有误差的传播和每一步新增的舍入。下面的积分递推说明，改变计算方向就能改变误差的传播规律。

表　条件性与数值稳定性的区别

比较项	条件性	数值稳定性
研究对象	数学映射、输入位置与扰动尺度	具体求值过程及其误差传播
主要问题	数据轻微改变，准确解改变多少	舍入和已有误差是否被额外放大
分析方法	输入扰动实验、导数与条件数	同一输入的算法比较、误差递推
应对误差的方法	提高数据准确性，按问题选定绝对或相对误差指标，必要时调整输出精度要求	改写公式、改变递推方向或更换算法

提高工作精度能降低舍入，且在输入允许时改善输出精度；它不会改变既定映射及误差尺度下的条件数。若原始观测只有有限准确位数，后来把它装进更多位的格式也不能补回未知数据。

正积分为什么会被算成负数

一个积分的被积函数处处非负，计算结果却可能是负数。下面从同一个积分出发，推导两种计算方法，再解释它们为什么会有不同的可靠性。

例 1.5　积分递推中的误差放大与缩小

设

$$I_n = \mathrm{e}^{-1}\int_0^1 x^n \mathrm{e}^{x}\,\mathrm{d}x, \qquad n = 0, 1, \ldots. \tag{1.28}$$

计算 I_0, I_8, I_9。先判断它们的正性与取值范围，再由分部积分建立递推。如果只用近似初值 $\widehat{I}_0 = 0.6321$ 向前计算，结果是否可靠？能否利用积分的范围，设计另一种递推方向来求这些积分？

解：先在实数中确定积分的性质和递推关系，再分析近似初值与舍入误差的影响。

（1）积分的正性与递推公式。在 $0 < x < 1$ 上，$x^n \mathrm{e}^{x-1} > 0$，且 $\mathrm{e}^{-1} < \mathrm{e}^{x-1} < 1$。分别乘以 x^n 并积分，由 $\int_0^1 x^n\,\mathrm{d}x = 1/(n+1)$ 得

$$\frac{\mathrm{e}^{-1}}{n+1} < I_n < \frac{1}{n+1}. \tag{1.29}$$

这个范围从积分定义直接得到，可以独立检查计算结果；任何负数都不可能是这些积分的准确值。

当 $n=0$ 时，直接积分给出 $I_0 = \mathrm{e}^{-1}(\mathrm{e}-1) = 1-\mathrm{e}^{-1} \approx 0.6321205588$。当 $n \geq 1$ 时，对 x^n 求导、对 e^x 积分，分部积分给出

$$\begin{aligned} I_n &= \mathrm{e}^{-1}\left([x^n \mathrm{e}^x]_0^1 - n\int_0^1 x^{n-1}\mathrm{e}^x \,\mathrm{d}x \right) \\ &= 1 - nI_{n-1}. \end{aligned}$$

于是得到向前递推

$$I_0 = 1-\mathrm{e}^{-1}, \qquad I_n = 1 - nI_{n-1} \quad (n \geq 1). \tag{1.30}$$

这里所求对象是积分定义的 I_n；I_0 是实现递推时使用的初值。若另外研究“任意初值到递推数列”的映射，那是具有不同输入模型的问题。

（2）正确公式为什么算出负数。取 $\widehat{I}_0 = 0.6321$。按递推的前两步，$\widehat{I}_1 = 1-0.6321 = 0.3679$，$\widehat{I}_2 = 1-2(0.3679) = 0.2642$；继续计算，得到 $\widehat{I}_8 \approx -0.728$、$\widehat{I}_9 \approx 7.552$。前者为负，后者超过 $I_9 < 0.1$ 的上界，二者都违背了先验范围。

为何只有约 2.06×10^{-5} 的初值误差，最后却引起如此大的偏差？暂时忽略后续舍入，记 $\delta_n = \widehat{I}_n - I_n$。将近似递推与准确递推相减，有

$$\delta_n = -n\delta_{n-1} = (-1)^n n!\,\delta_0. \tag{1.31}$$

每向前一步，已有误差的符号改变、大小乘以当前的 n；到第 8 步，初值误差已被乘以 $8! = 40320$，其大小约为 0.829，远大于 I_8 本身。若第 n 步又引入局部舍入 ρ_n，则 $\delta_n = -n\delta_{n-1} + \rho_n$；这些新误差也会在后续步骤受到放大。公式在实数中成立，并不保证这个递推方向在有限精度下可靠。

（3）反向递推怎样利用粗近似。将同一恒等式改写为

$$I_{n-1} = \frac{1 - I_n}{n}. \tag{1.32}$$

取末端 $N=9$，由式 (1.29) 有 $0.0367879\ldots < I_9 < 0.1$。区间中点约为 0.068394，取其四位小数 $\widehat{I}_9 = 0.0684$，作为一个有范围依据的粗近似。它不是准确的 I_9，也不是由向前递推取得的参考值。

忽略新增舍入时，误差满足 $\delta_{n-1} = -\delta_n/n$。从 N 逐步代入至 n，得到

$$\delta_n = (-1)^{N-n}\frac{n!}{N!}\delta_N, \qquad 0 \leq n \leq N.$$

本例从 9 反推到 8 时，末值误差大小除以 9；反推到 0 时，除以 $9! = 362880$。由末值区间可知 $|\delta_9| < 0.031613$，因而在忽略新增舍入的模型中，$|\delta_0| < 8.72\times10^{-8}$。末值虽然粗糙，却足以使低阶积分明显改善；给定的末端 I_9 本身不会因此变准确。

（4）用独立参考检查两种算法。下面的代码分别向前计算到 9 和由 9 反推到 0，均使用 Python 的 binary64 浮点数。先用误差传播公式预测结果，再比较实际输出。配套 Notebook §1.3 保存了这两种计算及误差的完整输出。

```
forward = [0.6321]
for n in range(1, 10):
    forward.append(1.0 - n * forward[-1])

backward = [0.0] * 10
```

```
backward[9] = 0.0684
for n in range(9, 0, -1):
    backward[n - 1] = (1.0 - backward[n]) / n
```

独立参考由积分定义展开为正项级数建立，避免用待检验的递推为自身作证。下面先比较输出；级数及尾项界的完整推导见随后的选读，习题3(b)可据此检验参考值精度。

表 1.10　同一积分族的正向与反向计算（均为binary64）

n	高精度参考	正向值	反向值	反向绝对误差
0	0.63212056	0.6321000	0.63212062	$\mathbf{6.397\times10^{-8}}$
2	0.26424112	0.2642000	0.26424125	$\mathbf{1.279\times10^{-7}}$
4	0.17089341	0.1704000	0.17089495	$\mathbf{1.535\times10^{-6}}$
6	0.12680236	0.1120000	0.12684841	$\mathbf{4.606\times10^{-5}}$
8	0.10093197	−0.7280000	0.10351111	$\mathbf{2.579\times10^{-3}}$
9	0.09161229	7.5520000	0.06840000	$\mathbf{2.321\times10^{-2}}$

正向结果在 $n=8$ 处变成负数，违反式 (1.29)。反向算法的低阶值则明显更准确；I_0 的绝对误差约为 $\mathbf{6.397\times10^{-8}}$。这与正向误差按阶乘放大、反向误差逐步缩小的分析一致。表中 $n=9$ 的反向值就是给定末值，低阶值的改善来自递推中的误差缩小。

这道例题展示了稳定性分析的基本步骤：先确定数学结果应满足的性质，再找出算法中的误差传播因子，最后用独立参考验证。向前和反向递推针对同一积分族，末值精度与误差传播方向共同决定输出精度。

例题出处：积分族、近似初末值与递推思路源自文献 1 §1.3.1 例1.5；上述推导、先验范围、独立参考及实际误差在本例中完整给出。

选读：怎样保证参考值可靠

只读递推主线可转至下一项输入扰动实验。

为了避免用待检查的递推为自身作证，独立参考从积分定义建立。这里的一致收敛是指：在整个 $[0,1]$ 上，级数尾项都可用同一个趋于零的界控制，因而可交换极限与积分。在 $[0,1]$ 上，$|x^k/k!|\le 1/k!$，而 $\sum_k 1/k!$ 收敛。因此指数级数 $\mathrm{e}^x=\sum_{k=0}^{\infty}x^k/k!$ 在整个区间上一致收敛，可以逐项积分，得到

$$I_n=\mathrm{e}^{-1}\sum_{k=0}^{\infty}\frac{1}{k!\,(n+k+1)}. \tag{1.33}$$

参考值以 100 位十进制精度求前 160 项，并以 120 位复查报告位数。尾项也可以直接控制：令 $a_k=1/[k!(n+k+1)]$，则 $k\ge 160$ 时，$a_{k+1}/a_k<1/161$，所以

$$0<R_{160}<\frac{\mathrm{e}^{-1}}{160!\,(n+161)}\,\frac{1}{1-1/161}<4.88\times10^{-288}.$$

这一截断界远小于表中报告的误差尺度。提高参考计算的精度用于检查求值，尾项界用于控制截断，两者分别承担不同的检验任务。

实验三：输入扰动怎样放大

实验 A（改变输入）：取 $f(x)=\log x$、$x=1.0001$，输入改成 $x(1+\eta)$，其中 η 从 10^{-8} 增至 10^{-5}。两次函数值都用高精度计算，比较输出相对变化与 $\kappa_f(x)\eta$。积分例题比较了同一目标的两种算法；这里固定计算方法、改变输入，观察问题自身的敏感性。完整结果见配套 Notebook §1.3。

表 1.9 $x=1.0001$ 时对数问题的高精度输入扰动实验

输入相对扰动 η	输出相对变化	一阶预测 $\kappa\eta$	实测放大率
10^{-8}	1.000050×10^{-4}	1.000050×10^{-4}	10000.49994
10^{-7}	1.000050×10^{-3}	1.000050×10^{-3}	10000.49949
10^{-6}	1.000049×10^{-2}	1.000050×10^{-2}	10000.49499
10^{-5}	1.000045×10^{-1}	1.000050×10^{-1}	10000.44999

阅读表 1.9 和下面代码时，比较“实测放大率”与条件数是否接近。字符串构造的十进制输入使这里主要观察输入扰动，而非binary32或binary64的求值误差。

在这个输入点，$\kappa_f(x)\approx10000.50$。表 1.9 中的放大率接近条件数：输入相对改变约 10^{-6}，准确输出就相对改变约 10^{-2}。高精度求值帮助我们看清这种变化来自输入本身。

```
from decimal import Decimal, localcontext
with localcontext() as ctx:
    ctx.prec = 100
    x = Decimal("1.0001")
    y = x.ln()
    kappa = Decimal(1) / abs(y)
    for s in ["1e-8", "1e-7", "1e-6", "1e-5"]:
        eta = Decimal(s)
        relative_change = abs(((x * (1 + eta)).ln() - y) / y)
        print(eta, relative_change, kappa * eta,
              relative_change / eta)
```

回到开场差额，条件数说明准确输入改变时，准确差额怎样变化；同一输入的两种算法比较则说明计算路径怎样影响误差。把这两步分开，才能分清数据精度与算法选择各自的作用。

讨论与自检

某问题的相对条件数约为 10^{4}，输入相对误差约为 10^{-6}，希望输出相对误差不超过 10^{-3}。用一阶估计判断输入数据是否足够准确；提高运算精度能解决这里的主要限制吗？若保持目标误差不变，输入相对误差应降到什么量级？

迁移自检：球体积为 $V=4\pi r^3/3$，$r>0$，把 π 视为准确常数。若要求由半径测量误差产生的体积相对误差不超过 1%，用一阶估计给出半径相对误差要求；再对 $\hat r=r(1+\eta)$ 写出准确相对变化 $(1+\eta)^3-1$，判断“一阶估计满足要求”是否足以构成严格保证。

本节讨论的提示见章末“讨论与自检提示”。只读主线可转至 §1.4；下面两项矩阵与后向误差分析为选读。

选读：从误差传播到后向误差

还可以从输入端解释计算误差：计算结果是否等于某个稍受扰动问题的准确答案？对于非零标量输入，若

$$\hat{y} = f(x + \Delta x), \qquad \frac{|\Delta x|}{|x|} \le C u,$$

就给出了关于输入的后向误差解释。例如，一个算出的平方根若是邻近输入 $x + \Delta x$ 的准确平方根，就可用 $|\Delta x|/|x|$ 衡量这一误差。输出非零且扰动足够小时，结合局部条件数可估计输出相对误差约为 $\kappa_f C u$。

前向误差衡量输出偏差，后向误差衡量解释这个输出所需的输入变化，条件数把二者联系起来。线性方程组也能直接展示这种联系[2]。设 $Ax = b$，A 可逆、$b \neq 0$，近似解为 $\hat{x}$。定义残差 $r = b - A\hat{x}$，则

$$A\hat{x} = b - r, \qquad \hat{x} - x = -A^{-1} r.$$

第一式表明：保持 A 不变时，近似解是右端项改为 $b - r$ 后的准确解，相应相对后向误差为 $\|r\|_\infty / \|b\|_\infty$。这里 $\|v\|_\infty = \max_i |v_i|$，与它相容的矩阵范数为 $\|A\|_\infty = \max_i \sum_j |a_{ij}|$，满足 $\|Av\|_\infty \le \|A\|_\infty \|v\|_\infty$。由第二式及 $\|b\|_\infty \le \|A\|_\infty \|x\|_\infty$，得到

$$\frac{\|\hat{x} - x\|_\infty}{\|x\|_\infty} \le \kappa_\infty(A) \frac{\|r\|_\infty}{\|b\|_\infty}, \qquad \kappa_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty.$$

因此，小残差只有在条件数也适当时才保证小的前向误差。上述结论使用准确残差；实际计算残差时，还须计入矩阵乘法和减法的舍入误差。

选读自检：若相对残差为 10^{-10}，且 $\kappa_\infty(A) = 10^{8}$，上述界能否保证相对解误差不超过 10^{-4}？它能否反过来证明实际解误差很大？区分“上界不足以保证准确”与“已经证实不准确”。

选读：接近奇异的方程组

例 1.6　参数改变与接近奇异的方程组

考虑

$$\begin{cases} x + \alpha y = 1, \\ \alpha x + y = 0. \end{cases}$$

求参数 α 对解的影响，并比较参数到解分量的条件数与矩阵条件数。

解：第一式减去第二式的 α 倍，得 $(1 - \alpha^2)x = 1$；第二式给出 $y = -\alpha x$。当 $\alpha \neq 0, \pm 1$ 时，$x(\alpha) = 1/(1 - \alpha^2)$，$y(\alpha) = -\alpha/(1 - \alpha^2)$。参数 α 到解分量 x 的局部相对条件数为

$$\kappa_x(\alpha) = \left| \frac{2\alpha^2}{1 - \alpha^2} \right|.$$

在 $\alpha = 0.99$ 处，$\kappa_x \approx 98.50$；把 α 改成 0.991，x 从约 50.2513 改为约 55.8067。这里的条件数针对参数 α 到解分量 x 的映射，两个非对角系数随同一个参数一起变化。

若改用上段定义的矩阵条件数，对 $A = \left(\begin{smallmatrix} 1 & \alpha \\ \alpha & 1 \end{smallmatrix}\right)$，当 $\alpha \neq \pm 1$ 时，$A^{-1} = (1 - \alpha^2)^{-1} \left(\begin{smallmatrix} 1 & -\alpha \\ -\alpha & 1 \end{smallmatrix}\right)$。按行绝对值之和计算，得 $\|A\|_\infty = 1 + |\alpha|$、$\|A^{-1}\|_\infty = (1 + |\alpha|)/|1 - \alpha^2|$，故

$$\kappa_\infty(A) = \frac{1 + |\alpha|}{|1 - |\alpha||}, \qquad \alpha \neq \pm 1.$$

当 $\alpha = 0.99$ 时，其值为 199。κ_x 衡量参数 α 变化对解分量 x 的影响；$\kappa_\infty(A)$ 用于指定范数下矩阵方程的敏感性分析。两者采用的输入、输出与扰动模型不同，因此数值可以不同。

例题出处：方程组源自文献 1 §1.3.2 例1.6；本例另给出指定范数下的矩阵条件数。

1.4 AI关联：训练中的数值问题（2学时）

神经网络训练包含矩阵运算、指数求值、归一化、损失计算与参数更新，也会遇到舍入、上溢和下溢。本节从分类概率与交叉熵出发，说明等价改写怎样改善计算，再用一个小梯度的存储实验解释精度选择。

Softmax 的定义与直接计算的风险

设 $n \geq 1$，有限实向量 $z = (z_1, \ldots, z_n)$ 表示分类模型的 logits，即未归一化的类别得分。Softmax 把它变成概率向量：

$$p_i = \frac{\exp(z_i)}{\sum_{j=1}^{n} \exp(z_j)}, \qquad i = 1, \ldots, n. \tag{1.34}$$

在精确实数运算中，$p_i > 0$，且 $\sum_i p_i = 1$。但直接计算 $\exp(z_i)$ 可能在概率本身完全正常时就上溢。例如，对 $(1000, 1001, 1002)$，真正的概率仅由三项的差异决定；binary64 中 $\exp(1000)$ 却已经超出有限范围。

根据表 1.2，指数函数的上溢门槛约为 $\log M_{\max}$：binary32 中约为 88.72，binary64 中约为 709.78。若所有 z_i 很负，指数还可能全部下溢为零，最终出现 $0/0$。概率原本只取决于得分之间的差异，直接指数却把共同的大偏移带入了中间计算。

减去最大值为何有效

对于任意实数 c，分子与分母同时乘 $\exp(-c)$，得

$$p_i = \frac{\exp(z_i - c)}{\sum_j \exp(z_j - c)}. \tag{1.35}$$

取 $m = \max_j z_j$，则 $z_i - m \leq 0$，至少一个指数为 1，因此在实数中

$$0 < \exp(z_i - m) \leq 1, \qquad 1 \leq \sum_j \exp(z_j - m) \leq n. \tag{1.36}$$

在有限输入、减法正常计算且向量长度不会使求和溢出的条件下，这一改写避免直接指数上溢，也避免所有指数同时下溢为零。很小的尾部指数仍可能舍为零，后面的例子将说明它对损失计算的影响。

指数和的对数（log-sum-exp）定义为

$$L(z) = \log \sum_j \exp(z_j).$$

同样可改写为

$$L(z) = m + \log \sum_j \exp(z_j - m). \tag{1.37}$$

若直接从这个 L 再减去很大的 z_i，可能重新引入抵消。计算对数概率时，更合适的表达式是

$$\log p_i = (z_i - m) - \log \sum_j \exp(z_j - m). \tag{1.38}$$

已知一个样本属于第 y 类，就希望模型给第 y 类较大的概率。本节用 $\ell = -\log p_y$ 衡量预测与已知类别的差距：p_y 越接近 1，损失越接近 0；p_y 越小，损失越大。例如，$p_y = 0.9$ 时损失约为 0.105，$p_y = 0.01$ 时约为 4.605。这种单样本损失称为交叉熵，以下只讨论它的计算，不要求先学习完整神经网络。

对真实类别为 y 的单样本交叉熵，

$$\ell = -\log p_y = \log \sum_j \exp(z_j - m) - (z_y - m). \tag{1.39}$$

式 (1.39) 从 logits 直接计算损失，避免先把极小概率存成零再取对数。

这些改写控制了中间量的尺度，减少上溢、下溢和抵消的影响。范围正常以后仍有指数求值、累加和除法的舍入，平移版也不保证在每一次测试中误差都更小。下面先用实验检查有限性、分量误差和损失；本节末的选读再分析各误差来源。平移算法与深入分析的出处为文献 4 §4，算法4.1及定理4.3。

实验四：大幅值 logits 与尾部概率

取三组确定性输入

$$\boldsymbol{z}^{(1)} = (0, 1, 2), \quad \boldsymbol{z}^{(2)} = (1000, 1001, 1002), \quad \boldsymbol{z}^{(3)} = (-1000, -999, -998).$$

在实数中，这三组 Softmax 相同。运行前先预测：哪一种实现会得到非有限输出？减去最大值后，三组输入的中间量是否相同？完整输出与误差计算见配套 Notebook §1.4。

```
import numpy as np

def stable_outputs(z):
    m = np.max(z)
    w = np.exp(z - m)
    s = np.sum(w, dtype=z.dtype)
    p = w / s
    logp = (z - m) - np.log(s)
    return p, logp, m + np.log(s)

cases = [[0, 1, 2], [1000, 1001, 1002], [-1000, -999, -998]]
for dtype in [np.float32, np.float64]:
    for values in cases:
        z = np.array(values, dtype=dtype)
        # Deliberate overflow/underflow demonstration.
        with np.errstate(over="ignore", under="ignore",
                         divide="ignore", invalid="ignore"):
            v = np.exp(z)
            naive = v / np.sum(v, dtype=dtype)
            p, logp, lse = stable_outputs(z)
        print(values, dtype.__name__, np.isfinite(naive).all(),
              p, -logp[0])
```

代码中的 `np.array` 把得分列表构造成数组，`z.dtype` 保留数组的工作精度；指数与减法按分量执行，`np.sum` 再求和。`np.isfinite(...).all()` 检查所有分量是否均为有限值。数学上的第 1 类对应 Python 下标 0，所以 `-logp[0]` 计算第 1 类损失。读输出时，先比较三组输入的有限性，再观察平移版的概率和损失是否一致。

以准确整数得分的高精度 Softmax 值为参考，报告最大分量绝对误差

$$E_\infty = \max_i |\widehat{p}_i - p_i^{\mathrm{ref}}|$$

以及归一化偏差

$$E_\Sigma = \left| \sum_i \widehat{p}_i - 1 \right|.$$

计算 E_Σ 时，用高精度累加实际存储的概率，以保留其中的小偏差。E_∞ 衡量各分量的准确性，E_Σ 衡量它们的和偏离 1 多少。

表 1.11　Softmax 实现的有限性与平移版输出误差

输入组	类型	直接版有限	平移版有限	E_∞	E_Σ
普通	`float32`	是	是	1.234×10^{-8}	2.235×10^{-8}
普通	`float64`	是	是	1.045×10^{-16}	1.110×10^{-16}
上溢测试	`float32`	否	是	1.234×10^{-8}	2.235×10^{-8}
上溢测试	`float64`	否	是	1.045×10^{-16}	1.110×10^{-16}
下溢测试	`float32`	否	是	1.234×10^{-8}	2.235×10^{-8}
下溢测试	`float64`	否	是	1.045×10^{-16}	1.110×10^{-16}

所有平移实现均得到接近

$$(0.09003057317,\ 0.24472847105,\ 0.66524095577)$$

的概率。三组输入各自减去最大值后都变成 $(-2,-1,0)$，因此同一精度下的输出与误差相同。共同的大偏移不再进入指数计算，三组输入都能得到准确的概率近似。

对第 2 组输入，binary32 的平移版 L 约为 1002.40759277，而按式 (1.39) 计算类别 1 的损失约为 2.40760612。若把已舍入的 L 再减去 1000，得到 2.40759277，会丢失小量中的准确位。先从各得分中减去最大值，再用式 (1.39) 计算损失，可以减少共同大偏移的影响。

例 1.7　概率和为 1 仍可能无法计算损失

取 $z=(0,-1000)$，真实类别是第二类。平移版 Softmax 在两种精度中均输出 $(1,0)$。第二个概率的高精度参考约为

$$p_2\approx 5.07596\times10^{-435},$$

小于 binary64 的最小正非规格化数。相对于 p_2，零输出的相对误差为 1；按绝对误差衡量，却极其微小。

若先计算 $-\log\widehat{p}_2$，结果是 $+\infty$。使用式 (1.39)，实际计算得到 1000，与

$$\ell=1000+\log(1+\mathrm{e}^{-1000})$$

在当前精度下相符。概率的绝对误差很小，却使先求概率再计算损失的方法失败；直接计算对数概率，则仍能保留损失需要的信息。若把概率截到某个小常数，计算的将是另一个损失函数。

梯度尺度与精度选择

梯度告诉我们怎样调整参数以减小损失。先看一个待调整的实参数 w：记 $g=\partial\ell/\partial w$，取学习率 $\alpha>0$，最简单的更新是

$$w_{\text{new}}=w-\alpha g.$$

若 $g>0$，就把 w 略微减小；若 $g<0$，就把它略微增大。在损失足够光滑、步长适当小时，这沿着损失下降的方向移动。实际模型有多个参数，使用梯度向量更新；这里先考察计算和保存梯度时会丢失什么信息。

得分 z_j 通常也依赖参数 w，链式法则给出 $\partial\ell/\partial w=\sum_j(\partial\ell/\partial z_j)(\partial z_j/\partial w)$。利用链式法则逐层计算参数导数的过程称为反向传播。因此，先算准损失关于得分的导数，是参数更新的一部分。

由 $\ell = \log \sum_i \exp(z_i) - z_y$，直接求导得

$$\frac{\partial}{\partial z_j} \log \sum_i \exp(z_i) = \frac{\exp(z_j)}{\sum_i \exp(z_i)} = p_j, \qquad \frac{\partial z_y}{\partial z_j} = \mathbf{1}_{j=y}.$$

相减得到单样本交叉熵关于得分的梯度

$$\frac{\partial \ell}{\partial z_j} = p_j - \mathbf{1}_{j=y}. \tag{1.40}$$

其中 $\mathbf{1}_{j=y}$ 在 $j=y$ 时为 1，否则为 0。对非真实类别 $j \neq y$，梯度等于 p_j，很小的概率会产生很小的梯度。对真实类别 $j=y$，梯度为 $p_y - 1$，只有 p_y 接近 1 时其绝对值才小；若 $p_y = 10^{-8}$，该分量为 -0.99999999，接近 -1。保存小梯度时，需要考虑浮点格式的数值范围。

binary16 的有效精度为 $p = 11$，最大有限正数为 65504，最小正非规格化数为 $2^{-24} \approx 5.9605 \times 10^{-8}$。若最近舍入，一个小于半个最小间距的正值就可能舍为零。本章用 NumPy 的 `float16` 做一个标量存储实验：比较直接存储 g，与先用准确十进制量计算 $1024g$、再存入 binary16、最后用 `float32` 除以 1024。

这里取 $1024 = 2^{10}$ 作为演示因子。在没有上溢、下溢且结果可表示的范围内，乘以 2 的整数次幂可精确调整二进制指数；本例选择它，是为了使小梯度能被低精度格式分辨。实际计算需根据数值范围选择缩放因子，1024 不是固定规定。

表 1.12　标量梯度的存储实验，缩放因子为1024；还原在binary32中完成

准确梯度 g	直接binary16存储	先缩放后存储并还原
10^{-7}	1.192×10^{-7}	1.000008×10^{-7}
10^{-8}	0	1.001172×10^{-8}

对 $g = 10^{-8}$，直接存入 binary16 会变成 0。若先乘以 1024，再存入 binary16，最后在 binary32 中除以 1024，就能得到约 1.001172×10^{-8}。缩放使小量进入低精度格式能够分辨的范围。顺序很关键：先放大梯度，再存储，最后在较高精度中还原。

训练中的损失缩放利用同样的思路。设参数为 $\boldsymbol{w}$、损失为 $\ell(\boldsymbol{w})$，选取在本次求导中保持不变的正因子 s，定义 $\tilde{\ell} = s\ell$。链式法则给出

$$\nabla_{\boldsymbol{w}} \tilde{\ell} = s \nabla_{\boldsymbol{w}} \ell, \qquad \boldsymbol{g} = \frac{\nabla_{\boldsymbol{w}} \tilde{\ell}}{s}.$$

例如，单参数更新应使用 $w_{\text{new}} = w - \alpha(\partial \tilde{\ell}/\partial w)/s$。若不除回 s，更新量也被乘以 s，不再是原来的更新。在较高精度中还原 $\boldsymbol{g}$ 后再更新参数，才保持原来的学习率。放大必须发生在反向传播及低精度存储之前；把已经舍为零的梯度事后乘 s，不能恢复信息。

讨论与自检

下面是一条待辨析的实现建议：“直接计算指数，再除以指数和；只要概率和为 1，交叉熵就能正确计算。”请从本节输入中选出反例，分别修改概率计算与交叉熵计算，并解释结果为什么改善。若只把 `NaN` 替换为 0，三个分量还能近似原来的概率吗？

本节讨论的提示见章末“讨论与自检提示”。下面的实现细节与误差推导均为选读，只读主线可转至本章小结。

选读：训练实现中的尺度检查

梯度裁剪是把过大的梯度缩小到指定阈值以内；若还要裁剪梯度，应先还原，再按原尺度裁剪，否则阈值的含义也会改变。

动态损失缩放在每次更新前检查梯度的有限性：发现 `inf` 或 `NaN` 时，跳过该次参数更新并减小 s；连续若干次正常时，可以尝试增大 s。这是在避免下溢与避免上溢之间调节尺度，并不保证所有模型都能用同一低精度格式计算。上述流程是框架损失缩放的主要机制7；表 1.12 只验证其中的小量存储环节。

选读：平移公式的舍入误差分析

这里把误差分为平移与指数求值、累加、除法三部分，给出在小相对误差假设下的一阶展开。这是便于理解误差来源的分析；文献 4 §4 定理4.3进一步把得分范围与舍入误差联系起来，其假设和界的形式与下面的展开应分别理解。

记准确的平移指数为 $t_i=\exp(z_i-m)$、$S=\sum_i t_i$。假设平移减法与指数求值造成的误差可合并写为 $\hat t_i=t_i(1+\varepsilon_i)$，$|\varepsilon_i|\le\eta\ll 1$；中间量和最后结果均处于相对舍入模型适用的范围，且 $(n-1)u<1$。逐项求和及最后除法给出

$$\widehat{S}=\Big(\sum_i \hat t_i\Big)(1+\sigma),\quad |\sigma|\le\gamma_{n-1},\qquad \widehat{p}_i=\frac{\hat t_i}{\widehat{S}}(1+\rho_i),\quad |\rho_i|\le u,$$

其中约定 $\gamma_0=0$。因为 $p_i=t_i/S$、$\sum_i\hat t_i=S(1+\sum_i p_i\varepsilon_i)$，令 $\omega=\eta+\gamma_{n-1}+u\ll 1$，展开分母可得

$$\frac{\widehat{p}_i-p_i}{p_i}=\varepsilon_i-\sum_j p_j\varepsilon_j-\sigma+\rho_i+O(\omega^2).$$

由于 $\sum_j p_j=1$，各分量相对误差至多为 $2\eta+\gamma_{n-1}+u+O(\omega^2)$。这把指数求值、累加次数与最后舍入分别计入；它不意味着平移版在每次测试中都比直接版误差更小。若尾部指数下溢为零，相应 $\varepsilon_i=-1$，上述小相对误差假设不再成立，须改看绝对误差和所需的损失信息。

选读：概率敏感性与参数更新

由 $\partial p_i/\partial z_j$ 组成的矩阵称为 Softmax 的雅可比矩阵（Jacobian）。求导得到

$$\frac{\partial p_i}{\partial z_j}=p_i(\delta_{ij}-p_j),\tag{1.41}$$

其中 δ_{ij} 在 $i=j$ 时为 1，否则为 0。对角项为 $p_i(1-p_i)$，其余项绝对值之和为 $p_i\sum_{j\neq i}p_j=p_i(1-p_i)$，所以第 i 行绝对值之和为 $2p_i(1-p_i)\le 1/2$，故 $\|J\|_\infty\le 1/2$。沿两组有限得分之间的线段积分，得到严格的绝对扰动界

$$\begin{aligned}\boldsymbol{p}(\boldsymbol{z}+\Delta\boldsymbol{z})-\boldsymbol{p}(\boldsymbol{z})&=\int_0^1 J(\boldsymbol{z}+t\Delta\boldsymbol{z})\Delta\boldsymbol{z}\,\mathrm{d}t,\\ \|\Delta\boldsymbol{p}\|_\infty&\le\tfrac{1}{2}\|\Delta\boldsymbol{z}\|_\infty.\end{aligned}$$

这里比较的是准确概率随准确得分的变化；算法舍入误差仍需另行计算。若某个概率接近零，同样的绝对变化仍可能对应很大的相对误差。

梯度能够保存之后，更新量还要与参数的浮点间距比较。若 $w=1$、学习率为 0.01、梯度约为 10^{-7}，更新量约为 10^{-9}。它远小于 binary16 在 1 附近的间距，也小于 binary32 中从 1 向下舍入的半间距，减去它可能仍得到 1。这是更新量被吸收，与梯度本身下溢为零是两个不同的步骤。

回到开场与本章小结

现在可以回答开场的问题。对于 $x = 10^{16}$，加法把 $x+1$ 舍回 x，两次平方根因而得到相同的机器数，直接求差成为 0。有理化式改用两个正数的和作分母，保留了约 5×10^{-9} 的差额。又由 $\kappa_d(x) < 1/2$ 可知，准确差额对输入 x 的相对扰动并不敏感。本例的大误差来自直接求值的路径。

浮点数以有限精度表示实数，相邻机器数的间距随数量级变化。舍入误差的大小应结合数值尺度判断，显示位数不能代替误差估计。

条件数衡量输入的微小变化会使准确结果改变多少。条件数较大时，要得到准确输出，输入数据也需要相应地准确。

稳定性描述算法如何传播和放大误差。有理化、改变递推方向和调整求和次序，都可能改善计算结果。分析算法时，需要把计算误差与问题本身的敏感性联系起来。

表 1.13　从观测现象到数值判断

观测现象	首先检查	可采取的措施
小增量或差额消失	间距、输入舍入、操作数已有误差	重组公式并比较独立参考
步长越小误差反而增大	截断项与舍入项、实际存储步长	按尺度平衡两类误差
输入微变、准确输出大变	输入模型与条件数	提高数据准确性或调整目标
同一输入、算法结果差异大	运算次序与误差传播	比较更稳定的算法
输出有限、损失仍失败	尾概率下溢及取对数的顺序	直接计算对数概率与损失

当计算机告诉你答案是 3.141592653589793 时，你凭什么相信它？要综合数据精度、问题条件性与算法误差，判断它是否达到所需精度。后续各章将继续使用这三个概念。

讨论与自检提示

先独立完成各节讨论，再核对思路与结论。四道课后综合题的完整参考解答仍独立保存。

§1.1：间距、舍入与显示　思路：画出相邻机器数，判断中点，再用准确整数或有理数作参考。

结论：binary64在 10^{16} 右侧的间距为 2，$10^{16}+1$ 在中点并舍回 10^{16}，绝对误差为 1，相对误差为 $1/(10^{16}+1)$。$(0.1101)_2 = 13/16 = 0.8125$，两种精度均可精确表示。取 $p=8$ 时，2^7 右侧间距为 1，故 $\mathrm{fl}(2^7+1)=129$；2^8 右侧间距为 2，257 位于 256 与 258 之间，二进制有效数最低位分别为 0 与 1，故舍为 256；259 位于 258 与 260 之间，最低位分别为 1 与 0，故舍为 260。这里的“偶数”指二进制有效数的最低位，而非十进制整数的奇偶。

§1.2：差额、步长与余弦改写　思路：有理化后比较分母；差分先检查 $1+h$；余弦题先确定准确结果尺度。

结论：$2\sqrt{x} < \sqrt{x+1}+\sqrt{x} < 2\sqrt{x+1}$，取倒数即得正文的严格范围，可排除零输出。最近舍入时，$0 < h \le u$ 会使 $1+h$ 舍回 1，中点也选 1；两种精度的阈值分别为 2^{-24}、2^{-53}。

当 $t \to 0$ 时，$1-\cos t \sim t^2/2$。若余弦绝对求值误差为 $O(u)$，直接式的相对误差可能达到 $O(u/t^2)$。若 $t/2$ 的表示和正弦求值均有 $O(u)$ 相对误差，且平方与最终结果处于正常范围，改写式可保持 $O(u)$ 相对误差；只有绝对函数误差界或发生下溢时，须重新分析。

§1.3：输入精度与条件数 思路：条件数乘输入误差给一阶预测；严格保证应检查准确变化或余项。

结论：$10^4 \times 10^{-6} = 10^{-2} > 10^{-3}$，输入误差需降至约 10^{-7}，并检查余项。提高工作精度不能补回原始数据的未知准确位。球体积的相对条件数为 3，一阶要求是 $|\eta| \lesssim 1/300$；但 $\eta = 1/300$ 时，$3\eta + 3\eta^2 + \eta^3 > 1\%$。对 $|\eta| \le q$（$0 \le q < 1$），正向体积变化大于负向变化的绝对值，故只需控制

$$(1+q)^3 - 1 \le 0.01 \quad \Longrightarrow \quad q \le (1.01)^{1/3} - 1 \approx 0.00332228.$$

选读自检：残差界只给出 $10^8 \times 10^{-10} = 10^{-2}$ 的前向上界，不能保证 10^{-4}，也不能反推实际误差很大。

§1.4：概率与损失的不同路径 思路：分别检查有限性、分量误差与损失，用 $(0,-1000)$ 且真实类别为第二类作反例。

结论：概率 $(1,0)$ 的和为 1，先对第二个存储概率取负对数却得 $+\infty$，直接对数路径得约 1000。大正偏移还会使直接指数上溢。应平移求概率，并直接计算对数概率或损失；把 `NaN` 换成 0 不能恢复概率比例，损失下降也不能证明每个样本和梯度都准确。

课后综合题见A1组-习题.md。四题对应四节，包含推导、实现与辨析；参考解答单独保存。

附录 A 有限步骤与迭代的两个例子

本附录保留基础数值分析中的两个算法例子，用于理解怎样把数学表达式组织成有限、可分析的计算过程。它们作为选读，不扩大本章四节的必修范围。

A.1 秦九韶算法

设

$$p(x) = a_0 x^n + a_1 x^{n-1} + \cdots + a_n.$$

逐项独立求幂再求和会重复计算。把式子重组为

$$p(x) = \big(\cdots((a_0 x + a_1)x + a_2)x + \cdots\big)x + a_n$$

得到递推

$$b_0 = a_0, \qquad b_i = x b_{i-1} + a_i, \quad i = 1, \dots, n. \tag{1.42}$$

归纳可知，$b_i = a_0 x^i + a_1 x^{i-1} + \cdots + a_i$，因此 $b_n = p(x)$。该算法需要 n 次乘法与 n 次加法。若每项从头连乘求幂再乘系数，乘法总数为 $\sum_{k=1}^{n} k = n(n+1)/2$；若依次生成并复用各次幂，逐项求值也可达到线性量级。这个成本比较针对明确的实现，运算更少并不代替误差分析。如果只求函数值，还可使用一个累计变量，而无须保存所有 b_i。一般实现的系数顺序与缺项零系数必须明确。

对 $p(x) = 2x^4 - 3x^2 + 3x - 4$，在 $x = -2$ 处依次得到

$$(b_0, b_1, b_2, b_3, b_4) = (2, -4, 5, -7, 10).$$

这些小整数都可被 binary64 精确表示。下面用浮点系数和浮点输入运行，输出为 10.0；本例中的每个中间量也都能精确表示。

```
def horner(coefficients, x):
    value = coefficients[0]
    for a in coefficients[1:]:
        value = value * x + a
    return value

print(horner([2.0, 0.0, -3.0, 3.0, -4.0], -2.0))  # 10.0
```

令

$$q(x) = b_0 x^{n-1} + b_1 x^{n-2} + \cdots + b_{n-1},$$

其中 b_i 在给定点 ξ 计算，则

$$p(x) = (x - \xi) q(x) + b_n, \qquad p'(\xi) = q(\xi).$$

因此对 q 再使用一次秦九韶算法即可求导数。在上述例子中，中间量为 $2, -8, 21, -49$，得到 $p'(-2) = -49$。

例题出处：秦九韶算法与多项式示例见文献 1 §1.4.1 式(1.8)、例1.11；求值、求导推导及实际代码在本附录中完整给出。

A.2 开方迭代与停止条件

对于 $a > 0$，求 $\sqrt{a}$ 等价于解 $x^2 - a = 0$。设 $x_k > 0$，令下一次近似为 $x_k + \Delta x$。忽略校正量的二阶项，

$$x_k^2 + 2x_k \Delta x \approx a,$$

得到

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{a}{x_k}\right). \tag{1.43}$$

记 $r = \sqrt{a}$、$e_k = x_k - r$，在精确运算中有

$$e_{k+1} = \frac{e_k^2}{2x_k}. \tag{1.44}$$

由式 (1.44) 知 $x_1 \ge r$，且 $k \ge 1$ 时，$r \le x_{k+1} \le x_k$。数列因此单调有界，极限满足迭代方程，故收敛于 r。此时 $|e_{k+1}| \le |e_k|^2 / (2r)$，给出二次收敛的误差界。

表 1.13　从 $x_0 = 2$ 开始求 $\sqrt{3}$ 的实际迭代

k	x_k（保留16位小数）	绝对误差
0	2.0000000000000000	2.679×10^{-1}
1	1.7500000000000000	1.795×10^{-2}
2	1.7321428571428572	9.205×10^{-5}
3	1.7320508100147274	2.446×10^{-9}
4	1.7320508075688772	1.004×10^{-16}

表 1.13 使用 binary64 从 $x_0 = 2$ 求 $\sqrt{3}$。到第 4 次时，误差已经达到约 10^{-16} 的量级，有限精度限制开始主导。此后应结合误差估计判断是否停止。

实际程序可同时检查相邻迭代差、尺度归一化的残差和最大迭代次数。若 $r_f = x_k^2 - a$ 能可靠计算，则关系

$$|x_k - \sqrt{a}| = \frac{|r_f|}{x_k + \sqrt{a}}$$

可将残差转化为误差估计。有限精度下，残差为 0 仍可能来自舍入，应结合误差估计判断停止。

选读推导：一般 Newton 法的局部收敛

Newton 法用切线的零点更新近似：$x_{k+1} = x_k - f(x_k)/f'(x_k)$。

设 r 是简单根，即 $f(r) = 0$、$f'(r) \neq 0$；f 在 $|x - r| \le \rho$ 上二阶连续可微，且 $|f'(x)| \ge m > 0$、$|f''(x)| \le M$。记 $e_k = x_k - r$，Taylor 展开给出

$$0 = f(x_k) - f'(x_k)e_k + \tfrac{1}{2}f''(\xi_k)e_k^2, \qquad e_{k+1} = \frac{f''(\xi_k)}{2f'(x_k)}e_k^2.$$

由此 $|e_{k+1}| \le M|e_k|^2/(2m)$。若 $M\rho/(2m) \le 1/2$ 且 $|e_0| \le \rho$，则每步误差至少减半，迭代保持在区间内并趋于 r，同时满足上述二次界。有限精度下仍需检查残差和停止条件。

实验说明与配套材料

四节实验与四题数值验证保存在 A1组-实验.ipynb；完整机器数、参考值、误差与实际运行环境保存在 `A1组-结果/`。本地配套版号为 2026-10-11-A1-LOCAL。Notebook 的环境单元记录真实运行时刻和实际库版本。所有输入均为各节明确给出的无量纲确定性数学量，没有数据集采样或划分，不需要随机种子。

参考计算使用100位 Decimal 运算，指定开方、指数和对数参考另以120位复查。准确十进制输入由字符串或整数建立，实际浮点值由 `Decimal.from_float` 读取；误差由未缩写数值计算。积分参考采用正项级数前160项，尾项界与提高工作精度承担不同的核验任务。每种工作精度扫描151个差分步长，积分计算到9阶，分类实验只用所列logits；实际耗时记录在结果文件中。

不同数学库、平台或硬件可能改变末位和实测最佳步长；复现时应同时检查输入类型、误差定义、有限性与整体趋势。本章没有运行完整神经网络训练，标量梯度存储实验也不能代替训练收敛性验证。含NaN、无穷值或空数组的Softmax输入不属于本章约定的有限非空输入。

正文、实验、习题及解答的来源与AI使用记录见 A1组-编写报告.md（原稿所引文件未提供） 和 A1组-AI对话记录.md（原稿所引文件未提供）。人工签认与B/C独立复核状态由报告如实记录。

指数范围与有效位数应分别比较。binary16的有效精度为 $p = 11$，采用5位指数；bfloat16的有效精度为 $p = 8$，采用8位指数，其单位舍入误差为 $2^{-8} = 0.00390625$，最大有限正数约为 3.39×10^{38}，格式参数见[4]§1表1。同样是16位存储，bfloat16能覆盖更大的数量级，却有较粗的相对间距；应根据算法、数据尺度和硬件选择，不能只比较总位数。这是格式比较，不是本章已运行的bfloat16训练实验。

参考资料

[1] 李庆扬. 数值分析. 第6版. 北京：清华大学出版社，2025. 第1章：§1.2.3 例1.4（面积）；§1.3.1 例1.5（积分递推）；§1.3.2 例1.6（方程组）；§1.3.3 例1.7（小正根）；§1.4.1 式(1.8)、例1.11（秦九韶算法）. ISBN 9787302693833. https://www.tup.com.cn/booksCenter/book_10017001.html.

[2] Higham N J. Accuracy and Stability of Numerical Algorithms. 2nd ed. Philadelphia: SIAM, 2002. 舍入模型见§2.2；连乘界见§3.1引理3.1；求和误差分析见第4章（79--92页）；条件数与前后向误差见第1、7章. DOI: 10.1137/1.9780898718027.

[3] Goldberg D. What every computer scientist should know about floating-point arithmetic. ACM Computing Surveys, 1991, 23(1): 5--48. DOI: 10.1145/103162.103163. 可公开阅读的作者论文转载：https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html.

[4] Blanchard P, Higham D J, Higham N J. Accurately computing the log-sum-exp and softmax functions. IMA Journal of Numerical Analysis, 2021, 41(4): 2311--2330. DOI: 10.1093/imanum/draa038. 格式参数见§1表1；条件性见§2；平移算法见§4算法4.1，Softmax误差界见定理4.3；预印本：https://arxiv.org/abs/1909.03469.

[5] NumPy Developers. NumPy 2.3 documentation: numpy.finfo. 用于核对格式范围、eps 与非规格化数的定义. https://numpy.org/doc/2.3/reference/generated/numpy.finfo.html. 链接复查日期：2026-10-11.

[6] Python Software Foundation. Python 3.12 documentation: Floating-Point Arithmetic: Issues and Limitations. §15. https://docs.python.org/3.12/tutorial/floatingpoint.html. 数学库 `fsum` 说明：https://docs.python.org/3.12/library/math.html#math.fsum. 链接复查日期：2026-10-11.

[7] PyTorch Contributors. PyTorch 2.14 documentation: Automatic Mixed Precision package. 尤其参见 Gradient Scaling. https://docs.pytorch.org/docs/2.14/amp.html. 链接复查日期：2026-10-11.

支撑内容	具体来源位置	本次核查与边界
面积误差、小正根、正积分递推、秦九韶算法	[1]第1章中已标出的具体例号	题面、推导、算法和结果均在本章完整呈现；出版社页面已复查，原书具体例号沿用此前本地材料记录，组员仍需人工确认。
舍入模型、误差连乘界、求和与稳定性	[2]§2.2、§3.1引理3.1、第4章；[3]Rounding Error	说明假设并给出本章所需推导；网页核查是AI辅助读取，不代替学生人工核对。
平移Softmax与log-sum-exp	[4]§4算法4.1、定理4.3	本章自行给出恒等变换、范围分析和小误差展开；研究论文用于进一步阅读。
格式参数、显示与存储、梯度缩放	[5]numpy.finfo；[6]§15、math.fsum；[7]Gradient Scaling	官方文档链接于2026-10-11复查；本章实验只验证所列函数级计算。

开场采用本机Python实际输出；数轴图为本组用绘图程序生成的概念示意，误差曲线来自Notebook实测数据。未采用仓库初始化漫画，也未使用AI图像生成器。

本章习题与参考解答

第01章 误差的代价：课后习题（A1组（G20）本地稿）

配套修订版：2026-10-11-A1-LOCAL。四题依次对应1.1—1.4节；各含概念/推导、实现/数值验证、陷阱辨析。题中的AI建议均为教学构造，并非真实AI输出。均为课后练习，不增加课堂学时。本地稿已完成题面及AI辅助数值自检；组员人工确认尚未记录，B组核验和C组复核待完成。学生版PDF中只收录题面，不收录参考解答。

预计用时以已配置好的Notebook模板为前提，包含推导、运行和整理结果，首次环境准备另计；用时为教学估计，未经学生试做计时。每题先写预测再运行。

题1：对应1.1 浮点数与舍入误差

- 完整题面： 准确输入为十进制有理数 $1/5$。假想二进制格式采用规格化表示、有效精度 $p=4$、舍入到最近值；实际实验比较 IEEE binary32 和 binary64（最近舍入）。
- 推导或概念小问： 手算 $1/5$ 的二进制展开、$p=4$ 的舍入值、绝对与相对误差；说明 0.625 为何能被有限二进制精确表示，而 0.2 不能。
- 代码或结果分析小问： 分别把 `numpy.float32("0.2")` 和 `numpy.float64("0.2")` 构造为相应浮点数，提交实际存储值、对准确 $1/5$ 的有符号误差、绝对误差、相对误差。可用 Python 标准库、NumPy 和 `Decimal.from_float`；参考必须用准确的 `Decimal("0.2")` 或有理数 $1/5$，不得把浮点值的格式化输出作为参考。
- 陷阱辨析小问： 辨析以下说法：“屏幕都显示0.2，所以两个值都准确；binary64机器精度约为 10^{-16}，因此任何数值计算的绝对误差都不超过 10^{-16}。” 提交格式（按测试输入扩展行数）：

存储与误差

类型	实际存储值	有符号误差	绝对误差	相对误差

- 结果核对： 手算值和误差应精确；实测值与 IEEE 舍入一致，误差报告至少6位有效数字；清楚区分显示、单步相对舍入界与最终算法误差。
- 难度、预计课后用时与来源： 基础，约25分钟；本章§1.1及正文参考资料中的 Goldberg、Python 浮点文献。

题2：等价公式与差分步长（对应1.2）

- 完整题面： 计算 $d(x)=\sqrt{x+1}-\sqrt{x}$：binary32 取准确整数 $x=2^{24}$，binary64 取 $x=2^{53}$。另以 $D_h=[e^{1+h}-e]/h$ 近似 e，步长集合为 $\{10^{-1},10^{-4},10^{-8},10^{-12},10^{-16}\}$。所有低精度运算在所标精度内完成；高精度参考使用准确 x 和实际存储的 h。
- 推导或概念小问： 有理化 $d(x)$；说明为何直接式可能为零而有理化式仍为正。求 D_h-e 的首个非零截断项；仅在舍入项可由 Bu/h 表述的模型下，推导平衡步长量级并说明 B 依赖运算路径。
- 代码或结果分析小问： binary32只取 $x=2^{24}$，binary64只取 $x=2^{53}$；分别比较直接式和 $1/(\sqrt{x+1}+\sqrt{x})$，列参考值及绝对误差。对所列五个 h 提交 D_h、绝对误差、`fl(1+h)==1` 的真值，并指出本集合最优步长。可用 Python 标准库、NumPy 和至少80位十进制参考；表或误差—步长图均可。
- 陷阱辨析小问： 辨析以下说法：“步长越小越准；代数等价保证浮点实现同样可靠。”指出各自的失败输入及原因。 提交格式（按测试输入扩展行数）：

差额

类型与 x	参考值	直接值	直接绝对误差	有理化值	有理化绝对误差

差分

类型	实际 h	D_h	绝对误差	$\mathrm{fl}(1+h)=1$?

- 结果核对： 有理化恒等式、$D_h - e = eh/2 + O(h^2)$ 和平衡量级正确；失败区间、误差指标、独立参考与运行环境可复核。不同数学库引起的末位或实测最优点差异，若有数据支持，可接受。
- 难度、预计课后用时与来源： 中等，约40分钟；本章§1.2及正文参考资料中的 Higham 文献。

题3：对应1.3 条件数与稳定性

- 完整题面： 实验A对 $F(x) = \ln x$ 从 $x = 1.0001$ 作正向相对扰动 η，观察准确输出的相对变化。实测：$\eta = 10^{-8}$ 时输出变化 $1.000049994 \times 10^{-4}$，$\eta = 10^{-6}$ 时变化 $1.000049499 \times 10^{-2}$。实验B以 $I_n = e^{-1}\int_0^1 t^n e^t\,dt$ 为准确目标，分别用 $I_n = 1 - nI_{n-1}$、初值 $\widehat{I_0} = 0.6321$ 正向递推，以及 $I_{n-1} = (1 - I_n)/n$、终值 $\widehat{I_9} = 0.0684$ 反向递推（binary64）。数据如下：

n	准确参考	正向值	反向值
0	0.63212056	0.63210000	0.63212062
8	0.10093197	-0.72800000	0.10351111
9	0.09161229	7.55200000	0.06840000

- 推导或概念小问： 求 F 的相对条件数与两种递推的初始误差传播因子；依表判断实验A的主要现象和实验B的主要现象，并解释反向终值为何仍不准确。
- 代码或结果分析小问： 用正项级数 $I_n = e^{-1}\sum_{k=0}^{\infty} 1/[k!(n+k+1)]$（至少100项且核验尾项，推导见正文选读“怎样保证参考值可靠”）或高精度数值积分，独立复核 I_0, I_8, I_9；报告两条递推各自的绝对误差。区分初值误差与新增舍入误差。
- 陷阱辨析小问： 辨析以下说法：“反向在 $n = 0$ 很准，故原问题良态；对数问题病态，故对数算法必定错误。”说明条件数、输入模型及算法稳定性分别回答什么。 提交格式（按测试输入扩展行数）：

积分递推

n	独立参考	正向值	正向绝对误差	反向值	反向绝对误差

另注明参考精度与尾项界。

- 结果核对： $\kappa_F(x) = 1/|\ln x|$，忽略新增舍入时，正向初值误差的大小按 $n!$ 放大；从末端 N 反向递推到 n，末值误差的大小乘以 $n!/N!$，本题 $N = 9$；对 $n = 0, 8, 9$ 分别给出正向值与反向值的绝对误差，共六个误差值。说明积分定义与任意初值递推问题有不同条件性。
- 难度、预计课后用时与来源： 中等，约40分钟；本章§1.3及正文参考资料中的 Higham 文献。

题4：对应1.4 AI关联——训练中的数值问题

- 完整题面： 待辨析的实现建议：“Softmax先直接算 e^{z_i}，除以指数之和即可。只要训练损失下降、程序没有NaN，就说明数值实现稳定。”测试 logits $A = (0, 1, 2)$、$B = (1000, 1001, 1002)$、$C = (-1000, -999, -998)$；另取 $T = (0, -1000)$，真实类别为第2类。比较 binary32 和 binary64。不要求运行完整训练。
- 推导或概念小问： 人工写出减最大值的 Softmax、指数和的对数（log-sum-exp）和交叉熵；对有限 logits 证明 $1 \le \sum_i e^{z_i - m} \le n$，解释 T 的低概率类可能下溢而交叉熵仍可计算。

- 代码或结果分析小问： 对 A, B, C，在两种精度下运行直接版与平移版；提交“输出全有限”、$E_\infty = \max_i |\hat{p}_i - p_i^{ref}|$、$E_\Sigma = |\sum_i \hat{p}_i - 1|$。在 T 上比较先求概率再取负对数与稳定对数路径。参考概率至少80位并以更高精度复查。
- 陷阱辨析小问： 逐条辨析建议；选出概率输出全部有限、但先求概率的损失路径仍失败的输入，解释原因；判断损失下降能否证明数值实现正确，还应检查哪些量，并结合本题测试说明。 提交格式（按测试输入扩展行数）：

概率

输入组	类型	实现	全有限?	E_∞	E_Σ

T 的损失

类型	尾概率	概率路径损失	对数路径损失

- 结果核对： 两种精度中直接版 B, C 出现非有限输出；平移版 A, B, C 有限，且本题三维测试的 $E_\infty < 10^{-6}$（binary32）、$E_\infty < 10^{-14}$（binary64）。须报告每组 E_Σ 实测值。T 的概率路径损失为非有限，而对数路径约1000。报告环境、代码和全部输入；阈值不推广到任意维数或极小尾部概率，不能以归一化替代尾概率的相对误差检查。
- 难度、预计课后用时与来源： 中等，约45分钟；本章§1.4及正文参考资料中的 Blanchard、Higham、Higham 文献。

四题的复现实验位于 A1组-实验.ipynb 的“课后习题数值核验”部分；原始结果位于 `A1组-结果/exercise_results.json`。

第01章 误差的代价：习题参考解答（A1组（G20）本地稿）

配套修订版：2026-10-11-A1-LOCAL。题号与A1组-习题.md一一对应。本文件不纳入学生版教材目录；公开仓库中的答案同样公开。本地稿已完成推导说明、实际运行和AI辅助自检；组员人工签认尚未记录；B组核验、C组复核均待完成，不得将本稿表述为已获B/C通过。

运行环境、命令、实际运行时刻与依赖版本见 A1组-编写报告.md（原稿所引文件未提供） 及 `A1组-结果/environment.json`。参考计算用100位十进制精度，关键量用120位复查。四题代码与实测输出位于 A1组-实验.ipynb 的“课后习题数值核验”部分；原始输出为 `A1组-结果/exercise_results.json`。误差均由实际浮点存储值与准确参考之差计算，表格为便于阅读保留部分位数。所有代码在独立的新内核按顺序实际执行并保存；这次执行由AI工具驱动，未代替学生在JupyterLab中的人工运行与审阅。

题1：对应1.1 浮点数与舍入误差

小问索引：(a)推导；(b)实现、参考与结果；(c)辨析。

- 条件与符号： 准确 $x = 1/5$，存储值 $\hat{x}$；有符号误差为 $\hat{x} - x$，绝对误差为 $|\hat{x} - x|$，相对误差为 $|\hat{x} - x|/|x|$。假想 $p = 4$ 的首位二进制数字计入有效位数。

- 手动推导或概念解释（以下为AI辅助整理，待组员逐步核验）：
  $1/5 = (0.0011001100\ldots)_2 = (1.1001100\ldots)_2 2^{-3}$；保留四位后的邻数为0.1875与0.203125，中点为0.1953125。准确值0.2严格超过中点，故最近舍入为 $(1.101)_2 2^{-3} = 13/64 = 0.203125$。绝对误差 $1/320 = 0.003125$，相对误差 $1/64 = 0.015625$。$0.625 = 5/8 = (0.101)_2$ 可有限表示；$1/5$ 因分母含因子5，二进制展开不终止。
- 实现位置与独立参考： Notebook §题1，代码单元 `exercise-1`。用 `Decimal("0.2")` 表示准确值，`Decimal.from_float(float(value))` 揭示存储值；二者不同路径。最近舍入选择与准确值距离最小的机器数，中点时选择有效数最低位为零者；文献来源为 Goldberg。
- 实际输出、误差和容差理由：两种格式的准确存储值分别为：
  - float32：`0.200000002980232238769531250`。
  - float64：`0.200000000000000011102230246251565404236316680908203125`。

类型	有符号误差	绝对误差	相对误差
binary32	$+2.98023223877 \times 10^{-9}$	$2.98023223877 \times 10^{-9}$	$1.49011611938 \times 10^{-8}$
binary64	$+1.11022302463 \times 10^{-17}$	$1.11022302463 \times 10^{-17}$	$5.55111512313 \times 10^{-17}$

  两者格式化到小数1位均显示0.2。两项误差为正，故绝对误差恰与有符号误差的大小相同。存储值由IEEE舍入确定；有限位打印的误差至少保留6位有效数字。
- 陷阱的正确解释及适用范围： 显示精度不等于存储值准确性。机器单位舍入误差只在特定单步浮点模型下控制相对舍入误差，并非任意算法最终绝对误差的上界；抵消、累积、条件性、范围问题都可能主导。binary64中 $10^{16}+1$ 舍为 10^{16}，相对于准确整数之和的绝对误差为1。
- B核验、C复核及版本： AI辅助推导与本环境运行完成；B核验待完成，C复核待完成；2026-10-11-A1-LOCAL。

题2：等价公式与差分步长（对应1.2）

小问索引：(a)推导；(b)实现、参考与结果；(c)辨析。

- 条件与符号： binary32取准确整数 $x = 2^{24}$，binary64取 $x = 2^{53}$；差分参考使用实际存储的 h。$d(x) > 0$；导数准确值为 e。
- 手动推导或概念解释（以下为AI辅助整理，待组员逐步核验）： $d(x) = 1/(\sqrt{x+1} + \sqrt{x})$。对有效精度为 p 的格式，$x = 2^p$ 右侧的相邻数是 $x+2$。准确的 $x+1$ 恰在中点，按最近舍入、遇中点取偶数的规则选择最低有效位为0的 x，故 `fl(x+1)==x`，直接版返回0，而实数值仍为正。泰勒展开得 $D_h = e + (e/2)h + O(h^2)$。在 $A, B > 0$ 的模型 $E(h) = Ah + Bu/h$ 下，$E'(h) = A - Bu/h^2 = 0$ 给出 $h_* = \sqrt{Bu/A}$；$E''(h) = 2Bu/h^3 > 0$，所以这是模型最小点。平衡量级为 $\sqrt{u}$；系数取决于实现，不能拿它当精确实测极小点。
- 实现位置与独立参考： Notebook §题2，代码单元 `exercise-2`。准确整数输入经100位 `Decimal.sqrt` 得有理化参考，120位复查；导数参考为100位 `Decimal(1).exp()`，120位复查。数值误差以实际存储的浮点值计算。
- 实际输出、误差和容差理由：

类型、x	参考 $d(x)$	直接值	有理化值	有理化绝对误差
binary32，2^{24}	$1.220703106810\times10^{-4}$	0	1.220703125×10^{-4}	1.819×10^{-12}
binary64，2^{53}	$5.268356063862\times10^{-9}$	0	$5.26835606386\times10^{-9}$	3.208×10^{-25}

类型	h	D_h	绝对误差	`fl(1+h)==1`
binary32	10^{-1}	2.85884141922	1.405596×10^{-1}	否
binary32	10^{-4}	2.71797180176	3.100267×10^{-4}	否
binary32	$10^{-8},10^{-12},10^{-16}$	0	2.718282	是
binary64	10^{-1}	2.85884195487	1.405601×10^{-1}	否
binary64	10^{-4}	2.71841774708	1.359186×10^{-4}	否
binary64	10^{-8}	2.71828182186	6.602751×10^{-9}	否
binary64	10^{-12}	2.71871414270	4.323142×10^{-4}	否
binary64	10^{-16}	0	2.718282	是

binary32合并列出的三个失败步长分别运行，结果均为0。在所列五个步长内，binary32最优为 10^{-4}，binary64最优为 10^{-8}。不同数学库可能改变末位或实测集合内的最优点；可接受可复现且解释明确的差异，不能接受未说明环境或以另一低精度公式代替参考。

- 陷阱的正确解释及适用范围： 缩小步长会减少截断误差，但分子抵消与输入舍入会令总误差回升，最终可得到0。实数等价不保证浮点路径同样可靠；有理化也不自动解决所有极端尺度的范围问题。
- B核验、C复核及版本： AI辅助推导、运行与120位复查完成；B核验待完成，C复核待完成；2026-10-11-A1-LOCAL。

题3：对应1.3 条件数与稳定性

小问索引：(a)推导；(b)实现、参考与结果；(c)辨析。

- 条件与符号： 实验A的扰动作用于准确输入；实验B真实目标由积分定义，递推仅是两种实现。δ_n 表示相应近似与真实 I_n 的差。
- 手动推导或概念解释（以下为AI辅助整理，待组员逐步核验）： 对 $F(x)=\ln x$，$\kappa_F(x)=|xF'(x)/F(x)|=1/|\ln x|$，在1.0001约为10000.4999917。积分分部积分给出 $I_0=1-e^{-1}$、$I_n=1-nI_{n-1}$。忽略新增舍入时，正向 $\delta_n=(-n)(-(n-1))\cdots(-1)\delta_0=(-1)^n n!\delta_0$；反向 $\delta_{n-1}=-\delta_n/n$，从末端 N 到 n 有 $\delta_n=(-1)^{N-n}(n!/N!)\delta_N$（$0\le n\le N$）。大小因子来自 $|\delta_n|=|\delta_N|/[N(N-1)\cdots(n+1)]=(n!/N!)|\delta_N|$（$n<N$）：向前反复乘整数，向后反复除整数，$n=N$ 时因子为1。本题 $N=9$，末值误差传到 $n=0,8,9$ 时的大小因子依次为 $1/9!$、$1/9$、1；反向终值本身仍是给定粗近似。
- 实现位置与独立参考： Notebook §题3，代码单元 `exercise-3`。实验A用100位十进制对数；(b)的参考值精度核验：实验B用正项级数 $I_n=e^{-1}\sum_{k\ge0}1/[k!(n+k+1)]$ 前160项，不调用被检查的递推生成参考。令 $a_k=1/[k!(n+k+1)]$，$k\ge160$ 时有 $a_{k+1}/a_k=(n+k+1)/[(k+1)(n+k+2)]<1/161$，故

$0 < R_{160} < e^{-1}/[160!(n+161)] \cdot 1/(1-1/161)$。对 $n = 0, 8, 9$，尾项界均小于 4.88×10^{-288}。另以120位重算，报告位数不变；提高运算精度不能替代尾项界。

- 实际输出、误差和容差理由： 实验A在 $\eta = 10^{-8}$ 与 10^{-6} 时输出相对变化约为 $1.000049994 \times 10^{-4}$ 与 $1.000049499 \times 10^{-2}$，分别与一阶预测吻合。实验B：

n	参考	正向绝对误差	反向绝对误差
0	0.6321205588	2.05588×10^{-5}	6.39669×10^{-8}
8	0.1009319674	8.28932×10^{-1}	2.57914×10^{-3}
9	0.0916122930	7.46039	2.32123×10^{-2}

  前向在 $n = 8$ 得到 -0.728，已违背正积分；在 $n = 9$ 得到7.552。该结论主要展示前向初值误差的放大；反向低阶值较准，不能推出反向整张表都准。不同实现的新增舍入可影响最后几位，但前向 $n!$ 与从 N 到 n 的反向 $n!/N!$ 的误差大小因子不变。

- 陷阱的正确解释及适用范围： 某个输出准确，不证明原问题对所有输入扰动良态；对数在近1处相对病态，也不意味着任一对数实现必然错误。若把 I_0 改成任意输入并把递推当成另一个数学问题，其条件数需另作分析；不能与原积分问题混同。

- B核验、C复核及版本： AI辅助推导、运行及独立级数复查完成；B核验待完成，C复核待完成；2026-10-11-A1-LOCAL。

题4：对应1.4 AI关联——训练中的数值问题

小问索引：(a)推导；(b)实现、参考与结果；(c)辨析。

- 条件与符号： 题面建议为教学构造，非真实AI引语。p_i 是准确Softmax概率，$\hat{p}_i$ 是浮点计算值；$E_\infty = \max_i |\hat{p}_i - p_i|$，$E_\Sigma = |\sum_i \hat{p}_i - 1|$。$T$ 的真实类别为第2类。

- 手动推导或概念解释（以下为AI辅助整理，待组员逐步核验）： 令 $m = \max_i z_i$。则 $p_i = e^{z_i - m}/\sum_j e^{z_j - m}$，$\mathrm{LSE}(z) = m + \log \sum_j e^{z_j - m}$，$\ell_y = \log \sum_j e^{z_j - m} - (z_y - m)$。因 $z_i - m \le 0$ 且至少一个为0，每项指数在 $(0, 1]$，至少一个为1，故 $1 \le \sum_j e^{z_j - m} \le n$。$T = (0, -1000)$ 中第2类准确概率约 $5.0759589 \times 10^{-435}$，两种精度均存为0；但 $\ell_2 = 1000 + \log(1 + e^{-1000}) \approx 1000$ 可由对数式算出。

- 实现位置与独立参考： Notebook §题4，代码单元 `exercise-4`，本Notebook中对应的函数定义。参考概率由100位十进制运算建立，并以120位复查。程序对两种精度同时运行直接指数版和平移版。

  演示中，`np.errstate` 仅用于暂时忽略预期的上溢、下溢、除零和无效运算告警；程序仍检查输出是否有限，忽略告警不改变计算结果。

  平移及对数损失的推导已在本题上方给出，Blanchard、Higham、Higham文献作为研究出处保留。

- 实际输出、误差和容差理由： A, B, C 的准确概率相同，约 $(0.09003057317, 0.24472847105, 0.66524095577)$。先看B、C的有限性，再看A的误差；避免范围失效不等于每个误差指标都下降。两种实现的指标分别列出；归一化偏差用高精度累加实际存储概率计算。直接版在B、C上为NaN，不能作有限误差比较。

输入	类型	实现	全有限	E_∞	E_Σ
A	binary32	直接版	是	$1.96554608 \times 10^{-8}$	$7.45058060 \times 10^{-9}$
A	binary32	平移版	是	1.23430×10^{-8}	2.23517×10^{-8}
A	binary64	直接版	是	$1.08467934 \times 10^{-17}$	0
A	binary64	平移版	是	1.04544×10^{-16}	1.11022×10^{-16}
B	binary32	直接版	否	NaN	NaN
B	binary32	平移版	是	1.23430×10^{-8}	2.23517×10^{-8}
B	binary64	直接版	否	NaN	NaN
B	binary64	平移版	是	1.04544×10^{-16}	1.11022×10^{-16}
C	binary32	直接版	否	NaN	NaN
C	binary32	平移版	是	1.23430×10^{-8}	2.23517×10^{-8}
C	binary64	直接版	否	NaN	NaN
C	binary64	平移版	是	1.04544×10^{-16}	1.11022×10^{-16}

平移版避免本题B、C的范围失效，但不保证每次运行的每个误差指标更小。本次A的binary64直接版两个指标都小于平移版；范围适用性与具体准确性应分别判断。

$\boldsymbol{T}$ 的结果（binary32、binary64相同）：尾概率0；概率路径损失 $+\infty$；对数路径损失1000。

建议验收界：binary32 的 $\boldsymbol{E_\infty < 10^{-6}}$，binary64 的 $\boldsymbol{E_\infty < 10^{-14}}$；两界宽于本机末位差异，但能识别明显失败。归一化误差须独立报告，不能代替分量误差或尾概率相对误差。

- 陷阱的正确解释及适用范围： 未出现NaN仍可能出现无穷值；例如 `np.isnan(np.inf)` 和 `np.isfinite(np.inf)` 都为False。本题T的概率为(1,0)，先取负对数得到的正无穷没有NaN，却不是有限值。即使输出全有限，也不能据此判断准确；应分别检查有限性，将概率、损失等量与独立参考比较。损失下降也不说明概率、梯度或参数更新都准确。应以独立参考、分量误差、极端输入和不同精度复算检查。本题只运行函数级CPU实验，未运行完整模型训练，不能证明收敛。
- B核验、C复核及版本： AI辅助推导、运行与120位复查完成；B核验待完成，C复核待完成；2026-10-11-A1-LOCAL。

第02章 逼近的智慧

> 编写组：A1（G8）；来源版本：`230873df074c`，main已合并稿。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

用有限的信息，换取可信的答案

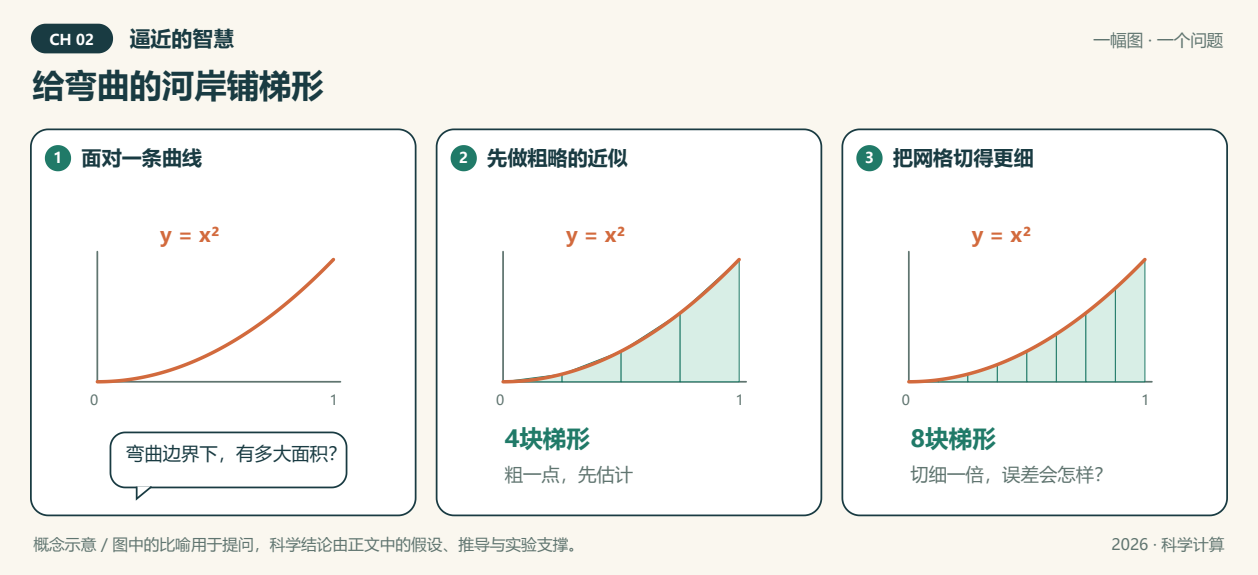


图02-1 • 给弯曲的河岸铺梯形。概念示意，非实测结果。

章节引入

丈量河岸边一块弯曲的地，尺子拉不直，面积也算不出来。常用的办法是"铺梯形"：把河岸切成若干段，每段用一根直尺代替弯曲线，算出各块梯形面积，再相加。段切得越细，答案越接近真值。

这个做法包含三个问题。

1. 用直线代替曲线，依据是什么？换一条更合适的曲线，结果会不会更好？
2. 代替之后误差有多大？能否在计算之前估计误差？
3. 如果曲线不是由公式给出，而是一组测量数据，还能不能代替？

这三个问题分别对应插值、误差估计与离散数据拟合。它们的历史线索并不相同，却指向同一套工具。

图02-2 • 逼近论的五位奠基者。标注年份为该工作的公开年份。

1795 年，拉格朗日在巴黎高等师范学校的讲稿中给出一个公式：给定 $n+1$ 个点，存在唯一的次数不超过 n 的多项式穿过它们。这就是 2.1 节的 Lagrange 公式。

1805 年，勒让德计算彗星轨道时遇到另一类问题：观测数据多于待定参数，且每个观测都带误差。让曲线穿过所有点，等于把误差当作数据。他改为要求偏差的平方和最小。1809 年，高斯在《天体运动论》中称自己从 1795 年起就这样计算。方法沿用至今，称为最小二乘。

1854 年，切比雪夫研究把圆周运动变为直线运动的连杆机构。机构只能近似走直线，问题于是变成：什么是最优近似？他的答案是让误差在若干点上正负交替、幅度相等，即"最坏处也尽量不坏"。由此得到一族以他命名的多项式，也得到 2.1 节末节点位置的答案。

1885 年，魏尔斯特拉斯证明：闭区间上的连续函数都可用多项式一致逼近到任意精度。存在性问题由此解决。

1901 年，龙格给出反例：一个任意阶光滑的函数，用等距节点插值，次数越高，两端振荡越剧烈。这说明"存在一个好多项式"与"给定的节点能算出它"是两件事。

本章讨论的逼近，是在信息有限的条件下，说明简单函数能替代到什么程度、代价是多少。

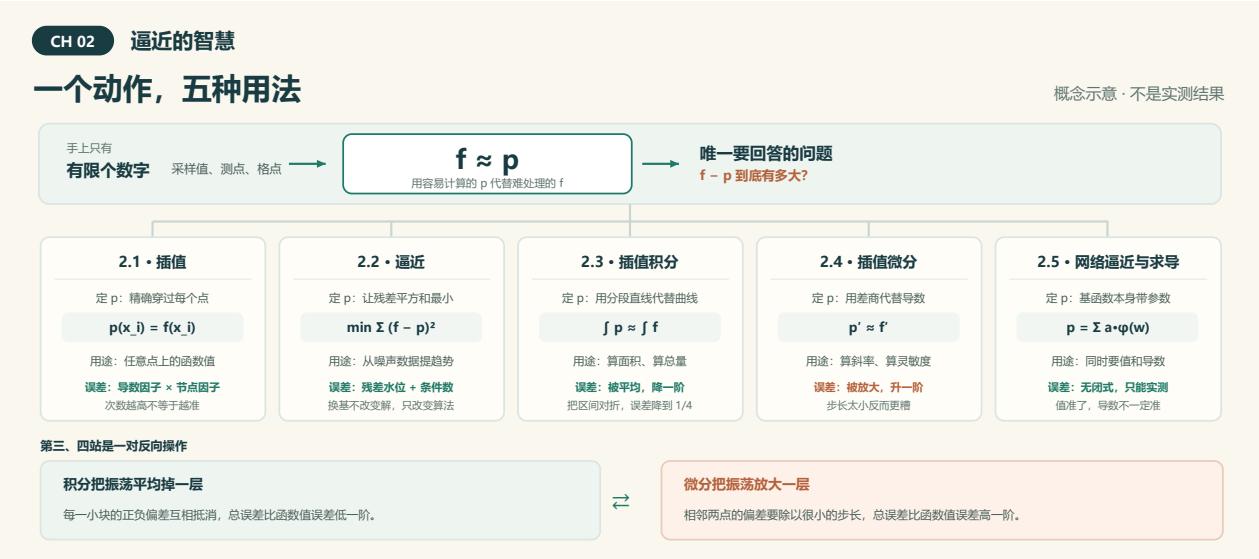


图02-3 • 本章五节的结构。

一条主线：一个公式，三种用法

全章围绕插值余项展开：

式（0.1）

$$f(x)-p_n(x)=\frac{f^{(n+1)}(\xi_x)}{(n+1)!}\,\omega_{n+1}(x),$$

$$\omega_{n+1}(x)=\prod_{k=0}^{n}(x-x_k).$$

右端是两个因子的乘积：一个只与函数有关，即高阶导数；一个只与节点有关，即节点多项式。这一分解是后面各节结论的来源。

对式 (0.1) 所作的运算	得到的结论	所在节
取绝对值、取最大	插值误差上界	2.1
改求残差平方和最小	最小二乘与基函数	2.2
两边对 x 积分	求积公式的误差	2.3
两边对 x 求导	差分公式的误差	2.4
把节点值换成带参数的基函数	网络逼近与求导	2.5

各节内容如下：

- 2.1 插值：给定节点与函数值，构造穿过它们的多项式；
- 2.2 逼近：数据多于待定系数且含噪声时，放弃逐点通过；
- 2.3 插值积分：对插值多项式积分，得到求积公式；
- 2.4 插值微分：对插值多项式求导，得到差分公式；
- 2.5 网络逼近：让基函数本身含参数。

每一节都回答同一个问题：近似在何处出错、误差多大、在什么条件下会失效。

记号与约定

- 全章各量无量纲，误差均指绝对误差；"最大误差"指在指定网格或区间上取的最大值。
- n 表示多项式的次数，节点数为 $n+1$。
- $\mathcal{P}_n$ 表示次数不超过 n 的实多项式全体；$C^r[a,b]$ 表示在 $[a,b]$ 上有直到 r 阶连续导数的函数全体。
- 范数、内积、正交与条件数的定义见附录 A、附录 B。
- 先修内容为微积分（Rolle 定理、Taylor 公式、定积分）、线性代数（矩阵、秩、内积），以及浮点数与实数的区别。

本章开头的问题可以先记下：区间数由 4 增至 8，梯形公式的误差变为原来的几分之一？2.3 节将给出精确到分数的答案。2.4 节将说明，同样加密步长，求导的误差为什么不一定随之减小。

2.1 插值：从点到线（2学时）

本节目标：写出插值多项式并证明其唯一性；把插值误差分解为函数因子与几何因子；说明节点分布对误差的影响。

概念讲解：把问题说明白

沿管道布置若干温度传感器，已知各传感器的位置与读数，要求估计两个传感器之间以及管道端点的温度。传感器位置称为节点，读数称为数据。用有限个点重建一条曲线，就是插值。

这里有两个不同的问题：

- 节点上的值是否正确，是线性代数问题，涉及存在性与唯一性；
- 节点之间的值是否准确，是分析问题，涉及误差估计。

前者只要节点互异即可保证。后者不然。即使每个读数精确到小数点后四位，节点之间的误差仍可能很大。极端情形是所有传感器都集中在管道左端，此时左端拟合得再准，右端的推测也只是外推。

本节按三步展开：2.1.1 建立插值问题，给出 Lagrange 公式及其便于计算的等价形式；2.1.2 导出余项，把误差分解为两个因子；2.1.3 固定函数、只改变节点，讨论第二个因子能压到多小，并给出一个反例。

本节只讨论一元多项式插值，不涉及样条、有理逼近与多元插值。先讲多项式，是因为它的误差可以分解，这一性质在后面几节反复使用。

关键推导：让结论有依据

2.1.1 插值问题与 Lagrange 公式

定义 2.1.1（多项式插值） 设 $x_0,\dots,x_n$ 是 $[a,b]$ 中 $n+1$ 个互异实数，对应数据为 $y_0,\dots,y_n$。若 $p_n\in\mathcal{P}_n$ 满足

$$p_n(x_j)=y_j,$$
$$j=0,1,\dots,n,$$

则称 p_n 为这组数据的插值多项式，x_j 称为节点。若 $y_j=f(x_j)$，也称 p_n 为 f 的插值多项式。

定义中"次数不超过 n"不能省去。若允许任意次数，在任一解上加 $c\prod_{k=0}^{n}(x-x_k)$ 仍满足同样的数据，解不唯一。

定理 2.1.1（存在唯一性与 Lagrange 公式） 对定义 2.1.1 中的数据，插值多项式存在且唯一，并可写成

式（2.1.1）

$$p_n(x)=\sum_{j=0}^{n}y_j\ell_j(x),$$

$$\ell_j(x)=\prod_{\substack{k=0\\k\neq j}}^{n}\frac{x-x_k}{x_j-x_k}.$$

证明 节点互异，故各分母不为零。每个 ℓ_j 的次数不超过 n，且 $\ell_j(x_i)=1$（$i=j$）、$\ell_j(x_i)=0$（$i\neq j$）。于是式 (2.1.1) 属于 $\mathcal{P}_n$，且在每个节点取到 y_i，存在性得证。

设 p_n, q_n 都插值同一组数据，则 $p_n - q_n \in \mathcal{P}_n$ 有 $n+1$ 个互异零点。次数不超过 n 的非零多项式至多有 n 个互异零点，故 $p_n \equiv q_n$。□

证明只用到两点：分母不为零，以及零点个数受限。2.2 节证明最小二乘解的唯一性时，用的是同一论证。

例 2.1.1 取节点 $-1, 0, 1$，数据 $1, 0, 1$。三个基函数为

$$\ell_0 = \frac{x(x-1)}{2},$$

$$\ell_1 = 1 - x^2,$$

$$\ell_2 = \frac{x(x+1)}{2},$$

故 $p_2 = \ell_0 + \ell_2 = x^2$。一般地，若数据来自某个 $q \in \mathcal{P}_n$，由唯一性得 $p_n = q$。这一性质称为多项式再现性，可作为检验程序的第一步：用低次多项式造数据，程序应在舍入误差内原样还原。

Lagrange 公式便于证明，不便于反复求值，因为每算一个点都要重算全部乘积。为此引入节点多项式与重心权重

式（2.1.2）

$$\omega_{n+1}(x) = \prod_{k=0}^{n} (x - x_k),$$

$$w_j = \frac{1}{\prod_{k \neq j} (x_j - x_k)},$$

得到下面的等价写法。

命题 2.1.2（重心求值公式） 对所有非节点 x，

式（2.1.3）

$$p_n(x) = \frac{\displaystyle\sum_{j=0}^{n} \frac{w_j y_j}{x - x_j}}{\displaystyle\sum_{j=0}^{n} \frac{w_j}{x - x_j}}.$$

证明 由 $\ell_j(x) = \omega_{n+1}(x) w_j / (x - x_j)$。对常数函数 1 应用定理 2.1.1，得 $\sum_j \ell_j(x) \equiv 1$，即 $\sum_j w_j/(x - x_j) = 1/\omega_{n+1}(x) \neq 0$。两式相除即得 (2.1.3)，公共因子 $\omega_{n+1}(x)$ 约去。□

权重只与节点有关，可预先算好。此后每求一个值只需两个长度为 $n+1$ 的和，数值上也比连乘稳定。代价是求值点落在节点上时须单独处理，否则除零。重心公式只是同一个多项式的另一种算法，不改变节点选择带来的误差。

2.1.2 余项

插值条件保证节点上误差为零。节点之间的误差由下述定理给出。

定理 2.1.3（插值余项） 设 $f \in C^{n+1}[a, b]$，节点互异且都在 $[a, b]$ 内。对每个非节点 $x \in [a, b]$，存在 ξ_x 位于 $x, x_0, \dots, x_n$ 的最小闭包区间内部，使

式（2.1.4）

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!}\, \omega_{n+1}(x).$$

证明 固定非节点 x，令

$$K=\frac{f(x)-p_n(x)}{\omega_{n+1}(x)},$$

$$G(t)=f(t)-p_n(t)-K\omega_{n+1}(t).$$

G 在 $x_0,\dots,x_n,x$ 这 $n+2$ 个互异点为零。相邻零点之间用一次 Rolle 定理，得 G' 的至少 $n+1$ 个零点；重复 $n+1$ 次，得 $G^{(n+1)}(\xi_x)=0$。因 $p_n^{(n+1)}=0$、$\omega_{n+1}^{(n+1)}=(n+1)!$，解出 K 即得 (2.1.4)。当 x 为节点时，插值条件直接给出误差为零。□

对 (2.1.4) 取绝对值再取上确界：

式（2.1.5）

$$\|f-p_n\|_{\infty,[a,b]}\le \underbrace{\frac{\|f^{(n+1)}\|_{\infty,[a,b]}}{(n+1)!}}_{\text{函数因子}}\cdot\underbrace{\|\omega_{n+1}\|_{\infty,[a,b]}}_{\text{几何因子}}.$$

右端两个因子互相独立：第一个取决于函数的高阶导数，第二个取决于节点的位置。提高精度于是分成两项可以分别处理的任务。2.2 节放松逐点通过的条件，2.3 节对 (2.1.4) 积分，2.4 节对它求导，2.5 节把节点换成可训练的参数。

对 (2.1.5) 有三点说明。

- 它是上界，不是等式。上界大不意味着实际误差一定大。
- n 增大时导数阶数也随之增大，$\|f^{(n+1)}\|_\infty$ 不能视为与 n 无关的常数。
- 它要求 $f\in C^{n+1}$。光滑性不足时公式失效，收敛阶可能下降，2.3 节将给出 $h^{\frac{3}{2}}$ 的例子。

2.1.3 节点的选择

函数不能改变，可以改变的只有几何因子。问题化为：在所有首一的 $n+1$ 次多项式中，哪一个在 $[-1,1]$ 上的最大模最小？

定理 2.1.4（首一多项式的最小最大模） 对任意首一的 $n+1$ 次实多项式 q，

式（2.1.6）

$$\|q\|_{\infty,[-1,1]}\ge 2^{-n}.$$

等号由 $\omega_{n+1}=2^{-n}T_{n+1}$ 达到，其中 T_k 由 $T_k(\cos\theta)=\cos k\theta$ 定义。因此取 T_{n+1} 的零点作节点时几何因子最小：

式（2.1.7）

$$x_j=\cos\frac{(2j+1)\pi}{2n+2},$$

$$j=0,1,\dots,n.$$

证明 记 $r=n+1$、$C=2^{-n}$，则 $Q=CT_r$ 首一且 $\|Q\|_\infty=C$。反设存在首一多项式 q 使 $\|q\|_\infty<C$。在 $r+1$ 个点 $t_k=\cos(k\pi/r)$ 上，$Q(t_k)=C(-1)^k$，故 $q(t_k)-Q(t_k)$ 的符号严格交替。由介值定理，相邻两点之间各有一个零点，共 r 个互异零点。但 $q-Q$ 的首项相消，次数不超过 $r-1$，不可能有 r 个零点。矛盾。

又 Q 与 ω_{n+1} 都是首一 r 次多项式且零点相同，故相等。□

证明中"正负交替、幅度相等"的构造，正是切比雪夫研究连杆机构时所用的思想：最坏的几处一样坏，就没有更坏的地方。

把 (2.1.6) 代入 (2.1.5)，得本章最常用的误差界：

式（2.1.8）

$$\|f-p_n\|_{\infty,[-1,1]} \le \frac{\|f^{(n+1)}\|_{\infty,[-1,1]}}{2^n(n+1)!}.$$

区间为 $[a,b]$ 时，先作代换 $t=(2x-a-b)/(b-a)$ 把区间化为 $[-1,1]$，右端再乘尺度因子 $\big((b-a)/2\big)^{n+1}$。T_{n+1} 的零点靠近两端更密，且不含端点；另一族常用节点 $\cos(j\pi/n)$（$n\ge 1$，$j=0,\dots,n$）含端点，是极值点而非零点，两者不可混用，重心权重也不同（零点节点可取 $w_j=(-1)^j\sin\theta_j$，等距节点可取 $w_j=(-1)^j\binom{n}{j}$）。

扩展 · 龙格现象（1901）

定理 2.1.4 只说明几何因子可以很小，并不说明任意节点都适用。取

$$f(x)=\frac{1}{1+25x^2},$$
$$x\in[-1,1],$$

它在实轴上任意阶光滑。用等距节点插值，在 6001 点独立网格上实测（同一函数、同一公式、同一精度，只换节点族）：$n=8$ 时等距 1.0452、Chebyshev 1.7084×10^{-1}；$n=16$ 时 14.394 与 3.2614×10^{-2}；$n=32$ 时 5059.0 与 1.4017×10^{-3}。完整对照表见本节"结果解释"。

到 $n=32$，两列相差约 3.6×10^{6} 倍。等距列的误差每步放大约 13.8 倍与 351 倍，倍数本身在增大，这是加速恶化，不是随机波动。5059 这个数曾用 90 位十进制精度在峰值点独立复算，与 float64 结果相对差 5.91×10^{-9}，量级可信。

原因是等距节点下插值算子对数据扰动的放大倍数（Lebesgue 常数）随 n 指数增长，Chebyshev 零点把它压到只随 $\log n$ 增长。本节不展开证明。结论是：节点分布与次数同等重要，二者不能互相替代。反过来，这也不构成放弃插值的理由，该换的是节点。

扩展 · 勒让德与正交多项式

切比雪夫多项式不是孤例。勒让德多项式 P_k 由 $(1-x^2)P_k''-2xP_k'+k(k+1)P_k=0$ 定义，在 $[-1,1]$ 上关于权 1 正交；T_k 关于权 $1/\sqrt{1-x^2}$ 正交。正交性使"求最佳平方逼近的系数"化为逐个独立的内积，不必解线性方程组，这是 2.2 节换基思想的连续版本。定义与正交关系见附录 B。

条件核查表：

结论	用到的假设	假设失效时的后果
插值多项式存在且唯一（定理 2.1.1）	节点互异；次数上限为 n	节点重合则分母为零，问题无定义；去掉次数上限则解不唯一
余项公式（定理 2.1.3）	$f\in C^{n+1}[a,b]$；节点在 $[a,b]$ 内；x 不是节点	f 只有 n 阶光滑时公式不成立；x 为节点时误差为零，需单独处理
一致范数上界（2.1.5）	同上	上界是充分条件，不是等式；上界大不证明实际误差大

结论	用到的假设	假设失效时的后果
Chebyshev 界（2.1.8）	区间 $[-1,1]$；节点取 T_{n+1} 的零点	一般区间需乘 $((b-a)/2)^{n+1}$；改用极值点则常数改变

结论与限制： 式（2.1.5）右端随 n 趋于零时，收敛性得证；不趋于零时，该估计本身给不出结论。龙格函数的渐近发散需要复分析，本节不证。文献定位：[1] §2.2.2 – §2.2.3（印刷页 25 – 27），[2] 第 13 章（印刷页 95 – 97）。

实验设计：先预测，再运行

实验任务：对龙格函数，在相同节点数下比较等距节点与 Chebyshev 零点，并在与节点不重合的独立稠密网格上评估误差。

要填写的项目	本组内容
要回答的问题与运行前预测	问：同次数下换节点族，能否改变区间误差的量级？预测：两种节点在节点处误差均为零；等距节点误差随 n 增大而增大且集中在端点；Chebyshev 零点误差单调下降。若实测与预测不符，先检查求值点是否意外落在节点上
输入、参数、单位与数据来源	$f=1/(1+25x^2)$，$x\in[-1,1]$；$n\in\{8,16,32\}$；评价网格 $z_k=-1+2(k+0.371)/6001$（$k=0,\dots,6000$），偏移 0.371 使网格点不落在任何节点上；数据由解析式生成，全部无量纲，无训练/测试划分
独立参考解与基线	参考解直接取解析 $f(z_k)$，不经过插值程序；另设基线 $2x^2-x+3\in\mathcal{P}_2$，在 3 个非节点处应被精确还原（再现性检查）；重复节点、点越界、$n<0$ 三类非法输入应被显式拒绝
误差指标与容差	指标为网格最大绝对误差，是连续一致范数的离散代理。节点误差断言 $\le 10^{-12}$：该处应精确返回数据值，容差按 $\lvert y\rvert\le 1$ 取机器精度量级。网格误差只记录不断言——插值误差不受程序控制，事后断言等于挑选结论
环境与运行预算	Python 3.12 + NumPy，CPU，float64；本节无随机数，种子不适用；单节不到 1 秒，完整实验脚本约 9 秒（本机实测 8.6 秒，其中网络训练约 2.7 秒）

本节代码在同号 Notebook 2.1 小节，重跑请用 `tools/execute_notebook.py` 空内核执行。

结果解释与本节结论

实测数据如下（同一函数、同一公式，仅换节点族）：

n	等距节点网格最大误差	Chebyshev 零点网格最大误差
8	1.0452	1.7084×10^{-1}
16	14.394	3.2614×10^{-2}
32	5059.0	1.4017×10^{-3}

两列的变化方向相反。等距列从 8 到 16 放大约 13.8 倍，从 16 到 32 放大约 351 倍；Chebyshev 列每步缩小约 5.2 倍与 23 倍。到 $n = 32$，两列相差约 3.6×10^6 倍。函数与公式都没有变，改变的只是节点的位置。

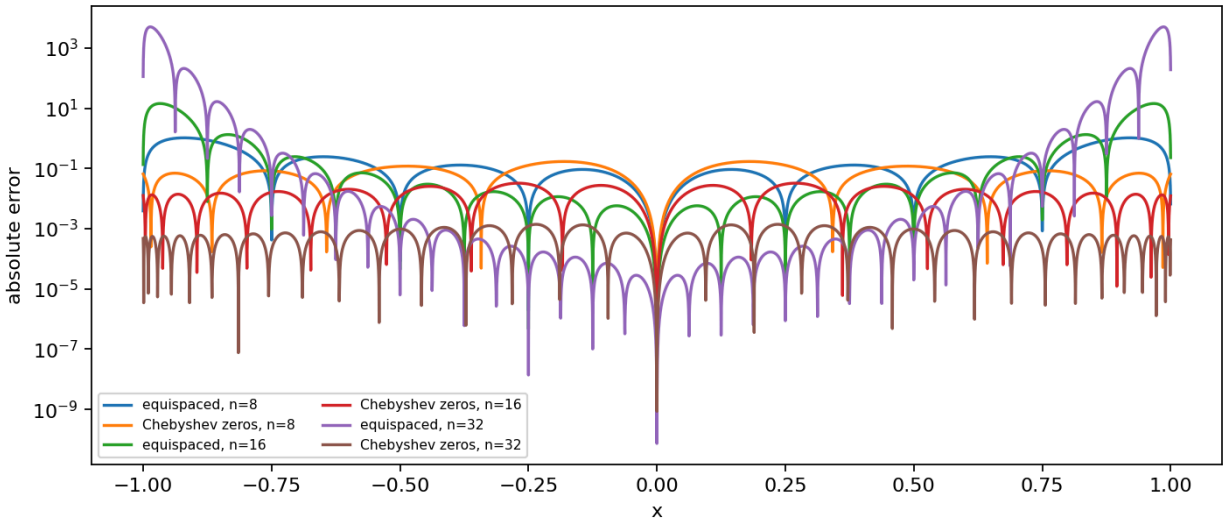


图02-4 • Runge 函数插值的绝对误差随次数的变化；纵轴为对数，横轴为插值次数，两条曲线分别是等距节点与 Chebyshev 零点。参数、评价网格与解析真值见 Notebook 2.1。图中是有限网格上的观测值，不是渐近证明。

`node_max_error` 三档全为 0.0。这只说明插值条件被满足，任何正确实现都必然如此，不能作为精度证据。

2^{-n} 不能用来预测 Chebyshev 那一列。它只控制几何因子，误差上界还含 $\|f^{(n+1)}\|_\infty/(n+1)!$，而龙格函数的高阶导数增长很快，抵消了相当一部分 2^{-n}。因此实测的下降速度（每步 5 至 23 倍）并不等于 2^n 的节奏。

本节结论： 插值误差由函数因子与几何因子相乘决定。函数不能改，节点可以选择。节点选得好，几何因子按 2^{-n} 缩小；选得不好，它可能指数放大。报告插值精度时，必须同时给出节点族、次数与评价网格。

边界： 本例只测了一个函数、三个次数。结论不能推广为"等距节点一律差"。对低次数或解析性很强的函数，等距插值足够使用。判断该用哪种节点，要看 f 的高阶导数行为。

易错点与讨论

一、把节点误差为零当作精度证据。 它只是插值条件的直接后果。有信息量的是与节点不重合的网格误差，这也是评价网格偏移 0.371 的原因。

二、把网格最大值当作连续范数。 6001 点上的最大值是 $\|f - p\|_\infty$ 的一个近似。网格加密时这个数通常变大。跨实验比较误差，必须同时交代网格密度与取点方式。

三、用 2^{-n} 直接外推收敛速度。 它只约束几何因子。对龙格函数，函数因子在高阶增长，实测下降慢于 2^{-n}，把上界的一部分当作整体速率会高估精度。

四、混淆次数提高与节点加密。 等距节点下增加 n 使误差从 1.04 涨到 5059，正确的结论是这族节点在高次数下失效，而不是多项式插值不可用。换节点族后同一套公式立即恢复收敛。

讨论题： 若把评价网格从 6001 点加密到 60001 点，$n = 32$ 那一格的数值会变大还是变小？这个变化能否用来估计真实的 $\|f - p\|_\infty$？

思考：

1. 若把 n 从 32 增到 64，等距节点的网格误差预计是多少量级？这个外推可信吗？还需要什么信息？
2. Chebyshev 零点不含端点，为什么反而比含端点的等距节点更准？端点附近的误差是如何被摊平的？
3. 若函数改为 $f(x)=e^x$，等距插值还会出现龙格现象吗？请从 $\|f^{(n+1)}\|_\infty$ 出发论证。

2.2 逼近：当数据不再听话（2学时）

本节目标：写出最小二乘目标与正规方程，说明唯一性的来源；解释换基为什么不改变解、只改变数值敏感性；区分训练残差、网格误差与条件数三个指标。

概念讲解：把问题说明白

2.1 节要求曲线穿过每一个点。实际数据往往做不到：仪器有读数误差，样本数远多于待定系数，强行穿过等于把噪声当作信号。

看一个最小的例子。真实函数为零，在 $-1,0,1$ 三处测得 $0,\varepsilon,0$，其中 $\varepsilon\neq 0$ 是测量误差。二次插值必须穿过这三点，结果是 $\varepsilon(1-x^2)$：一条在中间鼓起、两端贴地的抛物线。误差被原样保留，还配上了一条款式完整的曲线。

换一个判据：不要求残差为零，只要求残差平方和最小。用常数 c 拟合这三点，

$$J(c)=c^2+(c-\varepsilon)^2+c^2=3\Big(c-\frac{\varepsilon}{3}\Big)^2+\frac{2\varepsilon^2}{3},$$

最优解为 $c=\varepsilon/3$。它不穿过任何一个点，却把单个扰动的影响压到三分之一。这就是最小二乘的基本思想：用整体偏差最小代替逐点通过。代价是必须接受"没有一条曲线真的经过全部数据"。

以下三个量回答三个不同的问题，不能互相替代：

指标	记号	回答的问题
训练残差	$\|Ac-y\|_2$	曲线在已知样本上贴得多紧
独立网格误差	$\max_z\|p(z)-f(z)\|$	曲线在未知位置上准不准
条件数	$\kappa_2(A)$	换一台机器算，答案会不会变

本节按三步展开：2.2.1 建立目标函数并说明为什么用平方；2.2.2 导出最优条件，给出正规方程与唯一性；2.2.3 固定数据与次数、只换基函数，用条件数与网格误差量化坐标选择的影响。

本节只讨论线性最小二乘，即模型对系数是线性的。多项式与 Chebyshev 多项式的线性组合都属于此类。网络参数是非线性的，不适用本节结论，2.5 节另行处理。一致最优逼近的完整理论不在此展开，只在扩展框中说明它与平方最优的区别。

关键推导：让结论有依据

2.2.1 目标函数

设样本为 (x_i,y_i)，$i=1,\dots,m$，且

$$y_i=f(x_i)+\eta_i,$$

其中 f 未知，η_i 是观测误差。本节不对 η_i 作随机分布假设。选定次数 d（通常 $d+1<m$）与一组基 $\phi_0,\dots,\phi_d$，令

$$p_c(x)=\sum_{j=0}^{d} c_j\phi_j(x).$$

定义 2.2.1（离散最小二乘） 记设计矩阵 $A_{ij}=\phi_j(x_i)$（$i=1,\dots,m$；$j=0,\dots,d$），残差 $r(c)=Ac-y$。离散最小二乘问题是求

式（2.2.1）

$$\min_{c\in\mathbb{R}^{d+1}} J(c),$$

$$J(c)=\|Ac-y\|_2^2=\sum_{i=1}^{m}\bigl(p_c(x_i)-y_i\bigr)^2.$$

采用平方而非绝对值或最大值，有三条理由。

1. 不抵消：正负残差都变为正数，不会互相抵消。
2. 大偏差受重罚：平方使偏离大的点付出更高代价，符合"宁可处处小错，不可一处大错"的要求。
3. 可微：平方和对系数是二次函数，求导得到线性方程组，解有闭式结构。

第三条使优化问题化为解线性方程组，这是最小二乘得以长期使用的技术原因。代价是平方对离群点敏感，一个离群点就能把整条曲线拽偏。若数据中确有离群点，应改用绝对值或稳健损失，那属于另一类问题。

还需注意：增加次数不会使最小训练残差变大，因为旧的候选多项式仍在新空间中。但这只说明训练集内更贴合，不保证样本之外更准确。噪声会随次数一起被拟合进去，2.2.3 节将用数据展示这一现象。

2.2.2 正规方程

先从几何角度看。A 的列空间

$$S=\{Ac:c\in\mathbb{R}^{d+1}\}\subseteq\mathbb{R}^m$$

是所有可实现拟合值组成的子空间。最小二乘就是在这个子空间中找离 y 最近的向量。

定理 2.2.1（存在性、最优条件与唯一性） 对任意 $A\in\mathbb{R}^{m\times(d+1)}$ 与 $y\in\mathbb{R}^m$，问题 (2.2.1) 至少有一个解。c_* 为解的充要条件是

式（2.2.2）

$$A^{\mathsf T}(Ac_*-y)=0,\quad \text{即}\quad A^{\mathsf T}Ac_*=A^{\mathsf T}y.$$

所有解给出相同的拟合值 Ac_*；系数解唯一当且仅当 A 的列线性无关。

证明 取 S 的一组标准正交基 $q_1,\dots,q_r$（对 A 的列作 Gram–Schmidt 即得），令 $z_*=\sum_{k=1}^{r}(q_k^{\mathsf T}y)q_k$。直接计算可知 $y-z_*$ 与每个 q_k 正交，从而与整个 S 正交。对任意 $z\in S$，交叉项为零，故

式（2.2.3）

$$\|y-z\|_2^2=\|y-z_*\|_2^2+\|z_*-z\|_2^2\ge\|y-z_*\|_2^2,$$

即 z_* 是 S 中唯一的最近向量；因 $z_*\in S$，存在 c_* 使 $Ac_*=z_*$。

再看最优条件。对任意 h 与实数 t，

$$J(c_* + th) - J(c_*) = 2t\,h^{\mathsf T}A^{\mathsf T}(Ac_* - y) + t^2\|Ah\|_2^2.$$

若一次项系数不为零，取足够小、符号相反的 t 会使右端为负，与最优性矛盾，故 (2.2.2) 成立。反之若 (2.2.2) 成立，取 $t=1$ 得 $J(c_*+h)-J(c_*)=\|Ah\|_2^2\ge 0$。

最后，若 c_1, c_2 都最优，则 $A(c_1-c_2)=0$。列满秩时零空间只有零向量，故 $c_1=c_2$；列不满秩时存在非零 h 使 $Ah=0$，c_* 与 c_*+h 都是解。□

式 (2.2.2) 称为正规方程（法方程）。其几何含义是：最优残差与设计矩阵的每一列都正交。最小二乘不要求 A 是方阵，只要列空间是有限维子空间，投影就存在。

例 2.2.1 在 $-1, 0, 1$ 处观测 $1, 0, 2$，用 $p = c_0 + c_1 x$ 拟合：

$$A = \begin{pmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad A^{\mathsf T}A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}, \quad A^{\mathsf T}y = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

解得 $p_*(x) = 1 + x/2$，残差 $(-\tfrac12, 1, -\tfrac12)^{\mathsf T}$。残差之和为零（与常数列正交），加权和 $\sum_i x_i r_i = 0$（与一次项列正交）。平方残差和为 $\tfrac32$，均方根残差为 $1/\sqrt{2}$，报告误差时须说明用的是哪一个。本例没有真实函数，这个残差不是对真实目标的逼近误差。

命题 2.2.2（多项式拟合的唯一性） 若样本位置中至少有 $d+1$ 个互异点，且 $\phi_0, \dots, \phi_d$ 是 $\mathcal{P}_d$ 的基，则设计矩阵列满秩，最小二乘多项式唯一。

证明 若 $Ac = 0$，则 p_c 在至少 $d+1$ 个互异点取零。次数不超过 d 的非零多项式至多有 d 个互异实根，故 $p_c \equiv 0$；再由基的线性无关性得 $c = 0$。□

这与定理 2.1.1 的唯一性证明用的是同一论证：零点个数不够用。插值与最小二乘形式不同，唯一性的依据相同。

秩亏时要区分"拟合值唯一"与"函数唯一"。例如用一次多项式拟合两个都位于 $x=0$ 的观测 $y_1=1, y_2=3$，则 $J(c) = 2(c_0-2)^2 + 2$，所有 $p(x) = 2 + c_1 x$ 都最优：在观测点给出相同的值 2，斜率无法确定。

2.2.3 换基

幂基 $1, x, \dots, x^d$ 直观，但次数较高时各列接近，系数对扰动敏感。先作区间归一化

$$t = \frac{2x - a - b}{b - a} \in [-1, 1],$$

再改用 Chebyshev 基 $T_0, \dots, T_d$。所有基函数在 $[-1,1]$ 上都被 1 控制，列与列的区别更明显。

定理 2.2.3（换基不改变精确解） 设 $\phi_0, \dots, \phi_d$ 与 $\psi_0, \dots, \psi_d$ 是同一多项式空间的两组基。对同一组数据与同一未加权平方残差目标，两组基给出的最优多项式集合相同；若设计矩阵列满秩，则最优多项式相同且唯一。

证明 存在可逆矩阵 B 使 $A_\psi = A_\phi B$。每个系数 γ 对应 $c = B\gamma$，且表示同一个函数；B 可逆保证这一对应遍历整个候选空间，而 $\|A_\psi\gamma - y\|_2 = \|A_\phi c - y\|_2$，故两个最小化问题等价。唯一性由定理 2.2.1 得到。□

换基没有改变候选函数空间与目标，只改变了坐标与算法。因此若两种基算出的曲线明显不同，原因必在舍入、数值秩判定或目标不一致，而不在"Chebyshev 基更聪明"。

描述求解过程稳定性的量是条件数（定义见附录 A）：

式（2.2.4）

$$\kappa_2(A) = \frac{\max_{\|v\|_2=1} \|Av\|_2}{\min_{\|v\|_2=1} \|Av\|_2}.$$

分母越小，说明某些系数组合在样本上几乎不可见，反推系数越容易被扰动带偏。又因 $\|Av\|_2^2 = v^{\mathsf{T}} A^{\mathsf{T}} A v$，所以

式（2.2.5）

$$\kappa_2(A^{\mathsf{T}}A) = \kappa_2(A)^2.$$

这就是"推导用正规方程、计算不用正规方程"的原因。显式构造 $A^{\mathsf{T}}A$ 会使条件数平方：$\kappa_2 = 10^4$ 时变为 10^8，float64 下损失 8 位有效数字。实现上应使用薄 QR 或专用最小二乘求解器，它们数学上等价，数值上不平方条件数。

数值观察。 取 41 个等距样本，目标 e^x，叠加标准差 0.015、种子 2026 的噪声，固定 $d = 12$：

基	$\kappa_2(A)$	训练残差 $\|Ac - y\|_2$	独立网格最大误差
幂基	1.6675×10^4	6.9326×10^{-2}	1.8921×10^{-2}
Chebyshev 基	2.4589	6.9326×10^{-2}	1.8921×10^{-2}

两种基给出的预测曲线最大差约 1.70×10^{-14}。条件数降了约 6782 倍，解却在十余位小数内一致。这与定理 2.2.3 相符。

扩展 • 三种"最优"不是一回事

同一个函数、同一个候选空间，换一种误差尺度，答案就变：

- 离散最小二乘：$\sum_i (p(x_i) - y_i)^2$ 最小；
- 连续最小二乘：$\int_a^b (p - f)^2\,dx$ 最小；
- 一致最优（最小最大）：$\max_{[a,b]} |p - f|$ 最小。

在 $[-1, 1]$ 上用常数逼近 x^2，三者分别给出 $\frac{2}{3}$（三点离散）、$\frac{1}{3}$（连续平方）、$\frac{1}{2}$（一致最优）。切比雪夫交错定理刻画了第三种：最优多项式的误差在 $d+2$ 个点上正负交替、幅度相等。它与最小二乘的解一般不同——用了 Chebyshev 基，不等于做了一致逼近。详见 [2] 第 10 章（印刷页 73 – 75）。

条件核查表：

结论	用到的假设	假设失效时的后果
正规方程给出最优解（定理 2.2.1）	J 可微（恒成立）	无
解唯一且为全局最小	A 列满秩	秩亏时解不唯一，需最小范数解或正则化
两种基给出同一多项式（定理 2.2.3）	精确算术；两基张成同一 $\mathcal{P}_d$	浮点下只在条件数允许的精度内相等（实测 1.70×10^{-14}）
条件数可作敏感性指标	列满秩；比较时区间、次数、列缩放一致	缩放某列会改变条件数而不改变函数空间

结论与限制： 本节只保证在给定样本、给定次数下平方残差最小，不保证该多项式接近真实函数。后者取决于样本代表性、噪声水平与次数选择，属于统计建模问题，线性代数无法回答。文献定位：[1] §3.4.1 式 (4.1)–(4.7)（印刷页 73–74），[3] §3.2–§3.3（印刷页 103、109–110）。

实验设计：先预测，再运行

实验任务：固定样本与次数，只更换基函数，比较条件数、残差与独立网格误差；再用同一样本比较三个次数，观察残差与网格误差是否同向。

要填写的项目	本组内容
要回答的问题与运行前预测	问一：同数据同次数换基，解与条件数各变多少？预测解在舍入范围内相同（差 $\lesssim 10^{-13}$），条件数下降数个量级。问二：次数增大是否总是更好？预测训练残差单调下降，网格误差先降后升
输入、参数、单位与数据来源	$m = 41$ 个等距样本，$x \in [-1, 1]$，目标为已知解析曲线，叠加标准差 0.015 的伪随机噪声，种子固定；$d \in \{3, 12, 30\}$，主实验取 $d = 12$；另设缩放区间 $[100, 102]$ 作对照；全部无量纲
独立参考解与基线	参考解为无噪目标在独立网格上的值，与拟合程序完全独立；基线为 `numpy.linalg.lstsq` 的薄 QR 解，用于核对自编途径（本组只用正规方程作代数核对，不用它求数值解）；秩亏输入应被拒绝
误差指标与容差	三个指标分开报告：训练残差、独立网格最大误差、$\kappa_2(A)$。解的一致性容差取 10^{-12}，依据是 $\kappa_2 \approx 1.7 \times 10^4$ 时 float64 的相对精度约 $10^{-16}\kappa_2 \approx 10^{-12}$；网格误差不作断言
环境与运行预算	Python 3.12 + NumPy，CPU，float64；随机种子固定，噪声可复现；本节约 1 秒

本节代码在同号 Notebook 2.2 小节。记录中 `normal_equations_solved = False` 是设计选择，不是遗漏。

结果解释与本节结论

主实验（$m = 41$，$d = 12$，噪声标准差 0.015，种子固定）：

基函数	$\kappa_2(A)$	训练残差	独立网格最大误差	与 `lstsq` 解的最大差
幂基 $1, x, \ldots, x^{12}$	1.6675×10^4	6.9326×10^{-2}	1.8921×10^{-2}	1.10×10^{-14}
Chebyshev 基	2.4589	6.9326×10^{-2}	1.8921×10^{-2}	3.11×10^{-15}

残差与网格误差在小数点后十余位内一致，条件数却降了约 6782 倍。解读如下。

- 两种基张成同一个 $\mathcal{P}_{12}$，精确算术下解必然相同，所以"解相同"是可预期的。
- 换基改变的是求解过程的舍入敏感性，不是模型本身。
- 本例中两种途径都停在 10^{-14} 与 10^{-15}，说明 float64 下二者都够用。条件数的巨大差异没有转化为可观测的精度差异。

这个结论不戏剧，但数据就是如此。

条件数的差异何时才会兑现？若显式构造并求解正规方程，矩阵变为 $A^{\mathsf{T}}A$，条件数为 $\kappa_2(A)^2 \approx 2.8 \times 10^8$，此时相对误差约 $10^{-16} \times 2.8 \times 10^8 \approx 3 \times 10^{-8}$ 会进入解，网格误差可能被污染。这是本组不显式解正规方程的理由。

次数对比（同一样本）：

次数 d	训练残差	独立网格最大误差
3	7.7736×10^{-2}	1.1541×10^{-2}
12	6.9326×10^{-2}	1.8921×10^{-2}
30	4.5376×10^{-2}	28.217

训练残差随次数单调下降，$d = 30$ 时最小；网格误差在 $d = 30$ 时增至 28.2，比 $d = 3$ 差约 2400 倍。两者方向相反。残差是训练集内的量，它只会因模型更灵活而下降，无法察觉过拟合。样本只有 41 个，$d = 30$ 有 31 个基函数，自由度几乎用尽，解在样本之间剧烈振荡。

本节结论： 最小二乘的解由子空间决定，换基不改变解，只改变求解的数值稳定性；评价模型必须使用与训练样本独立的网格，训练残差不能替代；次数应由独立误差决定，而不是由残差最小决定。

边界： 本例的噪声水平（0.015）与样本量（41）共同决定了最优次数落在 3 至 12 之间。换一组数据，最优点会移动。因此不能把"$d = 12$ 最好"当作结论，只能说在本例的样本量与噪声下，$d = 30$ 已经过拟合。

易错点与讨论

一、把 Chebyshev 基的最小二乘解当成最佳一致逼近。 两者目标不同：前者最小化离散 ℓ_2 残差，后者最小化 $\|p - f\|_\infty$，解一般不同。本节实验用的是前者，只是换了基。

二、只看条件数就断言精度更好。 κ_2 从 1.6675×10^4 降到 2.4589，降了 6782 倍，但网格误差在小数点后十余位内没有变化。条件数刻画的是最坏情况下的误差放大上界，不是实际误差。

三、把 $d = 30$ 的最小残差当成最优模型。 残差 4.54×10^{-2} 确实最小，但网格误差 28.2 是 $d = 3$ 的 2400 倍。用训练残差选模型，会系统性地偏向过拟合的一端。

四、忽略同子空间这一前提。 幂基与 Chebyshev 基张成同一个 $\mathcal{P}_{12}$，解相同是必然的。若两次实验同时改了次数、样本或正则项，就不能把差异归因于换基。对照实验一次只改一个因素。

讨论题： 若把样本区间从 $[-1, 1]$ 平移到 $[100, 102]$ 而不作变量替换，幂基的条件数会变成多少量级？此时 Chebyshev 基还有优势吗？实验里设了 `scaling_interval` 这一组对照，请先预测再看结果。

思考：

1. 换基后解相同、条件数降了 6782 倍，为什么网格误差没变？什么条件下这个差异才会真正显现？
2. 若样本数从 41 增到 410 而次数固定为 12，条件数与网格误差各会怎么变？哪个更可能先改善？
3. $d = 30$ 的训练残差最小、网格误差最大。若只能用训练集内的信息选模型，有没有办法察觉这种过拟合？

2.3 插值积分：把面积交给多项式（2学时）

本节目标：说明求积公式就是对插值多项式积分；写出梯形误差并解释它为什么是二阶；用 $\int\omega_{n+1}$ 这一几何量统一解释误差的大小与符号。

概念讲解：把问题说明白

计算 $\int_a^b f$ 时，计算机只能取有限个函数值。最自然的做法是先用一个容易积分的函数 p 代替 f，再对 p 精确积分。多项式满足这一要求：它的原函数仍是多项式。

于是 2.1 节的方法可以直接使用。设 p_n 是 f 在节点 $x_0,\dots,x_n$ 上的插值多项式，则

式（2.3.1）

$$\int_a^b f(x)\,dx \approx \int_a^b p_n(x)\,dx = \sum_{k=0}^{n} f(x_k)\underbrace{\int_a^b \ell_k(x)\,dx}_{=:A_k}.$$

右端是求积公式：把积分近似为函数值的加权和，权重 A_k 只与节点有关，与 f 无关。凡能写成这种形式的公式，称为插值型求积公式。数值积分由此纳入插值的框架。

本节只讨论一个例子：$n=1$、节点取区间两端。此时 p_1 是连接两端点的直线，积分为梯形面积；把区间等分为 N 段、每段照此处理，得到复合梯形公式。选它作主要对象，原因有三：几何意义清楚；误差可写成闭式，能对分数核验；误差阶为二阶，可以看清"加密一倍、误差四分之一"的来源。

全节围绕三个问题：误差是几阶？误差为正还是为负？结论依赖什么光滑性？

关键推导：让结论有依据

2.3.1 复合梯形公式

把 $[a,b]$ 等分为 N 段，$h=(b-a)/N$，$x_k=a+kh$。在 $[x_k,x_{k+1}]$ 上，两点插值多项式为连接端点的直线

$$\ell_k(x)=\frac{x_{k+1}-x}{h}f(x_k)+\frac{x-x_k}{h}f(x_{k+1}).$$

积分得该段梯形面积 $h[f(x_k)+f(x_{k+1})]/2$。各段相加，内部节点被左右两段各算一次，合并后得

式（2.3.2）

$$T_N=h\left[\frac{f(a)+f(b)}{2}+\sum_{k=1}^{N-1}f(x_k)\right].$$

这就是复合梯形公式。T_N 在此表示求积值，与 2.1 节的 Chebyshev 多项式 $T_k(x)$ 不是同一记号。

定理 2.3.1（梯形公式是分段线性插值的精确积分） 设 p_h 是 f 在节点 $a,x_1,\dots,x_{N-1},b$ 上的分段线性插值函数（每段一次，整体连续），则

式（2.3.3）

$$T_N=\int_a^b p_h(x)\,dx,$$

且当 $f\in C[a,b]$ 时，$T_N\to\int_a^b f$。

证明 第一式由逐段计算直接得到。第二式：在每段上 p_h 是端点值的凸组合，故 $|f(x)-p_h(x)|\le\omega_f(h)$，其中 ω_f 是 f 的连续模。积分得

$$\left|\int_a^b f - T_N\right| \le (b-a)\,\omega_f(h).$$

闭区间上的连续函数一致连续，故 $\omega_f(h)\to 0$。$\square$

定理说明：梯形公式的收敛性只需要连续性，不需要光滑性。光滑性影响收敛速度，不影响是否收敛。

2.3.2 误差

下面把 2.1 节的余项（2.1.4）在每段上积分。

定理 2.3.2（梯形误差） 设 $f\in C^2[a,b]$，则存在 $\xi\in(a,b)$，使

式（2.3.4）

$$I - T_N = -\frac{(b-a)h^2}{12} f''(\xi),$$

从而

式（2.3.5）

$$|I - T_N| \le \frac{(b-a)h^2}{12}\|f''\|_{\infty,[a,b]}.$$

证明 在 $[u,v]=[x_k,x_{k+1}]$ 上，两点插值的余项为

$$f(t) - \ell_k(t) = \frac{f''(\eta_t)}{2}(t-u)(t-v).$$

$(t-u)(t-v)\le 0$ 在整段上不变号，且

$$\int_u^v (t-u)(t-v)\,dt = -\frac{h^3}{6}.$$

对 f'' 用加权中值定理（权 $(t-u)(t-v)$ 非正、f'' 连续），存在 $\eta_k\in(u,v)$ 使

$$\int_u^v (f-\ell_k)\,dt = \frac{f''(\eta_k)}{2}\left(-\frac{h^3}{6}\right) = -\frac{h^3}{12} f''(\eta_k).$$

对 $k=0,\dots,N-1$ 求和，得 $I - T_N = -\dfrac{h^2}{12}\displaystyle\sum_{k=0}^{N-1} h\,f''(\eta_k)$。右端的和是 f'' 在 $[a,b]$ 上的加权平均（权重都是 h），由 f'' 的连续性，它等于某个 $f''(\xi)$，$\xi\in(a,b)$。代回即得（2.3.4），取绝对值即得（2.3.5）。$\square$

证明结构与 2.1 节相同：

2.1 插值	2.3 积分
误差 $=\dfrac{f''(\xi)}{2}\omega(x)$	误差 $=\displaystyle\int \frac{f''(\eta_t)}{2}\omega(t)\,dt$
几何因子 $\|\omega\|_\infty$	几何因子 $\left\|\int\omega\right\| = \dfrac{h^3}{6}$
取最大，得误差上界	取积分，得误差公式

同一个余项，取范数得插值误差，积分得求积误差。几何因子变为 $\int\omega$ 之后，误差的大小与符号同时由它决定。

- $\int_u^v \omega = -h^{\frac{3}{6}} < 0$，所以 $f''>0$（凸函数）时 $I - T_N < 0$，梯形公式高估积分。几何上，凸函数的弦在曲线上方，梯形面积大于曲边面积。

- 若 f 为直线（$f''=0$），误差恒为零。梯形公式对一次多项式精确，即代数精度至少为 1。

由 (2.3.5) 得收敛阶：$h\to 0$ 时 $|I-T_N|=O(h^2)$。区间数加倍，h 减半，误差上界变为四分之一。

例 2.3.1 对 $f(x)=x^2$、$[0,1]$，有 $f''=2$，于是

式（2.3.6）

$$I=\frac{1}{3},$$

$$T_N=\frac{1}{3}+\frac{1}{6N^2}.$$

特别地 $T_4=\frac{11}{32}$、$T_8=\frac{43}{128}$，绝对误差分别为 $\frac{1}{96}$ 与 $\frac{1}{384}$，恰为四分之一。开篇问题的答案是四分之一，这是精确的代数事实。

（式 (2.3.6) 可用 $\sum_{k=1}^{N-1}k^2=(N-1)N(2N-1)/6$ 直接验证。）

光滑性不能省。 若 f 只有连续性，定理 2.3.2 失效，收敛阶可能低于二阶。以 $f(x)=\sqrt{x}$ 在 $[0,1]$ 上为例：它连续、凹，每段梯形都不超过积分值。仅第一段就产生

$$\int_0^h\sqrt{x}\,dx-\frac{h}{2}\sqrt{h}=\frac{h^{3/2}}{6},$$

故 $I-T_N\geq h^{\frac{3}{2}}/6$。若存在与 N 无关的常数 C 使 $|I-T_N|\leq Ch^2$，则要求 $C\geq 1/(6\sqrt{h})$，令 $h\to 0$ 得矛盾。因此实际阶为 $h^{\frac{3}{2}}$，不是 h^2。

扩展 • 从梯形到 Simpson 与 Gauss

梯形是"一次插值后积分"。若每段改用二次插值（端点与中点三个节点）再积分，得到 Simpson 公式

$$\int_u^v f\approx\frac{h}{6}\left[f(u)+4f(u+h)+f(v)\right],$$

其代数精度为 3（对三次多项式精确），误差阶为 $O(h^4)$。继续提高插值次数，得到 Newton–Cotes 公式；若节点位置也可自由选取，则得到 Gauss 求积，$n+1$ 个节点可达到 $2n+1$ 阶代数精度。

这条路线有一个共同瓶颈：高次等距插值会遇到龙格现象，所以 Newton–Cotes 一般只用低阶；Gauss 求积依靠正交多项式选节点，避开了这一问题。本章只要求理解"求积等于插值后积分"这一原理。选读边界见章末。

条件核查表：

结论	用到的假设	假设失效时的后果
$T_N=\int p_h$（定理 2.3.1）	分段线性插值；节点含端点	节点不含端点时公式形状改变
$T_N\to I$	f 在 $[a,b]$ 连续	有界但间断时需另作分析
误差公式 (2.3.4)	$f\in C^2[a,b]$；等距分划	f'' 无界或不存在时公式失效，阶可降到 $h^{\frac{3}{2}}$ 或更低
误差阶 $O(h^2)$	同上	只有 C^1 时不能断言二阶

结论与限制： 求积误差只依赖 h 与 f''，与浮点精度无关；其符号由 f'' 的符号决定。式 (2.3.5) 是上界，不是等式；一次函数的误差为零，说明上界取等需要额外条件。文献定位：[1] §4.3.1 式 (3.1)–(3.3)（印刷页 106–107）。

实验设计：先预测，再运行

实验任务：对 $f(x)=x^2$、$[0,1]$，用 $N=4,8,16,32,64$ 计算复合梯形值，报告绝对误差与相邻误差比。

要填写的项目	本组内容
要回答的问题与运行前预测	问：N 加倍时误差是否严格变为 $\frac{1}{4}$？预测是，比值稳定在 0.2500，因为闭式（2.3.6）给出的正是 $1/(6N^2)$
输入、参数、单位与数据来源	$f=x^2$，$[0,1]$，$N\in\{4,8,16,32,64\}$；真值 $\frac{1}{3}$ 由解析式计算；全部无量纲，无随机数
独立参考解与基线	参考解为解析积分值，不经过任何求积程序；基线为 T_N 的闭式 $\frac{1}{3}+1/(6N^2)$，用于核对实现；矩形公式作为一阶方法的对照
误差指标与容差	绝对误差 $\lvert I-T_N\rvert$ 与相邻比值。断言与闭式的差 $\le 10^{-14}$
环境与运行预算	Python 3.12 + NumPy，CPU，float64；无随机数；本节约 0.5 秒

附加手算（不运行程序）： 若把 f 换成 $\sqrt{x}$，比值还是 $\frac{1}{4}$ 吗？由 $I-T_N\ge h^{\frac{3}{2}}/6$，阶只能到 $h^{\frac{3}{2}}$，预测比值约 $2^{-1.5}\approx 0.354$。这一问考的是定理的条件是否满足。

本节代码在同号 Notebook 2.3 小节。

结果解释与本节结论

积分结果（$f=x^2$，$[0,1]$，真值 $\frac{1}{3}$）：

N	T_N	绝对误差	相邻误差比
4	0.34375	1.0417×10^{-2}	—
8	0.3359375	2.6042×10^{-3}	0.2500
16	0.333984375	6.5104×10^{-4}	0.2500
32	0.33349609375	1.6276×10^{-4}	0.2500
64	0.3333740234375	4.0690×10^{-5}	0.2500

比值四步全部为 0.2500，逐位相同。这与式（2.3.6）吻合：$T_4=\frac{1}{3}+\frac{1}{96}=0.34375$。所以这里的 0.25 不是统计趋势，而是可由闭式验证的代数事实。

再看符号：x^2 是凸函数，$T_N>I$，$I-T_N<0$，与定理 2.3.2 的判断一致。若换成凹函数 $\sqrt{x}$，梯形低估积分，符号翻转，但"$\int\omega<0$ 决定符号"这一逻辑不变。

本节结论： 复合梯形公式是分段线性插值的精确积分，其误差等于余项的积分。误差由几何因子 $\int\omega=-h^{\frac{3}{6}}$ 与函数因子 f'' 相乘决定，因此是二阶的，符号由 f'' 决定，且对一次函数精确。光滑性不足时阶会下降（$\sqrt{x}$ 为 $h^{\frac{3}{2}}$），但连续性已足以保证收敛。

边界： "区间加倍、误差四分之一"只对 C^2 函数成立，且指上界的阶；具体常数依赖 f''。区间长度也进入公式：把 $[0,1]$ 换成 $[0,10]$ 而 N 不变，h 变为 10 倍，误差约变为 100 倍，但比值仍是 0.25，因为比值只与 h 的缩放有关。

易错点与讨论

一、把误差比 0.25 当成普适结论。 它来自 $f=x^2$ 的精确展开。若 f 只有 C^1 光滑性，实际阶低于二阶，比值不再是 0.25。收敛阶的验证必须绑定光滑性假设。

二、只看误差大小，不看误差符号。 梯形对凸函数高估、对凹函数低估，这不是实现错误，而是 $\int\omega<0$ 的必然结果。若程序对 x^2 给出低估，通常说明求和或端点系数写错。

三、把加密等同于提高精度。 加密降低的是离散化误差；浮点求和还会引入舍入误差，N 极大时后者可能占主导。二者来源不同，不能合并为一个"总误差"。

四、以为梯形公式太粗糙而不值得使用。 它的二阶收敛可以证明，且对周期光滑函数，实际精度常常很高（谱精度）。方法的好坏取决于问题结构，不取决于公式形式是否复杂。

讨论题： 若把区间改为 $[0,10]$、N 固定为 8，误差绝对值大约是原来的多少倍？误差比还是 0.25 吗？请先用尺度变换论证，再运行代码核对。

思考：

1. 积分误差比四步都等于 0.2500。这是数值巧合还是必然？请写出 T_N 对 x^2 的闭式来回答。
2. 若 f 是三次多项式，梯形公式的误差还是零吗？为什么？由此能推出它的代数精度是几？
3. 把 f 换成 $|x-0.5|$，相邻误差比还会稳定在 0.2500 吗？请分"尖点恰好落在节点上"与"尖点落在某段内部"两种情形回答。

2.4 插值微分：把斜率交给多项式（2学时）

本节目标：说明差分公式就是插值多项式的导数；写出差分的截断误差与舍入误差，解释最优步长的来源；区分差分、符号微分与自动微分三种途径的误差结构。

概念讲解：把问题说明白

2.3 节对插值多项式作了积分。本节对它求导。

设 p_1 是 f 在 x_0,x_1 上的线性插值，则 $p_1'(x)\equiv(f(x_1)-f(x_0))/h$ 是常数。在 x_0 处取值，得前向差分

$$D_h^+f(x_0)=\frac{f(x_0+h)-f(x_0)}{h}.$$

设 p_2 是 f 在 x_{-1},x_0,x_1（等距）上的二次插值，则 $p_2'(x_0)=(f(x_1)-f(x_{-1}))/(2h)$，即中心差分

$$D_h^0f(x_0)=\frac{f(x_0+h)-f(x_0-h)}{2h}.$$

差分公式就是插值多项式的导数。由此可以直接从 2.1 节的余项推出误差公式，也可以解释为什么求导比积分困难：积分把误差平均掉，求导把误差除以步长，因而放大。

本节回答三个问题：差分公式的误差是几阶？为什么 h 不是越小越好，最优步长约为多少？若不选 h，还有没有别的途径，自动微分与差分、符号微分差在何处？

第三个问题与现代计算直接相关。训练神经网络时，反向传播用的就是自动微分，其依据仍是链式法则。

关键推导：让结论有依据

2.4.1 截断误差

为避免与积分步长混淆，用 $\delta>0$ 表示差分步长。

定理 2.4.1（差分的截断误差） 设 $f\in C^2$ 于 x_0 的邻域，则存在 ξ 介于 x_0 与 $x_0+\delta$ 之间，使

式（2.4.1）

$$f'(x_0)-D_\delta^+ f(x_0)=-\frac{\delta}{2}f''(\xi).$$

若 $f\in C^3$，则存在 η 介于 $x_0-\delta$ 与 $x_0+\delta$ 之间，使

式（2.4.2）

$$f'(x_0)-D_\delta^0 f(x_0)=-\frac{\delta^2}{6}f'''(\eta).$$

证明 先证前向。由 Taylor 公式，$f(x_0+\delta)=f(x_0)+\delta f'(x_0)+\frac{\delta^2}{2}f''(\xi_1)$，移项除以 δ 即得 (2.4.1)。

再证中心。把 $f(x_0\pm\delta)$ 在 x_0 展开到三阶：

$$f(x_0\pm\delta)=f(x_0)\pm\delta f'(x_0)+\frac{\delta^2}{2}f''(x_0)\pm\frac{\delta^3}{6}f'''(\xi_\pm).$$

两式相减，偶次项抵消，除以 2δ 得

$$D_\delta^0 f(x_0)=f'(x_0)+\frac{\delta^2}{12}\big[f'''(\xi_+)+f'''(\xi_-)\big].$$

方括号内是 f''' 在两点的平均值。由导数的介值性（Darboux 定理），它等于某个 η 处的 $f'''(\eta)$。故误差为 $-\delta^2 f'''(\eta)/6$。□

公式	截断误差	阶
前向差分	$-\frac{\delta}{2}f''(\xi)$	$O(\delta)$
中心差分	$-\frac{\delta^2}{6}f'''(\eta)$	$O(\delta^2)$

中心差分的阶更高，代价是多算一个函数值，且要求 f 多一阶光滑。这与 2.3 节的梯形公式形成对照：积分用一次插值即达二阶，求导用一次插值只有一阶。原因仍是误差在积分中被平均、在求导中被除以步长放大。

2.4.2 舍入误差与最优步长

定理 2.4.1 只给出截断误差。实际计算还有另一项：函数值本身只有有限位有效数字。

设每次求值的绝对误差至多为 $\varepsilon|f|$，其中 $\varepsilon\approx 2.2\times10^{-16}$ 为 float64 的相对精度。前向差分的分子是两个相近数相减，误差最多 $2\varepsilon|f|$，除以 δ 后放大为 $2\varepsilon|f|/\delta$；中心差分的分子误差约 $2\varepsilon|f|$，除以 2δ 后为 $\varepsilon|f|/\delta$。

定理 2.4.2（总误差与最优步长） 在定理 2.4.1 的条件下，前向差分的总误差满足

式（2.4.3）

$$|E^{+}| \le \frac{M_2\delta}{2} + \frac{2\varepsilon|f|}{\delta},$$
$$M_2 = \max|f''|,$$

中心差分的总误差满足

式（2.4.4）

$$|E^{0}| \le \frac{M_3\delta^2}{6} + \frac{\varepsilon|f|}{\delta},$$
$$M_3 = \max|f'''|.$$

两式的极小点分别在 $\delta \sim \sqrt{\varepsilon}$ 与 $\delta \sim \varepsilon^{\frac{1}{3}}$ 量级，对应的最小误差分别为 $O(\sqrt{\varepsilon})$ 与 $O(\varepsilon^{\frac{2}{3}})$。

证明 把截断界与舍入界相加即得 (2.4.3)、(2.4.4)。对 (2.4.3) 关于 δ 求导并令其为零：$M_2/2 - 2\varepsilon|f|/\delta^2 = 0$，得 $\delta_* = \sqrt{4\varepsilon|f|/M_2}$，代回得最小误差 $2\sqrt{\varepsilon|f|M_2}$。对 (2.4.4) 同理：$M_3\delta/3 - \varepsilon|f|/\delta^2 = 0$，得 $\delta_* = (3\varepsilon|f|/M_3)^{\frac{1}{3}}$，代回得最小误差 $\sim \varepsilon^{\frac{2}{3}}$。□

两式说明：步长越小，误差不一定越小。截断项随 δ 减小而下降，舍入项随 δ 减小而上升，两者在一处相等，该点即最优步长。继续减小步长只会放大舍入误差。

以本节实验的函数为例：$f(x) = \sin(x^2) + e^x$，$x = 0.7$，$|f| \approx 3.25$，$M_2 \approx 2.86$。代入得前向最优步长约 $\sqrt{4 \times 2.2 \times 10^{-16} \times 3.25/2.86} \approx 3.2 \times 10^{-8}$，实测 5.62×10^{-9}；中心最优步长约 $(3 \times 2.2 \times 10^{-16} \times 3.25/M_3)^{\frac{1}{3}}$，实测 5.62×10^{-6}。量级相符。理论给出的是量级，不是逐位预言，因为 M_2, M_3 只是上界。

2.4.3 自动微分

差分需要选 δ，符号微分需要展开表达式。第三条途径是自动微分（AD）：把程序拆成基本运算，沿计算图传播导数。

以前向模式为例。把 x 写成 $x + 1\cdot\varepsilon$，其中 $\varepsilon^2 = 0$、$\varepsilon \neq 0$。这是一个代数记号，不是"很小的实数"。每个数存成一对 $(v, \dot v)$，运算规则为

$$(u + \dot u\varepsilon) + (v + \dot v\varepsilon) = u + v + (\dot u + \dot v)\varepsilon,$$
$$(u + \dot u\varepsilon)(v + \dot v\varepsilon) = uv + (\dot u v + u\dot v)\varepsilon.$$

对可微基本函数 ϕ，规定 $\phi(v + \dot v\varepsilon) = \phi(v) + \phi'(v)\dot v\varepsilon$。连续复合两次，$\varepsilon$ 的系数正好是 $\phi'(\psi(v))\psi'(v)\dot v$，即链式法则。这里没有忽略高阶项的近似，$\varepsilon^2 = 0$ 是定义。

以 $F(x) = \sin(x^2) + e^x$ 为例，程序写成

$$v_0 = x,\quad v_1 = v_0 v_0,\quad v_2 = \sin v_1,\quad v_3 = e^{v_0},\quad v_4 = v_2 + v_3.$$

前向模式从 $\dot v_0 = 1$ 出发，一次遍历得 $\dot v_4 = 2x\cos(x^2) + e^x$。反向模式先正向求值并保存中间量，再从 $\bar v_4 = 1$ 逆序累加：

式（2.4.5）

$$\bar v_j \mathrel{+}= \bar v_i \frac{\partial v_i}{\partial v_j}.$$

符号 $\mathrel{+}=$ 表示累加，不能写成覆盖。平方节点 $v_1 = v_0 v_0$ 有两条边指向 v_0，两条都要加。

命题 2.4.3（AD 的合法性） 考虑由有限个基本运算构成的无环计算图。若各基本运算在所用点附近连续可微，程序在该邻域内执行同一条运算路径，且使用精确算术与正确的局部导数，则：前向模式以 $\dot{x}$ 为种子得到输出的方向导数；标量输出的反向模式以 1 为种子得到全部输入的偏导数。

证明 把节点按依赖关系排序，设 $v_i = \phi_i(v_{j_1}, \ldots, v_{j_r})$，父节点都排在前面。若父节点的切向量正确，链式法则给出 $\dot{v}_i = \sum_k (\partial \phi_i / \partial v_{j_k}) \dot{v}_{j_k}$，由输入开始归纳即得前向结论。反向模式从输出逆序处理：某个节点的微小变化沿每条出边影响输出，多元链式法则要求把各路径贡献相加；逆序时所有后继已处理完，式 (2.4.5) 正好完成累加。归纳至输入即得偏导数。□

条件的边界如下。

- 浮点运算仍有舍入误差。AD 消除的是截断误差，不是全部误差。本例实测与解析值差 10^{-15} 量级，这是 float64 的舍入水平。
- 分支与不可微点须注意。程序"若 $x = 0$ 返回 0，否则返回 x"在实数意义下就是 $F(x) = x$，但在零点只跟踪常数分支会给出导数 0。AD 求的是所走路径的导数，不是抽象函数的导数。
- 代价在存储与遍历次数。多参数、单输出时反向模式一次遍历得到全部梯度；前向模式一次只给一个方向。

三种途径的对比：

方法	输入与核心操作	输出	主要误差
有限差分	在邻近点调用函数，作差商	导数的近似值	截断 + 舍入，依赖 δ
符号微分	对表达式应用求导规则	导数表达式	表达式膨胀；求值仍有浮点误差
自动微分	执行基本运算时传播局部导数	指定点的导数值	仅浮点舍入；需可微路径

扩展 • 复步长差分

若函数能接受复变量，取 δ 为纯虚数 ih，由 Taylor 展开

$$\operatorname{Im} f(x + ih) = h f'(x) + O(h^3),$$

于是 $f'(x) \approx \operatorname{Im} f(x + ih)/h$。没有相减，也就没有灾难性抵消，误差可到 $O(h^2)$ 且不受 ε/h 限制。代价是函数须支持复数运算。这说明差分的精度瓶颈不是数学，而是"两个相近数相减"这一操作。

条件核查表：

结论	用到的假设	假设失效时的后果
差分误差 (2.4.1)(2.4.2)	$f \in C^2$ 或 C^3；等距节点	光滑性不足时阶下降；非等距需重算系数
最优步长 (2.4.3)(2.4.4)	舍入误差模型 $\le \varepsilon\lvert f\rvert$；$M_2, M_3$ 有界	实际常数依赖实现；只保证量级
AD 给出正确导数（命题 2.4.3）	基本运算可微；执行路径固定；精确算术	分支跳变处返回单侧或约定导数；ReLU 折点无经典导数

结论与限制： 求导误差同时含截断与舍入，不能合成一个数报告；AD 没有截断误差，但有舍入误差与路径依赖。文献定位：[4] §2.1、§3.1 – §3.2（印刷页 4 – 13）。

实验设计：先预测，再运行

实验任务：对 $F(x)=\sin(x^2)+e^x$ 在 $x=0.7$ 处比较解析导数、前向与中心差分（61 点对数步长扫描）以及自编 AD（Dual、Graph、Jet2）。

要填写的项目	本组内容
要回答的问题与运行前预测	问一：差分误差随 δ 减小是否单调下降？预测不，先降后升，存在最优 δ。问二：三种求导途径是否一致？预测两种 AD 与解析差在 10^{-15} 量级，差分在 $\delta=0.1$ 时差 10^{-1} 量级
输入、参数、单位与数据来源	$F=\sin(x^2)+e^x$，$x=0.7$；δ 取 61 点对数网格，从 10^{-1} 到 10^{-16}；全部无量纲，数据由解析式生成，无划分
独立参考解与基线	解析式 $F'=2x\cos(x^2)+e^x$、$F''=2\cos(x^2)-4x^2\sin(x^2)+e^x$。选解析值而非高精度数值，是为了同时提供真值与闭式表达，避免同一族算法互相验证；AD 为自编教学代码，不是外部库
误差指标与容差	绝对误差 $\lvert d_{\text{方法}}-d_{\text{解析}}\rvert$。AD 容差 10^{-13}（约 60 倍机器 epsilon，为复合运算累积留余量）；差分按最优 δ 单独给出，不设统一阈值
环境与运行预算	Python 3.12 + NumPy，CPU，float64；无随机数；本节约 1 秒，含 61 点扫描

本节代码在同号 Notebook 2.4 小节。61 点扫描区间固定，报告最优点时必须同时给出该区间。

结果解释与本节结论

（$x=0.7$，解析值 $F'=3.2490187095246466$，$F''=2.85599168387525$）

方法	一阶导	绝对误差
解析式	3.2490187095246466	—
前向 Dual	3.2490187095246466	$<10^{-16}$
反向 Graph	3.249018709524647	1.1×10^{-15}
二阶 Jet2（对 F''）	2.85599168387525	$<10^{-15}$
前向差分，$\delta=10^{-1}$	3.3836	1.346×10^{-1}
中心差分，$\delta=10^{-1}$	3.2563	7.302×10^{-3}
中心差分，最优 δ	—	4.225×10^{-11}

三种 AD 途径与解析值的差都在 10^{-15} 量级，即 float64 的舍入水平。同一批运算用不同模式得到一致结果，说明实现之间互相独立地吻合。这比"与解析式对上"更有说服力，因为它排除了同一个错误被重复三次的可能。

差分侧的情况不同。

- $\delta = 10^{-1}$ 时，前向误差 1.35×10^{-1}，中心 7.3×10^{-3}。中心二阶、前向一阶，在 $\delta = 0.1$ 处正好相差一个数量级，与理论一致。
- 沿 61 点网格从 10^{-1} 扫到 10^{-16}，误差先降后升。中心差分最低点在 $\delta = 5.6234 \times 10^{-6}$，误差 4.225×10^{-11}；理论量级 $\varepsilon^{\frac{1}{3}} \approx 6.1 \times 10^{-6}$，相符。
- 前向差分最低点在 $\delta = 5.6234 \times 10^{-9}$，误差 6.4×10^{-9}；理论量级 $\sqrt{\varepsilon} \approx 1.5 \times 10^{-8}$，也相符。
- 到 $\delta = 10^{-16}$，$x + \delta$ 在 float64 下已等于 x，前向差分为 0，中心差分退化为噪声。这一端的具体数值随平台变化，本节只报告量级。

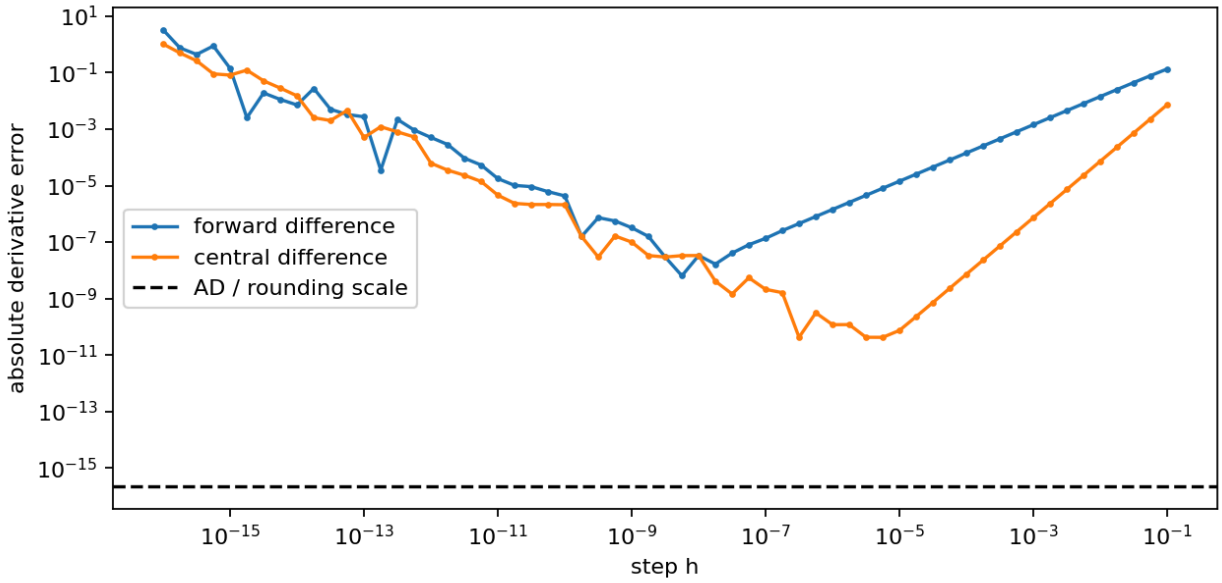


图02-5 • 对 $F(x) = \sin(x^2) + e^x$ 在 $x = 0.7$ 处的求导误差。两坐标轴均为对数；虚线为 AD 误差或浮点尺度。最优步长是本次有限扫描的结果。

本节结论： 差分公式是插值多项式的导数，截断误差可从插值余项读出：前向一阶、中心二阶。差分还有舍入误差，它随步长减小而增大。两项交汇给出最优步长，前向约 $\sqrt{\varepsilon}$，中心约 $\varepsilon^{\frac{1}{3}}$。自动微分不作除法、不选步长，沿计算图执行链式法则，误差只剩浮点舍入；代价是需要可微路径，且只对所写的程序负责。

边界： AD 的精确是相对差分而言的，不是零误差。若程序含分支、$|x|$、$\max$ 等不可微结构，AD 返回所走分支上的导数，在跳变点不唯一。2.5 节的 ReLU 反例将具体展示这一点。

易错点与讨论

一、把有限差分与自动微分混为一谈。 两者都给出导数值，但误差结构不同：差分有 $O(\delta)$ 或 $O(\delta^2)$ 的截断误差，且随 δ 变化；AD 没有截断误差，只有舍入累积。

二、报告差分结果却不给 δ。 同一函数在 $\delta = 10^{-1}$ 与 5.6×10^{-6} 下，中心差分误差相差七个数量级（7.3×10^{-3} 对 4.2×10^{-11}）。脱离 δ 的误差数字无法复现，也无法比较。

三、以为 δ 越小越准。 误差是截断项与舍入项之和。前向差分的最优步长约 $\sqrt{\varepsilon} \approx 10^{-8}$，中心约 $\varepsilon^{\frac{1}{3}} \approx 10^{-5}$，再小就进入舍入主导区。

四、把"AD 与解析值一致"当成"积分也对"。 求导与积分是两个独立问题：前者验证链式法则的实现，后者验证求积公式与离散化。一个成立不能推出另一个成立。

讨论题： 若把 F 换成一个函数值只能通过黑箱获得、无法修改程序的场景，三种方法还剩几种可用？若只能选一种，选哪个，为什么？

思考：

1. 中心差分在 $\delta = 0.1$ 时误差 7.3×10^{-3}，在 5.6×10^{-6} 时降到 4.2×10^{-11}，再减小又变差。请分别指出两个方向上起主导作用的误差项。
2. 若只能报告一个"导数精度"数字，你选最优 δ 下的差分还是 AD？各自的代价是什么？为什么 AD 的 10^{-15} 不能简单理解为比差分好 10^4 倍？
3. 前向差分的最优步长为什么比中心差分大约三个数量级？请从两条误差界中 δ 的幂次解释。

2.5 网络逼近与求导：当基函数也会学习（2学时）

本节目标：把网络看作基函数带参数的逼近；区分对输入求导与对参数求导；说明激活函数的光滑性如何决定高阶导数能否使用，以及为什么函数值准不代表导数准。

概念讲解：把问题说明白

前四节有一个共同前提：基函数事先选定。多项式是 $1, x, \ldots, x^n$，Chebyshev 多项式是 $T_0, \ldots, T_n$，数据只决定系数。神经网络改变了这一点：基函数的形状也由参数决定。

本节使用单隐层标量网络：

式（2.5.1）

$$u_\theta(x) = c + \sum_{j=1}^{q} a_j \tanh(w_j x + b_j),$$

其中 $\theta = (a_1, \ldots, a_q, w_1, \ldots, w_q, b_1, \ldots, b_q, c) \in \mathbb{R}^{3q+1}$ 是待定参数，按 a, w, b, c 分块排列，与 Notebook 里的参数打包顺序一致。

把 $\tanh(w_j x + b_j)$ 看作一排可移动、可伸缩的基函数：b_j 控制位置，w_j 控制陡峭程度，a_j 控制它对总和的贡献。$q = 12$ 时有 37 个参数，比 2.2 节的 13 个系数多，但仍是有限维的。网络没有跳出"用有限个参数表示函数"的框架，只是让这些参数用得更灵活。

以下三件事必须分开，否则讨论会混乱。

问题	数学性质	本节的讨论
表达能力：是否存在 θ 使 $u_\theta \approx f$	存在性（逼近论）	定理 2.5.1 给出一个构造
可训练性：优化能否找到这组 θ	优化	只作小规模演示
泛化：未参与训练的点上是否仍准	统计	不展开

三者没有逻辑蕴含关系。存在一个好网络，不等于梯度下降能找到它；训练损失小，不等于未见过的位置上仍准。2.2 节已用 $d = 30$ 的过拟合演示过这一点，这里只是换了一个更灵活的函数族。

两个导数也必须分开。

- 对输入求导 $\partial u_\theta / \partial x$：网络作为函数的斜率。PINN 的方程残差使用它。
- 对参数求导 $\partial u_\theta / \partial \theta$：损失对参数的敏感度。训练使用它。

两者共用计算图，传播方向可以不同，但数学对象完全不同。

本节只讨论一维输入、一维输出、单隐层、$\tanh$ 激活的网络，不涉及完整的 PINN 训练（第 6 章）。选 $\tanh$ 而非 ReLU，是因为 $\tanh$ 任意阶光滑，二阶导数存在且有界；2.5.3 节将用 ReLU 说明这一选择并非可有可无。

关键推导：让结论有依据

2.5.1 表达能力

先看一个不需要优化、可以直接写出权重的构造。记 $\rho(z)=\max(0,z)$，称为 ReLU 激活。

定理 2.5.1（折线表示与一致逼近） 对任意 $f\in C[-1,1]$ 与 $\varepsilon>0$，存在有限个 ReLU 单元组成的单隐层网络 p，使 $\|p-f\|_\infty<\varepsilon$。

证明 取等距节点 $t_j=-1+2j/n$（$j=0,\dots,n$），记斜率 $s_j=[f(t_{j+1})-f(t_j)]/(t_{j+1}-t_j)$。连接相邻数据点的折线可写成

式（2.5.2）

$$p(x)=f(t_0)+s_0(x-t_0)+\sum_{j=1}^{n-1}(s_j-s_{j-1})\rho(x-t_j).$$

在 (t_k,t_{k+1}) 上，斜率为 $s_0+\sum_{j=1}^{k}(s_j-s_{j-1})=s_k$，且 $p(t_0)=f(t_0)$，故它正是所述折线。又因 $x=\rho(x)-\rho(-x)$，线性项也能用两个 ReLU 单元表示。

若 $x=(1-\lambda)t_j+\lambda t_{j+1}$（$0\le\lambda\le1$），则

$$|p(x)-f(x)|\le(1-\lambda)|f(t_j)-f(x)|+\lambda|f(t_{j+1})-f(x)|.$$

f 在闭区间上一致连续，故当网格宽度 $2/n$ 足够小时，右端小于 ε，且该选择对所有 x 同时有效。□

证明给出的不是抽象的存在性，而是一组可以写出的权重。第 j 个 ReLU 的系数是斜率的改变量 s_j-s_{j-1}：折线每转一次弯，就增加一个 ReLU。这也是 ReLU 网络天然表示分段线性函数的原因。

但定理只保证函数值的一致逼近，不保证导数。折线的二阶导数在段内为零、在折点不存在，与光滑目标无关。这一缺口是 2.5.3 节的主题。

2.5.2 两个导数

输入导数。 记 $z_j=w_jx+b_j$、$t_j=\tanh z_j$、$s_j=1-t_j^2$。由 $dt_j/dz_j=s_j$ 与 $ds_j/dz_j=-2t_js_j$，逐项用链式法则：

式（2.5.3）

$$u_\theta'(x)=\sum_{j=1}^{q}a_jw_js_j,$$

$$u_\theta''(x)=-2\sum_{j=1}^{q}a_jw_j^2t_js_j.$$

两式只用到 $\tanh$ 的一、二阶导数，且对 x 连续。这是 $\tanh$ 网络能直接用于二阶 PDE 强形式残差的原因：u_{xx} 有经典意义。

参数导数。 对参数求偏导时，x 与其余参数固定：

式（2.5.4）

$$\frac{\partial u_\theta}{\partial c}=1,\quad \frac{\partial u_\theta}{\partial a_j}=t_j,\quad \frac{\partial u_\theta}{\partial w_j}=a_j s_j x,\quad \frac{\partial u_\theta}{\partial b_j}=a_j s_j.$$

式（2.5.4）给出 $3q+1$ 维向量 $\nabla_\theta u_\theta(x)$，式（2.5.3）给出标量 $u'_\theta(x)$。维数不同，含义不同，不能互相代替。

手算一例。 取 $q=1$、$(a,w,b,c)=(2,3,0,1)$、$x=0$，则 $z=0$、$t=0$、$s=1$，于是

$$u=1,$$

$$u'=aw\cdot 1=6,$$

$$u''=-2aw^2 ts=0.$$

单点损失 $L=(u-y)^2$（取 $y=0$）为 1。按 (a,w,b,c) 顺序，

$$\nabla_\theta L=2(u-y)\big(t,\ asx,\ as,\ 1\big)=(0,\ 0,\ 4,\ 2).$$

注意 $\partial_w u=0$ 与 $u'=6$ 可以同时成立：前者是"改变参数 w 时输出如何变化"，后者是"改变输入 x 时输出如何变化"。两者求导对象不同，不能混用。

损失梯度。 对均方损失 $L_{\text{data}}=\frac{1}{m}\sum_i(u_\theta(x_i)-y_i)^2$，

式（2.5.5）

$$\nabla_\theta L_{\text{data}}=\frac{2}{m}\sum_{i=1}^{m}\big[u_\theta(x_i)-y_i\big]\nabla_\theta u_\theta(x_i).$$

反向模式一次前向加一次反向即可得到全部 $3q+1$ 个分量，代价与前向一次同阶；若用前向模式逐参数扰动，需要 $3q+1$ 次遍历。这是多参数、少输出时反向传播更省的原因。

若损失中还含输入导数（如 PINN 的残差），则要对 u'_θ、u''_θ 再求参数偏导，会用到 $\tanh$ 的三阶导数。自动微分可以沿图完成，前提仍是这些导数存在：

参数分量	$\partial_\alpha u_\theta$	$\partial_\alpha u'_\theta$	$\partial_\alpha u''_\theta$
c	1	0	0
a_j	t_j	$w_j s_j$	$w_j^2 h_j$
w_j	$a_j s_j x$	$a_j s_j + a_j w_j h_j x$	$2a_j w_j h_j + a_j w_j^2 \dot{h}_j x$
b_j	$a_j s_j$	$a_j w_j h_j$	$a_j w_j^2 \dot{h}_j$

其中 $h_j=-2t_j s_j$、$\dot{h}_j=2s_j(3t_j^2-1)$。这张表可作为小规模程序的独立基准：手算与 AD 相符，才说明求导实现无误。

2.5.3 光滑性、反例与残差界

定理 2.5.2（光滑性传递与 ReLU 的限制） 设 r 为非负整数。若有限层网络的每层由仿射映射与逐分量的 C^r 激活组成，则输出关于输入是 C^r 的。若标量输入的有限网络只使用 ReLU 激活与仿射映射，则其输出是有限分段仿射函数。

证明 第一句：输入坐标函数光滑；若某层各分量是 C^r，其有限线性组合及与 C^r 激活的复合仍是 C^r；按层数归纳即得。$\tanh$ 的分母在实轴上恒正，故为 C^∞，式 (2.5.1) 可作任意有限阶经典求导。第二句：第一层仿射函数在一段区间内至多有一个非恒零根，取正部后仍分段仿射；设某层已有有限个分段点，下一层仿射组合在每段仍仿射，再取正部每段至多新增一个根；有限层、有限单元保证分段数有限。□

于是 ReLU 网络在每个开分段上的二阶导数为零；在斜率跳变的折点，一阶经典导数不存在。用这样的二阶导数构造强形式残差，无法表示一般的非零右端项。这不是实现缺陷，而是表示能力的边界。框架在折点返回约定值，不会使经典导数存在。

光滑还不够：函数值准不代表导数准。 由式 (2.5.3)，一个单元对一阶、二阶导数的贡献分别含 w_j 与 w_j^2；不同单元的函数值可以互相抵消，导数中的这些因子却不同。更根本地，有下述反例。

例 2.5.1（一致逼近仍有大的导数误差） 在 $[-1,1]$ 上令

$$g_k(x)=\frac{\sin(kx)}{k^2},$$
$$k=2,3,\dots$$

则

$$\|g_k\|_\infty\le\frac{1}{k^2},$$
$$\|g_k'\|_\infty\le\frac{1}{k},$$
$$\|g_k''\|_\infty\le 1.$$

取 $x=\pi/(2k)$ 使 $\sin(kx)=1$，取 $x=0$ 使 $\cos(kx)=1$，故三个上界都能达到：$\|g_k\|_\infty=1/k^2$、$\|g_k'\|_\infty=1/k$、$\|g_k''\|_\infty=1$。$k=100$ 时依次为 10^{-4}、10^{-2}、1。

函数值乃至一阶导数都趋于零，二阶导数却不趋于零。这说明"函数拟合好了，导数自然就好"缺少额外条件。它否定的是这一般性推断，并不声称每个训练网络都必然有大导数误差。

方程残差小能否保证解误差小？在一个具体模型问题上，可以给出完整的初等结论。

定理 2.5.3（连续残差给出的误差界） 考虑边值问题

式（2.5.6）

$$-u''(x)=\pi^2\sin(\pi x),\quad -1<x<1;,$$
$$u(-1)=u(1)=0,$$

其解为 $u_*(x)=\sin(\pi x)$。设 $v\in C^2[-1,1]$，记残差 $r(x)=-v''(x)-\pi^2\sin(\pi x)$，

$$M=\|r\|_\infty,$$
$$\eta=\max\{|v(-1)|,|v(1)|\}.$$

则

式（2.5.7）

$$|v(x)-u_*(x)|\le\eta+\frac{M}{2}(1-x^2),$$
$$\|v-u_*\|_\infty\le\eta+\frac{M}{2}.$$

证明 令 $e=v-u_*$，则 $e''=-r$。取 $\phi(x)=\eta+\frac{M}{2}(1-x^2)$，则

$$(e-\phi)'' = M - r \geq 0,$$
$$(-e-\phi)'' = M + r \geq 0.$$

二阶导数非负的函数是凸函数，图像不高于两端点连成的弦。$e-\phi$ 与 $-e-\phi$ 在 $x=\pm 1$ 处的值都不大于零，故在整个区间不大于零，即 $-\phi \leq e \leq \phi$。再用 $0 \leq 1-x^2 \leq 1$ 即得 (2.5.7)。□

定理把解的准确性归结为两个可检查的连续量：边界误差 η 与残差上确界 M。若两者都为零，则 $v = u_*$，顺带证明了唯一性。但它要求 M 是整个区间的最大残差。

有限个配点上的残差为零，推不出 M 小。对任意有限个互异配点 ξ_i，多项式

$$B(x) = (1-x^2)\prod_{i=1}^{N_r}(x-\xi_i)^3$$

非恒零，且 $B(\pm 1) = 0$、$B''(\xi_i) = 0$。于是 $u_* + AB$ 在所有配点上的残差与边界损失都为零，函数误差却是 $|A|\|B\|_\infty$，可以任意大。这是 PINN 用有限配点训练时难以从理论上保证精度的根源。

数值例 2.5.2（只训练函数值的光滑网络） 取 $q = 12$，在 81 个等距点上最小化 L_{data}，同时训练 37 个参数；种子 2026，float64，Adam 迭代 14000 步。在与训练点不重合的 2001 点偏移网格上：

指标	实测值
最终训练均方损失	约 1.1×10^{-6}
训练网格函数值最大误差	2.91×10^{-3}
独立网格函数值最大误差	2.86×10^{-3}
独立网格一阶导数最大误差	1.171×10^{-1}
独立网格二阶导数最大误差	1.647

为区分"导数算错"与"导数逼近不好"，实验把自编 AD 的输入导数与式 (2.5.3) 的闭式比较，最大差不超过 4.0×10^{-15}。求导实现正确，误差来自网络本身。

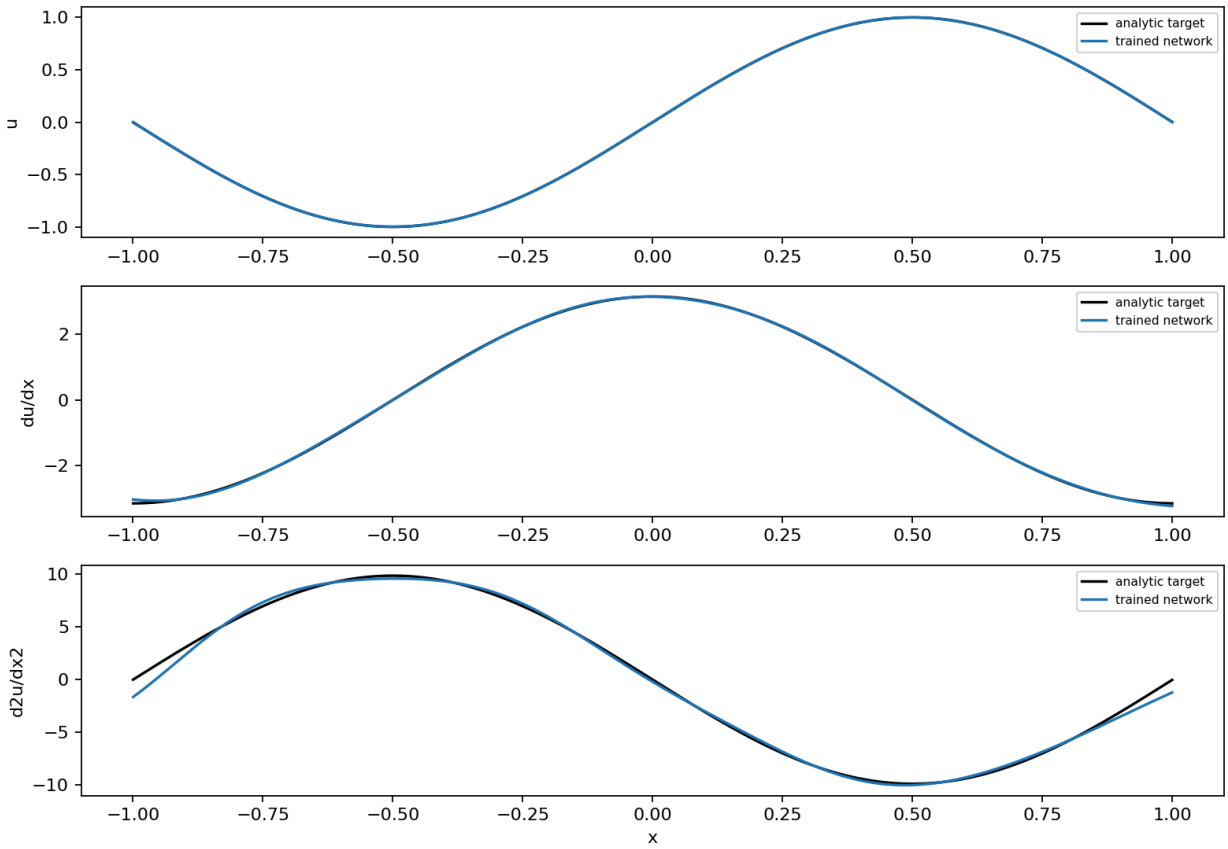


图02-6 • 同一已训练网络与解析目标的函数值、一阶、二阶导数。训练只用 81 点函数值，评价用 2001 个不重合内点；配置与保存参数见 Notebook 2.5。

条件核查表：

结论	用到的假设	假设失效时的后果
输入一、二阶导数有闭式 (2.5.3)	激活二阶可微（$\tanh$）	ReLU 在折点二阶导数不存在，差商约 $1/\delta$
一次反向得到全部参数梯度	计算图可微；控制流不依赖参数的跳变	含 `if/max` 的分支在跳变处梯度不唯一
ReLU 网络一致逼近连续函数（定理 2.5.1）	f 连续；宽度可随精度增大	只保证函数值，不保证导数；不保证训练找得到
残差界 (2.5.7)	$v \in C^2$；M 为连续最大残差	只控配点时 M 未知，界不可用

结论与限制： 表达能力是存在性结论，训练是优化结论，泛化是统计结论。本节只演示第二个，不能反推第一个，也不能当作 PDE 求解的验证。文献定位：[5] 第 6 章（印刷页 157 – 160），[6] §3.1（印刷页 688），[7] 定义 3.2、推论 3.5（印刷页 556）。

实验设计：先预测，再运行

实验任务：用 $\tanh$ 小网络拟合 $f(x) = \sin(\pi x)$，只在函数值上训练，再在独立网格上比较函数值与一、二阶导数。

要填写的项目	本组内容
要回答的问题与运行前预测	问：只用函数值训练，能否同时得到准确的一、二阶导数？预测函数值误差约 10^{-3}，一阶约 10^{-1}，二阶约 10^{0}，逐阶放大约一个到两个数量级。若二阶误差与函数值误差同量级，先怀疑评价代码把导数算错
输入、参数、单位与数据来源	目标 $\sin(\pi x)$，$x \in [-1,1]$，解析导数 $\pi\cos(\pi x)$、$-\pi^2\sin(\pi x)$；训练点 81 个等距点，测试点 2001 个偏移网格点（偏移 0.371）；宽度 $q=12$，参数 37 个；全部无量纲；训练与测试严格分离
独立参考解与基线	参考解为解析 f 及其导数，独立于网络；基线取零函数（误差上界约 1）与最佳常数（约 0.64），用于判断网络是否学到结构；导数基线用解析式而非差分，避免把差分误差混入评价
误差指标与容差	最终训练均方损失、训练网格最大误差、独立网格函数值与一、二阶导数的最大绝对误差。函数值容差 5×10^{-3}（约振幅的 0.5%）；导数不设统一容差，只报量级，因为训练目标不含导数，任何阈值都是事后选择
环境与运行预算	Python 3.12 + NumPy，CPU，float64；种子 2026；Adam（$\beta_1=0.9,\beta_2=0.999,\varepsilon=10^{-8}$），学习率 0.01 走 8000 步后降为 0.002，共 14000 步；单机约 1–3 秒，随 BLAS 与线程数变化

本节代码在同号 Notebook 2.5 小节。种子固定只保证同一环境下可复现，不保证跨平台逐位相同。

结果解释与本节结论

先看一个容易忽略的对照：训练网格与独立网格的函数值误差分别为 2.91×10^{-3} 与 2.86×10^{-3}，几乎相同。这说明本例没有过拟合：37 个参数对 81 个点，容量并不富余，泛化间隙很小。因此后面导数误差大，原因不在过拟合，而在训练目标中不含导数信息。

再看逐阶放大：一阶误差 1.17×10^{-1} 是函数值误差的 41 倍，二阶误差 1.65 又是一阶的 14 倍。这一放大不是程序缺陷，自编 AD 与闭式相差不超过 4.0×10^{-15}。也就是说，0.117 与 1.647 是网络本身对真实导数的逼近误差，评价环节没有引入可察觉的额外误差。先用 AD 验证评价器，再讨论网络的精度，这是本节的方法论要点。

逐阶放大的原因是：若 $u-f$ 含频率 ω 的分量，求导一次放大 ω 倍，两次放大 ω^2 倍。网络为贴合函数值会在小尺度上作细微调整，这些调整在函数值上微不足道，在曲率上却很明显。这与优化器好坏无关。

本节结论： 只用函数值训练，函数值可以拟合到 10^{-3}，但导数精度逐阶退化到 10^{-1} 与 10^{0}。若下游任务需要导数（PDE 残差、灵敏度、最优控制），必须把导数信息显式放入损失，或改用导数性质更好的表示。同时，残差界 (2.5.7) 要求连续最大残差，有限配点上的小残差不构成精度保证。

边界： 本例是 $q=12$、一维、光滑目标、14000 步 Adam。结论不能外推为"神经网络不能求导"：加大宽度、加入导数损失、改用光滑基都可以显著改善。本节只主张：在当前设置下，函数值精度不能作为导数精度的代理指标。

易错点与讨论

一、把光滑激活当成定理。［7］推论 3.5 给出的是充分条件：在开集、经典导数存在、激活为 ℓ-finite 等前提下，存在网络同时逼近函数与导数。充分条件不等于必要条件。工程上说"用 $\tanh$ 更稳"是对的，说"必须用光滑激活，否则定理不成立"则把建议讲成了定理。

二、以为函数值拟合好，导数就一定好。 实测 $2.86\times10^{-3}\to1.171\times10^{-1}\to1.647$，逐阶放大。机制是求导放大高频成分，与优化好坏无关。

三、用差分评价网络的导数。 差分自身在 δ 不当时误差可达 10^{-1}，与待测的一阶导数误差同量级，两者混在一起无法分离。本组改用自编 AD 交叉核对（差 $\le 4.0\times10^{-15}$），才使"0.117 是网络的误差"这一判断成立。

四、把固定种子当成跨平台可复现。 种子只固定初值与样本顺序；Adam 的更新依赖梯度求和，而 BLAS 与多线程的归约顺序在不同机器上不同，末位会分叉并被混沌放大。本组重跑时 `final_loss` 在 1.06×10^{-6} 与 1.10×10^{-6} 之间变动，二阶导数误差在 1.64688 与 1.64694 之间变动。凡训练末态量，只报有效数字与量级。

讨论题： 若在损失中加入 $\lambda\sum_i|u_\theta'(x_i)-f'(x_i)|^2$，二阶导数误差会改善吗？改善的机制是"学到了正确的曲率"，还是仅仅"把评价指标写进了目标"？设计一个能区分这两种解释的实验。

思考：

1. 训练网格与独立网格的函数值误差几乎相同（2.91×10^{-3} 与 2.86×10^{-3}），这说明什么？如果二者差很多，又说明什么？
2. 一阶导数误差是函数值误差的 41 倍，二阶又是它的 14 倍。这个放大能通过增加训练点消除吗？能通过增大网络宽度消除吗？
3. ReLU 反例中差商约 10^4，真实导数 $O(1)$。若把 δ 从 10^{-4} 改成 10^{-2}，差商会变成多少？这个变化说明问题出在步长，还是出在表示本身？

回到开场

以章节开场中 $f(x)=x^2$ 在 $[0,1]$ 上的积分为例，2.3 节已证明：将区间等分为 N 个子区间时，复合梯形公式满足

$$T_N-\int_0^1 x^2\,dx=\frac{1}{6N^2}.$$

因此，子区间数从 4 增加到 8 时，绝对误差由 $\frac{1}{96}$ 减小到 $\frac{1}{384}$，后者为前者的四分之一。实验记录中的 $T_4=0.34375$、$T_8=0.3359375$ 与上述精确公式一致。

开场图示中的梯形面积之和，实质上是分段线性插值函数的积分。对一般函数，若 $f\in C^2[a,b]$，则等距复合梯形公式具有 $O(h^2)$ 的误差上界；仅凭几何示意或函数连续性，不能断言具有二阶收敛性。即使满足二阶误差上界，每次步长减半后的误差比也不一定恰为 $\frac{1}{4}$；该比值的渐近结论还需误差具有非零的二阶主项。

结合本章讨论，可以从以下三个方面回应章节引入的问题。

1. 近似表示的依据。 对 $n+1$ 个互异节点及其函数值，次数不超过 n 的插值多项式存在且唯一。分段线性插值在每个子区间内用直线连接端点数据；在相应光滑性条件下，可通过插值余项估计其与原函数

的偏差。节点处吻合仅是插值条件，并不保证整个区间内的误差足够小。

2. 不同运算下的误差。 函数值、积分和导数对应不同的评价目标。2.1 节利用插值余项建立函数值误差界；2.3 节对局部插值误差积分，得到求积误差；2.4 节利用插值多项式的导数构造差分公式，并借助 Taylor 展开等方法分析截断误差。含有未知中间点的余项表达式不能未经论证直接求导，浮点差分计算还须考虑舍入误差。因此，各种误差结论均须在明确的光滑性、节点及步长条件下讨论。
3. 含噪数据与可调表示。 对含噪或较多的观测数据，最小二乘方法以给定样本上的残差平方和最小为目标，不再要求逐点插值（2.2）。神经网络进一步采用可训练的参数化函数表示（2.5）。离散拟合误差较小，并不自动保证独立数据上的误差或导数误差较小，仍须针对实际目标分别检验。

全章结论可汇总为下表。

对误差公式所作的运算	得到的结论	误差阶	关键条件
取绝对值、取最大	插值误差界 $\frac{\|f^{(n+1)}\|_\infty}{(n+1)!}\|\omega_{n+1}\|_\infty$	由节点与 f 共同决定	$f \in C^{n+1}$
两边积分	梯形误差 $-\frac{(b-a)h^2}{12}f''(\xi)$	$O(h^2)$	$f \in C^2$
构造差分（一次插值）	前向差分误差 $-\frac{h}{2}f''(\xi)$	$O(h)$	$f \in C^2$
构造差分（二次插值）	中心差分误差 $-\frac{h^2}{6}f'''(\eta)$	$O(h^2)$	$f \in C^3$
换成最小二乘	正规方程与投影	由独立网格评估	A 列满秩
换成参数化基	网络逼近与求导	逐阶退化	激活足够光滑

下一章从微分方程的离散化研究"用有限个数求解函数"的另一途径。逼近与误差仍以同样的方式出现，只是未知函数不再由公式给出，而由方程本身定义。

课后可完成五道配套习题。本章传统方法的依据为李庆扬等《数值分析》第五版第 2、3、4 章；Chebyshev 与 Runge 的进一步解释、AD 及网络相关材料见参考文献与定位。

本章小结与课后习题

插值。 已知有限个点的值，求其他位置的值。适用条件明确：节点互异、候选空间给定，解才唯一；区间误差还依赖节点分布与函数的高阶变化。两处易混：节点处误差为零不代表区间内准确；次数提高不代表精度提高（等距节点下 n 从 8 到 32，误差从 1.04 涨到 5059）。

最小二乘。 数据多于未知量时的压缩方法：把 41 个观测压成 13 个系数。适用条件是设计矩阵列满秩，秩亏时必须显式拒绝，而不是给出一个看似合理的解。三处易混：训练残差、独立网格误差与一致范数误差是三个不同目标，$d=30$ 一例中残差最小（4.54×10^{-2}）而网格误差最大（28.2）；换基不改变解，条件数下降 6782 倍改变的是舍入敏感性。

插值积分。 求积公式就是对插值多项式积分，权重 $A_k=\int \ell_k$ 只与节点有关。梯形误差的几何因子为 $\int\omega=-h^{\frac{3}{6}}$，因此是二阶的，符号由 f'' 决定，对一次函数精确。光滑性不足时阶会下降，连续性已足以保证收敛。

插值微分。 差分是插值多项式的导数：一次插值给前向差分，二次插值给中心差分。误差含两项：截断随 h 减小而下降，舍入随 h 减小而上升，两者交汇出最优步长（前向 $\sqrt{\varepsilon}$，中心 $\varepsilon^{\frac{1}{3}}$）。自动微分不作除法，只沿计算图传播链式法则，误差只剩浮点舍入。

网络逼近与求导。 基函数也由数据塑形。须分开三件事（表达、训练、泛化）与两个导数（对输入、对参数）。ReLU 能一致逼近连续函数，但分段仿射的结构使它不适合表示有非零连续二阶导数的目标；$\tanh$ 光滑，但函数值准不代表导数准，实测逐阶退化为 $10^{-3} \to 10^{-1} \to 10^{0}$。残差界要求连续最大残差，有限配点的小残差不构成精度保证。

五节由一条链相连：插值给出用有限信息重建函数的框架；最小二乘把条件过强的问题放宽为残差最小；积分与微分是对同一插值式的两种运算，误差因而呈现不同的阶；网络把固定的基换成可学的基，链式法则始终是求导的工具。

课后题按五节对应填写，见A1组-习题.md；参考解答单独放在A1组-习题参考解答.md。

附录 A　范数、条件数与误差的度量

正文中省略的若干定义集中于此，需要时查阅。

A.1 向量范数。 对 $x \in \mathbb{R}^m$ 与 $p \geq 1$，

$$\|x\|_p = \Big(\sum_{i=1}^{m} |x_i|^p\Big)^{1/p},$$

$$\|x\|_\infty = \max_{1 \leq i \leq m} |x_i|.$$

最常用的是 $p = 2$（欧氏范数）与 $p = \infty$（最大范数）。三者等价：$\|x\|_\infty \leq \|x\|_2 \leq \sqrt{m}\,\|x\|_\infty$。范数把误差大小化为一个数；不同范数给出的数不同，报告时须说明所用范数。

A.2 函数范数。 对有界函数 g 在区间 $[a,b]$ 上，

$$\|g\|_{\infty,[a,b]} = \max_{x \in [a,b]} |g(x)|$$

称为一致范数（最大范数）。若 g 连续、区间闭，最大值一定取到。连续平方范数为

$$\|g\|_{2,[a,b]} = \Big(\int_a^b |g(x)|^2\,dx\Big)^{1/2}.$$

两者区别重要：$\|\cdot\|_2$ 允许函数在零测集上很大，一个尖峰可以不影响积分；$\|\cdot\|_\infty$ 抓住最坏的一点。2.2 节强调的三个误差目标不同，根源在此。

A.3 离散网格最大误差。 若只在网格 $z_0, \ldots, z_M$ 上评价，记

$$E_{\text{grid}} = \max_{0 \leq k \leq M} |p(z_k) - f(z_k)|.$$

它是 $\|p - f\|_\infty$ 的一个近似：网格越密通常越大，但不会超过连续范数（除非漏掉峰值）。因此"网格最大误差"不能直接称为"最大误差"，跨实验比较必须固定网格。

A.4 绝对误差、相对误差与机器 epsilon。 绝对误差为 $|\tilde{x} - x|$；相对误差为 $|\tilde{x} - x|/|x|$（$x \neq 0$）。float64 的相对精度 $\varepsilon \approx 2.2 \times 10^{-16}$，即约 16 位十进制有效数字。本章所说"$10^{-15}$ 量级"即指这一尺度：误差已小到浮点表示的下限。

A.5 矩阵条件数。 对可逆矩阵 A，

$$\kappa_p(A) = \|A\|_p \|A^{-1}\|_p,$$

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)},$$

其中 $\sigma_{\max}, \sigma_{\min}$ 为最大、最小奇异值。对列满秩的长矩阵，式 (2.2.4) 用 $\max/\min$ 的比值定义 κ_2，与奇异值定义一致。基本性质：

- $\kappa_p(A) \geq 1$，等号当且仅当 A 是标量乘正交矩阵；
- $\kappa_2(A^{\mathsf{T}}A) = \kappa_2(A)^2$，正规方程把病态平方；
- $\kappa_2(AB) \leq \kappa_2(A)\kappa_2(B)$。

条件数不是误差，而是误差放大倍数的上界。它大，说明问题本身敏感；它小，不保证这次算得准。

附录 B　内积、正交与正交函数系

B.1　内积与正交。　实向量空间上的内积 $\langle u, v\rangle$ 满足对称性、对第一变元线性、正定性。$\mathbb{R}^m$ 的标准内积为 $\langle u, v\rangle = u^{\mathsf{T}}v$。若 $\langle u, v\rangle = 0$，称 u, v 正交；一组两两正交且范数为 1 的向量称为标准正交系。

B.2　函数的内积与正交函数系。　在 $[a, b]$ 上取权函数 $w(x) > 0$，定义

$$\langle g, h\rangle_w = \int_a^b g(x)h(x)w(x)\,dx.$$

若 $\langle \phi_j, \phi_k\rangle_w = 0$（$j \neq k$），称 $\{\phi_k\}$ 为关于权 w 的正交函数系；若再有 $\|\phi_k\|_w = 1$，称为标准正交系。若每个多项式都可由 $\{\phi_k\}$ 的有限线性组合表示，则称其为正交多项式系。

B.3　正交带来的简化。　在连续最小二乘（最佳平方逼近）中，若用正交系展开 $p = \sum_k c_k\phi_k$，则

$$c_k = \frac{\langle f, \phi_k\rangle_w}{\langle \phi_k, \phi_k\rangle_w},$$

每个系数独立计算，不必解线性方程组。这是 2.2 节换基思想的连续版本：正交基使正规方程的矩阵成为对角阵。

B.4　两个常用的正交多项式系。

名称	权函数	区间	三项递推	主要用途
Legendre P_k	1	$[-1, 1]$	$(k+1)P_{k+1} = (2k+1)xP_k - kP_{k-1}$	Gauss–Legendre 求积
Chebyshev T_k	$\dfrac{1}{\sqrt{1-x^2}}$	$[-1, 1]$	$T_{k+1} = 2xT_k - T_{k-1}$	插值节点、最小最大逼近

Chebyshev 的正交关系可显式写出：

$$\int_{-1}^{1} \frac{T_j(x)T_k(x)}{\sqrt{1-x^2}}\,dx = \begin{cases} 0, & j \neq k, \\ \pi, & j = k = 0, \\ \pi/2, & j = k \geq 1. \end{cases}$$

作代换 $x = \cos\theta$，左端化为 $\int_0^{\pi} \cos(j\theta)\cos(k\theta)\,d\theta$，即可验证。

B.5　连续正交不等于离散正交。　上述积分关系是对连续区间而言的。任意采样点上的离散和 $\sum_i T_j(t_i)T_k(t_i)$ 一般不为零；只有在特定节点（如 Chebyshev 零点或极值点）上才有离散正交性。连续最佳平方逼近与离散最小二乘是两个不同的问题，换基不会把前者变为后者。

选读边界

Monte Carlo 与高维积分移出必修主线，仅作选读；数值积分主线只用一维复合梯形，目的是把逼近与误差连接起来，而不是穷举求积公式。Simpson、Gauss 求积、Romberg 外推与自适应积分均不在必修范围内（2.3 节扩展框只作提及）。正交多项式的一般理论、Lebesgue 常数的估计、龙格现象的复分析证明，也不作要求。

选读内容不计入本章 10 学时（五节各 2 学时），也不计入最低完成要求。当前可直接引用的已有材料：

- 逐项复核记录：审校/来源核查.md
- 独立高精度补核（90 位 Decimal 复算 $n = 32$ 峰值）：审校/独立数值补核.json
- 可复现实验脚本：ch2_numerics.py（章目录内；Notebook 空内核执行用仓库根目录的 `tools/execute_notebook.py`）
- 离线预览：本章 `预览/` 目录下的 HTML 页面，由仓库根目录的 `tools/build_preview.py` 生成；两个工具脚本不在本章目录内，故此处只记名称，不设相对链接

参考资料、图片来源与AI使用

下表把正文中的主张、公式与图片逐一对应到文献位置。页码只对用户提供的本地 PDF 负责：PDF 页从 **1** 起计，印刷页按页面显示的数字计；两者偏移已在参考文献与定位.md中逐条说明（[1] 相差 **+13**、[3] 相差 **+31**、[5] 仅第 6 章读段相差 **+12**、[6] 的 PDF 1 为印刷 686、[7] 的 PDF 1 为印刷 551）。[2] 的偏移随位置变化，不能整本套用一个常数。

支撑的主张/公式/图片	作者、标题、版本/年份	页码/章节/链接	本组人工核对
拉格朗日插值唯一性与余项	[1] 李庆扬、王能超、易大义《数值分析》第五版，清华大学出版社（出版年未从版权页核实）	§2.2.2 – §2.2.3，式 (2.8) – (2.12)，印刷页 25 – 27	未核对
Chebyshev 递推、零点与插值误差界	[1] 同上	§3.2.3 – §3.2.4，印刷页 61、63 – 64	未核对
重心插值求值公式	[2] Trefethen, Approximation Theory and Approximation Practice, Extended Edition, SIAM, 2019	第 5 章，式 (5.6) – (5.11)，印刷页 35 – 36	未核对
Runge 现象与等距节点失效	[2] 同上	第 13 章，印刷页 95 – 97	未核对
Lebesgue 常数与节点稳定性	[2] 同上	第 2 章极值点定义、第 15 章 Lebesgue 函数，印刷页 7 – 8、107 – 109	未核对
离散最小二乘、法方程与唯一性条件	[1] 同上	§3.4.1，式 (4.1) – (4.7)，定义 7，印刷页 73 – 74	未核对
正规方程条件数平方、薄 QR 求解	[3] Driscoll、Braun, Fundamentals of Numerical Computation, SIAM, 2017	§3.2 – §3.3，定理 3.2.1、式 (3.3.3)，印刷页 103、109 – 110	未核对

支撑的主张/公式/图片	作者、标题、版本/年份	页码/章节/链接	本组人工核对
一致最优逼近不等同平方最优	[2] 同上	第 10 章交错刻画，印刷页 73–75	未核对
复合梯形公式及误差展开	[1] 同上	§4.3.1，式 (3.1)–(3.3)，印刷页 106–107	未核对
差分、符号微分与 AD 的区别；前向/反向模式	[4] Baydin、Pearlmutter、Radul、Siskind, “Automatic Differentiation in Machine Learning: a Survey”, JMLR 18(153), 2018	§2.1、§3.1–§3.2，表 2–3，印刷页 4–13	未核对
网络作为参数化函数；表达能力与可训练性之别	[5] Goodfellow、Bengio、Courville, Deep Learning, MIT Press, 2016（本地为含编辑提示的稿件）	本地稿件第 6 章 §6.1、§6.4、§6.5，印刷页 157–160、174、188–189	未核对
PINN 的方程残差与数据/物理损失构造	[6] Raissi、Perdikaris、Karniadakis, J. Comput. Phys. 378: 686–707, 2019	§3.1，式 (3)–(4)，印刷页 688	未核对
函数与导数同时逼近的充分条件	[7] Hornik、Stinchcombe、White, Neural Networks 3(5): 551–560, 1990	定义 2.2、定义 3.2、推论 3.5，印刷页 553、556	未核对
历史背景：最小二乘的提出	Legendre, Nouvelles méthodes pour la détermination des orbites des comètes, 1805	附录（最小二乘）	未核对
历史背景：最小二乘的早期使用与优先权声明	Gauss, Theoria motus corporum coelestium, 1809	第 2 卷相关章节	未核对
历史背景：连杆机构与最小最大逼近	Chebyshev, “Théorie des mécanismes connus sous le nom de parallélogrammes”, 1854	全文	未核对
历史背景：多项式一致逼近定理	Weierstrass, “Über die analytische Darstellbarkeit…”, 1885	全文	未核对
历史背景：等距插值的反例	Runge, “Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten”, Z. Math. Phys. 46 (1901), 224–243	全文	未核对
章节导入示意图	仓库初始化助手绘制的概念示意，非文献插图	本地仓库 `ABC共用-图片/A-章节导入.svg`	未核对
数学家时间线示意图	同上，年份取公开文献年份	本地仓库 `ABC共用-图片/A-人物时间线.svg`	未核对

支撑的主张/公式/图片	作者、标题、版本/年份	页码/章节/链接	本组人工核对
本章主线示意图	同上，五站结构与正文一致	本地仓库 ABC共用-图片/A-本章主线.svg	未核对

图片来源与使用边界。 本章无文献插图复制件。图02-1 至图02-3 为概念示意与人物时间线，由仓库初始化助手绘制，不承担定量主张；图02-4 至图02-6 为 Notebook 实跑输出（图02-6 见 2.5 节）。比喻的局限已在"回到开场"一段写明：分段线性插值只能说明梯形法的几何来源，不能保证任意河岸都有二阶收敛率。人物年份取自公开出版信息，正文已注明标注年份为该工作的公开年份，不作为史学结论。

AI 使用说明。 AI 参与三类工作：按五节结构重整章节、起草栏目文字、核对数值与文献页码。所有数值均来自本机实跑 `ch2_numerics.py` 与 Notebook 的输出记录（见实验结果.json），不是生成式估计；本次改写未新增、未修改任何实验数值，只改变了叙述与呈现顺序。文献页码取自参考文献与定位.md并标注了偏移关系。AI 起草与人工结论须分开看待：本表"本组人工核对"一列全部标为"未核对"，表示尚无学生逐条比对原书页面；AI 内部审查不等于正式 B/C 验证，也不等于学生人工核验。

本章习题与参考解答

第02章 逼近的智慧：课后习题

本习题为教材编写稿，五道题依次对应正文 2.1—2.5 节。所有变量无量纲。建议先完成手算，再运行实验。配套程序为 ch2_numerics.py，参考解答见习题参考解答。下面的辨析语句均为教学构造。

题1：插值精确性与区间误差（对应 2.1）

设 $\mathcal{P}_n$ 是次数不超过 n 的实多项式空间，$f(x)=1/(1+25x^2)$，$x\in[-1,1]$。

1. 对节点 $-1,0,1$，写出三个Lagrange基函数，并求数据 $q(x)=2x^2-x+3$ 的插值多项式。为什么任意 $\mathcal{P}_2$ 中的多项式都被这三个节点精确恢复？用 $-0.73,0.23,0.89$ 三个非节点检验你的实现。
2. 对一般互异节点 $x_0,\dots,x_n$，说明Lagrange基函数的节点性质如何给出插值存在性和唯一性。若 $f\in C^{n+1}[-1,1]$，写出插值余项及其上界，并指出其中随 n 改变的两种量。该上界是否自动随 n 减小？
3. 取 $n=8,16,32$，分别使用 $x_j=-1+2j/n$ 和Chebyshev零点 $x_j=\cos((2j+1)\pi/(2n+2))$，$j=0,\dots,n$。用同一6001点独立网格 $z_\ell=-1+2(\ell+0.371)/6001$，$\ell=0,\dots,6000$，计算 $\max_\ell\vert p_n(z_\ell)-f(z_\ell)\vert $，列出六组结果并画误差曲线。可用NumPy及附带的重心插值实现；须说明它计算的仍是同一Lagrange插值多项式。另报节点最大误差及节点命中处理方式。
4. 辨析：“节点误差为零，所以增加节点一定提高整个区间及区间外的精度。”解释第3问的证据能说明什么、不能说明什么；Chebyshev零点是否包括区间端点？

题2：相同多项式空间与不同误差目标（对应 2.2）

1. 用常数 c 拟合 x^2。先对样本 $\{-1,0,1\}$ 最小化 $\sum_{x\in\{-1,0,1\}}(c-x^2)^2$，再在连续区间 $[-1,1]$ 最小化 $\max_{x\in[-1,1]}\vert c-x^2\vert $。分别求唯一最优常数和目标函数最小值，解释两者不同的原因。
2. 对41个样本 $x_i=-1+2(i-1)/40$，$i=1,\dots,41$，取 $y_i=e^{x_i}+\eta_i$。噪声由 `np.random.default_rng(2026).normal(0.,.015,41)` 一次生成后共用。固定次数 $d=12$，分别以

$1, x, \dots, x^{12}$ 和 $T_0(x), \dots, T_{12}(x)$ 建立设计矩阵。写出尺寸、最小二乘目标及正规方程；说明满列秩条件为何成立，以及两种基在精确算术下得到同一最优多项式的理由。无需实现QR或SVD分解。

3. 用 `numpy.linalg.lstsq` 求两种基下的系数，在共用独立网格 $z_\ell = -1 + 2(\ell + 0.371)/2001$，$\ell = 0, \dots, 2000$，分别报告训练残差 $\|Ac - y\|_2$、相对无噪目标 e^x 的网格最大误差及两条预测曲线的最大差异。选做：用 `numpy.linalg.cond(A)` 报告矩阵的2范数条件数，作为数值敏感性的指标；本题不要求推导该例程内部算法。
4. 辨析：“Chebyshev基的条件数更小，因此它在同一数据和次数下定义了另一个、更优的最小二乘问题，并自动给出连续最优一致逼近。”指出错误；若同时改变次数，是否还能把效果全部归因于换基？为什么训练残差更小也不能保证无噪目标误差更小？

题3：插值积分——误差从余项里积出来（对应 2.3）

1. 对 $I = \int_0^1 x^2\,dx$，手算复合梯形值 T_4, T_8，并推导一般 $N \ge 1$ 时的 $I - T_N$。验证加密一倍后的绝对误差比为4，并与 $|I - T_N| \le (b-a)h^2\|f''\|_\infty/12$ 对照。说明这条误差公式是如何由插值余项积分得到的，几何因子 $\int \omega$ 等于什么。
2. 在 $[u, v]$ 上令 $h = (v-u)/2$，用二次插值（端点与中点三点）推导 Simpson 公式 $S = \frac{h}{3}[f(u) + 4f(u+h) + f(v)]$ 在单区间上的形式，并验证它对三次多项式精确、对四次多项式不精确。由此说明它的代数精度是几，误差阶为何是 $O(h^4)$。选做：用复合 Simpson 重算第1问的积分，比较 $N = 4$ 时它与 T_4 的误差。
3. 设 $f(x) = \sqrt{x}$、$[0, 1]$。证明仅第一段就给出 $I - T_N \ge h^{3/2}/6$，因而 $|I - T_N| \le Ch^2$ 不可能对与 N 无关的 C 成立。由此预测 N 加倍时误差比的极限值，并解释它为什么不是 0.25。
4. 辨析：“积分误差比是 0.25，所以任何连续函数的梯形误差都按四分之一缩小。”指出错误所在：问题出在阶、出在常数，还是出在对齐方式？再说明为什么“梯形公式对一次函数精确”不能推出“对光滑函数误差很小”。

题4：插值微分——截断、舍入与自动微分（对应 2.4）

1. 令 $F(x) = \sin(x^2) + e^x$。画出节点 $v_0 = x, v_1 = v_0 v_0, v_2 = \sin v_1, v_3 = e^{v_0}, v_4 = v_2 + v_3$ 的计算图，写出每个节点的前向值与切向导数，初值 $\dot v_0 = 1$。另从 $\bar v_4 = 1$ 写出反向伴随量；说明 v_0 从两条分支得到的贡献为什么要相加。
2. 用一次插值导出前向差分、用二次插值导出中心差分，并写出各自的截断误差主项。说明为什么前向是一阶、中心是二阶，以及"差分公式 = 插值多项式的导数"这句话在这里的具体含义。
3. 在 $x = 0.7$ 运行前向和反向自动微分，与解析导数比较。对 $h = 10^{-1-j/4}$，$j = 0, \dots, 60$，同时计算前向差分和中心差分，画绝对误差随步长变化的双对数图，报告误差最小点及最小步长处的结果。可用附带的教学 `Dual`、`Graph`、`Jet2` 类；说明这三个类是链式法则的实现，不含差分近似。
4. 辨析：“自动微分就是令差分步长非常小；AD 结果接近解析导数，就证明了积分实验也准确。”指出两处错误，区分截断误差、舍入误差，以及"对已给计算过程求导"的浮点误差；并说明为什么前向与中心差分的最优步长相差约 $\varepsilon^{-1/6}$ 倍。

题5：网络的输入导数、参数导数与光滑性（对应 2.5）

令 $u_\theta(x) = c + \sum_{j=1}^{q} a_j \tanh(w_j x + b_j)$，$x \in [-1, 1]$，所有参数为实数，$\theta = (a, w, b, c) \in \mathbb{R}^{3q+1}$。

1. 记 $t_j=\tanh(w_jx+b_j)$，推导 $u'_\theta(x),u''_\theta(x)$ 及对 a_j,w_j,b_j,c 的偏导。对训练点 x_i、数据 y_i 和均方损失 $L(\theta)=m^{-1}\sum_i(u_\theta(x_i)-y_i)^2$，写出参数梯度。用 $q=1,(a,w,b,c)=(2,3,0,1),x=0,y=0$ 手算 u,u',u'' 及该单点损失的参数梯度。
2. 读取本章保存的网络参数或复现实验：目标 $f(x)=\sin(\pi x)$，81个等距训练点，$q=12$，其余初始化与迭代设置以 `train_network` 为准。用 $z_\ell=-1+2(\ell+0.371)/2001$ 的2001点独立网格评估函数值、一阶和二阶导数误差，解析参考分别为 $f,f'=\pi\cos(\pi x),f''=-\pi^2\sin(\pi x)$。分别报告训练均方损失与这三个网格最大误差。以第1问公式和教学 `Jet2` 的输出相互核查；不得只报训练损失。
3. 取 $g_k(x)=\sin(kx)/k^2$，$k=2,3,\dots$。证明 $\|g_k\|_\infty\to0$、$\|g'_k\|_\infty\to0$，但 $\|g''_k\|_\infty\not\to0$，范数均指 $[-1,1]$ 上的连续一致范数。代入 $k=100$ 给出三个精确范数。它否定了什么推断？是否说明每个实际训练网络都必然有大导数误差？
4. 对 $\rho(x)=\max(0,x)$，讨论一阶与二阶经典导数及 $x=0$ 的情形。再讨论由有限多个ReLU单元组成的上述单隐层结构，为什么不适合直接表示有非零连续二阶导数的经典解。借助形式残差 $r_\theta(x)=-u''_\theta(x)-\pi^2\sin(\pi x)$，解释为什么求残差先需要输入导数，而训练损失需要参数导数；值拟合实验是否已求解带边界条件的微分方程？

第02章 逼近的智慧：习题参考解答

对应习题，五道题依次对应正文 2.1—2.5 节。本文件为公开的编写稿参考解答，不纳入学生版教材正文目录。手算结论与实际程序观测分开列示；数值结果摘自本章实验结果.json，实现见 ch2_numerics.py。来源编号与正文一致。正式B组核验、C组复核及人工签认均尚无记录；本文件的AI内部审读不代表这些状态。

题1：插值精确性与区间误差（对应 2.1）

1.1 低阶基准

三个基函数为

$$\ell_0(x)=\frac{x(x-1)}{2},\qquad \ell_1(x)=1-x^2,\qquad \ell_2(x)=\frac{x(x+1)}{2}.$$

数据为 $(6,3,4)$，因此

$$p_2=6\ell_0+3\ell_1+4\ell_2=2x^2-x+3=q.$$

三个非节点的精确值依次为 $4.7958,2.8758,3.6942$。这不是仅在三个检验点成立的近似：两个次数不超过2的多项式若在三个互异点相等，其差有三个根，只能是零多项式，因此在实轴上恒等。

1.2 一般插值与余项

对互异节点，

$$\ell_j(x)=\prod_{r\neq j}\frac{x-x_r}{x_j-x_r},\qquad p_n(x)=\sum_{j=0}^{n}f(x_j)\ell_j(x).$$

由 $\ell_j(x_i)=\delta_{ij}$ 得存在性。若两插值多项式均在 $\mathcal{P}_n$ 中，其差有 $n+1$ 个互异根，故为零，得唯一性。

当 $f\in C^{n+1}[-1,1]$，非节点 $x\in[-1,1]$ 满足

$$f(x)-p_n(x)=\frac{f^{(n+1)}(\xi_x)}{(n+1)!}\omega_{n+1}(x),\qquad \omega_{n+1}(x)=\prod_{j=0}^{n}(x-x_j),$$

其中 $\xi_x \in (-1,1)$。固定该非节点，令 $K=(f(x)-p_n(x))/\omega_{n+1}(x)$，则 $f-p_n-K\omega_{n+1}$ 在 $n+2$ 个互异点为零。连续使用Rolle定理得其 $n+1$ 阶导数某处为零，从而 $f^{(n+1)}(\xi_x)-K(n+1)!=0$。节点处误差本来为零。于是

$$\|f-p_n\|_\infty \le \frac{\|f^{(n+1)}\|_\infty}{(n+1)!}\|\omega_{n+1}\|_\infty.$$

导数阶次、节点乘积均随 n 改变，不能只见分母阶乘就判断右侧递减。该误差结论仅在给定区间及光滑条件下成立。

1.3 实测与实现核查

`interpolation_experiment` 用同一6001点偏移内部网格评估六个插值多项式。观测值如下，保留的位数用于阅读，完整数据见JSON。

次数 n	等距节点网格最大误差	Chebyshev零点网格最大误差
8	1.04517659	0.170835560
16	14.3937633	0.0326135736
32	5059.01137	0.00140174227

重心权重 $w_j=1/\prod_{r\neq j}(x_j-x_r)$，在非节点处计算

$$p_n(x)=\frac{\sum_j w_j f(x_j)/(x-x_j)}{\sum_j w_j/(x-x_j)}.$$

将Lagrange式分子分母同除 $\omega_{n+1}(x)$ 即得该表达式；分母恒等于 $1/\omega_{n+1}(x)$，可从常函数1的插值恒等式推出。共同缩放全部权重不改变商。实现对节点命中直接返回给定值，不做 ∞/∞ 运算，故六例节点误差均记录为零。非节点低阶测试还独立使用直接Lagrange乘积式核查。

1.4 辨析

节点误差检验的是约束满足程度；网格误差检验节点之间的行为。表中等距节点误差随这三次加密急剧增大，Chebyshev零点的网格误差减小，与Runge现象及端点附近节点配置的作用相符。Chebyshev零点不含 ±1。表格只证明这些固定设置下的数值现象，不证明所有光滑函数都收敛，也不控制区间外误差；偏移内部网格还不包含端点。题面容差允许高次等距插值的浮点消去影响，不能用放宽容差掩盖数量级错误。

题2：相同多项式空间与不同误差目标（对应 2.2）

2.1 两个最优常数

离散最小二乘目标

$$J(c)=2(c-1)^2+c^2=3\left(c-\frac{2}{3}\right)^2+\frac{2}{3},$$

故唯一解 $c_{\mathrm{LS}}=2/3$，最小平方和为 $2/3$。若写均方误差，最小值为 $2/9$，须说明归一化。

连续区间上 x^2 的值域为 $[0,1]$，因此

$$\max_{[-1,1]}|c-x^2|=\max\{|c|,|1-c|\}.$$

三角不等式给出 $1 \le |c| + |1-c| \le 2\max(|c|, |1-c|)$，所以误差至少 $1/2$。取 $c = 1/2$ 达到下界。若误差等于 $1/2$，同时 $|c| \le 1/2$、$|1-c| \le 1/2$，必有 $c = 1/2$，故唯一。离散平方和和连续最大误差是两个不同目标，换多项式基不会自动把前者变为后者。

2.2 设计矩阵与换基

设计矩阵均为 $A \in \mathbb{R}^{41\times 13}$，$c \in \mathbb{R}^{13}$，$y \in \mathbb{R}^{41}$。目标是 $\min_c \|Ac - y\|_2^2$，展开目标在任意方向 v 上的导数可得最优条件

$$A^{\mathsf{T}}(Ac - y) = 0.$$

该条件也充分：若 $A^T r = 0$，则对任意 v，$\|r + Av\|_2^2 = \|r\|_2^2 + \|Av\|_2^2 \ge \|r\|_2^2$。样本互异且41大于12，若 $Ac = 0$，对应 $\mathcal{P}_{12}$ 多项式有41个互异根，只能恒零。基的线性无关性再给 $c = 0$，故满列秩，最优系数唯一。

Chebyshev递推为 $T_0 = 1, T_1 = x, T_{k+1} = 2xT_k - T_{k-1}$。归纳可知 T_k 的次数恰为 k，首项系数非零，因此 $T_0, \ldots, T_{12}$ 也是 $\mathcal{P}_{12}$ 的一组基。存在可逆换基矩阵 S，使 $A_T = A_{\text{power}} S$。两个问题遍历同一多项式空间与同一预测向量集合，精确最优函数一致，但系数坐标不同。正规方程用于推导条件；本实验直接调用最小二乘求解器，并不显式求 $(A^T A)^{-1}$。

2.3 实测

`least_squares_experiment` 固定同一噪声样本、$d = 12$ 和2001点网格，得到：

基	训练残差 $\|Ac - y\|_2$	相对 e^x 的网格最大误差	2范数条件数（选做）
幂基	0.0693262986649065	0.0189213264675918	16674.6857
Chebyshev基	0.0693262986649069	0.0189213264676025	2.45887085

两基独立网格预测最大差约为 1.70×10^{-14}，小于 10^{-9}；精确记录见JSON的 `basis_prediction_max_difference`。该微小差异来自浮点计算，不是两个不同的精确优化问题。附带求解器还用可手算直线数据 $(-1,-1), (0,2), (1,5)$ 核查得到 $2 + 3x$，并拒绝秩亏的二次拟合；这些独立检查有助于发现矩阵构造和秩处理错误。

2.4 辨析

较小的条件数有助于改善该坐标表示下的数值敏感性，但不改变目标的定义。基于Chebyshev基的离散最小二乘不能据此改称连续最优一致逼近。改变次数则改变了搜索空间：高次数可更贴近噪声，而偏离无噪目标。共用数据的附加实验中，次数3、12、30的训练残差约为 $0.07774, 0.06933, 0.04538$，相应独立网格目标误差约为 $0.01154, 0.01892, 28.2171$。该例呈现过拟合风险，不构成每次加次数必然更差的一般定理。

题3：插值积分——误差从余项里积出来（对应 2.3）

3.1 积分的精确基准

$I = 1/3$。由 $\sum_{j=1}^{N-1} j^2 = (N-1)N(2N-1)/6$，

$$T_N = \frac{1}{N}\left[\sum_{j=1}^{N-1}\left(\frac{j}{N}\right)^2 + \frac{1}{2}\right] = \frac{1}{3} + \frac{1}{6N^2}.$$

因此 $T_4 = 11/32 = 0.34375$，$T_8 = 43/128 = 0.3359375$，$I - T_4 = -1/96$，$I - T_8 = -1/384$，绝对误差比恰为 4。这里 $f'' = 2$，通用误差上界 $(b-a)h^2\|f''\|_\infty/12 = 1/(6N^2)$ 正好取等；梯形对凸函数高估积分，符合负的 $I - T_N$。程序对 $N = 4, 8, 16, 32, 64$ 均与该解析式核对。

误差公式的来源就是插值余项积分。在单个子区间 $[x_k, x_{k+1}]$ 上，p_h 是过两端点的一次插值，故

$$I_k - T_{k,k+1} = \int_{x_k}^{x_{k+1}} \frac{f''(\xi_x)}{2}(x - x_k)(x - x_{k+1})\,dx.$$

几何因子与 h 无关地只依赖区间长度：

$$\int_{x_k}^{x_{k+1}} (x - x_k)(x - x_{k+1})\,dx = -\frac{h^3}{6}.$$

对全部子区间求和并用加权中值定理，即得 $|I - T_N| \le (b-a)h^2\|f''\|_\infty/12$。

3.2 Simpson：把插值次数提到二次

在 $[u, v]$ 上令 $h = (v-u)/2$、中点 $m = u + h$，取过 u, m, v 三点的二次插值 p_2。作代换 $t = (x-u)/h \in [0, 2]$，三个Lagrange基函数对 t 的积分为

$$\int_0^2 \frac{(t-1)(t-2)}{2}dt = \frac{1}{3}, \qquad \int_0^2 \big(-t(t-2)\big)dt = \frac{4}{3}, \qquad \int_0^2 \frac{t(t-1)}{2}dt = \frac{1}{3}.$$

乘回 $dx = h\,dt$，得

$$\int_u^v p_2(x)\,dx = \frac{h}{3}\big[f(u) + 4f(m) + f(v)\big],$$

即 Simpson 公式。它对三次多项式仍精确：取 $u = -h, v = h, m = 0$，$f(x) = x^3$ 时两端值相消、中点值为零，公式给 0，而 $\int_{-h}^{h} x^3 dx = 0$。对 $f(x) = x^4$ 则

$$\int_{-h}^{h} x^4 dx = \frac{2h^5}{5}, \qquad S = \frac{h}{3}\big(h^4 + 0 + h^4\big) = \frac{2h^5}{3},$$

二者不等，故代数精度恰为 3。误差主项正比于 $f^{(4)}$，复合后为 $O(h^4)$。

选做：对 x^2 用 $N = 2$ 的复合 Simpson（即单区间一次），$S = \frac{1/2}{3}(0 + 4\cdot\frac{1}{4} + 1) = \frac{1}{3}$，误差恰为零——这正是"对三次以下精确"的一个实例；而同一被积函数用 T_4 的误差是 $1/96 \approx 1.04 \times 10^{-2}$。四块梯形比不过一次 Simpson，差别不在运算量，而在插值次数。

3.3 光滑性失效：$\sqrt{x}$ 的实际阶

对 $f(x) = \sqrt{x}$、$[0, 1]$，取第一段 $[0, h]$：

$$T = \frac{h}{2}\big(\sqrt{0} + \sqrt{h}\big) = \frac{h^{3/2}}{2}, \qquad \int_0^h \sqrt{x}\,dx = \frac{2}{3}h^{3/2},$$

故

$$I - T_N \ge \frac{2}{3}h^{3/2} - \frac{h^{3/2}}{2} = \frac{h^{3/2}}{6}.$$

若存在与 N 无关的 C 使 $|I - T_N| \le Ch^2$，则需 $C \ge \frac{1}{6\sqrt{h}}$，令 $h \to 0$ 得矛盾。所以实际阶是 $h^{3/2}$，N 加倍时误差比趋向

$$2^{-3/2} = \frac{1}{2\sqrt{2}} \approx 0.3536,$$

而不是 0.25。原因不在常数，而在阶本身变了：$f''=-\frac{1}{4}x^{-3/2}$ 在 0 附近无界，定理 2.3.2 的 C^2 条件不成立。

3.4 辨析

"误差比 0.25"是 $f\in C^2$ 且等距分划下的结论，它依赖光滑性；光滑性不足时阶下降，比值随之改变（$\sqrt{x}$ 给 0.354）。另一方面，即使阶不变，常数也可能随"尖点与节点的对齐方式"跳动，使比值不稳定，这属于可预测性问题而非阶的问题。"对一次函数精确"只说明误差公式的首项在 $f''=0$ 时消失，并不给出误差大小的上界；误差界仍由 $\|f''\|_\infty$ 与 h^2 共同决定，f'' 很大时误差照样很大。

题4：插值微分——截断、舍入与自动微分（对应 2.4）

4.1 计算图上的前向与反向

前向模式逐节点传播一对数 $(v_i,\dot v_i)$：

节点	值	切向导数
v_0	x	1
v_1	x^2	$2x$
v_2	$\sin(x^2)$	$2x\cos(x^2)$
v_3	e^x	e^x
v_4	$\sin(x^2)+e^x$	$2x\cos(x^2)+e^x$

反向模式先保存这些值，从 $\bar v_4=\partial v_4/\partial v_4=1$ 开始。加法给 $\bar v_2=1,\bar v_3=1$，正弦给 $\bar v_1=\cos v_1$。指数分支向输入贡献 e^{v_0}，平方节点 v_0v_0 有两个输入边，各贡献 $v_0\bar v_1$，所以

$$\bar v_0=e^x+2x\cos(x^2)=F'(x).$$

遗漏任何一条分支或把累加改为覆盖都会得到错误答案。

4.2 差分公式就是插值导数

前向差分来自过 $(x,f(x))$ 与 $(x+h,f(x+h))$ 的一次插值：$p_1(s)=f(x)+\frac{f(x+h)-f(x)}{h}(s-x)$，故 $p_1'(x)=\frac{f(x+h)-f(x)}{h}$，正是前向差分。其截断误差由插值余项求导给出

$$D_h^+F-F'(x)=\frac{h}{2}F''(\xi),\qquad \xi\in(x,x+h),$$

是一阶的。

中心差分来自过 $x-h,x,x+h$ 三点的二次插值，在 x 处求导得 $\frac{f(x+h)-f(x-h)}{2h}$（二次项因对称而消去）。相应截断误差为

$$D_h^0F-F'(x)=-\frac{h^2}{6}F'''(\xi),\qquad \xi\in(x-h,x+h),$$

是二阶的。两式的共同点是：差分的截断误差可以从插值余项直接读出，插值几次，导数就精确到几阶。

4.3 实测与误差图

在 $x = 0.7$，独立解析参考为

$$F'(0.7) = 1.4\cos(0.49) + e^{0.7}.$$

程序 `differentiation_experiment` 的解析参考浮点值为3.2490187095246466，前向 `Dual` 为3.2490187095246466，反向 `Graph` 为3.249018709524647；后两者与参考的绝对差均小于 10^{-12}。具体记录在JSON的 `analytic_d1`、`dual_d1`、`reverse_d1` 字段，不以解析式冒充运行记录。

步长扫描同时暴露第二类误差：相减再除以 h 会放大舍入误差，所以步长极小不保证更准确。该次实测如下：

差分	扫描中最优步长	扫描中最小绝对误差	最小步长 $h = 10^{-16}$ 的绝对误差
前向	5.6234×10^{-9}	6.3995×10^{-9}	3.24901871
中心	5.6234×10^{-6}	4.2252×10^{-11}	1.02857266

这些只是所扫步长与当前平台中的最小点，不是精确最优步长定理。最小步长处，两种误差均远大于各自扫描最小误差。图见差分步长误差。

4.4 辨析

第一处错误：AD沿给定程序的基本运算应用链式法则，不取差分极限，也不搜索步长；它仍有基本函数计算及浮点运算误差。第二处错误：积分实验另有离散化参数 N、积分规则和独立基准 $1/3$，其正确性不能由另一段求导程序的误差推出。

最优步长之比可以定量说出。把截断界与舍入界相加并对 h 求极小：前向得 $h_* \sim \sqrt{\varepsilon}$，中心得 $h_* \sim \varepsilon^{1/3}$，二者之比

$$\frac{\varepsilon^{1/3}}{\varepsilon^{1/2}} = \varepsilon^{-1/6} \approx (2.2 \times 10^{-16})^{-1/6} \approx 10^{2.7},$$

与实测的 5.6234×10^{-6} 对 5.6234×10^{-9}（正好 10^3）量级相符。

题5：网络的输入导数、参数导数与光滑性（对应 2.5）

5.1 完整导数公式与手算

令 $s_j = 1 - t_j^2$，由 $(\tanh z)' = 1 - \tanh^2 z$，

$$u'_\theta(x) = \sum_j a_j w_j s_j, \qquad u''_\theta(x) = -2\sum_j a_j w_j^2 t_j s_j.$$

对输入求导时参数固定；对参数求偏导时 x 和其余参数固定：

$$\partial_{a_j} u = t_j, \quad \partial_{w_j} u = a_j s_j x, \quad \partial_{b_j} u = a_j s_j, \quad \partial_c u = 1.$$

记 $r_i = u_\theta(x_i) - y_i$，任一参数分量 α 的损失导数为

$$\partial_\alpha L = \frac{2}{m}\sum_{i=1}^{m} r_i \partial_\alpha u_\theta(x_i).$$

它是参数向量的梯度分量，不是输入方向的 u'。在指定单单元样例 $x=0$ 时，$t=0, s=1$，所以 $u=1, u'=6, u''=0$。单点损失为1，按 (a,w,b,c) 顺序，$\nabla_\theta L=(0,0,4,2)$。注意 $\partial_w u=0$ 与 $u'=6$ 可以同时成立，两者求导对象不同。

5.2 保存参数的实际评估

实验采用float64、种子2026、12个隐单元（37个参数），对81个等距训练点的函数值均方损失训练全部参数。训练设置及保存参数见JSON和 `train_network`；具体优化器更新仅用于复现，题目不要求提前掌握优化器理论。该次最终训练均方损失约为 1.05664×10^{-6}，训练点最大函数误差约为 0.00290832。在与训练点分离的2001点偏移网格上，观测如下：

量	独立解析参考	网格最大误差
函数值	$\sin(\pi x)$	0.00286475788
一阶输入导数	$\pi\cos(\pi x)$	0.117168496
二阶输入导数	$-\pi^2\sin(\pi x)$	1.64694191

`Jet2` 与第1问显式公式在函数值、一阶、二阶输出上的最大差分别约为 $1.00\times10^{-15}, 1.78\times10^{-15}, 4.00\times10^{-15}$。这核查的是对同一个网络的导数计算，不说明网络已逼近目标的导数。反向参数梯度还与显式公式比较，并用随机单位参数方向的中心差分核查：$h=10^{-5}$ 时绝对差约 6.06×10^{-12}。三个层次必须分开：网络内部导数实现正确、函数值拟合较好、目标导数逼近较好。

5.3 连续范数反例

$$g_k=\frac{\sin(kx)}{k^2},\qquad g_k'=\frac{\cos(kx)}{k},\qquad g_k''=-\sin(kx).$$

因 $|\sin|,|\cos|\le1$，有 $\|g_k\|_\infty\le1/k^2$、$\|g_k'\|_\infty\le1/k$、$\|g_k''\|_\infty\le1$。对整数 $k\ge2$，$x=\pi/(2k)\in[-1,1]$ 使 $\sin(kx)=1$，$x=0$ 使 $\cos(kx)=1$，故三个上界均达到。因此 $k=100$ 时精确范数依次为 $10^{-4}, 10^{-2}, 1$。函数值乃至一阶导数一致趋零，仍不保证二阶导数一致趋零；该反例否定的是缺少额外条件的一般推断，不声称每个训练网络一定有大导数误差，也不是训练结果。

5.4 ReLU与残差

对 $x<0$，$\rho'=0$；对 $x>0$，$\rho'=1$；在0左右导数不相等，经典一阶导数不存在。因此二阶经典导数在非零点为0，在0不存在。有限个一维ReLU单元之和是连续分段仿射函数，在折点之间二阶导数为0；发生真实折角的点处通常不可微。即使若干折角抵消后函数更光滑，其分段仿射结构也不能精确表示 $\sin(\pi x)$ 这样在开区间内具有非零连续二阶导数的函数。框架在折点约定返回某个数，不会使经典导数存在；这里不讨论分布导数或弱形式。

残差 $r_\theta=-u_\theta''-\pi^2\sin(\pi x)$ 需要在固定参数下对输入求二阶导数。若以残差及边界误差构造损失，训练还要对参数求梯度，这与输入导数承担不同任务。$\sin(\pi x)$ 满足 $-u''=\pi^2\sin(\pi x)$、$u(-1)=u(1)=0$，但本章实际训练只最小化函数值数据损失，并未训练该残差与边界条件。因此它是光滑网络拟合与求导演示，不能称为已经求解此边值问题。

核验状态

手算结论可按上述代数、Rolle定理和链式法则逐步复核。实测值依据本章保存JSON及相应程序，采用的容差用于发现实现错误，不是一般数学误差界。正式B组核验、C组复核及人工版本签认：待相应人员实际执行后另行记录；未填写姓名、贡献或人工工时。

第三章　离散的世界：微分方程的数值解法

> 编写组：A1（G3）；来源版本：`230873df074c`，main已合并稿。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

从一条折线、一张网格、一顶帽子，到一个向量场

第一节（3.1）　ODE 初值问题：Euler 与 RK4

从 Euler 的折线到 Runge–Kutta 的四段斜率

3.1.0　引子：一条折线与四段斜率

1768 年，Euler 在圣彼得堡出版《积分学原理》第一卷[1]。此时他的视力已经很差，书稿多由他口述、助手笔录。书中有一个今天看来极为朴素的构造：要求一条积分曲线，不必先解出它，只要从已知点出发，沿该点的切线方向走一小步；到了新位置，重新看切线方向，再走一小步。如此反复，得到的是一条折线。没有计算机的年代，折线是少数能真正算出来的东西。

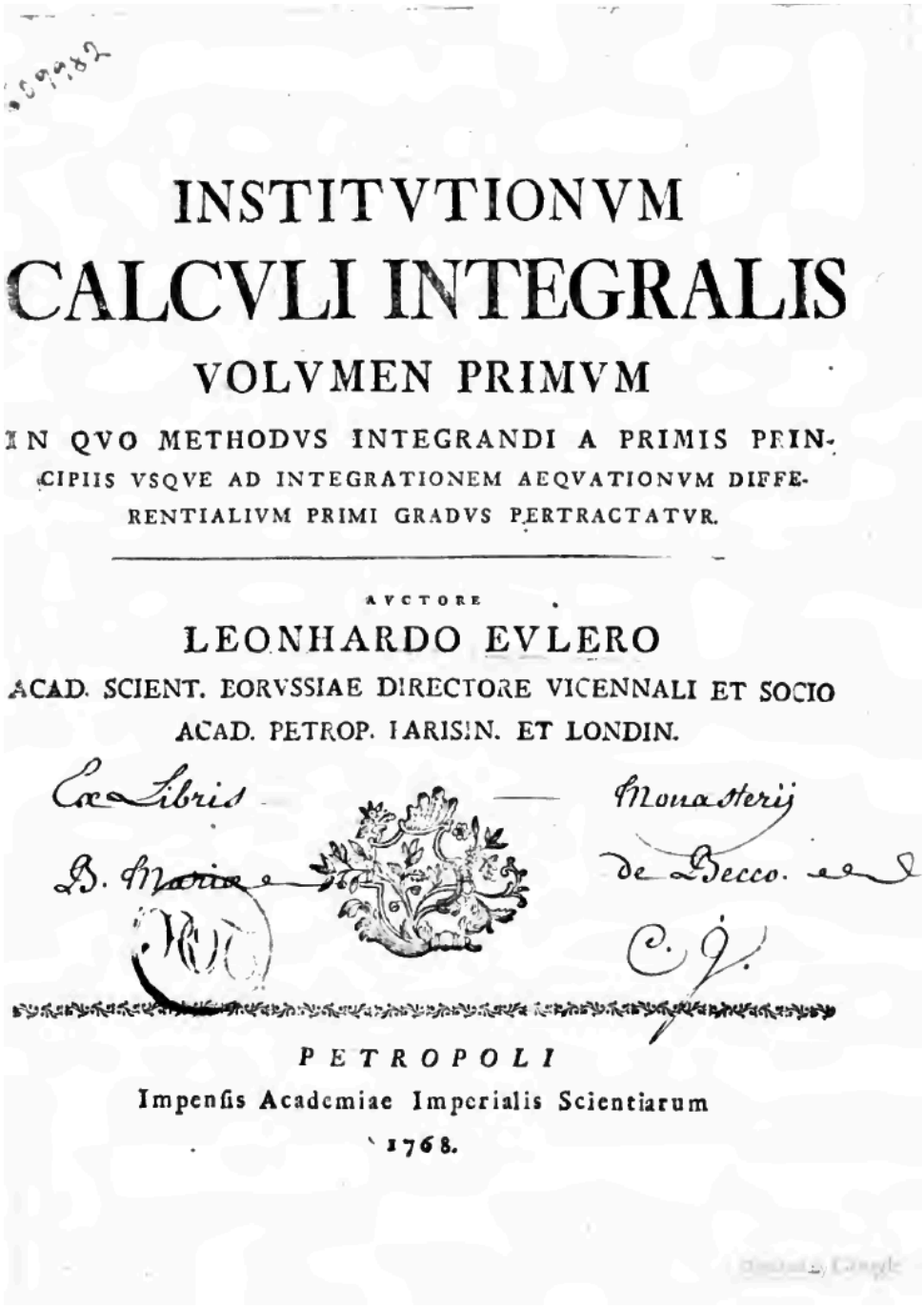


图 1　《积分学原理》第一卷（1768）扉页，Euler 口述、助手笔录。图片来源：Internet Archive

这条折线留下了一个直到二十世纪才被认真回答的问题：步长越来越小，折线是否真的收敛到真解？收敛得多快？要回答它，先得说清楚“解”是什么、在什么条件下解存在且唯一、初值的微小扰动会不会被放大。这些问题构成了 3.1.1 节的内容，也是所有数值方法的共同前提。

一百多年后，Göttingen 的 Runge 在 1895 年的《数学年刊》上换了一个问法[2]：与其把步长缩到无穷小，能否在一步之内多取几个斜率，用它们的加权平均代替切线，从而用更少的步数换到更高的精度？他给出了二阶格式。1900 年，Heun 把“预测—校正”的想法写成一类平均斜率格式[4]。1901 年，Kutta 在《物理学与数学杂志》上给出了四个斜率、四阶精度的格式[3]，也就是今天教科书里的经典 RK4。此后一百多年，RK4 一直是科学计算中最常用的通用积分器之一；Hairer、Nørsett 与 Wanner 的《Solving Ordinary Differential Equations I》系统整理了这一族方法的阶条件与稳定性理论[5]。

图 2　左：Euler（1707–1783）；中：Runge（1856–1927）；右：Kutta（1867–1944）。图片来源：MacTutor History of Mathematics Archive

三个人的问题其实是同一个：怎样用有限个函数值，把一个连续演化过程“压缩”成可以递推的公式。Euler 用一段切线，Runge 用两段斜率的平均，Kutta 用四段斜率的加权平均。本节沿着这条线索展开：3.1.1 节把初值问题与适定性说清楚，3.1.2 节给出显式与隐式 Euler 格式，3.1.3 节构造经典 RK4，3.1.4.1 至 3.1.4.3 节分别讨论误差、稳定性与计算成本，3.1.5 节用一枚拦截弹把前面的公式串起来，3.1.6 节再看这族方法今天在工程上的位置。

在进入公式之前，先请一位向导带我们走两夜雾隐岭。这座山只有一条下山的老路：山顶海拔 1.000 km，山脚 0 km；路面越往下越缓，坡度恰好等于“当前海拔的相反数”，即 $y' = -y$（y 以 km 计，t 以小时计）。山里常年起雾，向导走到哪里，只能摸到脚下那一处的坡度；他每半小时走一步，也就是 $h = 0.5$。

3.1.2.1 节跟向导走第一夜，3.1.3.3 节走第二夜。同一座山、同一场雾、同样的脚力，唯一的差别只有一句话：一步之内，问几次坡度。

3.1.1　初值问题与适定性

3.1.1.1　微分方程与积分曲线

设 $D \subset \mathbb{R}^2$ 是开区域，$f : D \to \mathbb{R}$ 是给定的函数。一阶常微分方程的初值问题是

$$y'(t) = f\big(t, y(t)\big), \qquad y(t_0) = y_0, \qquad t \in I, \qquad (1)$$

其中 I 是含 t_0 的区间，$(t_0, y_0) \in D$。

定义 1（经典解）　若 $y \in C^1(I)$，对一切 $t \in I$ 有 $\big(t, y(t)\big) \in D$，且逐点满足 (1)，则称 y 是初值问题 (1) 在 I 上的经典解，简称解。

例 1　$f(t, y) = y$，$(t_0, y_0) = (0, 1)$：解为 $y = e^t$，在 $\mathbb{R}$ 上处处有定义且唯一。

例 2　$f(t, y) = \sqrt{|y|}$，$(t_0, y_0) = (0, 0)$：右端连续，但 $y \equiv 0$ 与 $y = t^2/4$（$t \ge 0$）都是解，两条积分曲线都从原点出发。

几何上，(1) 在 D 上给出一个方向场：每一点 (t, y) 画一条斜率为 $f(t, y)$ 的短线段；解是一条处处与方向场相切的曲线，称为积分曲线。初值条件指定这条曲线经过 (t_0, y_0)。数值方法要做的，是不去找这条曲线，而用一串短直线段把它“折”出来。

3.1.1.2 Lipschitz 条件

例 2 说明：仅要求 f 连续，不足以保证解唯一。症结在于 f 在 $y=0$ 附近变化得太陡；把“陡”量化，就得到下面的条件。

定义 2（Lipschitz 条件） 若存在常数 $L>0$，使得

$$|f(t,u)-f(t,v)| \le L\,|u-v|, \qquad (t,u),(t,v)\in D, \qquad (2)$$

则称 f 在 D 上关于 y 满足 Lipschitz 条件， L 称为 Lipschitz 常数。

命题 1 设 D 是凸区域， f 在 D 上连续可微，且存在常数 M 使 $|\partial f/\partial y|\le M$。则 f 在 D 上关于 y 满足 Lipschitz 条件，常数可取 $L=M$。

证明 固定 $(t,u),(t,v)\in D$。由 D 的凸性，线段 $(t, v+s(u-v))$， $0\le s\le 1$ 含于 D。令 $g(s)=f(t,v+s(u-v))$，则 g 在 $[0,1]$ 上可微，且 $g'(s)=\partial_y f(t,v+s(u-v))(u-v)$。由微分中值定理，存在 $\xi\in(0,1)$ 使

$$|f(t,u)-f(t,v)| \le M\,|u-v|,$$

取绝对值并用 $|\partial_y f|\le M$，即得 (2)。证毕。

三个常见例子。

（a） $f(t,y)=-y$： $|f(t,u)-f(t,v)|=|u-v|$，故 $L=1$，且是全局 Lipschitz。

（b） $f(t,y)=t^2+y^2$：在带形 $|y|\le R$ 上 $|\partial f/\partial y|=2|y|\le 2R$，故在该带形上 $L=2R$。这是局部 Lipschitz： R 越大 L 越大，不存在对全平面成立的有限 L。

（c） $f(t,y)=\sqrt{|y|}$：取 $v=0$、 $u>0$，差商 $\sqrt{u}/u=1/\sqrt{u}\to+\infty$，故在含 $y=0$ 的任何区域上都不满足 Lipschitz 条件。这与例 2 中解不唯一是同一件事的两面。

3.1.1.3 存在唯一性与连续依赖

把微分方程换成积分方程，是处理存在性的标准做法。在连续函数类中，

$$y(t)=y_0+\int_{t_0}^{t} f\big(s,y(s)\big)\,\mathrm{d}s$$

与 (1) 等价。于是求 (1) 的解，等于求这个积分方程的不动点。

定理 1（Picard–Lindelöf） 设 $R=\{(t,y):|t-t_0|\le a,\ |y-y_0|\le b\}\subset D$， f 在 R 上连续，$|f|\le M$，且关于 y 满足 Lipschitz 条件，常数为 L。令 $\delta=\min\{a,\,b/M\}$，则初值问题 (1) 在 $I=[t_0-\delta,\,t_0+\delta]$ 上存在唯一解。

证明 取 $y^{(0)}\equiv y_0$，递推定义 Picard 迭代

$$y^{(m+1)}(t)=y_0+\int_{t_0}^{t} f\big(s,y^{(m)}(s)\big)\,\mathrm{d}s, \qquad m=0,1,2,\dots$$

由归纳法可证每个 $y^{(m)}$ 连续且不越出 R，并有估计

$$\big\|y^{(m+1)}-y^{(m)}\big\|_{\infty}\le \frac{ML^m}{(m+1)!}\,\delta^{m+1}.$$

右端是收敛级数的通项，故 $\{y^{(m)}\}$ 在 $C(I)$ 的上确界范数下是 Cauchy 列；由 $C(I)$ 的完备性，它一致收敛到某个连续函数 y。在迭代式中令 $m\to\infty$，即知 y 满足积分方程，从而是 (1) 的解。

唯一性：设 y, z 都是解，令 $u = |y - z|$。两式相减并用 Lipschitz 条件，得 $u(t) \le L\left|\int_{t_0}^{t} u(s)\,\mathrm{d}s\right|$；由引理 1（Gronwall 不等式）的 $\alpha = 0$ 情形（或直接对 $e^{-Lt}v$ 求导）得 $u \equiv 0$，故 $y \equiv z$。证毕。完整证明见 [6][7]。

存在唯一性回答“解有没有”；适定性还要回答“解稳不稳”：初值差一点，解会不会差很多。下面的不等式是估计误差的基本工具。

引理 1（Gronwall 不等式） 设 $u, \beta \ge 0$ 在 $[t_0, t_1]$ 上连续，$\alpha \ge 0$，且

$$u(t) \le \alpha + \int_{t_0}^{t} \beta(s)\,u(s)\,\mathrm{d}s, \qquad t \in [t_0, t_1]. \tag{3}$$

则

$$u(t) \le \alpha \exp\!\left(\int_{t_0}^{t} \beta(s)\,\mathrm{d}s\right), \qquad t \in [t_0, t_1]. \tag{4}$$

证明 令 $U = \alpha + \int \beta u$，则 $u \le U$，$U' = \beta u \le \beta U$，故 $(e^{-\int \beta} U)' \le 0$，在 t_0 处取 $U(t_0) = \alpha$ 即得。$\alpha = 0$ 时对 $\alpha = \varepsilon \downarrow 0$ 取极限。

定理 2（对初值的连续依赖） 设 y, z 分别是初值 y_0, z_0 的解，都在 I 上定义，f 的 Lipschitz 常数为 L。则

$$|y(t) - z(t)| \le |y_0 - z_0|\, e^{L|t - t_0|}, \qquad t \in I. \tag{5}$$

证明 两解相减，取绝对值，用 Lipschitz 条件得 $u(t) \le |y_0 - z_0| + L\int_{t_0}^{t} u(s)\mathrm{d}s$，其中 $u = |y - z|$；再由 (3)(4) 即得 (5)。

3.1.1.4 单步法框架与相容性

Euler、Heun 与 RK4 都是同一个框架的特例，先把框架抽象出来。

定义 3（单步法） 若格式可以写成

$$y_{n+1} = y_n + h\,\Phi(t_n, y_n, h), \tag{6}$$

其中 Φ 是给定函数，称为增量函数，则称该格式为单步法。若 Φ 不含 y_{n+1}，格式是显式的；若含 y_{n+1}，格式是隐式的，每一步需要解一个方程。

显式 Euler 的增量函数是 $\Phi(t, y, h) = f(t, y)$。后向 Euler 格式为 $y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$，其 Φ 通过 y_{n+1} 隐式定义，属于隐式单步法。Heun 格式（改进 Euler）先试走一步，再取两端斜率的平均：

$$\Phi(t, y, h) = \frac{1}{2}\Big[f(t, y) + f\big(t + h,\; y + h f(t, y)\big)\Big].$$

定义 4（相容性） 称单步法 (6) 与方程 (1) 相容，若

$$\Phi(t, y, 0) = f(t, y), \qquad (t, y) \in D. \tag{7}$$

步长趋于零时，格式的一步增量应与方程的一阶增量一致；显式 Euler 与 Heun 都满足 (7)。

3.1.2 显式与隐式 Euler 格式

3.1.2.1 显式 Euler 格式

(1) 说的是每一点的导数由 f 给出。最直接的离散方式，是把导数换成前向差商。把区间 $[t_0, T]$ 等分为 M 份，步长 $h = (T - t_0)/M$，节点 $t_n = t_0 + nh$。在 t_n 处

$$y'(t_n) \approx \frac{y(t_{n+1}) - y(t_n)}{h}.$$

把它代入 $y' = f(t, y)$，并把精确值换成数值近似，得

$$y_{n+1} = y_n + h\, f(t_n, y_n), \qquad n = 0, 1, \ldots, M-1. \tag{8}$$

同一个格式也可以从积分方程得到：在 $[t_n, t_{n+1}]$ 上 $y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(s, y(s))\,\mathrm{d}s$，用左端点的值乘区间长近似积分，同样得到 (8)。差商推导强调“用切线代替曲线”，矩形公式强调“用左端点值代替整段积分”，舍去的是同一个高阶项。

故事 | 雾隐岭的第一夜。　向导凌晨出发，站在海拔 $y_0 = 1.000$ km 的起点，摸到的坡度是 -1.000。按 Euler 的办法，他只问这一次：半小时后海拔下降 $0.5 \times 1 = 0.500$ km，于是他在本子上写下“从海拔 0.500 km 继续走”。

可这条路的真实位置是 $e^{-0.5} = 0.607$ km。他一步多下了 0.107 km——106 米，差不多是三十层楼。更糟的是，从这一刻起，他手里的“当前海拔”已经偏低，下一次摸到的坡度也跟着偏缓（ -0.500 而不是 -0.607），误差就这样一步一步累积下去。

错在哪里？不在脚力，也不在算术：他只是把全程最陡的那一处坡度，当成了接下来半小时的平均坡度。而这条路越往下越缓，起点的坡度恰恰是最陡的。“只问一次”在起点附近没问题，走得越远越离谱。第二夜换一种问法，见 3.1.3.3 节。

科普 | Euler 格式在今天。　它仍然是最常用的方法之一，只是用在“便宜”比“精确”更要紧的地方。手机游戏里，物体每画一帧（约 $1/60$ 秒）更新一次位置与速度，用的就是这种一路向前的显式推进：每步只问一次右端函数，算得快、写得短。工程师通常还会加一个小修正（半隐式 Euler——先用旧斜率更新速度，再用新速度更新位置），以免能量越算越多。

历史上最有名的一次 Euler 式尝试发生在 1922 年。英国气象学家 Richardson 把大气方程离散[12]，用手摇计算器整整算了六个星期，才得到一份六小时的天气预报；结果地面气压的变化大得不合物理，同行把这份结果当成笑话。事后的分析指出两个原因：初值观测太稀、噪声太大；时间步长相对网格又取得太大，误差被迅速放大。今天看，这两条都指向同一件事——显式推进的步长必须受限制。数值天气预报后来靠计算机和更稳的格式才走通，但那六个星期的手算，是这门学科真正的起点。

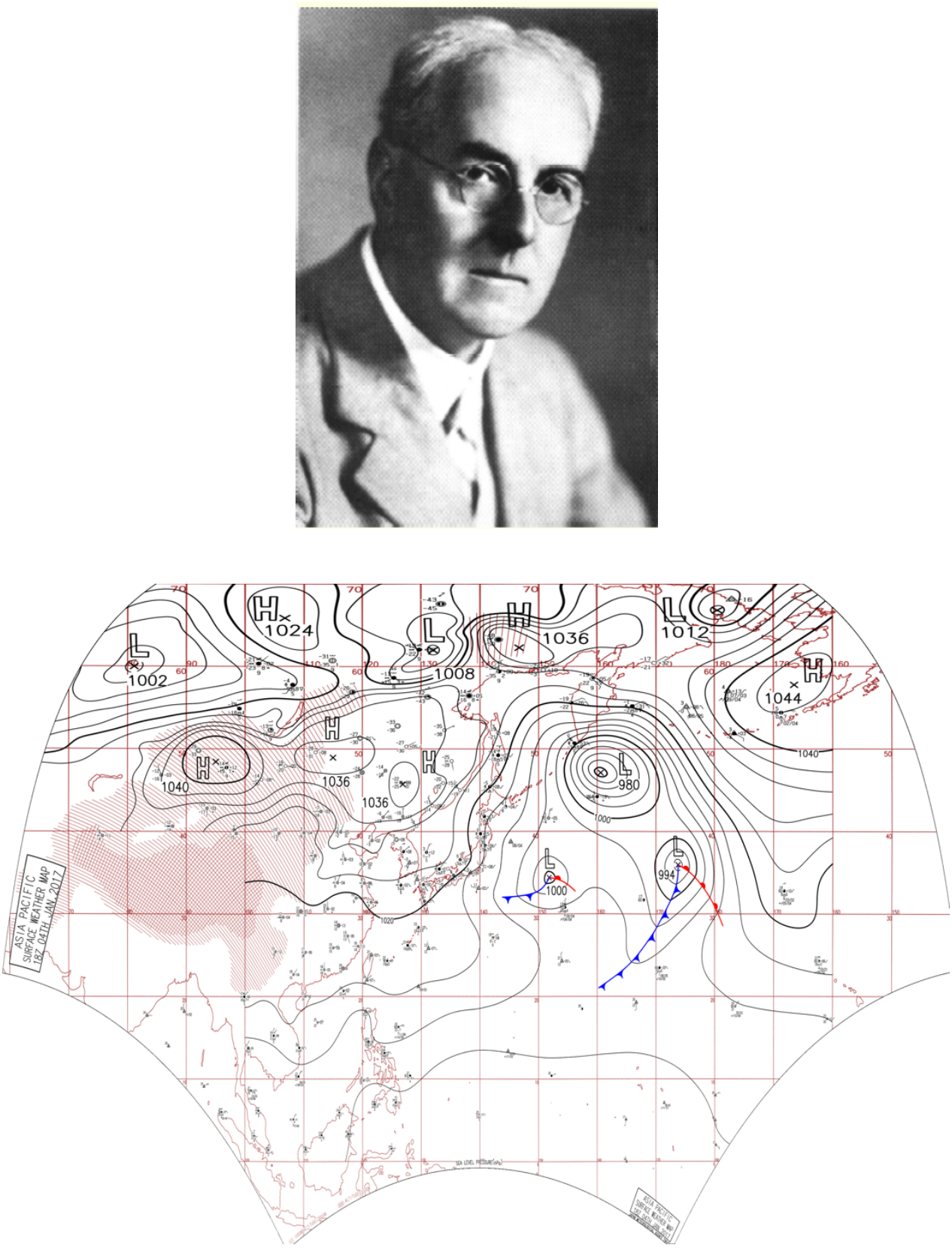


图 3　左：Richardson（1881－1953），1922 年用手摇计算器做出第一次数值天气预报；右：今天的一张地面天气分析图（日本气象厅，CC BY 4.0，via Wikimedia Commons）

3.1.2.2 隐式 Euler 格式

显式 Euler 用左端点的斜率；若改用右端点，就得到隐式 Euler 格式（后向 Euler）——相当于先猜终点、再回头对账，猜得不对就迭代。格式为

$$y_{n+1} = y_n + h\, f(t_{n+1}, y_{n+1}).$$

右端含 y_{n+1}，每一步都要解一个方程：线性问题解线性方程组，非线性问题用 Newton 迭代。它的局部截断误差同样是 $O(h^2)$（定义见 3.1.2.3 节），一阶精度并不比显式格式高，但稳定性好得多。对测试方程 $y' = \lambda y$，放大因子为

$$R(z) = \frac{1}{1 - z}, \quad z = h\lambda.$$

当 $\mathrm{Re}\lambda < 0$ 时对任意 $h > 0$ 都有 $|R(z)| < 1$，即无条件稳定；代价是每步的隐式求解。

梯形格式：

$$y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_{n+1}, y_{n+1})].$$

把两端斜率平均，是二阶隐式格式。显式与隐式的取舍，本质上是每步成本与稳定性之间的取舍。

3.1.2.3 几何意义与局部误差

把 (8) 改写成积分形式：$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\,\mathrm{d}t$。Euler 格式相当于用一块矩形代替这段积分——矩形的宽是 h，高取哪一端的函数值，就决定了是显式还是隐式。图 4 把两种取法并排画出。

左图是显式 Euler：每块矩形的高取左端点的 $f(t_i, x(t_i))$，所以曲线上升时矩形面积偏小，算出的下一步偏低；右图是隐式 Euler：高取右端点的 $f(t_{i+1}, x(t_{i+1}))$，矩形面积偏大，算出的下一步偏高。两者都是一阶格式，区别只在“用哪一端的高度”。

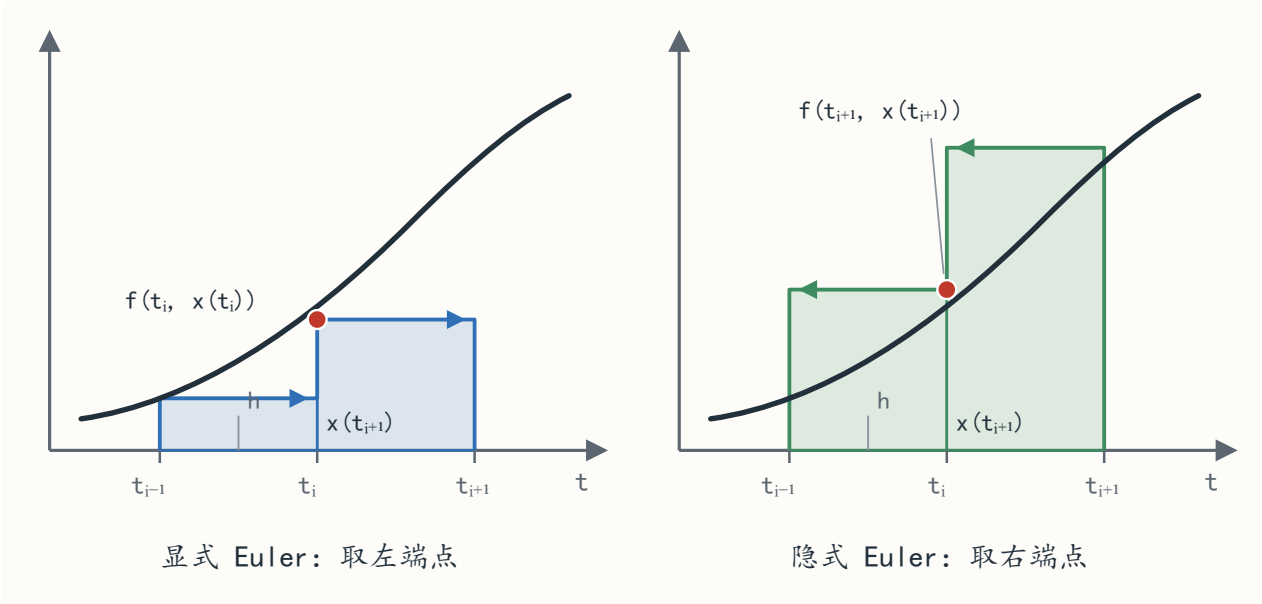


图 4　两种 Euler 的几何差别：左图矩形取左端点的高度（显式），右图取右端点的高度（隐式）；同一段曲线，一块偏矮、一块偏高

矩形面积与真实积分之差，就是局部截断误差的来源：每块矩形都少算或多算了一个小曲边三角形，其高约为 $\frac{h}{2}|y''|$，底为 h，面积是 $O(h^2)$。步长减半，这块三角形缩到四分之一；但步数翻倍，所以走到终点时，误差只按 h 缩小——这就是“局部二阶、全局一阶”的几何来源。

把精确解在 t_n 处 Taylor 展开，

$$y(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)) + \frac{h^2}{2}y''(\xi_n),$$

与 (8) 相减，得 Euler 的局部截断误差

$$\tau_{n+1} = \frac{h^2}{2}y''(\xi_n) = O(h^2). \qquad (9)$$

3.1.3 经典 Runge–Kutta 方法

3.1.3.1 基本思想与一般形式

一步推进可以写成积分形式

$$y(t_n + h) = y(t_n) + \int_{t_n}^{t_n+h} f\big(t, y(t)\big)\,\mathrm{d}t.$$

Euler 格式相当于用左端点值乘区间长度近似这个积分；若在区间内部若干点取样、再按权重组合，就能在不计算高阶导数的前提下提高精度。这是 Runge‑Kutta 方法的基本思想：用若干个斜率 k_i 的加权平均代替单点斜率。

最简单的两步取样是 Heun 格式：先用 Euler 试走一步得到预测值 $\tilde{y}_{n+1} = y_n + hf(t_n, y_n)$，再用两端斜率的平均校正：

$$y_{n+1} = y_n + \frac{h}{2}\Big[f(t_n, y_n) + f\big(t_n + h, \tilde{y}_{n+1}\big)\Big]. \qquad (10)$$

把 (10) 在 $h = 0$ 处展开并与精确解比较，局部误差从 $O(h^2)$ 降到 $O(h^3)$，即格式是二阶的。

一般的显式 s 级 Runge‑Kutta 格式写成

$$\begin{aligned} k_i &= f\Big(t_n + c_i h,\; y_n + h\sum_{j=1}^{i-1} a_{ij}k_j\Big), \qquad i = 1, \dots, s, \\ y_{n+1} &= y_n + h\sum_{i=1}^{s} b_i k_i. \end{aligned} \qquad (11)$$

式 (11) 中的 c_i 为节点系数，a_{ij} 为级间耦合系数，b_i 为组合权重。系数一旦取定，方法就完全确定，可以用 Butcher 表紧凑地写出。

3.1.3.2 经典 RK4：格式与推导

经典四阶 Runge‑Kutta 取 $s = 4$，在步内四个位置各取一次斜率：

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f\big(t_n + h/2,\; y_n + hk_1/2\big), \\ k_3 &= f\big(t_n + h/2,\; y_n + hk_2/2\big), \\ k_4 &= f\big(t_n + h,\; y_n + hk_3\big), \end{aligned} \qquad (12)$$

再以权重 $1, 2, 2, 1$ 加权平均：

$$y_{n+1} = y_n + \frac{h}{6}\big(k_1 + 2k_2 + 2k_3 + k_4\big). \qquad (13)$$

对应的 Butcher 表为

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

每一步需要四次右端函数求值，记 $n_f = 4M$。四个斜率的取法不是唯一的：只要系数满足下面列出的阶条件，就能得到四阶格式。

把 (13) 与精确解在 t_n 处的 Taylor 展开逐项比较。记 $F = f(t_n, y_n)$，沿解曲线取全导数，则

$$y(t_n + h) = y_n + hF + \frac{h^2}{2}\left(f_t + f_y F\right) + O(h^3).$$

把 (12) 的各级斜率在 (t_n, y_n) 处展开，代入 (13)，按 h 的幂次合并，得

$$\begin{aligned} y_{n+1} = y_n &+ hF\Big(\sum_i b_i\Big) + \frac{h^2}{2}\Big(\sum_i b_i c_i\Big)\left(f_t + f_y F\right) \\ &+ \frac{h^3}{6}\Big(\sum_i b_i c_i^2\Big)\left(f_{tt} + 2 f_{ty} F + f_{yy} F^2\right) + \frac{h^3}{6}\Big(\sum_{i>j} b_i a_{ij} c_j\Big) f_y\left(f_t + f_y F\right) + O(h^4). \end{aligned}$$

令同次幂的系数逐个相等，得 8 条阶条件：

$$\begin{aligned} &\sum_i b_i = 1, \qquad \sum_i b_i c_i = \frac{1}{2}, \qquad \sum_i b_i c_i^2 = \frac{1}{3}, \\ &\sum_{i>j} b_i a_{ij} c_j = \frac{1}{6}, \qquad \sum_i b_i c_i^3 = \frac{1}{4}, \qquad \sum_{i>j} b_i c_i a_{ij} c_j = \frac{1}{8}, \\ &\sum_{i>j} b_i a_{ij} c_j^2 = \frac{1}{12}, \qquad \sum_{i>j>k} b_i a_{ij} a_{jk} c_k = \frac{1}{24}. \end{aligned}$$

这 8 个方程不足以唯一确定 10 个未知系数，故四阶显式方法不唯一[5][10]。把 (12) 与 (13) 的节点、权重代入，前两条为 $\sum_i b_i = 1$、$\sum_i b_i c_i = \frac{1}{2}$，其余各条同样成立，于是 y_{n+1} 与真解的展开在 h^4 项以内完全一致。经典 RK4 的局部截断误差为

$$\tau_{n+1} = O(h^5). \qquad (14)$$

四个阶段是达到四阶精度的最低要求：s 级显式 Runge–Kutta 的最高阶为 s（$s \le 4$），再高的阶需要更多级。这也是 RK4 在精度与每步成本之间取得平衡的原因。

3.1.3.3 几何意义与权重来源

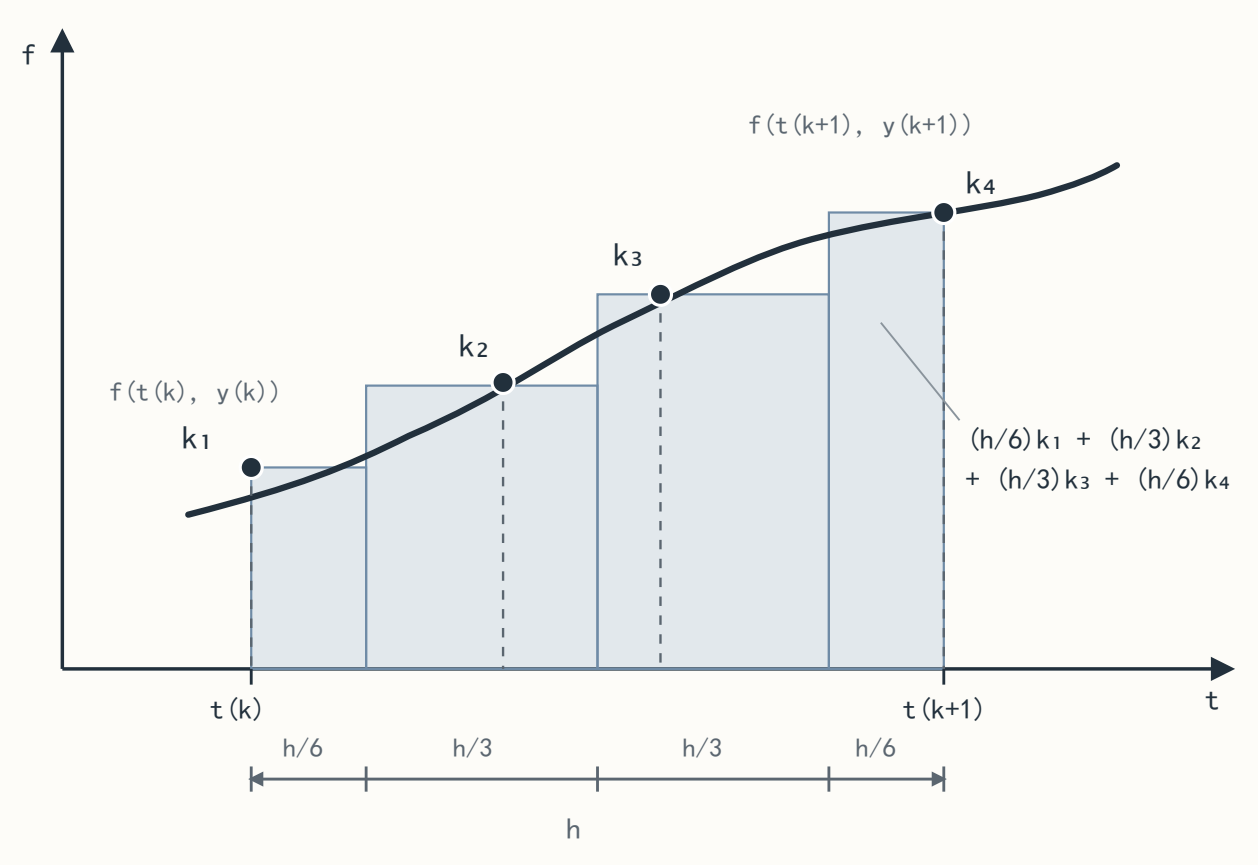


图 5　RK4 的积分账本：一步的积分被切成四块，宽依次是 h/6、h/3、h/3、h/6，高分别是四个斜率 k_1–k_4；四块面积相加，就是这一步的增量

把 (13) 按积分写开：

$$y_{n+1} = y_n + \frac{h}{6}k_1 + \frac{h}{3}k_2 + \frac{h}{3}k_3 + \frac{h}{6}k_4.$$

图 5 把右边的和画成四块矩形：宽依次是 $h/6$、$h/3$、$h/3$、$h/6$，高分别是四个斜率。四块面积之和，就是用四个采样值对 $\int_{t_n}^{t_{n+1}} f(t,y(t))\,dt$ 的近似——RK4 的“四阶”，说的是这四块面积把积分算得足够准。

为什么是这四个宽度？把一步的积分用 Simpson 公式近似，需要在左端点、中点、右端点取值，权重比为 $1:4:1$；中点被第 2、3 级各取一次，4 便拆成 $2+2$，这就是 $1:2:2:1$ 的由来；再统一乘以 $h/6$，就得到 $h/6, h/3, h/3, h/6$ 这四个宽度。节点 $0, \frac{1}{2}, \frac{1}{2}, 1$ 也由这三处取样位置决定[11]。

拿 $y' = -y$、$y(0)=1$、$h=0.5$ 验算一遍：四个斜率依次是 $k_1=-1$、$k_2=-0.75$、$k_3=-0.8125$、$k_4=-0.5938$，加权平均

$$\bar{k} = \frac{1}{6}\big(-1 + 2(-0.75) + 2(-0.8125) - 0.59375\big) = -0.78646,$$

于是 $y_1 = 1 + 0.5\times(-0.78646) = 0.6067708$，与真值 $e^{-0.5}=0.6065307$ 相差 2.4×10^{-4}。加权平均落在最陡的 k_1 与最平的 k_4 之间，相当于用一条斜率居中的直线代替整段曲线，其截断误差比单点切线小一个量级以上。

把图 4 与图 5 并排看，区别一目了然：Euler 只用一块矩形、一个高度；RK4 用四块矩形、四个高度，且中间两块各占 $h/3$——多出的三次求值，买的就是这四个宽度上的“分摊”。

四阶之所以常用，是因为在这一阶数上阶条件仍可手工核验；继续提高阶数需要增加级数，阶条件迅速增多，精度收益与公式复杂度不再同步改善。

故事 | 雾隐岭的第二夜。　第二天夜里，还是那座山、那场雾，向导带了三个侦察兵。规矩定得很死：每走一步，四条坡度按 $1:2:2:1$ 加权平均，只走一步。

出发前，他自己先摸脚下的坡度：$k_1=-1.0000$。然后派第一个侦察兵“按起点坡度往下试跑半小时”，到中点 $(0.25,\,0.75)$ 再摸一次：$k_2=-0.7500$。他嫌这个位置是“猜”出来的，又派第二个侦察兵从中点出发，改用 k_2 重新推算中点 $(0.25,\,0.8125)$，摸到 $k_3=-0.8125$。最后派第三个侦察兵沿 k_3 一路试跑到终点 $(0.50,\,0.59375)$，摸到 $k_4=-0.5938$。

四条坡度加权：$[-1.0000+2\times(-0.7500)+2\times(-0.8125)+(-0.5938)]/6=-0.78646$。一步之后他报出的海拔是 $1+0.5\times(-0.78646)=0.6068$ km；真实位置 0.6065 km，差 2.4×10^{-4} km，也就是 24 厘米。

同一座山、同一段路：第一夜错 106 米，第二夜错 24 厘米。代价只有一个——这一步里多问了三处坡度。三个侦察兵的花销，换来的是四百多倍的精度。

3.1.4　数值实验

3.1.4.1　误差与收敛阶

局部截断误差衡量单步离散本身带来的误差：假设第 n 步以精确解 $y(t_n)$ 为输入，一步之后与精确解的偏差。Euler 的局部误差由 (9) 给出，RK4 的局部误差由 (14) 给出。全局误差描述多步迭代后的整体偏差：

$$E_n=y(t_n)-y_n. \tag{15}$$

单步误差会随时间传播并被放大，因此全局误差一般大于局部误差的简单累加。由 (15)，把 Euler 格式与精确解相减，用 Lipschitz 条件估计，再由 Gronwall 不等式 (3)(4)，得

$$|E_n|\le\frac{M_2}{2L}\left(e^{L(T-t_0)}-1\right)h,\qquad M_2=\max|y''|. \tag{16}$$

放大因子仍是 $e^{L(T-t_0)}$：区间越长、L 越大，误差界越松。

这张不等式值得用一张图记住：误差不是攒出来的，是滚出来的。每一步注入一小块局部误差，它随后按 e^{Lt} 复利放大。同一个局部误差 $\varepsilon=10^{-3}$，到 $t=5$ 时，$L=2$ 的累计量是 $L=0$ 的两千多倍（图 7）。

(16) 说明 Euler 的全局误差与 h 的一次方成正比。对 RK4，同样的推导给出

$$\max_{0\le n\le M}|y(t_n)-y_n|=O(h^4). \tag{17}$$

式 (17) 是四阶收敛的定量表述。总步数 $M=O(h^{-1})$ 的累积效应使全局阶比局部阶低一阶：Euler 的局部误差是 $O(h^2)$、全局是 $O(h)$；RK4 的局部误差是 $O(h^5)$、全局是 $O(h^4)$。这一“降一阶”的规律，是单步法误差分析的基本结论[8]。

若误差近似满足 $E(h)\approx Ch^p$，则步长减半后 $E(h/2)\approx C(h/2)^p$，两式相除取对数，得观测收敛阶

$$p_{\text{obs}}=\log_2\frac{E(h)}{E(h/2)}. \tag{18}$$

式 (18) 就是由相邻两次误差估计收敛阶的公式。对模型问题 $y'=-y$、$y(0)=1$、$t\in[0,1]$，精确解为 e^{-t}。取一系列减半的步长，结果列于表 1，误差随步长的变化画在图 6。

表 1　终点误差与观测收敛阶（$y'=-y$，$y(0)=1$，$T=1$）

步长 h	Euler 求值次数	Euler 误差	Euler 观测阶	RK4 求值次数	RK4 误差	RK4 观测阶
1/2	2	1.18×10^{-1}	—	8	2.91×10^{-4}	—
1/4	4	5.15×10^{-2}	1.20	16	1.48×10^{-5}	4.30
1/8	8	2.43×10^{-2}	1.08	32	8.31×10^{-7}	4.15
1/16	16	1.18×10^{-2}	1.04	64	4.93×10^{-8}	4.08
1/32	32	5.82×10^{-3}	1.02	128	3.00×10^{-9}	4.04
1/64	64	2.89×10^{-3}	1.01	256	1.85×10^{-10}	4.02

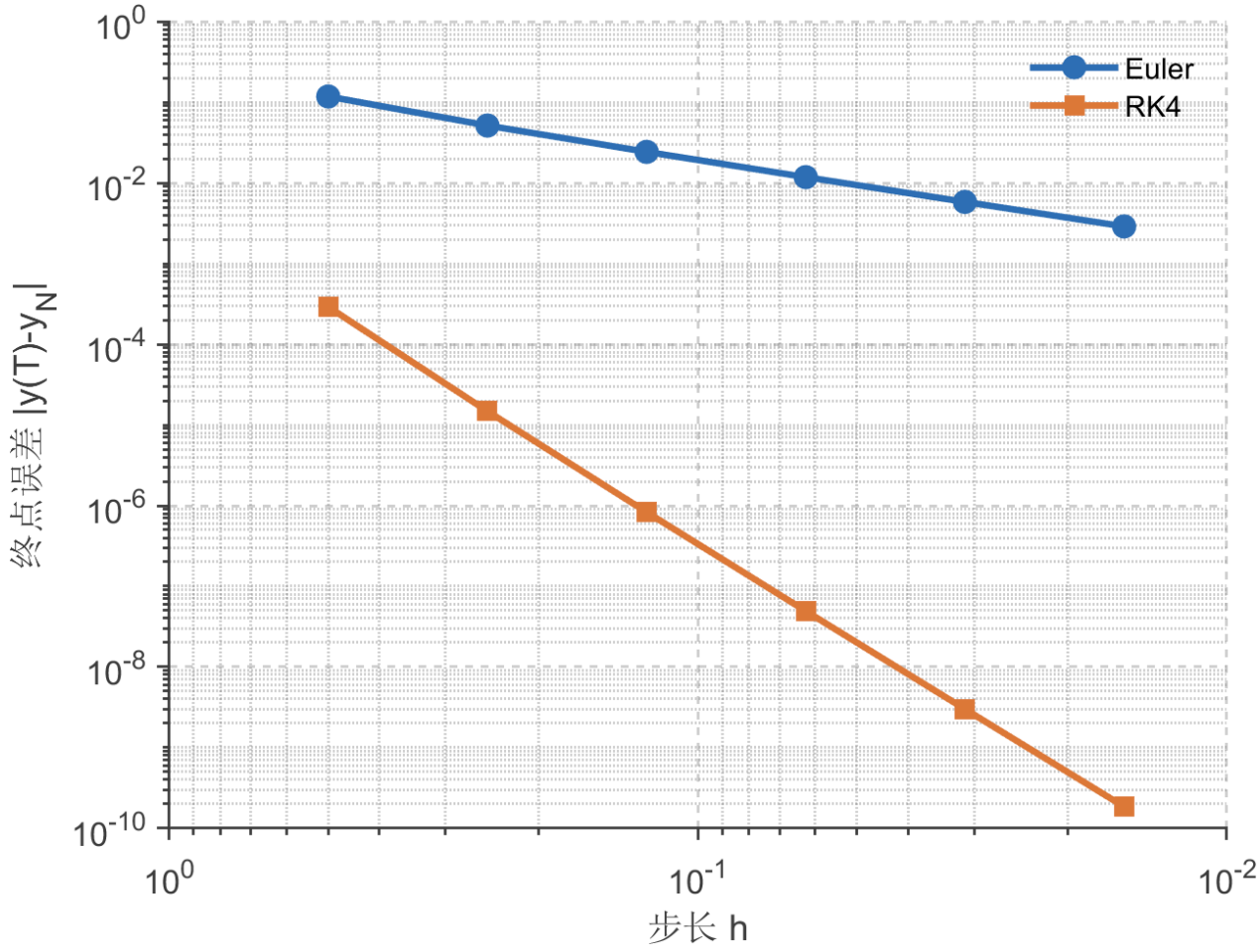


图 6　终点误差随步长的变化（双对数坐标）。斜率约为 1 的是 Euler，约为 4 的是 RK4

表 1 与理论一致：Euler 的观测阶由 1.20 逐步趋近 1，RK4 的观测阶由 4.30 趋近 4。步长较大时观测阶偏高，是因为高阶截断项尚未充分衰减；步长很小时观测阶偏离理论值，是因为舍入误差开始占主导。

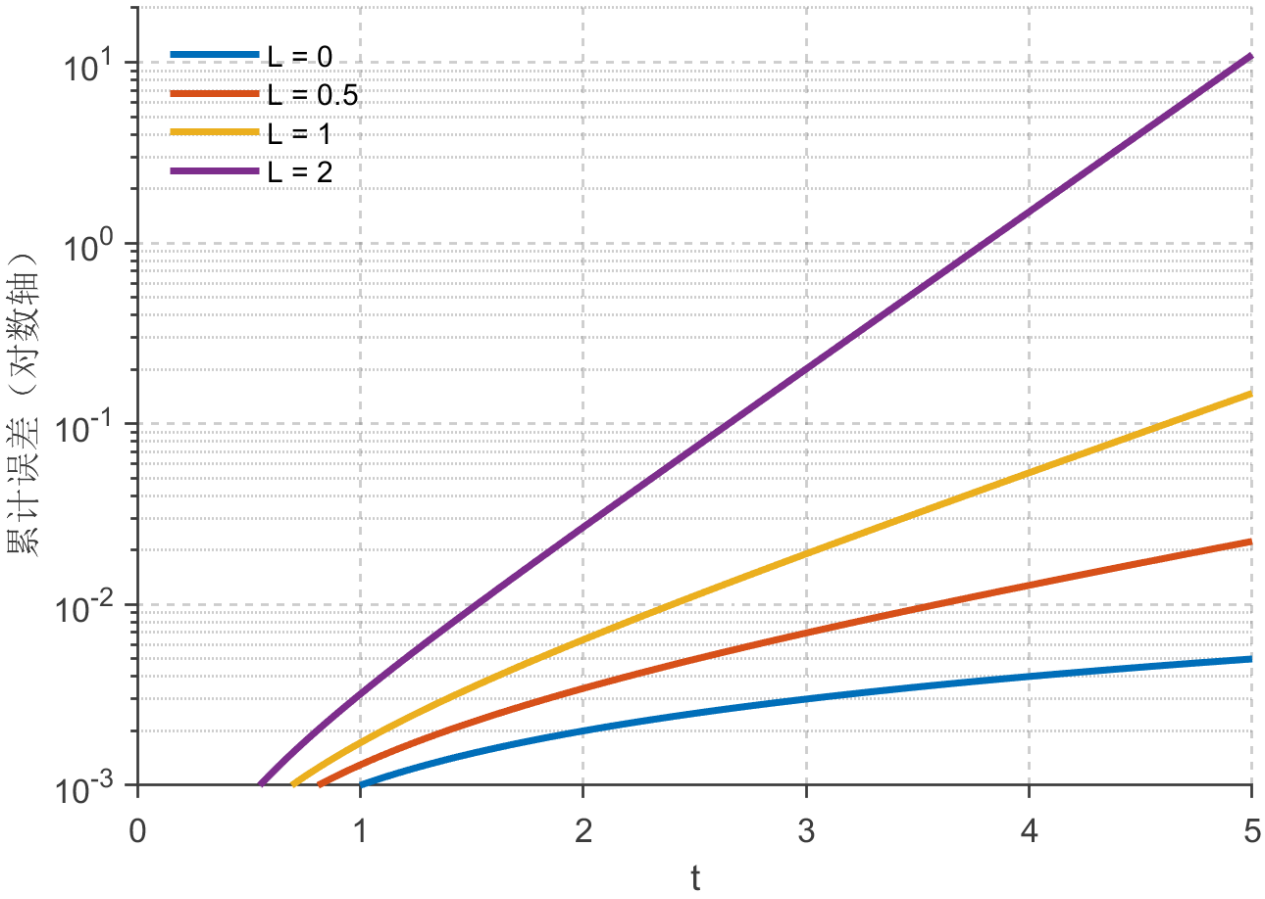


图 7　同一个局部误差在不同 L 下的累计量

3.1.4.2　稳定性

精度问的是“走得准不准”，稳定性问的是“会不会走飞”。步长一旦越过某条线，数值解就不再是解的近似，而是一个自己长出来的东西。收敛阶描述 $h \to 0$ 时的渐近精度，无法回答固定步长下误差是否会被放大。为分析稳定性，考虑线性测试方程

$$y' = \lambda y, \qquad \lambda \in \mathbb{C}. \qquad (19)$$

把数值格式作用于 (19)，一步迭代可以写成

$$y_{n+1} = R(z)\, y_n, \qquad z = h\lambda,$$

$R(z)$ 称为该格式的稳定函数；要数值解不增长，需要

$$|R(z)| \le 1. \qquad (20)$$

满足 (20) 的复数 z 的集合称为绝对稳定区域。精度与稳定性是相互独立的性质：高阶格式在步长过大时同样可能失去稳定性[9]。

对显式 Euler，$y_{n+1} = y_n + h\lambda y_n = (1+z)y_n$，故 $R(z) = 1 + z$。条件 $|1+z| \le 1$ 在复平面上是以 -1 为圆心、半径为 1 的圆盘。对模型问题 $\lambda = -1$，$z = -h$，由 $|1-h| \le 1$ 得

$$0 \le h \le 2.$$

若额外要求解保持非负且单调衰减，则需 $1 - h \ge 0$，即 $h \le 1$。

对经典 RK4，把测试方程代入 (12)(13)，得

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}. \qquad (21)$$

式 (21) 中的多项式是指数函数 e^z 的 Taylor 展开截断到四次，故在测试方程上单步放大因子与 e^z 的差异起始于 z^5 量级。在实负轴上，$|R(z)| \le 1$ 的区间约为

$$-2.785 \le z \le 0,$$

对 $\lambda = -1$ 即 $h \lesssim 2.785$。图 8 给出两种方法的放大因子随步长的变化。

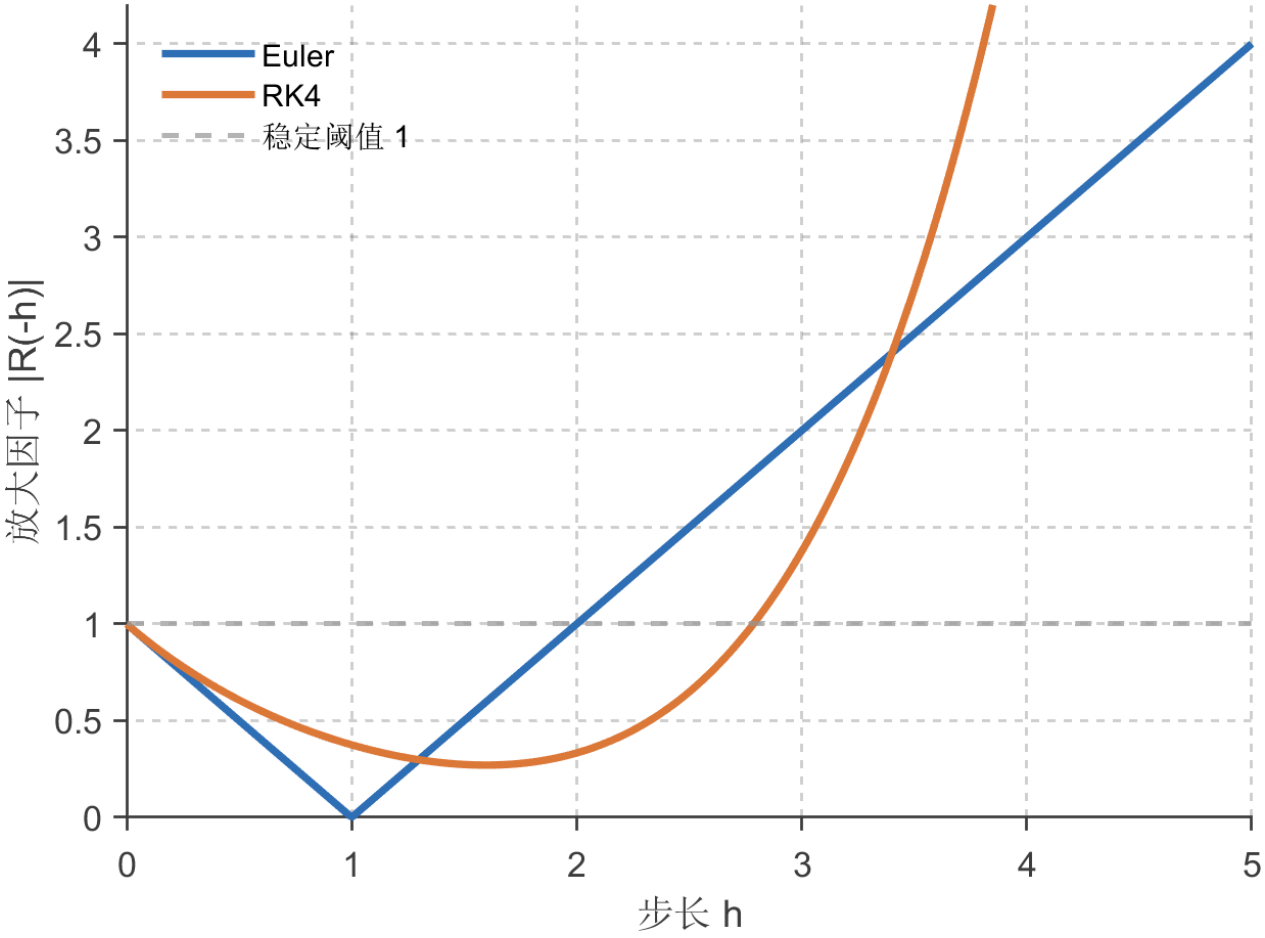


图 8　放大因子 $|R(-h)|$ 随步长的变化，虚线为稳定阈值 1。Euler 在 h=2 处越过阈值，RK4 在 h≈2.785 处越过阈值

步长越过稳定区间后，数值解会以交替符号振荡并迅速增长。图 9 是 $h = 3$、$T = 9$ 时两种方法的数值解与精确解的对比：Euler 的放大因子为 2，几步之后幅值已达 10^0 量级；RK4 的放大因子为 1.375，同样增长，只是速度稍慢。稳定性是格式的性质，不是初值的性质。

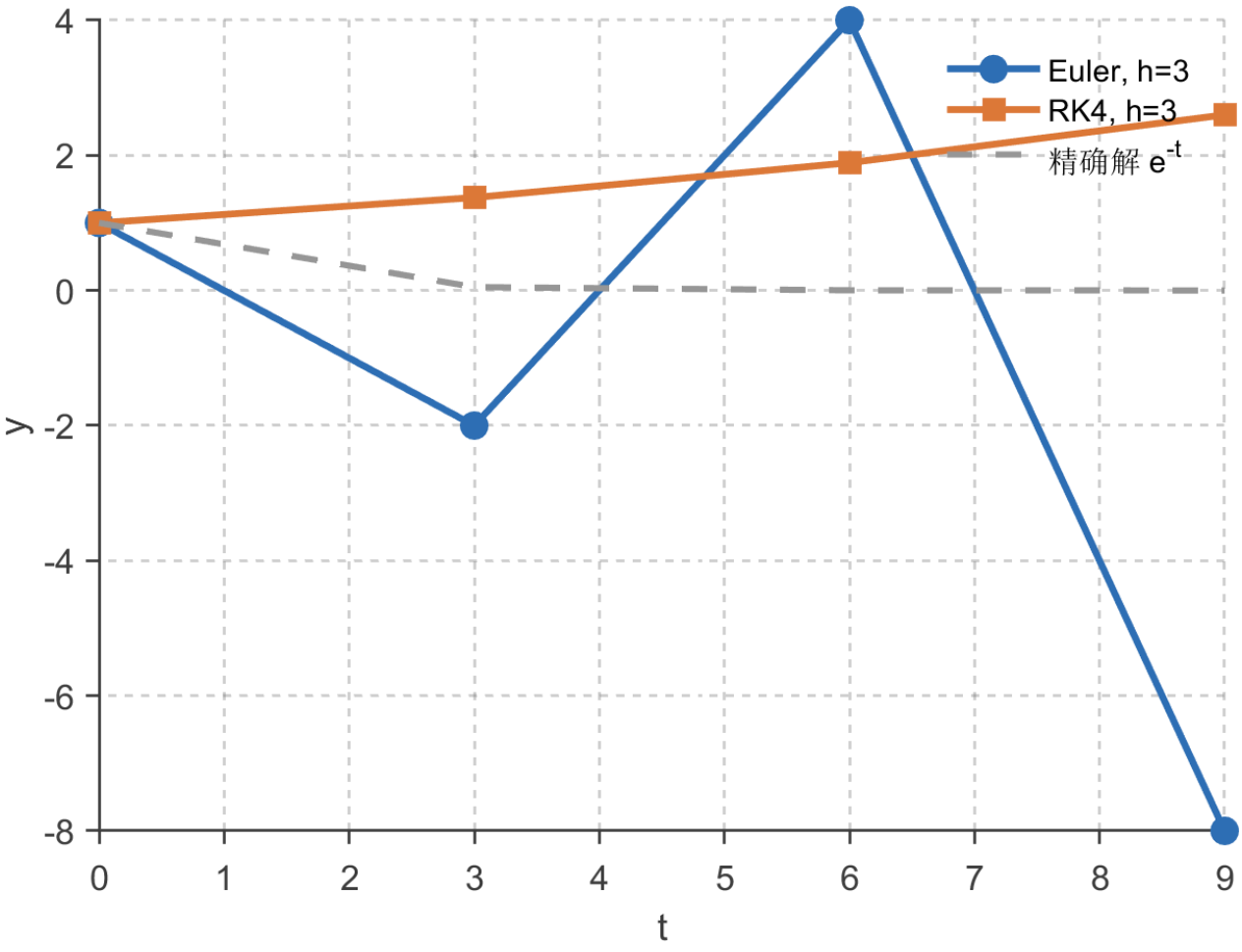


图 9　h=3 时两种方法的数值解与精确解（T=9）

3.1.4.3　计算成本

计算成本可用两类指标衡量：右端函数求值次数 n_f，与编程语言、硬件无关，适合理论比较；CPU 运行时间，反映实际开销，受实现与硬件影响。Euler 每步求值一次，RK4 每步求值四次，故相同步数下 $n_f^{\mathrm{RK4}} = 4n_f^{\mathrm{Euler}}$。

但单步成本不决定总成本。若要求两种方法达到相同精度，设 $E_{\mathrm{Euler}} \approx ah$、$E_{\mathrm{RK4}} \approx bh^4$，令 $ah_{\mathrm{Euler}} = bh_{\mathrm{RK4}}^4$，得

$$h_{\mathrm{Euler}} \approx \frac{b}{a}\, h_{\mathrm{RK4}}^4 .$$

相应的求值次数之比为

$$\frac{n_f^{\mathrm{Euler}}}{n_f^{\mathrm{RK4}}} = \frac{T/h_{\mathrm{Euler}}}{4T/h_{\mathrm{RK4}}} = \frac{a}{4b\,h_{\mathrm{RK4}}^3}.$$

步长越小，这个比值增长得越快：RK4 每步四次求值的代价，很快被它允许的大步长抵消。一句话：Euler 每步便宜，却要多走几个数量级的步数；RK4 每步贵四倍，买回来的正是这些步数。

取 $T = 1$、$h = 1/128$，实测结果列于表 2：Euler 与 RK4 的求值次数分别为 128 与 512，误差分别为 1.44×10^{-3} 与 1.15×10^{-11}；求值次数之比为 4，而实际耗时之比为 5.9（不同步长的中位数）。耗时比高于 4，是因为 RK4 每步除四次求值外还要组合四个阶段。只有在同一编程语言、同一实现与同一硬件下，耗时比较才有意义。

表 2　计算成本对比（T=1，h=1/128）

方法	求值次数	终点误差	相对耗时
Euler	128	1.44×10^{-3}	1.0
RK4	512	1.15×10^{-11}	5.9

3.1.5　趣味编程——基于单步法的导弹轨迹仿真

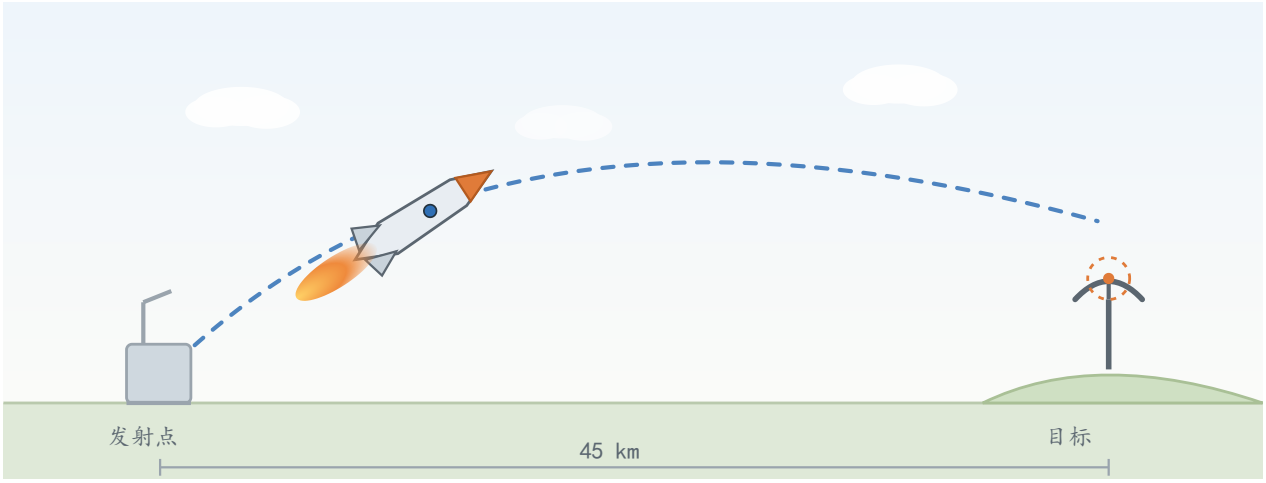


图 10　拦截场景：从发射点打出一枚拦截弹，目标是 45 km 外、6 km 高的阵地

考虑一枚拦截弹：由原点以初速 $v_0=800$ m/s 发射，目标位于 $(45,\,6)$ km。将弹体视为质点，忽略姿态与推力，只计重力与二次空气阻力。以位置与速度 $\boldsymbol{s}=(x,y,v_x,v_y)^{\mathsf T}$ 为状态，运动方程为

$$\dot{\boldsymbol{s}}=f(\boldsymbol{s})=\begin{pmatrix} v_x \\ v_y \\ -k\|\boldsymbol{v}\|v_x \\ -g-k\|\boldsymbol{v}\|v_y \end{pmatrix},\qquad \|\boldsymbol{v}\|=\sqrt{v_x^2+v_y^2},$$

其中 $k=5\times10^{-6}\ \mathrm{m^{-1}}$，$g=9.81\ \mathrm{m/s^2}$。

需要确定的是发射角 θ。给定 θ 后，初值取

$$\boldsymbol{s}_0=(0,\,0,\,v_0\cos\theta,\,v_0\sin\theta)^{\mathsf T},$$

弹道由该初值问题唯一确定。记弹道在下降段穿过 $y=6$ km 时的横坐标为 $X(\theta)$，则 θ 应满足

$$X(\theta)=45\ \mathrm{km},\qquad \theta\in[5^\circ,50^\circ].$$

由于 $X(\theta)$ 无解析表达式，上述问题归结为一元方程的数值求根。

先计算 $X(\theta)$。取步长 h，用 RK4 将 $\boldsymbol{s}_n$ 推进到 $\boldsymbol{s}_{n+1}$：

$$\boldsymbol{s}_{n+1}=\boldsymbol{s}_n+\frac{h}{6}(\boldsymbol{k}_1+2\boldsymbol{k}_2+2\boldsymbol{k}_3+\boldsymbol{k}_4),$$

$$\boldsymbol{k}_1=f(\boldsymbol{s}_n),\quad \boldsymbol{k}_2=f\!\left(\boldsymbol{s}_n+\tfrac{h}{2}\boldsymbol{k}_1\right),\quad \boldsymbol{k}_3=f\!\left(\boldsymbol{s}_n+\tfrac{h}{2}\boldsymbol{k}_2\right),\quad \boldsymbol{k}_4=f\!\left(\boldsymbol{s}_n+h\boldsymbol{k}_3\right).$$

若某一步满足 $y_n\ge 6\ \mathrm{km}>y_{n+1}$，则在该步内以二分法求 $\tau\in[0,h]$，使 $[\mathcal{S}(\boldsymbol{s}_n,\tau)]_y=6$，并取 $X(\theta)=[\mathcal{S}(\boldsymbol{s}_n,\tau)]_x$。

再求方程的根。$X(\theta)$ 关于 θ 单调递增，取 $a^{(0)}=5^\circ$、$b^{(0)}=50^\circ$，按

$$\theta^{(m+1)}=\frac{a^{(m)}+b^{(m)}}{2}$$

试探：若 $X(\theta^{(m+1)})<45$ km，令 $a^{(m+1)}=\theta^{(m+1)}$、$b^{(m+1)}=b^{(m)}$；否则令 $a^{(m+1)}=a^{(m)}$、$b^{(m+1)}=\theta^{(m+1)}$。当 $b^{(m)}-a^{(m)}<10^{-6}$ rad（约 6×10^{-5} 度）时终止迭代。

将 RK4 换成显式 Euler 格式

$$s_{n+1}=s_n+h\,f(s_n),$$

其余步骤不变，即得 Euler 的相应结果。

计算得 $\theta^\star=39.6859^\circ$。图 11 给出两种格式的仿真弹道，表 3 给出不同步长下的落点偏差。

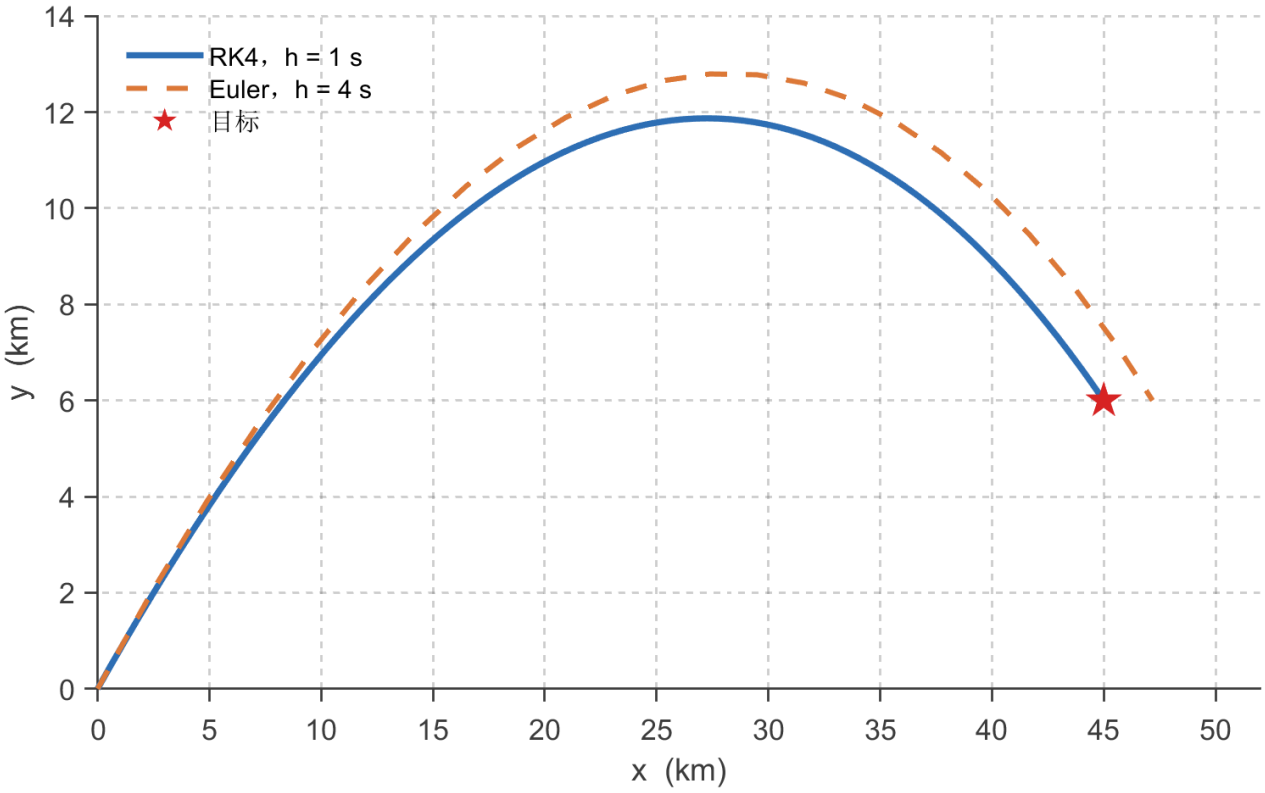


图 11　仿真弹道：RK4（h=1 s）在 (45,6) km 处命中目标；Euler（h=4 s）的弹道整体偏高，落点偏出约 2.2 km

表 3　同一发射角下的落点偏差（目标 x=45 km，y=6 km）

步长 h (s)	Euler 落点偏差 (m)	Euler 求值次数	RK4 落点偏差 (m)	RK4 求值次数
16	8249.6	7	0.320	24
8	4239.9	12	0.0195	44
4	2182.2	22	0.00169	84
2	1099.8	43	0.000603	168
1	554.1	85	0.000535	336
0.5	277.7	168	0.000531	668

由表 3 可见，Euler 的落点偏差随步长减半近似减半（相邻比值 1.95、1.94、1.98、1.98、2.00），为一阶收敛；RK4 在 $h = 2$ s 时偏差已小于 1 mm，进一步减小步长偏差不再下降，受参考解精度限制。步长同为 **0.5** s 时，Euler 的偏差为 278 m，RK4 仅为 0.53 mm。由图 11 可见，RK4 的弹道平滑地穿过目标点；Euler 在 $h = 4$ s 时的弹道整体偏高，落点明显偏出。

3.1.6 RK4 在今天

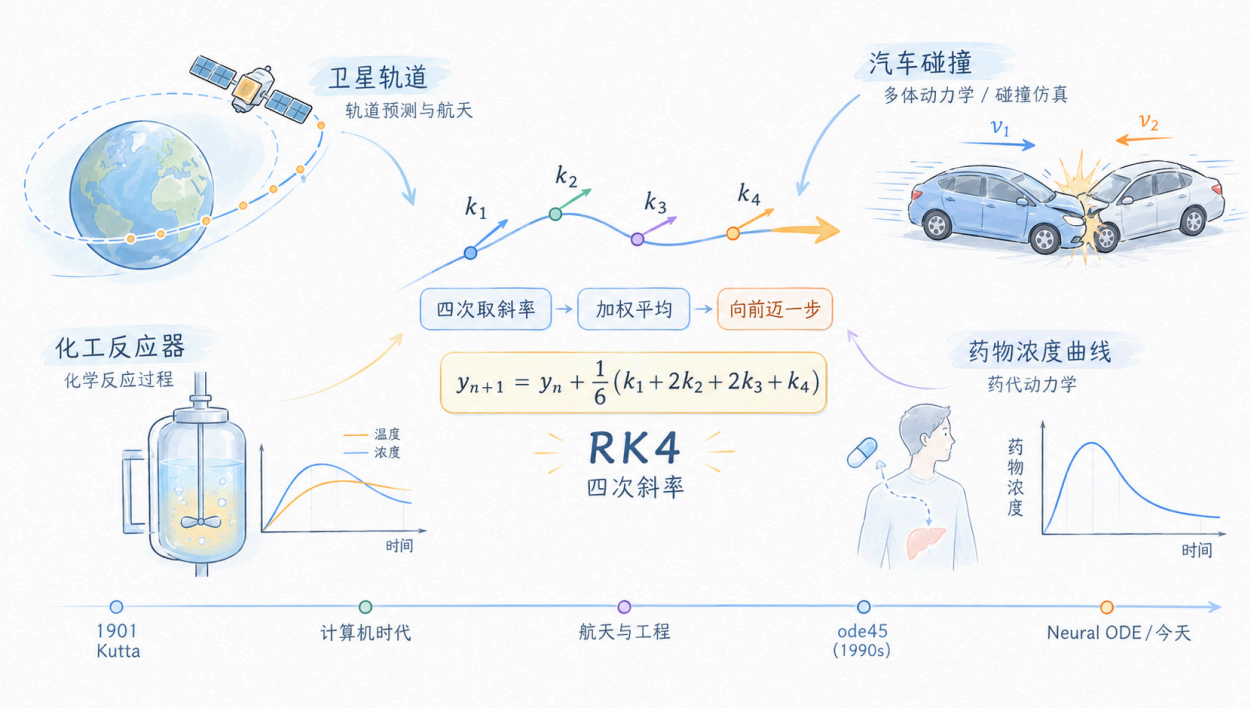


图 12　四类常见应用：卫星轨道、汽车碰撞、化工反应器、药物浓度曲线

自从 1901 年 Kutta 在纸上写下那四个斜率，这个只有几行的公式就悄悄走出了教科书，不断换上新身份。

最先接纳它的是刚诞生的电子计算机。上世纪五六十年代，卫星上天、飞船奔月，工程师第一次可以用机器代替手算表格；预报轨道、计算弹道，最顺手的工具就是每步问四次坡度的 RK4。

七十年代，它又走进化工与制药：反应器里的温度与浓度怎样随时间变化，药物在血液里的浓度怎样升、怎样降，都被写成联立的常微分方程，再用 RK4 一步步推出来。

1980 年，Dormand 和 Prince 想出了新点子：同一步里同时算出四阶与五阶两个答案，用两者的差估计误差，据此自动调整步长。这对四阶与五阶的搭档[13] 后来成了 MATLAB 里 `ode45` 的心脏；上世纪九十年代起，工程师不必再自己写积分器，一行 `ode45` 就够了。

到了今天，它的舞台更大：航天任务分析、汽车碰撞与多体动力学仿真、锂电池的等效电路、新药研发中的药代动力学曲线，背后都有它的身影。2018 年，Neural ODE 干脆把 `ode45` 装进了神经网络，让网络的“层”变成一段微分方程。

跨越一百多年，它万变不离其宗：每一步，在几个位置摸一摸坡度，加权平均，再往前迈一步。多问三次，误差小四百多倍——我们在纸上写下的 RK4，正是这个朴素动作最简洁的数字化表达。

第二节（3.2）　PDE 有限差分：热传导与 CFL 条件

从傅里叶的热流到网格上的稳定边界

3.2.0　引子：一本书与一台机器

一块烧红的铁，从一头热到另一头，需要多久？

这不是一道纯粹的数学题。冶金作坊要知道铸件多久才能冷透，玻璃工匠要知道退火炉里的温度怎样变化，工程师要知道汽缸壁散失多少热量。更远一点，还有人追问：地球内部为什么越深越热？这些问题的共同点是——只知道此刻的温度，却想预测下一刻、乃至任意时刻的温度。

1807 年 12 月 21 日，巴黎科学院。三十九岁的傅里叶宣读了一篇论文，题目是《固体中热的传播》。他当时的身份不是大学教授，而是伊泽尔省省长：修过格勒诺布尔到都灵的大道，排过沼泽地的水，也跟随拿破仑远征过埃及。白天处理行政，夜里回到书房摆弄铁棒、玻璃球和温度计。热，从工业炉（处理金属）、地球内部的温度及其随时间的变化，到他一路见过的地中海与埃及的烈日，都是他长期惦记的问题[14][35][36]。

傅里叶的出发点很朴素：不去追踪每一个分子，而是把温度看成一个场 $u(x,t)$。取细杆上的一小段，算清流入、流出与内部产生的热量，再假设热流与温度梯度成正比（这套推导写进了他 1807 年的论文[15]）：

$$q = -k\frac{\partial u}{\partial x}.$$

这就是后来以他命名的傅里叶定律。把守恒律与它联立，消去热流 q，就得到

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}.$$

这条方程说的是：温度的变化率正比于温度分布的“弯曲程度”。曲线上凹得越厉害，该点温度变得越快[16]；温度分布越平，它就越安静。

真正的困难在这里：对任意的初始温度 $u_0(x)$，方程怎么解？傅里叶给出的答案是——把 u_0 拆成无穷多条正弦曲线，让每一条按自己的速率衰减，再把结果叠加起来：

$$u(x,t) = \sum_{n=1}^{\infty} b_n\, e^{-\kappa (n\pi/L)^2 t} \sin\frac{n\pi x}{L}.$$

为了说清“任意函数都能这样拆”，他被迫去研究三角级数的收敛性，这就是今天的傅里叶级数。数学史上因此有一个说法：傅里叶级数不是从纯数学的头脑里掉下来的，它是被热传导方程逼出来的[16][36]。

这条路并不平坦。1807 年的论文由拉格朗日、拉普拉斯、蒙日与拉克鲁瓦评审，拉格朗日对“任意函数展开成正弦级数”深表怀疑；1811 年巴黎科学院以“固体中热的传播”设奖，傅里叶拿下了奖金，评审报告却仍说他的推导“在严格性和一般性上还有欠缺”；1822 年《热的解析理论》出版。批评没有阻止这条方程，反而把它推到了数学分析的中心。

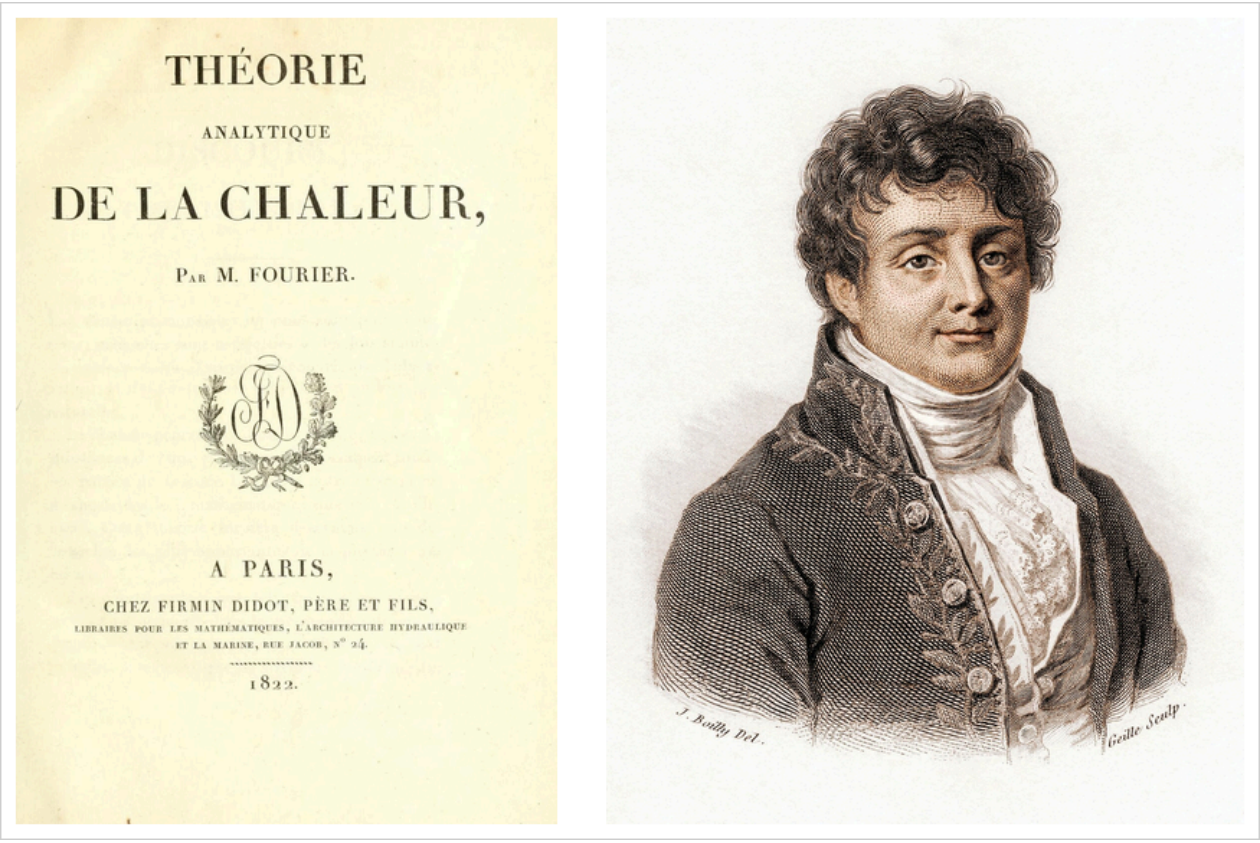


图 13　《热的解析理论》初版扉页（1822，左）与傅里叶像（右，J.-L. Boilly 绘）。图片来源：Internet Archive、Wikimedia Commons

一百三十八年后，这件事在另一台机器上以另一种形式重新出现。

周毓麟原本走的是拓扑学，路已经铺开。后来，他把这条走顺的路收起来，转攻当时国内尚属空白的非线性偏微分方程，在渗流方程上做出了被公认为经典的结果。如果故事停在这里，这是一位数学家漂亮的转身。

但一纸调令，把他推到了另一类问题面前。他加入核武器研制队伍，成为理论部“八大主任”之一，站的是数学这一侧。理论突破中最硬的一道坎是“九次计算”——为核对一个关键数据，一群人反复验算了九个月。周毓麟负责的是让机器能算下去：找方法、编程序，把方程变成机器能执行的东西。他参与设计的一些程序，用了近二十年。论文会署名，程序不会说话；但一个程序能跑近二十年，本身就是一种回答。

有人问他，放下已经做出结果的方向，后不后悔。他的回答只有一句话：“党叫干啥就干啥。”后来他写过一句注脚：“国家需要我，是我的荣幸。”[17]

等到年近花甲，他又转身一次。这一次不是换问题，而是往问题底下钻：网格上的方程为什么能稳稳地算下去？步长能取多大？越过那道限制，会发生什么？他开创离散泛函分析这一新领域，为非线性发展方程的差分方法建立了严谨的理论体系。从把方程算出来，到追问“为什么这个数可信”，这是一次更安静、也更难的转身[17][37]。

1964 年 10 月 16 日，罗布泊上空升起蘑菇云[29]。蘑菇云是看得见的答案；看不见的，是云柱下方那些把方程变成程序、把程序变成可信数字的判断。

图 14　周毓麟（1923–2021，左）与 1964 年 10 月 16 日罗布泊上空的蘑菇云（右）。图片来源：中国科学家博物馆、Wikimedia Commons

本节从这道方程出发：3.2.1 节由守恒律导出方程，给出定解条件与最大值原理；3.2.2 节把方程搬到网格上，设计出可以变成 $A\boldsymbol{x}=\boldsymbol{b}$ 的差分格式，再让程序跑出数值解与收敛阶；3.2.3 节讲稳定性——步长能取多大，越过限制会怎样；3.2.4 节用这条方程修复一张被划花的二维码。3.2.5 节把视角拉远，看这道方程今天还在哪些地方工作。

3.2.1　热传导方程：从守恒律到适定性

3.2.1.1　热传导方程的导出

取细杆上的一小段 $[x, x+\Delta x]$（图 15），截面积为 S，单位体积热容为 ρc，温度为 $u(x,t)$，热流密度为 $q(x,t)$（规定向右为正）。这一段储存的热量为 $\rho c u S\Delta x$，它随时间的变化率，等于从左侧流入的热量减去从右侧流出的热量，再加上段内源项产生的热量：

$$\rho c\, S\Delta x\,\frac{\partial u}{\partial t} = S\,q(x,t) - S\,q(x+\Delta x,t) + S\Delta x\, f(x,t).$$

两端同除以 $S\Delta x$ 并令 $\Delta x\to 0$：左端化为温度对时间的偏导数，右端化为热流沿杆方向的梯度；再把傅里叶定律 $q=-k\partial u/\partial x$ 代入，即得

$$\rho c\,\frac{\partial u}{\partial t} = \frac{\partial}{\partial x}\Big(k\frac{\partial u}{\partial x}\Big) + f.$$

若杆的材料均匀，k 为常数，则可记 $\kappa = k/(\rho c)$、$g = f/(\rho c)$，把方程整理成热传导方程的标准形式

$$\frac{\partial u}{\partial t} = \kappa\,\frac{\partial^2 u}{\partial x^2} + g(x,t). \tag{22}$$

定义 1（热方程）　形如 (22) 的方程称为热传导方程；$\kappa>0$ 为热扩散率，g 为源项，$g\equiv 0$ 时称为齐次热方程。

以上的推导只用到三个假设：介质各向同性；热流对温度梯度的响应是瞬时的；材料没有记忆效应。在常规尺度下，这三条都足够好；但在飞秒激光加热、纳米薄膜与超低温等情形，就需要 Cattaneo–Vernotte 一类的非傅里叶模型。本书只讨论傅里叶模型。

值得指出，按傅里叶模型，热的传播速度是无限的：只要 $t>0$，杆内部任何一点都会受到初值的影响，只是离得越远、偏离越小。这与“热量一点点渗过去”的直觉不同，也是傅里叶模型的代价；Cattaneo－Vernotte 模型给热流引入有限的松弛时间，修正的正是这一点。

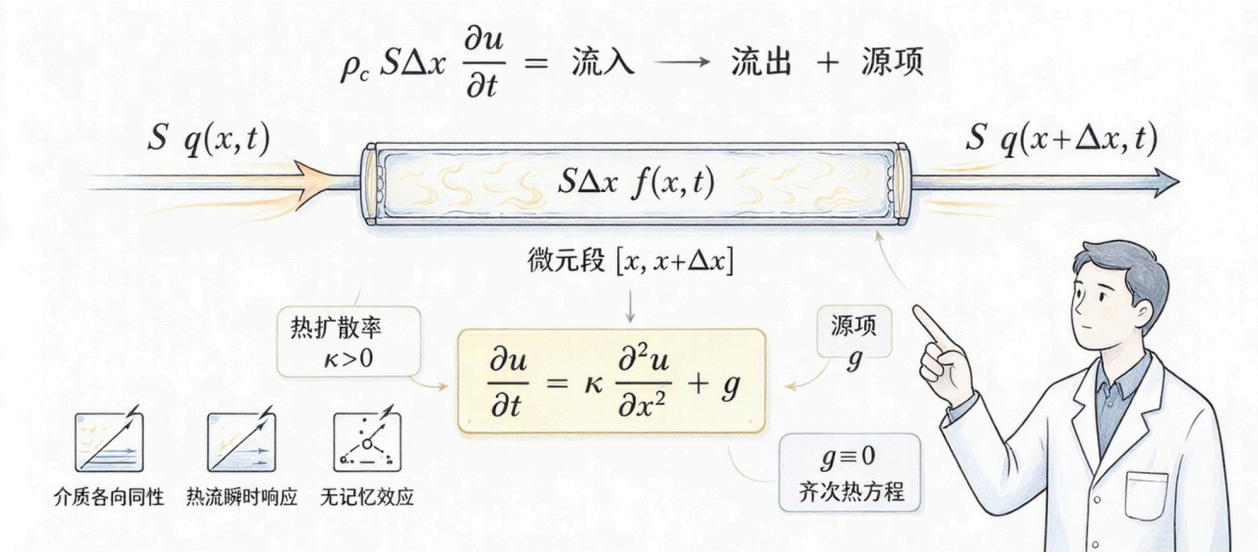


图 15　杆元的热量守恒：流入、流出与源项共同决定这一小段的温度变化

3.2.1.2　定解问题与三类边界条件

热方程对时间是一阶的，因此需要一个初始条件；对空间是二阶的，因此需要两个边界条件：

$$\begin{cases} u_t = \kappa u_{xx} + g(x,t), & 0<x<L, \quad 0<t\le T, \\ u(x,0) = u_0(x), & 0\le x\le L, \\ \alpha_0 u(0,t) + \beta_0 u_x(0,t) = \gamma_0(t), & 0<t\le T, \\ \alpha_L u(L,t) + \beta_L u_x(L,t) = \gamma_L(t), & 0<t\le T. \end{cases} \tag{23}$$

定义 2（古典解）　设 $u\in C^{2,1}\big((0,L)\times(0,T]\big)\cap C\big([0,L]\times[0,T]\big)$。若 u 在内部逐点满足 (23) 的方程，在 $t=0$ 与 $x=0,L$ 上逐点满足初值与边界条件，则称 u 为 (23) 的古典解。

系数 α、β 取不同的值，就得到三类边界条件；三者的物理含义不同，离散时也必须分别处理（表 4）：

表 4　三类边界条件及其离散方式

名称	取法	物理含义	离散方式
Dirichlet	$\alpha=1,\ \beta=0$	端点温度给定	边界值直接代入
Neumann	$\alpha=0,\ \beta=1$	端点热流给定，零热流即绝热	虚拟节点
Robin	$\alpha=h,\ \beta=\pm k$	端点与外界对流换热	虚拟节点 + 消元

这三个名字听上去抽象，用一根铁棒就能把它们的区别讲清楚（图 16）：

- Dirichlet（第一类）：把铁棒的一端插进一杯冰水里。只要杯里还有冰，这一端的温度就被“钉”在 0 ℃ 上，无论传过去多少热都一样。这就是“端点温度给定”——这一端的值不是未知数，每一步直接赋成已知值。

- Neumann（第二类）：给铁棒的一端裹上厚厚的保温棉。裹得足够好的时候，几乎没有任何热量从这一端进出，也就是“端点热流为零”，即绝热。这就是“端点热流给定”：它不告诉你这一端是多少度，只告诉你从这端流走了多少热（零热流时总热量守恒）。
- Robin（第三类）：把铁棒的一端露在空气里，用风扇对着它吹。这一端与空气之间发生对流换热：流走的热量既与“铁棒这端有多热”有关，也与“空气有多冷”有关，二者之间是一个线性的比例关系。这就是“端点对流换热”，也是烧红的铁块在风里凉得更快的数学描述。

三类边界的差别可以一句话概括：Dirichlet 管住“温度”，Neumann 管住“流量”，Robin 把两者按比例绑在一起。若 $u_0(0) \neq \gamma_0(0)$，解在角点 $(0,0)$ 处不连续，古典解不存在。

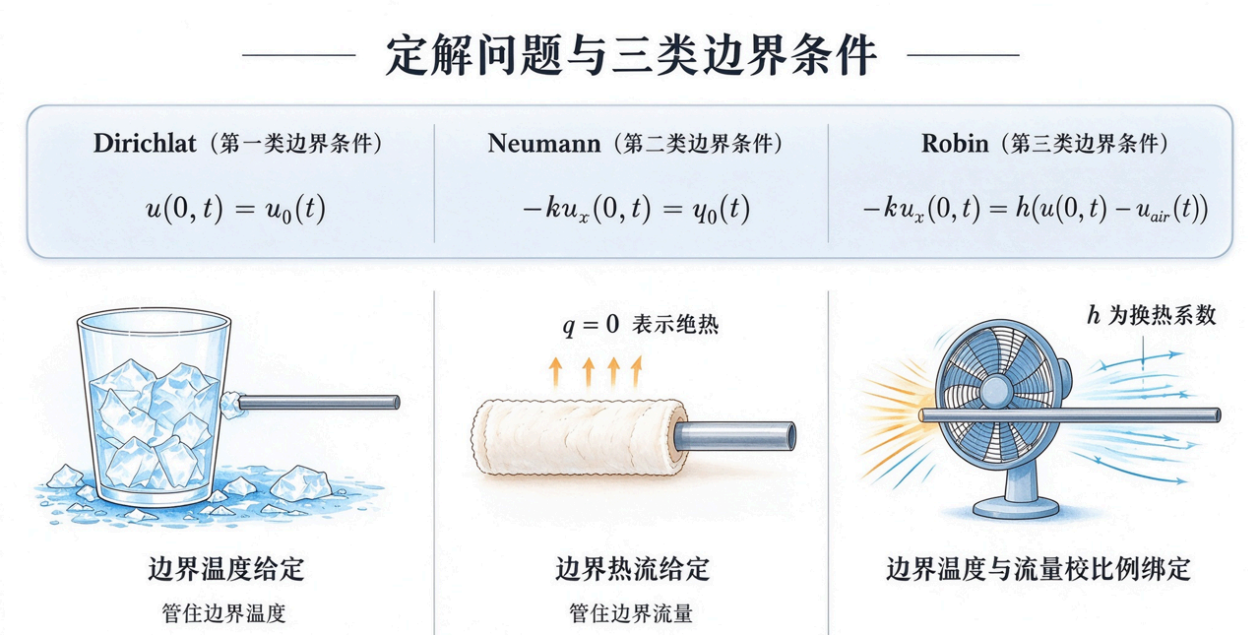


图 16　一根铁棒上的三类边界：冰水定温（Dirichlet）、保温棉绝热（Neumann）、风扇换热（Robin）

3.2.1.3　最大值原理与适定性

在讨论数值方法能否“收敛”之前，先要确认解本身存在、唯一，并且对初边值的微小扰动不敏感，这就是适定性。最大值原理给出的结论很直观：解不会在区域内部自己长出新的极值[22][23]。

定理 1（最大值原理）　设 u 在 $Q_T = (0,L) \times (0,T]$ 内满足 $u_t = \kappa u_{xx}$，在 $\bar{Q}_T$ 上连续。则 u 在 $\bar{Q}_T$ 上的最大值与最小值都在抛物边界

$$\Gamma_T = \{(x,0) : 0 \le x \le L\} \cup \{(0,t),(L,t) : 0 \le t \le T\}$$

上取到。

证明　记 $M = \max_{\Gamma_T} u$。先看严格不等式 $u_t - \kappa u_{xx} < 0$ 的情形。若 u 在内部点 (x_0,t_0)，$0 < x_0 < L$，$0 < t_0 \le T$，取到最大值，则 $u_x(x_0,t_0) = 0$、$u_{xx}(x_0,t_0) \le 0$，且对 t 的偏导非负，于是 $u_t - \kappa u_{xx} \ge 0$，矛盾。所以最大值只能在 Γ_T 上取到。

以上论证要求严格不等式成立。对一般的 u，可以用一个扰动把严格性“造”出来：令 $v = u - \varepsilon t$，$\varepsilon > 0$，则 $v_t - \kappa v_{xx} = u_t - \kappa u_{xx} - \varepsilon < 0$。由上面的论证，$v$ 的最大值在 Γ_T 上取到，于是

$$u(x,t) - \varepsilon t = v(x,t) \le \max_{\Gamma_T} v \le \max_{\Gamma_T} u = M,$$

令 $\varepsilon\downarrow 0$ 得 $u\le M$。对 $-u$ 重复上述论证得最小值。证毕。

推论 1（唯一性与连续依赖） 若 (23) 的解存在，则唯一，且

$$\max_{\bar{Q}_T}|u|\le\max_{\Gamma_T}|u|. \tag{24}$$

证明 设 u_1,u_2 是两个解，$w=u_1-u_2$ 满足齐次方程与零初边值，由 (24) 得 $\max|w|\le 0$，故 $w\equiv 0$。连续依赖同理：初边值之差不超过 ε，解之差也不超过 ε。证毕。

换一个角度，能量方法可以给出 L^2 估计。设 u 满足齐次方程与齐次 Dirichlet 条件，令 $E(t)=\frac{1}{2}\int_0^L u^2\,\mathrm{d}x$，则

$$\frac{\mathrm{d}E}{\mathrm{d}t}=\int_0^L u\,u_t\,\mathrm{d}x=\kappa\int_0^L u\,u_{xx}\,\mathrm{d}x=-\kappa\int_0^L u_x^2\,\mathrm{d}x\le 0.$$

由 Poincaré 不等式得 $E(t)\le E(0)e^{-2\kappa\pi^2 t/L^2}$。最大值原理看的是极值，能量估计看的是平方积分，两种方法说明的却是同一件事：初值的扰动只会被抹平，不会被放大。离散格式能不能保住这条性质，正是 3.2.3 节要回答的问题。

上式中的 π^2/L^2 来自最慢的模态 $\sin(\pi x/L)$：弯曲得最少的曲线被抹平得最慢，整条曲线的衰减速度就由它决定。

故事 | 钎尾还是凉的。 老陶把一根冷铁钎插进窑火，钎尖很快烧红，钎尾还是凉的。他不能把整根钎子抽出来看，只能用手摸钎尾。

铁钎里的热不会凭空冒出来：某一点升温的快慢，取决于它两侧的温度差——温度曲线弯得越厉害，热流进（出）得越快；钎尖在火里，是一个边界；钎尾在空气里，也是一个边界。老陶不懂偏微分方程，但他知道一条：钎子上最烫的地方一定在火上，不会在中间某处自己长出一个更高的温度。

这条经验正是最大值原理：边界定了，内部不会自己冒出新的极值。老陶据此断定，钎身里不会有比火端更热的点；要问的是另一件事——热量传到钎尾要多久，温度沿钎子怎样分布。那得把钎子切成小段再算，是 3.2.2 节的事（图 17）。

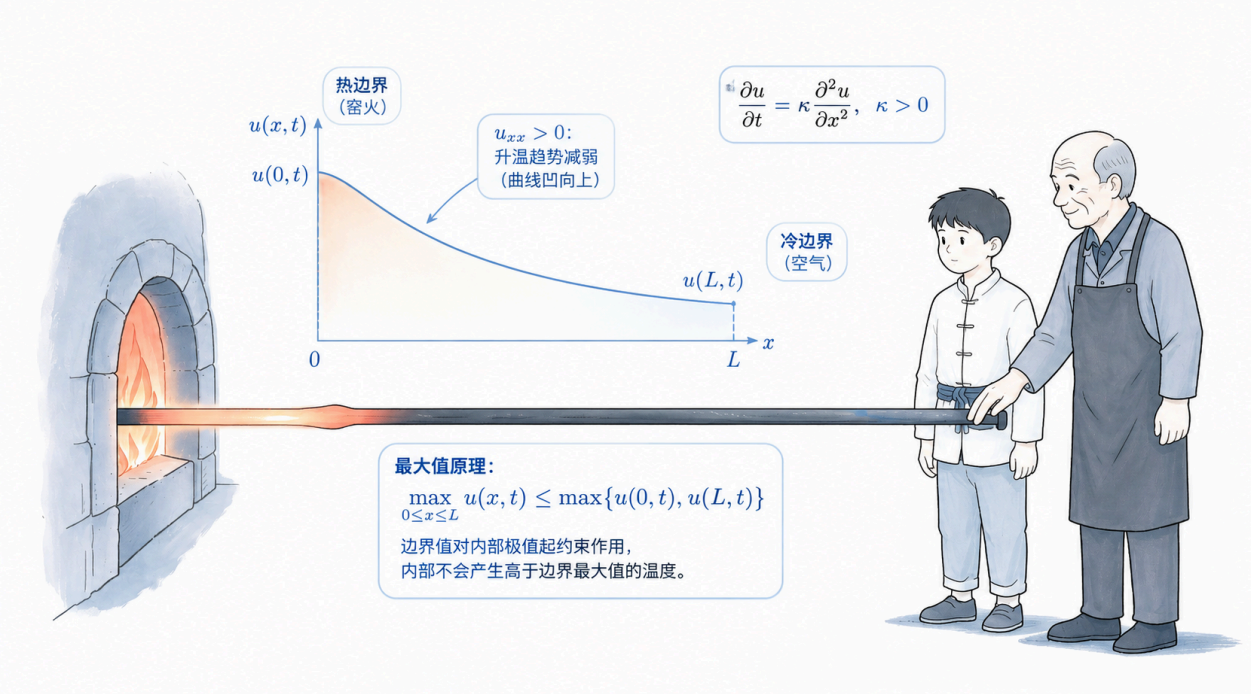


图 17　钎尖与钎尾：边界温度、温度剖面与最大值原理

3.2.2　空间差分与显式时间推进

傅里叶把解写成无穷多条正弦曲线之和，但计算机只能处理有限个数。于是问题变成：把一条连续的温度分布换成一排离散数值之后，下一时刻的这排数值应当如何计算？

先看清这幅图景（图 18）：一条直线上等分着 $N+1$ 个节点，每个节点上的温度按 $U_j^n \to U_j^{n+1}$ 逐层向前推进，而推进的依据只有它自己和左右两个邻居。二维情形把这条直线铺成一个平面，每个内点看上下左右四个邻居（图 19）。本节的差分格式，说到底都是在回答同一件事：取哪几个邻居、各给多大权重。

3.2.2.1　一维热传导方程的离散与求解

热方程 $u_t = \kappa u_{xx}$ 的右端衡量的是温度曲线弯曲的程度：某点高于左右邻居的平均值，热量便从该点向两侧流走，温度下降；低于平均值，热量向该点汇聚，温度上升。取节点 $x_j = jh$，用二阶中心差商代替二阶导数，就可以把这一含义写成

$$U_{xx} \approx \frac{U_{j-1} - 2U_j + U_{j+1}}{h^2} = \frac{2}{h^2}\left(\frac{U_{j-1} + U_{j+1}}{2} - U_j\right),$$

括号里恰好是“邻居平均减自己”，即该点温度与近邻平均温度之差。再往前推一步：经过一个时间步长，温度的变化量约等于变化率乘以 Δt。两者合起来，得

$$U_j^{n+1} = U_j^n + r\left(U_{j-1}^n - 2U_j^n + U_{j+1}^n\right) + \Delta t\, g_j^n, \qquad r = \frac{\kappa \Delta t}{h^2}, \qquad (25)$$

其中 $j = 1, \dots, N-1$。 r 叫网格比，可以读成“一个时间步之内，扩散在网格上推进的相对强度”。初值与 Dirichlet 边界条件相应离散为

$$U_j^0 = u_0(x_j), \qquad U_0^n = \gamma_0(t_n), \qquad U_N^n = \gamma_N(t_n).$$

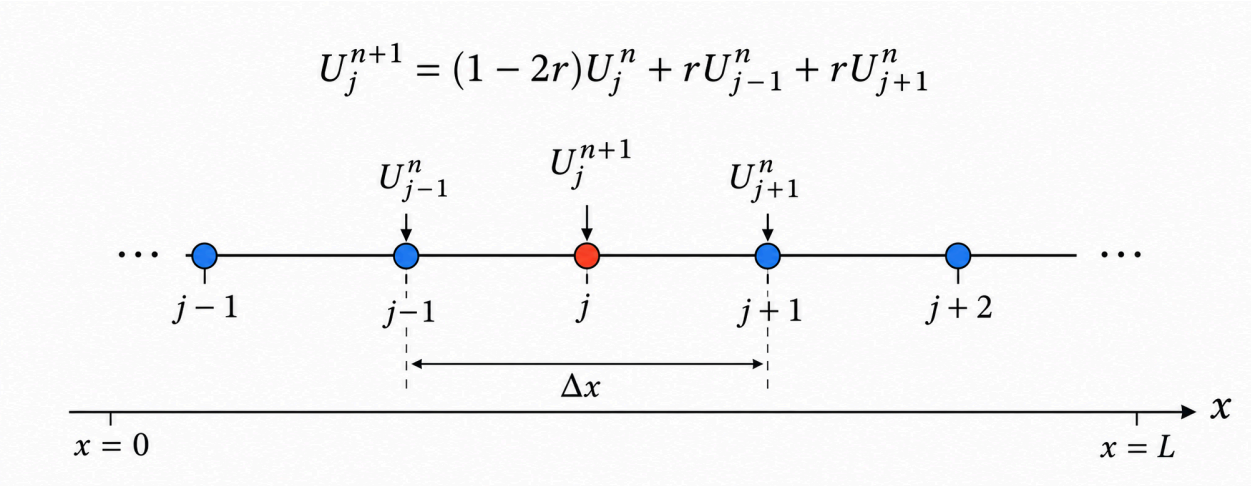


图 18　一维显式格式：新值由同层三个旧值加权得到，两端为已知边界

一维的矩阵形式

要把整个一维网格写成 $A\boldsymbol{x} = \boldsymbol{b}$，只需把内点排成一个列向量 $\boldsymbol{U}^n = (U_1^n, \dots, U_{N-1}^n)^{\mathsf{T}}$。先写出离散 Laplace 矩阵

$$L=\begin{pmatrix}-2 & 1 & & \\ 1 & -2 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & -2\end{pmatrix},$$

这是一个 $(N-1)$ 阶三对角矩阵：第 j 行依次给出 U_{j-1}、U_j、U_{j+1} 的系数，即 -2 与两个 1。于是显式格式可以写成如下的矩阵形式

$$U^{n+1}=BU^n+\Delta t\,g^n,\qquad B=I+rL=\begin{pmatrix}1-2r & r & & \\ r & 1-2r & r & \\ & \ddots & \ddots & \ddots \\ & & r & 1-2r\end{pmatrix}. \tag{26}$$

每步只需一次矩阵—向量乘法，不必解方程。

真正需要解方程的是隐式格式：把右端取在新一层，得到 $(I-rL)U^{n+1}=U^n+\Delta t g^{n+1}$。这就是标准的 $Ax=b$，其中 $x=U^{n+1}$ 是新一层的未知温度，而

$$A=I-rL=\begin{pmatrix}1+2r & -r & & \\ -r & 1+2r & -r & \\ & \ddots & \ddots & \ddots \\ & & -r & 1+2r\end{pmatrix}.$$

右端 $b=U^n+\Delta t g^{n+1}$ 收集的是旧一层的温度与源项。矩阵 A 对称、三对角且严格对角占优，因此每步只需求解一个三对角方程组；用追赶法即可在 $O(N)$ 次运算内完成。

表 5　边界条件如何进入 A 与 b

情形	处理	A 的首（末）行	b 的首（末）分量
内部点 j	三个未知量	$(-r,\ 1+2r,\ -r)$	$U_j^n+\Delta t g_j^{n+1}$
左端 Dirichlet	边界值移到右端	$(1+2r,\ -r)$	$U_1^n+\Delta t g_1^{n+1}+r\gamma_0(t_{n+1})$
左端 Neumann	设虚拟点 $U_{-1}=U_1-2h\sigma$	$(1+2r,\ -2r)$	$U_0^n+\Delta t g_0^{n+1}-2rh\sigma(t_{n+1})$

表 5 概括了“组装矩阵”的全部内容：先按内部点填好 A 的各行，再看两端属于哪一类边界，替换首末两行，并把边界值产生的项移到 b 中。

3.2.2.2　二维热传导方程的离散与求解

二维区域取 $\Omega=(0,1)^2$，两个方向都分成 N 段，$h=1/N$，内点 $(x_i,y_j)=(ih,jh)$。每个内点有四个邻居，把一维的推理照搬过来，离散 Laplace 算子就写成五点模板：

$$U_{i,j}^{n+1}=U_{i,j}^n+r\big(U_{i-1,j}^n+U_{i+1,j}^n+U_{i,j-1}^n+U_{i,j+1}^n-4U_{i,j}^n\big).$$

二维画不出“时间轴”，图 19 只画出同一时刻的网格：每个内点向上下左右各取一个邻居。把它写成加权平均，权重为 $(1-4r,r,r,r,r)$，五项之和为 1。中心权重必须非负，由此立刻得到二维的上界 $r\le 1/4$，恰是一维上界的一半。

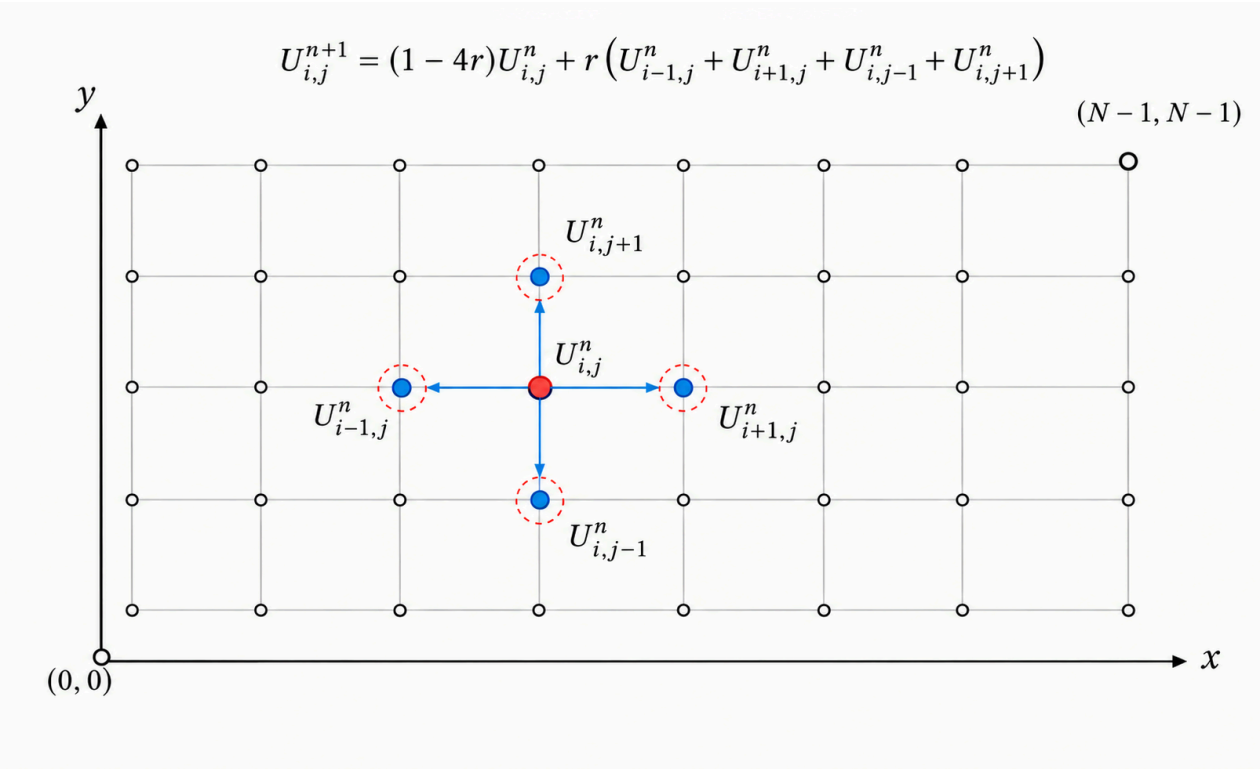


图 19　二维五点模板：新值只用自身与上下左右四个邻居，对角邻居不参与

二维的矩阵形式

二维与一维的本质区别，在于未知量的排列方式。设 x 方向和 y 方向的内点数均为 $N-1$，记 $r=\kappa\Delta t/(\Delta x)^2$（为简洁起见取 $\Delta x=\Delta y$）。把二维网格的内点按行优先排成一个长向量：先排第一行内点，再排第二行，依此类推，

$$\boldsymbol{x}=\left(U_{1,1}^{n+1},U_{2,1}^{n+1},\dots,U_{N-1,1}^{n+1},\,U_{1,2}^{n+1},\dots,U_{N-1,2}^{n+1},\,\dots,\,U_{1,N-1}^{n+1},\dots,U_{N-1,N-1}^{n+1}\right)^{\mathsf{T}},$$

其中 $U_{i,j}^{n+1}$ 表示第 j 行、第 i 列内点的新一层温度。该向量共有 $m=(N-1)^2$ 个分量。因此，二维隐式格式对应的线性方程组

$$A_{2D}\boldsymbol{x}=\boldsymbol{b},$$

其中，A_{2D} 是一个 $m\times m$ 的大型稀疏矩阵，$\boldsymbol{x}$ 和 $\boldsymbol{b}$ 均为 m 维列向量。当 N 较大时，$m=(N-1)^2$ 增长很快，例如 $N=101$ 时 $m=10000$，此时 A_{2D} 为 10000×10000 矩阵；但每行至多只有五个非零元，实际计算中并不存储全部零元。

为看清 A_{2D} 的具体结构，将 $\boldsymbol{x}$ 按行分块：

$$\boldsymbol{x}=\begin{pmatrix}\boldsymbol{x}_1\\ \boldsymbol{x}_2\\ \vdots\\ \boldsymbol{x}_{N-1}\end{pmatrix},\qquad \boldsymbol{x}_j=\begin{pmatrix}U_{1,j}^{n+1}\\ U_{2,j}^{n+1}\\ \vdots\\ U_{N-1,j}^{n+1}\end{pmatrix},\qquad j=1,\dots,N-1.$$

其中每个块 $\boldsymbol{x}_j$ 对应第 j 行上的 $N-1$ 个内点。相应地，右端向量也按同样方式分块：

$$\boldsymbol{b}=\begin{pmatrix}\boldsymbol{b}_1\\ \boldsymbol{b}_2\\ \vdots\\ \boldsymbol{b}_{N-1}\end{pmatrix},\qquad \boldsymbol{b}_j=\boldsymbol{U}_j^{n}+\Delta t\,\boldsymbol{g}_j^{n+1},$$

其中 $\boldsymbol{U}_j^{n}$ 和 $\boldsymbol{g}_j^{n+1}$ 分别是第 j 行内点的旧层温度和源项。若边界条件非齐次，则边界值产生的已知项也应并入相应的 $\boldsymbol{b}_j$ 中。

在上述按行分块的排列下，二维隐式格式的系数矩阵 $\boldsymbol{A_{2D}}$ 具有分块三对角形式：

$$A_{2D}=\begin{pmatrix} D & -rI & 0 & \cdots & 0\\ -rI & D & -rI & \ddots & \vdots\\ 0 & -rI & D & \ddots & 0\\ \vdots & \ddots & \ddots & \ddots & -rI\\ 0 & \cdots & 0 & -rI & D \end{pmatrix},\qquad (27)$$

其中 I 是 $N-1$ 阶单位矩阵，而对角块 D 为

$$D=\begin{pmatrix} 1+4r & -r & 0 & \cdots & 0\\ -r & 1+4r & -r & \ddots & \vdots\\ 0 & -r & 1+4r & \ddots & 0\\ \vdots & \ddots & \ddots & \ddots & -r\\ 0 & \cdots & 0 & -r & 1+4r \end{pmatrix}_{(N-1)\times(N-1)}.$$

于是， $\boldsymbol{A_{2D}}$ 是一个由 $(N-1)\times(N-1)$ 个块组成的分块矩阵，每个块本身又是 $(N-1)\times(N-1)$ 矩阵；因此 $\boldsymbol{A_{2D}}$ 的总阶数为 $m=(N-1)^2$，其对角块 D 描述同一行内相邻内点之间的耦合，故 D 本身是三对角矩阵；相邻块之间的非对角块为 $-rI$，描述上下相邻两行中同一列内点之间的耦合。除这些位置外，其余块均为零块。因此 $\boldsymbol{A_{2D}}$ 是一个大型、稀疏、分块三对角的矩阵。

将 (27) 的第 j 个块行展开，即得到第 j 行内点所满足的方程组：

$$-r\boldsymbol{x}_{j-1}+D\boldsymbol{x}_j-r\boldsymbol{x}_{j+1}=\boldsymbol{b}_j,\qquad j=1,\dots,N-1,$$

其中边界行中越界的 $\boldsymbol{x}_0,\boldsymbol{x}_N$ 由边界条件处理。若写成单个内点 (i,j) 的方程，则为

$$-rU_{i-1,j}^{n+1}-rU_{i,j-1}^{n+1}+(1+4r)U_{i,j}^{n+1}-rU_{i,j+1}^{n+1}-rU_{i+1,j}^{n+1}=b_p,$$

其中行优先编号 $p=(j-1)(N-1)+i$。

这正说明：二维隐式格式中，每个未知量只与自身及上下左右四个邻居耦合，因此矩阵 $\boldsymbol{A_{2D}}$ 的每一行至多有五个非零系数。与一维情形相比，一维矩阵是三对角的；二维矩阵则是分块三对角的，其对角块 D 仍为三对角矩阵，非对角块为 $-rI$。

显式二维格式每步一次五点更新， $O(N^2)$，不需要解方程；隐式格式每步要解一个 $m=(N-1)^2$ 阶的稀疏系统，实际计算多用稀疏直接解或迭代法。这就是二维问题里“显式简单、隐式昂贵”的具体含义。

需要强调的是，二维并不是把一维做两遍。按行优先排成一根长向量之后，矩阵由三对角变为分块三对角，行与行之间的耦合以非对角块 $-rI$ 的形式出现；若改按列优先排列，得到的只是同一个矩阵的一次置换相似变换，谱不变，分块结构随之转置。

3.2.2.3 数值解与收敛阶

数值实验要回答两个问题：格式是否收敛、观测阶是否与理论一致；误差主要由空间离散还是时间离散决定。统一用最大范数度量误差：固定终端时刻 T，记

$$E(h)=\max_{0\le j\le N}\left|U_j^M-u(x_j,T)\right|,\qquad M=\frac{T}{\Delta t},$$

其中 u 为精确解。若 $E(h)\approx Ch^p$，则网格减半后误差约缩小 2^p 倍，观测阶可由相邻两次的比值估计：

$$p_{\text{obs}}=\log_2\frac{E(h)}{E(h/2)}.$$

实验分两步。第一步固定网格比 r，即令 $\Delta t=rh^2$，让 h 逐次减半：空间误差与时间误差同阶下降，测到的是“时空同步加密”的阶。第二步固定网格、只缩小 Δt：此时空间误差是一块不随 Δt 下降的“地板”，必须改用同一网格上半离散方程的精确解作参考，才能单独测出时间方向的阶。

3.2.2.3.1 一维：数值解与收敛阶

取一维模型问题

$$u_t=u_{xx},\qquad u(x,0)=\sin(\pi x),\qquad u(0,t)=u(1,t)=0,$$

其精确解为 $u(x,t)=e^{-\pi^2 t}\sin(\pi x)$。固定 $r=0.4$、$T=0.1$，令 $N=10,20,40,80,160$，相应地 $h=1/N$、$\Delta t=rh^2$、$M=T/\Delta t$。$N=20$ 时数值解与精确解几乎完全重合（图 20(a)）；逐次加密后，最大误差在双对数坐标下按斜率 2 下降（图 20(b)），具体数字列于表 6。

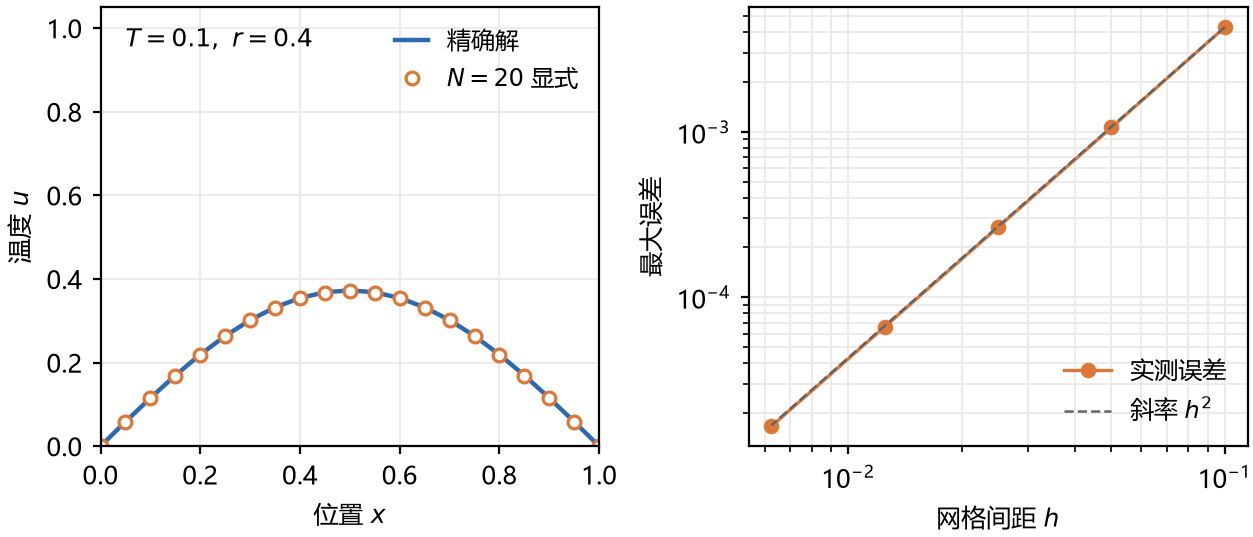


图 20　一维算例：（a）数值解与精确解重合；（b）最大误差以斜率 2 下降

表 6　一维空间收敛（T=0.1，r=0.4）

网格数 N	网格间距 h	时间步 M	最大误差	观测阶
10	1.00×10^{-1}	25	4.29×10^{-3}	—
20	5.00×10^{-2}	100	1.06×10^{-3}	2.01
40	2.50×10^{-2}	400	2.65×10^{-4}	2.00

网格数	网格间距	时间步	最大误差	观测阶
80	1.25×10^{-2}	1 600	6.62×10^{-5}	2.00
160	6.25×10^{-3}	6 400	1.66×10^{-5}	2.00

由表 6， N 每翻一倍误差约降到四分之一（比值 4.05、4.00、4.00、3.99），观测阶稳定在 2.00；首行 2.01 略高，是 $N=10$ 时高阶项尚未衰减。最细网格误差 1.66×10^{-5} 远高于舍入水平，说明误差由截断而非舍入主导。

图 20(b) 的数据点落在斜率 2 的直线上，与表 6 一致；图 20(a) 两条曲线已重合，说明仅凭图形判断精度并不可靠。

严格说，表 6 的观测阶 2 是空间与时间误差共同作用的结果： $\Delta t=rh^2$ 使时间方向的一阶误差也按 h^2 下降，两种误差混在一起；要单独看时间阶，需固定网格。

为了把时间方向的阶单独测出来，固定 $N=40$，让 r 从 $1/2$ 逐次减半。此时与连续精确解比较，空间误差是一块不随 Δt 下降的“地板”，观测阶会被压平；因此参考解换成同一网格上半离散方程的精确解：

$$u_h(x_j,t)=e^{\lambda_h t}\sin(\pi x_j),\qquad \lambda_h=-\frac{4\sin^2(\pi/2N)}{h^2}.$$

结果见表 7：误差逐次减半，观测阶稳定在 1.00，与显式 Euler 的一阶精度相符。这里必须换成半离散参考解，否则空间误差的“地板”会把观测阶压平。

表 7　一维时间收敛：固定网格、只缩步长（N=40，T=0.1）

实际 r	时间步长 Δt	时间步 M	最大误差	观测阶
0.5000	3.13×10^{-4}	320	5.68×10^{-4}	—
0.2500	1.56×10^{-4}	640	2.84×10^{-4}	1.00
0.1250	7.81×10^{-5}	1 280	1.42×10^{-4}	1.00
0.0625	3.91×10^{-5}	2 560	7.09×10^{-5}	1.00
0.0312	1.95×10^{-5}	5 120	3.54×10^{-5}	1.00

一维的结论：空间二阶、时间一阶；时空同步加密时观测阶为 2，固定网格只缩步长时为 1。

3.2.2.3.2　二维：数值解与收敛阶

二维取 $\Omega=(0,1)^2$，初值为两个模态的叠加

$$u(x,y,0)=\sin(\pi x)\sin(\pi y)+\tfrac{1}{2}\sin(2\pi x)\sin(\pi y),$$

两端取零边界。对应地，精确解为

$$u(x,y,t)=e^{-2\pi^2 t}\sin(\pi x)\sin(\pi y)+\tfrac{1}{2}\,e^{-5\pi^2 t}\sin(2\pi x)\sin(\pi y).$$

取 $N=40$、 $r=0.2$、 $T=0.05$，图 21 给出精确解、数值解、误差场与中心剖面。两张热图几乎无差别；误差场在区域中心最大（温度最高、曲率最大），最大误差 3.51×10^{-4}，与 $h^2=6.25\times10^{-4}$ 同量级。

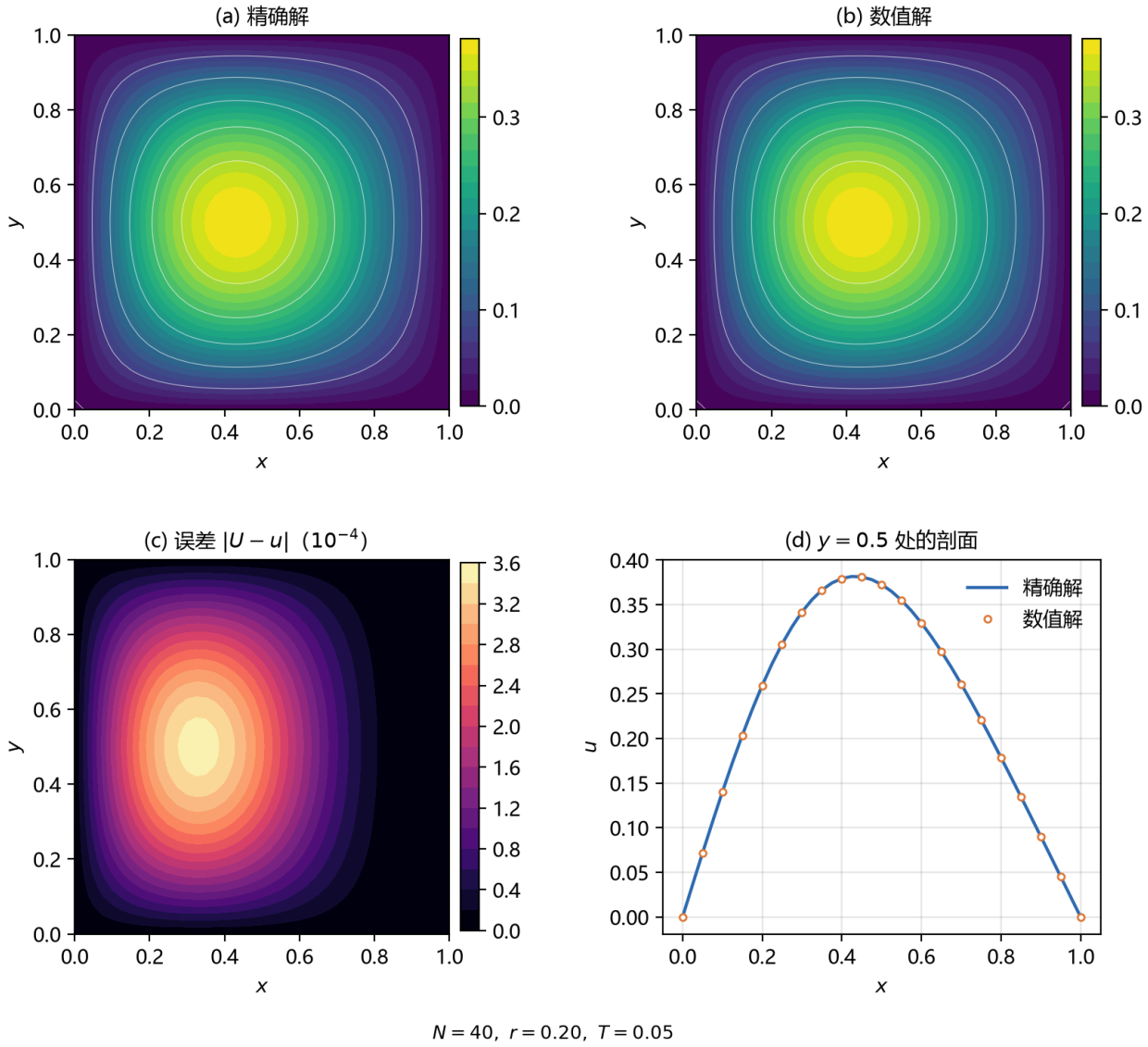


图 21　二维算例：精确解、数值解、误差场与中心剖面

保持 $r=0.2$、令 $N=10\to160$，结果见表 8：误差比值 4.02、3.99、4.01、4.00，观测阶稳定在 2.00；图 22(a) 的斜率同样为 2。

表 8　二维空间收敛（T=0.05，r=0.2）

网格数 N	网格间距 h	时间步 M	最大误差	观测阶
10	1.00×10^{-1}	25	5.63×10^{-3}	—
20	5.00×10^{-2}	100	1.40×10^{-3}	2.01
40	2.50×10^{-2}	400	3.51×10^{-4}	2.00
80	1.25×10^{-2}	1 600	8.76×10^{-5}	2.00
160	6.25×10^{-3}	6 400	2.19×10^{-5}	2.00

同一 N 下二维误差略大于一维（$N=10$ 时 5.63×10^{-3} 对 4.29×10^{-3}）：离散 Laplace 算子是两项二阶差商之和，误差常数相加；这也解释了图 21 中误差场为何在中心最大。

固定 $N=40$、只缩 Δt，结果见表 9 与图 22(b)：误差逐次减半，观测阶 1.00，与一维相同。

表 9　二维时间收敛：固定网格、只缩步长（N=40，T=0.05）

实际 r	时间步长 Δt	时间步 M	最大误差	观测阶
0.25000	1.56×10^{-4}	320	8.44×10^{-4}	—
0.12500	7.81×10^{-5}	640	4.22×10^{-4}	1.00
0.06250	3.91×10^{-5}	1 280	2.11×10^{-4}	1.00
0.03125	1.95×10^{-5}	2 560	1.05×10^{-4}	1.00
0.01562	9.77×10^{-6}	5 120	5.27×10^{-5}	1.00

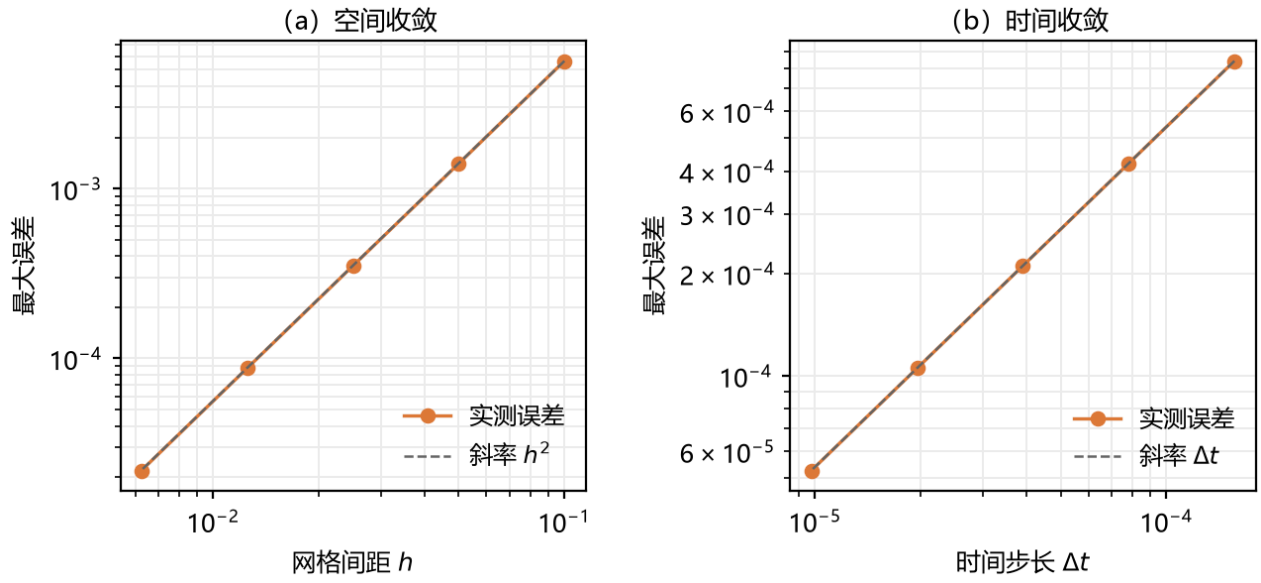


图 22　二维收敛阶：（a）误差随 h^2 下降；（b）固定网格时误差随 Δt 线性下降

二维的代价比一维多一笔：网格加密一倍，未知量变成 4 倍，时间步数也变成 4 倍，总工作量约 16 倍。精度提高一倍，一维的代价约为 8 倍，二维约为 16 倍。这笔账的根源是显式格式的稳定限制，下一节把这条限制说清楚。

故事丨十个人的账本。　老陶的徒弟小满想替师父分担：铁钎烧到哪一段、哪一刻该翻面，不能总靠手背去试。他把钎子量了量，均分成十段，每段正中插一根热电偶。一根连续的钎子，从此变成十个数。

头一晚，他守着炉口记时辰。每隔一小会儿，他先照着上一遍的十个数，把每一段的新温度写下来：本段与左右两段一掺，往邻居那边靠一靠。中间那段若和两边一样热，就原地不动；比两边热，就分一点给邻居，自己凉下来；比两边凉，就反过来。十段各算各的，谁也不许偷看刚写下的新数——师父定过规矩：先抄旧账，再记新账。

一夜下来，账本积了厚厚一叠：一个时辰一页，每页十个数。后来小满发现，若把“谁分给谁多少”也列成一张表，这张表每行只有三个位置有数，其余都是空格；有了它，可以不必一页页抄，一次把整页账算平——只是每算一页都要解一串方程，不如抄账轻快。他把两种法子都记在扉页：一种快，一种稳。

老陶翻着账本笑道：你这不是把一根钎子切成了十个人么？每人只问左右邻居上一遍多少度，问完各记各的账。小满点头。他并不知道，自己刚才做的两件事，一件叫“把空间切成格子”，一件叫“让格子排着队往前走”（图 23）。

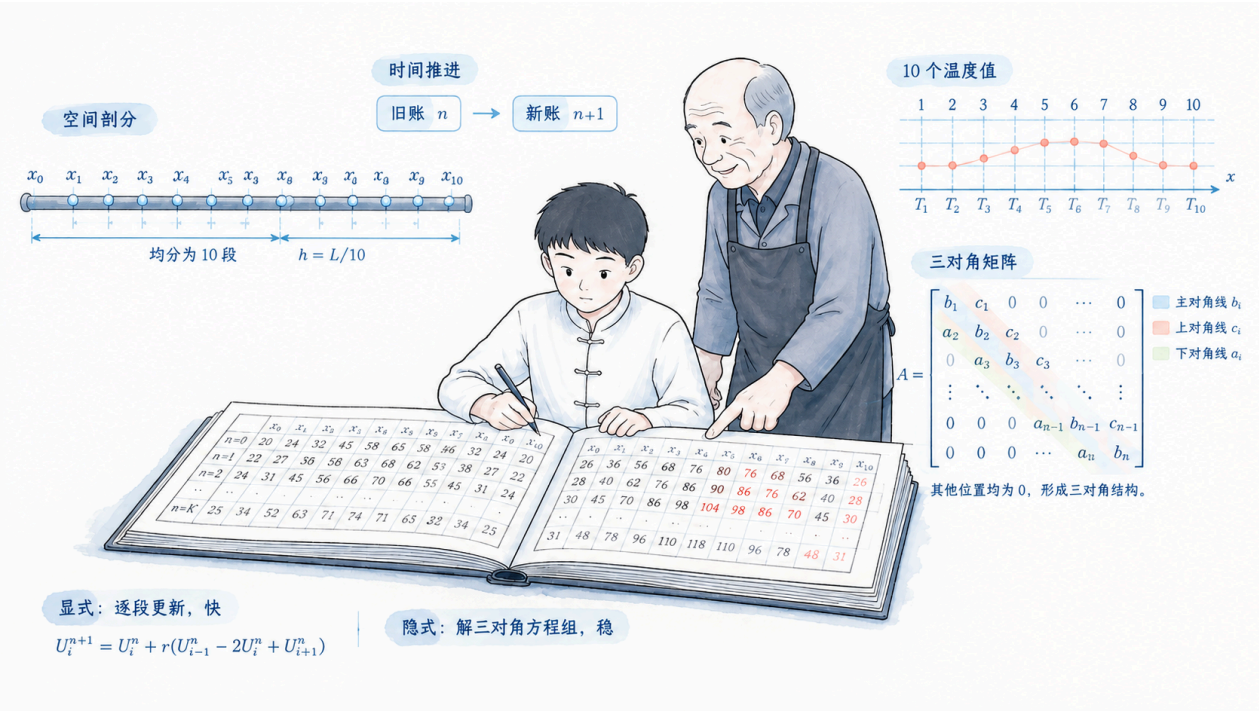


图 23　十个人的账本：空间剖分、逐层更新与三对角矩阵

3.2.3　稳定性的限制与网格实验

3.2.2 节的格式每步只做几次乘加，但时间步不能任意取大：一旦超过某个界限，误差会在迭代中被放大，数值解不再逼近真解，而是自己长成一条振荡的曲线。本节回答两个问题：这条界限在哪里？越过界限后会发生什么、代价有多大？

回答第一个问题有三种视角。局部视角看每个点上的权重，回答“凭什么不能更大”；Fourier 视角把一个误差拆成不同波长的波，回答“哪个波长最先出问题”；矩阵视角看整步迭代的谱半径，回答“最坏情况有多坏”。三者不是三套理论，而是同一个限制的三种说法，最后给出同一个数 $r \le 1/2$（二维为 $r \le 1/4$）。第二个问题用网格实验回答：扫描网格比、观察失稳形态、固定 Δt 加密网格，看误差怎样累积、代价怎样增长（图 24）。

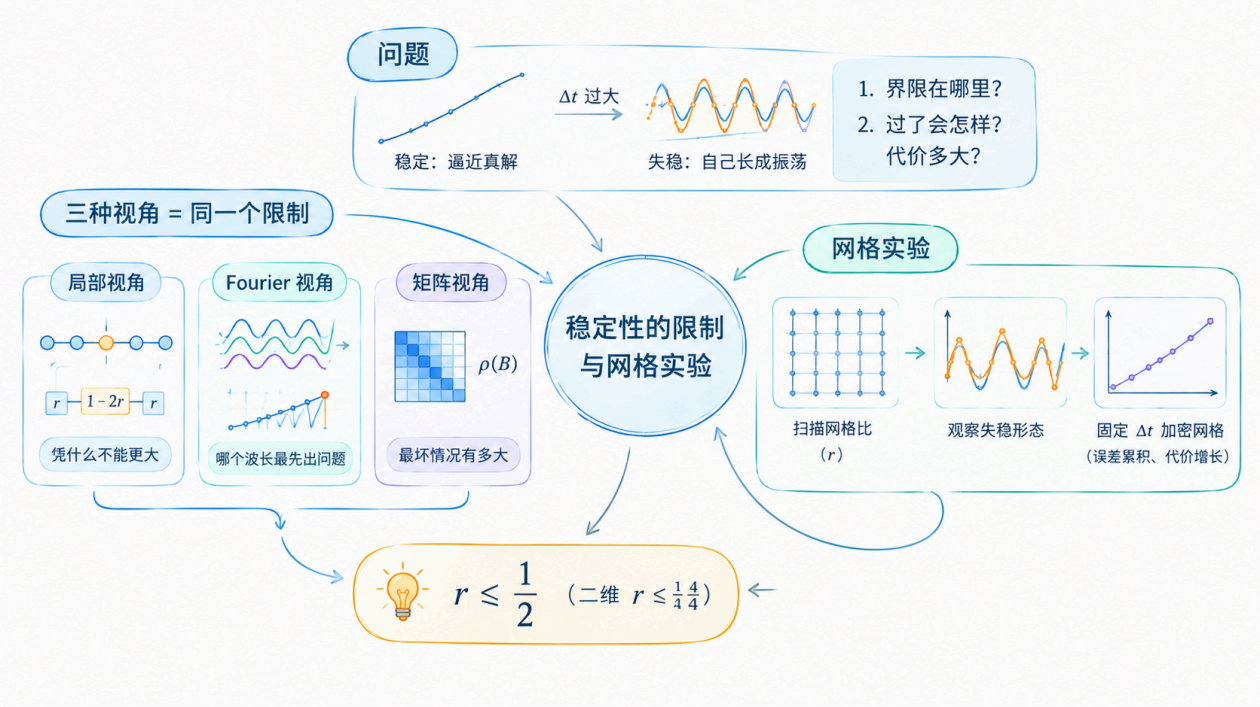


图 24　本节的结构：三种视角给出同一个限制，网格实验检验越过限制的后果

3.2.3.1　稳定性的限制：负权重与离散最大值原理

把显式格式 (25) 写成加权平均：

$$U_j^{n+1} = (1-2r)\,U_j^n + r\,U_{j-1}^n + r\,U_{j+1}^n.$$

$r \le 1/2$ 时三个权重都非负，新值必落在三个旧值的最小值与最大值之间：网格上的最高温度只降不升，最低温度只升不降。这就是离散最大值原理，它与 3.2.1 节的连续版本对应。 $r > 1/2$ 时对角权重 $1-2r$ 变负，新值不再被旧值夹住，一个局部尖峰可以被越抬越高。限制由此出现：

$$r = \frac{\kappa \Delta t}{h^2} \le \frac{1}{2} \qquad \Longleftrightarrow \qquad \Delta t \le \frac{h^2}{2\kappa}.$$

这是一条硬约束：空间步长减半，时间步长必须减到四分之一。二维的权重是 $(1-4r, r, r, r, r)$，五项之和为 1，中心权重非负给出 $r \le 1/4$，恰是一维上界的一半。

负权重给出了“为什么”，但它只检查了一个点上的系数，没有回答“哪个波长最先出问题”。下一节的 Fourier 分析补上这一问。

3.2.3.2　放大因子：逐波长看

把数值解按波长分解，取一个模态 $U_j^n = g^n e^{\mathrm{i}j\theta}$ 代入显式格式 (25)，约去公因子，得

$$g(\theta) = 1 + r\left(e^{-\mathrm{i}\theta} - 2 + e^{\mathrm{i}\theta}\right) = 1 - 4r\sin^2\frac{\theta}{2}.$$

$g(\theta)$ 与步数无关，只取决于波长（用相位角 θ 表示）与网格比。要求所有模态都不增长，即 $|g(\theta)| \le 1$。 $\theta = 0$ 对应常值模态， $g = 1$，不涨不落； $\theta = \pi$ 对应网格上能够表示的最高频模态（相邻节点一步一变色），它的放大因子 $1-4r$ 最不利：

$$|g(\pi)| = |1-4r| \le 1 \qquad \Longleftrightarrow \qquad r \le \frac{1}{2}. \qquad (28)$$

式 (28) 既是必要的也是充分的：$r<1/2$ 时锯齿被压平，$r=1/2$ 时锯齿幅值不变、每步变号，$r>1/2$ 时按 $|1-4r|$ 层层放大。

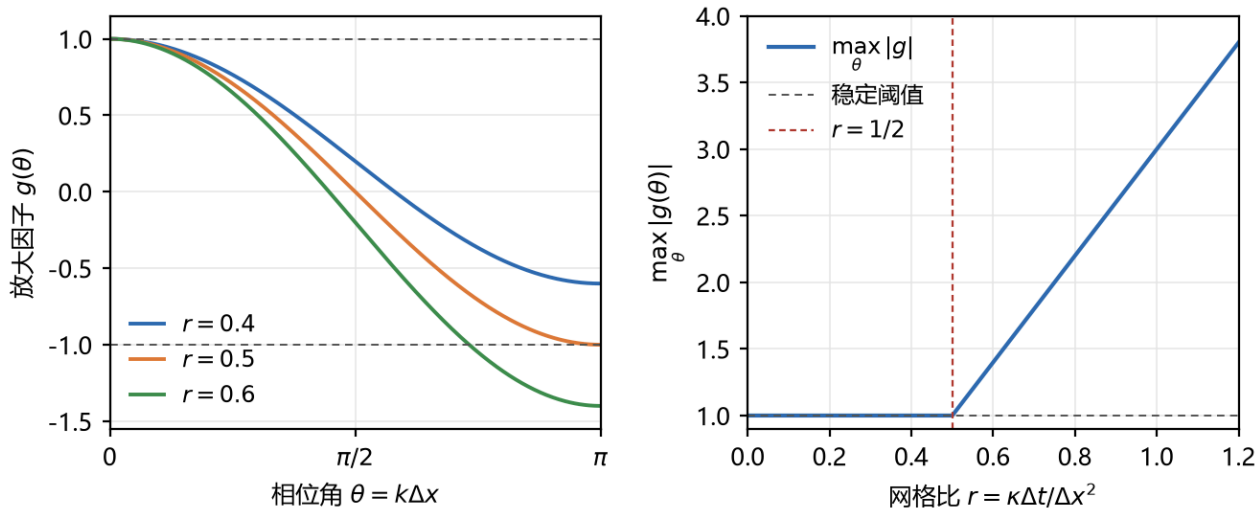


图 25　放大因子：左为 g(θ) 随波长的变化，右为最坏模态随网格比的变化（虚线为阈值 ±1）

图 25 左图给出三条曲线：$r=0.4$ 时整条曲线落在 ±1 之间；$r=0.5$ 时右端恰好触到 -1；$r=0.6$ 时曲线在 $\theta\approx2.28$ 处穿过 -1，此后越往高频越放大。右图把最坏模态单独画出来：$|1-4r|$ 随 r 线性变化，在 $r=1/2$ 处穿过 1，$r>1/2$ 后一路上升。

与负权重相比，放大因子多回答了一件事：不是所有波长同时失稳，而是最靠近网格尺度的那个波长最先出问题。这也解释了后面实验中“光滑初值看起来没事、舍入误差却迟早把解吞掉”的现象。

3.2.3.3　谱半径：整体看

放大因子逐波长地看，矩阵的语言则是一步看全体。显式格式 (26) 的误差满足 $e^{n+1}=Be^n$，即 $e^n=B^ne^0$，是否被放大完全由 B 的幂决定。

定义 3（谱半径）　设 B 为 n 阶方阵，$\lambda_1,\dots,\lambda_n$ 为它的特征值。称

$$\rho(B)=\max_{1\le i\le n}|\lambda_i|$$

为 B 的谱半径。

定理 2（稳定性判据）　若 B 可对角化，则显式格式稳定当且仅当 $\rho(B)\le1$。

证明　设 $B=Q\Lambda Q^{-1}$，$\Lambda=\mathrm{diag}(\lambda_1,\dots,\lambda_n)$，记 $\kappa(Q)=\|Q\|_2\|Q^{-1}\|_2$，则

$$\|e^n\|_2\le\|Q\|_2\|\Lambda^n\|_2\|Q^{-1}\|_2\|e^0\|_2\le\kappa(Q)\,\rho(B)^n\,\|e^0\|_2.$$

若 $\rho(B)\le1$，误差不放大；若 $\rho(B)>1$，取最大特征值对应的特征向量作初值，误差按 $|\lambda_{\max}|^n$ 无限增长。证毕。

对本题，$B=I+rL$ 对称，故 $\kappa(Q)=1$。L 的特征值与特征向量为

$$\lambda_k=-4\sin^2\frac{k\pi}{2N},\qquad v_k=\Big(\sin\frac{jk\pi}{N}\Big)_{j=1}^{N-1},\qquad k=1,\dots,N-1.$$

证明只需一个三角恒等式：Lv_k 的第 j 个分量为 $\sin((j-1)\theta)-2\sin(j\theta)+\sin((j+1)\theta)=-2\sin(j\theta)(1-\cos\theta)$，记 $\theta=k\pi/N$ 即得 $-4\sin^2(\theta/2)\sin(j\theta)$。于是

$$\lambda_k(B) = 1 - 4r\sin^2\frac{k\pi}{2N}, \qquad \rho(B) = \max_{1\le k\le N-1}\left|1 - 4r\sin^2\frac{k\pi}{2N}\right|.$$

$\rho(B)\le 1$ 最坏的情形发生在 $\sin^2 = 1$，即 $1-4r\ge -1$，于是 $r\le 1/2$，与 (28) 一致：矩阵的谱恰好对应所有网格波长的放大因子。二维重算一遍，最负的特征值约为 -8，限制随之减半为 $1/4$。

矩阵视角的价值在于它不依赖 Fourier 模态。对二维、变系数或复杂区域，逐波长的分析不再方便，但误差递推与谱半径仍然成立；只要估出 B 的谱，就能判断稳定性。本题中 B 对称，$\kappa(Q)=1$，谱半径判据还是“紧”的，因此三种视角给出同一个数。

限制还有一处细节值得说明。最大值原理要求 $r\le 1/2$，而谱半径判据只要求 $\rho(B)\le 1$，即

$$r \le \frac{1}{2\cos^2(\pi/2N)},$$

后者略宽：$N=20$ 时约为 0.503。差别来自边界——$\theta=\pi$ 的锯齿模态在两端取值为 ±1，不满足零边界条件，不属于离散正弦基，最负的特征值实际出现在 $k=N-1$；$N\to\infty$ 时两界重合。实用中仍以 $r\le 1/2$ 为准，因为它保证的是更强的性质：离散最大值原理。

以 Courant、Friedrichs、Lewy 的经典论文为起点，这套“网格比要有上界”的思想后来被统称为 CFL 条件[19]。三种视角，同一个数（表 10）：

表 10　同一道限制的三种推导

角度	依据	一维结论	二维结论
负权重	加权平均的系数非负	$r\le 1/2$	$r\le 1/4$
Fourier 模态	$\lvert g(\theta)\rvert\le 1$，最坏在 $\theta=\pi$	$r\le 1/2$	$r\le 1/4$
矩阵谱半径	$\rho(B)\le 1$，最坏特征值 $1-4r$ 或 $1-8r$	$r\le 1/2$	$r\le 1/4$

负权重说明“凭什么不能更大”，放大因子指出“哪个波长最先出问题”[20]，谱半径给出“最坏情况有多坏”。三种说法合起来，限制才算说清楚。

3.2.3.4　网格实验：越过限制会怎样

限制说完了，接下来看实验。实验都在一维模型问题（$u_t=u_{xx}$，$u(x,0)=\sin\pi x$，两端为零）上进行，只在初值上叠加一个幅值 10^{-3} 的网格尺度扰动 $\sin((N-1)\pi x)$。由于格式是线性的，把扰动前后的两个解相减，就单独分离出这个高频模态；它的最大范数除以 10^{-3}，即为实测的增长倍数。

固定 $N=20$、$T=0.05$，让网格比跨过 $1/2$，结果列于表 11。

表 11　网格比扫描：理论与实测（N=20，T=0.05）

目标 r	实际 r	$\lvert g(\pi)\rvert$	理论增长	实测增长	判定
0.40	0.4000	0.600	3.53×10^{-12}	3.66×10^{-12}	稳定
0.45	0.4545	0.818	7.98×10^{-5}	7.98×10^{-5}	稳定
0.50	0.5000	1.000	6.09×10^{-1}	6.09×10^{-1}	稳定
0.55	0.5556	1.222	9.15×10^{2}	9.15×10^{2}	失稳

目标	实际		理论增长	实测增长	判定
0.60	0.6061	1.424	8.27×10^{4}	8.27×10^{4}	失稳

表 11 的五个网格比分成两段：$r \le 0.5$ 的三行，高频扰动被压低或保持不变；$r > 0.5$ 的两行，扰动被放大到 10^{2} 与 10^{4} 量级。“理论增长”按初始扰动的实际波数 $(N-1)\pi/N$ 计算 $|g|^{M}$，与实测值在三位有效数字上相符。

表 11 中 $r = 0.5000$ 一行的两个数字需要解释：$|g(\pi)|$ 记为 1.000，理论增长却是 6.09×10^{-1}。原因是实验叠加的扰动为 $\sin((N-1)\pi x)$，对应 $\theta = (N-1)\pi/N$，并非 $\theta = \pi$——后者的锯齿在两端取 ±1，同样不满足零边界条件。代入真实波数得 $|1 - 4r\sin^{2}(\theta/2)| = 0.9877$，40 步后为 $0.9877^{40} \approx 0.609$，与表中数值一致。

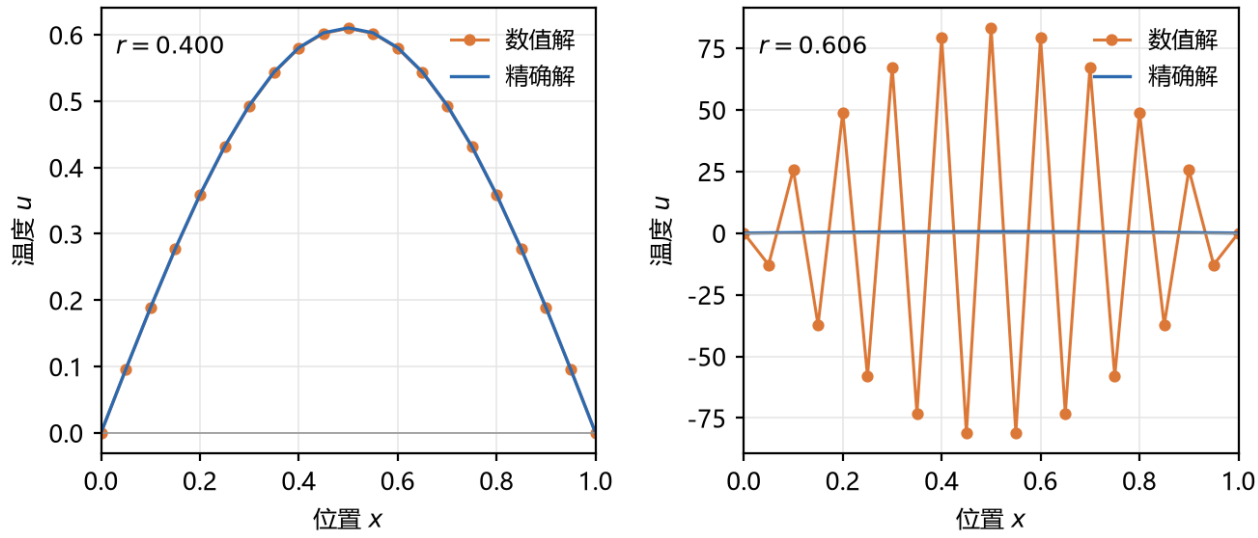


图 26　越过限制后的棋盘振荡：r=0.4 与精确解重合，r=0.606 在 ±83 之间交替

图 26 给出两种网格比下的数值解。$r = 0.4$ 时解与精确解重合；$r = 0.606$ 时解呈典型棋盘格振荡，19 个内点中变号 18 次。值得注意的是，初值 $\sin \pi x$ 本身是 $N = 20$ 下最光滑的模态，它在 $r = 0.606$ 时仍按 0.9851 衰减——只拿光滑初值做实验，解不会立刻爆炸。被放大的是高频扰动：浮点舍入误差里就有这一分量，它迟早会把解吞掉。稳定性是格式的性质，不是初值的性质。

另一个后果是“加密反而更糟”。固定 $\Delta t = 0.002$、$T = 0.02$，网格数从 10 加倍到 80，r 从 0.20 涨到 12.80，幅值被放大到 10^{14} 量级。要保持 $r \le 1/2$，必须让 Δt 与 h^{2} 同比例缩小（图 27）：

$$h \to \frac{h}{2} \quad\Longrightarrow\quad \Delta t \to \frac{\Delta t}{4}, \qquad M \to 4M.$$

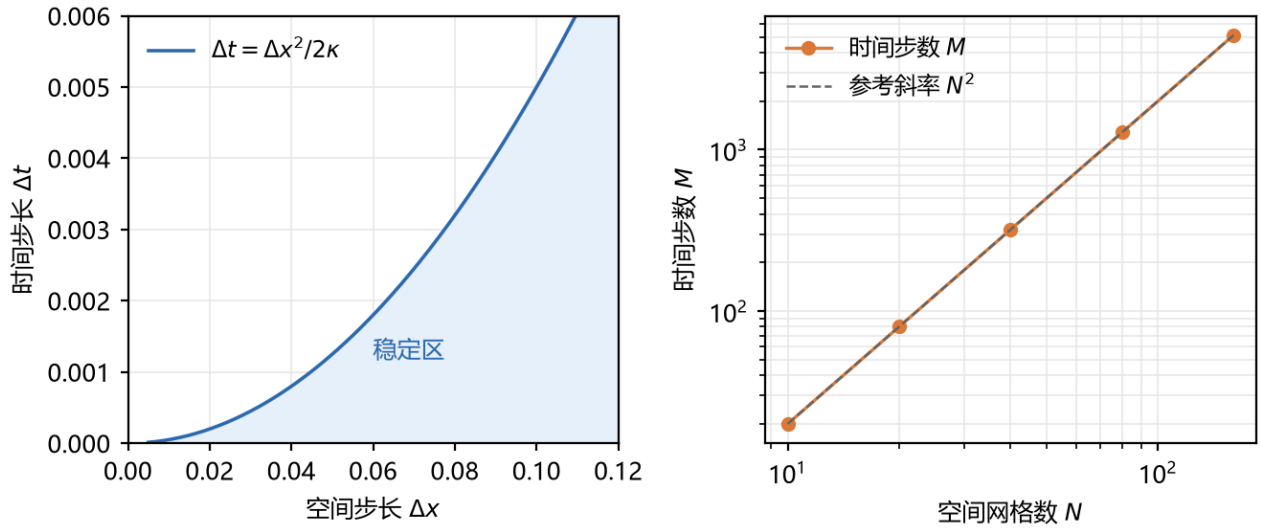


图 27　稳定区域与代价：左为可取的 Δt 上界，右为步数随 N 的增长

网格数每加倍一次，时间步数增到 4 倍；二维还要再乘上未知量变 4 倍这一笔。把代价写成数量级：一维的网格数 $\propto h^{-1}$、时间步数 $\propto h^{-2}$，总工作量 $\propto h^{-3}$；二维的未知量 $\propto h^{-2}$、时间步数 $\propto h^{-2}$，总工作量 $\propto h^{-4}$。精度提高一倍，代价分别涨到 8 倍与 16 倍。

若这一代价难以接受，可以改换时间离散。把右端整体取在新一层（隐式格式），模态代入后得到 $g(\theta)=1/\big(1+4r\sin^2(\theta/2)\big)$，它对一切 $r>0$ 都落在 $(0,1]$ 内；若取前后两层的平均，则得 Crank–Nicolson 格式。三者的对照列于表 12。

表 12　三种时间离散的放大因子

格式	$g(\theta)$	稳定条件	时间精度
显式（FTCS）	$1-4r\sin^2(\theta/2)$	$r\le 1/2$	一阶
隐式（后向 Euler）	$1/\big(1+4r\sin^2(\theta/2)\big)$	无条件	一阶
Crank–Nicolson	$\dfrac{1-2r\sin^2(\theta/2)}{1+2r\sin^2(\theta/2)}$	无条件	二阶

无条件稳定并不意味着可以随意取大步长：步长增大，时间离散误差随之增大（显式与隐式均为一阶，Crank–Nicolson 为二阶）。它解除的只是 Δt 必须跟随 h^2 的约束，并未取消精度要求。

故事 | 锯齿爬上账本。　为了看清钎尖附近的温度，小满把格子从十段加到八十段，却没改时间步。头几轮无事，到了后半夜，账本上冒出一排锯齿：相邻两段一正一负，越往后越尖。

老陶看了一眼说：格子细了，步子也得跟着细——格子缩一半，步子得缩到四分之一。小满只缩了格子，没缩步子，账上的数越滚越大，锯齿被放大到百万亿倍。

他换成隐式格式，锯齿平了，步子也能放大；老陶又提醒：稳不等于准，步子太大，时间上的误差照样找上门。小满把两条记在扉页：显式格式里，步子最多只能跟着格子平方的一半走（二维是四分之一）；越过这条线，先坏的不是精度，是稳定（图 28）。

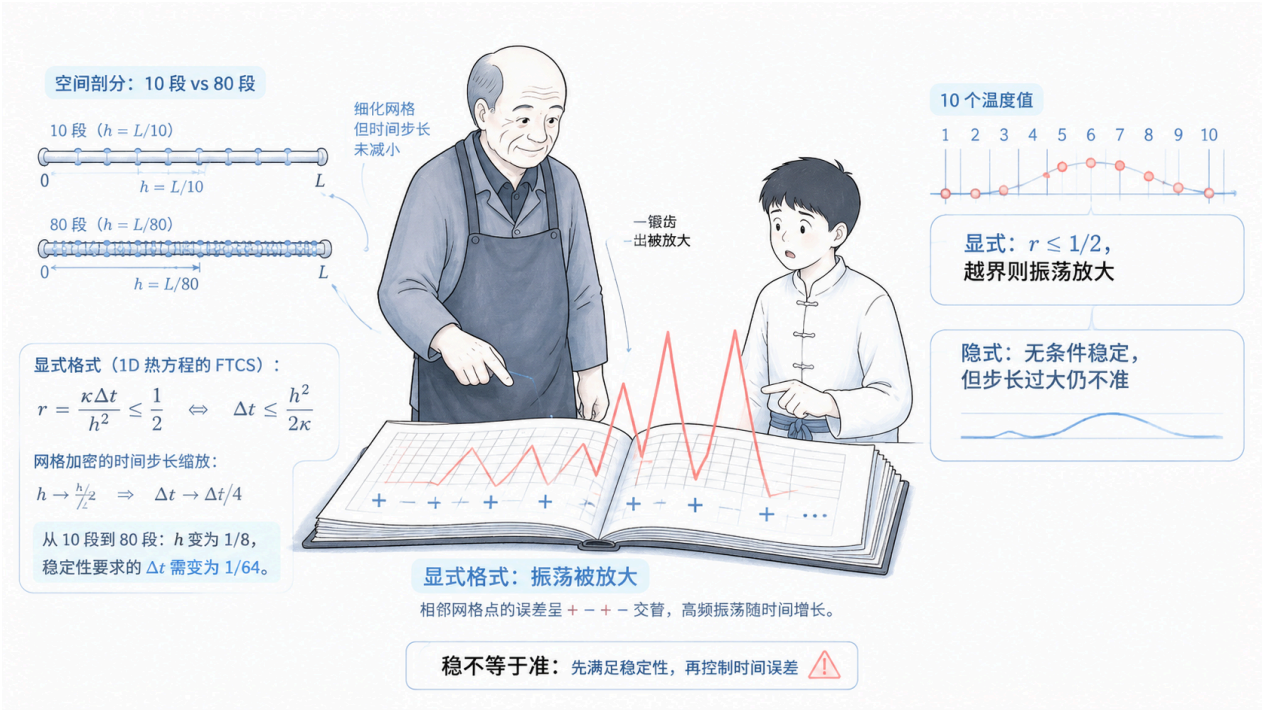


图 28　锯齿爬上账本：网格加密而步长未减，振荡被放大

3.2.4　趣味编程——“热传导方程”修复破损二值图像

图 29(a) 是一张 588×588 的二维码照片，被人用灰色笔划了几道，又盖上一个图案；手机扫不出来，解码器连位置都找不到。输入是一张灰度图，输出是一张 0/1 图（黑模块记 0，白模块记 1），要解决的问题就是把被覆盖的模块补回来。

二维码是一个 0/1 矩阵。完好的像素已知，被覆盖的像素未知；已知像素构成边界，未知像素是待求区域。这就是图像修复（image inpainting）——基于 PDE 的做法由 Bertalmío、Sapiro 等人在 2000 年的 SIGGRAPH 论文里系统提出[30]，Chan 与 Shen 随后把它写成了严格的变分模型[31]。

这里只用其中最简单的一档，叫膜模型（membrane model，也叫调和修复）：在缺失区域 Ω_{miss} 里找一张“最光滑”的曲面，让它贴住已知像素构成的边界。写成变分问题，就是最小化 Dirichlet 能量

$$\min_{U} \mathcal{J}(U) = \int_{\Omega_{\text{miss}}} |\nabla U|^2 \,\mathrm{d}x\,\mathrm{d}y, \qquad U = U^0 \text{ 于 } \partial\Omega_{\text{miss}}.$$

对 $\mathcal{J}$ 取变分，得 Euler–Lagrange 方程 $\Delta U = 0$。也就是说，膜模型的解就是 Laplace 方程在缺失区域上的 Dirichlet 问题，而它正是热方程的稳态：

$$\Delta_h U = 0 \quad \text{于 } \Omega_{\text{miss}}, \qquad U = U^0 \quad \text{于 } \partial\Omega_{\text{miss}}.$$

这一步把“补图”换成了“解一个椭圆边值问题”：不用先验知识，也不用训练，信息全部来自那条边界。热方程在这里不是比喻——让缺失区域按热方程一直扩散下去，温度不再变化的那一刻，就是 $\Delta U = 0$ 的解。

用显式热流来求这个稳态：取 $r = 1/4$，缺失点每步取四个邻居的平均，已知像素每步重置为原值，

$$U_{i,j}^{n+1} = \begin{cases} \dfrac{1}{4}\left(U_{i-1,j}^{n} + U_{i+1,j}^{n} + U_{i,j-1}^{n} + U_{i,j+1}^{n}\right), & (i,j) \in \Omega_{\text{miss}}, \\ U_{i,j}^{0}, & (i,j) \notin \Omega_{\text{miss}}. \end{cases} \tag{29}$$

这正是 Jacobi 迭代；$r=1/4$ 恰是二维显式格式的限制（最大值原理意义下的上界），所以迭代一定收敛到离散调和函数[31]——收敛性与 3.2.3 节那道限制是同一件事。

热方程给出的是连续灰度，而二维码只有黑白两种模块，所以最后还要取一次阈值。这一步只是把结果读成 0/1 交给解码器，不属于模型：修复本身由 Laplace 方程决定，阈值只影响“读”的方式。取 $1/2$，是因为灰度会收敛到笔画两侧的加权平均，过半即判为白：

$$Q_{i,j}=\begin{cases}1, & U_{i,j}>1/2,\\ 0, & U_{i,j}\le 1/2.\end{cases}\qquad (30)$$

至于计算速度：若嫌显式迭代太慢，可以换成隐式格式或两层平均的 Crank–Nicolson 格式[21]。

具体做法分四步：第一步，估计缺失区域（笔画是“颜色居中、局部平坦”的灰，用局部标准差筛出，再做闭运算与膨胀）；第二步，按 (29) 迭代，每步把已知像素重置；第三步，按 (30) 二值化；第四步，用标准解码器读取。

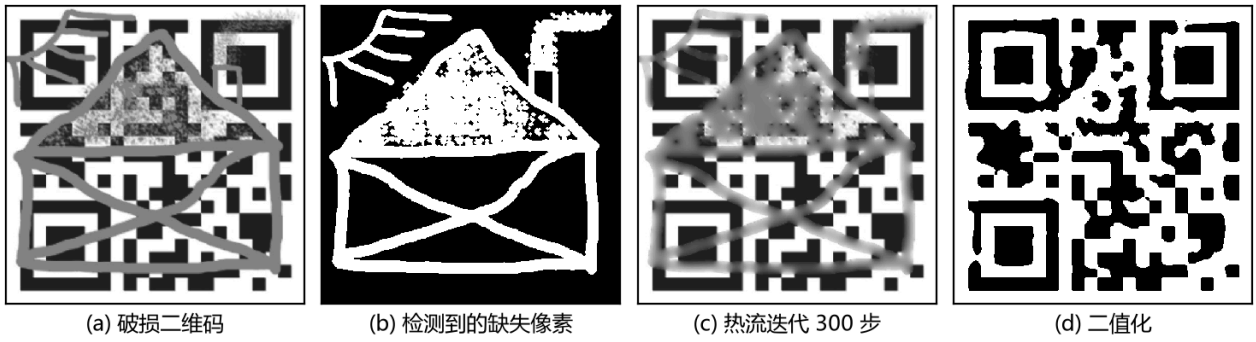


图 29　热方程修复二维码：（a）破损原图；（b）掩膜（35%）；（c）300 步后的灰度场；（d）二值化结果。解码为 “I like coffee”

修复前，笔画盖住了定位角与大片数据区；修复后，定位角恢复成完整的黑白方块。缺损 35% 里有很大一块是中部那块不携带数据的装饰图案，真正影响解码的笔画只占一部分。表 13 给出迭代步数与解码结果：20 步不够，40 步才行——热量要从笔画两侧“走”到中央，笔画宽十几个像素，而热扩散一次只走一个像素。

表 13　迭代步数与解码结果（缺损 35%）

迭代步数	0	10	20	40	100	300	600
能否解码	否	否	否	是	是	是	是

表 13 中步数的量级可以用扩散尺度来估计：热扩散的典型距离随步数按 $\sqrt{n}$ 增长，与笔画半宽（十余像素的一半）相当所需的步数为 $7^2\approx 50$，与“20 步不够、40 步可解”相符。

必须说明：用 PDE 调和延拓修复二维码，并不是主流方法。

另一方面，缺失区域并非越大越保险。区域越大，Laplace 解越平坦，原有细节被一并抹去；把完好的模块也标为缺失，等于把可用信息变成未知，只会加重解码器的负担。掩膜应尽量贴近真实缺损。

近年的工作多用深度学习[33][34]。PDE 修补的优点是简单、可解释、无需训练；缺点只能补“平滑”的缺失，补不了纹理。调和延拓也只保证“最光滑”，不保证“正确”：被覆盖模块原本是什么，信息已经丢失，最后要靠二维码自带的 Reed–Solomon 纠错码兜底，按版本与等级可容忍约 7%–30% 的码字错误

[32]。

3.2.5 热方程的今天

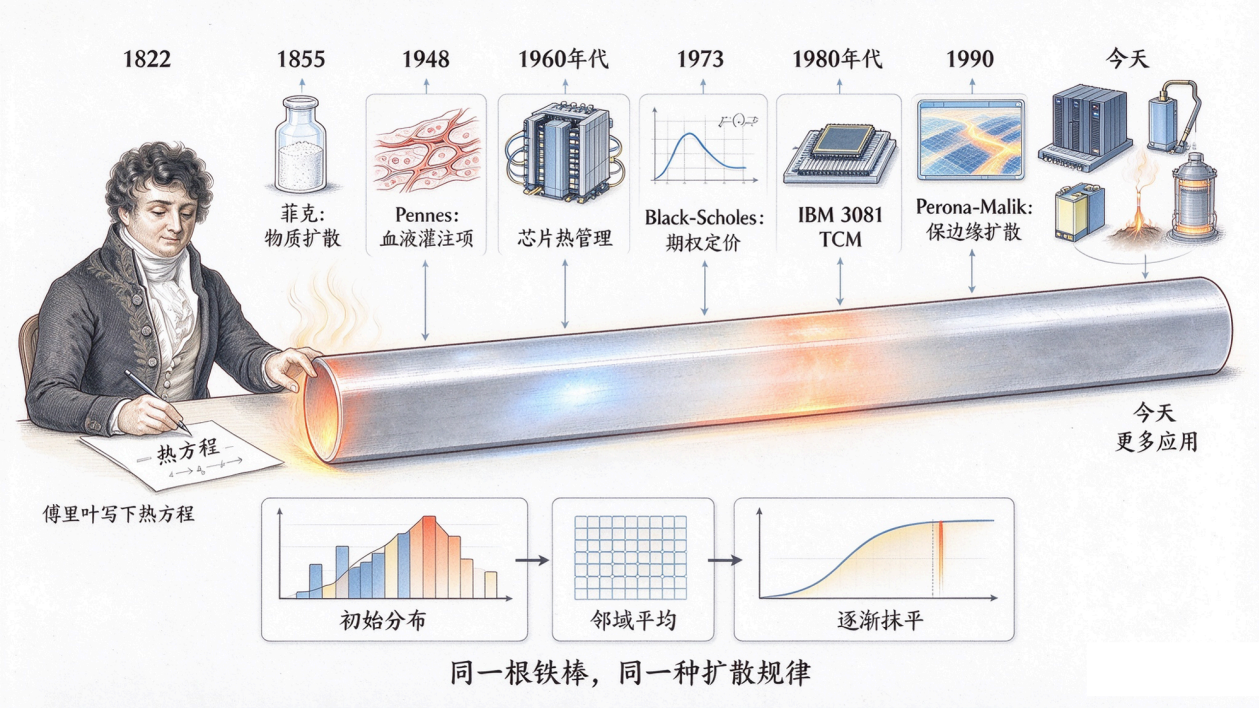


图 30　热方程的两百年：从铁棒到数据中心

自从 1822 年傅里叶写下热方程之后，这个简单的抛物型方程就悄悄走出了铁棒、火炉的世界，不断换上新身份，去解决各个领域的难题。

1855 年，菲克发现它还能描述物质的扩散，于是用它来刻画溶质的慢慢渗透。到了 1948 年，Pennes 给热方程添上了血液灌注项，让它走进生物体内，用来模拟人体组织的传热[28]。

时间来到上世纪 60 年代，计算机产业兴起，IBM 等公司开启了芯片热管理的探索，最早的大型计算机依靠传导模块带走热量。1973 年，Black 和 Scholes 又改造了这个方程，得到期权定价公式[27]，热方程就此闯入金融领域。

80 年代，IBM 3081 登场，它的导热模块靠着弹簧活塞把芯片紧紧压在水冷板上，单个模块就能扛下几百瓦的热量。1990 年，Perona 和 Malik 想出了新点子：让扩散系数跟着梯度变化，用改良后的扩散方程做图像处理，平滑噪声的同时，守住图片里清晰的边缘[26]。

到了今天，热方程的舞台变得更大。数据中心用上冷板液冷，单机柜的功率攀升到几十千瓦；锂电池、激光焊接、地热勘探、返回舱的热防护，背后都有它的身影。

跨越两百年，它万变不离其宗：本质都是让一个初始分布，靠着“邻居互相平均”慢慢演化。我们在网格上写下的离散格式，正是这个古老规律简洁的数字化表达。

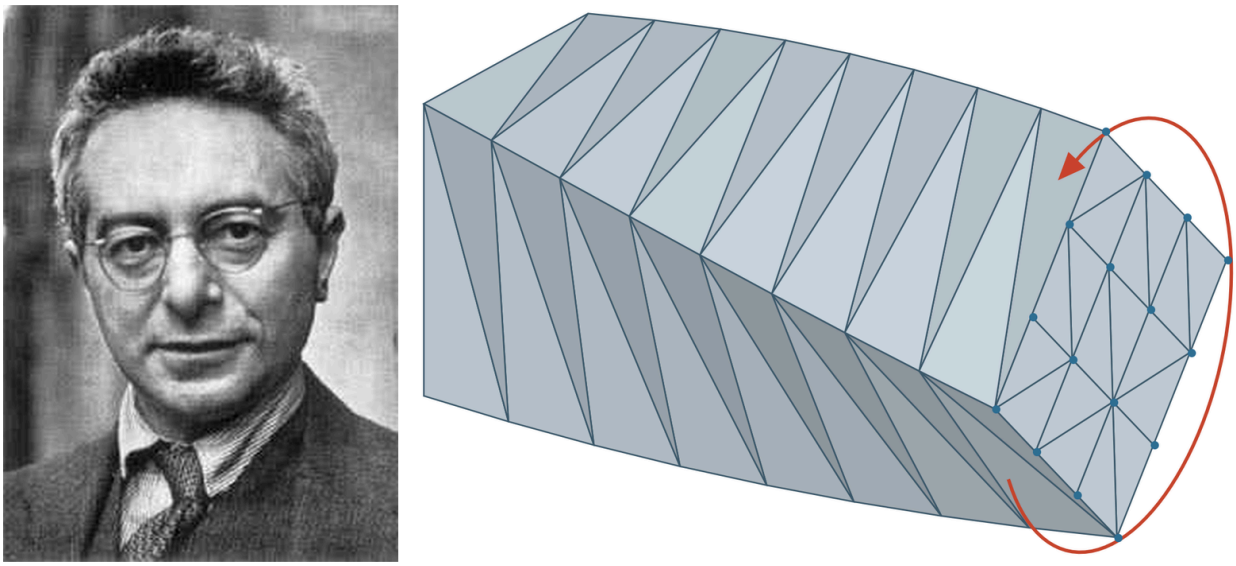
第三节（3.3）　有限元：弱形式与 Galerkin 方法

3.3.0　引子：一部手机的半毫秒

把一部手机举到胸口，松手。它落到地上只要半秒；真正决定命运的，是触地后的半毫秒。外壳凹下去又弹回来，应力波在玻璃、铝合金和电池之间来回反射，某一条倒角上的拉应力先越过极限，裂纹从那里开始。这个过程太快，肉眼看不见，高速摄像机也只能拍到几十帧。

工程师的办法是把它搬进计算机。手机被切成几十万个小四面体，每个小块写一个刚度矩阵，再按节点编号拼成一个巨大的方程组；解出来，就知道哪一角先裂。同一套算法也在桥上、机翼上、鱼竿上工作。它的名字叫有限元（finite element）。今天，汽车碰撞、手机跌落、心脏支架、摩天大楼的抗风，底层都有一张网格和一次装配。

一部手机跌落的半毫秒：外壳被切成几十万个小四面体，应力波在网格间来回反射，先越过极限的那一角先裂。



Richard Courant（1888–1972，左）与他求解的圣维南扭转问题（右）。1943 年，他用三角形网格求解圣维南扭转问题，并讨论了收敛性，这被公认为有限元的数学雏形[[49]](#a1-ch3-ref49)。左图来源：MacTutor History of Mathematics Archive；右图为自绘示意：方杆扭转 48°，表面与端面都是三角形网格。

有限元不是一个人发明的。它更像三条河：数学家从能量出发，计算数学家从残差出发，工程师从一根杆的刚度出发。三条河各自流了半个世纪，最后汇进同一本矩阵账。本节沿着汇合后的路线走一遍：先把微分方程改写成弱形式，再用一顶顶“帽子”把它压成矩阵，然后按编号装配、上边界条件、量误差。最后，用一根三节鱼竿把整本账从头到尾记一遍。

先说清楚范围：本节只做最简单的一维线性元。二维三角形只作选讲，梁的弯曲是四阶方程，这里不碰。这不是偷懒。一维情形已经包含了有限元的全部关键动作——变分、投影、装配、边界、误差；把一维的账记明白，二维只是每个节点多几个邻居。

把这五步串起来，就是图 31：连续问题先改写成弱形式，再剖网格、造分片函数、组装矩阵，最后施加边界并求解。

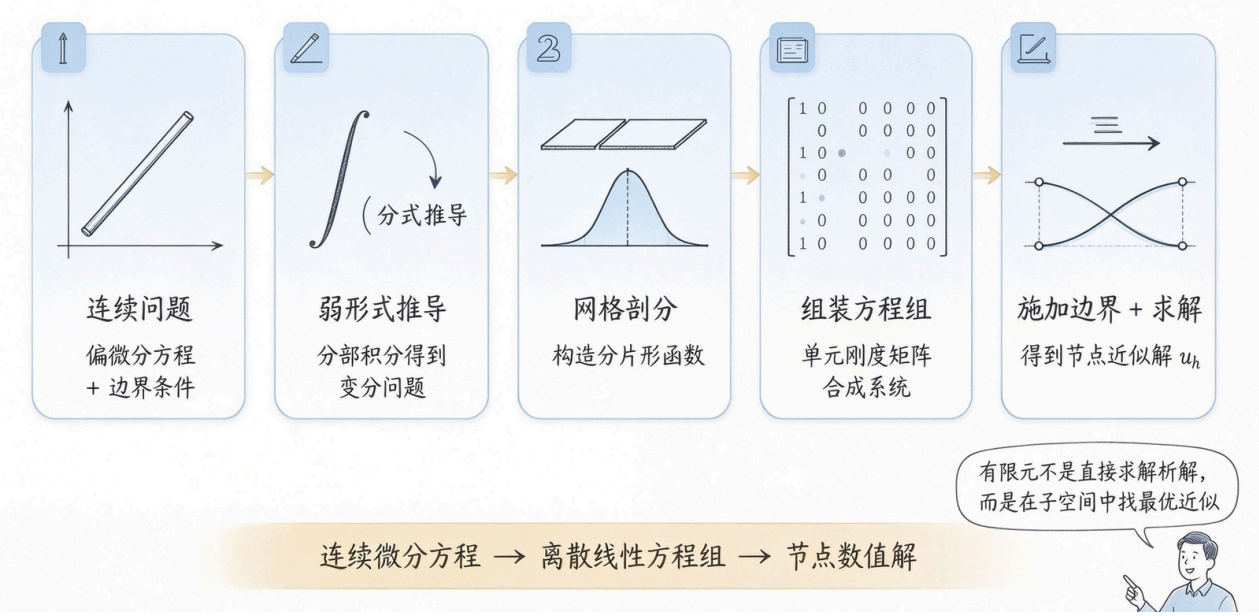


图 31　有限元的五步：从连续问题到节点解

3.3.1　三条血脉

3.3.1.1　三个人的三条路

1956 年，西雅图。波音公司的工程师把一架飞机的机翼拆成几百个小三角形。每个三角形写一个小刚度矩阵，再按节点编号拼成整体矩阵；解出矩阵，就知道机翼在载荷下怎样变形。这是工程界用“拆成小块、再拼回去”分析整架飞机的开端。

要看清这件事，得把时钟拨回半个世纪。

1908 年，哥廷根。Ritz 的肺结核已经拖了很久。他想的不是“怎样把方程解出来”，而是“怎样让能量最低”：把未知函数写成一组试函数的线性组合，代入能量泛函，让系数自己找到最小值。解方程于是变成求极值。1909 年，Ritz 去世，年仅 31 岁。他没看到这个方法成为工程标准，但“Ritz 法”留在了每一本有限元教材里[39]。

1915 年，彼得格勒。Galerkin 是铁路与结构工程师，面对的是杆与板的弹性平衡。他手里没有能量泛函，于是换了一个问法：把近似解代入方程，得到残差；要求残差与每一个基函数正交。这就是 Galerkin 方法[40]。它不要求算子对称，适用范围比 Ritz 法宽。

把近似解 $u_n = \sum_{j=1}^{n} c_j \varphi_j$ 代入 $Lu = f$，得到残差 $R = Lu_n - f$。Galerkin 的条件是

$$(R, \varphi_i) = 0, \qquad i = 1, \dots, n.$$

也就是：残差在每个基函数上的投影为零。没有能量泛函，照样能算。

1943 年，纽约。Courant 在《美国数学会公报》上发表《平衡与振动问题求解的变分方法》[41]。他把区域剖分成三角形，在每个三角形上用分片线性函数，求解圣维南扭转问题，并讨论了网格加密时的收敛性。这就是“Courant 三角形”。

三个人彼此不知道对方的工作。Ritz 从能量出发，Galerkin 从残差出发，Courant 从三角形出发；三条路看起来毫不相干，却都在做同一件事：把无限维的微分方程，压成有限维的线性方程组。

3.3.1.2 汇合：从机翼到名字

1956 年的机翼并不是凭空出现的。Turner、Clough、Martin 与 Topp 发表《复杂结构的刚度与挠度分析》[42]。他们把机翼离散成三角形与四边形“单元”，对每个单元写出一个小刚度矩阵，再按节点编号组装成整体刚度矩阵。这是工程界独立发明的有限元。

1960 年，Clough 在匹兹堡的第二届 ASCE 电子计算会议上第一次使用“finite element”这个词[43]，名字从此固定下来。1960 年代，Argyris、Zienkiewicz 等人把方法系统化[45]，有限元迅速成为航空、土木、机械行业的标准工具。1973 年，Strang 与 Fix 的《An Analysis of the Finite Element Method》给出了严格的误差理论[44]，Céa 引理与 Aubin–Nitsche 对偶论证这才把工程方法与泛函分析彻底接上。

三条线在这里接上。Ritz 和 Galerkin 说明“为什么可以这样算”：把无限维方程投影到有限维子空间，能量最小与残差正交给出同一个方程组。波音的结构分析说明“怎么算才高效”：先算单元，再按编号累加，稀疏矩阵与快速求解都从这里来。Courant 的三角形说明“复杂区域怎么办”：网格可以贴着边界走，不必是矩形。

把三条线按年代排开，顺序更清楚：1908 年 Ritz 在哥廷根把能量极小写成线性方程组；1915 年 Galerkin 在彼得格勒用残差正交绕开能量；1943 年 Courant 在纽约把三角形网格带进变分问题；1956 年波音把单元装配用于机翼；1960 年 Clough 给它起名；1973 年 Strang 与 Fix 补上误差理论。工程在前，理论在后，名字最后才出现。

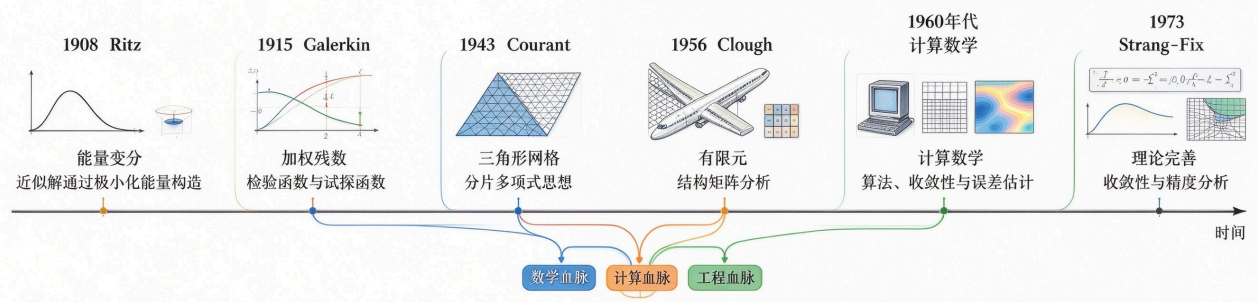


图 32　三条血脉的时间线：数学、计算与工程在 1950—1960 年代汇合

三条线各有各的出发点。数学家从能量出发：Euler 和 Lagrange 把“能量取极值”写成变分法，Rayleigh 用一组试函数去逼近，Ritz 把它变成解线性方程组；在他们手里，解方程就是找最低点。计算数学家从残差出发：Galerkin 不管能量，只要近似解在每个基函数方向上的残差为零；Strang 和 Fix 后来补上误差理论。工程师从结构出发：Courant 把区域切成三角形，波音把机翼切成单元，Clough 给这个方法起名；在他们手里，解方程就是把许多小矩阵拼成一个大矩阵。

三条线各有局限：Ritz 要有能量，Galerkin 要选基函数，Courant 要处理网格。合在一起，局限才被补上。这三根支柱缺一根，有限元都站不稳。

1956 年波音的工程师未必读过 Ritz 和 Galerkin，Courant 也未必关心机翼。三条线在各自的行业里长出来，最后在“把方程变成矩阵”这件事上碰头。

这也是有限元有意思的地方：它不是先有一套完整理论，再去工程里试用；理论、算法、结构分析几乎同时成熟，才在 1960 年代被叫同一个名字。

图 33 把三条线并排画在一起，底部汇到同一个方框。合流之后的活很具体：选基函数，算单元矩阵，按编号装配，施加边界，估计误差。后面的几节按这个顺序走。

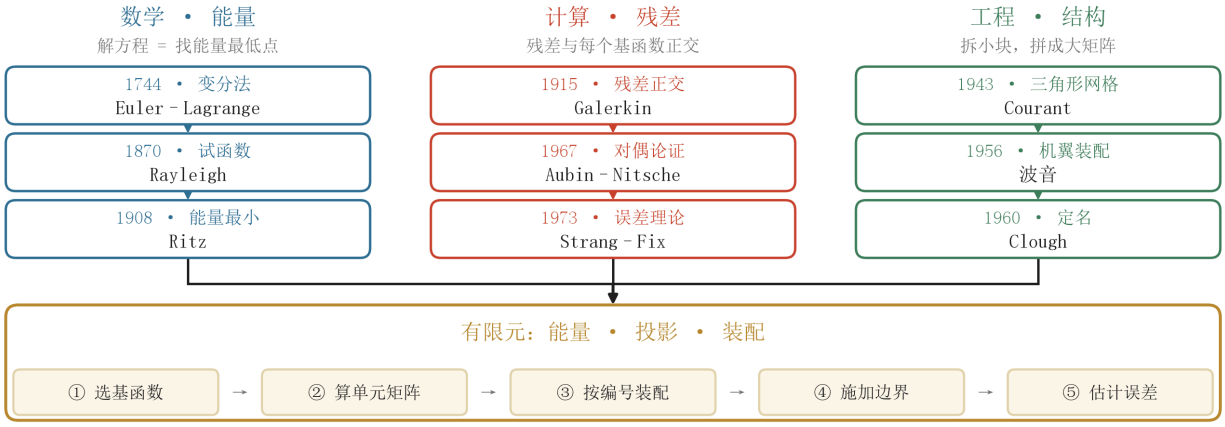


图 33　三条血脉：能量、残差与装配，最后汇成有限元

3.3.2　从强形式到弱形式

3.3.2.1　强形式与它的软肋

考虑一维变系数两点边值问题：

$$-(a(x)u'(x))' = f(x), \qquad 0 < x < 1, \qquad u(0) = u(1) = 0, \qquad (31)$$

其中 $a(x) \geq a_0 > 0$ 是材料系数（例如导热系数或弹性模量乘截面积），f 是载荷。本节始终假设 $a \in L^\infty(0,1)$，存在常数 $0 < a_0 \leq a_1$ 使 $a_0 \leq a(x) \leq a_1$ 几乎处处成立，$f \in L^2(0,1)$；讨论强形式时再额外假设 $a \in C^1[0,1]$、$f \in C^0[0,1]$、$u \in C^2[0,1]$。这就是“强形式”。它要求 u 二阶可导，au' 一阶可导，逐点满足方程。

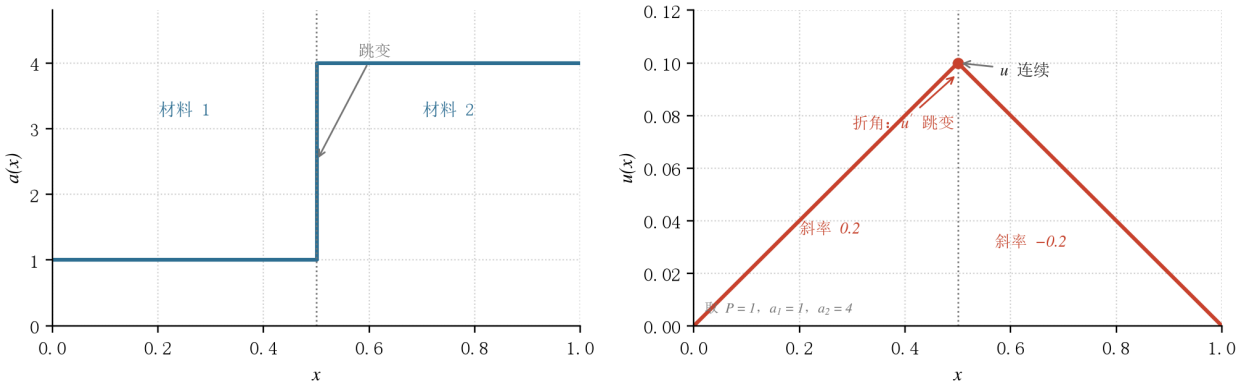


图 34　材料界面：a 跳变，u 连续而 u’ 折角，u’’ 在接口不存在

这个要求有时太挑食。一根由两段不同材料粘成的杆，a 在接口处跳变；接口上还可能作用集中力。图 34 是这种情形：左图 a 在 $x = 1/2$ 处从 1 跳到 4，右图位移 u 连续，u' 在接口折了一下，u'' 在接口根本没有定义。强形式要求逐点满足 $-(au')' = f$，可接口处连 u'' 都不存在；弱形式把导数分一半出去，只要求 $u, u' \in L^2$，这道坎就绕过去了。

3.3.2.2　分部积分与边界项：谁进空间，谁进右端

弱形式的做法是：不要求方程在每一点成立，只要求它在“平均意义”下成立。任取一个检验函数 v，两边相乘，在 $(0,1)$ 上积分：

$$\int_0^1 -(au')'v\,\mathrm{d}x = \int_0^1 fv\,\mathrm{d}x.$$

对左边做分部积分：

$$\int_0^1 -(au')'v\,\mathrm{d}x = \int_0^1 au'v'\,\mathrm{d}x - \big[au'v\big]_0^1. \qquad (32)$$

一次分部积分，把 u 的二阶导数分了一半给 v：左边只剩 u' 与 v' 的乘积，对两者的光滑性要求都降到一阶。代价是多出一个边界项。这一步的几何意思很清楚：把求导的负担分一半给检验函数；图 35 把它画成一次“分摊”。

现在挑检验函数。若要求 $v(0)=v(1)=0$，边界项立刻消失，得到

$$\int_0^1 au'v'\,\mathrm{d}x = \int_0^1 fv\,\mathrm{d}x, \qquad \forall v \in H_0^1(0,1). \qquad (33)$$

这就是弱形式。 $H_0^1(0,1)$ 是“一阶导数平方可积、在两端取零”的函数空间；它比 C^2 大得多，允许导数有间断。

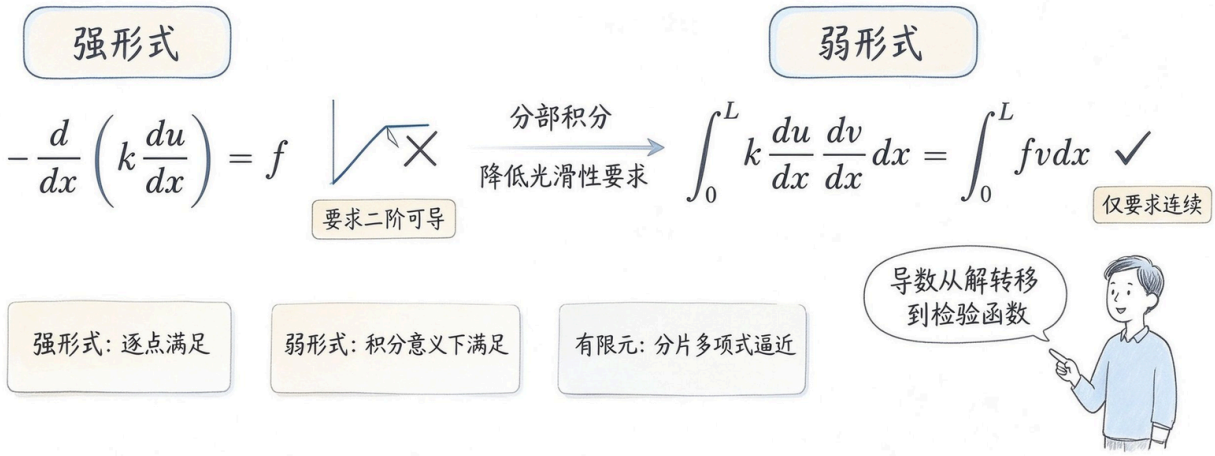


图 35　强形式到弱形式：分部积分把导数分给检验函数

把双线性形式记作

$$a(u,v)=\int_0^1 au'v'\,\mathrm{d}x, \qquad (f,v)=\int_0^1 fv\,\mathrm{d}x,$$

弱形式就可以写成一句很短的话：求 $u \in V = H_0^1(0,1)$，使

$$a(u,v)=(f,v), \qquad \forall v \in V. \qquad (34)$$

注意“ $\forall v$”三个字的分量。它不是对某一个 v 成立，而是对所有检验函数成立；正是这一点，把逐点方程换成了积分方程。

定义 1（弱解与函数空间） 记 $L^2(0,1)$ 为平方可积函数空间，$H^1(0,1)=\{v\in L^2(0,1): v'\in L^2(0,1)\}$，其中 v' 是弱导数；其范数与半范数为

$$\|v\|_{H^1}^2=\|v\|_{L^2}^2+\|v'\|_{L^2}^2, \qquad |v|_{H^1}=\|v'\|_{L^2}.$$

$H_0^1(0,1)$ 是 $C_c^\infty(0,1)$ 在 H^1 范数下的闭包；对连续代表元，它等价于“ $v(0)=v(1)=0$”。称 $u\in V=H_0^1(0,1)$ 为弱形式 (34) 的弱解，若 $a(u,v)=(f,v)$ 对一切 $v\in V$ 成立。

定义 2（连续性与强制性） 若存在常数 $M>0$，使

$$|a(u,v)| \le M\|u\|_{H^1}\|v\|_{H^1}, \qquad \forall u,v\in V,$$

则称 a 在 V 上连续；若存在常数 $\alpha>0$，使

$$a(v,v) \ge \alpha\|v\|_{H^1}^2, \qquad \forall v\in V,$$

则称 a 在 V 上强制。对 $a(u,v)=\int_0^1 au'v'\,\mathrm{d}x$，由 $a_0\le a\le a_1$ 得

$$|a(u,v)| \le a_1|u|_{H^1}|v|_{H^1} \le a_1\|u\|_{H^1}\|v\|_{H^1},$$

故 $M=a_1$。强制性的常数要用下面的 Poincaré 不等式。

引理 1（Poincaré 不等式） 对 $v\in H_0^1(0,1)$，

$$\|v\|_{L^2} \le |v|_{H^1}, \qquad \|v\|_{H^1} \le \sqrt{2}\,|v|_{H^1}.$$

证明 先取 $v\in C_c^\infty(0,1)$。由微积分基本定理，

$$v(x)=\int_0^x v'(s)\,\mathrm{d}s,$$

于是 $|v(x)|^2 \le x\int_0^x |v'(s)|^2\,\mathrm{d}s \le \int_0^1 |v'(s)|^2\,\mathrm{d}s$。对 x 从 0 到 1 积分，得 $\|v\|_{L^2}^2 \le |v|_{H^1}^2$。$C_c^\infty$ 在 H_0^1 中稠密，取极限即得一般情形。证毕。

由引理 1，$a(v,v)\ge a_0|v|_{H^1}^2 \ge \frac{a_0}{2}\|v\|_{H^1}^2$，故强制常数可取 $\alpha=a_0/2$。

命题 1（弱形式与强形式等价） 设 $a\in C^1([0,1])$。若 $u\in C^2[0,1]$ 满足强形式 (31)，则 u 满足弱形式 (34)；反之，若 $u\in H_0^1(0,1)$ 满足 (34) 且 $u\in C^2[0,1]$，则 u 满足 (31)。

证明 前半段就是 (32) 的分部积分：取 $v\in H_0^1$，边界项为零，得 (33)。后半段反过来：对任意 $v\in C_0^\infty(0,1)$，由 (34) 与分部积分，

$$\int_0^1 \big[-(au')'-f\big]v\,\mathrm{d}x=0.$$

由变分法基本引理，$-(au')'=f$ 在 $(0,1)$ 内处处成立；$u\in H_0^1$ 又给出 $u(0)=u(1)=0$。证毕。

边界项没有消失的情形，恰好对应另一类边界条件。设右端是 Neumann 条件

$$a(1)u'(1)=g,$$

左端仍是 $u(0)=0$。此时 $v(0)=0$，但 $v(1)$ 不为零，边界项只剩 $gv(1)$：

$$a(u,v)=(f,v)+g\,v(1), \qquad \forall v\in V_0=\{v\in H^1(0,1):v(0)=0\}.$$

这就是 Neumann 边界在弱形式里的位置：它不进函数空间，而是进右端。Dirichlet 条件则相反，必须写进试探空间与检验空间。

把这句话说死一点：Dirichlet 是本质边界条件，Neumann 是自然边界条件。本质条件要“请进空间”，自然条件只需“请进右端”。这就是为什么有限元程序里，Dirichlet 要改矩阵，Neumann 只改载荷向量。

第三类是 Robin 条件：端点与外界换热，流出的热流与端点温度成正比。设

$$a(1)u'(1)+\sigma u(1)=g,$$

其中 $\sigma \ge 0$ 是换热系数。乘 v 分部积分后，边界项变成 $[g-\sigma u(1)]v(1)$，于是弱形式右端多出 $gv(1)$，左端多出 $\sigma u(1)v(1)$：

$$a(u,v)+\sigma u(1)v(1)=(f,v)+g\,v(1).$$

Robin 条件既进右端，也进矩阵：它给端点对角元加 σ，给右端加 g。这正是“自然边界条件”的一般形态——Neumann 是 $\sigma=0$ 的特例，Dirichlet 则必须进空间。

非齐次 Dirichlet 也不难。设 $u(0)=g_0$、$u(1)=g_1$，取一个满足边界值的 lift 函数 u_g，令 $u=u_0+u_g$，其中 $u_0\in H_0^1$。代入弱形式，把已知项移到右边：

$$a(u_0,v)=(f,v)-a(u_g,v),\qquad \forall v\in H_0^1.$$

未知量仍是 u_0，只是右端多了一项。

3.3.2.3 能量泛函与 Galerkin 投影

当 a 对称、正定时，弱形式还可以从能量角度理解。定义

$$J(v)=\frac{1}{2}a(v,v)-(f,v)=\frac{1}{2}\int_0^1 a(v')^2\,\mathrm{d}x-\int_0^1 fv\,\mathrm{d}x.$$

定理 1（能量极小与弱形式等价） 设 $a(\cdot,\cdot)$ 对称、连续（常数 M）、强制（常数 α），$f\in V^*$，$J(v)=\frac{1}{2}a(v,v)-(f,v)$。则 u 满足弱形式 (34)，当且仅当 u 是 J 在 V 上的最小值点；若 a 强制，最小值点唯一。

证明 对任意 $w\in V$，展开

$$J(u+w)=J(u)+\big[a(u,w)-(f,w)\big]+\frac{1}{2}a(w,w).$$

若 u 满足 (34)，中括号为零，于是

$$J(u+w)=J(u)+\frac{1}{2}a(w,w)\ge J(u),$$

等号只在 $a(w,w)=0$ 时成立；由强制性，这蕴含 $w=0$。故 u 是严格最小值点。

反过来，设 u 是最小值点。对任意 $w\in V$ 与实数 t，函数 $\phi(t)=J(u+tw)$ 在 $t=0$ 取最小，故 $\phi'(0)=0$。直接计算

$$\phi'(0)=a(u,w)-(f,w),$$

由 w 的任意性即得 (34)。证毕。

定理 1 有一个很具体的图像。把 V_h 里的函数写成 $v_h=c_1\varphi_1+c_2\varphi_2$，$J$ 就是 c_1,c_2 的二次函数：二次项正定，所以它是一个开口朝上的碗；Galerkin 方程 $Ac=b$ 恰好是碗底的条件 $\nabla J=0$。图 36 画的是 $N=3$ 时的等高线，碗底就是有限元解。

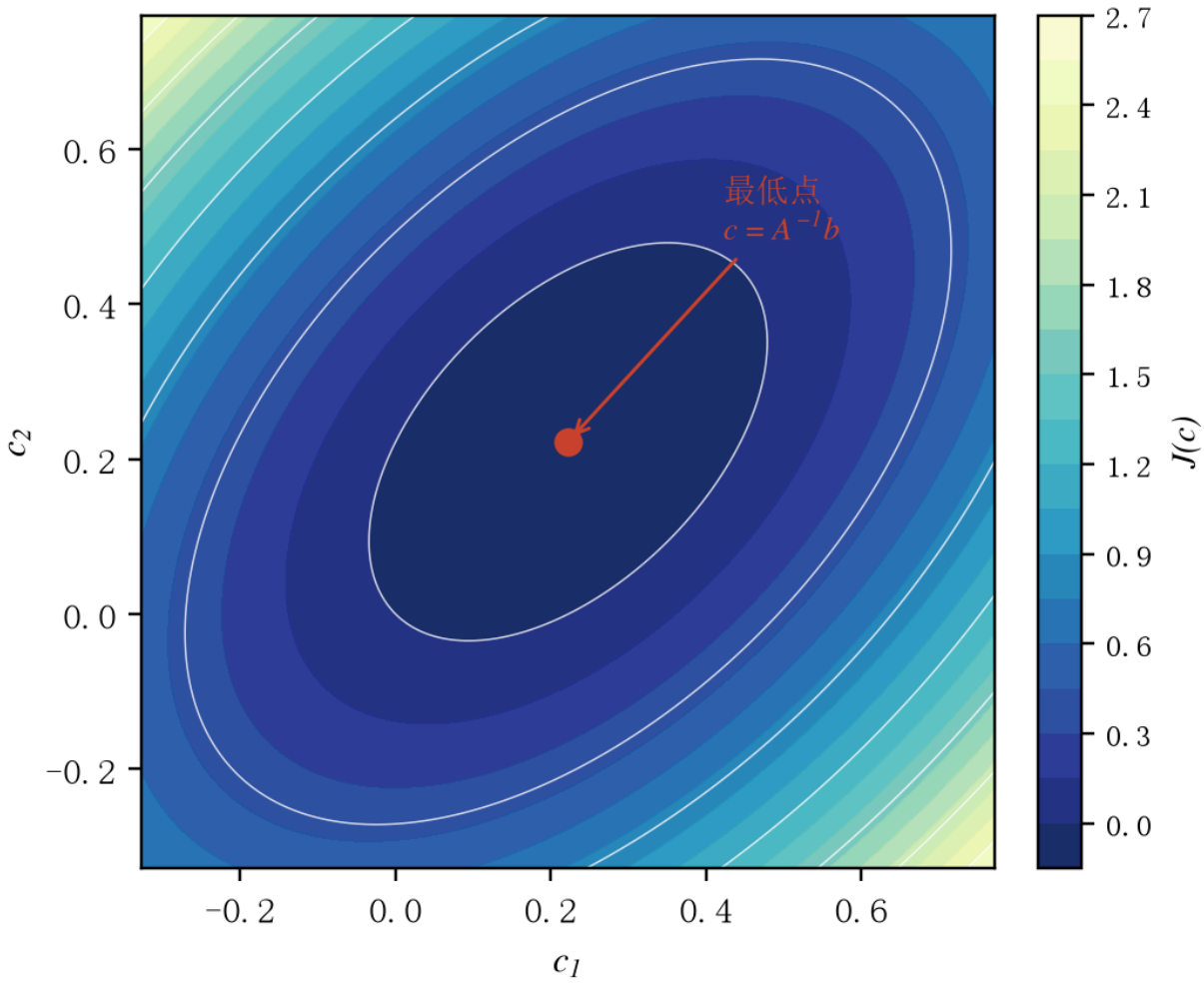


图 36　能量泛函的碗：Galerkin 解落在最低点，∇J=0 与 Ac=b 是同一句话

这个定理解释了 3.3.1.1 节那句话：对称情形下，Ritz 的“能量最小”与 Galerkin 的“残差正交”是同一件事。它也为有限元提供了存在唯一性的入口。

定理 2（Lax–Milgram，引用）　设 V 是 Hilbert 空间，$a: V \times V \to \mathbb{R}$ 连续、强制，$f: V \to \mathbb{R}$ 连续线性。则存在唯一的 $u \in V$，使 $a(u,v) = f(v)$ 对一切 $v \in V$ 成立，且有估计 $\|u\|_V \le \|f\|_{V^*}/\alpha$。证明见 [46][47]。

把 $f(v) = (f,v)$ 代入，弱形式 (34) 在 a 连续、强制时存在唯一弱解；Céa 引理中的常数 M/α 也因此有明确上界。

弱形式仍在无限维空间里。Galerkin 的一步，是取一个有限维子空间 $V_h \subset V$，在 V_h 里找近似解。

设 $\dim V_h = n$，基函数为 $\varphi_1, \dots, \varphi_n$。令

$$u_h(x) = \sum_{j=1}^{n} U_j \varphi_j(x),$$

要求 u_h 对每个基函数都满足弱形式：

$$a(u_h, \varphi_i) = (f, \varphi_i), \qquad i = 1, \dots, n.$$

图 37 是这一步的几何图像：把精确解 u 投影到有限维子空间 V_h，误差 $u - u_h$ 与 V_h 正交。

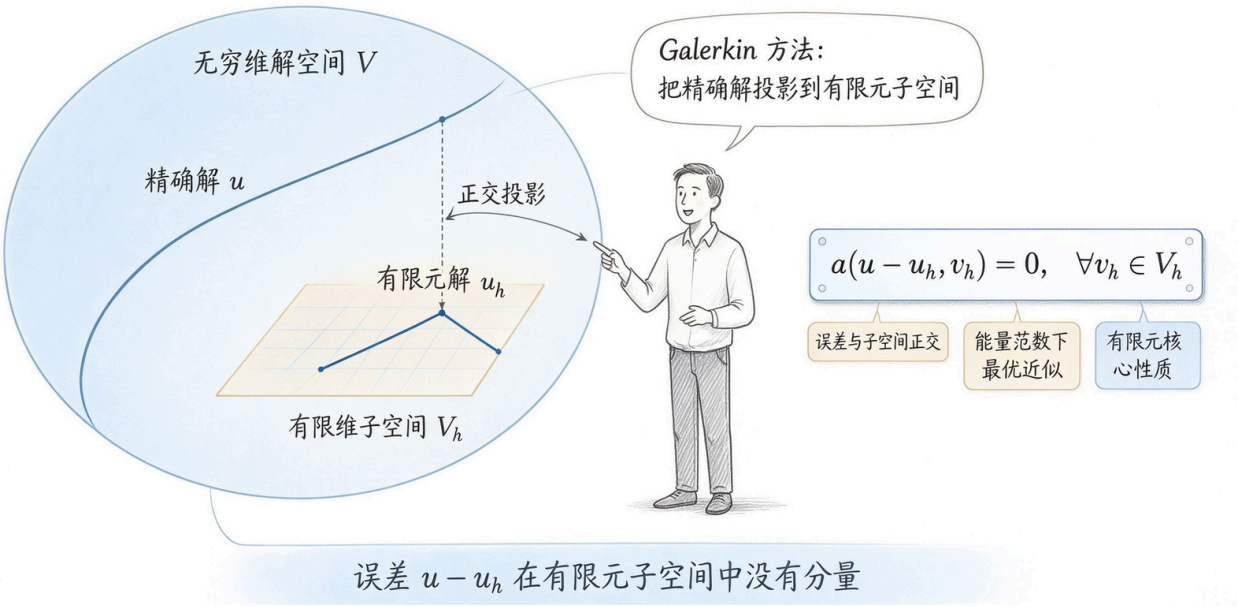


图 37　Galerkin 投影：误差与有限元子空间正交

把 u_h 的展开代入，得线性方程组

$$AU=b, \qquad A_{ij}=a(\varphi_j,\varphi_i), \qquad b_i=(f,\varphi_i).$$

矩阵 A 称为刚度矩阵，b 称为载荷向量。A 对称：$A_{ij}=a(\varphi_j,\varphi_i)=a(\varphi_i,\varphi_j)=A_{ji}$；$A$ 正定：$U^{\mathsf T}AU=a(u_h,u_h)>0$（$u_h\neq 0$）。于是方程组有唯一解。

命题 2（刚度矩阵的对称正定性）　设 $\{\varphi_1,\dots,\varphi_n\}$ 线性无关，a 对称、强制（常数 α），$A_{ij}=a(\varphi_j,\varphi_i)$。则 A 对称正定，且

$$U^{\mathsf T}AU=a(u_h,u_h)\geq \alpha\|u_h\|_{H^1}^2.$$

证明　对称性由 $a(u,v)=a(v,u)$ 直接得到。对任意 $U=(U_1,\dots,U_n)^{\mathsf T}\neq 0$，令 $u_h=\sum_j U_j\varphi_j$；由基函数线性无关，$u_h\neq 0$。于是

$$U^{\mathsf T}AU=\sum_{i,j}U_iA_{ij}U_j=a(u_h,u_h)\geq\alpha\|u_h\|_{H^1}^2>0.$$

故 A 正定。证毕。

剩下的问题只有一个：这 n 个基函数怎么选？选得越“局部”，矩阵越稀疏；选得越“光滑”，精度越高。有限元的答案是：选帽子。

3.3.3　线性单元与刚度矩阵

3.3.3.1　帽函数与 2×2 刚度矩阵

把区间 $[0,1]$ 分成 N 个单元，节点 $x_i=ih$，$h=1/N$。在内部节点 x_i 上定义

$$\varphi_i(x)=\begin{cases}\dfrac{x-x_{i-1}}{h}, & x_{i-1}\leq x\leq x_i,\\[2mm] \dfrac{x_{i+1}-x}{h}, & x_i\leq x\leq x_{i+1},\\[2mm] 0, & \text{其它}.\end{cases}$$

它的图像是一顶三角形的“帽子”：在 x_i 处高 1，在左右两个邻居处降到 0，再往外一直是 0。更准确地说，$\varphi_i(x_j)=\delta_{ij}$——在自家节点上取 1，在别人家的节点上取 0。

故事 | 帽子铺。　帽子铺的老板只做一种帽子：每顶帽子有三个落脚点，中间高 1，两边落回 0。顾客问：能不能做一顶盖住整条街的帽子？老板说：可以，但那就不是帽子了，是一块布；有限元要的是许多小帽子，不是一块大布。

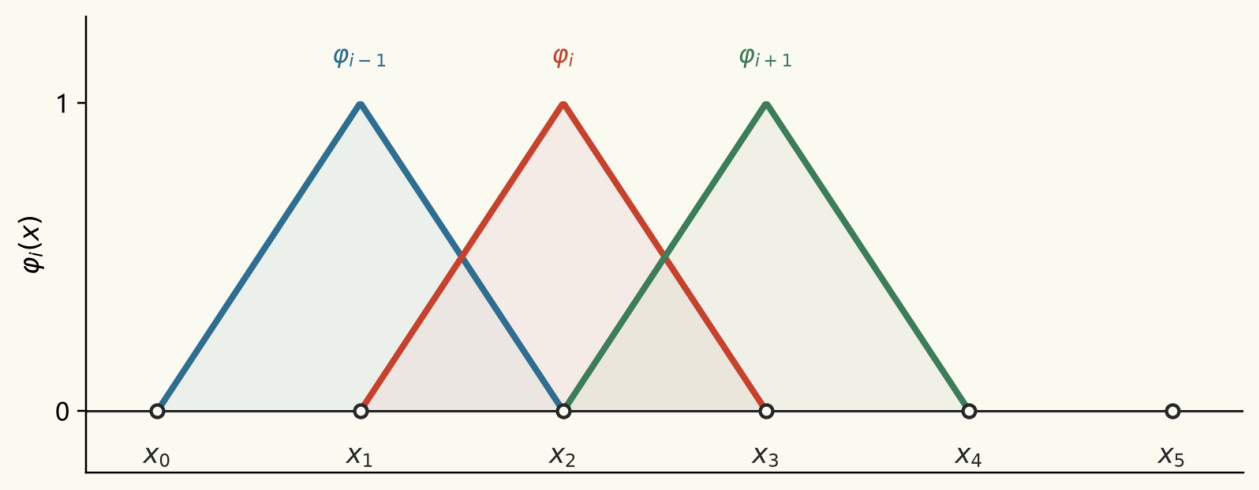


图 38　三顶相邻的帽函数

把其中一顶单独拿出来看，它的两支斜率、面积与支撑区间就一目了然。

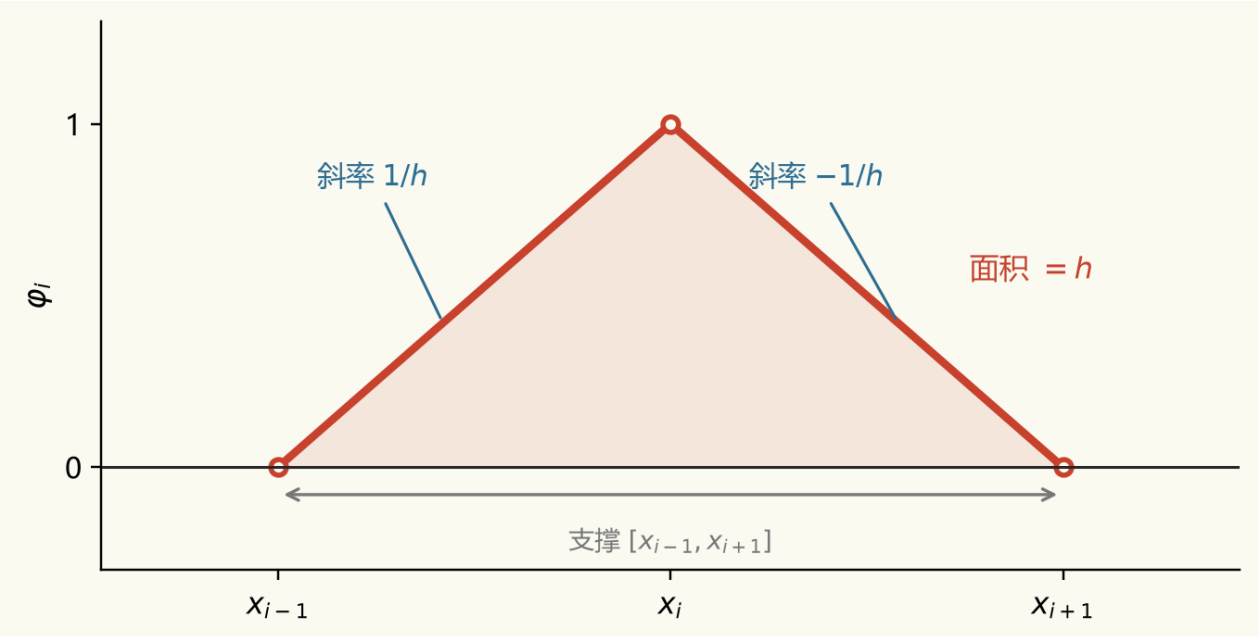


图 39　单顶帽子的斜率、面积与支撑

定义 3（线性有限元空间）　取 $V_h=\operatorname{span}\{\varphi_1,\dots,\varphi_{N-1}\}\subset V$，维数 $N-1$。为什么不要 φ_0 与 φ_N？因为它们对应的节点在边界上，而 $V_h\subset H_0^1$ 要求边界值为零；把边界帽子去掉，剩下的函数在两端自动为零。

帽子的局部性带来一个直接后果：$a(\varphi_j,\varphi_i)=0$，只要 $|i-j|>1$。两个相隔两个节点以上的帽子，支撑集不相交，乘积的积分为零。于是 A 是三对角矩阵。$N=10^4$ 时，矩阵有 10^8 个位置，非零元只有约 3×10^4 个。

在单元 $e=[x_{i-1},x_i]$ 上，只有两顶帽子非零：φ_{i-1} 与 φ_i。它们的导数分别是

$$\varphi_{i-1}'=-\frac{1}{h},\qquad \varphi_i'=\frac{1}{h}.$$

于是单元刚度矩阵是

$$K^{(e)}=\int_e\begin{pmatrix}(\varphi_{i-1}')^2 & \varphi_{i-1}'\varphi_i'\\ \varphi_i'\varphi_{i-1}' & (\varphi_i')^2\end{pmatrix}\mathrm{d}x=\frac{1}{h}\begin{pmatrix}1 & -1\\ -1 & 1\end{pmatrix}.$$

这个 2×2 矩阵只有四个数，却是整个方法的心脏。$h=0.1$ 时它等于

$$10\begin{pmatrix}1 & -1\\ -1 & 1\end{pmatrix},$$

单元越短，刚度越大，符合“短弹簧更硬”的直觉。

若 a 随位置变化，把 a 换成它在单元上的平均值：

$$K^{(e)}=\frac{\bar a_e}{h}\begin{pmatrix}1 & -1\\ -1 & 1\end{pmatrix},\qquad \bar a_e=\frac{1}{h}\int_{x_{i-1}}^{x_i}a(x)\,\mathrm{d}x.$$

为什么是平均值？因为 $\int_e a\varphi_{i-1}'\varphi_i'=(-1/h^2)\int_e a$，$a$ 只以积分的形式出现，不需要对 a 求导。若 a 在单元内光滑，两点 Gauss 求积就够；若 a 在单元内跳变，就把跳变面设成单元边界，让每个单元内的 a 是常数。

3.3.3.2 载荷向量与求积

载荷向量是

$$b_i=\int_0^1 f\varphi_i\,\mathrm{d}x.$$

它不是一个点值，而是 f 按帽子的形状加权平均。以 $f=2$ 为例，$\int\varphi_i=h$（内部节点），两端只有半个帽子，$\int\varphi_0=h/2$。于是

$$b=(h,\,2h,\,2h,\,\dots,\,2h,\,h)^{\mathsf T}.$$

这是有限元与差分的第一处实质差别：差分的右端是 $f(x_i)$，有限元的右端是 $\int f\varphi_i$。f 光滑时两者只差 $O(h^2)$；f 有间断或剧烈变化时，这个加权平均会把 f 抹平一点，差别就出来了。

一般情形用 Gauss 求积。图 40 把参考单元与 Gauss 点画在一起：物理单元 $[x_{i-1},x_i]$ 先映到参考单元 $[-1,1]$，

$$x(\xi)=x_{i-1}+\frac{h}{2}(\xi+1),\qquad \xi\in[-1,1],$$

再在参考单元上取 Q 个 Gauss 点 ξ_q 与权重 w_q：

$$\int_{x_{i-1}}^{x_i}f\varphi_i\,\mathrm{d}x=\frac{h}{2}\int_{-1}^{1}f(x(\xi))\varphi_i(x(\xi))\,\mathrm{d}\xi\approx\frac{h}{2}\sum_{q=1}^{Q}w_q f(x(\xi_q))\varphi_i(x(\xi_q)).$$

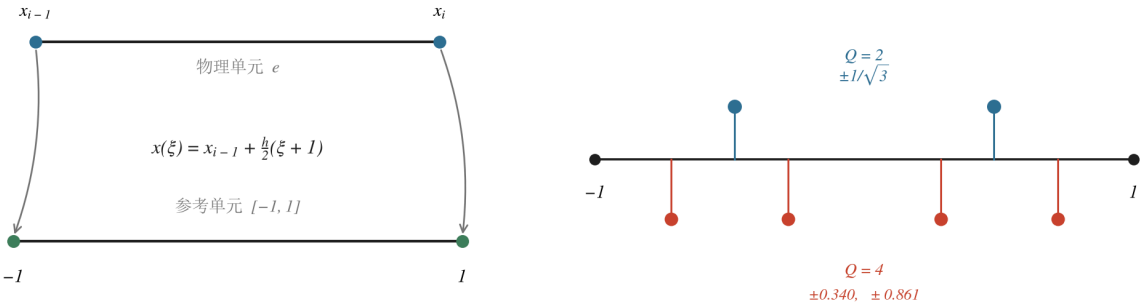


图 40　参考单元与 Gauss 点：物理单元先映到 $[-1,1]$，再在参考坐标上取点

注意 f 与 φ_i 都要在物理点 $x(\xi_q)$ 上取值，不能把参考坐标直接代进 f。这是实现里最常见的错之一：单元很短时它只带来一个小偏差，网格一加密就变成不收敛的常数项。

被积函数是“ f 乘一个线性函数”。 f 是常数或线性时，2 点 Gauss 已经精确； f 是 $\sin$、 $\exp$ 这类光滑函数时，4 点通常足够。求积阶数不够，会把求积误差混进离散误差，后面 3.3.5.2 节会看到后果。

3.3.3.3　二维三角形与条件数（选讲）

二维的线性单元是三角形。把区域剖分成三角形，每个三角形上取三个顶点处的“帽函数”，在自家顶点取 1、另外两个顶点取 0。单元刚度矩阵有一个只用顶点坐标的漂亮公式：

$$K^{(e)} = \frac{1}{4|T|}\left(bb^{\mathsf{T}} + cc^{\mathsf{T}}\right), \qquad (35)$$

其中 $|T|$ 是三角形面积，顶点按逆时针编号为 i, j, k， $b_i = y_j - y_k$， $c_i = x_k - x_j$，其余两个分量循环排列。

以单位直角三角形为例：顶点按逆时针取 $(0,0),(1,0),(0,1)$， $|T| = 1/2$， $b = (-1,1,0)$， $c = (-1,0,1)$。代入 (35) 得

$$K^{(e)} = \frac{1}{2}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}.$$

每一行之和为零，说明常向量（整体平移）仍在零空间里；这与一维整体矩阵“角上 $1/h$、内部 $2/h$”是同一本账。

这个公式不需要任何数值积分，装配时只用到三个顶点的坐标。二维的装配逻辑与一维完全一样：局部 3×3 块按全局节点编号累加。不同的只是每个节点多了几个邻居，矩阵从三对角变成带状。

二维三角形是 1943 年 Courant 的武器，也是 1956 年波音机翼的积木。一维的帽子是“线段上的三角形”，二维的帽子是“平面上的小帐篷”，高维只是把“邻居”换成“相邻的单纯形”。

公式能不能算，要拿数据说话。取单位方形 $\Omega = (0,1)^2$，精确解 $u = \sin \pi x \sin \pi y$，右端 $f = 2\pi^2 u$，四边为零。把每边分成 N 段，每个小方形沿对角线切成两个三角形； $N = 8$ 的网格见图 41(a)。单元矩阵用 (35)，载荷用 7 点三角形求积，装配与一维完全相同，只是每个节点最多连 6 个邻居。

表 14　二维三角形单元的误差

N	h	自由度	L^2 误差	阶	H^1 误差	阶
4	0.2500	9	7.91×10^{-2}	—	8.39×10^{-1}	—
8	0.1250	49	2.11×10^{-2}	1.90	4.32×10^{-1}	0.96
16	0.0625	225	5.38×10^{-3}	1.97	2.18×10^{-1}	0.99
32	0.03125	961	1.35×10^{-3}	1.99	1.09×10^{-1}	1.00

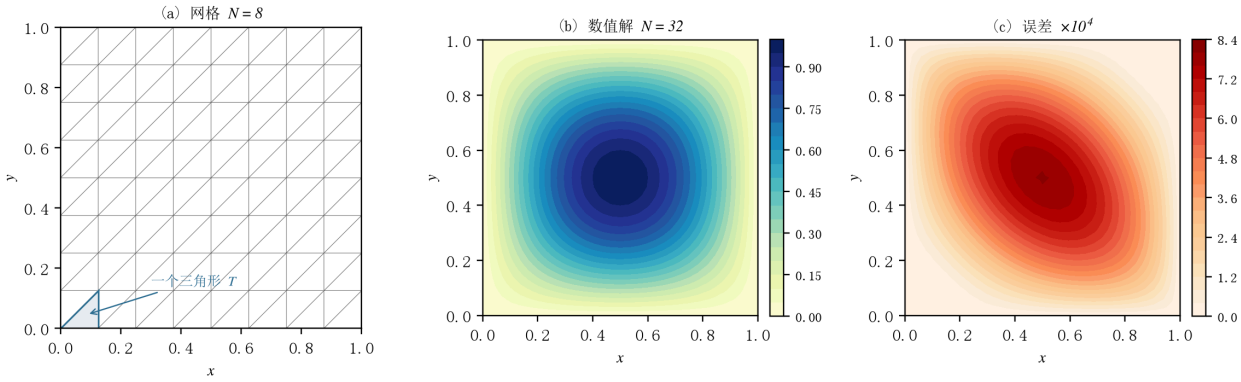


图 41　二维三角形：网格、数值解与误差分布（N=32 时最大误差约 8.4×10^{-4}）

两个阶仍是 2 与 1，只是常数比一维多。误差最大处在区域中心，也就是温度最高、曲率最大的地方； $N=32$ 时最大误差约 8.4×10^{-4}，与 $h^2=9.8\times10^{-4}$ 同量级。这张表说明：二维不是把一维做两遍，未知量按 $(N-1)^2$ 增长，矩阵从三对角变成带状；但装配规则、边界处理和误差阶都没有变[52][53]。

刚度矩阵对称正定，但它的条件数随网格加密迅速变坏。对一维 Poisson 问题，内部刚度矩阵的特征值是

$$\lambda_k=\frac{4}{h}\sin^2\frac{k\pi h}{2},\qquad k=1,\dots,N-1,$$

对应的特征向量是 $v_j=\sin(jk\pi/N)$， $j=1,\dots,N-1$；直接代入差分方程即可验证。于是

$$\kappa(A_{\text{int}})=\frac{\lambda_{\max}}{\lambda_{\min}}=\cot^2\frac{\pi h}{2}\sim\frac{4}{\pi^2h^2}.\qquad(36)$$

网格加密一倍， h 减半，条件数变四倍。

表 15　内部刚度矩阵的条件数

N	h	$\kappa(A_{\text{int}})$	$4/(\pi^2h^2)$	κh^2
4	0.2500	5.83	6.48	0.364
8	0.1250	25.27	25.94	0.395
16	0.0625	103.09	103.75	0.403
32	0.03125	414.35	415.01	0.405
64	0.015625	1659.38	1660.05	0.405

图 42 把表 15 画成双对数：实测曲线与 $4/(\pi^2h^2)$ 几乎重合，斜率是 -2。这说明 (36) 不只是一个渐近公式，常数也已经在数值上对上了。

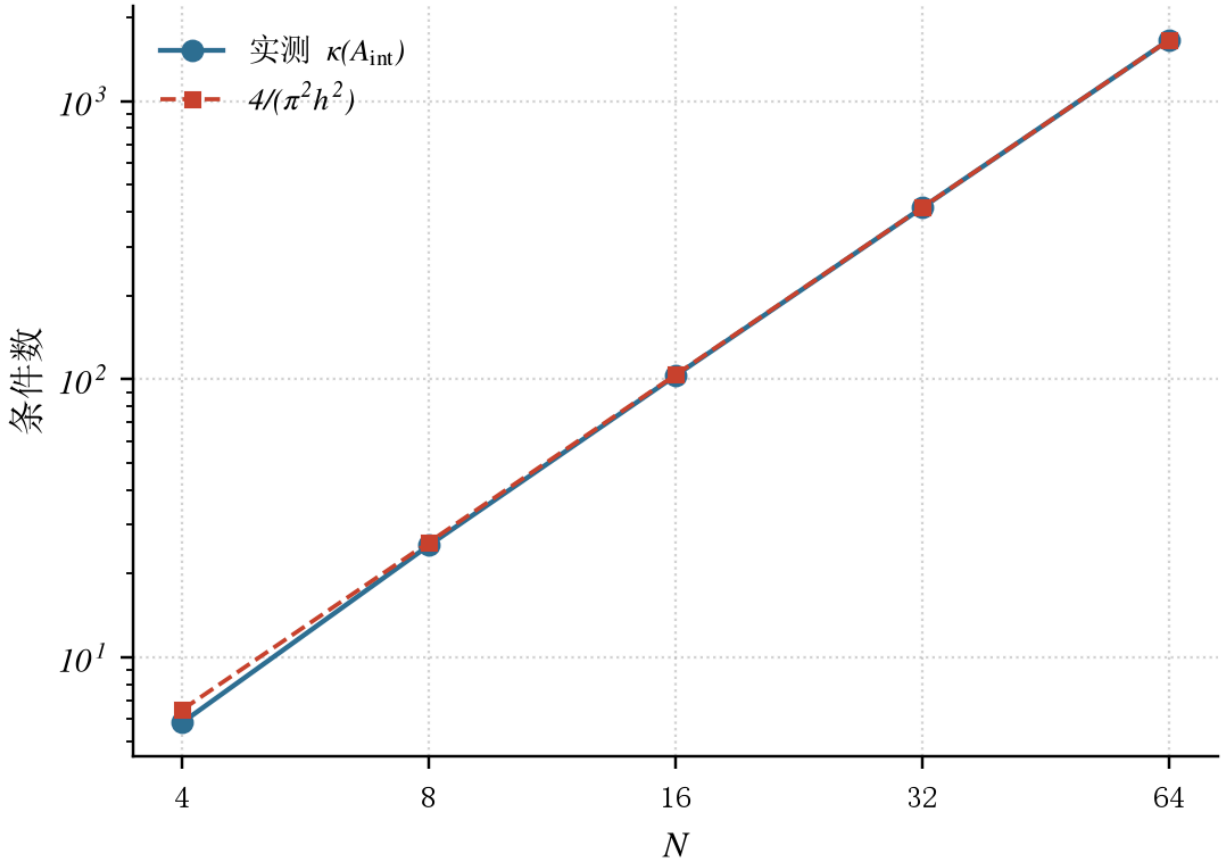


图 42　条件数随网格加密按 h^{-2} 增长

最后一列稳定在 0.405 附近，理论值是 $4/\pi^2 = 0.4053$；这是对 (36) 中常数的定量确认。

条件数变大有两个后果：迭代法收敛变慢，浮点舍入误差被放大。对对称正定系统，共轭梯度法的误差有一个只依赖 κ 的界：

$$\|u_k - u\|_A \le 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|u_0 - u\|_A,$$

其中 k 是迭代步数，$\|\cdot\|_A$ 是能量范数。由 (36)，$\sqrt{\kappa} \sim 2/(\pi h)$：网格加密一倍，迭代步数也大致翻倍。预处理、多重网格的目的，就是把 κ 压回与 h 无关的量级。

这不是有限元独有的缺点，而是所有离散化的共同代价；有限元的优势在于矩阵稀疏，稀疏性让迭代法有办法。共轭梯度法的原始文献见 [50]，稀疏迭代法的系统讨论见 [51]。

3.3.4　装配与边界

3.3.4.1　两套编号与按编号累加

装配这一步没有新公式，只有一本账。单元 $e_i = [x_{i-1}, x_i]$ 上只有两个节点，局部编号取 $0, 1$；整体问题有 $N+1$ 个节点，全局编号取 0 到 N。两套编号之间需要一个对应：

$$e_i \longmapsto \{i-1,\ i\}, \qquad i = 1, \dots, N.$$

映射说的是：单元 i 的局部 $0,1$，分别记到全局的 $i-1$ 与 i。

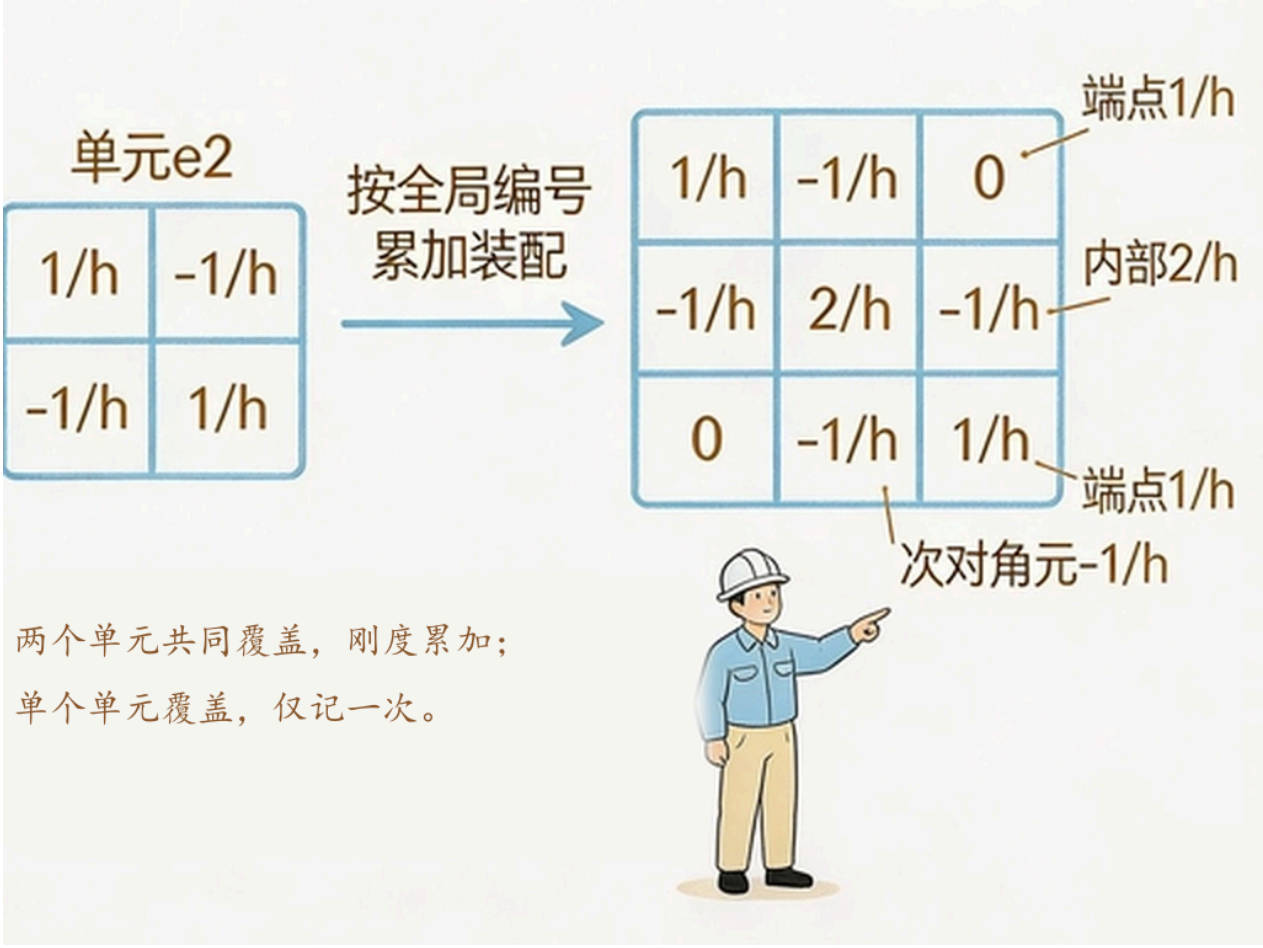


图 43　装配映射：单元 e_2 的 2×2 小块按全局编号累加

映射定下来，装配就是按编号累加：

$$A_{i-1,i-1} \mathrel{+}= \frac{1}{h},\quad A_{i-1,i} \mathrel{+}= -\frac{1}{h},\quad A_{i,i-1} \mathrel{+}= -\frac{1}{h},\quad A_{i,i} \mathrel{+}= \frac{1}{h}. \tag{37}$$

注意是 $+=$，不是 $=$。一个内部节点同时属于左右两个单元，它会被记两次；端点只记一次。这一句差别，就是下面角上 $1/h$ 与内部 $2/h$ 的全部来历。

右端向量走同一本账：$b_j = \int f\varphi_j$，在每个单元上用 Gauss 求积算，再按同样的全局编号累加。

故事丨装配车间。　车间里，小工面前摆着一盒一模一样的 2×2 铁片，每片四角写着 $1/h$、$-1/h$、$-1/h$、$1/h$。师傅给他一张节点表：第 i 片装到第 $i-1$ 与第 i 号位置。小工不用懂有限元，只要把四角对准行号列号，往上加。装完一数，中间的节点被左右两片各压了一次，变成 $2/h$；两头的节点只被压一次，还是 $1/h$。师傅说：对了。小工问：为什么？师傅说：因为你没有把端点当成内部节点。

装配车间：小工不懂有限元，只要把每块 2×2 铁片的四角对准大账本上的行号列号往上加；中间节点被左右两片各压一次，叠成 2/h。

把 (37) 逐单元执行完，得到

$$A=\frac{1}{h}\begin{pmatrix} 1 & -1 & & & & \\ -1 & 2 & -1 & & & \\ & -1 & 2 & -1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & -1 & 2 & -1 \\ & & & & -1 & 1 \end{pmatrix}. \tag{38}$$

看两个角： x_0 上的对角元是 $1/h$，内部的 x_1 是 $2/h$。这不是笔误。 x_0 只被单元 e_1 覆盖，只记一次； x_1 被 e_1 与 e_2 同时覆盖，记两次。

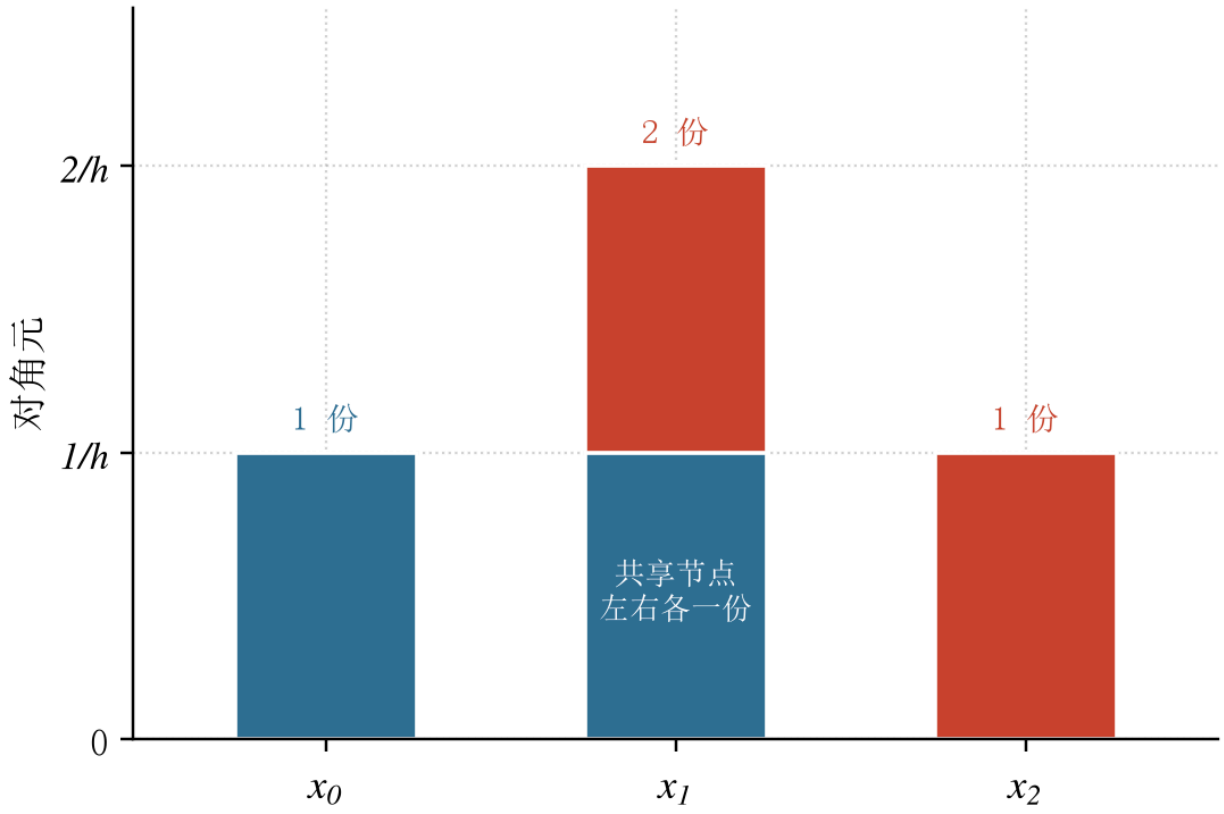


图 44　端点一份、内部两份

这里要分清两件事。整体矩阵的角上是 $\mathbf{1/h}$，这是覆盖次数；一旦用消元法把端点行列删去，角上的值就不再进入方程组，内部块的对角全是 $\mathbf{2/h}$。真正危险的是把内部块的首末对角也写成 $\mathbf{1/h}$：这时矩阵仍对称，却对应一个错误的边值问题，解在端点附近出现 $O(1)$ 的边界层，程序不会报错。3.3.4.2 节用实验把这件事量出来。

一个十秒检查：装配完成、尚未施加边界条件时，$\boldsymbol{A1}$ 应当等于零向量。常向量代表整体平移，没有边界约束时它落在零空间里。若 $\boldsymbol{A1}$ 的两个端点分量不为零，整体矩阵的角上就写错了；注意这一步检查的是整体矩阵，不是消元后的内部块。

3.3.4.2　稀疏结构、存储与角上为什么是 1/h

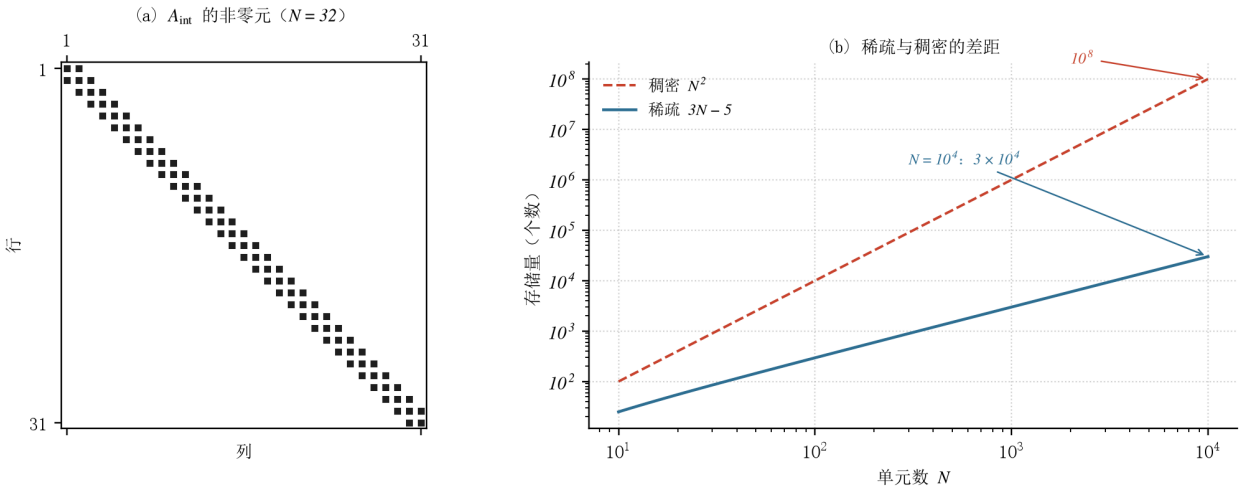


图 45　稀疏结构：(a) A_{int} 的非零元只占三条对角线；(b) 稀疏存储 3N−5 与稠密 N^2 的增长差距

装配得到的 A 不只是三对角，它的绝大多数位置是零。以 $N=32$ 为例，A_{int} 是 31×31，只有 91 个非零元；若按稠密矩阵存，要存 $31^2=961$ 个数，九成以上是零。图 45(a) 是它的 spy 图：非零元只落在主对角与两条次对角上，半带宽为 1。

一般的计数是：A_{int} 有 $N-1$ 行，首末行各 2 个非零元，其余行各 3 个，故

$$\mathrm{nnz}(A_{int}) = 3(N-1)-2 = 3N-5.$$

$N=10^4$ 时，A_{int} 有 10^8 个位置，非零元只有 3×10^4 个，相差三千多倍。稀疏存储只记非零元与下标，内存从 $O(N^2)$ 降到 $O(N)$；这也是有限元能算几十万、几百万个单元的原因。

带宽同样重要。一维线性元的半带宽是 1；二维按行优先编号时，半带宽约为一行的未知量数 N，矩阵从三对角变成带状。3.3.4.1 节的两套编号在这里第二次起作用：编号顺序决定带宽，带宽决定直接法的填充与迭代法的收敛[51]。

“角上是 $1/h$”不是一条需要背下来的规则，而是“端点只被一个单元覆盖”的直接推论。记住覆盖次数，比记住结论可靠。

这条推论也解释了一个常见 bug，但方向和很多人想的不一样。错误不是“整体矩阵用了 $2/h$”——用消元法时端点行列会被删去，整体矩阵角上写多少都进不了内部块。错误是把内部块的首末对角写成 $1/h$，相当于在 x_1 或 x_{N-1} 处错装了一个自由端。图 46 是实测：N 从 8 加密到 128，最大误差从 0.77 涨到 0.98，不下降；误差集中在端点附近，向内部线性衰减。若两个角都写错，矩阵直接奇异，`numpy.linalg.solve` 会报错。

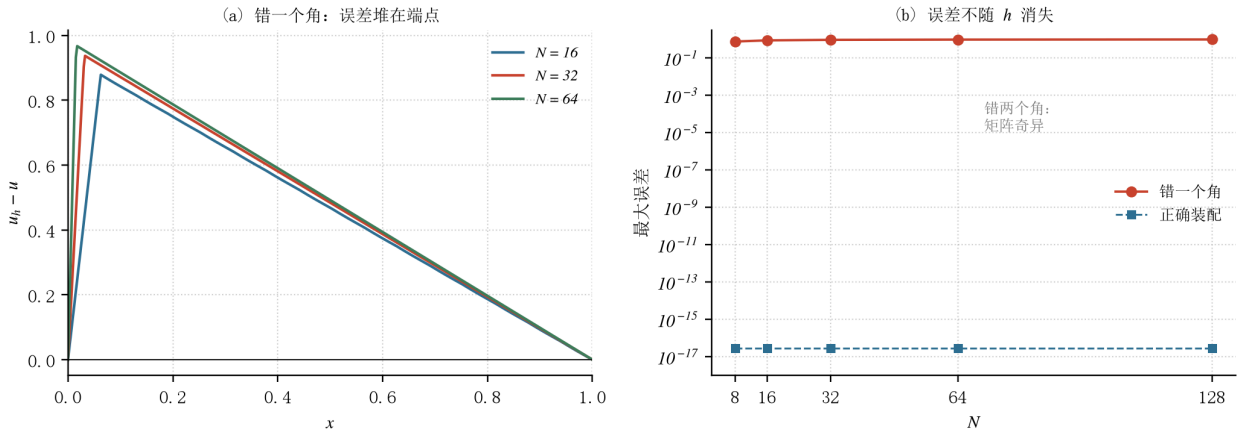


图 46　错角矩阵的边界层：错一个角时误差不随 h 消失；错两个角时矩阵奇异

若把两个端点消去，剩下的内部块是干净的常系数三对角：

$$A_{int} = \frac{1}{h}\,\mathrm{tridiag}(-1,\,2,\,-1)_{(N-1)\times(N-1)}.$$

这个闭式只对均匀网格成立。非均匀网格上，每个单元有自己的 h_e，整体矩阵不再等对角；装配规则 (37) 仍然正确，只是把 $1/h$ 换成 $1/h_e$。

3.3.4.3　十秒自检与差分的亲戚关系

映射写反、或漏加相邻单元的贡献，程序往往照样能跑，也能解出一条光滑曲线。所以“看着像”不能当验收标准。

可靠的办法是用另一种方式算同一个矩阵，再逐位对账。均匀网格上可以按（38）直接构造，与循环装配的结果逐位比对。表 16 是四个网格上的对账结果。

表 16　装配结果与直接构造的比对

N	h	最大差	非零元	半带宽	判定
4	0.2500	0	7	1	通过
8	0.1250	0	19	1	通过
16	0.0625	0	43	1	通过
32	0.03125	0	91	1	通过

差值是严格的 0，不是“很小”。若这里出现 10^{-1} 量级的偏差，几乎一定是映射写反了；这不是精度不够，是记账记错了位置。

这个自检比“看解像不像”可靠，原因很简单：矩阵承载了离散问题的全部信息。它不依赖右端项，也不依赖求解器；而错误的矩阵照样能解出一条光滑曲线，下游没有任何一道检查会拦住它。

再补三条快速检查：A 对称；A 三对角；角上 $1/h$、内部 $2/h$。任一不满足，先别怀疑求解器，去查 `gdof`。

在均匀网格上，把内部块与 $-u''=f$ 的中心差分矩阵放在一起看：

$$A_{\text{int}} = h\,B, \qquad B = \frac{1}{h^2}\,\text{tridiag}(-1,\,2,\,-1).$$

刚度矩阵只差一个因子 h。不少人由此得出结论：“均匀网格上的线性元就是差分。”这个结论不成立，因为两种方法除了矩阵，还有两处不同。

第一，右端项不同。差分的右端是点值 $f(x_i)$；有限元的右端是加权平均 $\int f\varphi_i$。f 光滑时差别是 $O(h^2)$；f 有间断时，有限元的右端会把间断抹平一点，这既是优点也是缺点。

第二，边界处理的含义不同。有限元的边界条件是在变分意义下施加的，差分的边界条件是直接代入。一个是在弱形式里“请进空间”，一个是在代数方程里“换掉一行”。

所以“刚度矩阵成比例”不等于“两种方法等价”。真正的差别不在矩阵，而在右端项与边界处理。这也是有限元在变系数、复杂区域上更占优势的原因。

3.3.4.4　矩阵的奇异性与三种上锁方式

施加 $u(0)=u(1)=0$ 之前，先要明白为什么必须施加。由装配规则，每一列的元素和都为零：内部列是 $\frac{2}{h}-\frac{1}{h}-\frac{1}{h}$，端列是 $\frac{1}{h}-\frac{1}{h}$。于是

$$A\mathbf{1} = \mathbf{0}.$$

常向量落在 A 的零空间里，A 奇异。物理含义清楚：没有边界约束，解可以整体平移一个常数；每个平移量都给出一个解。

这不是数值 bug，而是物理上的刚体位移。必须补上边界条件，解才唯一。

补边界条件有三种常见做法，工程量差别很大。

消元法最直接：删去第 0、 N 行与列，只解 $N-1$ 个内部自由度。

$$A_{\text{int}}U_{\text{int}} = b_{\text{int}}, \qquad A_{\text{int}} = (A_{ij})_{1\le i,j\le N-1}.$$

置 1 法保留尺寸：把第 d 行与第 d 列（对角元除外）清零，对角元置 1，右端置 g_d：

$$A_{dd} = 1, \qquad A_{dj} = A_{jd} = 0\ (j \neq d), \qquad b_d = g_d.$$

第 d 个方程就变成 $1 \cdot u_d = g_d$，边界值被直接写进方程。

置大数法最省事：把 A_{dd} 改成 10^{12}，右端置 $10^{12}g_d$：

$$A_{dd} = 10^{12}, \qquad b_d = 10^{12}g_d.$$

它的效果是把边界条件写成一个极强的惩罚： u_d 被压到 $g_d + O(10^{-12})$，而不是精确等于 g_d。三种做法的效果列于表 17。

表 17　三种边界处理给出同一个解（N=16）

做法	与消元法的最大差	$u_h(0)$	$u_h(1)$	$\kappa(A)$
消元法	0	0	0	1.03×10^{2}
置 1 法	0	0	0	1.03×10^{2}
置大数法	9.38×10^{-13}	9.375×10^{-13}	9.375×10^{-13}	1.63×10^{12}

前两种给出完全相同的结果，边界值精确为零，条件数也一样。置大数法差 10^{-12} 量级，边界值也只是近似零；更麻烦的是条件数：它从 1.03×10^{2} 抬到 1.63×10^{12}，涨了十个数量级。

差别看似微不足道，后果却有两个。一是边界值的精度被永久限制在 10^{-12}；二是 10^{12} 的尺度进入矩阵，条件数被恶化到 10^{12} 量级。当 h 继续减小，有限元矩阵本身的条件数已按 h^{-2} 增长，再乘上这个惩罚因子，误差会更快放大。

实践上应优先用消元法或置 1 法。置大数法像一位“差不多先生”：临时凑一个能跑的结果可以，进生产代码不行。

3.3.5　误差与实验

本节按“先预测、再运行”的顺序组织：先把预期写下来，再让程序去检验。实验设计列于表 18。

表 18　实验设计：先预测，再运行

要填写的项目	本组内容
要回答的问题与运行前预测	线性元在 L^2 下二阶、 H^1 下一阶；求积不足会破坏节点精确；界面与网格不重合会掉阶。预测 $p_{L^2} \approx 2$、 $p_{H^1} \approx 1$。
输入、参数、单位、数据来源与划分	一维 Poisson $-u'' = f$， $u(0) = u(1) = 0$；制造解 $u = \sin(\pi x)$， $f = \pi^2 \sin(\pi x)$； $N = 8, 16, \ldots, 128$；变系数 $a = 1 + \frac{1}{2}\sin 2\pi x$；界面 $x = 0.5$（对齐）与 $x = 0.53$（错位）。

要填写的项目	本组内容
独立参考解/基线及选择理由	解析解作参考；另用差分格式交叉核对矩阵与解，避免“自己验自己”。
误差指标、容差及其依据	$\|u-u_h\|_{L^2}$、$\|u-u_h\|_{H^1}$，观测阶 $p=\log_2(e_h/e_{h/2})$；双精度，节点误差容差 10^{-12}。
环境、种子、设备与运行预算	Python 3.12、NumPy 2.x、Matplotlib 3.11；固定种子 2026（问题本身无随机性）；单次运行 < 10 s。

3.3.5.1 Céa 引理与插值估计

两个范数　有限元的误差要在范数下衡量，而这直接决定“几阶”这个说法的含义。设 u_h 是有限元解，两个标准选择是

$$\|u-u_h\|_{L^2}=\Big(\int_0^1 (u-u_h)^2\,\mathrm{d}x\Big)^{1/2},\qquad |u-u_h|_{H^1}=\Big(\int_0^1 (u'-u_h')^2\,\mathrm{d}x\Big)^{1/2}.$$

L^2 范数看的是“整体高度差”，H^1 半范数看的是“坡度差”。同一个解，两把尺子给出两个阶：

$$\|u-u_h\|_{L^2}=O(h^2),\qquad |u-u_h|_{H^1}=O(h).\tag{39}$$

为什么 H^1 低一阶？因为 H^1 里对导数求误差，而 u_h 是分段线性的，它的导数在每段上是常数。用常数去逼近 u'，只有一阶精度。

定义 4（收敛阶）　若误差满足 $E(h)=O(h^p)$，称格式为 p 阶收敛；若 $E(h)\approx Ch^p$，由相邻两次计算可估计观测阶

$$p_{\text{obs}}=\log_2\frac{E(h)}{E(h/2)}.$$

下面的数值实验就用这个公式判阶。

所以，说“有限元是二阶的”必须先说清是哪个范数。只报“二阶”而不说 L^2，等于没说；导数的误差其实只有一阶。

尺子有了，接下来量误差。Galerkin 解有一个关键性质：误差与整个子空间正交。对任意 $v_h\in V_h$，

$$a(u-u_h,v_h)=a(u,v_h)-a(u_h,v_h)=(f,v_h)-(f,v_h)=0.$$

这叫 Galerkin 正交性。它是所有误差估计的起点。

引理 2（Céa）　设 a 连续（常数 M）、强制（常数 α），u 是弱解，$u_h\in V_h$ 是 Galerkin 解。则

$$\|u-u_h\|_a\le\frac{M}{\alpha}\inf_{v_h\in V_h}\|u-v_h\|_a,\tag{40}$$

其中 $\|v\|_a=\sqrt{a(v,v)}$ 是能量范数。

证明　由强制性、Galerkin 正交性与连续性，有下面的估计。因为 $u_h-v_h\in V_h$，正交性把 $u-u_h$ 与 u_h-v_h 的配对消掉，于是

$$\|u-u_h\|_a^2=a(u-u_h,u-u_h)=a(u-u_h,u-v_h)\le\frac{M}{\alpha}\|u-u_h\|_a\|u-v_h\|_a.$$

两边约去 $\|u-u_h\|_a$（若为零则结论显然），再对 v_h 取下确界，即得 (40)。证毕。

Céa 引理把“有限元误差”转化成了“逼近论误差”：u_h 与 u 的距离，不超过 V_h 中最好的那个函数与 u 的距离，只差一个常数 M/α。剩下的问题是：分片线性函数能把 u 逼近得多好？

引理 3（一维线性插值） 设 $u\in H^2(0,1)$，$I_h u$ 是 u 在节点上的分片线性插值。则存在只依赖区间长度的常数 C，使

$$|u-I_h u|_{H^1}\le Ch|u|_{H^2},\qquad \|u-I_h u\|_{L^2}\le Ch^2|u|_{H^2}. \tag{41}$$

证明 在一个单元 $e=[x_{i-1},x_i]$ 上，记 $w=u-I_h u$。w 在两端为零。由 Taylor 展开与插值条件，w 可以写成 Green 函数形式

$$w(x)=\int_{x_{i-1}}^{x_i} G(x,s)u''(s)\,\mathrm{d}s,$$

其中 $|G(x,s)|\le h/4$，$|\partial_x G(x,s)|\le 1$；直接写出 G 的两段表达式即可验证这两个界。于是

$$|w(x)|\le \frac{h}{4}\int_e |u''|,\qquad |w'(x)|\le \int_e |u''|.$$

对 x 积分并用 Cauchy–Schwarz，得

$$\|w\|_{L^2(e)}\le Ch^2\|u''\|_{L^2(e)},\qquad |w|_{H^1(e)}\le Ch\|u''\|_{L^2(e)}.$$

对全部单元求和，即得 (41)。证毕。

图 47 把误差拆成两段：插值误差 $u-u_I$ 与离散误差 u_I-u_h。前者由网格和光滑性决定，后者由 Galerkin 投影决定；两条都控制住，才有 H^1 一阶、L^2 二阶。

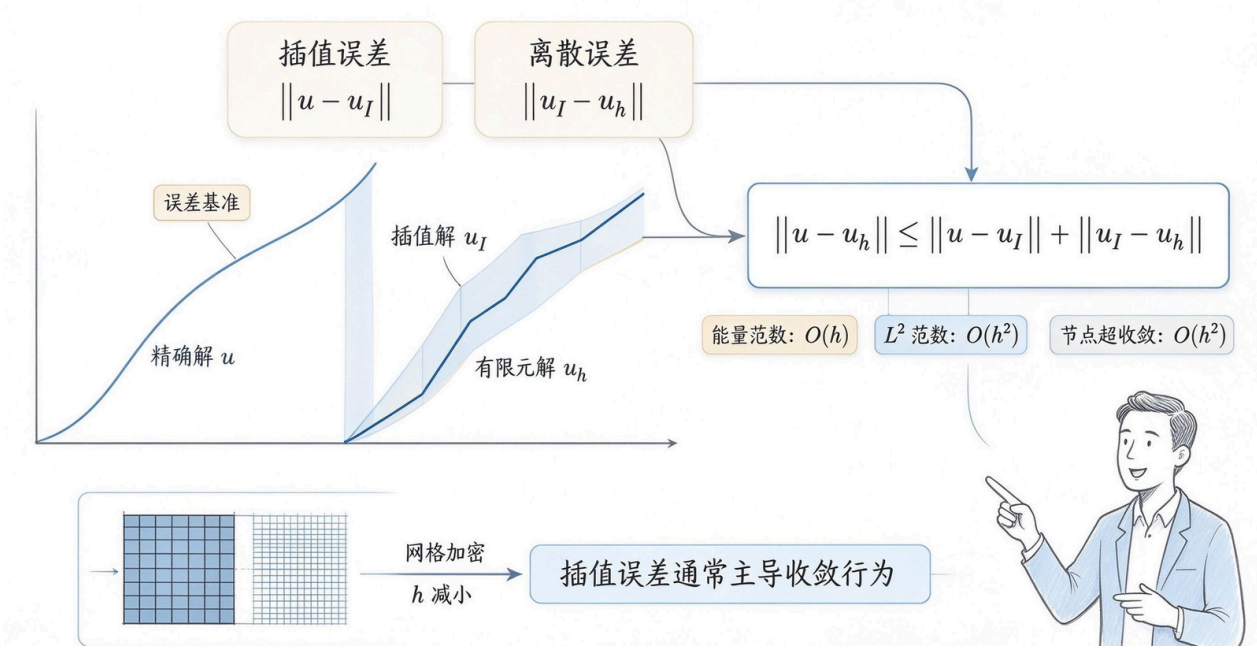


图 47 误差分解：插值误差 + 离散误差

把 Céa 引理与插值估计接起来，H^1 误差是 $O(h)$。L^2 误差的高一阶需要另一个工具：对偶论证。

定理 3（Aubin–Nitsche） 设 a 对称、连续（常数 M）、强制（常数 α），$u\in H^2(0,1)$ 是弱解，$u_h\in V_h$ 是 Galerkin 解。假设对偶问题 $a(w,v)=(e,v)$（$\forall v\in V$，$e=u-u_h$）的解满足正则性估计 $\|w\|_{H^2}\le C_{\mathrm{reg}}\|e\|_{L^2}$。则存在与 h 无关的常数 C，使

$$\|u-u_h\|_{L^2}\le Ch|u-u_h|_{H^1}.$$

证明　记 $e=u-u_h$，取对偶问题的解 $w\in V$。由对称性与 Galerkin 正交性（ $I_hw\in V_h$），

$$\|e\|_{L^2}^2=a(e,w)=a(e,w-I_hw).$$

由连续性、引理 3 与正则性假设，

$$\|e\|_{L^2}^2\le M|e|_{H^1}|w-I_hw|_{H^1}\le Ch\|e\|_{L^2}|e|_{H^1}.$$

若 $\|e\|_{L^2}=0$ 则结论显然；否则约去 $\|e\|_{L^2}$，得 $\|e\|_{L^2}\le Ch|e|_{H^1}$。证毕。

推论 1（误差阶）　在引理 2、引理 3 与定理 3 的条件下，存在与 h 无关的常数 C，使

$$|u-u_h|_{H^1}\le Ch|u|_{H^2},\qquad \|u-u_h\|_{L^2}\le Ch^2|u|_{H^2}.$$

证明　由 Céa 引理与插值估计，$|u-u_h|_{H^1}\le Ch|u|_{H^2}$；再由定理 3，$\|u-u_h\|_{L^2}\le Ch|u-u_h|_{H^1}\le Ch^2|u|_{H^2}$。证毕。

这就是“ L^2 二阶、 H^1 一阶”的完整来源[47][48]。

3.3.5.2　节点超收敛与三条失效条件

对一维 Poisson 问题，还有一个更强的结果。

定理 4（节点精确）　设 $a=1$， $f\in L^2(0,1)$， u 是 $-u''=f$、 $u(0)=u(1)=0$ 的解， u_h 是线性元 Galerkin 解。若载荷 $\int f\varphi_i$ 被精确计算，则

$$u_h(x_i)=u(x_i),\qquad i=1,\dots,N-1. \qquad (42)$$

证明　固定节点 x_j，令 $G_j(x)=G(x,x_j)$ 是 $-d^2/dx^2$ 在 $(0,1)$ 上、零 Dirichlet 边界的 Green 函数。 G_j 连续、分段线性，拐点只在 x_j，且 $G_j(0)=G_j(1)=0$，所以 $G_j\in V_h$。

记 $e=u-u_h$。由 Galerkin 正交性， $a(e,G_j)=0$。另一方面，分部积分两次并用 $-G_j''=\delta_{x_j}$：

$$a(e,G_j)=\int_0^1 e'G_j'\,\mathrm{d}x=-\int_0^1 e\,G_j''\,\mathrm{d}x=e(x_j).$$

两式相减， $e(x_j)=0$。证毕。

这个证明值得多看一眼：它没有估计任何误差，只用了两件事——Green 函数恰好落在有限元空间里，以及 Galerkin 误差与整个空间正交。节点上的误差就这样被“正交”掉了。

故事 | 节点上的巧合。　第一次跑出 2.78×10^{-17} 时，组里先怀疑是 bug：节点误差怎么会比 L^2 误差小十几个数量级？查了映射、查了装配、查了求积，最后才发现不是 bug——是 Green 函数在替我们做消去。程序没有错，错的是“误差一定随网格连续变小”的直觉。

推论：既然 u_h 与 u 在所有节点上重合，而两者都是分片线性的，那么 u_h 就是 u 的分片线性插值 I_hu。于是 L^2 误差完全来自单元内部，量级是 $O(h^2)$； H^1 误差是 $O(h)$。这与 3.3.5.1 节的估计一致，但节点精确是一个更强的逐点结论。

节点精确有一个不能漏的前提：载荷必须精确积分。若 $\int f\varphi_i$ 只算到某个求积阶，节点值就不再精确。下面给出实测。

(39) 与 (42) 各依赖一条假设，另有一个附带前提。三条都在实际计算中失效过。

第一条是载荷的精确积分。若 $\int f\varphi_i$ 只算到某个求积阶，节点值就只能精确到求积误差的阶。表 19 是实测：取 $u=\sin\pi x$、$f=\pi^2\sin\pi x$，用 $Q=1,2,4$ 点 Gauss 算载荷，节点误差随 N 的变化一目了然。$Q=1$ 时节点误差只按 h^2 下降；$Q=2$ 按 h^4 下降，$N=64$ 已到 4.0×10^{-9}；$Q=4$ 在 $N\geq16$ 就碰到机器精度。

表 19　求积阶数与节点误差

N	h	$Q=1$	$Q=2$	$Q=4$
4	0.2500	2.71×10^{-2}	2.73×10^{-4}	5.86×10^{-10}
8	0.1250	6.51×10^{-3}	1.67×10^{-5}	2.24×10^{-12}
16	0.0625	1.61×10^{-3}	1.03×10^{-6}	8.66×10^{-15}
32	0.03125	4.02×10^{-4}	6.45×10^{-8}	3.11×10^{-15}
64	0.015625	1.00×10^{-4}	4.03×10^{-9}	9.10×10^{-15}

图 48 把表 19 的数据画成双对数：$Q=1$ 的斜率约 2，$Q=2$ 的斜率约 4。换成可精确积分的 $f=2$ 之后，$Q=1$ 就已达到 2.78×10^{-17}。同一套代码、同一个 N，只因载荷算法不同，“节点精确”就从成立变成不成立。

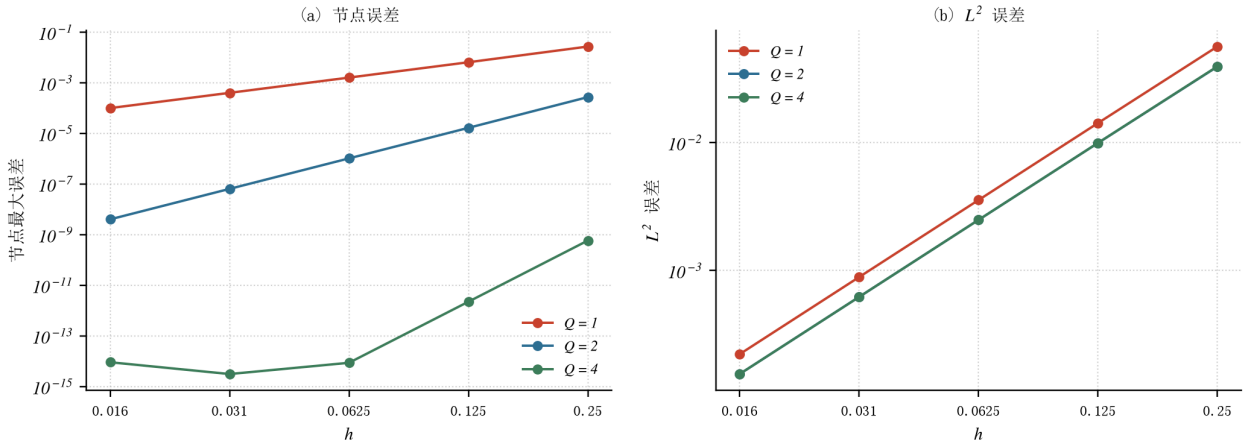


图 48　求积阶数：Q=1 只按 h^2 收敛，Q=2 按 h^4 收敛，Q=4 碰到机器精度

为什么 $Q=1$ 只掉到二阶，而不是更差？被积函数是 f 乘一个线性帽函数，$Q=1$ 对线性部分是精确的，误差只来自 f 的变化；f 光滑时，中点公式是二阶。$Q=2$ 对三次多项式精确，误差降到四阶；$Q=4$ 对七次精确，对 $\sin$ 已经足够。求积阶数不够不会让解爆掉，只会把节点超收敛悄悄拿掉——这是最容易被忽略的一类误差。

第二条是解的正则性。L^2 二阶来自 Céa 加插值估计，后者要求 $u\in H^2$。一维里，只要 a 足够光滑（或分段常数且界面与单元边界对齐），$f\in L^2$ 就保证 $u\in H^2$，所以 f 有间断并不会掉阶；真正让阶数掉下来的是 $f\notin L^2$（点载荷、$x^{-\beta}$ 型奇性右端），或二维区域的角点——那里 u 可能只有 H^{1+s}（$s<1$）的正则性，此时 L^2 的阶会掉到 $1+s$，而不是 2。这就是为什么真实问题的收敛阶常常“跑不到 2”。不是程序错了，是解不够光滑，或右端不够好。

第三条是网格的均匀性。(38) 那个漂亮的三对角闭式依赖 h 处处相同。非均匀网格上，装配规则 (37) 依然正确，但每个单元要用自己的 h_e，整体矩阵不再等对角；不能再用“角上 $1/h$、内部 $2/h$”去构造，必须老老实实按映射累加。

三条失效条件有一个共同点：它们都不是程序 bug，而是假设不成立。报误差时必须说清范数；说节点精确时必须说清载荷精确积分；说二阶收敛时必须说清解的正则性。

3.3.5.3 数值实验：变系数与混合边界

取 $-u''=2$、$u(0)=u(1)=0$，精确解

$$u(x)=x(1-x).$$

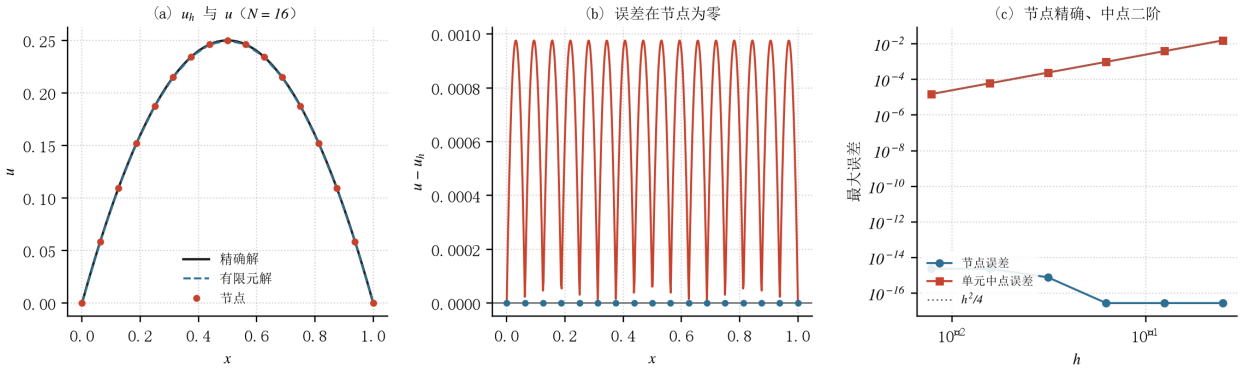


图 49　解、误差与节点超收敛：N=16 的解；误差在节点为零；节点误差停在机器精度，中点误差按 h^2 下降

先看解长什么样。图 49 给了三个画面：$N=16$ 时 u_h 与 u 几乎重合；误差 $u-u_h$ 在每个节点为零，在单元中点达到 $h^2/4$；把节点误差与中点误差画在同一张双对数图上，节点误差停在 10^{-16}，中点误差按 h^2 下降。节点精确不是“误差很小”，而是“节点上恰好为零”。

表 20　两个范数下的误差

N	h	L^2 误差	阶	H^1 半范数	阶	节点最大误差
4	0.2500	1.1411×10^{-2}	—	1.4434×10^{-1}	—	2.78×10^{-17}
8	0.1250	2.8527×10^{-3}	2.00	7.2169×10^{-2}	1.00	2.78×10^{-17}
16	0.0625	7.1318×10^{-4}	2.00	3.6084×10^{-2}	1.00	2.78×10^{-17}
32	0.03125	1.7830×10^{-4}	2.00	1.8042×10^{-2}	1.00	7.77×10^{-16}
64	0.015625	4.4574×10^{-5}	2.00	9.0211×10^{-3}	1.00	2.39×10^{-15}

图 50 把表 20 的两个阶画成斜率：L^2 的斜率约为 2，H^1 的斜率约为 1。

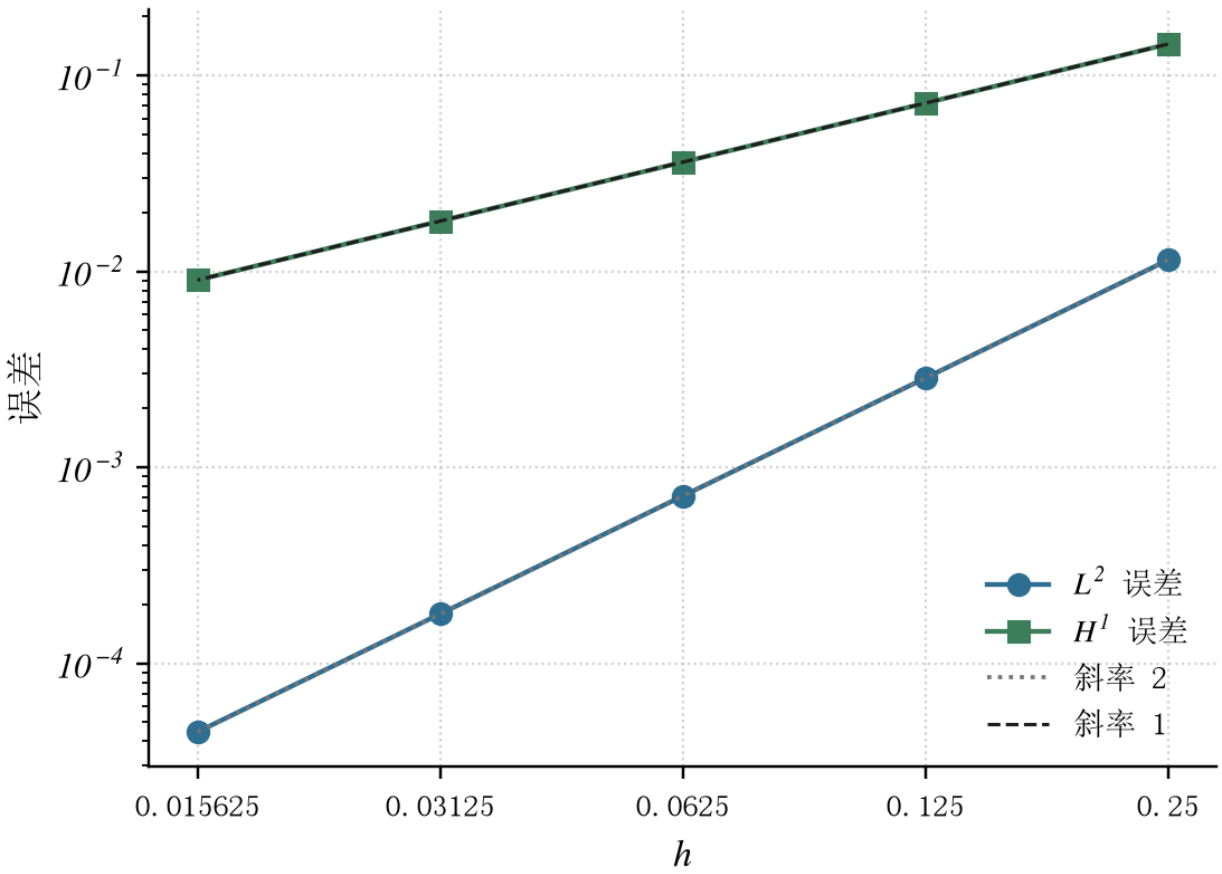


图 50　两个范数下的误差随 h 的变化；虚线为斜率 2 与 1

表 20 同时验证了三件事。第一，两个范数的阶严格等于 2 与 1；观测阶全是 $\mathbf{2.00}$ 与 $\mathbf{1.00}$，没有“接近”的余地。第二，节点误差是机器精度（ $\mathbf{2.78\times10^{-17}}$），证实 (42) 成立。第三， L^2 误差的量级与插值估计相符：既然 $u_h = I_h u$，误差应约为 $\frac{h^2}{8}\max|u''| = \frac{h^2}{4}$； $N=64$ 时 $\frac{1}{4\cdot 64^2} = 6.1\times10^{-5}$，实测 4.46×10^{-5}，同一量级。

真实材料很少是常系数。取

$$-(a u')' = f, \qquad a(x) = 1 + \frac{1}{2}\sin 2\pi x, \qquad u(x) = \sin \pi x,$$

则右端为

$$f(x) = \pi^2\Big[\big(1+\tfrac{1}{2}\sin 2\pi x\big)\sin \pi x - \cos 2\pi x \cos \pi x\Big].$$

单元刚度用 a 在单元中点的值，载荷用 4 点 Gauss 求积。表 21 和图 51 是 $N = 8, 16, 32, 64, 128$ 的结果。

表 21　变系数问题的误差

N	h	L^2 误差	阶	H^1 误差	阶
8	0.1250	9.42×10^{-3}	—	2.53×10^{-1}	—
16	0.0625	2.35×10^{-3}	2.00	1.26×10^{-1}	1.01
32	0.03125	5.89×10^{-4}	2.00	6.30×10^{-2}	1.00

		误差	阶	误差	阶
64	0.015625	1.47×10^{-4}	2.00	3.15×10^{-2}	1.00
128	0.0078125	3.68×10^{-5}	2.00	1.57×10^{-2}	1.00

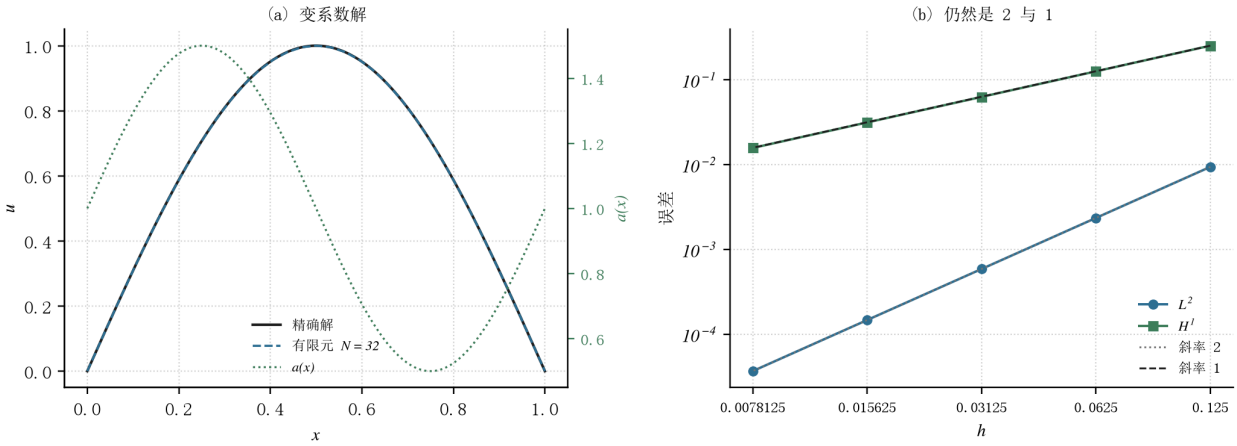


图 51　变系数问题：(a) a(x) 与 u_h；(b) L^2 与 H^1 的收敛；虚线为斜率 2 与 1

两个阶仍是 2 与 1。原因不神秘：a 光滑，单元中点平均是二阶近似；Céa 引理与插值估计的形式没有变，误差仍由“最佳逼近误差”控制。

系数跳变的情形要小心。若 a 有跳变但解仍光滑，阶数通常还是 2/1，只是误差常数变大、节点超收敛失效；真正会掉阶的，是解本身有折角而折角没有落在节点上。看一个能算清楚的例子：两段材料 $a=1$（左）与 $a=4$（右）在 $x=1/2$ 处粘接，接口作用一个单位集中力，两端固定。精确解是一个帐篷：

$$u(x)=\begin{cases}2Hx, & 0\le x\le \tfrac12,\\ 2H(1-x), & \tfrac12\le x\le 1,\end{cases}\qquad H=\frac{P}{2(a_1+a_2)}=\frac{1}{10}.$$

左斜率 0.2，右斜率 -0.2，接口处折角。若把接口取成节点，线性元空间包含这个帐篷，三个单元就精确到机器精度；若接口落在单元内部，含折角的单元上只能用一条直线去逼近两条直线，L^2 误差掉到一阶、H^1 误差掉到半阶。表 22 和图 52 是实测。

表 22　界面/集中力不对齐的误差

N	h	L^2 误差	阶	H^1 误差	阶	最大误差
7	0.142857	3.98×10^{-3}	—	8.11×10^{-2}	—	8.04×10^{-3}
15	0.066667	1.76×10^{-3}	1.18	5.53×10^{-2}	0.55	3.89×10^{-3}
31	0.032258	8.33×10^{-4}	1.08	3.84×10^{-2}	0.53	1.91×10^{-3}
63	0.015873	4.05×10^{-4}	1.04	2.69×10^{-2}	0.51	9.46×10^{-4}
127	0.007874	2.00×10^{-4}	1.02	1.90×10^{-2}	0.51	4.71×10^{-4}

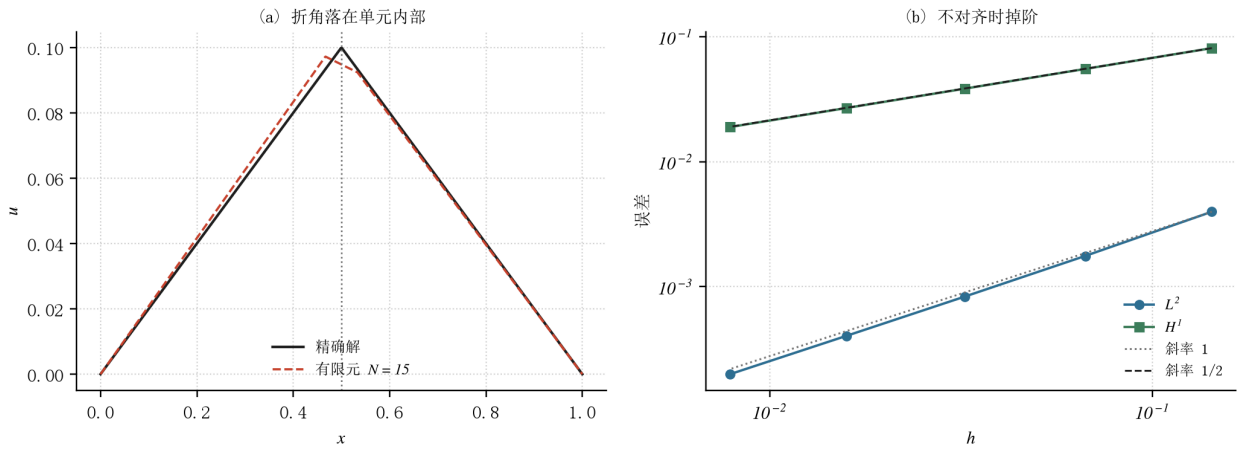


图 52　界面/集中力不对齐：(a) 折角落在单元内部；(b) L^2 掉到一阶、H^1 掉到半阶

对齐时（ $N = 8, 16, 32, 64, 128$）三个误差都在 10^{-14} 以下；不对齐时 L^2 阶约 1.0、 H^1 阶约 0.5，最大误差也按一阶下降。这不是程序 bug，是离散空间不包含精确解。把界面（或集中力作用点）设成节点，阶数立刻回来——这也是 3.3.3.1 节“跳变面设成单元边界”那句话的真正含义。

混合边界也不难。把右端改成非齐次 Neumann，只需把 g 加进载荷向量，矩阵不用动；Robin 条件给端点对角元加 σ，给右端加 g；若边界值非零，取一个满足边界值的提升函数 u_g，令 $u = u_g + w$，$w \in H_0^1$，弱形式右端多出 $a(u_g, v)$ 一项。矩阵仍是同一个刚度矩阵，变的只是右端和端点对角元。这也是有限元在工程里好用的原因：几何、材料、边界条件换的时候，装配的账本不用重写。

本节的一维实验覆盖常系数、光滑变系数、界面失配三种情形；二维在 3.3.3.3 节，混合边界只改右端与端点对角元，留作练习。

3.3.5.4　实验复现与本节结论

这一节的实验不多，但每一组都对应一条结论。表 23 把它们放在一起：算例、数据文件、图/表、脚本。所有脚本在 `code/` 下，数据在 `工作/数据/` 下；运行环境是 Python 3.12、NumPy 2.5、SciPy 1.18、Matplotlib 3.11，线性方程组用 `numpy.linalg.solve`，误差积分用 8 点 Gauss。

表 23　实验与复现清单

实验	算例	数据	图/表	脚本
常系数收敛	$-u'' = 2$，$u = x(1-x)$	表 3.3.3-3-收敛阶.csv	表 20 / 图 49、图 50	`make_fig_data.py`
求积阶数	$u = \sin \pi x$，$Q = 1, 2, 4$	表 3.3-8-求积阶数.csv	表 19 / 图 48	`make_exp33_ext.py`
变系数	$a = 1 + \frac{1}{2}\sin 2\pi x$	表 3.3-7-变系数收敛.csv	表 21 / 图 51	`make_exp33.py`
界面失配	$a = 1/4$，接口集中力	表 3.3-11-界面失配.csv	表 22 / 图 52	`make_exp33_ext.py`
二维三角形	单位方形，$u = \sin \pi x \sin \pi y$	表 3.3-10-二维收敛.csv	表 14 / 图 41	`make_exp33_ext.py`
条件数	一维 Poisson	表 3.3-5-条件数.csv	表 15 / 图 42	`make_exp33.py`

实验	算例	数据	图/表	脚本
鱼竿	三节变截面杆	表 3.3-4-鱼竿.csv	表 24	正文代码

表 23 的最后一列是“账本”：常系数与变系数验证 2/1；求积实验说明节点超收敛的前提；界面实验说明折角必须落在节点上；二维实验说明同一套装配在三角形上仍然成立。把四组放在一起，才能说“有限元是二阶的”这句话有边界条件：说范数、说光滑性、说网格与界面对齐、说载荷积分足够准。

复现时最容易错的三处也列在这里：第一，误差积分要在物理坐标上做，不能把参考坐标直接代进 f；第二，误差范数要用足够高阶的求积算，否则会把求积误差当成离散误差；第三，画收敛阶时横轴用 h 而不是 N，否则斜率会差一个符号。三处都不难，但每一处都能让“看起来对”的图变成错的。

科普丨有限元在今天。　今天的商用仿真软件里，装配这一步仍然叫 assemble。一个手机模型几十万单元，整体矩阵上亿个非零元，靠稀疏直接法或共轭梯度法求解；工程师不会手写 2×2 小块，但每一步做的仍是本节这三件事：算单元、按编号装、解矩阵。区别只在规模，不在思想。

- 常系数、精确求积： L^2 观测阶约 2， H^1 约 1，与 Céa 引理 + 一维线性插值估计一致。
- 载荷改用 1 点 Gauss：节点值不再精确，节点误差降到 $O(h^2)$；这是“求积不足”的直接后果。
- 变系数 $a=1+\frac{1}{2}\sin 2\pi x$： a 光滑且求积足够时阶不变；用中点 a 近似会引入额外误差。
- 界面 $x=0.5$ 与节点对齐时仍是 $2.00/1.00$；错位到 $x=0.53$ 时（本包 Notebook 的均布载荷算例） L^2 误差在 2×10^{-5} 附近不再下降， H^1 阶降到约 0.7；正文 3.3.5.3 节的集中力算例给出 $L^2\approx 1.0$ 、 $H^1\approx 0.5$。两个算例都说明：界面必须落在节点上。
- 结论：二阶/一阶不是“有限元天然如此”，而是“弱形式 + 分片线性 + 足够求积 + 网格贴合界面”共同保证的。

3.3.6　趣味编程：给鱼竿量“腰力”

周末，老陈在河边掏出一根三节鱼竿，说：“这竿子腰力好。”他把竿梢弯成一个圆弧，又弹回来。不妨算一算：如果一条鱼沿着竿轴方向拉，竿梢会被拉长多少？

先泼一盆冷水：鱼竿真正弯的是横向，那是梁的四阶方程，本节不碰。但“拉长”这一小截，恰好是一道标准的一维变系数问题，而且可以用三节竿把装配这本账从头到尾记一遍。老陈听完，说：“行，先算拉，再算弯。”

本节只做三件事：第一，把三节竿写成变截面杆，得到方程 (43)；第二，用三个单元装配，得到 (44) 和那段十几行代码；第三，用四种算法和自重收敛验证，落到表 24 与图 54。三件事做完，这根竿的“拉长账”就记平了。

把鱼竿看成一根变截面杆， x 从手柄（ $x=0$）指向竿梢（ $x=L$）。设三节长度相同，截面积分别为 A_1,A_2,A_3，弹性模量 E，材料密度 ρ。轴向位移 $u(x)$ 满足

$$-(EAu')'=\rho gA,\qquad u(0)=0,\qquad EA(1)u'(1)=P,\qquad (43)$$

其中 P 是竿梢的集中拉力， ρgA 是自重。左端固定是本质边界，右端受力是自然边界——正好把 3.3.2.2 节的两句话各用一次。

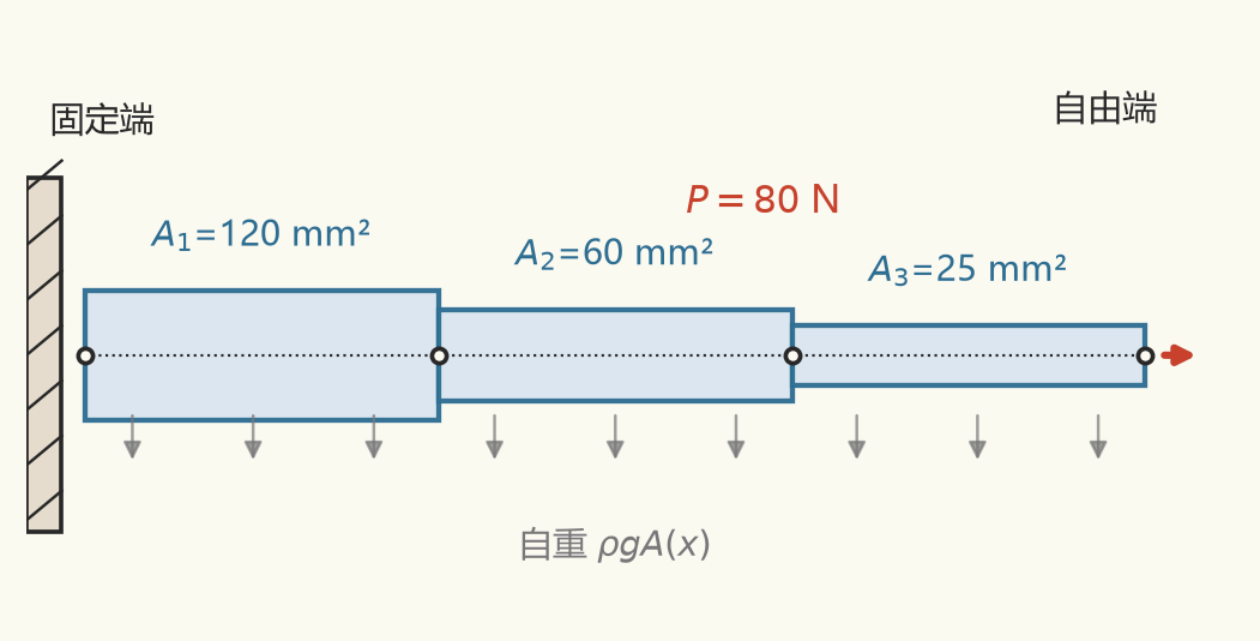


图 53　三节鱼竿的截面与受力

取 $L=1.8$ m，三节各 0.6 m，$A_1=120$ mm²、$A_2=60$ mm²、$A_3=25$ mm²，$E=120$ GPa，$\rho=1600$ kg/m³，$P=80$ N。先只算集中力，忽略自重。精确解是分段线性的，竿梢位移为

$$\delta=\frac{P}{E}\sum_{e=1}^{3}\frac{l_e}{A_e}.$$

代入数字：$\delta=26.0$ μm，约一根头发丝的三分之一。老陈听了直摇头：“我钓一条八斤的鱼，它猛地一挣（瞬时拉力按 80 N 估），竿子才拉长这么点？”——对，鱼竿主要弯，不怎么伸。

用有限元算，装配规则一字不改：每个单元用自己的 h_e 与 A_e，

$$K^{(e)}=\frac{EA_e}{h_e}\begin{pmatrix}1&-1\\-1&1\end{pmatrix},\qquad b^{(e)}=\frac{\rho g A_e h_e}{2}\begin{pmatrix}1\\1\end{pmatrix},\tag{44}$$

再按全局编号累加，把 P 加到最后一个自由度上。这段代码只保留骨架：`fem` 按单元循环，先算 $K^{(e)}$ 与 $b^{(e)}$，再把 2×2 小块加到全局矩阵的对应位置；`A[min(...)]` 用单元中点判断它属于哪一节，`tip` 与 `weight` 两个开关分别控制集中力与自重。代码如下（只保留核心十几行）：

```
import numpy as np
E, rho, g, P = 120e9, 1600.0, 9.81, 80.0
L, A = 1.8, np.array([120.0, 60.0, 25.0]) * 1e-6
def fem(N, tip=True, weight=True):
    h = L / N; K = np.zeros((N + 1, N + 1)); b = np.zeros(N + 1)
    for e in range(N):
        Ae = A[min(int((e + 0.5) * h / 0.6), 2)]
        K[e:e+2, e:e+2] += E * Ae / h * np.array([[1, -1], [-1, 1]])
        b[e:e+2] += rho * g * Ae * h / 2 if weight else 0.0
    if tip: b[-1] += P
    u = np.zeros(N + 1); u[1:] = np.linalg.solve(K[1:, 1:], b[1:])
    return u[-1]
print(fem(3) * 1e6)   # 26.000
```

表 24　四种算法给出的竿梢位移

算法	竿梢位移（μm）	相对误差	说明
逐段精确（三节）	26.000	—	基准
三单元 FEM	26.000	0	与基准一致
全竿按平均截面 68.3 mm²	17.561	−32.5%	把竿当成等截面
全竿按手柄截面 120 mm²	10.000	−61.5%	只用手柄截面

三个结果放在一起，说明了两件事。

第一，三单元 FEM 与精确解一模一样，误差在机器精度。原因不是巧合：精确解是分段线性的，而线性元空间恰好包含所有分段线性函数；当网格节点与材料接口重合时，精确解就落在 V_h 里，Galerkin 解只能等于它。这叫“空间包含解”，比“高阶精度”更彻底。

这个“一模一样”有一个前提：三个单元边界与材料接口重合。若把接口挪到单元内部，三单元就不再精确，误差会按 3.3.5.3 节的界面失配方式掉阶。

第二，按平均截面算，少算 32.5%；按手柄截面算，少算 61.5%。差别全在最软的那一节：$A_3 = 25$ mm² 只有手柄的 $1/4.8$，它的柔度 $\int \mathrm{d}x/(EA)$ 却占了总柔度的 $24000/39000 \approx 62\%$。把整根竿当成一根粗杆，等于把最软的一节忽略了。

再把自重加回去。自重的总重量只有 1.93 N，额外位移 0.12 μm，约占 0.5%；但它的误差随网格按 L^2 二阶、H^1 一阶收敛（图 54），因为自重对应的精确解是分段二次的，不再落在 V_h 里。集中力让解落进空间，自重把解推出空间——同一根竿，两种行为。

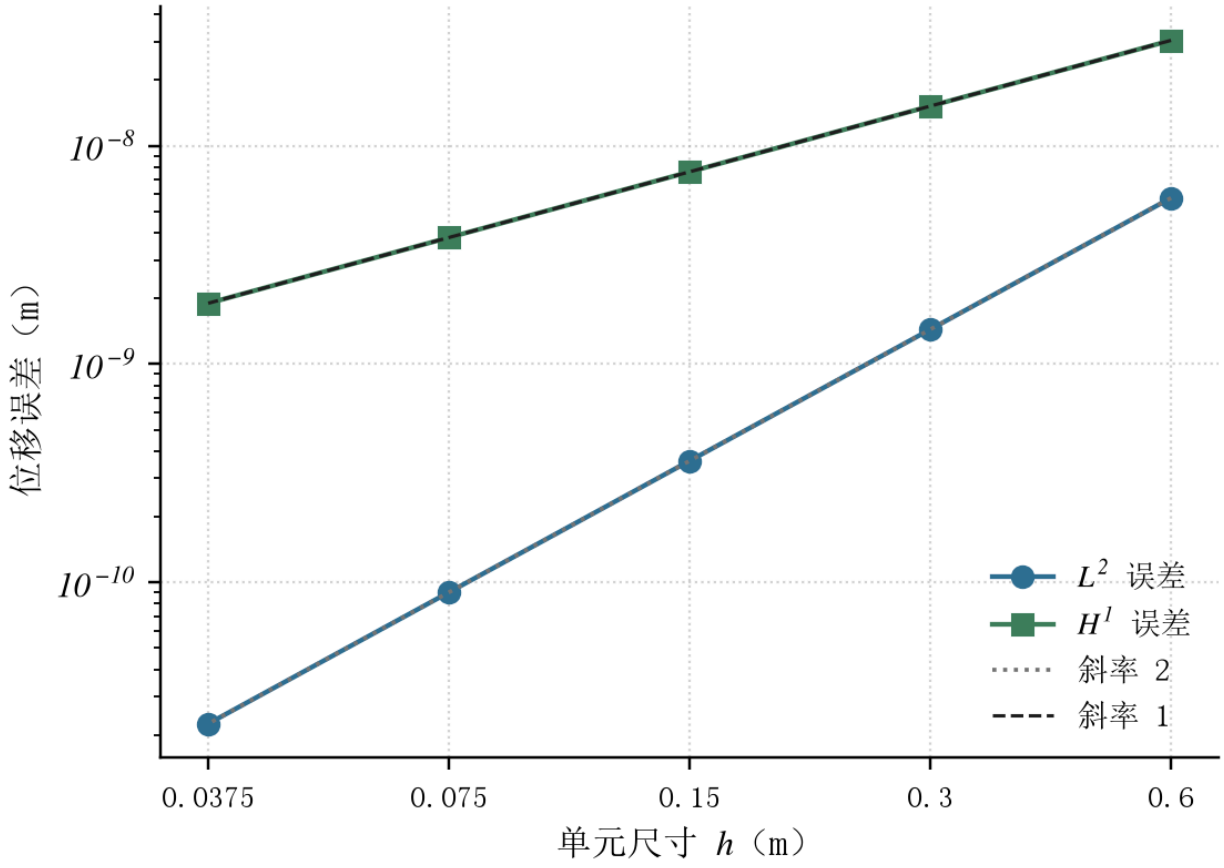


图 54　自重位移的收敛

老陈最后问：“那我这竿子到底算好还是不好？”答案不在这里：竿子的“腰力”取决于弯曲刚度 $EI(x)$ 的分布，要解四阶方程，需要 C^1 单元或混合格式。本节只算清了“拉长”这一小截，但装配的账、边界的账、误差的账，已经从头到尾记了一遍。下一根竿，换一个方程，账本还是这本账。

3.3.7　有限元的今天

1960 年代，有限元从论文走进工业。NASA 在 1965 年组织开发 NASTRAN，1970 年 ANSYS 成立，1978 年 Abaqus 出现；到 1980 年代，汽车厂已经用整车模型做碰撞，飞机厂用整机模型做静强度。方法没有变，变的是规模：今天一辆车的碰撞模型上千万单元，一部手机跌落的模型几十万到上百万单元，一张芯片的封装模型要同时算热和力。

规模变大，算法也要跟着变。三对角在二维变成带状、在三维变成更宽的带状；直接法要靠稀疏消元、嵌套剖分和 GPU 加速，迭代法要靠共轭梯度、多重网格和预处理。3.3.4.2 节的“非零元只有三万个”到了三维会变成几亿个，但“只存非零元”这句话没有变。

有限元还从“算一个结果”变成了“改一个设计”。1988 年 Bendsøe 与 Kikuchi 把材料密度当作设计变量，提出拓扑优化：在给定的载荷与体积约束下，让算法自己决定材料该放在哪里[54]。今天的汽车支架、飞机舱门、人工关节，很多是先由拓扑优化给出形状，再拿有限元复核，最后交给 3D 打印。图 55 是这条流水线。

另一条线是“算得更快”。显式动力学把每一步写成矩阵—向量乘法，适合碰撞、冲击、爆炸；隐式动力学每步解一个稀疏系统，适合长时间、准静态。GPU 把这两类计算都推快了一个数量级；区域分解把上亿自由度的模型拆到几千张卡上。今天的工程师不必手写 2×2 小块，但 3.3.4.1 节的装配账仍然是并行剖分的依据。

还有两条支线。等几何分析（2005）把 CAD 的 NURBS 基函数直接当单元基函数，省掉“几何—网格”转换，让复杂曲面在离散前后保持同一张脸[55]。数字孪生则把传感器数据回灌到模型：飞机机翼、风机叶片、桥梁，一边测一边更新有限元模型，让“算出来的”和“量出来的”慢慢对齐。

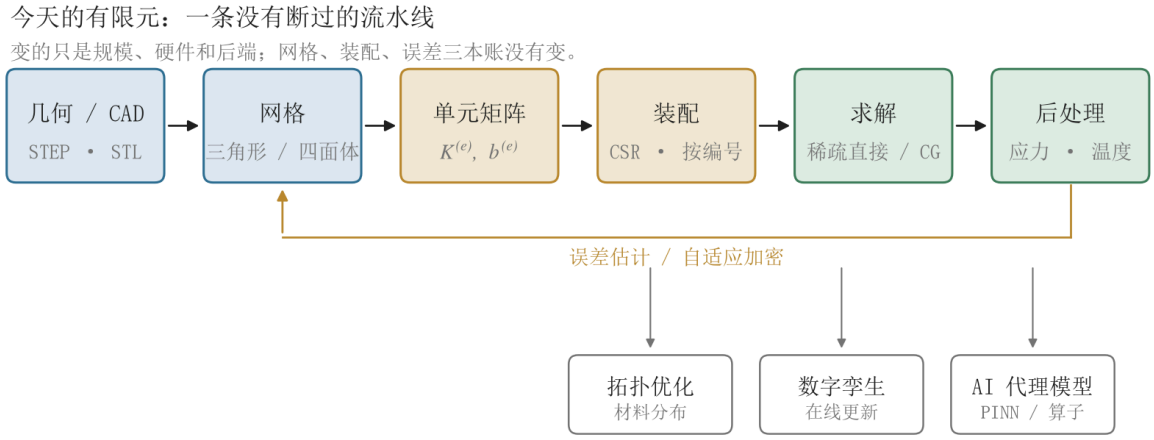


图 55　有限元的今天：从几何、网格、装配到求解与后处理；误差估计把后处理接回网格

最近十年，AI 也进来了。2019 年 Raissi 等人提出 PINN，把方程残差写进损失函数[56]；2021 年 Li 等人提出 Fourier 神经算子，直接学“输入函数到输出函数”的映射[57]。它们的目标不是替代有限元，而是补上有限元最贵的那一段：要算几千次参数扫描时，用一个训练好的代理模型先筛，再用有限元复核。图 56 是四个典型场景：整车碰撞、手机跌落、人工关节、芯片与电池。它们的单元数、时间尺度、材料模型都不同，但都要回答同一件事：网格够不够细，误差可不可信。

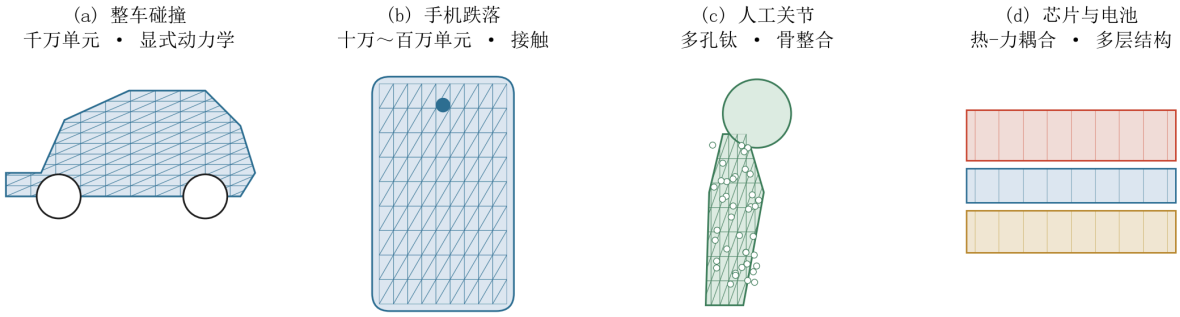


图 56　有限元的四个今天：整车碰撞、手机跌落、人工关节、芯片与电池；尺度不同，账本相同

所以“有限元的今天”不是某一项新技术，而是三条线同时变粗：规模从万到亿，目标从“算一次”到“改设计”，工具从手写代码到商业软件、开源库和 AI 代理模型。三条线底下还是那三本账：网格、装配、误差。网格可以自适应，装配可以并行，误差可以用 AI 估计，但账本没有换。

下次你看到一辆车的碰撞视频、一部手机的跌落仿真、一颗人工关节的应力云图，可以想起这一节：那些彩色云图的背后，仍然是一顶顶帽子、一个个 2×2 小块，和一次按编号的累加。

第四节（3.4）　AI 关联：Neural ODE 与 PINN

从“拟合一条曲线”到“学一个向量场”

3.4.0　引子：一条曲线，两台机器

一位做微生物实验的朋友发来三条几乎重合的生长曲线：横轴是时间，纵轴是菌落面积。她要的不是再拟合出一条形状相同的曲线，而是回答“如果第一天温度低两度，第三天会长成什么样”。换句话说，需要确定的不是 y，而是 $y'=f(t,y)$ 中的 f——曲线背后的规律。

这就是“反问题”。正问题手里有方程，任务是把解算准；反问题手里只有解的若干次观测，方程未知，要把它反推出来。它并不新鲜：开普勒用第谷二十年的观测反推行星轨道，工程师用测得的应力反推材料参数，走的都是这条路。新鲜的是工具：过去五年，神经网络把这件事从“专家手工建模”变成了“可以训练的对象”。

2024 年 10 月的诺贝尔奖使这一趋势受到广泛关注：化学奖的一半给了用 AlphaFold 预测蛋白质结构的 Demis Hassabis 与 John Jumper，另一半给了用计算方法设计蛋白质的 David Baker；物理学奖给了神经网络的两位奠基者 John Hopfield 与 Geoffrey Hinton。前者是“用机器学习解决科学问题”，后者是“用物理学的语言理解神经网络”——两条线索在同一周交汇，这就是今天所说的 AI for Science。

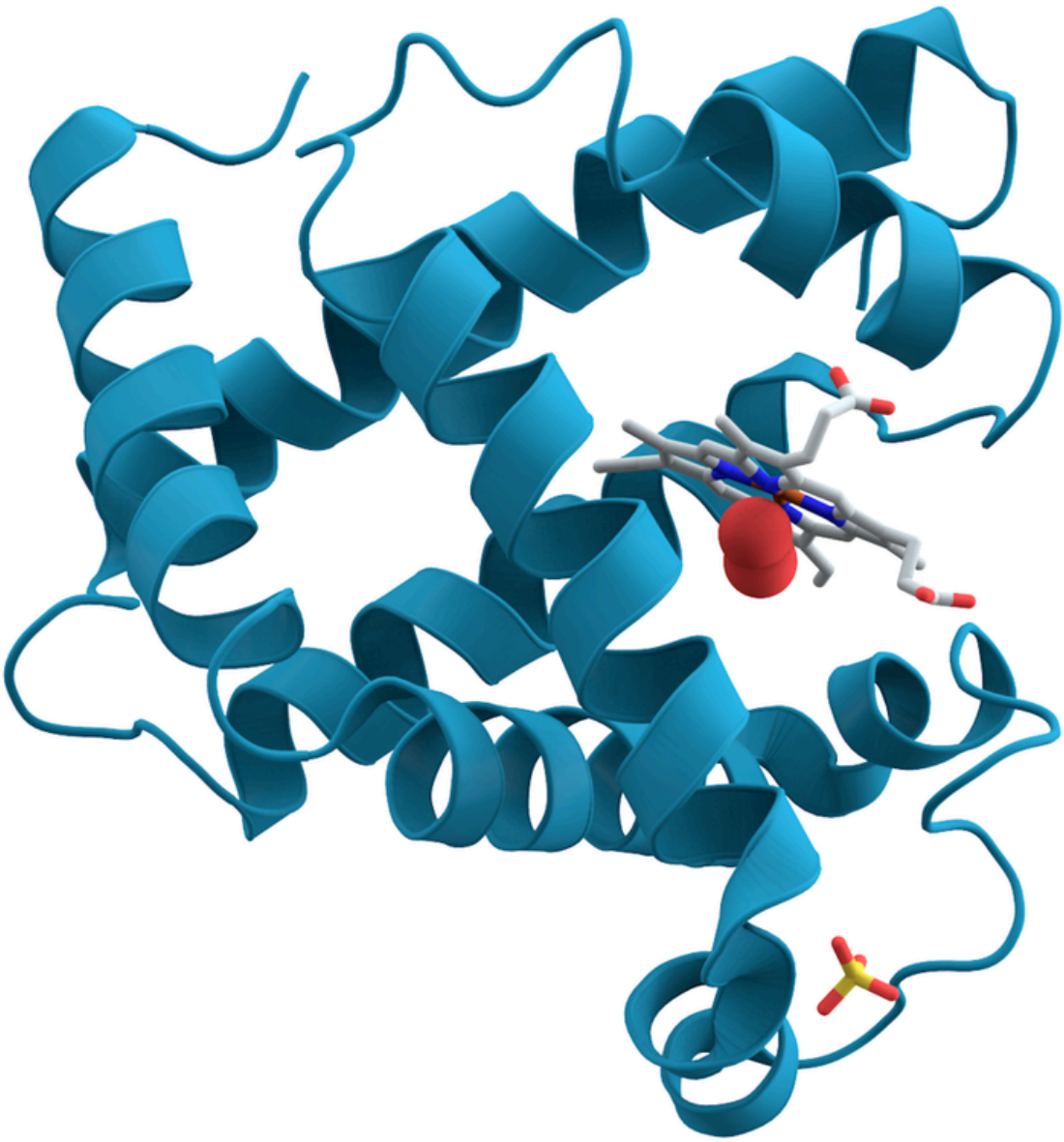


图 57　鲸肌红蛋白的三维结构：人类解析出的第一个蛋白质结构（肯德鲁，1958；1962 年诺贝尔化学奖）。AlphaFold 这类模型要预测的，正是这样的形状。图片来源：Wikimedia Commons，公有领域

就“用机器学方程”而言，2018 年以后出现了两条清晰的技术路线，时间顺序十分清楚。

先是 2018 年底：Chen、Rubanova、Bettencourt 与 Duvenaud 把 ResNet 的“一层一次加和”看成“一步 Euler 积分”，进而问——为什么不把层数取到无穷、让网络直接学连续时间的向量场？这就是 Neural ODE[58]。它学的是 f，不是 y。

紧接着是 2019 年：Raissi、Perdikaris 与 Karniadakis 提出 PINN[56]——不拟合数据曲线，而是让网络的输出直接满足方程，把“方程残差”当损失。两者的区别可以概括为一句：Neural ODE 学右端函数，PINN 解已知方程。

两条路线可用图 58 对照。二者都遵循同一条“输入 → 网络 → 输出 → 损失”的流程，但网络所处的位置不同，所得模型也随之不同：置于右端，网络成为 f_θ，需经积分器才得到轨迹；置于解上，网络成为 u_θ，需对其求一次导数才得到残差。前者学的是规律，后者学的是这一次的具体轨迹。

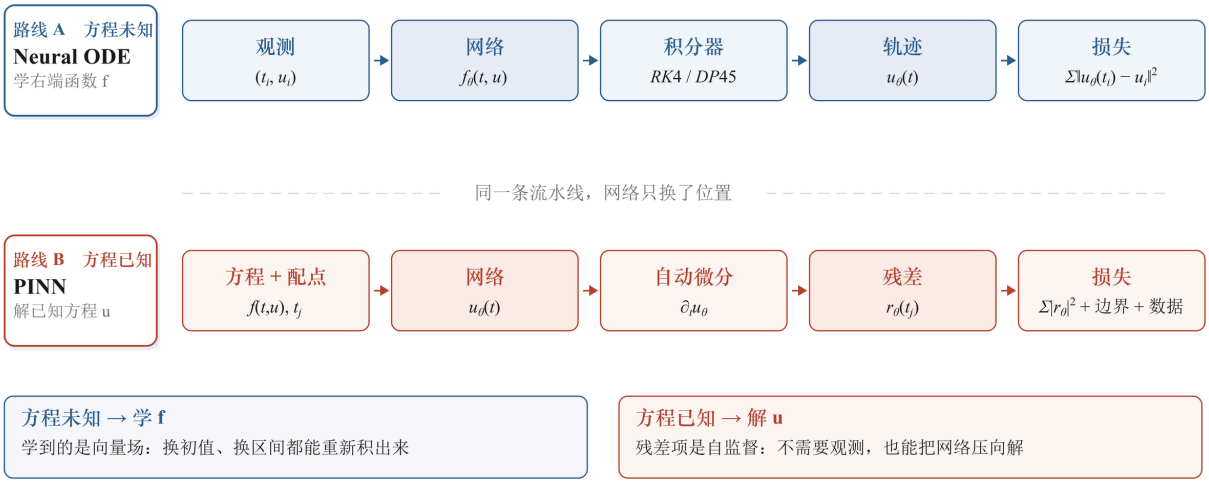


图 58　两条训练路线并排：同一条流水线，网络置于右端还是置于解上，决定了所学的是 f 还是 u。图片来源：本组绘制

用机器学习的语言表述：Neural ODE 是函数空间中的回归——回归的对象不是数据点，而是“从状态到斜率”的映射；PINN 则是以方程为约束的正则化——损失中 $\sum|r_\theta|^2$ 一项不含任何标签，属于自监督项（前者对应 3.4.1.1，后者对应 3.4.2.1）。

这两条路线在国内都有落脚点，而且起步并不算晚。由鄂维南等学者倡导的“AI for Science”，在 2018 年前后同时在国内学界与产业界铺开：飞桨的 PaddleScience 把 PINN、XPINNs 等格式做成了可直接调用的求解器与示例[61]，昇思的 MindSpore Science 提供微分方程求解与科学计算算子库，北京大学团队还基于它发布了二维偏微分方程的基础模型 PDEformer-2[62]；DeepModeling 社区的 DeePMD-kit 则把机器学习势函数送进了材料与化学的日常计算。这些工具的共同点在于：本书讲过的积分器、残差与边界条件，都被封装成了参数，因此选型的差别最终落在求解器的选择与残差的容忍度上。

图 59　鄂维南院士在 2021 世界人工智能大会上作报告。图片来源：百度百科

图 60　左：飞桨（PaddlePaddle）标识；右：昇思（MindSpore）标识。图片来源：飞桨官网、百度百科

本节的任务是把这套方法拆解开来。可以用一个比喻概括主线：正问题相当于照着菜谱做菜；反问题则相当于只尝一口，就要把菜谱反推出来，而且写出的菜谱必须能做出整桌菜，而不只是还原这一口。

下面三节依次回答三个问题。3.4.1 节：方程怎样写成一个网络？关键不在于记住某一条轨迹，而在于学出“在每一点、每一刻该往哪走”的向量场，并弄清它成立需要什么条件。3.4.2 节：结果好不好，用什么标准衡量？是让轨迹贴合观测，还是让方程处处成立——两种标准各有各的缺陷。3.4.3 节：怎样确认结果确实正确？最容易被忽略的是求解过程本身也会出错：积分器的离散误差会进入参数，因此必须在未用于训练的初值、步长和区间上重做一遍。

3.4.4 节以一条真实的菌落生长曲线为例，用几十行代码把这三件事完整实现一遍；3.4.5 节转而讨论这两条线今天的产业形态、它们与 AI 的关系，以及尚待解决的问题。

3.4.1　学习右端函数

1601 年第谷去世，留下二十多年、精度空前的天文观测；开普勒用这些点，最终拟合出火星的椭圆轨道——他知道行星“会经过哪里”，却说不清“为什么”。半个多世纪后，牛顿给出万有引力：每一点上都有一个确定的加速度，行星只是顺着它走。开普勒学到的是一条曲线，牛顿学到的是产生这条曲线的场。

本节学的也是后者。给的只有若干观测点，要的是一张网 f_θ：它给出每一个时刻、每一个状态上的“下一步往哪走”。曲线只是这个场的一个投影——这一观点贯穿后续各节。

图 61　开普勒的椭圆轨道与牛顿的运动方程：从观测曲线，到受力分析，再到微分方程。图片来源：豆包生成

3.4.1.1　从拟合曲线到拟合斜率

先把两个对象分开：曲线是一条轨迹留下的痕迹，向量场则是每一点、每一刻都存在的“水流”。观测数据只提供了几张照片——船在水面上的位置，而 Neural ODE 要学的是水流本身。学到水流之后，无论把船从何处、何时放入，都能推断它的走向；只学到那条曲线，换一条船就必须重来。

已知观测点 (t_i, u_i)，$i=1,\dots,N$，来自某个未知的初值问题

$$u' = f(t,u), \qquad u(0)=u_0. \tag{45}$$

传统做法是先假定 $u(t)$ 的函数形式（如 logistic），再用最小二乘确定几个参数。Neural ODE 改变的正是被假设的对象：它假设的是 f。

定义 1（参数化向量场与轨迹）　设 $f_\theta : \mathbb{R}\times\mathbb{R}^d \to \mathbb{R}^d$ 是由参数 θ 决定的神经网络。称初值问题

$$\frac{\mathrm{d}u_\theta}{\mathrm{d}t} = f_\theta(t,u_\theta), \qquad u_\theta(0)=u_0 \tag{46}$$

的解 $u_\theta(t)$ 为该网络给出的轨迹。

注意式 (46) 与式 (45) 形状完全相同，只是把 f 换成了 f_θ。这一步替换带来两个后果：有利的一面是，f_θ 一旦学到，任意初值、任意区间的轨迹都可以重新积出；代价是式 (46) 没有解析解，只能依赖数值积分，而积分器的误差会进入训练过程。

训练目标就是让轨迹贴合观测：

$$\mathcal{L}_{\text{traj}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \big\| u_\theta(t_i) - u_i \big\|^2 + \lambda \big\| u_\theta(0) - u_0 \big\|^2. \tag{47}$$

式 (47) 里没有出现 f 的任何“真值”，f 只是在积分过程中被反复调用。这正是它与普通回归的本质区别：拟合的对象是曲线背后的向量场，曲线只是这个向量场的一个投影。

把这件事写成映射会更清楚。固定 θ 与初值 u_0，式 (46) 的解定义了一个映射 $\varphi_\theta:(t,u_0)\mapsto u_\theta(t)$，称为流映射。它满足半群性质

$$\varphi_\theta(t+s,u_0)=\varphi_\theta\big(t,\varphi_\theta(s,u_0)\big),$$

先走 s 再走 t，与一次走 $t+s$ 相同。这条性质有两个直接后果。其一，学到 f_θ 之后，任意初值、任意时刻的轨迹都是同一个映射的不同切片——“换初值外推”不是额外的技巧，而是这一表示的必然结果。其二，只要 f_θ 是 C^1 的，$\varphi_\theta(t,\cdot)$ 对每个 t 都可逆（反向积分即可还原），这是 ResNet 那类离散网络做不到的。连续归一化流（CNF）正是靠它把“可逆变换 + 密度变换”搬进了生成模型：密度沿轨迹的变化率是 $-\nabla\cdot f_\theta$，积分起来就得到对数似然，不必去算 Jacobian 行列式[58]。

由此产生一个自然的问题：既然要拟合的是一个函数，为什么必须用神经网络？多项式、样条、小波同样是函数族，而且更古老、更便于分析。

答案在于两者的参数增长方式不同。以三次张量积多项式为例：d 个变量、每个方向 ≤ 3 次，基函数有 $\binom{d+3}{3}$ 个，是 $O(d^3)$ 量级；而要把精度做到 ε，需要的基函数个数是 $O(\varepsilon^{-d/3})$——指数上挂着维数 d。神经网络的情形则不同：宽度 w、深度 L 的网络大约有 $O(Lw^2)$ 个参数，与 d 无关；对 Barron 型函数类，逼近误差是 $O(1/\sqrt{N})$，N 是参数个数，指数上也没有 d[74]。换言之，维数升高时，基函数一侧的参数按 10^d 的方式增长，而网络一侧只需适当增加宽度。这是深度学习能在高维问题上立足的根本原因，也是 3.4.2.4 中“PDE 基础模型开始讨论规模”的伏笔。

这一比较有前提：它针对的是“光滑度受限、维数却很高”的函数类。若问题是一维的、函数是解析的，谱方法与 Chebyshev 基函数明显优于网络：速度快、精度高，而且附带误差估计。所以准确的说法不是“神经网络万能”，而是“神经网络是维数高、结构弱、样本多时的默认选择”。

图 62 把这两件事画在一起：左、中两格是“学一个场”的示意——灰色箭头是真实的 f，蓝色箭头是学到的 f_θ，三条轨迹换了初值仍然贴着真解；右一格对应上面的比较，横轴是状态维数，纵轴是达到同一精度所需的参数个数。

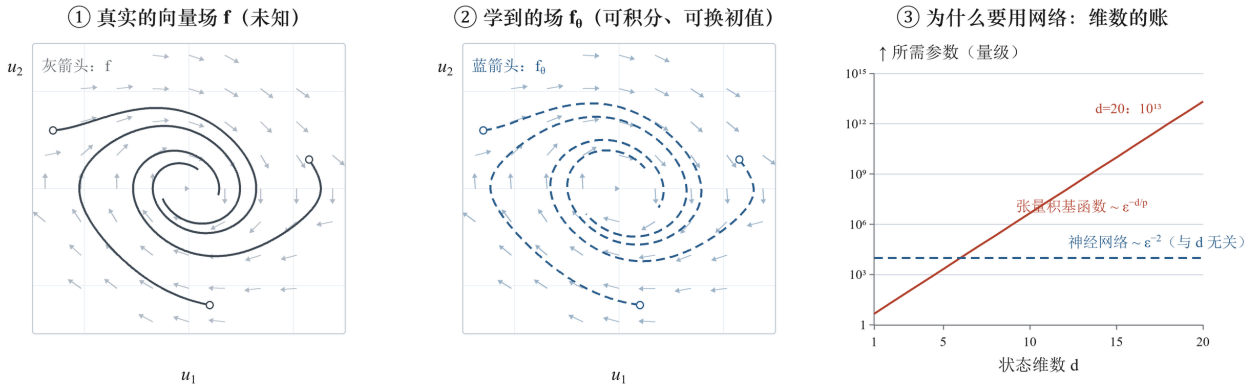


图 62　表示能力的两个侧面：左、中，学到的向量场（蓝）与真场（灰）几乎重合，换初值仍然适用——学到的是场而不是曲线；右，达到同一精度所需参数个数随维数的增长（量级估计，取 $\varepsilon=10^{-2}$、光滑度 p=3）。本组计算并绘制

科普 | 一层网络就是一步 Euler　ResNet 的一个残差块写成 $h_{t+1} = h_t + F(h_t)$，与显式 Euler 的 $u_{k+1} = u_k + h\,f(t_k, u_k)$ 对照：只要规定“每穿过一层就前进一个单位时间”，F 就是那个 f，层数就是步数[76]。Neural ODE 把这条递推取极限——层数趋于无穷、步长趋于零，离散的层层堆叠就变成一段连续的时间。

这类离散化并不遥远。手机游戏、动画乃至支付成功后那段“到账”动效，都是一帧一帧算出来的：每一帧只问一次“下一瞬间在哪”，就是 Euler 走一步；帧率从 24 提高到 120，本质上是把 h 缩小，使离散过程逼近连续过程，画面才显得连贯。反过来，Neural ODE 让求解器自行选择步长：平缓处取大步、陡峭处取小步。这正是“学一个连续系统”在算力上优于“固定层数堆叠”的原因。

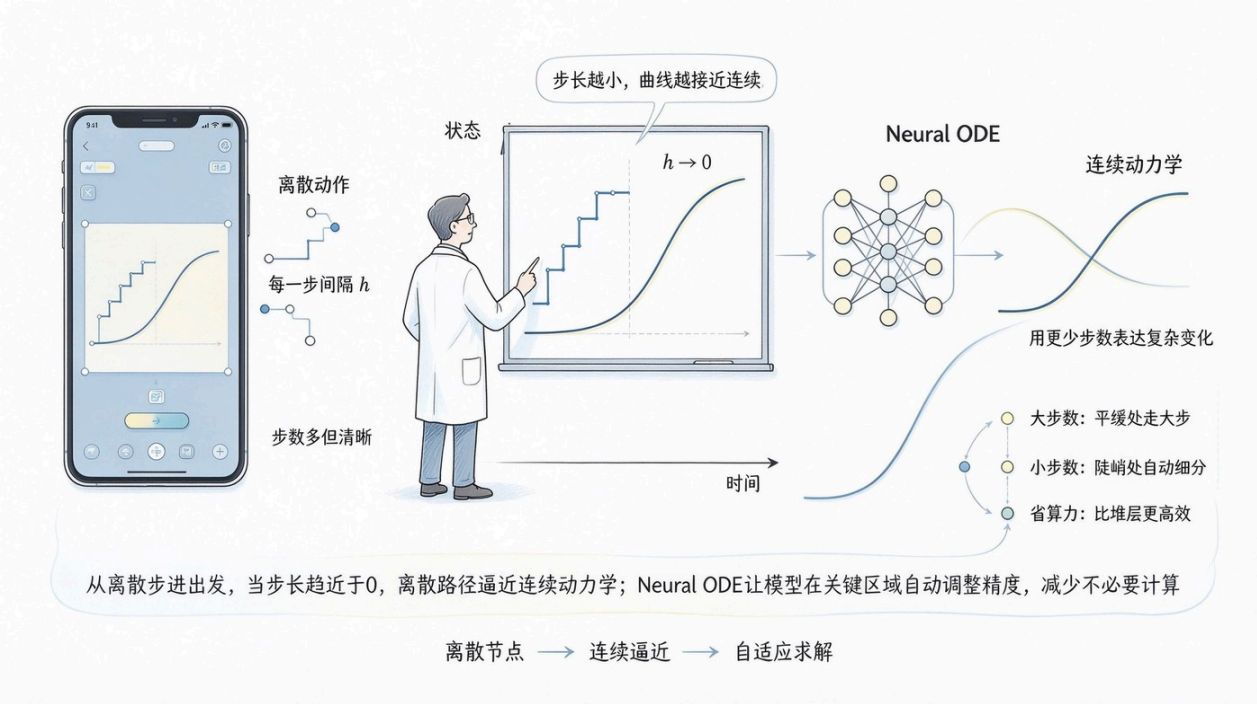


图 63　从离散步进到连续动力学：步长 h→0 时离散路径逼近连续曲线，Neural ODE 让模型在关键区域自动加密。图片来源：豆包生成

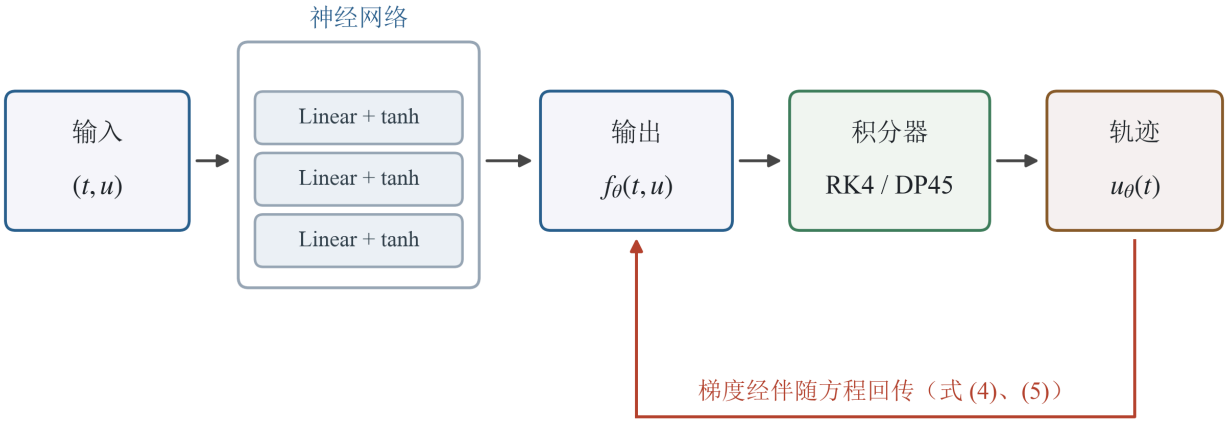


图 64　Neural ODE 的结构：网络只负责给出斜率 $f_\theta(t,u)$，推进轨迹的是积分器。本组绘制

3.4.1.2 存在唯一性与连续依赖

把 f 换成网络，存在唯一性与连续依赖这两条结论能不能照搬？能，只要网络满足 Lipschitz 条件。

这个条件并非多余。取 $f(u)=\sqrt{|u|}$、$u(0)=0$：$u\equiv 0$ 与 $u(t)=t^2/4$ 都满足方程，初值问题有无穷多个解——残差处处为零，却不能说“解对”。网络里若出现这类“平方根型”的尖点（非光滑激活、手工拼接的项），同样的失效就会出现。

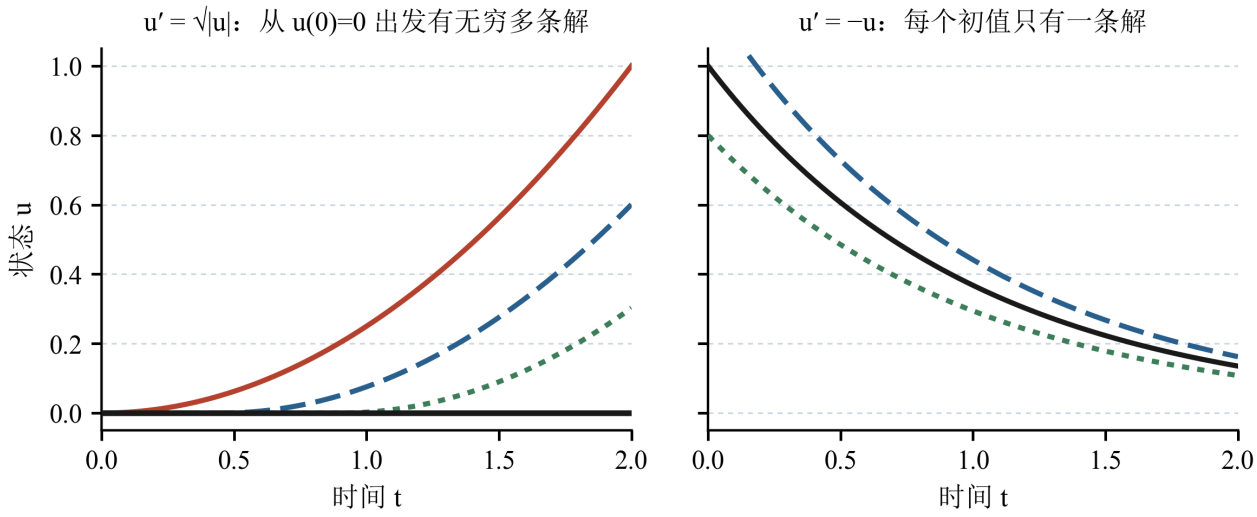


图 65　Lipschitz 条件并非多余：左图 u’=√|u| 从 u(0)=0 出发有无穷多条解，右图 u’=−u 的每个初值只有一条解。本组计算并绘制

命题 1　设 f_θ 在 D 上连续、关于 u 满足 Lipschitz 条件（常数 L）。则对任意初值 $(0,u_0)\in D$，式 (46) 在某个区间 $[0,\delta]$ 上存在唯一解，且解连续依赖于 u_0 与 θ。

证明　重复 Picard–Lindelöf 定理的证明，只把 f 换成 f_θ：f_θ 连续有界、Lipschitz，故 Picard 迭代是压缩映射，存在唯一不动点。连续依赖性由 Gronwall 估计直接给出。证毕。

证明里最要紧的一步是压缩常数。式 (46) 等价于积分方程 $u(t)=u_0+\int_0^t f_\theta(s,u(s))\,\mathrm{d}s$，Picard 映射 T 满足

$$\|T(u)(t)-T(v)(t)\|\le L\int_0^t\|u(s)-v(s)\|\,\mathrm{d}s\le L\delta\,\|u-v\|_\infty,$$

只要 $\delta<1/L$，T 就是压缩映射，Banach 不动点定理直接给出存在唯一。这也解释了命题 1 为什么只保证“某个小区间”上可解：$L\delta<1$ 是局部条件，L 越大、能保证的区间越短。

这条命题的实用价值在于：它说明了哪些网络“可用”。带 $\tanh$ 或 SiLU 激活的多层感知机，其输出关于输入是 Lipschitz 的（复合函数的 Lipschitz 常数是各层常数之积），所以命题 1 成立；而带 ReLU 的网络在原点附近虽然仍 Lipschitz，但其导数不连续，会让后面的伴随方程变得敏感。

网络的 Lipschitz 常数可以直接算出：若第 i 层权重矩阵的谱范数为 s_i、激活函数的 Lipschitz 常数为 ℓ（$\tanh$、SiLU、ReLU 的 ℓ 都等于 1），则整网 $L\le\prod_i s_i\ell$。深度增加时该乘积指数增长——“深层网络对输入极其敏感”并非修辞，而是这个乘积的直接结果。工程上的对策是谱归一化：把每层的 s_i 约束在 1 附近，让 L 不随深度爆炸[67]。

这一性质在深度学习中有更出名的名字：对抗样本。在一张熊猫照片上叠加肉眼几乎不可见的扰动（$\|\delta\|_\infty \approx 10^{-2}$），网络却以很高的置信度把它判为长臂猿——这不是网络“看错”，而是它的 L 过大：输入上 10^{-2} 的误差被放大成输出上的巨大变化[77]。2013 年这一现象刚被发现时被视为深度学习的缺陷，后来逐渐被理解为高维函数逼近的固有性质： L 大既是网络能够拟合复杂函数的前提，也是它脆弱的根源。对 Neural ODE 来说，这件事直接落在积分上—— L 大， e^{Lt} 就把初值噪声迅速放大，学到的那条轨迹只在训练区间起点附近才可信。

还要注意一个容易被忽略的细节： L 越大，连续依赖估计 $|u(t)-z(t)| \le |u_0-z_0|e^{Lt}$ 越松。深层网络的 L 往往很大，这意味着初值的一点点噪声会被放大成轨迹上的大偏差——这也是 3.4.3 节“换初值验证”的动机之一。

这不是抽象的顾虑。1963 年，气象学家 Edward Lorenz 在计算机上重新执行一次数值实验，为了省事把初值从 0.506127 记为 0.506——一个看似无碍的省略。头几天两条曲线几乎重合，几个“模式日”之后天气图彻底分道扬镳。九年之后，他把这件事概括成一句更出名的话：一只蝴蝶在巴西扇动翅膀，会不会在得克萨斯引起一场龙卷风？[63] 用上面的估计来读：初值只差 10^{-3} 量级，可大气的 L 大到让 e^{Lt} 在两周之内就把它放大到与气候信号同量级——因此两周以上的天气预报只能给出概率，无法给出确定答案。对训练 Neural ODE 的人而言，这一教训同样成立： L 大的网络，观测里的一点噪声会被积分成轨迹上的大偏差。

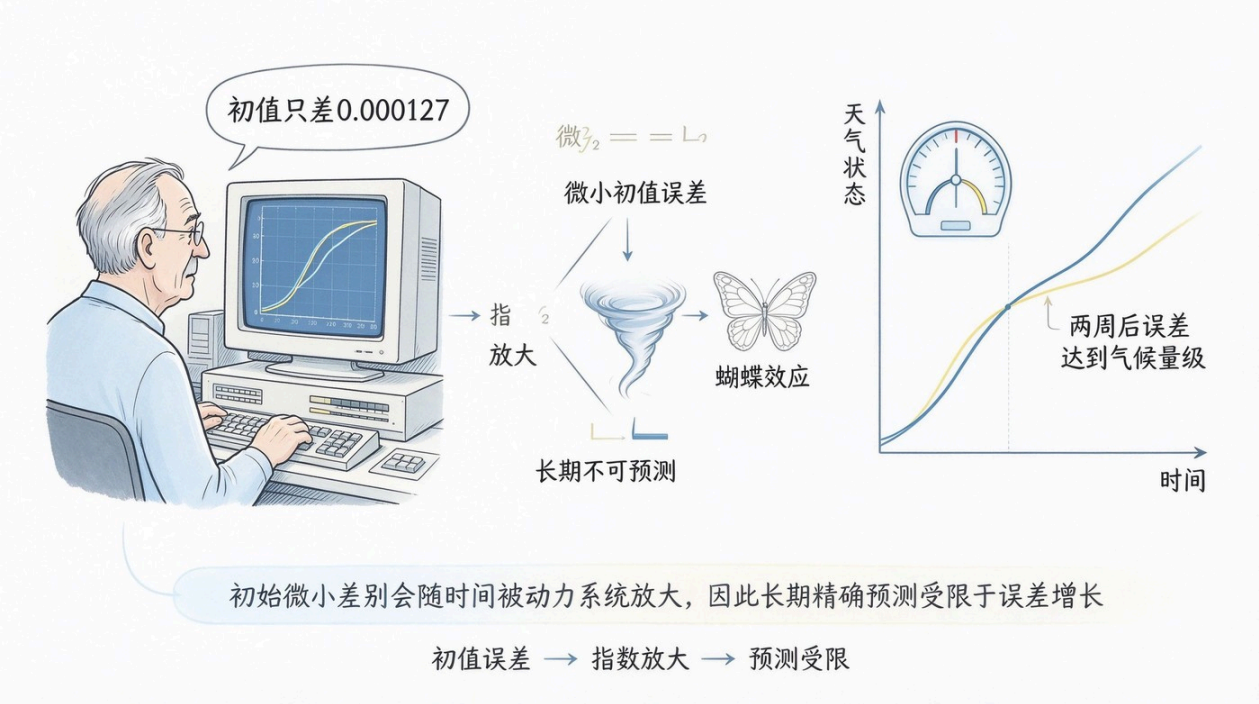


图 66　洛伦兹的两次模拟：初值只差 0.000127，误差被指数放大，两周之后已到气候量级。图片来源：豆包生成

3.4.1.3　伴随法：怎样穿过积分器求梯度

式 (47) 的梯度要穿过数值积分器。最直接的做法是“保存每一步、反向逐层求导”（离散反传）：前向 M 步需要保存 M 组中间量，内存随步数线性增长；步长小到 10^{-3}、区间长到 10^{3} 时，内存将无法承受。

Neural ODE 原论文给出的替代方案是伴随法：不解轨迹的离散链，而是另外解一个反向的连续方程 [58]。记伴随变量（拉格朗日乘子） $\boldsymbol{a}(t)=\partial\mathcal{L}/\partial u_\theta(t)$，可以证明它满足

$$\frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}t}=-\boldsymbol{a}^{\mathsf{T}}\frac{\partial f_\theta}{\partial u}(t,u_\theta),\qquad \boldsymbol{a}(T)=\frac{\partial\mathcal{L}}{\partial u_\theta(T)},\tag{48}$$

而参数梯度是一个积分

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\theta}=-\int_T^0\boldsymbol{a}(t)^{\mathsf{T}}\frac{\partial f_\theta}{\partial\theta}(t,u_\theta)\,\mathrm{d}t.\tag{49}$$

命题 2（伴随法的代价） 用伴随法求一次梯度需要额外解一个与状态同维的反向 ODE，内存为 $O(d)$，与时间步数无关；计算量为前向积分的约两倍。

证明要点 式 (48) 是把式 (46) 与 $\mathcal{L}$ 一起写成约束优化的 Lagrange 函数后，对 u 变分得到的：由于约束是逐点成立的 ODE，乘子也满足一个 ODE。反向积分从 T 到 0，每步只需当前状态与乘子，故内存与步数无关。计算量约为前向的两倍，因为每一步要算两次 Jacobian－向量积。证毕。

这一成本需单独计算。图 67 的横轴是参数个数 P，纵轴是计算一次全梯度所需的前向次数：有限差分为一条斜线（ $2P$ 次），反向模式与伴随法为两条水平线（约 2 次）。 $P=10^6$ 时两者相差 6 个数量级，这是“能否实现”的差别，而非“快慢”的差别。今天任何大模型的训练都建立在“梯度计算量与参数个数无关”这一事实之上；伴随法的作用，是把这一事实从“层”推广到了“时间”。

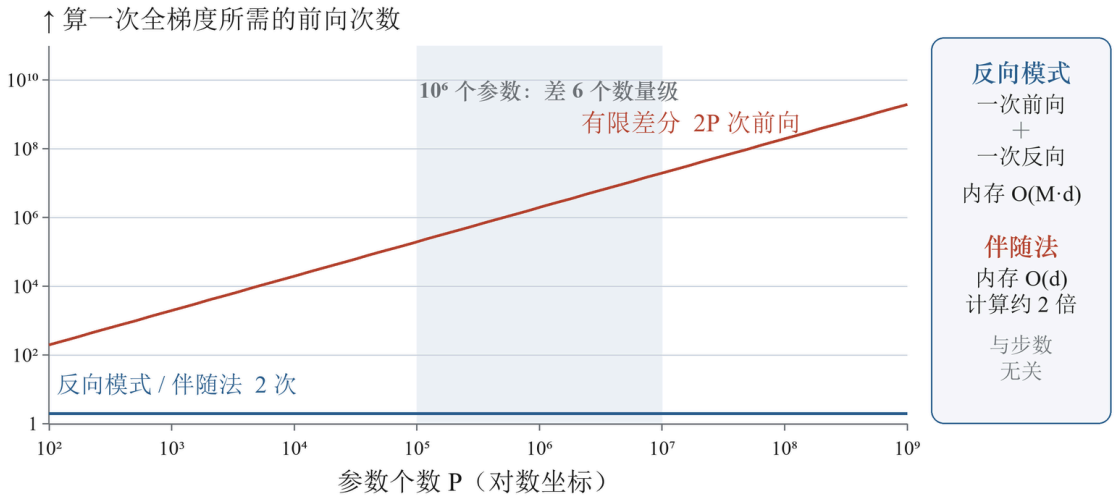


图 67 计算一次全梯度所需的前向次数：有限差分随参数个数线性增长，反向模式与伴随法均为常数。本组绘制

这条结论体现的正是精度与成本之间的权衡：伴随法用两倍时间换常数内存；离散反传用线性内存换一次前向。工程上通常按区间长度选择：短区间直接反传，长区间使用伴随法。

这里有个容易被忽略的细节：式 (49) 是“先连续求导、再离散”，而反向传播是“先离散、再求导”。两者给出的梯度并不严格相等，差一个 $O(h^p)$ 的量级（ p 为积分器的阶）。前者与积分过程解耦，训练中可以更换步长与格式；后者精确对应某一次前向计算，梯度与该次计算完全一致。实践中“训练中途更换求解器、损失曲线突然波动”，根源正在于此。

内存与计算之间还有第三条路：分段存、分段反传（checkpointing）。把前向切成若干段，只记段首状态，反传时重算段内——内存降到 $O(\sqrt{M}\,d)$，计算约两倍。分段数按显存容量确定，因此这是工程上最常用的折中[66]。三种做法没有绝对优劣，取决于这次训练更缺内存还是更缺时间。

这件事在 AI 里有更熟悉的名字：反向传播。一层层反传是把 $\partial\mathcal{L}/\partial\theta$ 沿计算图逐层链式相乘；伴随法是把同一个链式法则放进时间维，沿 ODE 的轨迹反向积分一次。差别只在“层”是离散还是连续——这也是为什么 PyTorch 的 `torchdiffeq` 只要接一个 ODE 求解器，就能把梯度算对[60]。今天大模型训练中用于节省显存的“分段重算”（gradient checkpointing），与 3.4.1.3 的分段存储、分段反传是同一思路：以重算换取内存。

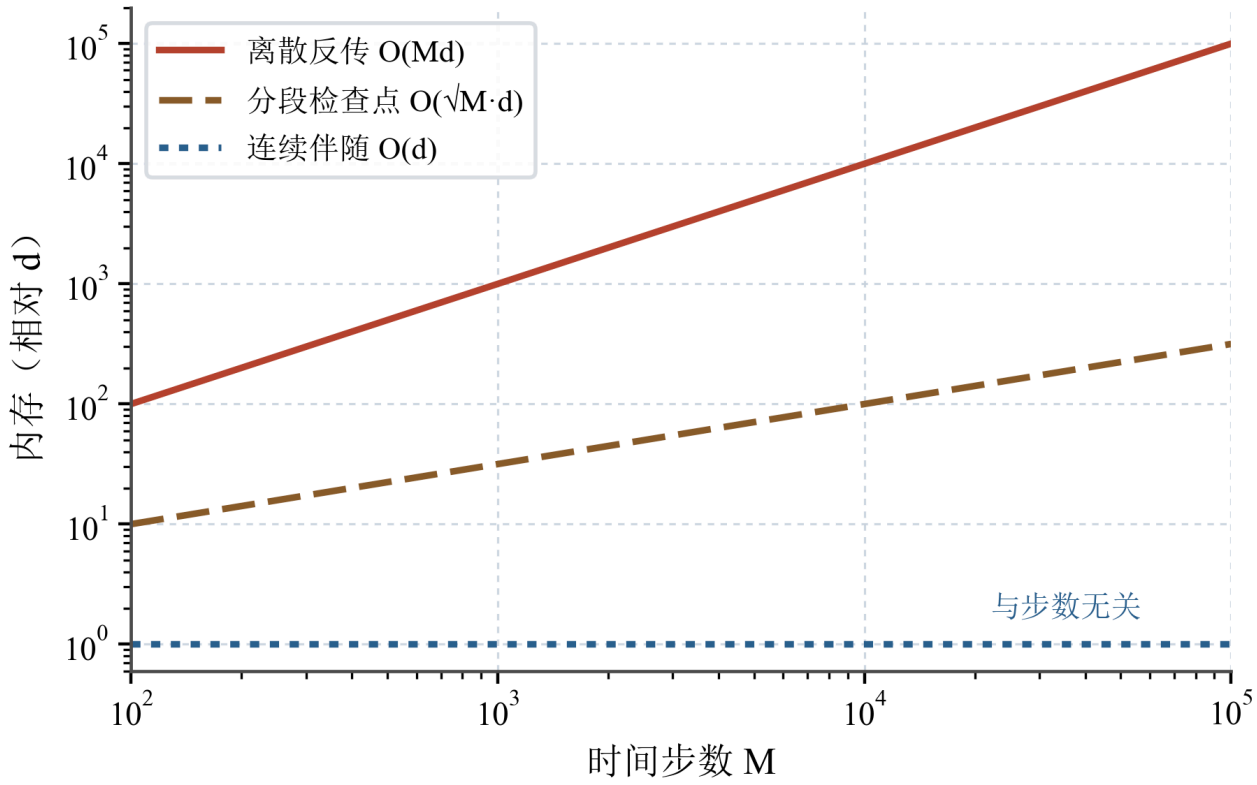


图 68　三种反向方式的内存开销随步数 M 的变化（取 d=1）：离散反传线性增长，分段检查点 √M 增长，连续伴随与步数无关。本组计算并绘制

> 科普 | 伴随法：从天气预报到火箭回收　“再解一个反向方程来求梯度”这件事，1950 年代就出现在最优控制里（Pontryagin 最大值原理中的协态方程）。1980 年代气象学家把它搬进资料同化，ECMWF 从 1997 年起用四维变分（4D-Var）做业务预报：想知道“初值场该改多少”，就把整个预报模式反向积分一遍，使误差回传到初值[64]。
>
> 如今这类反向计算十分常见：手机导航重新规划路线、无人机在气流中保持姿态、猎鹰 9 号回收时最后几十秒的制导，本质上都是“给定目标、反推当前应如何动作”的最优控制问题；而计算它的梯度，内存占用最小的办法就是伴随法。Neural ODE 的贡献不在于发明它，而在于把它纳入深度学习的显存预算：内存 $O(d)$、计算量约两倍。

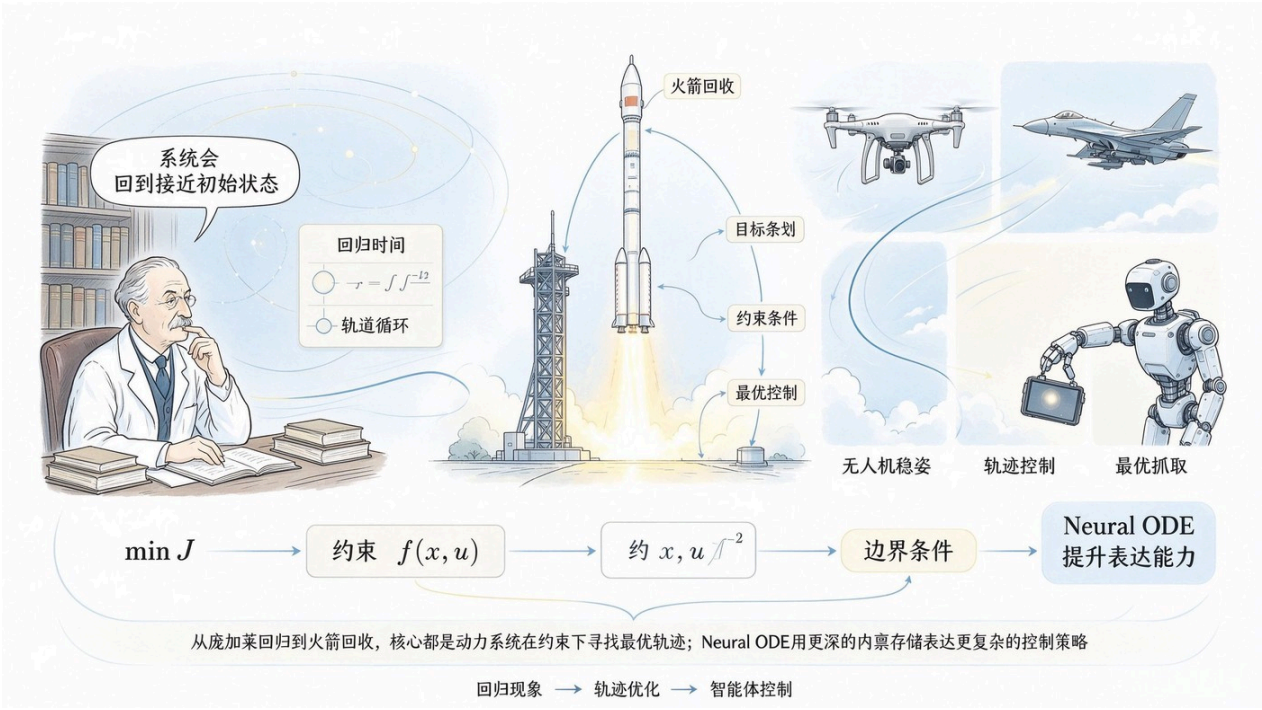


图 69　从庞加莱回归到火箭回收：核心都是动力系统在约束下寻找最优轨迹，梯度靠伴随法回传。图片来源：豆包生成

3.4.1.4　数值实验：一步反解出 θ

取一个最简单的例子来说明上述内容： $u' = -u$、 $u(0) = 1$，真解 e^{-t}。设只给出一个观测点 $(0.5,\ 0.6065)$，并限定网络只能取线性形式

$$f_\theta(t, u) = -\theta u.$$

这就是“结构对、参数未知”的最小模型。用前向 Euler 走一步（ $h = 0.5$）：

$$u_1 = 1 - 0.5\theta.$$

令 $u_1 = 0.6065$ 得 $\theta = 0.787$。若改用经典 RK4 走同一步，放大因子是 $1 - z + z^2/2 - z^3/6 + z^4/24$（ $z = 0.5\theta$），令其等于 0.6065，解得 $\theta = 1.0009$。同一个观测点、两种积分器，反解出的参数相差 27%。

表 25　同一步反解出的 θ 与它在 t=2 处的外推误差（观测点 (0.5,0.6065)，真值 e^{-2}=0.1353）

积分器	步长 h	反解 θ	训练点误差	t=2 处预测	相对真值偏差
前向 Euler	0.5	0.787	0.0	0.2072	+53%
前向 Euler	0.25	0.885	0.0	0.1704	+26%
经典 RK4	0.5	1.0009	0.0	0.1351	−0.2%
经典 RK4	0.25	1.0001	0.0	0.1353	0.0%

表 25 中最需注意的是“训练点误差”一列：四种组合都是 0，因为 θ 本来就是从该点反解得到的。可见训练误差为零并不能说明模型正确。真正的差别在最后一列：Euler 解出的 θ 系统性偏小，把衰减率学慢了，外推到 $t = 2$ 则多计算了 53%。这不是程序的错误，而是离散误差被吸收进了参数：Euler 对衰减

问题本身偏快（ $1-h<e^{-h}$ ），网络只能把 θ 调小来补偿。

这个把衰减率量小 21% 的系统偏差可以写成公式。Euler 走 $n=0.5/h$ 步落到观测点，要求 $(1-h\theta)^n=0.6065$，解出

$$\theta_{\text{Euler}}(h)=\frac{1-0.6065^{\,h/0.5}}{h}.$$

把 $0.6065\approx e^{-0.5}$ 代进去，$\theta_{\text{Euler}}(h)\approx(1-e^{-h})/h=1-\frac{h}{2}+\frac{h^2}{6}-\cdots$。$h=0.5$ 时给出 $1-0.25+0.042=0.792$，与数值 0.787 的差异仅来自舍入；首项 $-h/2$ 表明“步长减半、偏差减半”，这正是图 71 中那条近似直线的来源。RK4 的同类展开首项为 $O(h^4)$，因此它在 $h=0.5$ 上已看不出偏差。

可以用一个比喻：Euler 相当于一把刻度偏短的尺子。用它测量同一个身高，读数会系统性偏小；若不知道尺子偏短，就会得出“身高只有 1 米 68”的错误结论。表 25 中的 0.787 正是这样一个读数——它在训练点上完全对齐（误差 0.0），却把衰减率量小了 21%，换成刻度准确的 RK4 立刻回到 1.0009。因此“训练误差为零”从不是合格的凭证，它只说明尺子与被测对象已经相互校准。

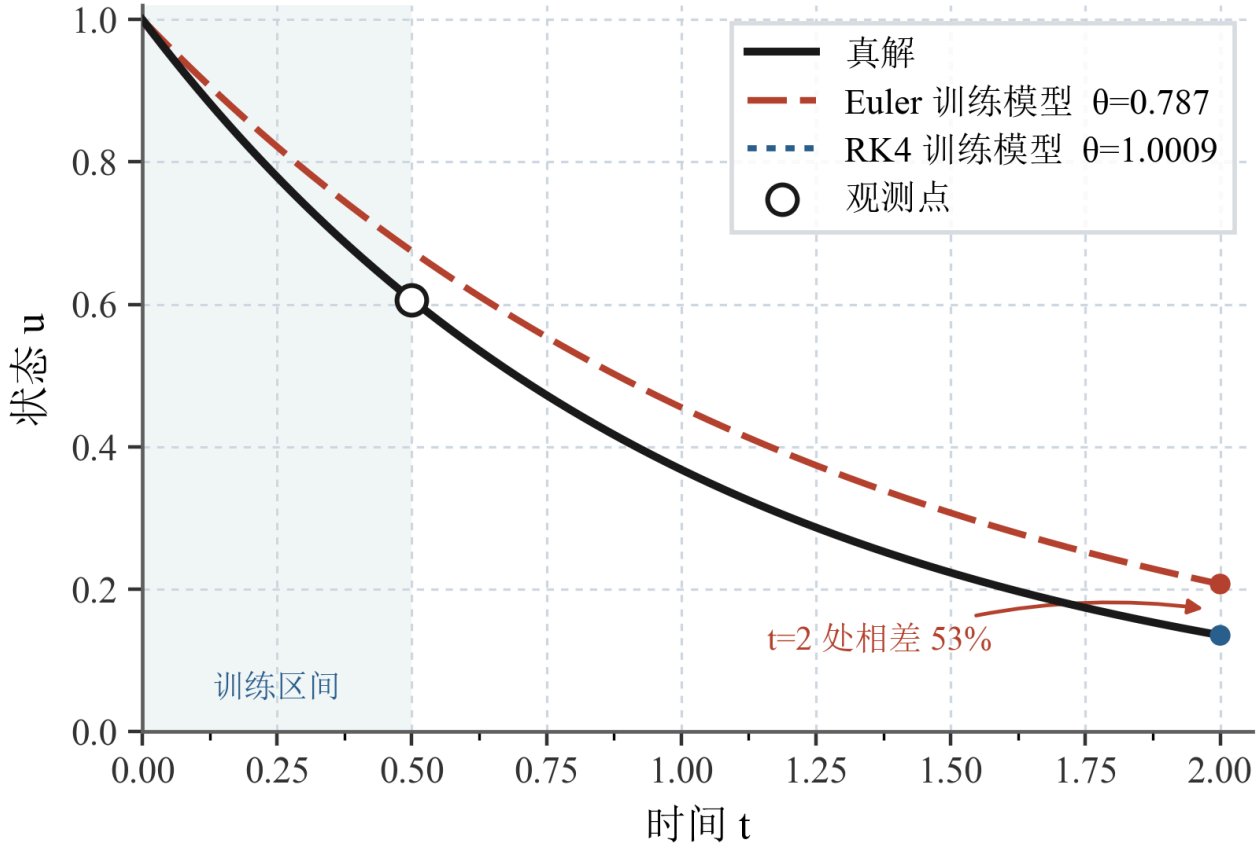


图 70　同一个观测点、两种积分器：Euler 训练出的衰减率 θ=0.787 明显偏小，外推到 t=2 差了 53%；RK4 的 θ=1.0009 几乎贴合真解。本组计算并绘制

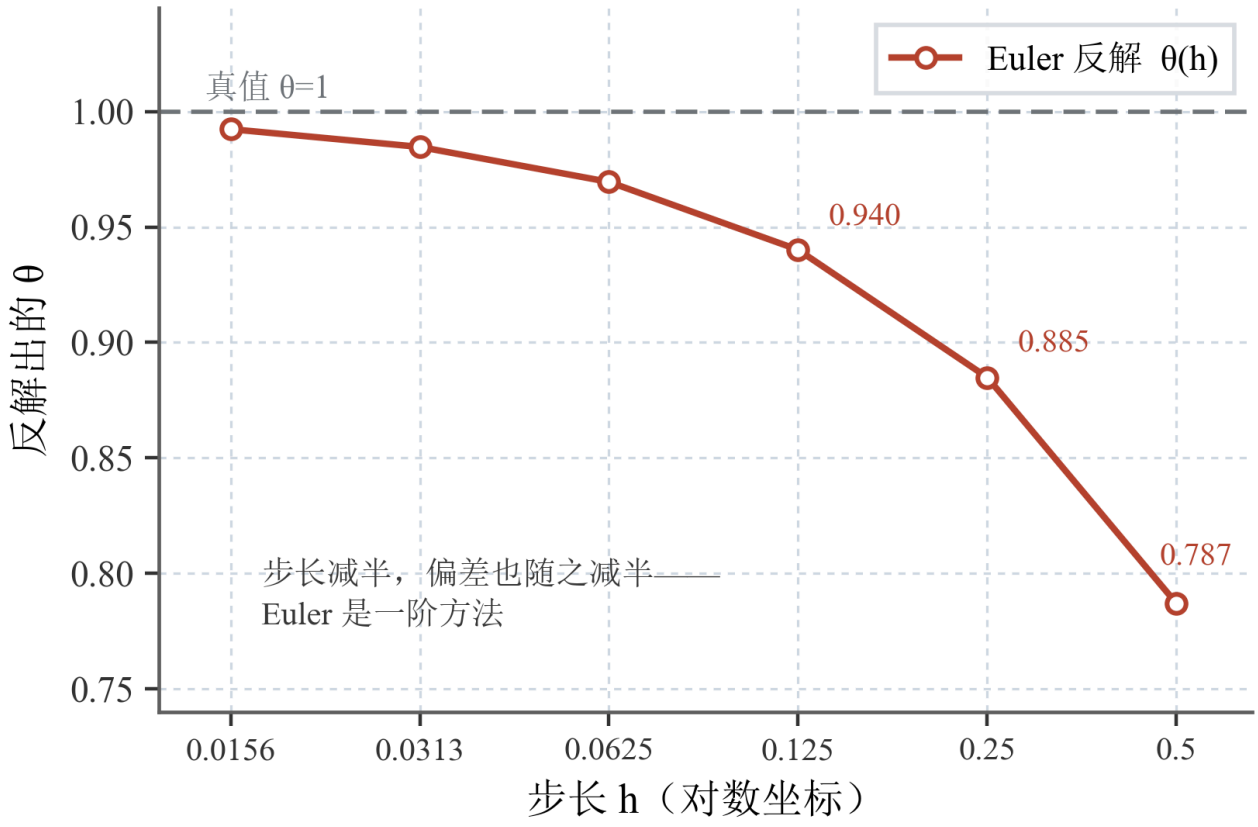


图 71　反解出的 θ 随步长减小而趋向真值 1：离散误差消失，参数才回到真值。本组计算并绘制

这段实验可以重做：取步长 h 依次为 0.5、0.25、0.125，重算 Euler 的 θ 与外推偏差。可以看到 θ 单调趋向 1、偏差单调趋向 0——离散误差随 $h \to 0$ 消失，这正是收敛阶在“反问题”里的体现。

3.4.2　轨迹拟合与方程残差

18 世纪的航海者靠“潮汐表”：把每个港口涨落的时间与高度记下来，编成表，直接查用。后来拉普拉斯用月球与太阳的引力写出方程，直接算出潮汐——表上没有的港口，方程也能给。

这两种“学”法，正是本节要比较的两条路线：一条把观测当答案背下来（拟合轨迹），一条把规律当答案学下来（拟合方程）。哪一条更省数据、更容易训练、各自的困难在哪里，是 3.4.2.1 到 3.4.2.3 要回答的问题。

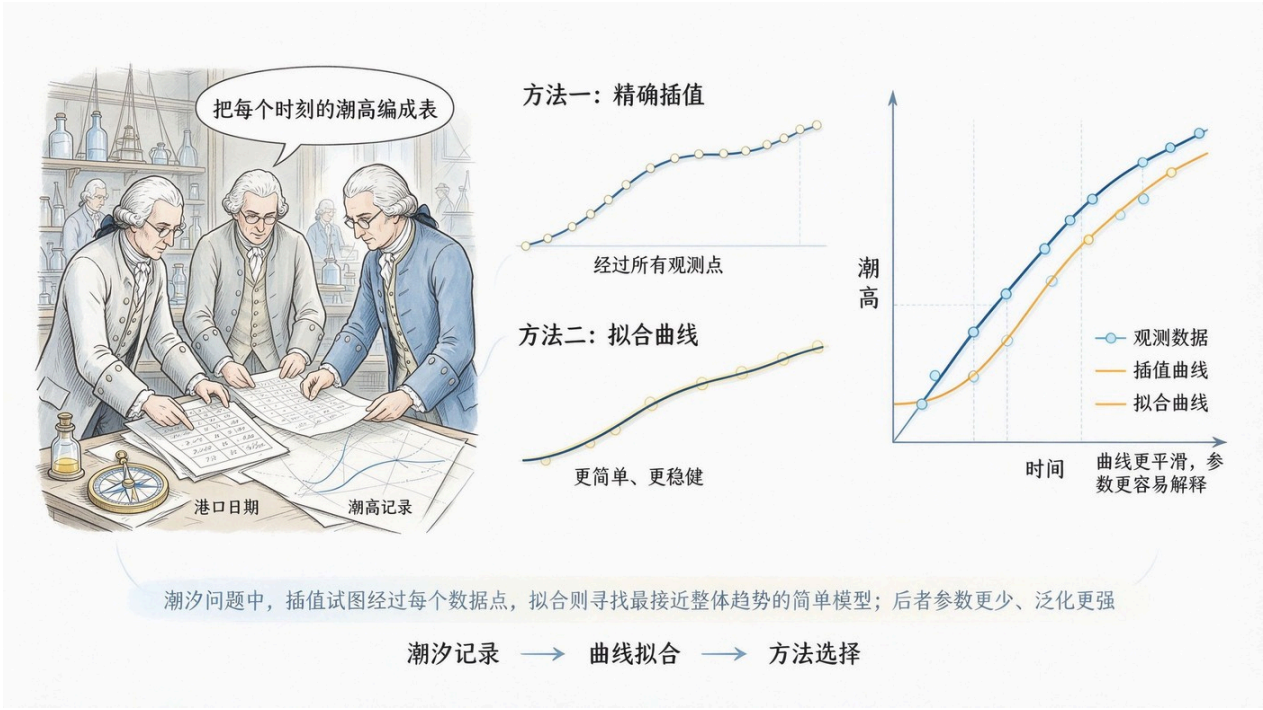


图 72　潮汐表的两种做法：精确插值要求曲线穿过每一个观测点，拟合只反映整体趋势。图片来源：豆包生成

3.4.2.1　残差算子与两种损失

引子里那两条路线的差别，可以用一个算子写清楚。

定义 2（方程残差）　对任意函数 u_θ，定义

$$r_\theta(t) = \frac{\partial u_\theta}{\partial t}(t) - f\big(t, u_\theta(t)\big).$$

称 r_θ 为该函数在方程 (45) 下的残差；$r_\theta \equiv 0$ 当且仅当 u_θ 是方程的解。

有了残差，PINN 的损失可以写成三部分：

$$\mathcal{L}_{\text{pinn}}(\theta) = \underbrace{\frac{1}{N_r}\sum_{j=1}^{N_r} \big|r_\theta(t_j)\big|^2}_{\text{方程残差}} + \lambda_1 \underbrace{\frac{1}{N_b}\sum_{k} \big|u_\theta(t_k) - g(t_k)\big|^2}_{\text{初边值}} + \lambda_2 \underbrace{\frac{1}{N_d}\sum_{i} \big|u_\theta(t_i) - u_i\big|^2}_{\text{数据}}, \tag{50}$$

其中配点 t_j 可以在区间内任意取（通常随机采样），g 是边界数据，λ_1, λ_2 是权重。与式 (47) 相比，式 (50) 里的 f 是真方程而不是网络——PINN 学的是解 u，Neural ODE 学的是右端 f。这是两条路线最根本的分野。

两者的适用条件也因此不同：有方程、数据稀少（甚至没有）时用式 (50)；有数据、方程未知时用式 (47)；两者都有，则三项同时启用，用 λ 调整比例。

从机器学习的角度看，这是一次典型的“先验换数据”。纯数据驱动往往需要成千上万个样本才能可靠外推；把方程当作先验，几百个点就能约束住解的形状。大模型时代讲 scaling law，而科学场景里样本永远是稀缺的——这正是“物理 + 少量数据”的组合在工程上长期有效的部分原因。

用监督学习的语言重述会更清楚：式 (50) 的第三项是有监督的（有标签 u_i），第一项是自监督的（没有标签，由方程自身提供监督信号），第二项是结构先验（边界条件是已知的硬性信息）。深度学习近年来最有用的几个想法，几乎都围绕“如何利用数据之外的结构构造监督信号”：对比学习用同一张图的两个视图构造标签，语言模型用下一个词构造标签，PINN 用方程残差构造标签。区别只在于，其他领域依靠数据增强构造监督，这里依靠物理定律构造监督。

两种目标对应两类实际情形。实验室里只在几个位置埋了热电偶，热传导方程却是确定的——这时式 (50) 更合适：一边用稀疏的实测点把场校正回来，一边要求场处处满足方程；工程上常说的“数字孪生”，骨架就是这种“方程 + 少量实测”的组合。反过来，如果只有一条形状近似 S 的曲线、连方程是什么都不知道，那就只能采用式 (47)，由网络自行学出 f。目标选择错误，后续调参都是徒劳。

式 (50) 的第一项是“强形式”残差：要求 u_θ 具有所需的连续导数（对一阶方程即 C^1），且方程逐点成立。也可以只要求“对任意测试函数 φ 都有 $\int r_\theta \varphi \,\mathrm{d}t = 0$”（弱形式），代价是积分计算量更大，优点是对光滑性要求低，且天然对接有限元式的离散。Deep Ritz 走得更远：直接最小化能量泛函，连残差都不用算，代价是必须先有变分结构——不是每个方程都有[70]。

还有一处常被忽略的细节：量纲。方程中系数相差几个数量级时（热传导系数 10^{-6} 与对流项 $O(1)$ 并存），残差项与数据项的数值大小完全不可比，无论怎样调整 λ，都得不到“两项同时下降”。先把方程无量纲化到 $O(1)$，是使 PINN 收敛代价最小的一步。

3.4.2.2 残差为零意味着什么

“残差小”究竟意味着什么？下面的命题给出确切回答。

命题 3　设 $u_\theta \in C^1$、f 连续且关于 u 满足 Lipschitz 条件。若 $r_\theta \equiv 0$ 于 $[0,T]$ 且 $u_\theta(0) = u_0$，则 u_θ 就是初值问题 (45) 的唯一解。

证明　$r_\theta \equiv 0$ 即 $u_\theta' = f(t, u_\theta)$，且初值相同，故 u_θ 满足 (45)；由 Picard–Lindelöf 定理，解唯一，故 u_θ 就是那个解。证毕。

命题 3 是定性的，下面的推论把它变为定量估计，也是本节最应记住的一个结果。

推论 1（残差到误差）　设 f 关于 u 满足 Lipschitz 条件（常数 L），$u^\star$ 是初值问题 (45) 的解，$u_\theta \in C^1$ 与 $u^\star$ 同初值，r_θ 为其残差。则对 $t \ge 0$，

$$\left| u_\theta(t) - u^\star(t) \right| \le \int_0^t \left| r_\theta(s) \right| e^{L(t-s)} \,\mathrm{d}s .$$

证明　记 $e = u_\theta - u^\star$。由 $r_\theta = u_\theta' - f(t, u_\theta)$ 与 $u^{\star\prime} = f(t, u^\star)$ 得 $\dot e = r_\theta + f(t,u_\theta) - f(t,u^\star)$，取绝对值并用 Lipschitz 条件，$|\dot e| \le |r_\theta| + L|e|$。两边乘 e^{-Lt}，得 $\frac{\mathrm{d}}{\mathrm{d}t}\left(e^{-Lt}|e|\right) \le e^{-Lt}|r_\theta|$；注意 $e(0) = 0$，从 0 到 t 积分即得。证毕。

这条不等式给出了明确结论：残差必须处处小，而且越靠前的残差被放大得越严重。若 $|r_\theta| \equiv \varepsilon$，误差不超过 $\varepsilon(e^{Lt} - 1)/L$：取 $L = 1$，$t = 1$ 的放大因子约 1.7，$t = 2$ 约 6.4，$t = 3$ 约 19；L 增大时，该因子随指数迅速变大。所以“残差 1%”与“解差 1%”之间没有等号，中间隔着一个由 L 与 t 决定的放大系数。

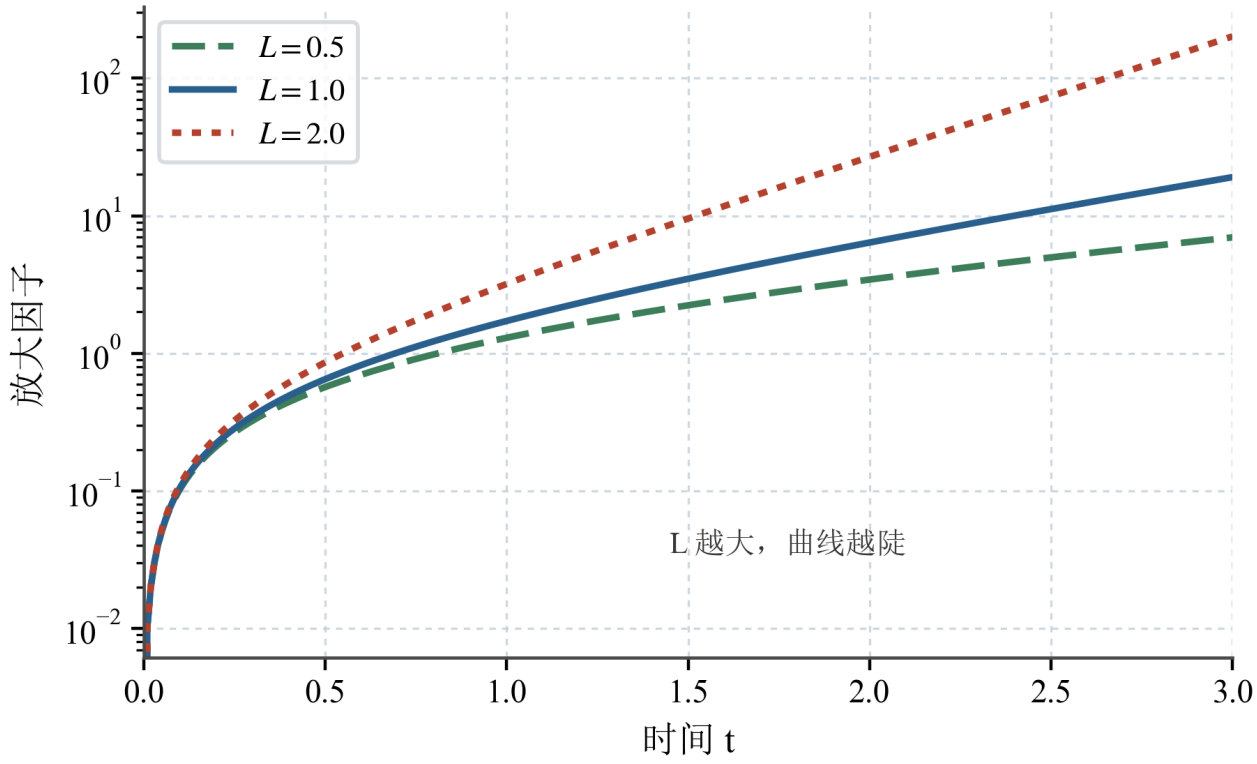


图 73　推论 1 里的放大因子 $(e^{Lt}-1)/L$：同一条残差曲线，L 越大放大越剧烈。本组计算并绘制

命题 3 看似平凡，却解释了 PINN 为什么在小数据下有效：只要残差处处为零，网络就“自动”是解，不需要任何观测。但它同时给出一个警告：命题 3 的前提是处处为零。式 (50) 的第一项只在有限个配点上算残差，配点之外可以任意差。这就是“残差小”与“解对”之间的鸿沟。

换言之，命题 3 要求“处处成立”，而式 (50) 只在有限个配点上检验。如同只在练习过的三道题上取得满分，并不能说明已经掌握这一类问题。

小实验 1（纸笔，10 分钟）｜残差为零的假象。　取 $u_\theta(t)=1+t$、方程 $u'=u$。算出 $r_\theta(t)=1-(1+t)=-t$。若只取一个配点 $t=0$，则 $r_\theta(0)=0$，残差损失为零；但在 $t=1$ 处 $r_\theta=-1$，解的真值是 $e\approx 2.718$，而网络只给 2。请把“配点数为 1、2、3”三种情形的残差损失与 $t=1$ 处误差列成一张小表，体会配点密度与可信度的关系。

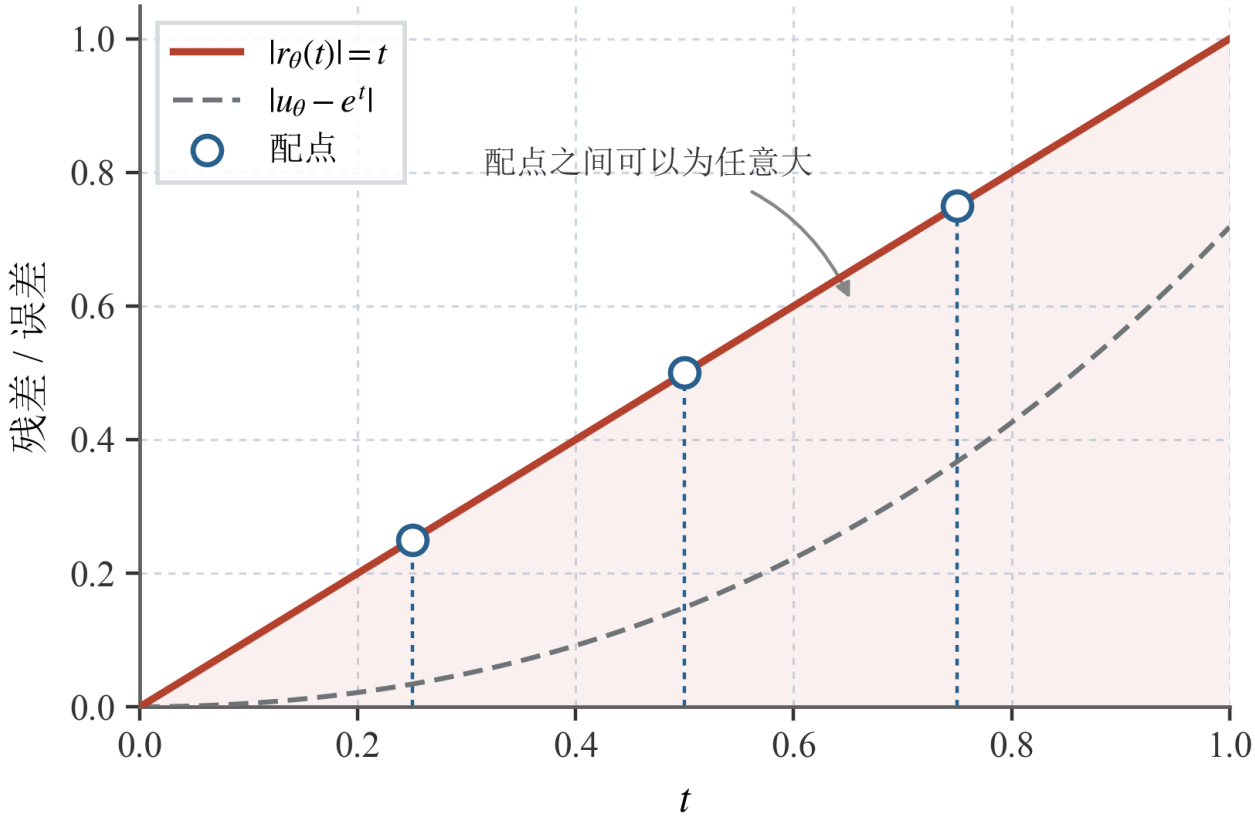
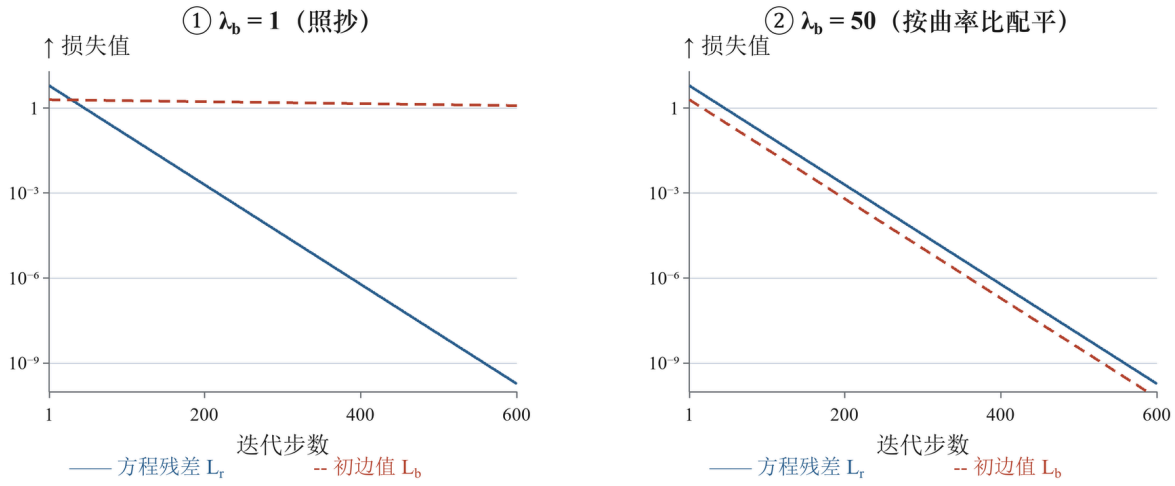


图 74　残差 $|r_\theta(t)|=t$ 只在配点（空心圆）处被“看见”，配点之间可以任意大；灰虚线是同一个模型与真解的差，它与残差并不相等。本组绘制

3.4.2.3　权重的代价：PINN 为什么难训

式 (50) 里的 $\boldsymbol{\lambda_1, \lambda_2}$ 并非无关紧要的超参数。Wang、Yu 与 Perdikaris 用神经正切核（NTK）分析了这件事[59]：训练时每一项损失下降的“速度”由该项对应的核特征值决定，若残差项与边界项的核尺度相差几个数量级，优化器就会偏向其中一项——表现是残差下降很快、边界条件却始终不满足，或者相反。这就是文献中“PINN 难以训练”的常见原因[78]。

这件事可以用一个极小的模型算出来。取一个只有两个参数的模型，让残差项的曲率比边界项大 50 倍（真实 PINN 中这个比值由无量纲化与网络的二阶导决定，通常还要大得多），用同一个学习率训练 600 步：权重取 1:1 时，残差项降到 $\boldsymbol{10^{-10}}$，边界项几乎没有变化；把权重按曲率比调到 50:1，两项才同时下降（图 75）。



同一个学习率下，收敛率 = η × 曲率：残差方向 0.02，边界方向 0.0004，差 50 倍

图 75　权重的代价：同一个学习率下，曲率大的残差项很快收敛，曲率小的边界项几乎不变；把权重调到曲率比，两项才同步下降。本组计算并绘制

原因并不复杂：梯度下降每步前进 $\eta\|\nabla L\|$，而可行步长由曲率决定。曲率大的一项几步即到底，余下几千步都用于曲率小的一项；但步长又受曲率大的一项限制（再大就发散，见 3.4.3.4），因此后者只能缓慢下降。所谓“权重没有调好”，本质是无法为两项分别指定步长，只能指定一个权重。

> 科普 | “偏科”不只发生在教室里　训练一个同时满足多项损失的网络，很像一个学生同时修两门课，而期末总评把两门按 99 比 1 加权——学生自然会把时间集中在第一门课上。NTK 给出的正是这个“学分”的数学版本：每一项损失下降的快慢由它对应的核特征值决定；两边尺度差上几个数量级，梯度就几乎全流向“容易的那门课”。 现实中这类“追逐指标”的现象十分常见：推荐系统若把点击率的权重压得远高于完播率，模型就会用夸张标题吸引点击；考核若只看销量、不看退货率，就会出现“销量越大、口碑越差”的局面。PINN 中的 λ_1,λ_2 就是这两门课的“学分权重”，权重失衡，网络自然偏重其中一项。

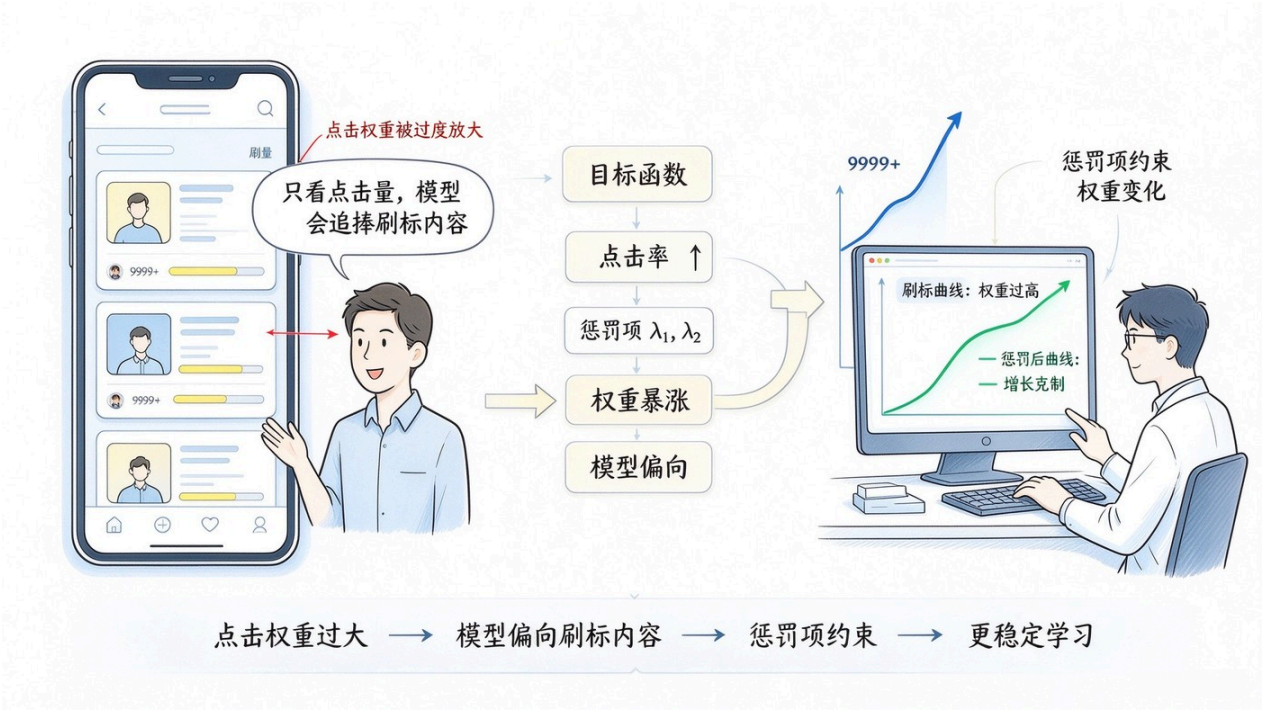
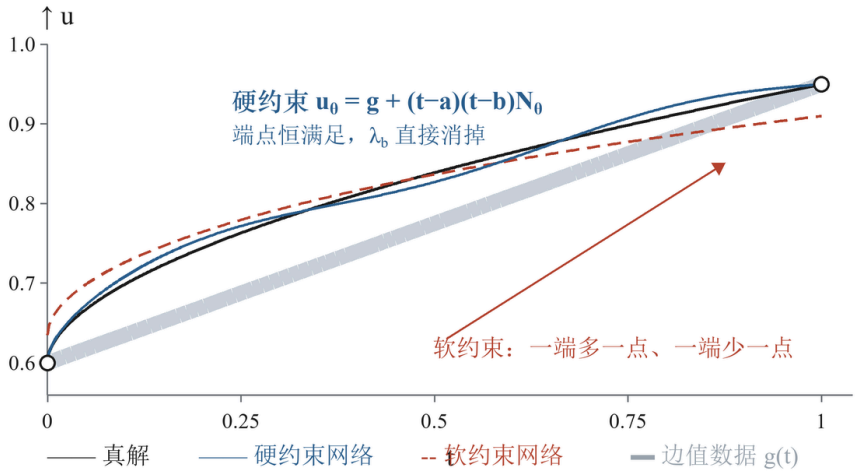


图 76　只看点击量，模型就会追捧刷标内容；把惩罚项的权重配好，才能把它拉回来。图片来源：豆包生成

实践中的对策有三条：其一，把 λ 当作可训练参数，让它随训练自适应（自适应权重）；其二，改用“硬约束”构造网络，例如令 $u_\theta(t) = g(t) + (t-a)(t-b)\,N_\theta(t)$，使边界条件恒满足，把 λ_1 彻底消掉；其三，分阶段训练，先拟合边界、再引入残差项。三条都不保证收敛，但都能明显降低调参的随机性。

第二条需要单独说明：它把“调权重”换成了“改结构”。图 77 给出了硬约束的构造：把边界数据 $g(t)$ 直接写进网络的表达式，端点上的值恒等于 g，λ_1 这一项从此不存在。代价是网络只能在这个流形上搜索，表达能力略减；优点在于训练目标少了一项，需要调整的超参数由三个减少为两个。



灰色粗带是给定的边值数据 g(t)（这里取两端点的连线）。软约束靠罚项逼近它，端点总差一点；硬约束把 g 直接写进结构，端点误差恒为零。

图 77　软约束与硬约束：罚项只能让端点“尽量靠近” g，而把 g 写进结构后端点误差恒为零。本组绘制

这类“把约束写进结构”的做法，今天有一个统一的名字：结构化先验。卷积网络把平移不变性写进权重共享，等变网络把旋转对称性写进层的设计，Hamiltonian 网络把能量守恒写进向量场的形式（3.4.5.2 还会碰到）。它们与硬约束是同一思路在不同尺度上的体现：能够由结构保证的量，就不必交给学习。

自适应权重并非经验之谈，常见写法有两种。其一是梯度范数平衡：每步算 $\|\nabla_\theta L_r\|$ 与 $\|\nabla_\theta L_b\|$，令 $\lambda \leftarrow \lambda \cdot (\|\nabla L_r\|/\|\nabla L_b\|)^{\alpha}$（ $\alpha \approx 0.9$），把两边的梯度拉回同一量级[69]。其二是不确定性加权：把两项写成 $L_i/\sigma_i^2 + \log \sigma_i^2$，让 σ_i 作为可训练参数——由模型自行学出“哪一项更可信”，等价于用最大似然为每个任务分配权重[68]。共同点是不再手工调整 λ，而让它随梯度的实际尺度变化。

同样的现象在多任务学习与大模型微调中很常见：几个损失的量级相差两个数量级时，梯度会被较大的那一项主导，模型表面上在训练，实际上只学会了一件事。工程上的解法与 3.4.2.3 相同——归一化、调整权重，或分阶段训练。

3.4.2.4　从 PINN 到数字孪生

式 (50) 这条“把残差当损失”的路线，今天主要落在工程侧。前面三节讨论的是它的数学性质，这一节看它在实际系统中的形态。最直接的是数字孪生：为一个真实设备（芯片、电池、血管、油气藏）配一个可实时计算的模型，传感器数据输入后即更新参数。这类场景的特点是——方程已知，但参数不准、边界条件不全，正是 PINN 所擅长的情形。

还有一条介于纯物理模型与纯数据驱动之间的路：通用微分方程（UDE）。方程的主体保留已知的物理项，只把说不清的那一部分换成网络： $\dot{u} = f_{\text{已知}}(u) + f_\theta(u)$[83]。这样既有外推能力（物理项负责远处），又有拟合能力（网络负责近处），训练时同样用伴随法求梯度。工程上这是最容易被接受的形式——并不推翻原有模型，只是为它增加一个“可学习的修正项”。

再往上一层是算子学习：不学某一个解，而学“给定参数即给出解”的映射。DeepONet 用分支网与主干网分别编码参数与坐标，傅里叶神经算子（FNO）在频域上学一个核[72][57]。它们一次训练可以服务成千上万个参数组合，代价是必须先有一批高质量的数值解作为训练数据。用机器学习的术语说，这是摊销（amortization）：把在线求解的昂贵计算转移到离线训练中一次完成。

最近两年最受关注的是 PDE 基础模型：把方程、边界与网格当作 token，用一个骨干网络覆盖多类方程。PDEformer-2、DPOT、M4 都采用这一思路，走的是语言模型“先统一表示、再讨论规模”的路径。它距离“通用”仍很远，但方向已经清楚——这也解释了这一领域近两年为何开始讨论 scaling law。

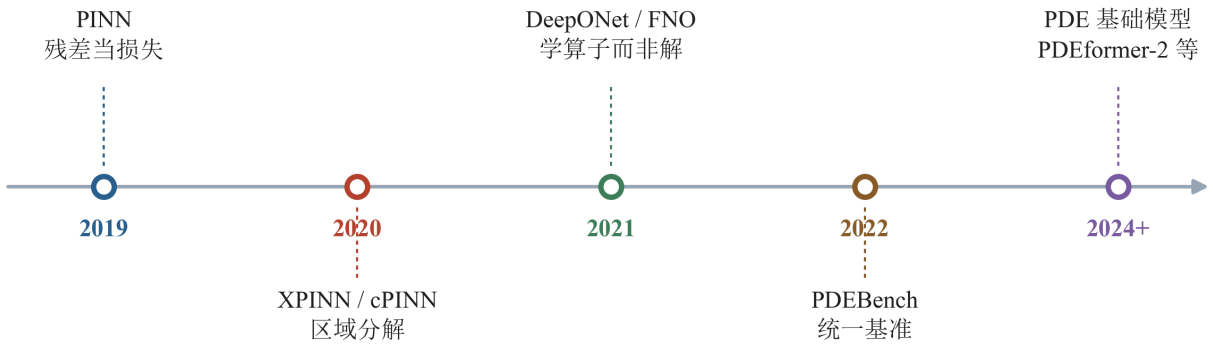


图 78　从 PINN 到 PDE 基础模型的几个节点（2019—2024）：损失怎么定、算子怎么学、基准怎么建。本组绘制

3.4.3 求解器误差与独立轨迹验证

1922 年，英国气象学家 Richardson 用手摇计算器算了六个星期，得到一份六小时的天气预报：结果地面气压的变化大得不符合物理，同行认为这份结果不可信。事后分析指出两个原因：初值观测太稀、噪声太大；时间步长相对网格取得太大，误差被迅速放大。

这两个原因，加上训练本身残留的优化误差，正好对应本节要分别估计的三类误差：模型本身的偏差、积分的离散误差、优化未收敛的残余。Richardson 那份“失败”的预报也说明：一份结果是否正确，必须换一套装置重算才能确认——这就是“三换”的由来。

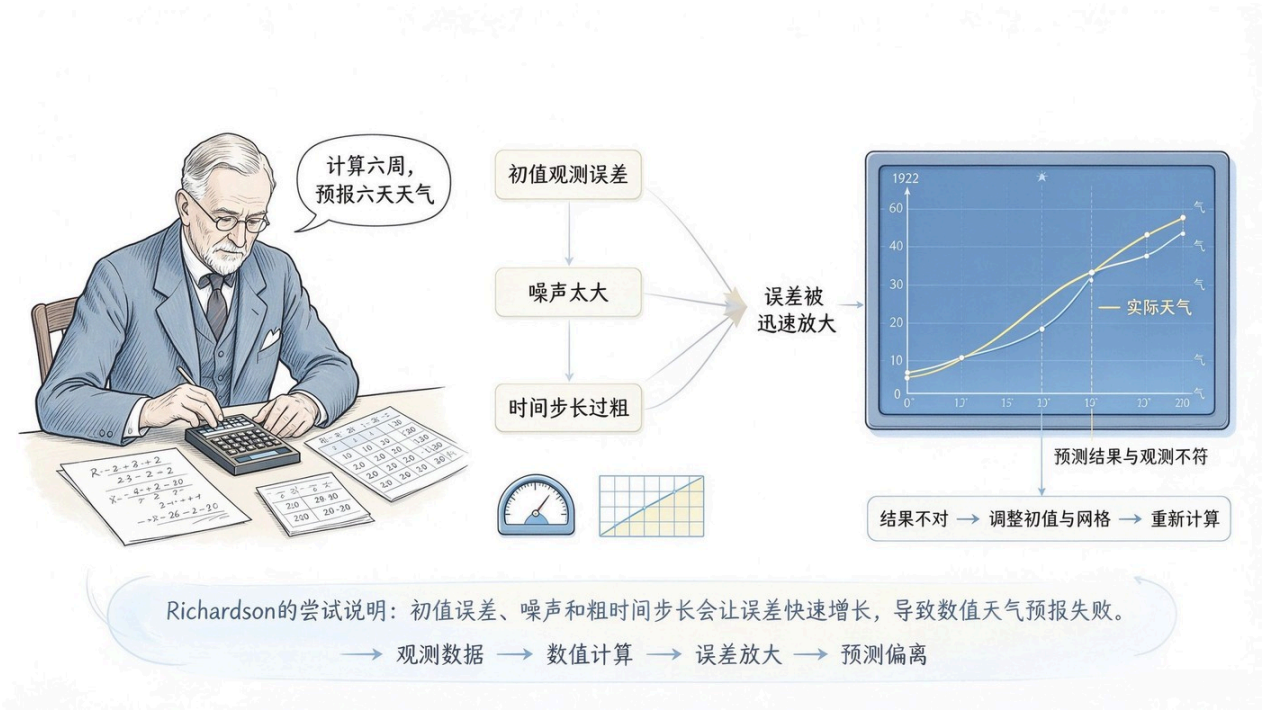


图 79　Richardson 1922 年的手算预报：初值误差、观测噪声与过粗的时间步长共同放大。图片来源：豆包生成

3.4.3.1 误差的三个来源

训练结束后通常只关注两个数值：训练损失与测试损失。但使用 Neural ODE 时还有第三项来源——积分器的离散误差。三者混在一起，就会得到“误差 10^{-4}”这类没有信息量的结论。

数值天气预报领域早在几十年前就对这两类误差作了明确区分：一边是初值误差（今天的探空数据不够准，明天的预报就不准），一边是模式误差（方程离散化、物理参数化带来的系统性偏差）；前者用集合预报处理，后者通过改进模式处理。若混在一起只报一个“预报误差 3℃”，就无法判断应当改进哪一方面。

命题 4（误差分解）　设 $\hat{u}$ 是用积分器 $\mathcal{I}$、步长 h 对 $f_{\theta^\star}$ 积分得到的数值轨迹，$u^\star$ 是真实系统的解。则

$$\|\hat{u}-u^\star\| \le \underbrace{\|u_{\theta^\star}-u^\star\|}_{\text{逼近误差}} + \underbrace{\|\hat{u}-u_{\theta^\star}\|}_{\text{离散误差}} + \underbrace{\|u_{\theta^\star}-u_\theta\|}_{\text{优化误差}}, \tag{51}$$

其中 u_θ 是训练得到的参数对应的精确轨迹，$u_{\theta^\star}$ 是“最优参数”对应的精确轨迹。

证明　三角不等式：$\|\hat{u}-u^\star\| \le \|\hat{u}-u_\theta\| + \|u_\theta-u_{\theta^\star}\| + \|u_{\theta^\star}-u^\star\|$，再把第一项拆成离散误差与优化误差。证毕。

第二项可以进一步具体化。对前向 Euler，局部截断误差是 $\frac{h^2}{2}|u''|$ 的量级，累积成全局误差

$$\|\hat{u}(T)-u_{\theta^*}(T)\| \le \frac{h}{2}\max_{[0,T]}|u''|\cdot\frac{e^{LT}-1}{L},$$

和推论 1 是同一个 Gronwall 引理，只是把“残差”换成了“每步的截断误差”[5]。这条式子解释了本节反复出现的两件事：误差随 h 一阶下降（表 25 的偏差一栏：步长减半，偏差减半）；L 大、T 长时步长必须取得极小——刚性方程计算代价高的根本原因，就在这个指数因子上。

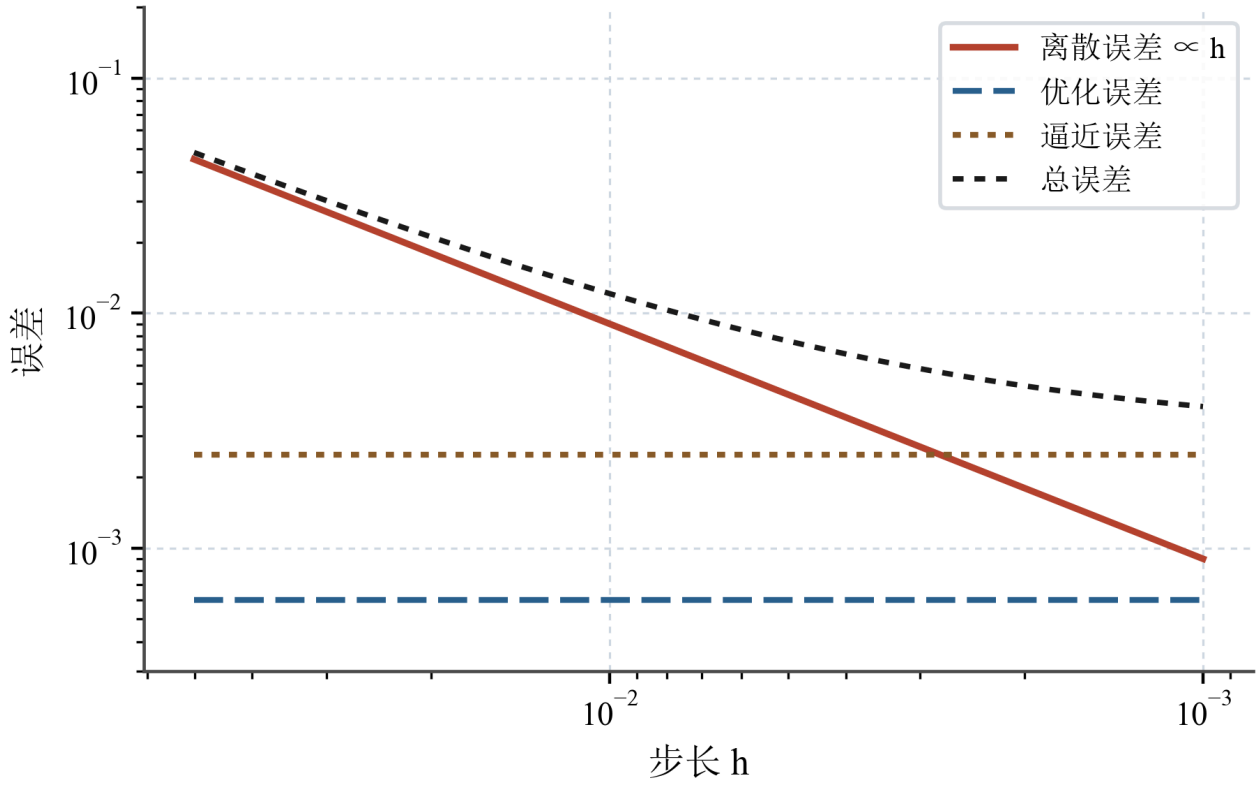


图 80　误差分解的实际表现：步长 h 减小，离散误差线性下降，但总误差最终被优化误差与逼近误差托住，不再改善。本组计算并绘制

式 (51) 的实用含义是：换一个积分器重算，就能把离散误差单独量出来。若两次结果差别明显，说明离散误差主导，此时讨论“模型是否学到”没有意义。

3.4.3.2　三换：独立轨迹验证协议

把 3.1 的建议做成可执行的检查清单，一共三换：

（i）换初值。训练只用 $u(0)=1$ 的轨迹，验证时取 $u(0)=0.5$、$u(0)=2$ 各积一条。若学到的是向量场，三条都应贴合；若只是记住了那一条曲线，换初值后会立刻失效。

用 3.4.1.1 的比喻说，就是把船放到另一处水面，检验那股“水流”是否依然成立。

（ii）换步长。用训练步长的 $1/2$ 与 2 倍各积一次。真解不随步长变，好模型也不该变；若轨迹随步长明显偏移，说明模型把离散误差吸收进了参数（表 25 已见）。

（iii）换区间。内插段验证通过后，再向两端各外推 20%。外推段的误差通常大一个量级，这是 Neural ODE 的固有局限，报告里应如实写出。

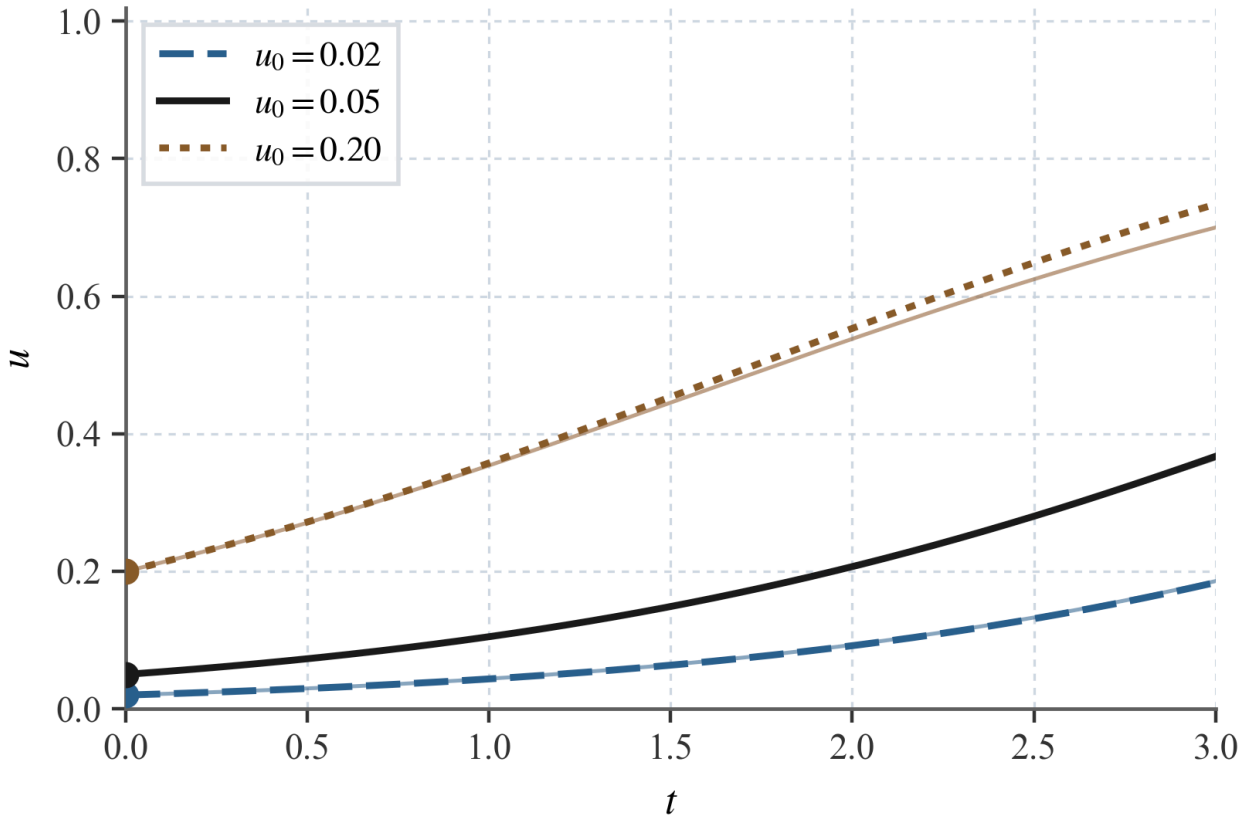


图 81　换初值验证：把学到的向量场放到三条不同初值上积分（细线），与真解（粗线）基本重合——学到的是向量场，而不是某一条曲线。本组计算并绘制

这套协议与用观测收敛阶检验格式的精神一致：不要用训练时用过的数据证明自己。

机器学习界对此有一个更直白的说法：公开榜过拟合。竞赛平台把测试集分为公开榜与私有榜，正是为了避免参赛者反复依据公开榜调参，使它逐渐变成训练集。三换协议是同一道理的小规模版本——凡是调参时看过的数据，都不再构成独立验证。

3.4.3.3　数值实验：换步长之后偏了多少

仍用 3.4.1.4 节的线性玩具模型，但这次把 θ 固定为用 Euler（$h = 0.5$）训练出的 0.787，然后更换积分器与步长重新积分到 $t = 2$，考察预测值的偏移。

表 26　固定 θ=0.787，换一套“装置”重算 t=2 的值（真值 0.1353）

复算方式	步长	t=2 处预测	相对真值偏差
解析解 $e^{-\theta t}$	—	0.2072	+53%
前向 Euler	0.5（训练时）	0.1353	0.0%
前向 Euler	0.25	0.1733	+28%
前向 Euler	0.125	0.1907	+41%
经典 RK4	0.5	0.2073	+53%
经典 RK4	0.25	0.2072	+53%

表 26 把同一个 $\theta = 0.787$ 交给不同的方法去算 $t = 2$：用训练时的 Euler（$h = 0.5$）重算，预测恰好落回真值 0.1353；步长减半，偏移到 0.1733；再减半，偏移到 0.1907，逐渐接近“拟合模型自身的真值”0.2072。换成 RK4 则不论步长如何，一律给出 0.2072，与解析解一致。

这正是“离散误差被吸收进参数”的直接证据：θ 是按 Euler（$h = 0.5$）和那一个观测点反解出来的，因此只有这一套装置能让误差相互抵消，表面上完美；换一套装置，抵消立即失效，53% 的偏差随即显现。表 25 与表 26 是同一诊断的两个方面：表 25 说明参数被污染了多少，表 26 说明换装置之后污染暴露得多彻底——两者的共同点是，训练点上的误差始终为 0，而 0 不能证明任何事情。

工程上把这一现象概括为：离线指标良好，部署后性能骤降。模型把训练环境中的误差固化进了参数，换一台设备或换一个季节，结果便无法对应。三换协议之所以强调“换”，正是为了在部署之前暴露这一问题。

3.4.3.4 学习率也是步长：优化器就是积分器

本节还有最后一处与 AI 有关的对应关系：将梯度下降写成一式，$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}(\theta_k)$。与之对照的是显式 Euler：$u_{k+1} = u_k + h\, f(t_k, u_k)$。把 u 换成 θ、把 f 换成 $-\nabla\mathcal{L}$、把 h 换成 η，两式形式完全相同。于是有四条对应关系：学习率就是步长；损失面就是向量场（只是它的方向指向极小点）；训练轮数就是积分步数；而损失函数在参数空间中的曲率，就是这一步所面对的刚性。

这并非修辞，它能直接给出可用的结论。把损失在极小点附近展开成二次型 $\mathcal{L} \approx \frac{1}{2}\mu|\theta - \theta^\star|^2$，梯度下降就是 $\theta_{k+1} - \theta^\star = (1 - \eta\mu)(\theta_k - \theta^\star)$。要收敛必须 $|1 - \eta\mu| < 1$，也就是

$$0 < \eta < \frac{2}{\mu_{\max}}.$$

这与 Euler 法的稳定域是同一个不等式。图 82 把它画在复平面上：Euler 的稳定域是圆心在 -1、半径为 1 的圆盘，学习率落在域内就收敛，落在域外则发散；右侧是四条真实的训练曲线，其中 $\eta = 1.0$（$\mu = 1$ 时一步收敛）、$\eta = 1.9$（振荡下降）、$\eta = 2.2$（发散）。

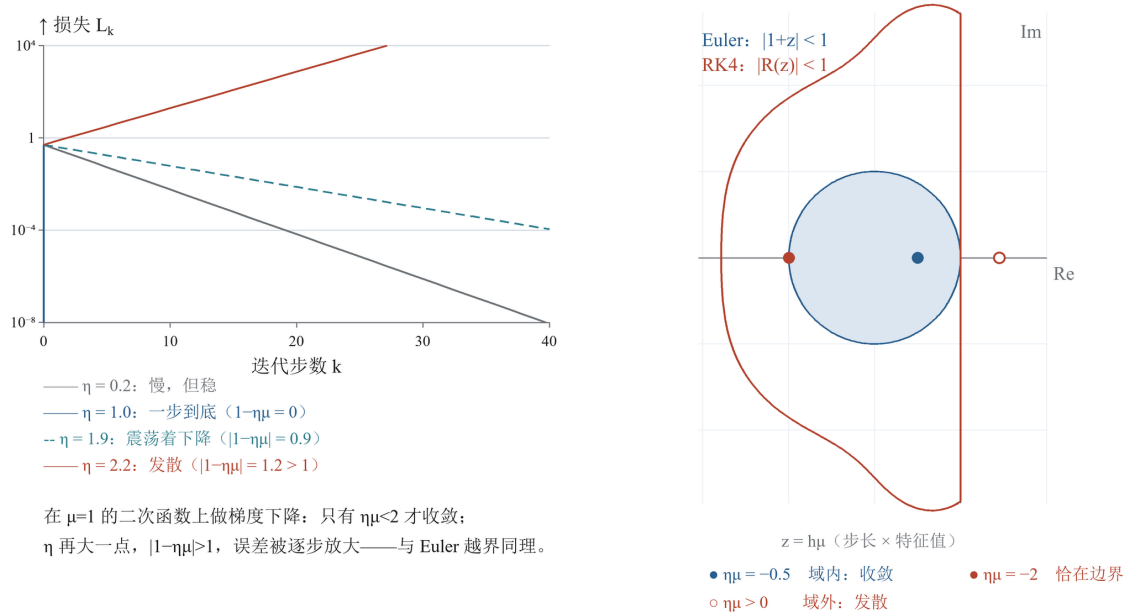


图 82　学习率就是步长：左，四条训练曲线（$|1 - \eta\,\mu|$ 决定收敛还是发散）；右，Euler 与 RK4 的稳定域，以及 $\eta\,\mu$ 取不同值时落在域内还是域外。本组计算并绘制

于是 3.4.1 节的三条结论可以照搬。第一，条件数即刚性： $\kappa = \mu_{\max}/\mu_{\min}$ 大，说明这个损失面“又长又窄”，梯度下降在窄方向取小步、在长方向缓慢推进，所需步数与 κ 成正比，与刚性方程“必须把步长取得极小”同理。第二，自适应步长对应自适应学习率：DP45 按局部误差调整 h，Adam 按梯度的一阶、二阶矩为每个参数调整步长[79]，两者都是“在陡峭处取小步”。第三，离散误差会进入参数：3.4.3.3 指出“换一套装置，参数里的污染就显现”，在深度学习中的对应现象是“更换优化器或 batch size，训练得到的模型不同”——这不是代码缺陷，而是积分器改变了。

这一线索近期又被推进了一步。Cohen 等人发现，实际训练中学习率常常停留在稳定边界附近（ $\eta \approx 2/\mu_{\max}$ ），损失在“尖峰—下降”之间反复振荡，他们把这个现象称为稳定性边缘（edge of stability）[80]。用本节的语言说：训练并不停留在稳定域内部，而是沿稳定域的边界进行。这也解释了损失曲线为何会波动——那多半不是随机性，而是积分器在边界附近振荡。

所以本节的结论还要再推进一步：既然训练本身就是一次积分，那么“求解器误差进入参数”在 AI 中是常态，而非意外。报告一个模型的效果时，至少要说清使用的是哪个优化器、多大的学习率、训练了多少步——正如报告数值解要说清格式与步长一样。三换协议至此也就有了完整的形式：它更换的不只是初值、步长与区间，还包括把参数走出来的那个“积分器”。

3.4.3.5 误差、成本与可信：工程上的三道难关

本节讨论的误差分解与三换验证，并不只适用于文中的玩具模型——在今天的实践中，它们仍是三道难关。

误差：本节已把逼近误差、离散误差与优化误差分开（命题 4），但多数论文仍只报告训练损失，而训练损失为零并不代表解对。

成本：Neural ODE 的一次前向需要几十到几百次网络调用，反向还要再计算一遍（3.4.1.3 的伴随法与 checkpointing）。生成模型中称为 NFE，科学计算中则意味着计算代价过高。

可信：是否在未用于训练的初值、步长与区间上验证过（3.4.3.2 的三换）？是否给出误差界或置信区间？换一组参数是否仍然有效？这三个问题没有回答，“AI for Science”就只能停留在演示阶段。

目前，社区已在用工程手段回应这些问题：统一的基准让“误差 10^{-3}”这一说法具有可比性[73]；可微求解器保证梯度计算正确；不确定性量化让模型输出一个分布而不只是一个数。

图 83　误差与验证这条线的几个节点：误差分解、自适应权重、可微求解器、基准与不确定性量化。本组绘制

这三道难关也解释了 3.4.3.2 的验证协议为何值得坚持：它们不是形式上的小心，而是把“能算”与“可信”区分开来的最低要求。

3.4.4　趣味编程：给一条生长曲线配一个向量场

把问题具体化：可用的只有 21 个数据点，即一条菌落生长曲线在两天里的等距采样，每个点都含 1% 的噪声；要回答的不是“第 22 个点是多少”，而是“这个菌落按什么规律生长”。这相当于只看一部影片的 21 帧，推断整部影片的叙事规则。

本节以这 21 个采样点为例，把前三节的内容完整实现一遍：构造数据、反演向量场、更换初值与步长验证。代码在 `A1-G3-3.4-实验.ipynb` 中，篇幅只有几十行，可以逐格运行。

这条 S 形曲线并非凭空构造。1838 年，比利时数学家 Verhulst 为了给“人口不可能一直指数增长”这一直觉一个表达式，写下了 $u' = ru(1-u/K)$，并称之为逻辑斯谛（logistic）曲线[65]。此后一个半世纪，同样的形状在互不相关的领域中反复出现：培养皿里的酵母、肿瘤的体积、一款新产品的市场渗透率，都呈现这一形态。

回到引子里那三条菌落生长曲线。取 logistic 型数据

$$u(t) = \frac{K}{1 + \left(\frac{K}{u_0} - 1\right)e^{-rt}}, \qquad K = 1,\ u_0 = 0.05,\ r = 0.8,$$

在 $t = 0, 0.1, \ldots, 2$ 上采样 21 个点，加 1% 的相对噪声作为观测。任务：只给这些点，反演向量场 $f_\theta(t, u)$。

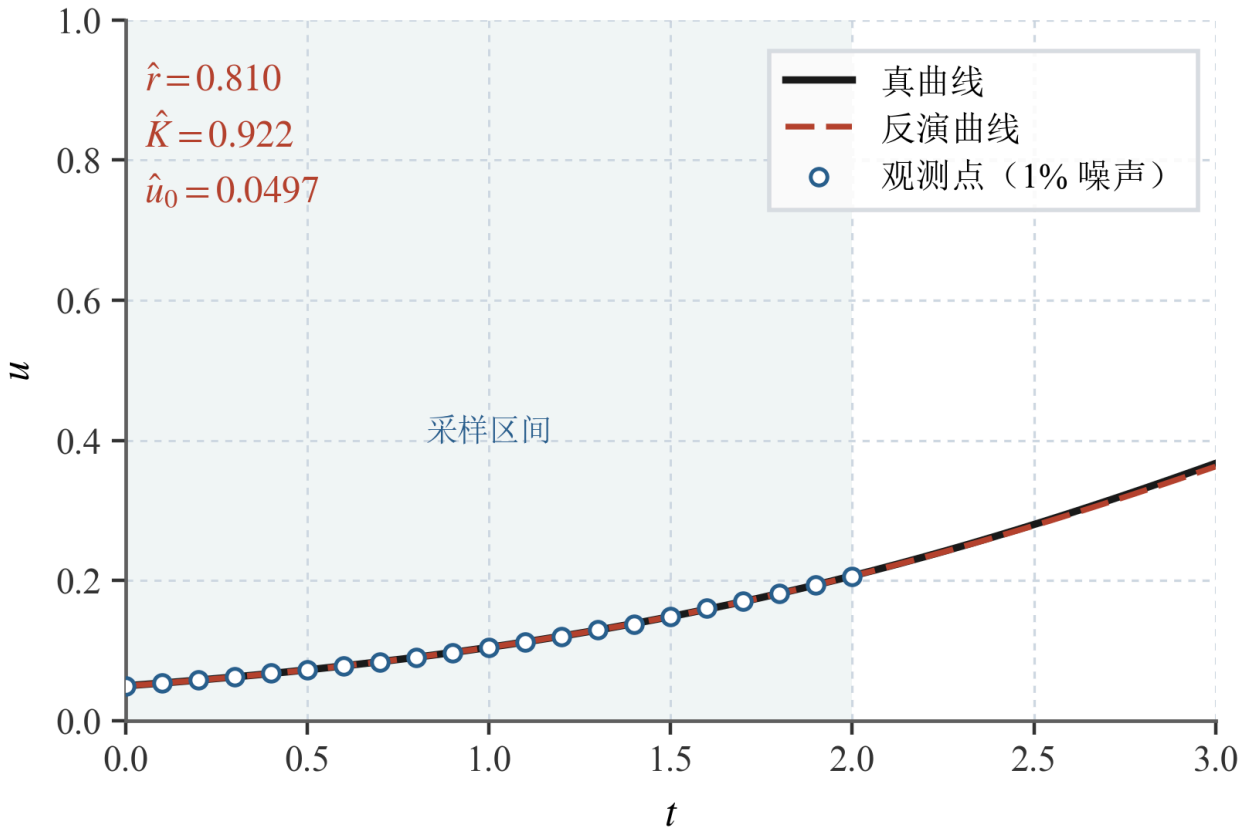


图 84　21 个含 1% 噪声的观测点（空心圆）、真曲线与反演出的曲线：$\hat{r}$=0.810，$\hat{K}$=0.922，$\hat{u}_0$=0.0497。本组计算并绘制

取 $\theta = (r, K, u_0)$ 三个参数（结构与真值相同），用 RK4（$h = 0.1$）积分，按式 (47) 训练。为了让读者能复现，实验脚本 `A1-G3-3.4-实验.ipynb` 做了四件事：

（1）生成含噪数据，画出观测点与真曲线；

（2）分别用 Euler（$h=0.2$）、RK4（$h=0.1$）、RK4（$h=0.05$）训练，比较反演出的 r；

（3）用训练好的模型换三个初值 $u(0)=0.02,0.05,0.2$ 各积一条，与真解对比，给出三条轨迹的最大偏差；

（4）把评估积分器换成步长的 $1/2$ 与 2 倍，给出“换步长偏移量”表（对应表 26 的框架）。

预期的结论是：h 越细，反演出的 r 越接近 0.8；换初值后三条轨迹都贴合真解（说明学到的是向量场而不是曲线）；换步长引起的偏移远小于换初值——若出现相反的情况，说明数据量不足或噪声过大。具体数值以 Notebook 的实际输出为准，正文不预先给定。

这里涉及一个真实反问题中几乎必然遇到的问题：可辨识性。拟合给出 $\hat{r}=0.810$、$\hat{K}=0.922$、$\hat{u}_0=0.0497$：r 准到 1%，K 却偏了 8%。这不是优化不充分，而是数据中没有关于 K 的信息。把灵敏度 $\partial u/\partial\theta$ 在采样区间上算出来，$\partial u/\partial r$ 的最大值约 0.33，$\partial u/\partial K$ 只有 0.034，相差近十倍。同样的噪声对 K 的干扰就大十倍；再加上曲线在 $[0,2]$ 上尚未出现明显弯曲，u 在早期几乎只以 u_0e^{rt} 的形式出现，K 被 u_0 与 r 的组合替代掉了。

这正是“换区间”这一步的价值：窗口扩展到曲线开始饱和处（$t\gtrsim 4$），$\partial u/\partial K$ 会增大到与 $\partial u/\partial r$ 同量级，K 才真正可辨识。在反问题中，能辨识什么取决于观测覆盖了哪一段数据——这一点比任何优化算法都更影响最终结果。

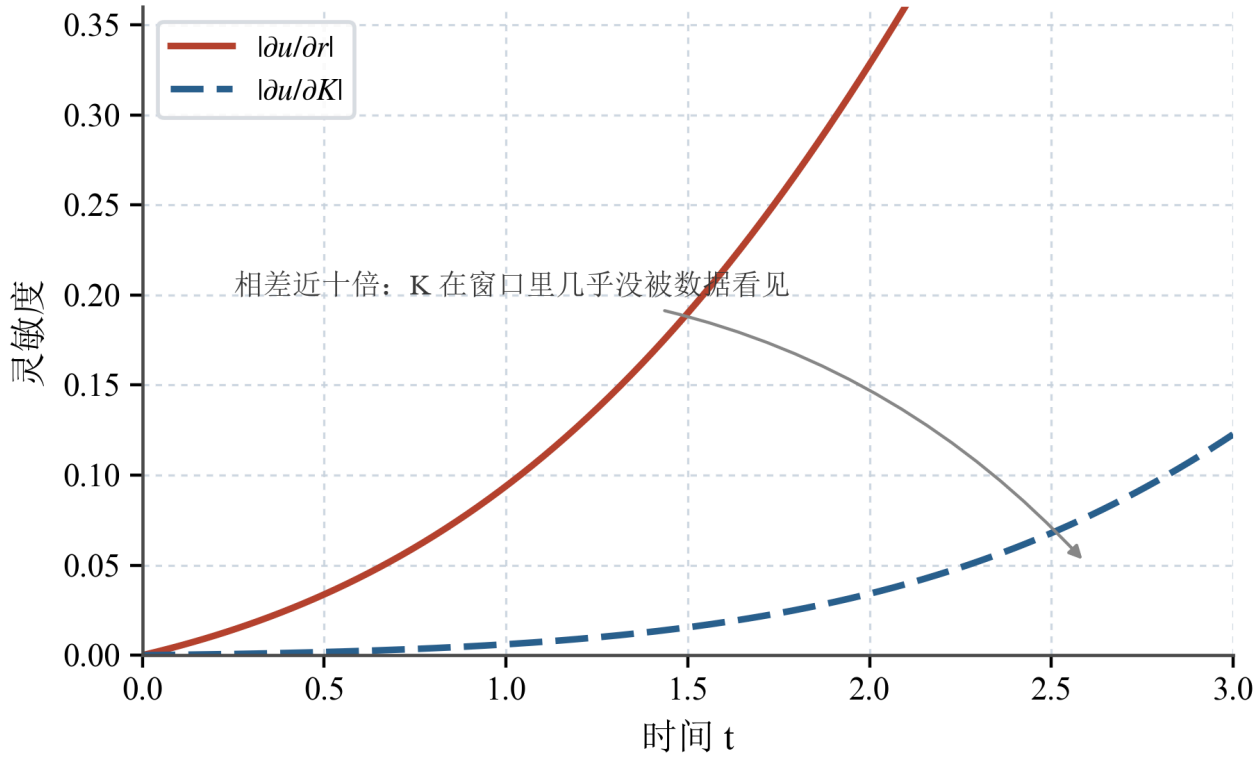


图 85　两个参数的灵敏度：采样窗口内 $\partial u/\partial r$ 明显大于 $\partial u/\partial K$，相差近十倍，因此 K 的反演精度较低。本组计算并绘制

3.4.5 Neural ODE 和 PINN 的今天

前四节讨论的是“怎么做”：怎样把 f 表示为一个网络、怎样判断模型是否学到、怎样把误差估计清楚。本节换一个角度，讨论“目前进展到哪一步”——这两条线分别发展成了什么、被谁使用、与 AI 的关系究竟在哪里，以及下一步通向何处。

先给出一张全景图。AI 与微分方程的结合可以分为四个方向，彼此的取向并不相同。

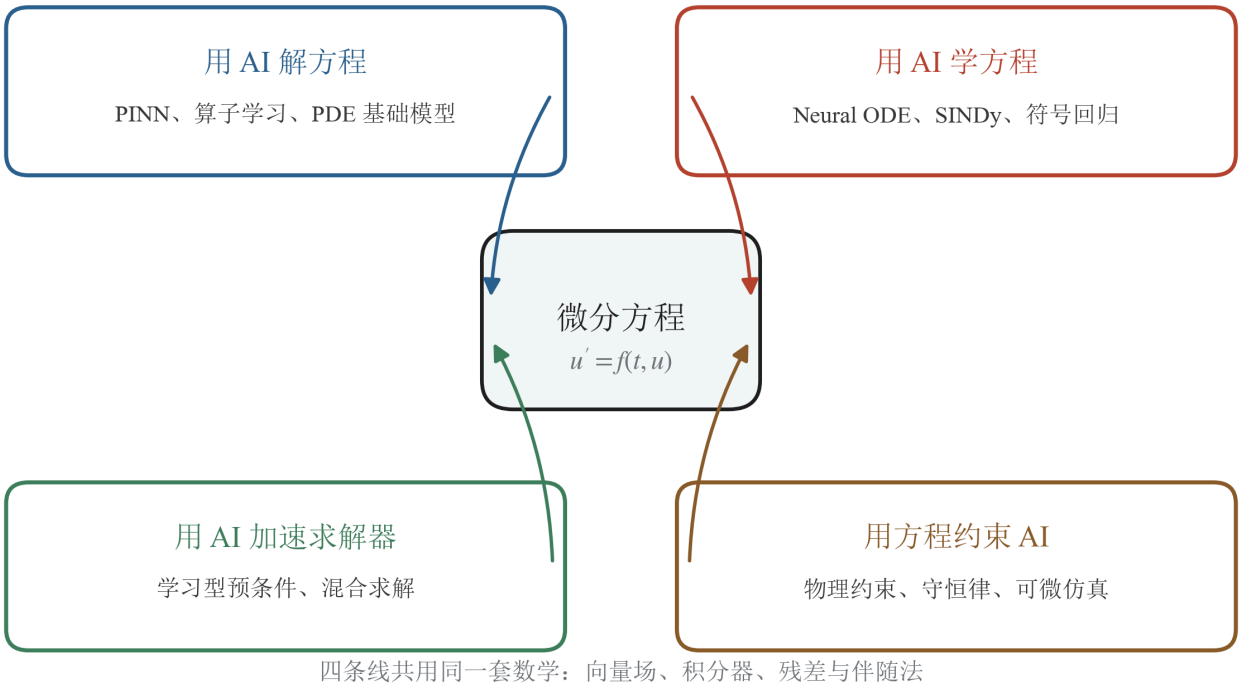


图 86　AI 与微分方程的四条交汇线：解方程、学方程、加速求解、约束 AI。四条线共用同一套数学——向量场、积分器、残差与伴随法。本组绘制

这四条线可以这样概括：用 AI 解方程——方程已知，让网络去满足它，PINN 与算子学习属于这一方向；用 AI 学方程——方程未知，从数据里反推向量场，Neural ODE 与符号回归走这条路；用 AI 加速求解器——网络不改答案，只做预条件、初值猜测、局部修正；用方程约束 AI——把守恒律、对称性、能量结构写进网络结构或损失函数。前两条是本节的主角（3.4.1、3.4.2 节），后两条是它们今天的产业形态。

3.4.5.1 用 AI 学方程：从 Neural ODE 到生成模型

首先需要明确“AI”在这一语境下的具体所指。本书所说“用 AI 学方程”，实际引入的是三项要素，它们分别对应后面的一个小节。

第一样，一张可微的函数族。多层感知机（MLP）不过是“仿射变换 + 一个非线性函数”重复若干次：

$$x_1 = \sigma(W_1[t;u] + b_1), \quad x_2 = \sigma(W_2 x_1 + b_2), \quad \cdots, \quad f_\theta(t,u) = W_L x_{L-1} + b_L,$$

其中 $\theta = \{W_i, b_i\}$ 是全部待定参数。写成这样，它就是一个“由 θ 决定的函数”：θ 改变，函数随之改变。1989 年 Cybenko、1991 年 Hornik 等人证明的万能逼近定理说：只要隐层足够宽（或者足够深），这样的函数就能在紧集上以任意精度逼近任何连续函数[74]。这是“把 f 交给一个网络”的依据所在——但

定理只保证“存在一组 θ”，并不保证“梯度下降能找到它”。前半句属于逼近论，后半句属于优化，两者之间的差距，正是后面所有困难的来源。

第二样，一次穿过计算图的求导。训练需要的是 $\partial\mathcal{L}/\partial\theta$。参数常有 10^5 到 10^6 个，逐个做有限差分要执行上百万次前向；而反向模式自动微分（即通常所说的反向传播）沿计算图反向遍历一次，代价只有前向的常数倍[75]。这条反向路径在 Neural ODE 中要穿过数值积分器，因而成为 3.4.1.3 的伴随法；在 PINN 中要穿过 $\partial_t u_\theta$ 这个微分算子，因而要求框架支持“对网络输出再求一次导”。

第三样，一条沿梯度推进的路径。有了表示和梯度，余下的步骤即 $\theta \leftarrow \theta - \eta\nabla_\theta\mathcal{L}$，重复数万次。这一式最容易被低估：$\eta$ 是学习率，也就是数值积分中的步长；η 过大时，损失会振荡甚至发散，这与 3.1.4.2 节 Euler 法“h 必须落在稳定域内”是同一件事（3.4.3.4 节给出了这一计算）。

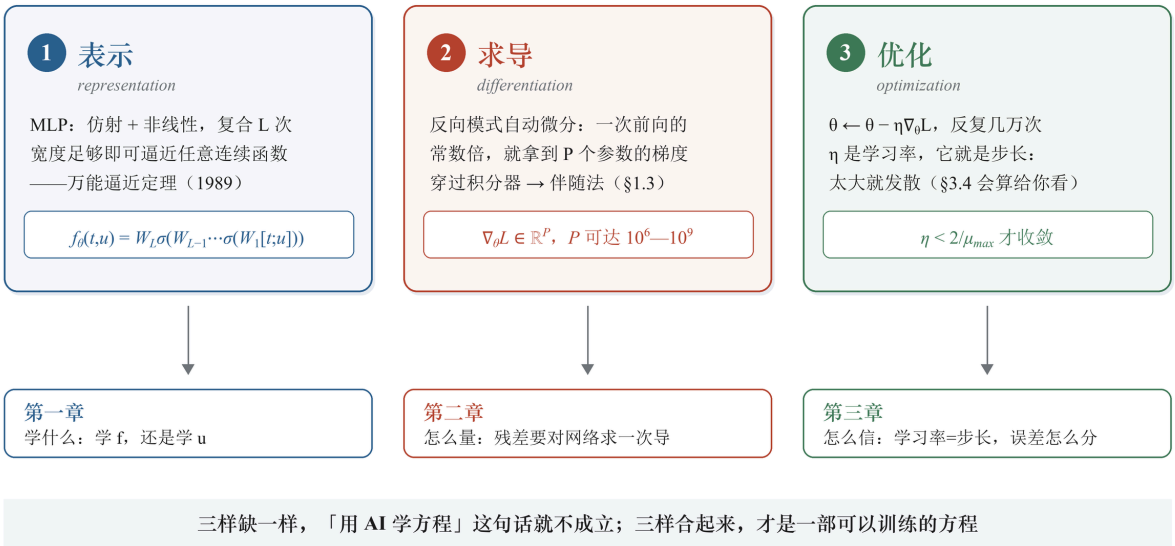


图 87　AI 加进来的三样东西，以及它们各自对应本书的哪一章：表示决定“学什么”，求导决定“怎么量”，优化决定“怎么信”。图片来源：本组绘制

三样合起来，才构成“用 AI 学方程”这一说法，缺一样都不成立：没有可微函数族，就没有候选；没有自动微分，就无法确定应当往哪个方向修改；没有梯度下降，就无法完成修改。反过来看，本节前三小节恰好对应这三样：3.4.1 节讨论“表示成什么”（学 f 还是学 u），3.4.2 节讨论“用哪一把尺子衡量”（轨迹还是残差），3.4.3 节讨论“这些误差怎样区分、怎样验证”。这也是标题中“关联”二字的本意——不是把 AI 与 ODE 并列作类比，而是把 AI 的三个部件，逐一装到微分方程的三个位置上。

引子中那两条路线，今天都已成为 AI 的主干应用。Neural ODE 这一支演化出连续归一化流与流匹配，成了图像与视频生成的一类主流方法——采样过程就是解一个 ODE，步数（NFE）直接决定生成成本；PINN 那一支演化出算子学习与 PDE 基础模型，被用于数字孪生、芯片散热、电池与血流仿真，这一支已在 3.4.2.4 讨论。前面几节反复出现的积分器、残差与伴随法，最终都落在这些产品之中。本节先讨论其中的第一条。

3.4.1 节开头“学 f 而不是学 y”的想法，在生成模型中被运用得最为彻底。把一张图片看成 $\mathbb{R}^d$ 里的一个点，把噪声看成另一个点：如果能学到一个速度场 $v(x,t)$，让噪声沿着它积分到 $t=1$ 时正好落在数据分布上，那么“生成一张图”就变成了“解一个 ODE”。

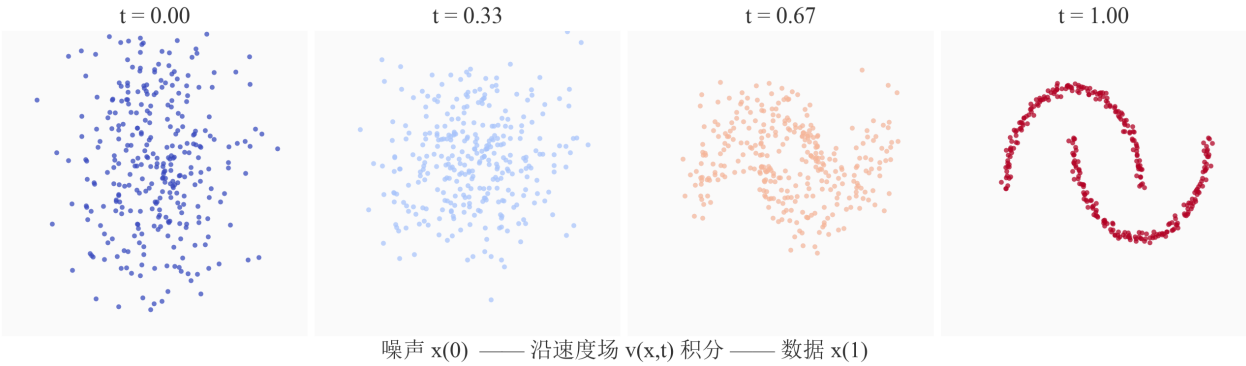


图 88　生成模型的 ODE 视角：噪声（左）沿速度场被推成数据分布（右），四个时刻是同一个积分的四帧。本组计算并绘制

这条路线的技术演进相当清晰：连续归一化流（CNF）第一次把它做成可训练的模型，代价是每一步都要计算散度；2022 年的流匹配（flow matching）[71] 绕开散度，直接回归速度场；2023 年之后的整流流（rectified flow）进一步把路径拉直，使得几步甚至一步即可采样。今天主流的图像与视频生成模型中，不少采样器就是 ODE 求解器：采样步数（NFE）直接换算为生成成本，蒸馏与一致性模型所做的正是“把步数压缩到 1—4 步”。

这条线上还有一次更彻底的“方程化”。2021 年之后，扩散模型把“加噪—去噪”写成了一个随机微分方程，而它的概率流 ODE 是一个确定性方程：把学到的得分函数代进去，采样就变成解一个 ODE[81]。于是“生成一张图”与“积一条轨迹”在数学上是同一件事——前者只是后者的高维、非物理版本。这也带来一个十分工程化的后果：采样质量与采样步数之间存在一条可测的曲线（图 89）。步数从 4 增加到 16，质量提升很快；从 64 增加到 256，几乎看不出差别，成本却增至四倍。因此这一代生成模型的竞争，一半在于改进训练目标（把曲线整体抬高），一半在于把拐点左移（蒸馏、一致性模型、少步采样器）。

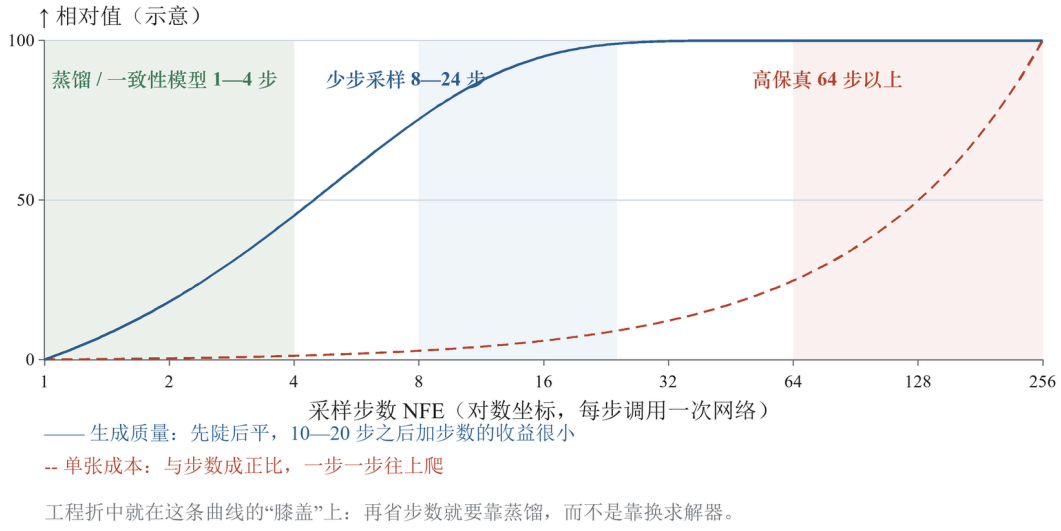


图 89　采样步数、生成质量与单张成本的关系（示意图）：质量先快速上升后趋于饱和，成本随步数线性增长，工程折中落在曲线的拐点附近。本组绘制

第二条应用是时间序列。医疗监护、传感器与金融数据都有两个特点：采样间隔不规则、存在缺失。Latent ODE 把观测映射到低维隐空间再连续积分，Neural CDE 用受控微分方程直接处理不规则的时间戳——相较 RNN 的“固定节拍”，它们更自然地处理“何时观测”这一问题。

第三条是控制与强化学习：把策略写成一个向量场，动作即沿它走一步。连续时间的表述使“何时决策”与“决策什么”解耦，采样效率通常更高。

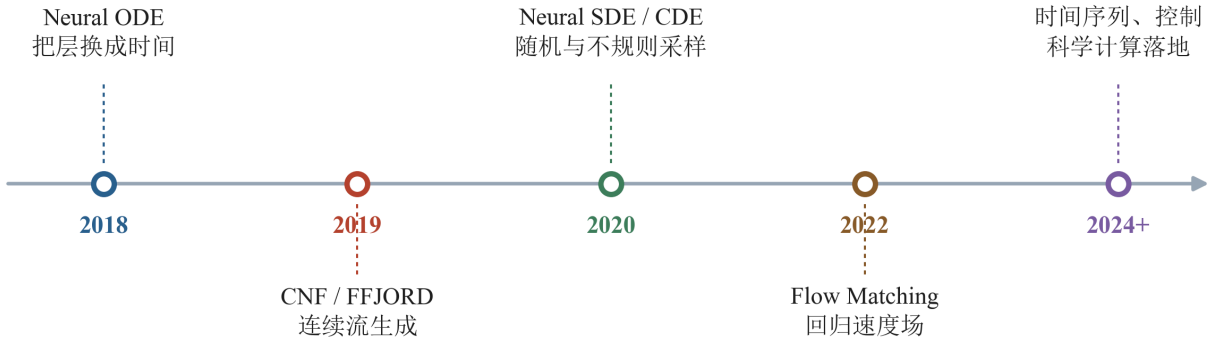


图 90　这一条线的时间线：2018 年 Neural ODE，2019 年连续归一化流，2020 年随机版本与不规则采样，2022 年之后的流匹配，2024 年之后的落地。本组绘制

3.4.5.2　用 AI 加速求解器，与用方程约束 AI

第三条线最不显眼，却最容易落地：网络不替代求解器，只负责“让求解器更快”。例如用网络给出初值猜测、学习预条件子、在粗网格上先算一遍再修正。这类做法不改变数值格式的收敛性，风险较小，工业界的接受度反而最高。

第四条线方向相反：用方程约束 AI。Hamiltonian 网络把向量场写成 $(\partial H_\theta/\partial p, -\partial H_\theta/\partial q)$，能量自动守恒；等变网络把旋转、平移对称性写进结构；物理约束的损失则把“不满足方程即惩罚”直接加入训练目标。约束越强，所需样本越少、外推越稳定，代价是表达能力受限——这是一条长期的权衡。

3.4.5.3　展望与结语

至于未来，大致有四个值得投入的方向：更大规模的 PDE 基础模型——能否像语言模型那样 scaling；可证明的误差界——目前多为经验结果，理论上仍有较大空白；物理与数据的混合建模——不是二选一，而是分工，物理负责外推、数据负责修正；自动发现方程——从数据直接得到可解释的方程，而不只是一个网络[82]。这四条都还在早期，也正好是本书留给读者的题目。

最后回到标题。所谓“AI 关联”，不是“AI 也能解方程”这种并列关系，而是三层嵌套：表示上，网络提供了一个高维、可微、参数共享的函数族，把“手工建模”换成了“可训练的对象”；求导上，自动微分把梯度送到每一个参数、每一步积分；优化上，梯度下降在损失面上走出的那条路径，本身也是一次积分。三层叠在一起，才有了“用数据反推方程、用方程约束网络”这两件事。也正因为是三层嵌套，任何一层出问题，现象都很相似——训练不收敛、结果不可信、换一个场景就失效。要区分它们，依靠的仍是那三样基本工具：误差分解、独立验证，以及换一套装置重算一遍。

参考文献

[1] L. Euler. Institutiones calculi integralis, Vol. 1. St Petersburg: Imperial Academy of Sciences, 1768.

[2] C. Runge. Über die numerische Auflösung von Differentialgleichungen. Mathematische Annalen, 1895, 46: 167–178.

[3] W. Kutta. Beitrag zur näherungsweisen Integration totaler Differentialgleichungen. Zeitschrift für Mathematik und Physik, 1901, 46: 435–453.

[4] K. Heun. Neue Methode zur approximativen Integration der Differentialgleichungen einer unabhängigen Veränderlichen. Zeitschrift für Mathematik und Physik, 1900, 45: 23–38.

[5] E. Hairer, S. P. Nørsett, G. Wanner. Solving Ordinary Differential Equations I: Nonstiff Problems, 2nd ed. Berlin: Springer, 1993.

[6] E. A. Coddington, N. Levinson. Theory of Ordinary Differential Equations. New York: McGraw-Hill, 1955.

[7] P. Hartman. Ordinary Differential Equations. New York: Wiley, 1964.

[8] J. Shiach. Ordinary Differential Equations: Error Analysis. https://jonshiach.github.io/ODEs-book/_pages/1.2_Error_Analysis.html

[9] J. Shiach. Ordinary Differential Equations: Absolute Stability. https://jonshiach.github.io/ODEs-book/_pages/4.3_Absolute_stability.html

[10] J. C. Butcher. Numerical Methods for Ordinary Differential Equations, 3rd ed. Chichester: Wiley, 2016.

[11] R. L. Burden, J. D. Faires. Numerical Analysis, 10th ed. Boston: Cengage Learning, 2016.

[12] L. F. Richardson. Weather Prediction by Numerical Process. Cambridge: Cambridge University Press, 1922.

[13] J. R. Dormand, P. J. Prince. A family of embedded Runge–Kutta formulae. Journal of Computational and Applied Mathematics, 1980, 6(1): 19–26.

[14] J. J. O'Connor, E. F. Robertson. Jean Baptiste Joseph Fourier. MacTutor History of Mathematics Archive, University of St Andrews. https://mathshistory.st-andrews.ac.uk/Biographies/Fourier/

[15] J. Fourier. Mémoire sur la propagation de la chaleur dans les corps solides. 提交巴黎科学院，1807-12-21.

[16] J. Fourier. Théorie analytique de la chaleur. Paris: Firmin Didot, 1822.

[17] 中国科学家博物馆. 周毓麟：为核武事业奉献终身（中国科学家，2021-03-05）. http://mmcs.cast.org.cn/kxjfc/kxjfc/zyl/bd7590/art/2023/art_d31374394f5a4f95879333fce4307b41.html

[18] 华冬英, 李祥贵. 微分方程的数值解法与程序实现——基于 Proteus 仿真. 3 版. 北京: 电子工业出版社, 2016.

[19] R. Courant, K. Friedrichs, H. Lewy. Über die partiellen Differenzengleichungen der mathematischen Physik. Mathematische Annalen, 1928, 100(1): 32–74.

[20] G. G. O'Brien, M. A. Hyman, S. Kaplan. A study of the numerical solution of partial differential equations. Journal of Mathematics and Physics, 1951, 29: 223–251.

[21] J. Crank, P. Nicolson. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. Proceedings of the Cambridge Philosophical Society, 1947, 43(1): 50–67.

[22] R. J. LeVeque. Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems. Philadelphia: SIAM, 2007.

[23] K. W. Morton, D. F. Mayers. Numerical Solution of Partial Differential Equations: An Introduction, 2nd ed. Cambridge: Cambridge University Press, 2005.

[24] J. Shiach. Finite Difference Methods（在线讲义）. https://jonshiach.github.io/files/notes/finite_difference_methods_notes.pdf

[25] B. Seibold. 18.336 Numerical Methods for Partial Differential Equations, MIT OpenCourseWare, Spring 2009. https://ocw.mit.edu/courses/18-336-numerical-methods-for-partial-differential-equations-spring-2009/

[26] P. Perona, J. Malik. Scale-space and edge detection using anisotropic diffusion. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1990, 12(7): 629–639.

[27] F. Black, M. Scholes. The pricing of options and corporate liabilities. Journal of Political Economy, 1973, 81(3): 637–654.

[28] H. H. Pennes. Analysis of tissue and arterial blood temperatures in the resting human forearm. Journal of Applied Physiology, 1948, 1(2): 93–122.

[29] 新华社. 珍贵！我国首颗原子弹爆炸成功画面. 2025-10-16. http://www.xinhuanet.com/politics/20251016/1efa1fbdc27440199e49eff76166891c/c.html

[30] M. Bertalmío, G. Sapiro, V. Caselles, C. Ballester. Image inpainting. Proceedings of SIGGRAPH 2000: 417–424.

[31] T. F. Chan, J. Shen. Image Processing and Analysis: Variational, PDE, Wavelet, and Stochastic Methods. Philadelphia: SIAM, 2005.

[32] ISO/IEC 18004:2015. Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification.

[33] P. Sirikongtham, A. Nimkoompai. Enhancing the scanability of damaged QR codes through image restoration using GANs combined with the spectral normalization technique. International Journal of Advanced Computer Science and Applications, 2025, 16(10).

[34] Scratched coating QR code image restoration based on the pluralistic image completion deep learning method. Journal of Imaging Science and Technology, 2025, 69(3): 030414.

[35] 贾随军. 傅立叶与热传导理论的数学化. 自然辩证法通讯, 2009, 31(5). https://jdn.ucas.ac.cn/index.php/home/journal/view/id/4816

[36] T. N. Narasimhan. Fourier's heat conduction equation: History, influence, and connections. Reviews of Geophysics, 1999, 37(1): 151–172. https://doi.org/10.1029/1998RG900006

[37] 吴明静. 采数学之美为吾美：周毓麟传. 北京: 中国科学技术出版社, 上海交通大学出版社, 2021.

[38] J. J. Koenderink. The structure of images. Biological Cybernetics, 1984, 50(5): 363–370. （“图像的多尺度描述由高斯卷积给出”的经典出处）

[39] W. Ritz. Über eine neue Methode zur Lösung gewisser Variationsprobleme der mathematischen Physik. Journal für die reine und angewandte Mathematik, 1909, 135: 1–61.

[40] B. G. Galerkin. Rods and plates: series solution of some problems of elastic equilibrium of rods and plates (in Russian). Vestnik Inzhenerov, 1915, 19: 897–908.

[41] R. Courant. Variational methods for the solution of problems of equilibrium and vibrations. Bulletin of the American Mathematical Society, 1943, 49: 1–23.

[42] M. J. Turner, R. W. Clough, H. C. Martin, L. J. Topp. Stiffness and deflection analysis of complex structures. Journal of the Aeronautical Sciences, 1956, 23(9): 805–823.

[43] R. W. Clough. The finite element method in plane stress analysis. Proceedings of the 2nd ASCE Conference on Electronic Computation, Pittsburgh, 1960: 345–378.

[44] G. Strang, G. J. Fix. An Analysis of the Finite Element Method. Englewood Cliffs: Prentice-Hall, 1973.

[45] O. C. Zienkiewicz, R. L. Taylor, J. Z. Zhu. The Finite Element Method: Its Basis and Fundamentals, 7th ed. Oxford: Butterworth-Heinemann, 2013.

[46] A. Quarteroni, R. Sacco, F. Saleri. Numerical Mathematics, 2nd ed. Berlin: Springer, 2007.

[47] S. C. Brenner, L. R. Scott. The Mathematical Theory of Finite Element Methods, 3rd ed. New York: Springer, 2008.

[48] J. Céa. Approximation variationnelle des problèmes aux limites. Annales de l'Institut Fourier, 1964, 14(2): 345–444.

[49] J. J. O'Connor, E. F. Robertson. Richard Courant. MacTutor History of Mathematics Archive, University of St Andrews. https://mathshistory.st-andrews.ac.uk/Biographies/Courant/

[50] M. R. Hestenes, E. Stiefel. Methods of conjugate gradients for solving linear systems. Journal of Research of the National Bureau of Standards, 1952, 49(6): 409–436.

[51] Y. Saad. Iterative Methods for Sparse Linear Systems, 2nd ed. Philadelphia: SIAM, 2003.

[52] C. Johnson. Numerical Solution of Partial Differential Equations by the Finite Element Method. Cambridge: Cambridge University Press, 1987.

[53] P. G. Ciarlet. The Finite Element Method for Elliptic Problems. Amsterdam: North-Holland, 1978.

[54] M. P. Bendsøe, N. Kikuchi. Generating optimal topologies in structural design using a homogenization method. Computer Methods in Applied Mechanics and Engineering, 1988, 71(2): 197–224.

[55] T. J. R. Hughes, J. A. Cottrell, Y. Bazilevs. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. Computer Methods in Applied Mechanics and Engineering, 2005, 194(39–41): 4135–4195.

[56] M. Raissi, P. Perdikaris, G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 2019, 378: 686–707.

[57] Z. Li, N. Kovachki, K. Azizzadenesheli, et al. Fourier neural operator for parametric partial differential equations. International Conference on Learning Representations (ICLR), 2021.

[58] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud. Neural Ordinary Differential Equations. NeurIPS, 2018: 6571–6583.

[59] S. Wang, X. Yu, P. Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. Journal of Computational Physics, 2022, 449: 110768.

[60] R. T. Q. Chen. torchdiffeq: 可微 ODE 求解器（开源实现）. https://github.com/rtqichen/torchdiffeq

[61] 飞桨 PaddleScience：面向科学计算的物理信息神经网络工具. https://www.paddlepaddle.org.cn/paddlescience

[62] MindSpore Science 文档（含北京大学团队基于昇思发布的 PDEformer-2）. https://www.mindspore.cn/mindscience/docs/zh-CN/master/index.html

[63] E. N. Lorenz. Deterministic nonperiodic flow. Journal of the Atmospheric Sciences, 1963, 20: 130–141.

[64] F.-X. Le Dimet, O. Talagrand. Variational algorithms for analysis and assimilation of meteorological observations: theoretical aspects. Tellus A, 1986, 38: 97–110.

[65] P.-F. Verhulst. Notice sur la loi que la population suit dans son accroissement. Correspondance Mathématique et Physique, 1838, 10: 113–121.

[66] A. Griewank. Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation. Optimization Methods and Software, 1992, 1(1): 35–54.

[67] T. Miyato, T. Kataoka, M. Koyama, Y. Yoshida. Spectral normalization for generative adversarial networks. ICLR, 2018.

[68] A. Kendall, Y. Gal, R. Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. CVPR, 2018: 7482–7491.

[69] S. Wang, Y. Teng, P. Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM Journal on Scientific Computing, 2021, 43(5): A3055–A3081.

[70] W. E, B. Yu. The deep Ritz method: A deep learning-based numerical algorithm for solving variational problems. Communications in Mathematics and Statistics, 2018, 6(1): 1–12.

[71] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. Flow matching for generative modeling. ICLR, 2023.

[72] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nature Machine Intelligence, 2021, 3: 218–229.

[73] S. Takamoto, T. Praditia, R. Leong, et al. PDEBench: An extensive benchmark for scientific machine learning. NeurIPS Datasets and Benchmarks, 2022.

[74] K. Hornik. Approximation capabilities of multilayer feedforward networks. Neural Networks, 1991, 4(2): 251–257.

[75] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind. Automatic differentiation in machine learning: a survey. Journal of Machine Learning Research, 2018, 18(153): 1–43.

[76] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. CVPR, 2016: 770–778.

[77] C. Szegedy, W. Zaremba, I. Sutskever, et al. Intriguing properties of neural networks. ICLR, 2014.

[78] A. Jacot, F. Gabriel, C. Hongler. Neural tangent kernel: Convergence and generalization in neural networks. NeurIPS, 2018.

[79] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. ICLR, 2015.

[80] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, A. Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. ICLR, 2021.

[81] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al. Score-based generative modeling through stochastic differential equations. ICLR, 2021.

[82] S. L. Brunton, J. L. Proctor, J. N. Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. PNAS, 2016, 113(15): 3932–3937.

[83] C. Rackauckas, Y. Ma, J. Martensen, et al. Universal differential equations for scientific machine learning. arXiv:2001.04385, 2020.

本章习题与参考解答

第 03 章　离散的世界 · 习题

> 本文件：`A1-G3-习题.md`（G3 = 第 03 章 A1 编写组） 范围：覆盖 3.1–3.4 全章，共 77 题——3.1 初值问题与 Runge–Kutta 方法（题 1–22）、3.2 热传导方程、差分格式与稳定性（题 23–42）、3.3 有限元：弱形式与 Galerkin 方法（题 43–55）、3.4 AI 关联：Neural ODE 与 PINN（题 56–77）。 配套文件：参考解答分上、下两册——A1-G3-习题参考解答.md（题 1–42）、A1-G3-习题参考

> 解答-3.3-3.4.md（题 43－77）；另配 A1-G3-正文.md、A1-G3-实验.ipynb。 版本：全章合并稿（2026-10-26 由 3.1－3.4 分节稿合并；题号全章连续，按小节分十一部分，版式照 3.1 节）。参考答案因 GitHub 单页公式数量上限（约 2000 个渲染单元）分为上、下两册，题号不变。 题号约定：全章连续编号，题号与解答文件一一对应，不得增删。标“快速”“找错”“开放”“陷阱”的题不设唯一答案，但需给出理由。 引用约定：题面中的定理、命题、推论、定义、公式、图、表编号，均指该题所属小节正文的编号；3.4 的四小节在正文中按“第一～四章”称呼（其内部再分 §1.1 等），本书按 3.4.1－3.4.4 编号。 记号：3.1 中 L 为 Lipschitz 常数、h 为步长；3.2 中空间步长 h、时间步长 Δt、网格比 $r=\kappa\Delta t/h^2$，节点 $x_j=jh$、$t_n=n\Delta t$，网格函数 U_j^n，统一取 $\kappa=1$、$L=1$；3.3 中 $-u''=f$、$x\in(0,1)$、$u(0)=u(1)=0$，$h=1/N$、节点 $x_i=ih$、单元 $e_i=[x_{i-1},x_i]$，单元刚度矩阵 $K^{(e)}=\frac{1}{h}\begin{pmatrix}1&-1\\-1&1\end{pmatrix}$；3.4 中 L 指关于 u 的 Lipschitz 常数、M 为步数或网络层数（按上下文区分）、d 为状态维数、θ 为网络参数。 改编说明：题 32－36 改编自教材《微分方程的数值解法与程序实现》第三章例题与习题（原书用 τ、m 记时间步长与空间分段数，称 $r=a\tau/h^2$ 为“网比”，网格函数写 u_i^k）。换记号时已把原题的 $u_t=u_{xx}+(t+1)\sin x$ 统一写成 $u_t=u_{xx}+g(x,t)$，并补出原题未给出的差分式与初边值离散写法；原题的“列表输出 5 个节点”按本文要求精简为 1－2 个节点的结果比较，便于手算核对。 难度标记：★☆☆ 基础（概念或一步计算）；★★☆ 综合（多步推导或需编程核对）；★★★ 综合运用（完整推导、编程复核或开放设计）。预计用时按熟手估算。

分值建议：全套 77 题，四节各成一块。主要权重放在需要完整推导、算账与判断的题上：3.1 的题 1－4、11－12、22；3.2 的题 32－36（改编教材题）与 42；3.3 的题 43－46；3.4 的题 56－59、64－71、74－75、77。快速题与找错题用于随堂检查；具体分值由教师按教学安排分配。

第一部分　3.1.1 初值问题与 Euler 方法（题 1–12）

题 1　Lipschitz 条件与经典解

对应： 3.1.1 ｜ 难度： ★★★ ｜ 预计用时： 30 分钟

设三个右端函数

$$f_1(t,y)=t^2+y^2,\qquad f_2(t,y)=\sqrt{|y|},\qquad f_3(t,y)=e^{-y}.$$

(1) 在带形 $|y|\le R$ 上分别求它们的 Lipschitz 常数，并指出哪个是全局 Lipschitz、哪个只是局部 Lipschitz。

(2) 指出哪一个在含 $y=0$ 的任何区域上都不满足 Lipschitz 条件，用差商说明。

(3) 对初值问题 $y'=\sqrt{|y|}$、$y(0)=0$，验证 $y(t)\equiv 0$ 与 $y(t)=t^2/4\ (t\ge 0)$ 都是经典解，并说明这违反了哪一条定理的哪一个前提。

(4) 填下表（f_2 在原点处的差商），并说明它与 (2) 的关系：

u	10^{-2}	10^{-4}	10^{-6}
$\lvert f_2(t,u)-f_2(t,0)\rvert/u$	?	?	?

(5) 陷阱题　判断：“f 连续，则初值问题的解唯一。”若错误，给出反例并说明唯一性真正需要什么条件。

题 2　Picard 迭代

对应： 3.1.1 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

对 $y'=y$、$y(0)=1$：

(1) 从 $y^{(0)}\equiv 1$ 出发，算出 $y^{(1)},y^{(2)},y^{(3)}$，写出 $y^{(m)}$ 的一般形式。

(2) 在矩形 $|t|\le T$、$|y-1|\le 1$ 上求 M 与 L，并对 $m=0,1,2$ 逐项验证估计式

$$|y^{(m+1)}(t)-y^{(m)}(t)|\le \frac{ML^m}{(m+1)!}\,|t|^{m+1}.$$

(3) 说明为什么 $\{y^{(m)}\}$ 在 $C[0,T]$ 上一致收敛。

(4) 填下表并指出极限值与哪个常数有关：

m	1	2	3	4	5	6	7	8
$y^{(m)}(1)$	?	?	?	?	?	?	?	?

(5) 陷阱题　判断：“Picard 迭代每算一步就是对解的一次更好的数值近似，因此可以取 $m=100$ 当数值解法用。”

题 3　Gronwall 与连续依赖

对应： 3.1.1 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

考察 $y'=-y$ 的两个解：$y(0)=1$ 与 $z(0)=1.01$。

(1) 写出精确解与精确差 $|y(t)-z(t)|$，判断它随 t 增大还是减小。

(2) 用定理 2 给出界，并计算 $t=10$ 处的界与精确差，求二者相差的倍数。

(3) 解释这个界为什么松，并计算 $[0,1]$ 上界与精确差之比。

(4) 填下表：

t	0	2	4	6	8	10
精确差	?	?	?	?	?	?
定理 2 的界	?	?	?	?	?	?

(5) 陷阱题　判断：“(2) 中界高达 220，说明这个问题数值上不可靠。”

题 4　解的存在区间与整体延拓

对应： 3.1.1 ｜ 难度： ★★★ ｜ 预计用时： 30 分钟

对 $y'=1+y^2$、$y(0)=0$：

(1) 在矩形 $|t|\le 1$、$|y|\le 1$ 上用定理 1 求存在区间的长度 δ。

(2) 精确解为 $y=\tan t$。令矩形半高为 b，求 $\delta(b)$，并求它的最大值。由此说明“只能在局部保证”是真的局限，而不是证明方法不够好。

(3) 下列右端函数中，哪一个可以把解延拓到任意有限区间？说明理由：$f(t,y)=1+y^2$；$f(t,y)=-y$。

(4) 取 $h=0.1$ 的 Euler 格式，填下表并与 $\tan t$ 比较：

t	1.0	1.4	1.5	1.6
Euler	?	?	?	?
$\tan t$	?	?	?	?

(5) 陷阱题　判断：“定理 1 只保证 $|t|\le\delta$ 上有解，所以 Euler 格式算到 δ 以后就没有意义了。”

题 5　快速题：初值问题与经典解

对应： 3.1.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

写出初值问题与经典解的定义，并说明为什么要求 $y\in C^1(I)$，而不是只要求 y 连续。

题 6　快速题：Lipschitz 常数

对应： 3.1.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

$f(t,y)=t^2+y^2$，在带形 $|y|\le 2$ 上求一个可用的 Lipschitz 常数。

题 7　快速题：存在区间半长

对应： 3.1.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

设 f 在矩形 $|t-t_0|\le 1$、$|y-y_0|\le 0.5$ 上连续且 $|f|\le 2$，满足 Lipschitz 条件。求定理 1 给出的存在区间半长 δ。

题 8　快速题：Gronwall 不等式

对应： 3.1.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

叙述 Gronwall 不等式，并说明当 $\alpha=0$ 时能得出什么更强的结论。

题 9　快速题：绝对值右端

对应： 3.1.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

$f(t,y)=|y|$ 是否在 $\mathbb{R}^2$ 上关于 y 满足 Lipschitz 条件？若是，给出常数。由此说明“Lipschitz”与“连续可微”哪一个更强。

题 10　快速题：Euler 一步

对应： 3.1.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

对 $y'=-y$、$y(0)=1$，用 $h=0.5$ 的显式 Euler 格式算一步，得 $y_1=$？与精确值 $y(0.5)=0.606531$ 相比是偏大还是偏小？用“斜率取在区间哪一端”解释。

题 11　找错题：Euler 的步数与末点

对应： 3.1.1 ｜ 难度： ★★☆ ｜ 预计用时： 15 分钟

下面的代码想用显式 Euler 计算 $y'=-y$ 在 $[0,1]$ 上的解，并与解析解比较最大误差：

```
T, t0 = 1.0, 0.0
h = 0.3
M = round((T - t0) / h)          # 步数
t, y = t0, 1.0
err = 0.0
for n in range(M):
    y = y + h * (-y)
    t = t + h
    err = max(err, abs(y - exact(t)))   # exact(t) = exp(-t)
```

(1) 求出实际执行的步数 M 与循环结束时 t 的值，说明误差为什么“偏离趋势”。　(2) 给出最小改动，使最后一步恰好落在 T 上。　(3) 这段代码会报错吗？它属于格式错误还是实现错误？

题 12　开放题：牛顿冷却

对应： 3.1.1 ｜ 难度： ★★★ ｜ 预计用时： 30 分钟

一杯 $80°\mathrm{C}$ 的热水放到 $20°\mathrm{C}$ 的房间里，用牛顿冷却定律建模：$\theta'=-k(\theta-\theta_{\text{env}})$，初值 $\theta(0)=\theta_0=80$。请回答：

(1) 求 Lipschitz 常数，说明它与温度范围是否有关；　(2) 定理 1 能保证在多大的区间上存在唯一解？能否据此断言“算到 $t=100$ 没问题”？　(3) 若初始温度有 $0.5°\mathrm{C}$ 的测量误差，用定理 2 给出 θ 的偏差上界；　(4) 本题的条件能否定出“步长最大取多少”？如果不能，说明应到哪一部分去找这条结论。

最后给出一句话结论：本节能保证什么，不能保证什么。

第二部分　3.1.2 经典 Runge–Kutta 方法（题 13–17）

题 13　RK4 手算一步

对应： 3.1.2 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

对 $y'=-y$、$y(0)=1$，取 $h=0.5$：

(1) 按四个阶段算出 k_1,k_2,k_3,k_4 与 y_1，保留到小数点后 6 位；　(2) 与精确值 $e^{-0.5}=0.6065307$ 比较，求误差；　(3) 显式 Euler 在同一问题、同一步长下给出 $y_1=0.5$，误差 1.065×10^{-1}。求两种方法的误差之比，并说明 RK4 为此付出的代价。

题 14　Butcher 表与阶条件

对应： 3.1.2 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

(1) 写出经典四阶 RK4 的 Butcher 表（节点 c_i、耦合系数 a_{ij}、权重 b_i）；　(2) 验证前三条阶条件 $\sum_i b_i=1$、$\sum_i b_i c_i=\frac12$、$\sum_i b_i c_i^3=\frac14$；　(3) $s=4$ 时，独立系数共几个（4 个权重、3 个节点、3 个耦合系数，节点由耦合系数的行和确定）？与 8 条阶条件相比，说明四阶格式为什么不唯一。

题 15　Kutta 3/8 规则

对应： 3.1.2 ｜ 难度： ★★★ ｜ 预计用时： 25 分钟

另一个四阶显式格式是 Kutta 的 3/8 规则：

$$k_1=f(t_n,y_n),\quad k_2=f\Big(t_n+\tfrac{h}{3},\,y_n+\tfrac{h}{3}k_1\Big),\quad k_3=f\Big(t_n+\tfrac{2h}{3},\,y_n+h\big(-\tfrac13k_1+k_2\big)\Big),$$

$$k_4=f\big(t_n+h,\,y_n+h(k_1-k_2+k_3)\big),\quad y_{n+1}=y_n+\frac{h}{8}\big(k_1+3k_2+3k_3+k_4\big).$$

(1) 写出它的节点 c_i 与权重 b_i，验证 $\sum_i b_i=1$、$\sum_i b_i c_i=\frac12$、$\sum_i b_i c_i^2=\frac13$；　(2) 用它解 $y'=-y$、$y(0)=1$、$T=1$，取 $h=1/2,1/4,\dots,1/64$，求观测收敛阶；　(3) 结合本题与题 14，解释“四阶显式方法不唯一”这句话的具体含义。

题 16　找错题：两个写错的 RK4

对应： 3.1.2 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

下面两段代码各有一处错误：

```
## 版本 A：权重写错
y_next = y + (h/4) * (k1 + k2 + k3 + k4)

## 版本 B：k3 的预测值用错
k3 = f(t + h/2, y + (h/2)*k1)     # 本该用 k2
```

(1) 版本 A 破坏了哪一条阶条件？请算出 $\sum_i b_i c_i$ 与 $\sum_i b_i c_i^2$ 来判断。　(2) 版本 B 破坏了哪个耦合系数 a_{ij}？它影响的是哪一级？　(3) 两个版本实测的观测收敛阶各是多少？与正确的 RK4 相比，$h=1/2$ 时的误差放大多少倍？

题 17　几何意义与权重来源

对应： 3.1.2 ｜ 难度： ★★☆ ｜ 预计用时： 15 分钟

取 $y'=-y$、$y(0)=1$、$h=0.5$：

(1) 写出中点试探点 M_1、修正中点 M_2、终点试探点 E_1 的坐标，指出 M_1 与 M_2 的 t 坐标是否相同；　(2) 说明第 2 级与第 3 级为什么不能合并为一级（它们在同一条竖线上，取值却不同）；　(3) 用 Simpson 求积公式的权重 $1:4:1$ 解释 RK4 权重 $1:2:2:1$ 的来源。

第三部分　3.1.3 误差、稳定性与计算成本（题 18–22）

题 18　局部阶与全局阶

对应： 3.1.3 ｜ 难度： ★☆☆ ｜ 预计用时： 10 分钟

(1) 写出显式 Euler 与经典 RK4 的局部截断误差阶和全局误差阶；　(2) 用下表计算观测阶 $p_{\text{obs}}=\log_2\dfrac{E(h)}{E(h/2)}$，填入空格，并与理论值比较：

h	Euler 误差	p_{obs}	RK4 误差	p_{obs}
$1/2$	1.179×10^{-1}	—	2.914×10^{-4}	—
$1/4$	5.147×10^{-2}	?	1.476×10^{-5}	?
$1/8$	2.427×10^{-2}	?	8.308×10^{-7}	?
$1/16$	1.181×10^{-2}	?	4.928×10^{-8}	?

(3) 为什么全局阶总比局部阶低一阶？

题 19　绝对稳定性

对应： 3.1.3 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

对测试方程 $y'=\lambda y$：

(1) 写出显式 Euler 与经典 RK4 的放大因子 $R(z)$，$z=h\lambda$；　(2) 求二者的绝对稳定区间在实负轴上的端点（数值保留三位小数）；　(3) 取 $\lambda=-10$，两种方法允许的最大步长各是多少？　(4) 取 $h=3$、$\lambda=-1$，计算两种方法的放大因子，并解释“RK4 是四阶格式，为什么这时也发散”。

题 20　隐式 Euler 与无条件稳定

对应： 3.1.3 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

(1) 写出后向 Euler 格式及其放大因子 $R(z)$；　(2) 证明：当 $\operatorname{Re}\lambda<0$ 时，对任意 $h>0$ 都有 $|R(z)|<1$；　(3) 取 $\lambda=-10$、$h=0.3$，分别计算显式与隐式 Euler 的放大因子，说明两者的差别；(4) 隐式格式的代价是什么？

题 21　计算成本

对应： 3.1.3 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

在 $T=1$、$h=1/128$ 下实测得：Euler 用 128 次右端求值，终点误差 1.442×10^{-3}；RK4 用 512 次求值，误差 1.149×10^{-11}。

(1) 求求值次数之比与误差之比；　(2) 若以误差常数近似（Euler 约 $0.236\,h$，RK4 约 $4.66\times10^{-3}h^4$），估计达到终点误差 10^{-6} 时两种方法各需多少步、多少次求值；　(3) 由此说明“RK4 每步贵 4 倍”是否意味着它总是不划算。

题 22　开放题：刚性问题的选型

对应： 3.1.3 ｜ 难度： ★★★ ｜ 预计用时： 30 分钟

某问题可近似为 $y'=\lambda y$，$\lambda=-50$，要在 $[0,20]$ 上求解。

(1) 由稳定性判据，求显式 Euler 与 RK4 的步长上限，以及在 $[0,20]$ 上至少需要的步数与右端求值次数；　(2) 仅就“保证不发散”这一条比较，哪一个更省？为什么高阶精度在这里没有带来好处？　(3) 取步长刚好在稳定上界（Euler 取 $h=0.04$、RK4 取 $h\approx0.0557$）时，两种方法的 $|R|$ 各等于多少？此时数值解与真解 e^{-50t} 的行为差多少？据此说明“稳定”与“准确”是不是一回事。　(4) 若要求终点误差 10^{-4}，仅靠本题的稳定性判据能否给出步长？还应到哪一部分去找依据？

最后给出一句话选型结论。

第四部分　3.2.1 热传导方程与定解问题（题 23–27）

题 23　傅里叶定律与方程

对应： 3.2.1 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

(1) 写出傅里叶定律

$$q=-k\frac{\partial u}{\partial x},$$

并说明各符号的名称与单位（SI）。

(2) 由“一维细杆一薄层的热量守恒”推出

$$\frac{\partial u}{\partial t}=\kappa\frac{\partial^2u}{\partial x^2}+g(x,t),\qquad \kappa=\frac{k}{\rho c},\quad g=\frac{f}{\rho c},$$

并说明 κ 的单位为什么是 m²/s。

题 24　定解条件与相容性

对应： 3.2.1 ｜ 难度： ★★☆ ｜ 预计用时： 15 分钟

考虑

$$u_t=u_{xx},\quad 0<x<1,\quad u(x,0)=u_0(x),\quad u(0,t)=0,\quad u(1,t)=1.$$

(1) 说明为什么本题需要一个初始条件和两个边界条件。

(2) 若取 $u_0(x)=0$，问初值与边界是否相容？在哪个点不相容？

(3) 当 $t\to\infty$ 时，数值解会趋于什么分布？（提示：令 $u_t=0$。）

题 25　验证精确解

对应： 3.2.1 ｜ 难度： ★☆☆ ｜ 预计用时： 10 分钟

试说明

$$u(x,t)=t\sin x$$

满足

$$u_t=u_{xx}+(t+1)\sin x,\quad 0<x<1,\ 0<t\le 1,\quad u(x,0)=0,\quad u(0,t)=0,\quad u(1,t)=t\sin 1.$$

（提示：分别核对方程的左端与右端，再核对初值与边界。）

题 26　最大值原理

对应： 3.2.1 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

叙述热方程的最大值原理，并利用它证明：若初值与边界值的绝对值都不超过 M，则解在整个区域上满足 $|u(x,t)|\le M$。

题 27　差分格式一步手算

对应： 3.2.1 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

取 $N=10$、$h=0.1$、$r=0.25$，初始温度 $U_j^0=1$（$1\le j\le 9$），$U_0^n=U_{10}^n=0$，源项 $g\equiv 1$。

(1) 写出本题的显式格式，并算出 Δt。

(2) 求 U_1^1（$j=1$，注意 $U_0^0=0$ 是边界值）与内点 U_5^1。

(3) 说明 U_1^1 为什么比 U_5^1 小。

第五部分　3.2.2 差分格式与 $Ax=b$ 的构造（题 28–37）

题 28　写出显式格式

对应： 3.2.2 ｜ 难度： ★☆☆ ｜ 预计用时： 15 分钟

对 $u_t=u_{xx}+g(x,t)$，取 N 段、M 步：

(1) 写出时间前向、空间中心差分格式；

(2) 写出初值与 Dirichlet 边界条件的离散式；

(3) 写出加权平均形式，并指出三个权重之和是多少。

题 29　构造系数矩阵

对应：　3.2.2　｜　难度：　★★☆　｜　预计用时：　20 分钟

取 $N=4$（即 U_0,U_1,U_2,U_3,U_4，内点为 U_1,U_2,U_3），$r=0.5$，隐式格式

$$-rU_{j-1}^{n+1}+(1+2r)U_j^{n+1}-rU_{j+1}^{n+1}=U_j^n,\qquad j=1,2,3,$$

两端取齐次 Dirichlet 条件。

(1) 写出 3×3 的系数矩阵 A 与右端向量 b。

(2) 若右端边界改为 $U_4^{n+1}=1$，A 是否改变？b 的最后一个分量变成什么？

题 30　从矩阵反写方程

对应：　3.2.2　｜　难度：　★★☆　｜　预计用时：　15 分钟

某同学给出的隐式矩阵为

$$A=\begin{pmatrix} 1.8 & -0.4 \\ -0.4 & 1.8 \end{pmatrix}.$$

(1) 求网格比 r。

(2) 写出与第一行对应的差分方程。

(3) 若这是一个两内点问题（$N=3$，内点为 U_1,U_2），两端的边界条件是什么？

题 31　追赶法

对应：　3.2.2　｜　难度：　★★☆　｜　预计用时：　20 分钟

用追赶法解三对角方程组

$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}\begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}=\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}.$$

(1) 写出前代、回代两步的中间量。

(2) 验证所得解满足原方程组。

题 32　（改编自教材习题三第 1 题）向前欧拉格式的误差

对应：　3.2.2　｜　难度：　★★★　｜　预计用时：　30 分钟

对

$$u_t=u_{xx}+(t+1)\sin x,\qquad 0<x<1,\ 0<t\le 1,$$

$$u(x,0)=0,\qquad 0\le x\le 1,$$

$$u(0,t)=0,\qquad u(1,t)=t\sin 1,\qquad 0<t\le 1.$$

精确解为 $u(x,t)=t\sin x$。取 $h=1/4$、$r=0.5$。

(1) 写出本题的向前欧拉格式与边界条件的离散写法。

(2) 求 Δt 与 M。

(3) 列表给出 $h=1/4,1/8,1/16$（均取 $r=0.5$）时节点 $(0.5,1)$ 处的数值解与误差；据此说明观测阶。

(4) 若把 r 从 0.5 改成 0.4（h 不变），这一点的数值解会怎样变化？用加权平均的权重解释。

题 33　（改编自教材习题三第 2 题）向后欧拉格式与网格比

对应： 3.2.2 ｜ 难度： ★★★ ｜ 预计用时： 25 分钟

题 32 的同一个问题，改用向后（隐式）欧拉格式。

(1) 写出格式，并说明为什么每一步都要求解一个三对角方程组。

(2) 取 $h=1/4$、$\tau=1/20$，求 r；再取 $\tau=1/40$，求 r。

(3) 计算节点 $(0.5,1)$ 处两种步长下的数值解与误差，说明 r 很大时（如 $h=1/8$、$\tau=1/40$，$r=1.6$）是否仍稳定。

(4) 比较题 32 与本题在该点误差的差别，说明前向与后向格式的误差方向为什么相反。

题 34　（改编自教材习题三第 3 题）Crank–Nicolson 格式

对应： 3.2.2 ｜ 难度： ★★★ ｜ 预计用时： 25 分钟

题 32 的同一个问题，改用 Crank－Nicolson 格式

$$\frac{U_j^{n+1}-U_j^n}{\Delta t}=\frac{1}{2}\Big(\frac{U_{j-1}^{n+1}-2U_j^{n+1}+U_{j+1}^{n+1}}{h^2}+\frac{U_{j-1}^{n}-2U_j^{n}+U_{j+1}^{n}}{h^2}\Big)+g_j^{n+\frac12}.$$

(1) 写出它的系数矩阵 A 的主对角元与两个次对角元（用 r 表示）。

(2) 取 $h=1/10$、$\Delta t=1/10$（$r=10$），说明格式为什么仍然稳定，而向前欧拉在此 r 下早已失稳。

(3) 说明 Crank－Nicolson 的时间精度是几阶（只需根据格式中取平均的两层判断）。

题 35　（改编自教材习题三第 6 题）带导数边界条件

对应： 3.2.2 ｜ 难度： ★★★ ｜ 预计用时： 25 分钟

考虑

$$u_t=u_{xx}+x^2(x^2-12)e^t,\qquad 0<x<1,\ 0<t\le 1,$$

$$u(x,0)=x^4,\qquad 0\le x\le 1,$$

$$u_x(0,t)=0,\qquad u_x(1,t)+u(1,t)=5e^t,\qquad 0<t\le 1.$$

精确解为 $u(x,t)=x^4e^t$。

(1) 验证 $u(x,t)=x^4e^t$ 满足 $u_x(0,t)=0$ 与 $u_x(1,t)+u(1,t)=5e^t$。

(2) 左端 $u_x(0,t)=0$ 是 Neumann 条件，右端 $u_x(1,t)+u(1,t)=5e^t$ 是 Robin 条件。写出这两个边界在网格上的离散方式（用虚拟节点 U_{-1}、U_{N+1}）。

(3) 说明用“虚拟节点 + 消元”、把边界值移到右端后，$\boldsymbol{b}$ 的首末分量各应增加什么。

题 36　（改编自教材习题三第 7 题）二维问题的五点显式格式

对应： 3.2.2 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

考虑

$$u_t=u_{xx}+u_{yy}+2(t-1)e^{t+x+y},\qquad (x,y)\in(0,1)^2,\ 0<t\le 2,$$

$$u(x,y,0)=e^{x+y},$$

$$u(0,y,t)=e^{t+y},\qquad u(1,y,t)=e^{t+y+1},$$

$$u(x,0,t)=e^{t+x},\qquad u(x,1,t)=e^{t+x+1}.$$

精确解为 $u(x,y,t)=e^{t+x+y}$。

(1) 写出五点显式格式（含源项），并写出四边边界条件的离散写法。

(2) 若取 $N=20$（每边 20 段）、$\Delta t=2.5\times10^{-4}$，求 r，判断是否满足二维稳定条件。

(3) 若要取 $\Delta t=1.0\times10^{-3}$（与一维常见的步长相当），$N=20$ 时是否稳定？

题 37　填空：矩阵的性质

对应： 3.2.2 ｜ 难度： ★☆☆ ｜ 预计用时： 10 分钟

填入恰当内容：

格式	矩阵	每行非零元个数	稳定上界
一维显式	$B=I+rL$	（a）	（b）
一维隐式	$A=I-rL$	（c）	（d）
二维隐式	$A_{2D}=I-rL_{2D}$	（e）	（f）

第六部分　3.2.3 数值解、收敛阶与稳定性（题 38–42）

题 38　观测阶

对应： 3.2.3 ｜ 难度： ★☆☆ ｜ 预计用时： 10 分钟

在模型问题 $u_t=u_{xx}$、$u(x,0)=\sin\pi x$、两端为零、$T=0.1$、$r=0.4$ 下测得：

N	10	20	40	80
最大误差	4.29×10^{-3}	1.06×10^{-3}	2.65×10^{-4}	6.62×10^{-5}

(1) 写出由相邻两次网格的最大误差计算观测阶的公式；

(2) 算出三组相邻网格的观测阶，并说明它们是否与“空间二阶”相符。

题 39　判断稳定与失稳

对应： 3.2.3 ｜ 难度： ★☆☆ ｜ 预计用时： 10 分钟

对下列每种情形，判断显式格式是否稳定（写出判据与结果）：

(1) 一维，$r=0.4$；

(2) 一维，$r=0.6$；

(3) 二维，$r=0.2$；

(4) 二维，$r=0.3$。

题 40　越过门槛会怎样

对应： 3.2.3 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

一维显式格式，初值 $\sin \pi x$ 上叠加一个 10^{-3} 的网格尺度扰动。取 $N=20$、$T=0.05$。

(1) 求 $r=0.4$ 时最危险模态的放大因子 $|1-4r|$，并估计 50 步后的放大倍数。

(2) 求 $r=0.6$ 时同样的量，并说明数值解会出现什么形状。

(3) 由此解释：为什么 $r=1/2$ 是“稳定与失稳的分界”。

题 41　网格加密的代价

对应： 3.2.3 ｜ 难度： ★★☆ ｜ 预计用时： 15 分钟

一维显式格式保持 $r=1/2$、$T=0.1$。

(1) 填写下表：

N	h	Δt	M
10	0.1	(a)	(b)
20	0.05	(c)	(d)
40	0.025	(e)	(f)

(2) N 加倍一次，M 变成原来的几倍？

(3) 若改用隐式格式，同样的 $N=40$ 下 Δt 还受这一限制吗？代价是什么？

题 42　开放题：显式还是隐式

对应： 3.2.3 ｜ 难度： ★★★ ｜ 预计用时： 25 分钟

要在 $[0,1]$ 上算 $u_t = u_{xx}$ 到 $T = 1$，取 $h = 10^{-3}$。

(1) 用显式格式，稳定条件要求 Δt 至多取多少？一共要算多少步？

(2) 换用隐式格式后，Δt 还受这个限制吗？代价是什么？

(3) 结合题 32–34 的结果，说明为什么教材在给出向前欧拉之后还要引入向后欧拉与 Crank–Nicolson。

第七部分　3.3 有限元：弱形式与 Galerkin 方法（题 43–55）

题 43　装配与自由度映射

对应： 3.3.4 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

取 $N = 4$（5 个节点、4 个单元），线性元。

(1) 写出单元 e_i（$i = 1,\dots,4$）的局部自由度到全局自由度的映射。

(2) 按装配规则手工写出整体刚度矩阵 A（5×5）。明确指出两个角上的对角元是多少，并解释为什么不是内部的 $2/h$。

(3) 若某同学写成“所有对角元都是 $2/h$”，会在哪些位置出错？这会导致解出现什么现象？

(4) 写出循环装配的伪代码，并给出一个“用另一种方式算同一个矩阵”的自检方法。为什么这个自检比“看解像不像”更可靠？

(5) 有同学把映射写成 `gdof = [i, i-1]`（顺序颠倒）。程序仍能运行并解出一个函数。请说明为什么它一定错，并给出一个能在 10 秒内发现它的检查方法。

题 44　边界处理的三种做法

对应： 3.3.4 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

沿用题 43 的 A 与右端向量 b（$-u'' = 1$ 时 $b_j = \int \varphi_j$，内部为 h、两端为 $h/2$），施加 $u(0) = u(1) = 0$。

(1) 说明为什么未施加边界条件的 A 是奇异的。给出它的零空间。

(2) 分别写出“消元法”“置 1 法”“置大数法”的具体操作。

(3) 三种做法给出的解是否相同？边界值的精度各是多少？条件数各有什么变化？

(4) 取 $N = 16$，用三种做法各解一次，报告它们与消元法解的最大差、$u_h(0)$ 与 $u_h(1)$。

(5) 有同学说：“三种做法结果一样，所以随便选一种。”请指出这个说法忽略了什么，并说明在网格加密时哪种做法会先出问题。

题 45　两个范数与收敛阶

对应： 3.3.5 ｜ 难度： ★★★ ｜ 预计用时： 30 分钟

取 $-u''=2$、$u(0)=u(1)=0$，精确解 $u=x(1-x)$。

(1) 写出 L^2 与 H^1 两个误差范数的定义，并说明为什么两者的收敛阶不同（提示：分段线性函数的导数有多准？）。

(2) 说明“一维线性元的节点值精确”这一性质，并写出它的前提条件。若右端项用低阶求积，这条性质还成立吗？

(3) 若节点值精确，则 L^2 误差由什么决定？试估计其量级与 h、$\max|u''|$ 的关系。

(4) 对 $N=4,8,16,32,64$ 计算 L^2、H^1 误差与节点最大误差，填表并给出观测阶。

N	$\|u-u_h\|_{L^2}$	阶	$\|u-u_h\|_{H^1}$	阶	节点误差
4		—		—	
…					
64					

(5) 有同学报告“有限元的收敛阶是 2”，但被追问时说不清是哪个范数。请说明这个说法为什么不够，并给出一个反例式的说明（哪个范数下是 1 阶）。

题 46　手算一次完整的有限元

对应： 3.3.3 – 3.3.4 ｜ 难度： ★★★ ｜ 预计用时： 35 分钟

取 $-u''=1$、$u(0)=u(1)=0$（精确解 $u=x(1-x)/2$），$N=4$，线性元。

(1) 手工写出整体刚度矩阵 A（5×5），指出两个角上的对角元。

(2) 写出右端向量 b（提示：$\int_0^1 \varphi_j\,\mathrm{d}x=h$ 对内部节点，两端为 $h/2$）。

(3) 施加零 Dirichlet 边界，写出 3×3 的内部方程组并解出 3 个内部节点值。

(4) 把 (3) 的结果与精确解在节点处比较。你能预测节点值是否精确吗？用正文的结论解释。

(5) 用程序复核 (1) – (4) 的全部数值，并报告残差 $\|Au-b\|$（在施加边界条件之后）。

(6) 有同学算出 $u_h(0.5)=0.13$（精确值 0.125），并解释为“有限元的离散误差”。请指出这个误差量级（0.005）是否正常，以及最可能的原因是什么。

题 47　弱形式与边界项

对应： 3.3.2 – 3.3.4 ｜ 难度： ★★☆ ｜ 预计用时： 15 分钟

考虑 $-u''+u=f$，$0<x<1$，$u(0)=0$，$u'(1)=\beta$（β 给定）。

(1) 取检验函数 v 满足什么边界条件？写出弱形式，明确 $x=1$ 处的边界项去了哪里。

(2) 若把 $u'(1)=\beta$ 当作本质边界 $u(1)=\beta$ 施加，方程和边界条件各发生了什么变化？

(3) 线性元的单元矩阵多出质量项 $\frac{h}{6}\begin{pmatrix}2&1\\1&2\end{pmatrix}$。写出完整的 2×2 单元矩阵。

(4) 在装配后的右端向量里，β 应该加到哪个分量？为什么？

(5) 有同学说“边界条件都是把对应行和列划掉”。这个说法对本题的 $x=1$ 端成立吗？说明理由。

题 48　快速题：对角元为什么不同

对应： 3.3.4 ｜ 难度： ★☆☆ ｜ 预计用时： 3 分钟

写出整体刚度矩阵内部节点与端点的对角元，并说明为什么不同。

题 49　快速题：稀疏度

对应： 3.3.4 ｜ 难度： ★☆☆ ｜ 预计用时： 3 分钟

$N=4$ 时 A 是几阶矩阵？非零元大约几个？

题 50　快速题：未加边界的奇异性

对应： 3.3.4 ｜ 难度： ★☆☆ ｜ 预计用时： 3 分钟

为什么未施加边界条件的 A 奇异？写出它的零空间。

题 51　快速题：两个范数

对应： 3.3.5 ｜ 难度： ★☆☆ ｜ 预计用时： 3 分钟

L^2 与 H^1 哪个阶高？为什么？

题 52　快速题：置大数法的代价

对应： 3.3.4 ｜ 难度： ★☆☆ ｜ 预计用时： 3 分钟

置大数法的精度代价是什么？

题 53　快速题：节点值精确的前提

对应： 3.3.5 ｜ 难度： ★☆☆ ｜ 预计用时： 3 分钟

“一维线性元节点值精确”这条性质的前提是什么？

题 54　找错题：装配漏掉非对角元

对应： 3.3.4 ｜ 难度： ★★☆ ｜ 预计用时： 10 分钟

下面这段装配代码声称“装配出了整体刚度矩阵”，但解出来的位移整体偏小、且随 N 增大不收敛到真解。请找出原因。

```
A = np.zeros((N + 1, N + 1))
for i in range(1, N + 1):
    h = 1.0 / N
    A[i-1, i-1] += 1.0 / h
    A[i,   i  ] += 1.0 / h      # 只加了对角元
```

题 55　开放题：五分钟查装配

对应： 3.3.4 ｜ 难度： ★★★ ｜ 预计用时： 20 分钟

有人交给你一段一维有限元的装配代码，你只有 5 分钟。怎么判断它装配得对不对？请写出不超过 150 字的检查方案，说明每个检查能排除什么错误。

> 选读（不计入本节学时）
>
> - 二维装配：三角形单元的局部矩阵是 3×3，装配时用单元-节点表（connectivity table）。与一维相比，唯一的新问题是“稀疏性”——必须用稀疏存储，否则内存会爆。
> - 非齐次 Dirichlet 边界：$u(0)=g$ 时需要把边界值移到右端向量，即 $b_i \mathrel{-}= A_{i0}g$。这一步漏掉会让解在边界附近整体偏移。

第八部分　3.4.1 学习右端函数（题 56–63）

题 56　Lipschitz 条件与命题 1

对应： 3.4.1 ｜ 难度： ★★★ ｜ 预计用时： 30 分钟

设三个可能的右端函数

$$f_1(u)=\tanh u,\qquad f_2(u)=\sqrt{|u|},\qquad f_3(u)=-\theta u\quad(\theta>0).$$

(1) 在带形 $|u|\le R$ 上分别求它们的 Lipschitz 常数，并指出哪个是全局 Lipschitz、哪个只是局部 Lipschitz。

(2) 指出哪一个在含 $u=0$ 的任何区域上都不满足 Lipschitz 条件，用差商说明。

(3) 对初值问题 $u'=\sqrt{|u|}$、$u(0)=0$，验证 $u(t)\equiv 0$ 与 $u(t)=t^2/4\ (t\ge 0)$ 都是经典解，并指出命题 1 的哪一条前提被破坏。

(4) 填下表（f_2 在原点处的差商），并说明它与 (2) 的关系：

u	10^{-2}	10^{-4}	10^{-6}
$\dfrac{\lvert f_2(u)-f_2(0)\rvert}{u}$	?	?	?

(5) 陷阱题　判断：“f_θ 是神经网络，处处连续，所以式 (2) 的解一定唯一。”若错误，给出反例，并说明唯一性真正需要什么条件。

题 57　Picard 迭代与 ResNet 的层层堆叠

对应： 3.4.1 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

对 $u'=-u$、$u(0)=1$：

(1) 从 $u^{(0)}\equiv 1$ 出发，算出 $u^{(1)},u^{(2)},u^{(3)}$，写出 $u^{(m)}$ 的一般形式，并说明它是 e^{-t} 的哪一类展开。

(2) 把 $[0,1]$ 分成 M 层，每层取残差块 $F(u)=-hu$（$h=1/M$），即

$$u_{k+1}=u_k+h\,(-u_k),\qquad k=0,1,\dots,M-1.$$

写出 u_M 的闭式，并填下表：

M	10	100	1000	10^4
$u_M=(1-1/M)^M$	?	?	?	?
与 e^{-1} 之差	?	?	?	?
误差 $/h$（$h=1/M$）	?	?	?	?

(3) 求 $\lim_{M\to\infty}u_M$，并证明误差满足 $\lvert u_M-e^{-1}\rvert\approx e^{-1}h/2$（$h=1/M$）；用 (2) 的最后一列核对这个常数。

(4) 比较两种“取极限”：Picard 迭代的 $m\to\infty$ 与 ResNet 的 $M\to\infty$，极限对象各是什么？各自的代价是什么？

(5) 陷阱题　判断：“ResNet 层数越多越准，所以只要无限加深，就等于解出了连续的 ODE。”

题 58　流映射、半群性质与可逆性

对应： 3.4.1 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

对 $f(u)=-\theta u$，取 $\theta=1$、$u(0)=u_0$：

(1) 写出流映射 $\varphi(t,u_0)$，并用 $s=0.3$、$t=0.7$、$u_0=1$ 的具体数值验证半群性质 $\varphi(t+s,u_0)=\varphi\big(t,\varphi(s,u_0)\big)$。（可保留 6 位小数。）

(2) 说明 $\varphi(t,\cdot)$ 可逆，给出逆映射。取 Euler 格式、$h=0.5$，从 $u_0=1$ 走 4 步，再按逆映射“除回去”，说明往返为什么恰好闭合。

(3) 对 1 维情形，密度沿轨迹满足 $\dfrac{\mathrm{d}}{\mathrm{d}t}\log p_t\big(u(t)\big)=-\dfrac{\partial f}{\partial u}$。取 $p_0=\mathcal{N}(0,1)$、$t=1$、$x=0.5$，分别用 “$\log p_1(x)$” 与 “$\log p_0(e^t x)+t$” 两条路各算一遍；再说明 $\mathrm{Var}(u_1)=e^{-2}$，并核对 $\int p_1=1$。

(4) 陷阱题　判断：“既然流映射可逆，反向积分就一定能精确还原真实的初始状态。”请用 $\theta=1$、$h=3$、走两步的 Euler 结果说明问题出在哪里。

题 59　伴随法的内存账

对应： 3.4.1 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

设状态维数 $d=256$，前向共 $M=10^5$ 步，单精度（每数 4 字节）：

(1) 写出离散反传需要保存的浮点数个数，并算出字节数（换成 MiB）。

(2) 写出伴随法的内存（$O(d)$）与分段检查点的内存（$O(\sqrt{M}\,d)$，按最优分段数 $k=\sqrt{M}$ 计），算出字节数，并给出二者相对离散反传的比值。

(3) 若可用显存只有 1 GiB，在 $d=1024$ 时离散反传最多能走多少步？伴随法受不受这一限制？

(4) 说明伴随法给出的梯度与离散反传给出的梯度为什么不严格相等，差什么量级；这对“训练到一半换求解器”意味着什么。

(5) 陷阱题　判断：“伴随法用两倍时间换常数内存，所以内存已经不是瓶颈，d 可以随便加大。”

题 60　快速题：定义 1 与参数化向量场

对应： 3.4.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

写出定义 1（参数化向量场与轨迹），并说明把 f 换成 f_θ 带来的一个好消息与一个坏消息。

题 61　快速题：网络 Lipschitz 常数

对应： 3.4.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

(1) $f(u)=3\tanh(2u+1)$，求整网的 Lipschitz 常数上界。

(2) 某两层网络的两层权重谱范数为 $s_1=2$、$s_2=0.5$，激活函数取 $\tanh$，求 L 的上界。

(3) 若每层谱范数都是 1.2、共 20 层，整网上界是多少？由此说明“深层网络对输入极敏感”这句话的来源。

题 62　快速题：连续依赖与 Lorenz

对应： 3.4.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

(1) 设两个初值之差为 10^{-3}、Lipschitz 常数为 $L=10$，用 3.4 正文 1.2 节的连续依赖估计求 $t=1$ 与 $t=0.1$ 处的偏差上界。

(2) Lorenz 的例子中，初值差约 10^{-3}，两周（14 天）内被放大到 $O(1)$。反推这需要多大的 L（按 e^{Lt} 计）。

题 63　快速题：伴随法的代价

对应： 3.4.1 ｜ 难度： ★☆☆ ｜ 预计用时： 5 分钟

叙述命题 2（伴随法的代价），并回答：为什么它的内存与时间步数无关？分段检查点的 $O(\sqrt{M})$ 是怎么来的（设分 k 段，每段 M/k 步）？

第九部分　3.4.2 轨迹拟合与方程残差（题 64–68）

题 64　残差、推论 1 与“配点为零”的假象

对应： 3.4.2 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

取模型 $u_\theta(t)=1+t$、方程 $u'=u$（即正文小实验 1）：

(1) 算出残差 $r_\theta(t)$，并在配点集 $\{0\}$、$\{0,0.5,1\}$、$\{0,0.25,0.5,0.75,1\}$ 上分别算残差损失（均方，三项损失中的第一项）。

(2) 用推论 1（$L=1$）写出误差界，算出 $t=1$ 处的值，并与真实误差 $|2-e|$ 比较；说明这个算例上界为什么恰好取等号。

(3) 算出放大因子 $(e^{Lt}-1)/L$ 在 $t=1,2,3$ 处的值；若 $|r_\theta|\equiv 0.01$，给出三个时刻的误差上界。

(4) 填下表，并说明配点数从 1 增到 5 时，残差损失与 $t=1$ 处误差各自怎样变化：

配点集	$\{0\}$	$\{0,0.5,1\}$	$\{0,0.25,0.5,0.75,1\}$
残差损失	?	?	?
$t=1$ 处误差	?	?	?

(5) 陷阱题　判断：“残差处处小于 1%，所以解的误差也小于 1%。”

题 65　两条路线与量纲

对应： 3.4.2 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

(1) 写出式 (3)（轨迹拟合）与式 (9)（PINN）两种损失，指出各自“学的是什么”，以及各自需要什么样的数据。

(2) 为下列三个场景各选一条路线并说明理由：(a) 只在几个位置埋了热电偶，热传导方程已知；(b) 手上只有一条 S 形曲线，方程未知；(c) 方程已知且数据较多。

(3) 量纲实验：方程含扩散项系数 $\alpha=10^{-6}$ 与对流项 $O(1)$，若 $|r_\theta|\sim 10^{-3}$、数据项的偏差 ~ 0.1，分别算出两项损失的量级与“两边拉平”所需的 λ_1；由此说明“先无量纲化”为什么是最便宜的一步。

(4) 说明强形式残差、弱形式残差与 Deep Ritz 三种写法各自对光滑性的要求与代价。

(5) 陷阱题　判断：“PINN 有三项损失，只要把 λ_1,λ_2 调好，就一定能收敛。”

题 66　权重的代价

对应： 3.4.2 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

(1) 用 NTK 的语言解释：残差项与边界项的核尺度差几个数量级时，为什么优化器会“只顾一头”。

(2) 梯度范数平衡：设 $\|\nabla_\theta L_r\| = 1$、$\|\nabla_\theta L_b\| = 10^2$、$\alpha = 0.9$，从 $\lambda_0 = 1$ 起按 $\lambda \leftarrow \lambda\big(\|\nabla L_r\|/\|\nabla L_b\|\big)^{\alpha}$ 迭代，填表：

k	0	1	2	3
λ_k	1	?	?	?

(3) 不确定性加权：证明 $L_i/\sigma_i^2 + \log\sigma_i^2$ 关于 σ_i 的最优解是 $\sigma_i^2 = L_i$；对 $L_r = 0.01$、$L_b = 1$ 给出两项权重之比。

(4) 三条对策（自适应权重、硬约束、分阶段训练）各消除哪一部分问题？

(5) 陷阱题　判断：“λ 只是超参数，调不好最多是慢一点，不会影响结论。”

题 67　找错题：两个写坏的残差损失

对应： 3.4.2 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

下面两段代码各有一处错误：

```
## 版本 A：配点只取一个
pts = np.array([0.0])
r = dudt(pts) - f(pts, u(pts))
L = np.mean(r**2)

## 版本 B：用前向差商代替自动微分（张量为 float32）
h = 1e-8
r = (u(t + h) - u(t)) / h - f(t, u(t))
```

(1) 设 $u_\theta(t) = 1 + t$、$f(t,u) = u$。版本 A 的残差损失是多少？$t = 1$ 处的误差是多少？这说明什么？

(2) 版本 B 中 $u(t) = e^{-t}$，要算的是 $u'(1) = -0.3679$。float32 下 h 取 10^{-4}、10^{-6}、10^{-8} 时，差商各给出什么值？解释 h 越小反而越糟的原因，并给出可用步长的量级。

(3) 两个版本分别属于哪一类错误（错误定义 vs 数值实现）？各给出最小改动。

题 68　开放题：算子学习要不要做

对应： 3.4.2 ｜ 难度： ★★★ ｜ 预计用时： 25 分钟

某热传导问题有 10000 组参数。单次高精度数值求解需 10 s；训练一个 DeepONet 需 2×10^4 s，训练好后单次推理 10^{-3} s。

(1) 算“逐个求解”与“训练一个算子网络”在 10000 组上的总成本，求加速比与盈亏平衡的参数组数 N。

(2) 算子学习相对 PINN 的优势是什么？前提是什么？

(3) PDEBench 这类基准解决了什么、没有解决什么？

(4) 若只允许做一件事来提高“误差 10^{-3}”这句话的可信度，你选什么？

最后给出一句话选型结论。

第十部分　3.4.3 求解器误差与独立轨迹验证（题 69–73）

题 69　误差分解

对应： 3.4.3 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

用 §1.4 - §3.3 的算例：观测点 $(0.5, 0.6065)$，真解 e^{-t}，用 Euler($h=0.5$) 反解出 $\hat{\theta}=0.787$（真值 1），真值 $u^\star(2)=0.1353$。

(1) 写出命题 4 的三项，说明哪一项只能靠“换一套装置”测出来。

(2) 用带符号的分解（以 $\hat{u}$ 偏大为正）

$$\hat{u}(T)-u^\star(T)=\underbrace{\left(\hat{u}-u_\theta\right)}_{\text{离散}}+\underbrace{\left(u_\theta-u_{\theta^\star}\right)}_{\text{优化}}+\underbrace{\left(u_{\theta^\star}-u^\star\right)}_{\text{逼近}}$$

填下表，$T=2$，其中 $u_\theta(2)=e^{-2\hat{\theta}}=0.2072$，逼近误差为 0（模型类 $f_\theta=-\theta u$ 含真解）：

复算方式	$\hat{u}(2)$	离散误差	优化误差	总误差
Euler $h=0.5$	?	?	?	?
Euler $h=0.25$	?	?	?	?
Euler $h=0.125$	?	?	?	?
RK4 $h=0.5$	?	?	?	?
RK4 $h=0.25$	?	?	?	?

(3) 表中 Euler $h=0.5$ 一行总误差几乎为 0，解释原因；并说明命题 4 用绝对值写出的三项和为什么只是一个上界。

(4) 陷阱题　判断：“误差分解只是把一个数拆成几项，对实际做法没有指导意义。”

题 70　换一套装置漂多少

对应： 3.4.3 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

固定 $\hat{\theta}=0.787$（$f_\theta=-\hat{\theta}u$、$u(0)=1$），真值 $u^\star(2)=0.135335$：

(1) 算出下列复算方式在 $t=2$ 处的值，填表：

复算方式	h	$u(2)$	相对真值的偏差
Euler	0.5（训练时）	?	?

复算方式			相对真值的偏差
Euler	0.25	?	?
Euler	0.125	?	?
Euler	0.0625	?	?
RK4	0.5	?	?
RK4	0.25	?	?

(2) 计算“换步长漂移量” $\left|u(2;h)-u(2;h/2)\right|$（Euler，$h=0.5\to0.25\to0.125\to0.0625$），说明它随 h 怎样变化。

(3) 解释：为什么训练时那一套装置恰好落回真值，而换任何一套都漂向 0.2072？

(4) 如果只允许报告一个数来反映“离散误差被吸收进参数”，你报告哪个？说明理由。

题 71　三换协议：偏差藏在哪里

对应： 3.4.3 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

仍用 $f_\theta=-\hat\theta u$、$\hat\theta=0.787$、真值 $\theta^\star=1$：

(1) 说明三换（换初值、换步长、换区间）各换什么、能发现什么、发现不了什么。

(2) 换初值：取 $u_0=0.02,\,0.05,\,0.2$，在 $t=2$ 处填表：

u_0	0.02	0.05	0.2
模型的解析解 $u_0e^{-\hat\theta t}$	?	?	?
真解 u_0e^{-t}	?	?	?
相对偏差	?	?	?
用训练装置 Euler($h=0.5$) 复算的相对偏差	?	?	?

并判断：这一步能否发现 §1.4 那个 53% 的偏差？

(3) 换步长：用题 70 的表说明这一步能发现什么。

(4) 换区间：算 $t=2.4$（外推 20%）与 $t=4$ 处的相对偏差。这对“外推能力”意味着什么？

(5) 陷阱题　判断：“三换全部通过，就说明模型学到的是向量场。”

题 72　快速题：离散误差界

对应： 3.4.3 ｜ 难度： ★☆☆ ｜ 预计用时： 10 分钟

(1) 写出 Euler 的全局误差界（3.4 正文 3.1 节末式）

$$\|\hat{u}(T)-u_{\theta^*}(T)\| \le \frac{h}{2}\max_{[0,T]}|u''|\cdot\frac{e^{LT}-1}{L}.$$

(2) 对 $u'=-u$、$u(0)=1$、$T=2$、$\max|u''|=1$、$L=1$，取 $h=0.5$ 与 $h=0.25$，算实际误差与界，求“界/实际误差”的倍数；说明这个界为什么“松但不废”。

(3) 取 $L=50$、$T=1$、$h=0.01$、$\max|u''|=1$，算这个界。由此说明刚性方程为什么“步长必须小得离谱”。

题 73　找错题：两个写坏的验证脚本

对应： 3.4.3 ｜ 难度： ★★☆ ｜ 预计用时： 20 分钟

下面两段验证脚本各有一处错误：

```
## 版本 A：换步长验证
ts = np.arange(0, T, h)
u  = integrate(theta, u0=1.0, ts)
print("终点误差", abs(u[-1] - true(T)))

## 版本 B：换初值验证
u2 = integrate(theta, u0=2.0, ts)
print("最大误差", max(abs(u2 - true(ts))))
```

(1) 版本 A：取 $T=2$。$h=0.5$ 与 $h=0.25$ 时 `ts` 的长度与末点各是多少？按 $\hat{\theta}=0.787$、Euler 格式算出脚本报出的“终点误差”，并说明这个检查会得出什么错误结论。

(2) 版本 B：`true()` 仍取 $u(0)=1$ 的解 e^{-t}。$u_0=2$、$t=2$ 时脚本报出的相对偏差是多少？正确的值是多少？

(3) 各给出最小改动，并说明这两个错误为什么“程序不报错、结论全错”。

第十一部分　3.4.4 趣味编程（题 74–77）

题 74　造数据与灵敏度

对应： 3.4.4 ｜ 难度： ★★☆ ｜ 预计用时： 25 分钟

取式 (11)：$K=1$、$u_0=0.05$、$r=0.8$，在 $t=0,0.1,\dots,2$ 上采样 21 点，加 1% 的相对噪声作为观测。

(1) 验证式 (11) 是方程 $u'=ru(1-u/K)$ 的解：取 $t=0.5$ 与 $t=1.5$，用差商算出 u'，与 $ru(1-u/K)$ 比较。

(2) 填表（5 个点），并给出每个点的噪声标准差 $\sigma=0.01u$：

t	0	0.5	1.0	1.5	2.0
$u(t)$	?	?	?	?	?
$\sigma=0.01u$	?	?	?	?	?

(3) 算出 $\partial u/\partial r$、$\partial u/\partial K$、$\partial u/\partial u_0$ 在 $[0,2]$ 上的最大值与出现位置，并求前两者之比。

(4) 解释：同样是 1% 的噪声，为什么对 K 的伤害约为对 r 的十倍？

(5) 陷阱题　判断：“有 21 个点、每点只有 1% 的噪声，所以三个参数都能反演到 1%。”

题 75　反演、可辨识性与窗口

对应： 3.4.4 ｜ 难度： ★★☆ ｜ 预计用时： 30 分钟

正文的反演结果为 $\hat r=0.810$、$\hat K=0.922$、$\hat u_0=0.0497$（真值 0.8、1、0.05）：

(1) 算出三个参数的相对误差，并与题 74(3) 的灵敏度比值对照，说明“K 反演不准”为什么不是优化没做好。

(2) 填表（拟合曲线与真曲线的相对偏差）：

t	0.5	1	2	4	6	8
拟合曲线 $(\hat r,\hat K,\hat u_0)$	?	?	?	?	?	?
真曲线 $(0.8,\,1,\,0.05)$	?	?	?	?	?	?
相对偏差	?	?	?	?	?	?

(3) 把采样窗口扩到 $[0,6]$ 上重算灵敏度：$\max\partial u/\partial r$ 与 $\max\partial u/\partial K$ 各是多少？比值比题 74(3) 改善了多少倍？

(4) 若只允许改一个条件（采样窗口、噪声水平、采样密度），改哪个最值钱？给出理由。

(5) 陷阱题　判断：“$\hat K$ 偏了 8%，说明优化没收敛，多训几轮就好了。”

题 76　快速题：logistic 的三件事

对应： 3.4.4 ｜ 难度： ★☆☆ ｜ 预计用时： 10 分钟

(1) 验证 $u(t)=\dfrac{K}{1+\left(\frac{K}{u_0}-1\right)e^{-rt}}$ 满足 Verhulst 方程 $u'=ru\left(1-u/K\right)$。

(2) 求 $u(t^*)=K/2$ 的 t^*（本题参数下），并说明这个数落在采样区间 $[0,2]$ 之外意味着什么。

(3) 判断平衡点 $u=0$ 与 $u=K$ 的稳定性（在 $u=K$ 处作线性化）。

题 77　开放题：一条真实生长曲线的完整实验设计

对应： 3.4.4 ｜ 难度： ★★★ ｜ 预计用时： 30 分钟

给你 21 个带 1% 噪声的观测点，要求交出“一条可外推的规律”，而不是“第 22 个点”：

(1) 列出必须分别报告的四个误差来源（逼近、离散、优化、数据噪声），并说明各自怎么估。

(2) 写出三换的具体做法与判定阈值（差多少算“漂”）。

(3) 参数不确定度怎么报？给出“灵敏度 + 后验”两种写法。

(4) 若把采样窗口从 $[0,2]$ 扩到 $[0,6]$，预期 $\hat{K}$ 的误差怎样变化？为什么？

最后给出一句话结论：本节能保证什么，不能保证什么。

第 03 章　离散的世界 • 习题参考解答（上）

> 本文件：A1-G3-习题参考解答.md（G3 = 第 03 章 A1 编写组；全章合并稿 • 上半册） 范围：覆盖 3.1 – 3.2（题 1 – 42，第一～第六部分）；3.3 – 3.4（题 43 – 77，第七～第十一部分）见 A1-G3-习题参考解答-3.3-3.4.md。 配套文件：A1-G3-习题.md、A1-G3-习题参考解答-3.3-3.4.md、A1-G3-正文.md、A1-G3-实验.ipynb 版本：全章合并稿（2026-10-26 由 3.1 – 3.4 分节解答合并；与习题逐题对应，版式照 3.1 节）。因 GitHub 单页数学公式数量上限（约 2000 个渲染单元），全章解答分为上、下两册，题号仍全章连续。 状态：解答由本组编写并逐题核对；题号与习题文件一一对应。本文件不纳入学生版教材目录。 数值来源：3.1 的题 6 – 10、13、15、16、18 – 22（算例 $y'=-y$、$y(0)=1$、$T=1$）与 3.2 的题 32、33、35、38、40、41（算例 $u_t=u_{xx}+(t+1)\sin x$）由配套程序实算后写入。其余数字为手算，过程已写出，便于复核。 引用约定：解答中引用的定理、命题、推论、定义、公式、图、表编号，均指该题所属小节正文的编号。解答中的“表 1”是本册自编的表号。

第一部分　3.1.1 初值问题与 Euler 方法（题 1–12）

题 1　Lipschitz 条件与经典解

(1) 三个右端的 Lipschitz 常数

（a）$f_1(t,y)=t^2+y^2$：$\dfrac{\partial f_1}{\partial y}=2y$，在 $|y|\le R$ 上 $\left|\dfrac{\partial f_1}{\partial y}\right|\le 2R$。由命题 1，$f_1$ 在带形 $|y|\le R$ 上满足 Lipschitz 条件，常数 $L=2R$。

（b）$f_2(t,y)=\sqrt{|y|}$：在 $|y|\le R$ 上不满足 Lipschitz 条件，见 (2)。

（c）$f_3(t,y)=e^{-y}$：$\dfrac{\partial f_3}{\partial y}=-e^{-y}$，在 $|y|\le R$ 上 $\left|\dfrac{\partial f_3}{\partial y}\right|\le e^{R}$。故 常数 $L=e^{R}$。

注意 (a) 与 (c) 的差别：f_1 的常数随 R 线性增长，f_3 的随 R 指数增长，但两者在固定的带形上都是有限常数。这正是“局部 Lipschitz”的意思。

(2) 哪个不是 Lipschitz

$f_2(t,y)=\sqrt{|y|}$。取 $u>0$、$v=0$，则

$$\frac{|f_2(t,u)-f_2(t,0)|}{|u-0|}=\frac{\sqrt{u}}{u}=\frac{1}{\sqrt{u}}\to+\infty\qquad(u\to 0^{+}).$$

差商无界，故不存在有限的 L 使 $|f_2(t,u)-f_2(t,0)|\le L|u|$ 对所有足够小的 u 成立。由于 $y=0$ 在任何含原点的区域里，所以 f_2 在含 $y=0$ 的任何区域上都不满足 Lipschitz 条件。

(3) 经典解与解的唯一性

定义 1 回顾： 若 $y\in C^1(I)$，对一切 $t\in I$ 有 $\big(t,y(t)\big)\in D$，且逐点满足式 (1)、初值条件，则称 y 是初值问题的经典解。

验证两个解：

• $y(t)\equiv 0$：$y'=0$，$\sqrt{|y|}=0$，两者相等；$y(0)=0$。故是解。　• $y(t)=t^2/4$（$t\ge 0$）：$y'=t/2$，而 $\sqrt{|y|}=\sqrt{t^2/4}=t/2$，两者相等；$y(0)=0$。故也是解。

同一个初值问题有两个经典解，所以解不唯一。这说明解的存在可以只靠 f 连续（Peano 定理），但唯一性需要更强的条件——Lipschitz 条件（定理 1）。

(4) 差商表

$f_2(t,u)-f_2(t,0)=\sqrt{u}$，故差商为 $1/\sqrt{u}$：

表 1　$f_2(t,y)=\sqrt{|y|}$ 在原点处的差商

u	10^{-2}	10^{-4}	10^{-6}
$\lvert f_2(t,u)-f_2(t,0)\rvert/u$	10	100	1000

每缩小 100 倍，差商放大 10 倍，无界增长。这与 (2) 的分析一致：“差商无界”就是“不存在有限 Lipschitz 常数”。

(5) 陷阱：连续不等于唯一

这句话错在把“存在”当成了“唯一”。f 连续只保证至少有一个解；唯一性还需要 f 关于 y 满足 Lipschitz 条件。

反例就是 (3) 中的 $y'=\sqrt{|y|}$、$y(0)=0$：f 在 $\mathbb{R}^2$ 上连续，但解不唯一。另一条思路：若 f 满足 Lipschitz，则定理 1 保证唯一；$\sqrt{|y|}$ 恰好不满足，所以定理 1 的假设不是多余的。

常见错误： 用“f 可微”代替“Lipschitz”，或只验证了 f 在 $y\neq 0$ 处 Lipschitz 就下结论。注意 (2) 的关键是 $y=0$ 这一点。

题 2　Picard 迭代

(1) 前三步迭代

$y^{(0)}=1$。

$y^{(1)}(t)=1+\int_0^t 1\,ds=1+t$。

$y^{(2)}(t)=1+\int_0^t(1+s)\,ds=1+t+\dfrac{t^2}{2}$。

$y^{(3)}(t)=1+\int_0^t\Big(1+s+\dfrac{s^2}{2}\Big)ds=1+t+\dfrac{t^2}{2}+\dfrac{t^3}{6}$。

一般地 $y^{(m)}(t)=\sum_{j=0}^{m}\dfrac{t^j}{j!}$，正是 e^t 的 Taylor 展开的前 $m+1$ 项。

(2) M、L 与估计式的验证

取矩形 $|t|\le T$、$|y-1|\le 1$。在该矩形上 $|f(t,y)|=|y|\le 2$，故可取 $M=2$；$|\partial f/\partial y|=1$，故 $L=1$。

逐项验证定理 1 证明中的估计式 $|y^{(m+1)}(t)-y^{(m)}(t)|\le \dfrac{ML^m}{(m+1)!}|t|^{m+1}$：

• $m=0$：$|y^{(1)}-y^{(0)}|=|t|$，右端 $=\dfrac{2\cdot 1^0}{1!}|t|=2|t|$，成立（左端只有右端的一半）。 • $m=1$：$|y^{(2)}-y^{(1)}|=\dfrac{t^2}{2}$，右端 $=\dfrac{2\cdot 1}{2!}|t|^2=t^2$，成立。 • $m=2$：$|y^{(3)}-y^{(2)}|=\dfrac{|t|^3}{6}$，右端 $=\dfrac{2}{3!}|t|^3=\dfrac{|t|^3}{3}$，成立。

可见估计式给出的是上界，常数并不紧——这不影响收敛性，因为在固定区间上 M 是有限的。

(3) 为什么一致收敛

由 (2) 的估计，在 $[0,T]$ 上

$$\|y^{(m+1)}-y^{(m)}\|_\infty\le \frac{ML^m}{(m+1)!}T^{m+1},$$

右端是收敛级数（指数级数）的通项，故 $\sum_m \|y^{(m+1)}-y^{(m)}\|_\infty<\infty$。于是 $\{y^{(m)}\}$ 是 $C[0,T]$ 中的 Cauchy 列；$C[0,T]$ 在 sup 范数下完备，故存在连续函数 y 使 $\|y^{(m)}-y\|_\infty\to 0$。这一步只用到“级数收敛”与“空间完备”，其余（极限满足积分方程、唯一性）见定理 1 证明的相应步骤。

(4) $t=1$ 处的迭代值

$y^{(m)}(1)=\sum_{j=0}^{m}1/j!$：

m	1	2	3	4	5	6	7	8
$y^{(m)}(1)$	2	2.5	2.6667	2.7083	2.71667	2.71806	2.718254	2.7182788

$e=2.7182818\ldots$，迭代值单调上升并逼近 e。这正是“$y^{(m)}$ 是 Taylor 部分和”的直接结果。

(5) 陷阱：Picard 迭代不是数值解法

两个问题：

第一，每一步要算的是积分 $\int f\big(s,y^{(m)}(s)\big)ds$，而且被积函数里含上一次迭代的整条曲线。对一般的 f，这个积分没有解析表达式，也就无法“取 $m=100$”来算；它存在的前提是能对函数做解析运算。

第二，Euler 方法只需要 f 在节点上的取值，不需要积分、不需要整条曲线。两者的信息需求完全不同：Picard 迭代是证明工具（构造存在性），Euler 是计算工具。

常见错误： 把 m（迭代次数）与 n（时间步数）混为一谈。

题 3　Gronwall 与连续依赖

(1) 精确解与精确差

$y(t)=e^{-t}$，$z(t)=1.01\,e^{-t}$，故

$$|y(t)-z(t)|=0.01\,e^{-t}.$$

它随 t 单调下降：初值差 0.01 被系统自身的衰减压小了。

(2) 定理 2 的界

$|y_0 - z_0| = 0.01$，$L = 1$，由式 (5)：

$$|y(t) - z(t)| \le 0.01\, e^{t}.$$

在 $t = 10$ 处，$0.01\, e^{10} = 0.01 \times 22026.47 = 220.26$。

精确值：$0.01\, e^{-10} = 0.01 \times 4.540 \times 10^{-5} = 4.54 \times 10^{-7}$。

两者相差 $e^{20} \approx 4.85 \times 10^{8}$ 倍，即八个多数量级。

(3) 界为什么松

定理 2 的证明只用了 Lipschitz 条件 $|f(t,u) - f(t,v)| \le L|u - v|$，它刻画的是“最坏情况下扰动被放大”。对 $y' = -y$，真实情形是扰动被缩小（e^{-t}），而 Lipschitz 条件对“放大还是缩小”不作区分——它只知道“变化率不超过 L 倍”。

换句话说，式 (5) 是对所有满足 Lipschitz 条件的方程都成立的统一上界，必然要用最坏情形。若把 $L = 1$ 换成带符号的信息（f 关于 y 单调递减），界可以收紧；这属于更细的分析，本节不做。

取区间 $[0, 1]$：界为 $0.01\, e^{1} = 0.02718$；精确差为 $0.01\, e^{-1} = 0.003679$。此时界只松了约 7.4 倍。

(4) 两条曲线

把 t 取 $0, 2, \ldots, 10$：

t	0	2	4	6	8	10
精确差 $0.01e^{-t}$	1.0×10^{-2}	1.35×10^{-3}	1.83×10^{-4}	2.48×10^{-5}	3.35×10^{-6}	4.54×10^{-7}
定理 2 的界 $0.01e^{t}$	1.0×10^{-2}	7.39×10^{-2}	0.546	4.03	29.8	220.3

一个指数下降、一个指数上升，只在 $t = 0$ 处相等。这提醒：上界不是预言。

(5) 陷阱：把上界当成实际误差

错在把“上界”当成了“实际差”。适定性要求的是：初值差 $\to 0$ 时解之差 $\to 0$。这里已经满足：$|y - z| = 0.01e^{-t} \le 0.01$，初值差越小，解之差越小。

(2) 的 220 只是“允许的最大值”，实际只有 4.5×10^{-7}。界松不等于问题不适定，恰恰相反，这个问题非常稳定（扰动还在衰减）。

常见错误： 用“界很大”推断“问题病态”；或忘记 L 与区间长度都会进入界。

题 4　解的存在区间与整体延拓

(1) 定理 1 给出的区间

在矩形 $|t| \le 1$、$|y| \le 1$ 上，$|f(t,y)| = |1 + y^2| \le 2$，故 $M = 2$；又 $a = 1$、$b = 1$，于是

$$\delta = \min\{a,\ b/M\} = \min\{1,\ 1/2\} = \frac{1}{2}.$$

解在 $I = [-\frac{1}{2}, \frac{1}{2}]$ 上存在唯一。

(2) 为什么只能是局部的

精确解 $y = \tan t$ 在 $t \to \pi/2^-$ 时无界（$\pi/2 \approx 1.571$）。定理 1 给的是 $[-0.5, 0.5]$，远小于真实的存在区间 $(-\pi/2, \pi/2)$。

把矩形的高 b 当参数：$|y| \le b$ 时 $M = 1 + b^2$，故

$$\delta(b) = \frac{b}{1+b^2}, \qquad \delta'(b) = \frac{1-b^2}{(1+b^2)^2},$$

δ 在 $b = 1$ 处取最大值 $\frac{1}{2}$。换多大的矩形，δ 都超不过 $\frac{1}{2}$。 这不是证明方法不够好，而是“矩形 + $|f|$ 上界”这套工具本身的极限：f 增长得越快，解越可能提前跑到无穷远。

真解确实在 $\pi/2$ 处爆破，所以“只在局部保证”是正确的结论，不是保守。

(3) 哪一个可以整体延拓

$f(t,y) = -y$：$|f| = |y| \le 1 \cdot (1 + |y|)$，故可取 $K = 1$；$|\partial f/\partial y| = 1$，故 f 在 $\mathbb{R}^2$ 上全局 Lipschitz，$L = 1$。

由 1.3 节末的整体延拓估计，

$$|y(t)| \le (1 + |y_0|)e^{K|t|} - 1 = 2e^{|t|} - 1,$$

右端在任何有限区间上都有界，解不可能在有限时刻跑到无穷远。于是每到一个新的时刻都可以用定理 1 重新出发，解可延拓到任意有限区间。

而 $f(t,y) = 1 + y^2$ 不满足这种线性增长条件，对应对 (2) 中会在有限时刻爆破的解，不能整体延拓。

(4) 数值现象

$h = 0.1$ 时 $y_{n+1} = y_n + 0.1(1 + y_n^2)$：

t	1.0	1.4	1.5	1.6
Euler	1.39639	3.38976	4.63881	6.89067
$\tan t$	1.55741	5.79788	14.10142	−34.23253

$t \le 1.4$ 时数值解偏小但走势一致；到了 $t = 1.5$ 差距迅速拉开；$t = 1.6$ 时精确解已经越过奇点、从 $-\infty$ 一侧回来，而 Euler 解仍在一路增长——数值解“看不见”奇点，它只是把递推继续做下去。

这正是局部存在性的意义：定理 1 只保证 $|t| \le 0.5$ 上的解，越过 $\pi/2$ 之后讨论的已经不是同一个解了。

(5) 陷阱：局部结论不等于局部算法

错在把“定理保证的区间”当成了“算法能算的区间”。

定理 1 是对解的存在性的陈述：它保证在 δ 内解存在且唯一，但解本身可以延拓得更远（如 (3) 所示）。Euler 格式是递推，只要右端有定义、数值不发散，就可以继续算；算得对不对是另一回事（属 4.1 节）。

常见错误： 把“存在区间”与“可计算区间”混为一谈；或以为 δ 小就说明方法差。

题 5　快速题：初值问题与经典解

初值问题：$y'(t)=f\big(t,y(t)\big)$、$y(t_0)=y_0$、$t\in I$。经典解：$y\in C^1(I)$、$(t,y(t))\in D$ 且逐点满足方程与初值条件。

要求 $y\in C^1$ 是因为方程里出现 y'；若只要求 y 连续，y' 可能不存在，“解”就无从谈起。

题 6　快速题：Lipschitz 常数

$\dfrac{\partial f}{\partial y}=2y$，$|y|\le 2$ 时 $\left|\dfrac{\partial f}{\partial y}\right|\le 4$。由命题 1，$L=4$ 可用。

题 7　快速题：存在区间半长

$\delta=\min\{a,\,b/M\}=\min\{1,\,0.5/2\}=\min\{1,\,0.25\}=0.25$。解在 $[t_0-0.25,\,t_0+0.25]$ 上保证存在唯一。

题 8　快速题：Gronwall 不等式

若 $u(t)\le \alpha+\int_{t_0}^{t}\beta(s)u(s)\,ds$（$u,\beta\ge 0$ 连续，$\alpha\ge 0$），则 $u(t)\le \alpha\exp\big(\int_{t_0}^{t}\beta(s)ds\big)$。

$\alpha=0$ 时右端为 0，故 $u(t)\le 0$；又 $u\ge 0$，所以 $u\equiv 0$。

题 9　快速题：绝对值右端

满足。对任意 u,v，由三角不等式

$$\big||u|-|v|\big|\le|u-v|,$$

故 $f(t,y)=|y|$ 在 $\mathbb{R}^2$ 上满足 Lipschitz 条件，常数 $L=1$（对 y 的 Lipschitz 条件是全局的）。

但 $|y|$ 在 $y=0$ 处不可微。所以“Lipschitz”比“连续可微”更宽：命题 1 只给出“偏导有界 $\Rightarrow$ Lipschitz”这一个方向，反过来不成立。$|y|$ 正是那个反例——它 Lipschitz，却不可微。

题 10　快速题：Euler 一步

$y_1=1+0.5\times(-1)=0.5$。与 $y(0.5)=0.606531$ 相比，偏小（这一步的斜率用的是起点处较陡的 -1）。

题 11　找错题：Euler 的步数与末点

（1）$M = \mathrm{round}((1-0)/0.3) = \mathrm{round}(3.333\ldots) = 3$。循环结束 $t = 0 + 3 \times 0.3 = 0.9$（浮点运算下 0.8999999999999999），不等于 $T = 1$。最后一步落在 $t = 0.9$，而 `exact(T)` 取的是 $t = 1$ 的值，两者不在同一时刻，误差自然“偏离趋势”。

（2）修正为：先算 M，再令 $h = (T - t_0)/M$：

```
M = round((T - t0) / h)
h = (T - t0) / M        # 修正步长，保证最后一层恰好落在 T
```

修正后 t 在循环结束时精确等于 T，误差才可比。

（3）程序不会报错，因为每一步都是合法的算术：M 是整数，循环次数正确，只是终点错了。这属于实现错误（步长没有取整修正），不是格式错误：Euler 格式本身没有变。

评分要点： 能算出 $M = 3$ 与 $t = 0.9$；能写出 `h=(T-t0)/M`；能分清“实现错”与“格式错”。只答“最后一步有问题”而不给数值，算半对。

题 12　开放题：牛顿冷却

（1）Lipschitz 条件。 $f(t,\theta) = -k(\theta - \theta_{\mathrm{env}})$，$\dfrac{\partial f}{\partial \theta} = -k$，故 $|f(t,u) - f(t,v)| = k|u - v|$，$L = k$。因为偏导是常数，这个常数与温度范围无关，是全局 Lipschitz。

（2）存在区间。 由定理 1，在矩形 $|t| \le a$、$|\theta - \theta_0| \le b$ 上，$M = \max|f| \le k\big(|\theta_0| + b + |\theta_{\mathrm{env}}|\big)$，$\delta = \min\{a,\ b/M\}$，解在 $|t| \le \delta$ 上存在唯一。

这个结论不能直接回答“能不能算到 $t = 100$”：定理 1 是局部的，δ 由矩形大小与 $|f|$ 的上界决定。要算到 $t = 100$，需要整体延拓（1.3 节末）：因为 $|f| \le k(1 + |\theta|)$，解可延拓到任意有限区间。严格说这一步超出了本节定理的范围，需要引用整体延拓的结论。

（3）初值差的影响。 由定理 2（式 (5)），

$$|\theta(t) - \theta'(t)| \le 0.5\, e^{kt}.$$

这就是全部能说的：它给出“最多差多少”，且随 t 指数增长。实际差通常是 $0.5e^{-kt}$（衰减），但本节只证明上界，不证明紧性。

（4）步长怎么选。 本节回答不了这个问题：本节只有存在唯一性与连续依赖，没有“步长上限”这一条。步长的稳定性限制（对这个线性问题，显式 Euler 要求 $kh \le 2$，即 $h \le 2/k$）属于 4.2 节的内容。

所以清单的最后一条应当写：**“本节保证解存在且稳定；步长上限要看 4.2 的稳定性判据。”**把这一条写出来，比给出任何一个具体步长都重要。

评分要点（四问各 2.5 分）： (1) 要写出 $L = k$ 并说明与范围无关； (2) 要说出“局部”二字，并指出整体延拓不是本节定理的直接结论； (3) 要写成 $0.5e^{kt}$ 的形式； (4) 要明确指出“步长上限不属于本节”，不能凭空给一个数。

第二部分　3.1.2 经典 Runge–Kutta 方法（题 13–17）

题 13　RK4 手算一步

(1) 四个阶段与一步推进

$y_0=1$，$h=0.5$，$f(t,y)=-y$：

$$k_1=f(0,1)=-1.000000,$$

$$k_2=f\big(0.25,\ 1+0.25\times(-1)\big)=f(0.25,\ 0.75)=-0.750000,$$

$$k_3=f\big(0.25,\ 1+0.25\times(-0.75)\big)=f(0.25,\ 0.8125)=-0.812500,$$

$$k_4=f\big(0.5,\ 1+0.5\times(-0.8125)\big)=f(0.5,\ 0.59375)=-0.593750.$$

$$y_1=1+\frac{0.5}{6}\big(-1.000000+2(-0.750000)+2(-0.812500)-0.593750\big)=1-0.3932291667=0.6067708333.$$

(2) 误差

精确值 $e^{-0.5}=0.6065306597$，故

$$\left|y_1-e^{-0.5}\right|=2.402\times10^{-4}.$$

(3) 与 Euler 的比较

Euler 一步给 $y_1=0.5$，误差 1.065×10^{-1}。两者误差之比为

$$\frac{1.065\times10^{-1}}{2.402\times10^{-4}}=4.4\times10^{2},$$

即同一步长下 RK4 精确了约 440 倍。代价是每步 4 次右端求值（Euler 只需 1 次），且要额外存储并组合四个阶段的斜率。

评分要点： 四个 k_i 的数值必须算对（这是本题唯一容易出错的地方，特别是 k_3 用 k_2 预测）；只写公式不代数值最多给一半分。

题 14　Butcher 表与阶条件

(1) Butcher 表

$$\begin{array}{c|cccc} 0 & & & & \\ \frac12 & \frac12 & & & \\ \frac12 & 0 & \frac12 & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac16 & \frac13 & \frac13 & \frac16 \end{array}$$

左列 $c_i=0,\frac12,\frac12,1$；右上区 $a_{21}=\frac12$，$a_{31}=0,a_{32}=\frac12$，$a_{41}=a_{42}=0,a_{43}=1$；末行 $b_i=\frac16,\frac13,\frac13,\frac16$。

(2) 验证三条阶条件

$$\sum_i b_i=\frac16+\frac13+\frac13+\frac16=1;$$

$$\sum_i b_ic_i=\frac16\cdot0+\frac13\cdot\frac12+\frac13\cdot\frac12+\frac16\cdot1=\frac16+\frac16+\frac16=\frac12;$$

$$\sum_i b_i c_i^3 = \frac{1}{6}\cdot 0 + \frac{1}{3}\cdot\frac{1}{8} + \frac{1}{3}\cdot\frac{1}{8} + \frac{1}{6}\cdot 1 = \frac{1}{24}+\frac{1}{24}+\frac{1}{6} = \frac{1}{4}.$$

三条全部成立。（其余五条同样成立，逐条代入见正文的核验段落。）

(3) 为什么四阶格式不唯一

$s=4$ 时，节点系数由耦合系数的行和确定（$c_i=\sum_{j<i}a_{ij}$），故独立系数为：权重 b_i 四个、节点 c_2,c_3,c_4 三个（$c_1=0$ 固定）、耦合系数 a_{32},a_{42},a_{43} 三个，共 10 个；而四阶的阶条件恰有 8 条。未知量多于方程，所以系数不唯一。

经典 RK4 只是这组解中系数最整齐的一组；题 15 的 3/8 规则是另一组解。不同的解给出不同的四阶格式，它们的阶数相同、误差常数与稳定性却可以不同——这就是“四阶格式不唯一”。

题 15　Kutta 3/8 规则

(1) 节点、权重与阶条件

由各阶段的定义读出 $c_i=0,\frac{1}{3},\frac{2}{3},1$；$b_i=\frac{1}{8},\frac{3}{8},\frac{3}{8},\frac{1}{8}$。于是

$$\sum_i b_i = \frac{1}{8}+\frac{3}{8}+\frac{3}{8}+\frac{1}{8} = 1;$$

$$\sum_i b_i c_i = \frac{3}{8}\cdot\frac{1}{3}+\frac{3}{8}\cdot\frac{2}{3}+\frac{1}{8}\cdot 1 = \frac{1}{8}+\frac{1}{4}+\frac{1}{8} = \frac{1}{2};$$

$$\sum_i b_i c_i^2 = \frac{3}{8}\cdot\frac{1}{9}+\frac{3}{8}\cdot\frac{4}{9}+\frac{1}{8}\cdot 1 = \frac{1}{24}+\frac{1}{6}+\frac{1}{8} = \frac{1+4+3}{24} = \frac{1}{3}.$$

三条成立，与经典 RK4 相同。

(2) 实测收敛阶

取 $y'=-y$、$y(0)=1$、$T=1$，误差随步长的变化为：$h=1/2$ 时 2.914×10^{-4}，$h=1/64$ 时 1.851×10^{-10}，末段观测阶

$$p_{\text{obs}} = \log_2\frac{3.0008\times10^{-9}}{1.8512\times10^{-10}} \approx 4.02,$$

即收敛阶为 4。

在这个线性算例上，它的误差与经典 RK4 到前四位有效数字完全相同。原因：任何四阶格式的单步放大因子 $R(z)$ 都与 e^z 在 z^4 以内一致（这正是前四条阶条件的内容），两者的差别从 z^5 项才出现；本题 $|z|=h\le 0.5$，该差别落在第六位有效数字之后。

(3) “四阶方法不唯一”的含义

题 14 (3) 说明满足 8 条阶条件的系数不唯一；本题给出其中另一组具体的系数，并用数值验证它确实是四阶。所以“四阶”刻画的是精度等级，不是某一个固定公式；选择具体格式时还要看误差常数、稳定性与实现代价。

评分要点： 能验证三条（尤其第三条 $\sum b_i c_i^2$ 需要正确取 c_i）；能给出实测阶接近 4 的证据。

题 16　找错题：两个写错的 RK4

(1) 版本 A：权重写成 $1:1:1:1$

此时 $b_i=\frac{1}{4}$，仍取 $c_i=0,\frac{1}{2},\frac{1}{2},1$：

$$\sum_i b_i=\frac{1}{4}\times 4=1 \quad \text{（成立），} \qquad \sum_i b_i c_i=\frac{1}{4}\left(0+\frac{1}{2}+\frac{1}{2}+1\right)=\frac{1}{2} \quad \text{（成立），}$$

$$\sum_i b_i c_i^2=\frac{1}{4}\left(0+\frac{1}{4}+\frac{1}{4}+1\right)=0.375\neq\frac{1}{3} \quad \text{（不成立），}$$

$$\sum_i b_i c_i^3=\frac{1}{4}\left(0+\frac{1}{8}+\frac{1}{8}+1\right)=0.3125\neq\frac{1}{4} \quad \text{（不成立）.}$$

即三阶条件开始失效，格式退化为二阶。

(2) 版本 B：k_3 用 k_1 预测

第 3 级的预测值应为 $y_n+\frac{h}{2}k_2$，写错后用的是 $y_n+\frac{h}{2}k_1$，相当于把耦合系数 a_{32} 从 $\frac{1}{2}$ 改成 0、同时把 a_{31} 从 0 改成 $\frac{1}{2}$。受影响的正是第 3 级，进而通过 k_3 影响 k_4 与最终加权和。

(3) 实测阶与误差放大

同一个算例（$y'=-y$，$T=1$），三种写法的观测阶与 $h=1/2$ 时的误差：

写法	h=1/2 的误差	末段观测阶	相对正确版的误差倍数
正确 RK4	2.914×10^{-4}	4.019	1
A：权重 1:1:1:1	1.287×10^{-3}	1.999	4.4
B：k_3 用 k_1	9.833×10^{-3}	2.011	34

两者都从四阶掉到二阶，比正确版差 4 倍与 34 倍。可见“权重看起来差不多”或“预测值随手取一个”都会让格式静默降阶，而程序不会报错——这是实现 RK4 时最需要防的错误。

评分要点：(1) 必须算出 $\sum b_i c_i^2=0.375$ 才能判定；(2) 要指出是 a_{32} 而不是权重；(3) 给出“掉到二阶”的结论即可，具体倍数视算例而定。

题 17　几何意义与权重来源

(1) 三个试探点的坐标

$M_1=\left(t_n+\frac{h}{2},\ y_n+\frac{h}{2}k_1\right)=(0.25,\ 0.7500)$；

$M_2=\left(t_n+\frac{h}{2},\ y_n+\frac{h}{2}k_2\right)=(0.25,\ 0.8125)$；

$E_1=(t_n+h,\ y_n+hk_3)=(0.50,\ 0.59375)$。

M_1 与 M_2 的 t 坐标相同，都是 $t_n+\frac{h}{2}$；不同的是 y 坐标。

(2) 为什么不能合并为一级

两个点在同一时刻，但用的是两个不同的预测值：M_1 由起点斜率 k_1 外推得到，偏低（对本题 0.7500）；M_2 用中点斜率 k_2 回头修正，偏高（0.8125）。因此 $k_2 \neq k_3$（-0.7500 与 -0.8125），两者携带的信息不同。把第 3 级写成“用 k_1 预测”，就退化为题 16 的版本 B，阶数掉到二阶。

换句话说：同一个取样时刻被取样两次，是 RK4 用四次求值换取四阶精度的关键。

(3) 权重 $1:2:2:1$ 的来源

把一步推进写成积分 $y(t_n+h)-y(t_n)=\int_{t_n}^{t_n+h} f\,dt$，则“走一步”就是算一个积分。用 Simpson 公式近似它，需要在左端点、中点、右端点取值，权重比为 $1:4:1$；中点被第 2、3 级各取一次，4 便拆成 $2+2$，于是权重成为 $1:2:2:1$，节点为 $0,\frac{1}{2},\frac{1}{2},1$。

评分要点：（1）三个坐标要算对；（2）要指出“同 t 不同 y”；（3）要说清 $4=2+2$ 这一拆分。

第三部分　3.1.3 误差、稳定性与计算成本（题 18–22）

题 18　局部阶与全局阶

(1) 两种方法的阶

方法	局部截断误差	全局误差
显式 Euler	$O(h^2)$，$\tau=\frac{h}{2}y''(\xi)$	$O(h)$
经典 RK4	$O(h^5)$	$O(h^4)$

(2) 观测阶

按 $p_{\text{obs}}=\log_2\dfrac{E(h)}{E(h/2)}$：

h	Euler 误差	p_{obs}	RK4 误差	p_{obs}
$1/2$	1.179×10^{-1}	—	2.914×10^{-4}	—
$1/4$	5.147×10^{-2}	1.195	1.476×10^{-5}	4.303
$1/8$	2.427×10^{-2}	1.085	8.308×10^{-7}	4.151
$1/16$	1.181×10^{-2}	1.040	4.928×10^{-8}	4.075

Euler 的观测阶趋于 1，RK4 的趋于 4，与理论一致。步长较大时观测阶偏高（$4.303>4$），是因为更高阶的截断项尚未充分衰减；再继续减半，观测阶会单调逼近 4（$h=1/64$ 时 4.019）。

(3) 为什么低一阶

全局误差是 $M=T/h$ 步局部误差的累积：

$$|y(t_n)-y_n|\lesssim\sum_j|\tau_j|\cdot(\text{放大因子})\approx\frac{T}{h}\cdot O(h^{p+1})=O(h^p),$$

步数按 $1/h$ 增长，正好抵消掉局部误差里的一个 h。这就是“降一阶”的来源，也是不能把局部阶当全局阶的原因。

题 19　绝对稳定性

(1) 放大因子

把格式作用于测试方程 $y'=\lambda y$，得 $y_{n+1}=R(z)y_n$，$z=h\lambda$：

$$R_{\text{Euler}}(z)=1+z,\qquad R_{\text{RK4}}(z)=1+z+\frac{z^2}{2}+\frac{z^3}{6}+\frac{z^4}{24}.$$

后者正是 e^z 的 Taylor 展开截断到四次。

(2) 实负轴上的稳定区间

要求 $|R(z)|\le 1$。

Euler：$|1+z|\le 1$ 给 $-2\le z\le 0$，故 $h|\lambda|\le 2$。

RK4：对 $z\le 0$ 解 $|R(z)|=1$，得 $z\approx -2.785$，故 $h|\lambda|\le 2.785$。

(3) $\lambda=-10$ 时的最大步长

Euler：$h\le \dfrac{2}{10}=0.2$；RK4：$h\le \dfrac{2.785}{10}=0.2785$。

RK4 的稳定上限约为 Euler 的 1.39 倍。

(4) $h=3$、$\lambda=-1$ 时为什么也发散

$z=-3$：

$$|R_{\text{Euler}}(-3)|=|1-3|=2.0>1,\qquad |R_{\text{RK4}}(-3)|=\left|1-3+\frac{9}{2}-\frac{27}{6}+\frac{81}{24}\right|=|1.375|=1.375>1.$$

两者都越过稳定边界，数值解按 $\times 2$ 与 $\times 1.375$ 的倍率振荡增长。

四阶精度与稳定性是两个独立的性质：阶数只描述 $h\to 0$ 时误差下降多快，稳定性描述固定 h 下扰动是否被放大。步长过大时，四阶格式同样会失效，只是比 Euler 稍晚失效。

评分要点：(2) 要给具体端点；(4) 要算到 1.375 并给出“独立性质”的判断。

题 20　隐式 Euler 与无条件稳定

(1) 格式与放大因子

后向 Euler：$y_{n+1}=y_n+hf(t_{n+1},y_{n+1})$。对测试方程 $y_{n+1}=y_n+h\lambda y_{n+1}$，故

$$y_{n+1}=\frac{1}{1-z}\,y_n,\qquad R(z)=\frac{1}{1-z}.$$

(2) 证明 $\mathrm{Re}\lambda<0$ 时 $|R|<1$

$|1-z|^2=(1-\mathrm{Re}z)^2+(\mathrm{Im}z)^2$。当 $\mathrm{Re}\lambda<0$、$h>0$ 时 $\mathrm{Re}z=h\mathrm{Re}\lambda<0$，故 $1-\mathrm{Re}z>1$，于是

$$|1-z|^2>1\quad\Longrightarrow\quad |R(z)|=\frac{1}{|1-z|}<1.$$

与 h 的大小无关，即无条件稳定（对左半平面）。

(3) $h = 0.3$ 时的比较

$z = h\lambda = -3$。

显式：$|R| = |1 - 3| = 2 > 1$，数值解以 $\times 2$ 振荡增长，发散。

隐式：$|R| = \left|\dfrac{1}{1+3}\right| = 0.25 < 1$，数值解单调衰减，稳定。

同一问题、同一步长，只因“斜率取在左端还是右端”而定性不同。

(4) 代价

右端含 y_{n+1}，每一步要解一个方程：线性问题解线性方程组，非线性问题用 Newton 迭代并计算 Jacobian。每步成本远高于显式格式，且实现复杂。显式与隐式的取舍，本质上是每步成本与稳定性之间的取舍。

题 21　计算成本

(1) 求值次数之比与误差之比

$$\frac{n_f^{\text{RK4}}}{n_f^{\text{Euler}}} = \frac{512}{128} = 4, \qquad \frac{E_{\text{Euler}}}{E_{\text{RK4}}} = \frac{1.442 \times 10^{-3}}{1.149 \times 10^{-11}} \approx 1.3 \times 10^{8}.$$

多花 4 倍求值，误差改善约 8 个数量级。

(2) 达到 10^{-6} 所需的步数与求值次数

用误差常数近似：$E_{\text{Euler}} \approx 0.236\,h$，$E_{\text{RK4}} \approx 4.66 \times 10^{-3} h^4$。

Euler：$h \approx \dfrac{10^{-6}}{0.236} = 4.2 \times 10^{-6}$，$M \approx 2.4 \times 10^{5}$，$n_f \approx 2.4 \times 10^{5}$。

RK4：$h \approx \left(\dfrac{10^{-6}}{4.66 \times 10^{-3}}\right)^{1/4} = 0.121$，$M \approx 8$，$n_f \approx 33$。

即达到同一精度，Euler 约需二十多万次求值，RK4 只需几十次。（这两个数是按渐近常数外推得到的量级估计，不保证在具体步长上逐位成立。）

(3) “每步贵 4 倍”是否意味着不划算

不是。比较成本必须在相同精度下进行：RK4 每步贵 4 倍，但允许的步长大得多（四阶 对 一阶），步数减少的量级远超过 4 倍。上例中 RK4 便宜了约 7×10^{3} 倍。

反过来的情形也存在：当步长被稳定性卡住、精度要求又很松时，高阶精度用不上，RK4 的 4 倍开销就变成纯粹的浪费（见题 22）。

题 22　开放题：刚性问题的选型

（1） 稳定上限与最少的求值次数

$\lambda = -50$，故 $|\lambda| = 50$。

Euler：$h \le \dfrac{2}{50} = 0.04$，$M \ge \dfrac{20}{0.04} = 500$，$n_f = 500$。

RK4：$h \le \dfrac{2.785}{50} = 0.0557$，$M \ge \dfrac{20}{0.0557} \approx 359$，$n_f = 4 \times 359 \approx 1436$。

（2）谁更省，为什么高阶没好处

仅就“不发散”这一条，Euler 更省（$500 < 1436$）：它的步长可以取到 0.04，每步只算一次右端；RK4 虽然步长可以大一些（0.0557），但每步要算四次，在步长被稳定性限制的情形下，四阶精度根本用不上——精度是“给多了”，代价却照付。

这就是“刚性”问题的典型特征：步长由稳定性决定而不是由精度决定，高阶显式格式的优势被抹平。

（3）稳定 $\neq$ 准确

取在稳定上界时，两种方法的放大因子绝对值都等于 1：

Euler：$z = h\lambda = -50 \times 0.04 = -2$，$R = 1 - 2 = -1$，$|R| = 1$，故 $y_n = (-1)^n y_0$——幅值恒为 1 的振荡。

RK4：$z = -2.785$，位于 $|R(z)| = 1$ 的边界上，$|R| = 1$，幅值同样不衰减。

而真解是 e^{-50t}，在 $t = 20$ 时为 $e^{-1000} \approx 0$。于是数值解与真解相差约 1（百分之百的相对误差），尽管它“没有发散”。稳定只保证扰动不被放大，不保证跟得上真解的衰减；要控制误差，步长必须取在稳定区间内部。

（4）精度要求下的步长

不能。本题的稳定性判据只回答“不发散”，不回答“误差多大”。要求终点误差 10^{-4}，需要误差阶与误差常数的估计（本节的 4.1 目：局部截断误差与收敛阶），再结合稳定性给出最终步长；对刚性更强的情形还要换用隐式方法（题 20）。

选型结论： 步长由稳定性决定时，先用稳定判据定上限，再用精度判据校验；若上限处误差不达标，要么缩小步长（代价高），要么改用隐式格式——高阶显式方法的优势只在“精度主导”的场景下成立。

评分要点（四问各 3 分）： （1）两个步长上限与步数要算对（$0.04/500$、$0.0557/359$）； （2）要明确“Euler 更省”并给出“步长受稳定性限制”这一理由； （3）要算到 $|R| = 1$ 并指出幅值不衰减、而真解趋于 0，误差约 1； （4）要指出稳定性判据不能替代精度分析，不能凭空给一个步长。

第四部分　3.2.1 热传导方程与定解问题（题 23–27）

题 23　傅里叶定律与方程

(1) 各符号与单位

q 为热流密度，单位 W/m²；k 为热导率，W/(m·K)；u 为温度，K；$\partial u/\partial x$ 为温度梯度，K/m。量纲核对：$k\frac{\partial u}{\partial x}$ 的单位为 $\mathrm{W/(m \cdot K) \cdot K/m = W/m^2}$，与 q 相同。

(2) 方程的导出

取细杆上厚 Δx、截面积 S 的一薄层，其热量为 $\rho c u S \Delta x$，热量变化率等于“流入 − 流出 + 内部产生”：

$$\rho c\,S\Delta x\,\frac{\partial u}{\partial t} = S\,q(x,t) - S\,q(x+\Delta x,t) + S\Delta x\,f(x,t).$$

两端除以 $S\Delta x$ 并令 $\Delta x \to 0$：

$$\rho c\,\frac{\partial u}{\partial t} = -\frac{\partial q}{\partial x} + f.$$

代入 $q = -k\,u_x$（k 为常数），再除以 ρc，即得

$$u_t = \kappa u_{xx} + g, \qquad \kappa = \frac{k}{\rho c},\quad g = \frac{f}{\rho c}.$$

κ 的单位：$\mathrm{W/(m\cdot K)} \div \big(\mathrm{kg/m^3}\cdot \mathrm{J/(kg\cdot K)}\big)$，而 $\mathrm{J} = \mathrm{W\cdot s}$，分母为 $\mathrm{W\cdot s/m^3}$，故 $[\kappa] = \mathrm{m^2/s}$。

题 24　定解条件与相容性

(1) 为什么一个初值、两个边界

方程只含 t 的一阶导数，故只需一个初始条件；含 x 的二阶导数，故在 $x=0$ 与 $x=1$ 各需一个边界条件，共两个。

(2) 不相容点

取 $u_0(x) = 0$，则 $u_0(0) = 0$ 与左边界 $u(0,t) = 0$ 相容；但 $u_0(1) = 0$，右边界为 $u(1,t) = 1$，故在角点 $(1,0)$ 处初值与边界矛盾，古典解在该点不连续，本题古典解不存在。

(3) $t \to \infty$ 的极限

令 $u_t = 0$，得 $u_{xx} = 0$，解为 $u = Ax + B$。由 $u(0) = 0$ 得 $B = 0$，由 $u(1) = 1$ 得 $A = 1$，故

$$u(x,\infty) = x,$$

即数值解最终趋于一条从左端 0 到右端 1 的直线。

题 25　验证精确解

记 $u(x,t) = t\sin x$。

（1）方程。 左端 $u_t = \sin x$；右端 $u_{xx} + (t+1)\sin x = -t\sin x + (t+1)\sin x = \sin x$。两端相等。

（2）初值。 $u(x,0) = 0\cdot\sin x = 0$，与 $u(x,0) = 0$ 一致。

（3）边界。 $u(0,t) = t\sin 0 = 0$；$u(1,t) = t\sin 1$，与题给边界一致。

四组条件全部满足，故 $u(x,t) = t\sin x$ 就是本题的解（由最大值原理可知它还唯一）。

题 26　最大值原理

(1) 叙述

若 u 在 $Q_T = (0,1)\times(0,T]$ 内满足 $u_t = u_{xx}$、在闭区域上连续，则 u 的最大值与最小值都在抛物边界

$$\Gamma_T = \{(x,0) : 0 \le x \le 1\} \cup \{(0,t),(1,t) : 0 \le t \le T\}$$

上取到。

(2) 证明 $|u| \le M$

设初值与边界值的绝对值不超过 M，则 $\max_{\Gamma_T}|u| \le M$。

$v = -u$ 也满足热方程。由最大值原理，u 的最大值不超过 Γ_T 上 u 的最大值，v 的最大值不超过 Γ_T 上 v 的最大值，于是

$$u \le \max_{\Gamma_T} u \le M, \qquad u \ge -\max_{\Gamma_T}(-u) \ge -\max_{\Gamma_T}|u| \ge -M.$$

故 $|u(x,t)| \le M$。结论的意义是“解不会自己长出新的极值”。

题 27　差分格式一步手算

(1) 格式与时间步长

$$U_j^{n+1} = U_j^n + r\big(U_{j-1}^n - 2U_j^n + U_{j+1}^n\big) + \Delta t, \qquad \Delta t = rh^2 = 0.25 \times 0.01 = 2.5 \times 10^{-3}.$$

(2) 一步之后的值

$j = 1$：$U_0^0 = 0$（边界），故

$$U_1^1 = (1-2r)U_1^0 + r(U_0^0 + U_2^0) + \Delta t = 0.5 \times 1 + 0.25 \times (0+1) + 0.0025 = 0.7525.$$

$j = 5$：两侧都是内点，

$$U_5^1 = 0.5 \times 1 + 0.25 \times (1+1) + 0.0025 = 1.0025.$$

(3) 为什么 U_1^1 更小

U_1 的左邻居是 0（边界），只有右邻居贡献热量；U_5 两边都是 1，近邻的平均较高。从“邻居平均”看，靠近边界的点被拉低，这正是扩散把热量从中间往两端摊开的表现。

第五部分　3.2.2 差分格式与 $Ax = b$ 的构造（题 28–37）

题 28　写出显式格式

(1) 格式

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \frac{U_{j-1}^n - 2U_j^n + U_{j+1}^n}{h^2} + g_j^n, \qquad j = 1, \dots, N-1,$$

即

$$U_j^{n+1} = U_j^n + r\big(U_{j-1}^n - 2U_j^n + U_{j+1}^n\big) + \Delta t\, g_j^n, \qquad r = \frac{\Delta t}{h^2}.$$

(2) 初值与边界

$$U_j^0 = u_0(x_j),\ j = 0, 1, \dots, N; \qquad U_0^n = \gamma_0(t_n), \quad U_N^n = \gamma_N(t_n).$$

(3) 加权平均

$$U_j^{n+1} = (1-2r)U_j^n + rU_{j-1}^n + rU_{j+1}^n + \Delta t\, g_j^n,$$

三个权重 $(1-2r,\ r,\ r)$ 之和为 1。

题 29　构造系数矩阵

(1) 矩阵与右端

$r=0.5$，$1+2r=2$：

$$A=\begin{pmatrix}2 & -0.5 & 0\\ -0.5 & 2 & -0.5\\ 0 & -0.5 & 2\end{pmatrix},\qquad \boldsymbol{b}=\begin{pmatrix}U_1^n\\ U_2^n\\ U_3^n\end{pmatrix}.$$

(2) 右端边界改为 $U_4^{n+1}=1$

A 不变，只有最后一个方程把 $-rU_4^{n+1}$ 移到右端：

$$\boldsymbol{b}=\begin{pmatrix}U_1^n\\ U_2^n\\ U_3^n+r\end{pmatrix}=\begin{pmatrix}U_1^n\\ U_2^n\\ U_3^n+0.5\end{pmatrix}.$$

要点： 边界条件只改首末两行与 $\boldsymbol{b}$ 的首末分量。

题 30　从矩阵反写方程

(1) 网格比

对角元 $1+2r=1.8$，故 $2r=0.8$，$r=0.4$。

(2) 第一行对应的方程

$$1.8U_1-0.4U_2=b_1,\qquad \text{即}\qquad -rU_0+(1+2r)U_1-rU_2=b_1,$$

其中 U_0 是左边界值。

(3) 两端边界条件

$N=3$ 时内点为 U_1,U_2，相邻边界点是 U_0 与 U_3。第一行只出现 U_1,U_2，说明 U_0 已被移到右端；最后一行同理。故两端都是 Dirichlet 边界（若为 Neumann，首行会多出一个 $-2r$ 的角元）。

题 31　追赶法

(1) 前代与回代

$a_2=a_3=-1$，$b_i=2$，$c_1=c_2=-1$，$d=(1,0,1)$。

$$c_1'=\frac{c_1}{b_1}=-\frac{1}{2},\qquad d_1'=\frac{d_1}{b_1}=\frac{1}{2},$$

$$c_2'=\frac{c_2}{b_2-a_2c_1'}=\frac{-1}{2-\frac12}=-\frac{2}{3},\qquad d_2'=\frac{d_2-a_2d_1'}{b_2-a_2c_1'}=\frac{0+\frac12}{\frac32}=\frac13,$$

$$d_3'=\frac{d_3-a_3d_2'}{b_3-a_3c_2'}=\frac{1+\frac13}{2-\frac23}=1.$$

回代：

$$u_3 = d_3' = 1, \quad u_2 = d_2' - c_2' u_3 = \frac{1}{3} + \frac{2}{3} = 1, \quad u_1 = d_1' - c_1' u_2 = \frac{1}{2} + \frac{1}{2} = 1.$$

(2) 验证

第 1 个方程：$2 - 1 = 1$；第 2 个方程：$-1 + 2 - 1 = 0$；第 3 个方程：$-1 + 2 = 1$。全部成立，故

$$(u_1, u_2, u_3) = (1, 1, 1).$$

题 32　（改编自教材习题三第 1 题）向前欧拉格式的误差

(1) 格式与边界

$$U_j^{n+1} = U_j^n + r\left(U_{j-1}^n - 2U_j^n + U_{j+1}^n\right) + \Delta t\,(t_n + 1)\sin x_j, \qquad r = \frac{\Delta t}{h^2},$$

$$U_j^0 = 0\ (0 \le j \le N), \qquad U_0^n = 0, \qquad U_N^n = t_n \sin 1.$$

(2) 时间步长与步数

$h = 1/4$，$h^2 = 1/16$，

$$\Delta t = r h^2 = 0.5 \times \frac{1}{16} = \frac{1}{32}, \qquad M = \frac{T}{\Delta t} = 32.$$

(3) 节点 $(0.5, 1)$ 的数值解与误差（精确值 $0.5 \sin 0.5 = 0.4794255$）

h	r	M	数值解	误差
$1/4$	0.5	32	0.479699	$+2.73 \times 10^{-4}$
$1/8$	0.5	128	0.479494	$+6.84 \times 10^{-5}$
$1/16$	0.5	512	0.479443	$+1.71 \times 10^{-5}$

误差每换一次网格约降为 $1/4$，观测阶约为 2。注意这里的误差主要来自空间离散（$O(h^2)$）；因为时间与空间同步加密，总误差的表现也是二阶。

(4) 把 r 改成 0.4 会怎样

h 不变、r 从 0.5 降到 0.4，Δt 也随之缩小，时间误差更小；但该点（$x = 0.5$ 附近、光滑解）的误差以空间误差为主，数值解几乎不变。若换成粗糙初值或离边界很近的点，权重变了（$1 - 2r$ 从 0 变为 0.2），单步的“邻居平均”方式改变，r 的影响才会在结果里显现。

题 33　（改编自教材习题三第 2 题）向后欧拉格式与网格比

(1) 格式

$$-rU_{j-1}^{n+1} + (1 + 2r)U_j^{n+1} - rU_{j+1}^{n+1} = U_j^n + \Delta t\,(t_{n+1} + 1)\sin x_j.$$

新一层的三个值同时出现在方程里，必须整层一起解，因此每步要解一个三对角方程组（矩阵 $A = I - rL$，对角 $1 + 2r$、次对角 $-r$）。

(2) 网格比

$h=1/4$、$\tau=1/20$：$r=\tau/h^2=0.05/0.0625=0.8$；$\tau=1/40$：$r=0.025/0.0625=0.4$。

(3) 数值解与误差（精确值 0.4794255）

h	τ	r	数值解	误差
1/4	1/20	0.80	0.479699	$+2.73\times10^{-4}$
1/4	1/40	0.40	0.479699	$+2.73\times10^{-4}$
1/8	1/40	1.60	0.479494	$+6.84\times10^{-5}$

$r=1.6$ 远大于 $1/2$，但数值解仍然正常：隐式格式对任意 $r>0$ 都稳定。

(4) 与向前欧拉比较

同一网格上两法的误差都在 5×10^{-4} 以内、符号相同，差别很小。原因是本题的误差主项由空间离散决定（$O(h^2)$），时间方向两种方法都是一阶，误差方向也一致。若把 r 取得很大（如 $r=10$），向后欧拉的“过度衰减”才会明显，而向前欧拉早已失稳。

题 34　（改编自教材习题三第 3 题）Crank–Nicolson 格式

(1) 系数矩阵

由格式整理得

$$-\frac{r}{2}U_{j-1}^{n+1}+(1+r)U_j^{n+1}-\frac{r}{2}U_{j+1}^{n+1}=\frac{r}{2}U_{j-1}^{n}+(1-r)U_j^{n}+\frac{r}{2}U_{j+1}^{n}+\Delta t\,g_j^{n+\frac12},$$

故 A 的主对角元为 $1+r$，两个次对角元为 $-r/2$。

(2) $r=10$ 时为什么仍稳定

Crank－Nicolson 是隐式（两层平均）格式，其放大因子为

$$g(\theta)=\frac{1-2r\sin^2(\theta/2)}{1+2r\sin^2(\theta/2)},$$

分子、分母都非负，$|g(\theta)|\le1$ 对任意 $r>0$、任意 θ 成立，故无条件稳定。向前欧拉在 $r=10$ 时 $|1-4r|=39>1$，高频模态每步放大近 39 倍，几步就发散。

(3) 时间精度

格式的时间导数取在 $t_{n+\frac12}$（两层中点）的中心差商，截断误差为 $O(\Delta t^2)$，故时间精度是二阶（空间仍是二阶）。

题 35　（改编自教材习题三第 6 题）带导数边界条件

(1) 验证边界条件

$u=x^4e^t$，$u_x=4x^3e^t$。

左端：$u_x(0,t)=4\cdot 0^3e^t=0$，成立。

右端：$u_x(1,t)+u(1,t)=4e^t+e^t=5e^t$，成立。

(2) 边界离散

左端 Neumann。 引入虚拟节点 U_{-1}，用中心差商表示导数：

$$\frac{U_1-U_{-1}}{2h}=0\implies U_{-1}=U_1.$$

代入 $j=0$ 的隐式方程 $-rU_{-1}+(1+2r)U_0-rU_1=b_0$，得

$$(1+2r)U_0-2rU_1=b_0.$$

右端 Robin。 引入虚拟节点 U_{N+1}，由

$$\frac{U_{N+1}-U_{N-1}}{2h}+U_N=5e^t\implies U_{N+1}=U_{N-1}+2h\,(5e^t-U_N)$$

代入 $j=N$ 处的方程，即得首末两行的形式。

(3) 移到右端后的 b

先看左端。$U_{-1}=U_1$ 代入后，首行变为

$$(1+2r)U_0-2rU_1=b_0,$$

即首行对角元仍为 $1+2r$、次对角元由 $-r$ 变为 $-2r$，这一步没有产生新的右端项（因为 $\sigma=0$）。

再看右端。把 $U_{N+1}=U_{N-1}+2h(5e^t-U_N)$ 代入末行 $-rU_{N-1}+(1+2r)U_N-rU_{N+1}=b_N$：

$$-rU_{N-1}+(1+2r)U_N-r\big(U_{N-1}+2h(5e^t-U_N)\big)=b_N,$$

整理得

$$-2rU_{N-1}+\big(1+2r+2rh\big)U_N=b_N+10rh\,e^t.$$

于是：末行对角元变为 $1+2r+2rh$，次对角元变为 $-2r$，b 的最后一个分量增加 $10rh\,e^t$（本题取 N 与 $h=1/N$ 时即 $10rh\,e^t$）。

要点： 带导数的边界条件用“虚拟节点 + 消元”，最终仍化成三对角方程组；它的影响只出现在首末两行与 b 的首末分量上。

题 36 （改编自教材习题三第 7 题）二维问题的五点显式格式

(1) 格式与边界

$$U_{i,j}^{n+1}=U_{i,j}^n+r\big(U_{i-1,j}^n+U_{i+1,j}^n+U_{i,j-1}^n+U_{i,j+1}^n-4U_{i,j}^n\big)+\Delta t\,2(t_n-1)e^{t_n+x_i+y_j},$$

边界：

$$U_{0,j}^n=e^{t_n+y_j},\quad U_{N,j}^n=e^{t_n+y_j+1},\quad U_{i,0}^n=e^{t_n+x_i},\quad U_{i,N}^n=e^{t_n+x_i+1},$$

初值 $U_{i,j}^0=e^{x_i+y_j}$。

(2) $N=20$、$\Delta t=2.5\times10^{-4}$

$$h=\frac{1}{20}=0.05,\quad h^2=2.5\times10^{-3},\quad r=\frac{2.5\times10^{-4}}{2.5\times10^{-3}}=0.1\le\frac{1}{4},$$

满足二维稳定条件，可以计算。

(3) $\Delta t=1.0\times10^{-3}$、$N=20$

$$r=\frac{1.0\times10^{-3}}{2.5\times10^{-3}}=0.4>\frac{1}{4},$$

不满足二维稳定条件，不能用五点显式格式直接推进（需要减小 Δt 或加密网格后同步缩小 Δt）。

题 37　填空：矩阵的性质

格式	矩阵	每行非零元个数	稳定上界
一维显式	$B=I+rL$	（a）3	（b）$r\le1/2$
一维隐式	$A=I-rL$	（c）3	（d）任意 $r>0$
二维隐式	$A_{2D}=I-rL_{2D}$	（e）5	（f）任意 $r>0$

二维每行五个非零元——中心 $1+4r$、上下左右各 $-r$；隐式格式无条件稳定。

第六部分　3.2.3 数值解、收敛阶与稳定性（题 38–42）

题 38　观测阶

(1) 公式

$$p=\log_2\frac{e_N}{e_{2N}},$$

其中 e_N 是网格数 N 时的最大误差（N 加倍，h 减半）。

(2) 三组观测阶

$10\to20$：$p=\log_2\dfrac{4.29\times10^{-3}}{1.06\times10^{-3}}\approx2.02$；

$20\to40$：$p=\log_2\dfrac{1.06\times10^{-3}}{2.65\times10^{-4}}\approx2.00$；

$40\to80$：$p=\log_2\dfrac{2.65\times10^{-4}}{6.62\times10^{-5}}\approx2.00$。

三组都接近 2，与“空间二阶”相符；粗网格那组略偏离，是更高阶误差项还没退干净。

题 39　判断稳定与失稳

判据：一维 $|1-4r|\le1$ 即 $r\le1/2$；二维 $r\le1/4$。

(1) $r=0.4\le1/2$，稳定。

(2) $r = 0.6 > 1/2$，失稳。

(3) $r = 0.2 \le 1/4$，稳定。

(4) $r = 0.3 > 1/4$，失稳。

题 40　越过门槛会怎样

(1) $r = 0.4$

$$|1 - 4r| = 0.6, \qquad 0.6^{50} \approx 8.1 \times 10^{-12}.$$

高频扰动被压到 10^{-12} 量级，数值解光滑。

(2) $r = 0.6$

$$|1 - 4r| = 1.4.$$

$N = 20$、$T = 0.05$ 时实际取到 $r = 0.6061$、$M = 33$ 步，$|1 - 4r| = 1.4242$，约 $1.42^{33} \approx 4.7 \times 10^{4}$ 倍。数值解出现正负交替的棋盘格振荡（实测在 ± 83 之间）。

(3) 为什么 $1/2$ 是分界

$|1 - 4r|$ 关于 r 单调：$r < 1/2$ 时小于 1，模态衰减；$r = 1/2$ 时等于 1，模态不增不减；$r > 1/2$ 时大于 1，模态增长。$r = 1/2$ 正好把“衰减”与“增长”分开。

题 41　网格加密的代价

(1) 填表 $(r = 1/2,\ \Delta t = h^2/2)$

N	h	Δt	M
10	0.1	(a) 5.0×10^{-3}	(b) 20
20	0.05	(c) 1.25×10^{-3}	(d) 80
40	0.025	(e) 3.125×10^{-4}	(f) 320

(2) 步数增长

$\Delta t = h^2/2$，$M = T/\Delta t = 2TN^2$，故 N 加倍一次，M 变成 4 倍。

(3) 隐式格式

隐式格式无条件稳定，Δt 不受 $r \le 1/2$ 的限制，可以按精度需要取（例如与 h 同阶）。代价是每步要解一个 $(N-1)$ 阶三对角方程组（追赶法 $O(N)$），程序更复杂，时间精度仍是一阶。

题 42　开放题：显式还是隐式

(1) 显式

$$\Delta t \le \frac{h^2}{2} = \frac{(10^{-3})^2}{2} = 5\times 10^{-7}, \qquad M \ge \frac{1}{5\times 10^{-7}} = 2\times 10^{6}.$$

(2) 隐式

隐式格式无条件稳定，Δt 不再受 $h^2/2$ 的限制，只受精度约束；但每步要解一个三对角方程组。

(3) 为什么教材还要引入后两种格式

题 32 的向前欧拉误差是 $O(\Delta t + h^2)$，且 $r > 1/2$ 就可能失稳；题 33 的向后欧拉去掉了稳定性限制，但时间仍是一阶；题 34 的 Crank－Nicolson 把时间提到二阶，且无条件稳定。教材依次引入这三者，正是按“稳定性 → 精度”两条线逐步改进差分格式的典型路径。

第 03 章　离散的世界 • 习题参考解答（下）

> 本文件：A1-G3-习题参考解答-3.3-3.4.md（G3 = 第 03 章 A1 编写组；全章合并稿 • 下半册） 范围：覆盖 3.3－3.4（题 43－77，第七～第十一部分）；3.1－3.2（题 1－42，第一～第六部分）见 A1-G3-习题参考解答.md。 配套文件：A1-G3-习题.md、A1-G3-习题参考解答.md、A1-G3-正文.md、A1-G3-实验.ipynb 版本：全章合并稿（2026-10-26 由 3.1－3.4 分节解答合并；与习题逐题对应，版式照 3.1 节）。因 GitHub 单页数学公式数量上限（约 2000 个渲染单元），全章解答分为上、下两册，题号仍全章连续。 状态：解答由本组编写并逐题核对；题号与习题文件一一对应。本文件不纳入学生版教材目录。 数值来源：3.3 的题 44－46 的数据见配套实验与 数据/；3.4 的题 57－59、64－75 由配套脚本实算（$f_\theta = -\theta u$、观测点 $(0.5, 0.6065)$；logistic 取 $K=1$、$u_0=0.05$、$r=0.8$）。其余数字为手算，过程已写出，便于复核。 引用约定：解答中引用的定理、命题、推论、定义、公式、图、表编号，均指该题所属小节正文的编号；3.4 的四小节按 3.4.1－3.4.4 编号（正文内部仍称“第一～四章”、再分 §1.1 等）。解答中的“表 2－8”是本册自编的表号。

第七部分　3.3 有限元：弱形式与 Galerkin 方法（题 43–55）

题 43　装配与自由度映射

(1) 映射

$$e_i \longmapsto \{i-1,\, i\}, \qquad i = 1,2,3,4.$$

(2) 整体刚度矩阵（$N=4$，$h=0.25$，$1/h=4$）

$$A = \begin{pmatrix} 4 & -4 & 0 & 0 & 0 \\ -4 & 8 & -4 & 0 & 0 \\ 0 & -4 & 8 & -4 & 0 \\ 0 & 0 & -4 & 8 & -4 \\ 0 & 0 & 0 & -4 & 4 \end{pmatrix}.$$

两个角上的对角元是 $4 = 1/h$，不是 $8 = 2/h$。 原因：节点 x_0 只被单元 e_1 覆盖，只收到 $\frac{1}{h}\cdot 1$ 一份贡献；内部节点 x_1 同时被 e_1 与 e_2 覆盖，收到两份，故为 $\frac{2}{h}$。

(3) 写成“所有对角元都是 $2/h$”会怎样

会在两个角（A_{00}、A_{NN}）各多算一份 $1/h$。整体矩阵变成

$$A' = \begin{pmatrix} 8 & -4 & & & \\ -4 & 8 & -4 & & \\ & \ddots & \ddots & \ddots & \\ & & -4 & 8 & -4 \\ & & & -4 & 8 \end{pmatrix}.$$

这等价于把边界“部分地”当成了内部节点，解在两端附近出现 $O(1)$ 量级的偏差，而且随着 $h \to 0$ 这个偏差不会消失（它不在正确的离散问题里）。一个直观的检查：$A\mathbf{1}$ 应当是零向量（常向量在零空间），而 $A'\mathbf{1} = (4, 0, \ldots, 0, 4)^{\mathsf{T}} \neq 0$。

(4) 伪代码与自检

```
A ← 0; b ← 0
对 i = 1..N:
    gdof ← [i-1, i]
    A[gdof, gdof] += (1/h) * [[1,-1],[-1,1]]
    b[gdof]       += ∫ f φ 在 e_i 上
```

自检方法： 对均匀网格，另按闭式直接构造

$$\text{diag} = \left(\tfrac{1}{h}, \tfrac{2}{h}, \ldots, \tfrac{2}{h}, \tfrac{1}{h}\right), \quad \text{off-diag} = -\tfrac{1}{h},$$

与装配结果逐位比对。本节实测差值为严格的 0。

为什么比“看解像不像”可靠： 矩阵是离散问题的完整信息，不依赖右端项、不依赖求解器；而“解看起来平滑/接近”完全无法排除映射错误——错误的矩阵往往也能解出一条光滑曲线。

(5) 映射颠倒

若写 `gdof = [i, i-1]`，单元矩阵就被放到转置的位置上。由于 $K^{(e)}$ 对称，局部上看似没变，但累加时“哪一行对应哪个节点”整体错位，得到的 A 不再对称三对角（会出现带状错位）。

10 秒检查法： 打印 A 的前三行，或检查三条性质：① 对称；② 三对角；③ 角上 $= \frac{1}{h}$、内部对角 $= \frac{2}{h}$。任一不满足就说明映射错了。

题 44　边界处理的三种做法

(1) 未施加边界时 A 奇异

由装配规则，每一列的元素和为 $\frac{1}{h} - \frac{1}{h} = 0$（角列只有 $\frac{1}{h} - \frac{1}{h}$，内部列 $\frac{2}{h} - \frac{1}{h} - \frac{1}{h} = 0$），故 $A\mathbf{1} = \mathbf{0}$：常向量在 A 的零空间中，A 奇异。物理含义：纯 Neumann 型的问题解不唯一，解可以整体加一个常数。

(2) 三种操作

• 消元法：删去第 0、N 行与列，只解 $N-1$ 个内部自由度。　• 置 1 法：把第 0 行与第 0 列（A_{00} 除外）清零，$A_{00} \leftarrow 1$，$b_0 \leftarrow 0$；另一端同理。　• 置大数法：$A_{00} \leftarrow 10^{12}$，$b_0 \leftarrow 0$；另一端同理。

(3) 解的异同

做法	解	$u_h(0)$	条件数
消元法	参考	恒 0	最好（尺寸最小）
置 1 法	与参考相同	恒 0	与原矩阵同量级
置大数法	与参考差 $\sim 10^{-12}$	$\sim 10^{-12}$	显著变差（$\sim 10^{12}$ 的尺度引入）

(4) $N = 16$ 的实测

做法	与消元法的最大差	$u_h(0)$	$u_h(1)$
消元法	0	0	0
置 1 法	0	0	0
置大数法	9.38×10^{-13}	9.375×10^{-13}	9.375×10^{-13}

完整数据见 `数据/表3.3.3-2-边界处理.csv`。

(5) 陷阱辨析

忽略了两件事：精度与条件数。

• 置大数法把边界条件变成“强惩罚”，边界值只能到 $\sim 1/M = 10^{-12}$；　• 它让 A 的条件数恶化约 M 倍。当 h 变小时，有限元矩阵本身的条件数已按 h^{-2} 增长，再乘上惩罚因子，误差会更快放大。所以网格加密时置大数法会先出问题。

结论：消元法与置 1 法可用；置大数法只适合“临时凑一个能跑的结果”，不适合作为正式实现。

题 45　两个范数与收敛阶

(1) 两个范数与阶差

$$\|u-u_h\|_{L^2} = \left(\int (u-u_h)^2\right)^{1/2} = O(h^2), \qquad |u-u_h|_{H^1} = \left(\int (u'-u_h')^2\right)^{1/2} = O(h).$$

H^1 里对导数求误差。u_h 是分段线性的，它的导数在每段上是常数，用常数逼近 u' 只有 $O(h)$ 的精度，所以 H^1 的阶比 L^2 低一阶。

(2) 节点值精确及其前提

性质：　对 $-u''=f$ 与线性元，$u_h(x_i)=u(x_i)$ 对每个节点精确成立。

前提：　右端向量 $b_j=\int f\varphi_j$ 必须精确计算（或用足够高阶的求积，使求积误差低于机器精度）。

若用低阶求积：　不再成立。节点值只能精确到求积误差的阶——本组实测：用 4 点 Gauss 求积解 $\pi^2\sin\pi x$ 时，$N=4$ 的节点误差为 5.86×10^{-10}（而非机器精度），且随 N 迅速下降（$N=16$ 时 8.7×10^{-15}）。

(3) L^2 误差由什么决定

节点值精确 $\Rightarrow u_h$ 就是 u 的分段线性插值 $I_h u$。于是

$$\|u-u_h\|_{L^2}=\|u-I_h u\|_{L^2}\le \frac{h^2}{8}\max_{[0,1]}|u''|\cdot(\text{常数}).$$

对 $u=x(1-x)$，$|u''|=2$，故误差量级 $\sim \frac{h^2}{4}$。$N=64$（$h=1/64$）时 $\frac{1}{4}\cdot\frac{1}{4096}=6.1\times10^{-5}$，实测 4.46×10^{-5}，同一量级。

(4) 误差表

N	h	$\|u-u_h\|_{L^2}$	阶	$\|u-u_h\|_{H^1}$	阶	节点误差
4	0.2500	1.1411×10^{-2}	—	1.4434×10^{-1}	—	2.78×10^{-17}
8	0.1250	2.8527×10^{-3}	2.00	7.2169×10^{-2}	1.00	2.78×10^{-17}
16	0.0625	7.1318×10^{-4}	2.00	3.6084×10^{-2}	1.00	2.78×10^{-17}
32	0.0312	1.7830×10^{-4}	2.00	1.8042×10^{-2}	1.00	7.77×10^{-16}
64	0.0156	4.4574×10^{-5}	2.00	9.0211×10^{-3}	1.00	2.39×10^{-15}

完整数据见 数据/表3.3.3-3-收敛阶.csv。

(5) “阶是 2”为什么不够

因为阶依赖于范数。同一个解，L^2 下是 2 阶，H^1 下是 1 阶。只报“2”会让人误以为导数也以二阶收敛，而实际上

$$|u-u_h|_{H^1}=O(h)\quad(\text{一阶}),$$

本组实测观测阶恰为 1.00。正确说法是：L^2 二阶、H^1 一阶。

题 46　手算一次完整的有限元

(1) 整体刚度矩阵（$N=4$，$h=1/4$，$1/h=4$）

$$A=\begin{pmatrix}4&-4&0&0&0\\-4&8&-4&0&0\\0&-4&8&-4&0\\0&0&-4&8&-4\\0&0&0&-4&4\end{pmatrix}.$$

角上对角元是 $4=\frac{1}{h}$。

(2) 右端向量

$\int_0^1\varphi_j\,\mathrm{d}x=h$（内部节点），两端为 $h/2$。故

$$b=(0.125,\ 0.25,\ 0.25,\ 0.25,\ 0.125)^{\mathsf T}.$$

(3) 施加边界后求解

删去第 0、4 行与列，得

$$\begin{pmatrix} 8 & -4 & 0 \\ -4 & 8 & -4 \\ 0 & -4 & 8 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \begin{pmatrix} 0.25 \\ 0.25 \\ 0.25 \end{pmatrix}.$$

两边同除 4：

$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \begin{pmatrix} 0.0625 \\ 0.0625 \\ 0.0625 \end{pmatrix}.$$

由对称性 $u_1 = u_3$。第二式：$-u_1 + 2u_2 - u_1 = 0.0625 \Rightarrow 2u_2 - 2u_1 = 0.0625$。第一式：$2u_1 - u_2 = 0.0625$。解得

$$u_1 = 0.09375, \qquad u_2 = 0.125, \qquad u_3 = 0.09375.$$

(4) 与精确解比较

精确解 $u = x(1-x)/2$：

x	u_h	$u(x)$	差
0.25	0.09375	$0.25 \times 0.75/2 = 0.09375$	0
0.50	0.125	$0.5 \times 0.5/2 = 0.125$	0
0.75	0.09375	0.09375	0

节点值精确成立，与正文（8）一致。原因：$\int f\varphi$ 对 $f = 1$ 是可精确积分的，满足（8）的前提。u_h 就是精确解的分段线性插值。

(5) 程序复核

本组用与正文同一套装配代码，在 $N = 4$、$f = 1$ 上单独跑了一遍（该算例不在 3.3.3 的 Notebook 里，Notebook 用的是 $f = 2$：这一条是另行核对的）：装配矩阵与手算的（1）逐位一致，右端向量为 $(0.125, 0.25, 0.25, 0.25, 0.125)$，解出 $(0.09375, 0.125, 0.09375)$，施加边界后的残差

$$\|Au - b\|_\infty = 1.11 \times 10^{-16},$$

节点最大偏差 1.39×10^{-17}。全部与手算一致。

(6) 陷阱辨析

0.005 这个误差量级不正常。

• 对 $N = 4$、二次精确解，节点值应当精确；若出现 0.005 的偏差，它比机器精度大 13 个数量级，说明存在真实的实现错误，而不是“离散误差”。 • 最可能的原因：右端向量的 $\frac{h}{2}$ 处理错了（把两端也当成内部节点用了 h），这会让边界附近的解整体偏低；其次是装配时角上对角元写成了 $2/h$。 • 判据：由（4）可知正确的节点值应为 $(0.09375, 0.125, 0.09375)$；若得到 0.13，可直接反推右端向量错了 0.5×0.0625 这一份。

题 47　弱形式与边界项

（1）取 $v \in H^1$ 且 $v(0) = 0$（$x = 1$ 端不固定）。乘 v 并分部积分：

$$\int_0^1 (-u''+u)v\,dx = \int_0^1 (u'v'+uv)\,dx - \big[u'v\big]_0^1.$$

$v(0)=0$ 消去左端；$x=1$ 处的 $-u'(1)v(1)=-\beta v(1)$ 移到右边，得

$$\int_0^1 (u'v'+uv)\,dx = \int_0^1 fv\,dx + \beta v(1).$$

（2）$u(1)=\beta$ 是本质边界，会限制试探空间；本题给的是 $u'(1)=\beta$，是自然边界，只进右端，不限制空间。两者不能互换。

（3）$K^{(e)}=\frac{1}{h}\begin{pmatrix}1&-1\\-1&1\end{pmatrix}+\frac{h}{6}\begin{pmatrix}2&1\\1&2\end{pmatrix}$。

（4）加到最后一个节点分量 F_N：因为 $v(1)$ 只由最后一个帽函数 φ_N 取到，$\beta v(1)$ 对应 $\beta\varphi_N$。

（5）不成立。划行划列只适用于本质边界；自然边界要留在弱形式里，作为右端的边界项。把 $x=1$ 的行列划掉等于强行加了 $u(1)=0$。

题 48　快速题：对角元为什么不同

内部节点（$1\le i\le N-1$）：$A_{ii}=2/h$；端点（$i=0$ 与 $i=N$）：$A_{ii}=1/h$。差别来自覆盖次数：内部节点同时属于左右两个单元，各贡献 $1/h$；端点只属于一个单元，只收一份。

题 49　快速题：稀疏度

$N=4$ 时 A 是 5×5。非零元：对角 5 个 + 次对角 $2\times 4=8$ 个，共 13 个（约 $3N+1$）。而全场有 25 个位置——稀疏度已近 50%，N 增大时非零元只按 $O(N)$ 增长，稠密存储却是 $O(N^2)$。

题 50　快速题：未加边界的奇异性

每一列元素和为零（内部列 $\frac{2}{h}-\frac{1}{h}-\frac{1}{h}=0$，端列 $\frac{1}{h}-\frac{1}{h}=0$），故 $A\mathbf{1}=\mathbf{0}$：常向量在零空间中。物理上，没施加边界条件时解可以整体加一个常数，不唯一。

题 51　快速题：两个范数

L^2 更高（二阶）。H^1 范数里含导数，而 u_h 分段线性、其导数是分段常数，用常数逼近 u' 只有一阶，所以 H^1 低一阶。本节实测：L^2 观测阶恒为 2.00、H^1 恒为 1.00。

题 52　快速题：置大数法的代价

边界值只能到 $\sim 1/M$（取 $M=10^{12}$ 时即 10^{-12}），不是精确的零；同时 10^{12} 的尺度进入矩阵，条件数被显著恶化。本节实测：与消元法差 9.38×10^{-13}，$u_h(0)=9.375\times 10^{-13}$。

题 53　快速题：节点值精确的前提

前提是载荷 $\int f\varphi_j$ 被精确积分。若用数值求积，节点值只能精确到求积误差的阶：本节实测用 4 点 Gauss 求积时 $N=4$ 的节点误差为 5.86×10^{-10}，换成可精确积分的 $f=2$ 后才降到 2.78×10^{-17}。

题 54　找错题：装配漏掉非对角元

原因是只加了对角元，丢掉了非对角元。

单元刚度矩阵是 2×2 的 $\frac{1}{h}\begin{pmatrix}1&-1\\-1&1\end{pmatrix}$，装配时 4 个元素都要累加。上面的代码只加了 $A_{i-1,i-1}$ 与 A_{ii}，漏掉了 $A_{i-1,i}=A_{i,i-1}=-1/h$。结果得到一个对角矩阵——它对应的不是双线性形式 $\int u'v'$，而是某种“每个节点独立一根弹簧”的模型。弹簧之间不耦合，解自然整体偏小，而且加密网格也不会收敛到真解。

修法：整体加，不要逐元素手工加。

```
for i in range(1, N + 1):
    gdof = [i - 1, i]
    A[np.ix_(gdof, gdof)] += Ke        # 一次加 4 个元素
```

这道题要记住的： 手工逐元素装配时，“漏一个”非常容易发生，而且漏掉非对角元不会让程序报错——矩阵仍然对称、仍然可解，只是解的是另一个问题。可靠的做法是用 `np.ix_` 一次加整块，并配合“装配 vs 直接构造”的逐位比对。

题 55　开放题：五分钟查装配

一种合格方案（约 140 字）：

分两步。第一步，用另一种方式算同一个矩阵：均匀网格上直接构造“对角 $2/h$（端点 $1/h$）、次对角 $-1/h$”，与循环装配逐位比对——排除映射写反、漏加相邻单元。这一步最关键，因为错误矩阵照样能解出光滑曲线。第二步查三条性质：$A\mathbf{1}=0$（排除对角元写错）、角上是否为 $1/h$（排除按内部规则构造）、对称性（排除索引错位）。若还有时间，用 $f=2$、$u=x(1-x)$ 跑一遍看节点误差是否在 10^{-16} ——排除右端项错。核心思想：用两种独立方式算同一个对象，比盯着代码读有效得多。

不合格的回答长什么样：

• 只说“检查循环边界有没有 off-by-one”——对了一半，但漏掉了最有效的交叉验证； • 只看代码是否“看起来对称”——对称不保证正确（对角矩阵也对称）； • 直接跑一个算例看解“像不像”——错误的矩阵常常解出很光滑的解。

> 本节核对说明：3.3 的数值答案均可在配套实验复现——数据/表3.3.3-1-装配检验.csv、表3.3.3-2-边界处理.csv、表3.3.3-3-收敛阶.csv、ch3_3_3_results.json；题 43(4) 的“逐位比对”由代码执行（差值严格为 0），题 46 的手算结果由代码复核（残差 10^{-16}）。待 B/C 组核验： 节点值精确的前提条件、$A_{\text{int}}=hB$ 的因子、以及“边界决定整体阶”在二维是否仍成立。

第八部分　3.4.1 学习右端函数（题 56–63）

题 56　Lipschitz 条件与命题 1

(1) 三个右端的 Lipschitz 常数

（a）$f_1(u)=\tanh u$：$f_1'(u)=1-\tanh^2 u$，故 $|f_1'|\le 1$ 对一切 $u\in\mathbb{R}$ 成立。由中值定理 $|f_1(u)-f_1(v)|\le|u-v|$，常数 $L=1$，且是全局 Lipschitz。

（b）$f_2(u)=\sqrt{|u|}$：在带形 $|u|\le R$ 上不存在有限 Lipschitz 常数（见 (2)）。若把带形改成 $|u|\ge\varepsilon$（避开原点），则 $|f_2'|=\dfrac{1}{2\sqrt{|u|}}\le\dfrac{1}{2\sqrt{\varepsilon}}$，可取 $L=\dfrac{1}{2\sqrt{\varepsilon}}$。

（c）$f_3(u)=-\theta u$：$f_3'(u)=-\theta$，故 $|f_3(u)-f_3(v)|=\theta|u-v|$，常数 $L=\theta$，全局 Lipschitz，与 u 的范围无关。

三者的差别值得记牢：f_1 的导函数有界但非恒定，f_3 的导函数是常数，f_2 的导函数在原点处发散。“局部 Lipschitz”指的是在任给的一个固定区域上常数有限——f_1、f_3 都满足，f_2 在含原点的区域上不满足。

(2) f_2 为什么不是 Lipschitz

取 $u>0$、$v=0$，则

$$\frac{|f_2(u)-f_2(0)|}{|u-0|}=\frac{\sqrt{u}}{u}=\frac{1}{\sqrt{u}}\xrightarrow[u\to 0^+]{}+\infty.$$

差商无界，故不存在有限的 L 使 $|f_2(u)-f_2(0)|\le L|u|$ 对足够小的 u 成立。由于 $u=0$ 落在任何含原点的区域里，f_2 在含 $u=0$ 的任何区域上都不满足 Lipschitz 条件。

注意这不是“f_2 不连续”——f_2 处处连续。问题出在它在原点处的斜率无穷：Lipschitz 条件管的是斜率的上界，连续只管取值不断裂。

(3) 两个经典解与命题 1 被破坏的前提

- $u(t)\equiv 0$：$u'=0$，$\sqrt{|u|}=0$，逐点相等；$u(0)=0$。故是经典解。
- $u(t)=t^2/4\ (t\ge 0)$：$u'=t/2$，$\sqrt{|u|}=\sqrt{t^2/4}=t/2$，逐点相等；$u(0)=0$。故也是经典解。

同一个初值问题有两个经典解，解不唯一。命题 1 的前提有两条——f_θ 在 D 上连续、关于 u 满足 Lipschitz 条件。被破坏的是第二条：f_2 连续（第一条成立），但在原点附近不 Lipschitz。所以命题 1 失效的责任不在“连续”这一条，而在“Lipschitz”这一条。

这也正是正文图 5 左图的样子：从 $u=0$ 出发可以“等一会儿再起飞”，起飞时刻任意，于是有无穷多条解。

(4) 差商表

$f_2(u)-f_2(0)=\sqrt{u}$，差商为 $1/\sqrt{u}$：

表 2　$f_2(u)=\sqrt{|u|}$ 在原点处的差商

u	10^{-2}	10^{-4}	10^{-6}
$\dfrac{\lvert f_2(u)-f_2(0)\rvert}{u}$	10	100	1000

u 每缩小 100 倍，差商放大 10 倍，无界增长；这与 (2) 的解析判断一致——“差商无界”就是“不存在有限 Lipschitz 常数”。反过来说，只要把 u 限制在离原点 ε 以外，差商就被 $\varepsilon^{-1/2}$ 卡住，这正是“局部”二字的含义。

(5) 陷阱：连续不等于唯一

这句话错在把“连续”当成了“Lipschitz”。反例就是 (3)：$f_2(u)=\sqrt{|u|}$ 在 $\mathbb{R}$ 上连续，$u'=\sqrt{|u|}$、$u(0)=0$ 却有无穷多解。

唯一性真正需要的是关于 u 的 Lipschitz 条件（命题 1 的第二条）。$f_1=\tanh$、$f_3=-\theta u$ 都满足，所以它们的初值问题解唯一；f_2 不满足，于是不唯一。

对 Neural ODE 而言，这句话有直接后果：网络里的非光滑部件——$\sqrt{\cdot}$、$|\cdot|$、手工拼接的项、某些非光滑激活——都可能在某个状态上制造这种无穷斜率。带 $\tanh$、SiLU 的多层感知机是 Lipschitz 的（复合函数的常数是各层常数之积），命题 1 可以照搬；带 ReLU 的网络虽然仍 Lipschitz，但导数不连续，会让后面的伴随方程变脆。

常见错误： 用“f 可微”代替“Lipschitz”；或只验证了 f_2 在 $u\neq 0$ 处 Lipschitz 就下结论。关键就是原点那一点。

题 57　Picard 迭代与 ResNet 的层层堆叠

(1) Picard 迭代序列

积分方程 $u(t)=1+\int_0^t\big(-u(s)\big)\mathrm{d}s$，从 $u^{(0)}=1$ 出发：

$$u^{(1)}(t)=1+\int_0^t(-1)\,\mathrm{d}s=1-t,$$

$$u^{(2)}(t)=1+\int_0^t\big(-(1-s)\big)\mathrm{d}s=1-t+\frac{t^2}{2},$$

$$u^{(3)}(t)=1+\int_0^t\Big(-\big(1-s+\tfrac{s^2}{2}\big)\Big)\mathrm{d}s=1-t+\frac{t^2}{2}-\frac{t^3}{6}.$$

一般地

$$u^{(m)}(t)=\sum_{j=0}^{m}\frac{(-t)^j}{j!},$$

即 e^{-t} 的 Taylor 展开的前 $m+1$ 项。这是交错级数，部分和 $u^{(m)}(0.5)$ 来回摆动地逼近 $e^{-0.5}=0.6065307$：

m	1	2	3	4	5	6
$u^{(m)}(0.5)$	0.5	0.625	0.6041667	0.6067708	0.6065104	0.6065321

误差按 $t^{m+1}/(m+1)!$ 下降——比任何多项式都快，这是“Picard 迭代在解析函数上极快”的来源，也是 (4) 要提醒的：快只在能算出积分时成立。

(2) ResNet 堆叠

每一步 $u_{k+1}=u_k(1-h)$，$h=1/M$，故

$$u_M = \left(1 - \frac{1}{M}\right)^M, \qquad h = \frac{1}{M}.$$

填表：

M	10	100	1000	10^4
$u_M = (1-1/M)^M$	0.348678	0.366032	0.367695	0.367861
与 e^{-1} 之差	1.92×10^{-2}	1.85×10^{-3}	1.84×10^{-4}	1.84×10^{-5}
误差 $/h$（$h = 1/M$）	0.1920	0.1847	0.1840	0.1839

$e^{-1} = 0.3678794$。层数每乘 10，误差缩到 $1/10$——一阶收敛，与显式 Euler 的阶数一致。这不是巧合：$F(u) = -hu$ 就是 Euler 走一步，堆 M 层就是 Euler 走 M 步。

(3) 极限与误差常数

由 $M\log\left(1 - \frac{1}{M}\right) = -1 - \frac{1}{2M} - \frac{1}{3M^2} - \cdots$ 得

$$u_M = e^{-1}\left(1 - \frac{h}{2} + O(h^2)\right), \qquad h = \frac{1}{M},$$

故 $\lim_{M\to\infty} u_M = e^{-1}$，且

$$\left|u_M - e^{-1}\right| \approx \frac{e^{-1}}{2}\, h = 0.18394\, h.$$

表中最后一列为 $0.1920 \to 0.1847 \to 0.1840 \to 0.1839$，正是向 0.18394 收敛。误差 = 常数 $\times h$，常数是 $e^{-1}/2$：这解释了 (2) 里“层数每乘 10、误差降到 1/10”。

(4) 两种“取极限”

Picard 迭代取的是 $m \to \infty$，极限对象是积分方程的不动点，即精确解。每一步要做的是“对整条已知曲线求积分”，前提是这个积分能解析地算出来；一般 f 做不到，所以 Picard 是证明工具。

ResNet 堆叠取的是 $M \to \infty$（$h \to 0$），极限对象是离散格式的极限，即 ODE 的解。每一步只需在一点上算 f 的取值；代价是计算量 M 次、内存随 M 增长，而且误差只按 h 一阶下降。

一句话：Picard 逼近的是“解”，ResNet 逼近的是“流”；前者用分析换速度，后者用算力换通用性。

(5) 陷阱：无限加深不等于解出 ODE

错在两点。

第一，阶。$F(u) = -hu$ 是一阶块，误差 $O(h)$；要把终点误差压到 10^{-6}，按 (3) 需要 $M \approx e^{-1}/(2 \times 10^{-6}) \approx 1.8 \times 10^5$ 层。层数与精度是“线性换指数”的关系，靠加深堆不出来。

第二，误差不止一项。由命题 4，总误差 = 逼近 + 离散 + 优化。加深只压住离散项；逼近误差由网络容量与结构决定，优化误差由训练决定（§3.1）。三者混在一起时，“更深”完全可能“更差”——这正是第三章要拆开的账。

常见错误： 把“层数极限存在”当成“有限层就够准”；或忽略 ResNet 与 Neural ODE 的差别在于求解器可以自适应换步长，而不是层数堆得多。

题 58　流映射、半群性质与可逆性

(1) 流映射与半群性质

解 $u' = -\theta u$ 得 $u(t) = u_0 e^{-\theta t}$，故

$$\varphi(t, u_0) = u_0 e^{-\theta t}, \qquad \theta = 1.$$

取 $s = 0.3$、$t = 0.7$、$u_0 = 1$：

$$\varphi(1, 1) = e^{-1} = 0.367879,$$

$$\varphi\big(0.7, \varphi(0.3, 1)\big) = \varphi(0.7, e^{-0.3}) = e^{-0.3} \cdot e^{-0.7} = 0.740818 \times 0.496585 = 0.367879.$$

两个数到 6 位小数完全一致。代数上这是 $e^{-\theta s} e^{-\theta t} = e^{-\theta(s+t)}$，即指数函数的乘法公式。正文说“先走 s 再走 t 与一次走 $t+s$ 相同”，讲的就是这一条；它把“轨迹”变成了“流”。

(2) 可逆性与往返闭合

$\varphi(t, \cdot) : u_0 \mapsto u_0 e^{-\theta t}$ 是线性映射，乘数 $e^{-\theta t} \neq 0$，故可逆，逆映射为

$$\varphi(t, \cdot)^{-1} = \varphi(-t, \cdot) : \ x \mapsto x e^{\theta t},$$

也就是“倒着积分”。存在唯一性（命题 1）保证了反向积分同样有解，所以这条逆是良定义的。

Euler、$h = 0.5$、$\theta = 1$、4 步：

$$u_4 = 1 \times (1 - 0.5)^4 = 0.0625.$$

按逆映射“除回去”：$0.0625 \div 0.5^4 = 0.0625 / 0.0625 = 1.0000$。往返恰好闭合，一点误差都没有。

原因很简单：离散格式本身是一个可逆的线性映射，只要 $1 - h\theta \neq 0$，除法就能精确还原。但请注意——它闭合的是这套格式走过的路，不是真解的路：真解在 $t = 2$ 处是 $e^{-2} = 0.1353$，Euler 给了 0.0625，差 54%。可逆 $\neq$ 准确，这是 (4) 要考的点。

(3) 密度沿轨迹的演化

1 维时 $\dfrac{\mathrm{d}}{\mathrm{d}t} \log p_t\big(u(t)\big) = -\dfrac{\partial f}{\partial u} = \theta$，故沿轨迹

$$\log p_t\big(u(t)\big) = \log p_0(u_0) + \theta t \quad \Longrightarrow \quad p_t(x) = e^{\theta t}\, p_0\big(e^{\theta t} x\big).$$

取 $p_0 = \mathcal{N}(0, 1)$、$\theta = 1$、$t = 1$、$x = 0.5$：$e^t x = e \times 0.5 = 1.35914$，于是

$$\log p_0(1.35914) = -\frac{1.35914^2}{2} - \frac{1}{2}\log(2\pi) = -0.92359 - 0.91894 = -1.84257,$$

两条路：

- $\log p_1(0.5) = \log\big[e \cdot p_0(e \times 0.5)\big] = 1 + (-1.84257) = -0.84257$；
- $\log p_0(e^t x) + t = -1.84257 + 1 = -0.84257$。

两者相同，这就是“密度沿轨迹的变化率是 $-\nabla \cdot f$”的算式版本。再核对两条：$\int p_1 \, \mathrm{d}x = 1$（数值积分给出 1.000，质量守恒）；$\mathrm{Var}(u_1) = e^{-2} = 0.135335$（线性流把方差乘 $e^{-2\theta t}$）。这就是连续归一化流（CNF）的对数似然可以“顺着轨迹积分”而不必算 Jacobian 行列式的原因：散度就藏在轨迹的增长里。

(4) 陷阱：可逆不等于能还原真实初值

取 $\theta=1$、$h=3$，Euler 走两步：

$$u_2 = 1\times(1-3)^2 = 4.$$

逆映射照样存在（除以 $(-2)^2$ 得 1），往返仍然闭合；但真解在 $t=6$ 处是 $e^{-6}=0.00248$，数值解给了 4——差了三个数量级，方向都不对（解该衰减，它却增长）。

问题出在 $|1-h\theta|=2>1$：放大因子的模大于 1，数值解在发散。可逆性只要求乘数非零，稳定性要求乘数模不超过 1，这是两个独立条件。所以“反向积分能精确还原”这句话，只有在那套格式本身稳定、且我们想还原的就是这套格式的解时才成立；想还原真解，还得让 $h\to 0$。

常见错误： 用“映射可逆”推断“数值解可信”；或把可逆性当成 CNF 的充分条件（CNF 还需要可积、散度可算、训练能收敛）。

题 59　伴随法的内存账

(1) 离散反传的内存

前向 M 步、每步存一个 d 维状态，需要 Md 个浮点数。代入 $d=256$、$M=10^5$：

$$Md = 2.56\times10^7 \text{ 个数},\qquad 2.56\times10^7\times 4\,\mathrm{B} = 1.024\times10^8\,\mathrm{B}\approx 97.7\,\mathrm{MiB}.$$

注意这是下界：实际训练中还要存每步的中间激活，真实占用通常再大几倍。

(2) 伴随法与分段检查点

表 3　三种反向方式的内存（$d=256$、$M=10^5$、单精度）

方式	浮点数个数	字节数	相对离散反传
离散反传	$Md=2.56\times10^7$	1.024×10^8（97.7 MiB）	1
分段检查点（$k=\sqrt{M}=316$）	$2\sqrt{M}\,d=1.62\times10^5$	6.48×10^5（632.5 KiB）	$1/158$
连续伴随	$2d=512$	2.05×10^3（2.0 KiB）	$1/5\times10^4$

伴随法只需要“当前状态 + 当前乘子”两组量，与 M 无关；分段检查点存 k 个段首状态、重算时再临时生成一段（M/k 步）的量，故内存 $\propto d\left(k+M/k\right)$，在 $k=\sqrt{M}$ 处取最小 $2\sqrt{M}\,d$——这就是 $\sqrt{M}$ 的来历。

(3) 1 GiB 显存能走多少步

每步 $d=1024$ 个浮点数、每数 4 B，即 4096 B。$1\,\mathrm{GiB}=1.0737\times10^9$ B，于是

$$M \le \frac{1.0737\times10^9}{4096}\approx 2.6\times10^5 \text{ 步}.$$

伴随法只用 $2\times1024\times4\,\mathrm{B}=8\,\mathrm{KB}$，与 M 无关，这一条限制对它不成立；但它的计算约为前向的两倍，且反向积分本身要重算整条轨迹，时间随 M 线性增长。内存换成了时间，账没有凭空消失。

(4) 两种梯度为什么不严格相等

式 (5) 的顺序是“先连续求导、再离散积分”：把伴随方程 (4) 当连续问题，用某个积分器数值求解；离散反传的顺序是“先离散前向、再对这个离散计算图求导”。前者算的是连续梯度的近似，后者算的恰好是那次前向计算图上的精确梯度，二者相差 $O(h^p)$（p 为积分器阶数）。

对训练的含义：伴随法的梯度与“当前这套步长”耦合较松，训练中途换步长、换格式时，梯度会跟着变，loss 会“抖一下”；离散反传的梯度与那次前向严丝合缝，但内存随步数线性增长。所以工程上按区间长度选：短区间直接反传，长区间用伴随，中间用 checkpointing。

(5) 陷阱：常数内存不是免费

错在把“$O(d)$”读成了“$O(1)$”。伴随法的内存随维数 d 线性增长：$d = 10^6$ 时要 $2d \times 4\,\mathrm{B} = 8$ MB，不算大；真正的问题是另外三条——(i) 计算量约为前向的两倍；(ii) 反向积分本身有离散误差，梯度精度受积分器控制；(iii) 实现复杂度高（要写出 $\partial f/\partial u$ 与 $\partial f/\partial \theta$ 的向量积）。

还有一条常被忽略：伴随法要求 f_θ 对 u 可微且 Jacobian 有界。ReLU 网络在原点处的不可微、以及大 L 带来的放大，都会让伴随方程“变脆”。所以“d 可以随便加大”这句话，在内存维度上或许成立，在精度与实现上并不成立。

评分要点： (1)(2) 三个字节数要算对（97.7 MiB / 632.5 KiB / 2.0 KiB）；(3) 要给出 2.6×10^5 这个量级；(4) 要说出“先连续后离散”与“先离散后连续”的顺序差别及 $O(h^p)$。

题 60　快速题：定义 1 与参数化向量场

定义 1。 设 $f_\theta : \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}^d$ 是由参数 θ 决定的神经网络，则初值问题

$$\frac{\mathrm{d}u_\theta}{\mathrm{d}t} = f_\theta(t, u_\theta), \qquad u_\theta(0) = u_0$$

的解 $u_\theta(t)$ 称为该网络给出的轨迹；f_θ 本身称为参数化向量场。式 (2) 与式 (1) 形状完全相同，只是把 f 换成了 f_θ——这一步替换带来下面两条后果。

好消息： 学到的是向量场而不是一条曲线。f_θ 一旦定下，任意初值、任意区间都可以重新积分得到轨迹（流映射的不同切片），“换初值外推”不需要额外本事。

坏消息： 式 (2) 一般没有解析解，轨迹只能数值积分，于是积分器的离散误差会混进训练——参数会把误差“背”进去（§1.4 的 $\hat\theta = 0.787$ 就是例子）。

题 61　快速题：网络 Lipschitz 常数

(1) $f'(u) = 3 \cdot \big(1 - \tanh^2(2u+1)\big) \cdot 2$，故 $|f'| \le 6$，$L \le 6$。

(2) 复合函数的 Lipschitz 常数不超过各层常数之积，$\tanh$ 的常数为 1，故 $L \le s_1 s_2 \cdot 1 = 2 \times 0.5 = 1$。

(3) $L \le 1.2^{20} = 38.34$。也就是说，输入相差 10^{-2}，输出可能相差 0.38 量级；若再做几层，乘积指数增长。这就是正文“深层网络对输入极敏感”的来源，也是谱归一化把每层 s_i 压到 1 附近的动机。

附带一条：L 越大，连续依赖估计 $|u(t)-z(t)| \le |u_0-z_0|e^{Lt}$ 越松，这也是第三章“换初值验证”的动机之一。

题 62　快速题：连续依赖与 Lorenz

（1）由连续依赖估计 $|u(t)-z(t)| \le |u_0-z_0|e^{Lt}$：

$$t=1:\quad 10^{-3}e^{10}=2.20\times 10^{1};\qquad t=0.1:\quad 10^{-3}e^{1}=2.72\times 10^{-3}.$$

注意 $t=1$ 那个界（22）比初值差大了四个数量级——界本身没有说“误差一定这么大”，它说的是“最多这么大”。

（2）要求 $10^{-3}e^{14L}\sim O(1)$，即 $e^{14L}\approx 10^{3}$，故

$$L\approx \frac{\ln 10^{3}}{14}=\frac{6.908}{14}\approx 0.49 \text{ 每天}.$$

这个数和大气的实际 Lyapunov 指数（约 0.5/天）同阶。于是 Lorenz 的故事可以完全用本节的公式读：10^{-3} 的初值差，在 $L\approx 0.5$ 的系统里两周就长到 $O(1)$，所以两周以上的天气预报只能给概率。

题 63　快速题：伴随法的代价

命题 2（伴随法的代价）。 用伴随法求一次梯度，需要额外解一个与状态同维的反向 ODE，内存为 $O(d)$、与时间步数无关；计算量为前向积分的约两倍。

为什么与步数无关。 离散反传要沿时间逐层回传，必须把 M 个中间状态都留着；伴随法把“回传”换成了“再解一个 ODE”：反向积分时，每一步只要有当前状态 $u(t)$ 与乘子 $a(t)$，就能往下推，其余状态按需重算。于是内存是 $2d$（状态与乘子），与 M 无关；代价是重算——计算翻倍。

$\sqrt{M}$ 从哪来。 分 k 段、每段 M/k 步：先存 k 个段首状态（kd 个数）；反传某一段时，从该段段首把这一段重算一遍，重算过程中另需 $\frac{M}{k}$ 个状态作为临时量（峰值 $\frac{M}{k}d$ 个浮点数）。合计

$$\text{内存}\ \propto\ d\Big(k+\frac{M}{k}\Big),$$

对 k 求极小：$1-\frac{M}{k^2}=0\Rightarrow k=\sqrt{M}$，最小值 $2\sqrt{M}\,d$。计算量约为前向的两倍（每段重算一次）。

第九部分　3.4.2 轨迹拟合与方程残差（题 64–68）

题 64　残差、推论 1 与“配点为零”的假象

(1) 残差与配点损失

$u_\theta(t)=1+t$，$u_\theta'=1$，$f(t,u_\theta)=u_\theta=1+t$，故

$$r_\theta(t)=1-(1+t)=-t.$$

残差损失（均方）：

- 配点 $\{0\}$：$r(0)=0$，损失 0。
- 配点 $\{0,0.5,1\}$：$\frac{0+0.25+1}{3}=0.4167$。
- 配点 $\{0,0.25,0.5,0.75,1\}$：$\frac{0+0.0625+0.25+0.5625+1}{5}=0.375$。

(2) 推论 1 的界，以及为什么取等号

$L=1$、$|r_\theta(s)|=s$，故

$$|u_\theta(t)-u^\star(t)| \le \int_0^t s\,e^{t-s}\,\mathrm{d}s.$$

这个积分有闭式：分部积分得 e^t-t-1。$t=1$ 时为

$$\int_0^1 s\,e^{1-s}\,\mathrm{d}s = e-2 = 0.718282.$$

而真实误差是 $u_\theta(1)-u^\star(1)=2-e=-0.718282$，绝对值恰好相等。

原因是这个算例把推论 1 证明里的每一步不等式都取成了等号：记 $\eta=u_\theta-u^\star$，则 $\dot\eta=r_\theta+\eta$，这里 $r_\theta=-s<0$ 且 $\eta<0$（模型偏低），两者同号，故 $|\dot\eta|=|r_\theta|+|\eta|$ 取等；再乘 e^{-Lt}、积分，也一路取等。推论 1 是紧的——至少在“残差不变号、且误差沿同一方向累积”时，它给出的就是精确的误差。

(3) 放大因子

$L=1$ 时 $(e^{Lt}-1)/L=e^t-1$：

t	1	2	3
(e^t-1)	1.718	6.389	19.086

若 $|r_\theta|\equiv 0.01$，误差上界为 $0.01(e^t-1)$：$t=1$ 时 0.0172，$t=2$ 时 0.0639，$t=3$ 时 0.191。

注意这个因子长得比线性快得多：残差同一个 1%，在 $t=3$ 处已经允许 19% 的误差。正文“残差 1% 与解差 1% 之间没有等号”说的就是这个数。

(4) 配点表

表 4　同一个模型、不同配点集的残差损失与终点误差

配点集	$\{0\}$	$\{0,0.5,1\}$	$\{0,0.25,0.5,0.75,1\}$
残差损失	0	0.4167	0.375
$t=1$ 处误差	0.7183	0.7183	0.7183

关键一行是第二行：误差一动不动。配点从 1 个加到 5 个，残差损失从 0 涨到 0.375（终于“看见”了错），可模型本身是固定的 $u_\theta=1+t$，它错多少与配点无关。而第一列的 0 更是假象：只在 $t=0$ 抽查一次，恰好抽在残差穿过零点的地方。

这正是命题 3 的前提“$r_\theta\equiv 0$”与式 (9)“在有限个配点上算残差”之间的鸿沟：命题 3 要求处处为零，配点法只能抽查。

(5) 陷阱：残差小不等于误差小

错在丢掉放大因子。由推论 1，若 $|r_\theta|\le\varepsilon$，则

$$|u_\theta(t)-u^\star(t)| \le \frac{\varepsilon}{L}\left(e^{Lt}-1\right),$$

取 $L=1$、$t=3$，这个因子是 19：残差 1% 允许误差 19%。L 越大、区间越长，放大越狠（正文图 10）。

另一方面，“残差 1%”本身也可能只是配点抽查的结果（见表 4 第一列）。要让它有意义，至少得：残差在整个区间上一致地小、L 是有限且被估出来的、并且用独立配点复核。

常见错误： 把“训练集上的残差损失”当成“解的误差”；或只报残差、不报 L 与区间长度。

题 65　两条路线与量纲

(1) 两种损失学的是什么

式 (3)：$\mathcal{L}_{\text{traj}}(\theta)=\frac{1}{N}\sum_i\|u_\theta(t_i)-u_i\|^2+\lambda\|u_\theta(0)-u_0\|^2$。它只要求轨迹经过观测点，学的是右端 f_θ；需要的是“u 的若干次观测”，方程可以是未知的。曲线只是这个向量场的一个投影。

式 (9)：$\mathcal{L}_{\text{pinn}}$ 由三项组成——方程残差、初边值、数据。第一项里出现的是真方程 f，不是网络，所以学的是解 u_θ；需要的是“已知的方程 + 少量边界/数据”。

这一分野决定了适用条件：有方程、数据稀少时用式 (9)；有数据、方程未知时用式 (3)；两者都有，就把三项都打开，用 λ 调比例。

(2) 三个场景

(a) 热电偶 + 已知热方程：用式 (9)。方程是确定的，观测稀疏，正好用残差项在无观测处“补”信息，用数据项把场校正回来——工程上“数字孪生”的骨架就是这个组合。

(b) 只有一条 S 形曲线：用式 (3)。连方程是什么都不知道，只能让网络把 f 猜出来；此时学到的 f 只有“这条曲线覆盖到的地方”才可信。

(c) 方程 + 较多数据：三项全开。残差项保证物理一致，数据项压制噪声，边界项钉住常数；比例由 λ 调（题 66）。

(3) 量纲实验

若 $|r_\theta|\sim 10^{-3}$，残差项 $\sim(10^{-3})^2=10^{-6}$；数据项的偏差 ~ 0.1，故数据项 $\sim 10^{-2}$。两项相差

$$\frac{10^{-6}}{10^{-2}}=10^{-4},$$

要让两边“分量相当”，需要 $\lambda_1\sim 10^4$。

问题在于这个 10^4 不是普适常数：换个单位制（用毫米代替米）、换个时间尺度，它立刻变成别的数，而训练过程对 λ 的量级很敏感（题 66）。先无量纲化——把方程除以特征量，让扩散项与对流项都变成 $O(1)$——就把 λ 拉回 $O(1)$ 附近。这是全部调参里最便宜的一步：一次代数变形，换掉后面几天的调参。

(4) 三种写法

强形式：直接要求 $r_\theta\equiv 0$，即式 (9) 第一项。要求 u_θ 二阶可导、方程逐点成立；实现简单，但二阶导的代价与噪声敏感度都高。

弱形式：只要求对任意测试函数 φ 有 $\int r_\theta\varphi\,\mathrm{d}t=0$（或等价的变分形式）。对光滑性要求低（一阶导甚至不导），天然对接有限元式离散；代价是积分更贵、需要选测试空间。

Deep Ritz：不算残差，直接最小化能量泛函。连“残差”都省了，代价是必须先有变分结构——不是每个方程都有（这一点把一大批方程排除在外）。

(5) 陷阱：三项损失调好就一定收敛

错。λ 的尺度问题只是第一层。还有三层：(i) 配点密度——残差只被抽查（题 67）；(ii) 量纲——两项数值不可比（(3)）；(iii) 优化路径——即使权重合适，“先满足谁”仍会决定收敛到哪个局部极小。正文引的 NTK 分析说的就是：核尺度差几个数量级时，优化器只顾一头，表现出来就是“残差降得很快、边界始终不对”。

所以正确的说法是：λ 决定“收敛到哪个解”，不只是“收敛多快”。

题 66　权重的代价

(1) NTK 的读法

把训练看作在函数空间里的梯度流：每一项损失下降的“速度”由该项对应的核特征值决定。若残差项的核尺度比边界项小几个数量级，则同一个参数更新里，残差方向的梯度分量远小于边界方向，优化器就把时间几乎全花在边界上——或者反过来。表现出来是“一项降得飞快、另一项纹丝不动”，不是学习率的问题，而是两项在同一个参数空间里的尺度差。

这与“一个学生两门课、期末按 99:1 加权”是同一件事：学分给歪了，他自然偏科。

(2) 梯度范数平衡

比值为 $\|\nabla L_r\|/\|\nabla L_b\| = 1/100 = 10^{-2}$，故每步乘 $0.01^{0.9} = 10^{-1.8} = 1.585\times10^{-2}$：

k	0	1	2	3
λ_k	1	1.585×10^{-2}	2.512×10^{-4}	3.981×10^{-6}

三步之内 λ 掉了五个多数量级，正好把两边的梯度拉到同量级。$\alpha = 0.9$ 而不是 1，是为了避免在梯度范数估计有噪声时来回震荡。

(3) 不确定性加权

记 L_i 为第 i 项的损失值，$g(\sigma) = L_i/\sigma^2 + \log\sigma^2$。求导：

$$g'(\sigma) = -\frac{2L_i}{\sigma^3} + \frac{2}{\sigma} = 0 \quad\Longrightarrow\quad \sigma^2 = L_i.$$

即模型学到的 σ_i 恰是该项损失的均方根，权重 $\dfrac{1}{\sigma_i^2} = \dfrac{1}{L_i}$。对 $L_r = 0.01$、$L_b = 1$：

$$\sigma_r = 0.1,\ \sigma_b = 1, \qquad \frac{w_r}{w_b} = \frac{1/0.01}{1/1} = 100:1.$$

损失小的那一项反而被加权 100 倍——看似反直觉，其实与 (2) 同向：损失小一般也意味着梯度小，而“拉平梯度尺度”的做法正是给梯度小的一项加大权重。两个公式，一个目标。

(4) 三条对策各治什么

• 自适应权重（梯度范数平衡 / 不确定性加权）：治“两项尺度差几个数量级”。不再手调 λ，让它跟着梯度的实际尺度走。

• 硬约束（如令 $u_\theta(t) = g(t) + (t-a)(t-b)N_\theta(t)$）：治“边界项永远差一点”。边界条件恒满足，$\lambda_1$ 直接消掉，优化器没有机会“偏科”。

• 分阶段训练（先边界、再残差）：治“训练初期优化路径混乱”。先把解的大形定住，再放开方程约束。

三条都不保证收敛，但都能显著降低“调参的运气成分”。

(5) 陷阱：λ 只影响速度

错。λ 改变的是目标函数本身，因此改变的是“收敛到哪个解”。极端情形很容易造：$\lambda_1 = 0$ 时，边界违约完全不计入损失，网络可以给出残差处处为零但初值全错的解（命题 3 的前提“同初值”被破坏）。反过来 $\lambda_1 \to \infty$ 时，边界完美、方程被忽略，就成了一个普通插值器。

正文引的文献把这个现象叫“PINN 训不动”，而不是“PINN 训得慢”——差别就在“不动”的项是被尺度压住的，不是被学习率拖慢的。

评分要点： (2) 要算出 1.585×10^{-2} 并说明它是 $0.01^{0.9}$；(3) 要证到 $\sigma^2 = L_i$ 并给出 $100:1$；(5) 要说出“改变目标函数、改变收敛到的解”。

题 67　找错题：两个写坏的残差损失

(1) 版本 A：只有一个配点

$u_\theta = 1 + t$、$f(t,u) = u$：$r_\theta(0) = 1 - 1 = 0$，故残差损失为 0，优化器“看到”的是完美拟合，梯度为零、参数不再更新。

而 $t = 1$ 处：模型给 2，真解 $e = 2.71828$，绝对误差 0.7183，相对误差 26.4%。

这说明：残差损失是抽查，不是证明。一个配点恰好落在残差零点上（$t = 0$ 是残差 $r = -t$ 的零点），损失为零却与“解对”无关（命题 3 要求的是处处为零）。

(2) 版本 B：float32 下的前向差商

$u(t) = e^{-t}$，真值 $u'(1) = -0.367879$。float32 下（机器精度 $\varepsilon = 1.19 \times 10^{-7}$）：

h	差商给出的 $u'(1)$	相对误差
10^{-4}	-0.36776	0.03%
10^{-6}	-0.32783	10.9%
10^{-8}	0	100%

原因是相减抵消：$u(t+h) - u(t) \approx h\,u' \approx 3.7 \times 10^{-9}$，而 $u \sim 1$ 的表示误差是 $\varepsilon|u| \approx 1.19 \times 10^{-7}$——差值比表示误差还小，被舍入整个吃掉。$h = 10^{-8}$ 时两个数在 float32 里是同一个数，差为 0，残差就成了 $-f$，损失变成 $|f|^2$ 的常数，梯度全部失真。

可用步长的量级是 $h \sim \sqrt{\varepsilon}\,\dfrac{|u|}{|u'|} \approx 3.4 \times 10^{-4}$（表的第一行已经落在这个量级）。所以别用有限差分算残差——PINN 的残差要用自动微分，它不经过“相减”这一步。

(3) 两类错误与改法

• 版本 A 是错误定义：配点密度不足/覆盖不当，监督信号本身错了。改法：配点覆盖整个区间（随机或分层采样）、加边界与数据项、并用独立配点复核（正文小实验 1 的做法）。

• 版本 B 是数值实现：公式没错，是算错了。改法：用自动微分求 u_t；若坚持差分，步长取 $\sim\sqrt{\varepsilon}$ 量级，并在 float64 下算。

常见错误： 看到残差损失降到 10^{-6} 就认为“残差很小”——版本 A、B 都会给出很小的损失，而解错得很远。

题 68　开放题：算子学习要不要做

(1) 成本账

逐个求解：$10^4 \times 10\,\mathrm{s} = 10^5\,\mathrm{s} \approx 27.8$ 小时。

算子网络：训练 2×10^4 s，推理 $10^4 \times 10^{-3} = 10$ s，合计 2.001×10^4 s ≈ 5.6 小时。

加速比 $10^5 / 2.001 \times 10^4 \approx 5.0$。盈亏平衡：

$$N^* = \frac{2 \times 10^4}{10 - 10^{-3}} \approx 2000 \text{ 组.}$$

即参数组数低于 2000 时，逐个求解更划算；高于它，训练算子网络开始回本。

(2) 优势与前提

优势：一次训练服务一族参数。给定参数、边界、初值，直接给出整条解（甚至整个场），不必重新迭代；推理快，可以塞进优化循环、实时控制、反问题。

前提：必须先有一批高质量的数值解当训练数据（这就是“数据换速度”），且参数/边界的分布与训练分布一致。分布外推是它的弱点——这与题 71 的“换区间”是同一类问题。

(3) 基准解决了什么

解决了可比性与可复现：同一批算例、同一套指标，“误差 10^{-3}”才有统一含义；换随机种子、换优化器导致的量级差异也因此暴露。正文说“基准存在的意义就在这里”。

没有解决：分布外推、多尺度/刚性/高维的困难，以及“可证明的误差界”。基准只让经验误差可比，不提供保证。

(4) 只做一件事

把三换做全并如实报告（换初值、换步长、换区间），同时给出误差分解（逼近/离散/优化/数据）。理由：单点误差可以靠调参“刷”出来，三换之后的漂移才是模型真正学到了什么。这与“过拟合排行榜”的道理相同——凡是调参时看过的数据，都不算独立验证。

(5) 一句话结论

有方程、数据稀少时用 PINN，有数据、方程未知时用 Neural ODE，要在成千上万组参数上反复求解时才值得训练算子网络；三者的可信度都不由训练损失决定，而由独立装置上的漂移决定。

评分要点（四问各 2.5 分）： (1) 要算出 5.0 与 $N^* \approx 2000$；(2) 要说出“需要数值解当数据”这一前提；(3) 要区分“可比”与“有保证”；(4) 要给可执行的验证动作，而不是“多训练”。

第十部分　3.4.3 求解器误差与独立轨迹验证（题 69–73）

题 69　误差分解

(1) 三项与“只能靠换装置测”的那一项

命题 4：

$$\|\hat{u}-u^\star\| \le \underbrace{\|u_{\theta^\star}-u^\star\|}_{\text{逼近}} + \underbrace{\|\hat{u}-u_{\theta^\star}\|}_{\text{离散}} + \underbrace{\|u_{\theta^\star}-u_{\hat{\theta}}\|}_{\text{优化}}.$$

三项分别对应：网络表达能力不够（逼近）、积分器走得不准（离散）、参数没训到位（优化）。

离散误差只能靠换装置测。逼近与优化误差都“属于模型本身”，换积分器改不了它们；只有离散误差随积分器与步长变化——把 h 减半、把 Euler 换成 RK4，结果的变化量就是它的直接读数。这也是正文说“换一个积分器重算，就能把离散误差单独量出来”的依据。

(2) 三项分解表

$u_{\hat{\theta}}(2)=e^{-2\hat{\theta}}=e^{-1.574}=0.20722$，$u^\star(2)=0.135335$，$u_{\theta^\star}=u^\star$（模型类含真解，逼近误差为 0）。

表 5　$T=2$ 处三种误差的带符号分解（$\theta^\star=1$、$\hat{\theta}=0.787$）

复算方式	$\hat{u}(2)$	离散误差 $\hat{u}-u_{\hat{\theta}}$	优化误差 $u_{\hat{\theta}}-u_{\theta^\star}$	总误差 $\hat{u}-u^\star$
Euler $h=0.5$	0.13531	−0.07191	+0.07188	−0.00003
Euler $h=0.25$	0.17330	−0.03392	+0.07188	+0.03797
Euler $h=0.125$	0.19073	−0.01649	+0.07188	+0.05539
RK4 $h=0.5$	0.20731	+0.00009	+0.07188	+0.07197
RK4 $h=0.25$	0.20722	+0.00000	+0.07188	+0.07189

三处值得讲：

• Euler 的离散误差全为负（解被算小了），且随 h 线性缩小：$-0.0719 \to -0.0339 \to -0.0165$，比值约 2.12、2.06。这是一阶格式的指纹。

• 优化误差恒为 $+0.0719$，与 h 无关——它属于参数，换积分器不会改。

• RK4 一行的离散误差几乎为零，总误差就等于优化误差。换到 RK4，账就只剩参数那一笔：这正是正文表 1 最后一列“$+53\%$”的来历。

(3) 为什么 $h=0.5$ 的 Euler 总误差几乎为 0

因为两项符号相反、大小几乎相等：离散误差 -0.0719 把解往下拉，优化误差 $+0.0719$ 把解往上抬，相加抵消到 -0.00003。

这个抵消的根源在 §1.4：$\hat{\theta}$ 就是拿 Euler($h=0.5$) 在观测点 $(0.5, 0.6065)$ 上反解出来的，所以这套装置在训练点必然对齐；又因为 $0.6065 \approx e^{-0.5}$，四次相乘恰好给出 e^{-2}，于是 $t=2$ 也几乎对齐。这是数值巧合，不是精度——表格第 2、3 行说明：h 一改，抵消立刻失效。

命题 4 用绝对值写，所以它给的是上界：三项都非负时，$|\text{总}| \le |\text{离散}| + |\text{优化}| + |\text{逼近}| = 0.1438$，而实际只有 0.00003。带符号的分解是等式，带绝对值的分解是不等式——前者解释“为什么抵消”，后者只保证“不超过”。

(4) 陷阱：误差分解不能指导做法

错。分解的用处恰恰是指路：

• 若换步长之后结果几乎不变、换初值就崩 → 离散误差不是主项，问题在逼近或数据（该换网络结构、加数据）。

• 若换步长漂移很大 → 离散误差主导，此时讨论“网络学得好不好”没有意义；该缩小步长或提高格式阶数。

• 若换积分器不变但 $h \to 0$ 仍不收敛到真解 → 优化没到位（或参数不可辨识）。

换句话说：只报一个总误差，无法判断该修哪一头；三项分开报，才知道钱该花在哪。这正是正文 3.1 节开头“混在一起就会得出误差 10^{-4} 这种没有信息量的结论”的意思。

评分要点： (2) 表中符号不能写反（离散负、优化正）；(3) 要说出“$\hat{\theta}$ 是 Euler($h = 0.5$) 反解出来的”这个因果，而不是“Euler 精度高”。

题 70　换一套装置漂多少

(1) 复算表

表 6　固定 $\hat{\theta} = 0.787$，换装置重算 $t = 2$ 的值（真值 0.135335）

复算方式	h	$u(2)$	相对真值的偏差
Euler	0.5（训练时）	0.13531	-0.02%
Euler	0.25	0.17330	$+28.1\%$
Euler	0.125	0.19073	$+40.9\%$
Euler	0.0625	0.19908	$+47.1\%$
RK4	0.5	0.20731	$+53.2\%$
RK4	0.25	0.20722	$+53.1\%$

$u(2) = e^{-2\hat{\theta}} = 0.20722$ 是模型自己的真值（解析解）。Euler 的序列 $0.135 \to 0.173 \to 0.191 \to 0.199$ 单调爬向它，$h \to 0$ 时落在 0.2072，即“$+53\%$”。RK4 一步到位：不论步长都给出 0.2072。

(2) 换步长漂移量

$$|u(2;0.5) - u(2;0.25)| = 0.0380, \qquad |u(2;0.25) - u(2;0.125)| = 0.0174, \qquad |u(2;0.125) - u(2;0.0625)| = 0.0084.$$

每把 h 减半，漂移也减半（$0.0380 \to 0.0174 \to 0.0084$），一阶收敛。第一步的漂移 0.0380 已是真值 0.1353 的 28%——比 1% 量级的噪声大了近三十倍，不可能被忽略。

(3) 为什么训练装置恰好落回真值

两层原因叠加：

第一，$\hat{\theta}$ 是按 Euler($h=0.5$) 反解的，定义上就让这套装置在训练点 $(t=0.5)$ 精确复现观测值 0.6065；这不是“算得准”，是“按这把尺子校准过”。

第二，观测值 0.6065 恰好取的是 $e^{-0.5}$，而 $t=2$ 要走 4 步，$0.6065^4=0.13531\approx e^{-2}$。于是终点也几乎对上——这是数值巧合。若把观测点换到 $t=0.4$、或把真解换成别的衰减率，这个巧合立刻消失。

换成任何其它装置，两层都失效：h 变了，尺子刻度变了；格式变了（RK4），校准也不再成立。剩下的就是模型自己的真值 0.2072 与真解的差距 53%。

(4) 报告哪一个数

报告**“换装置后的漂移”**（例如“Euler $h=0.5\to0.25$ 漂移 28%”或“RK4($h\le0.5$) 复算得 0.2072，与训练装置相差 53%”），而不是训练损失。

理由：训练损失是“按这把尺子校准”的产物，定义上接近零；漂移量才反映参数里背了多少离散误差。这与“离线指标漂亮、一上线就崩”是同一件事——上线换的是装置，不是数据。

评分要点：(1) 六个值要与表中一致；(2) 要指出漂移随 h 一阶下降；(3) 要说出“反解”与“$0.6065=e^{-0.5}$”两层原因，只答“Euler 精度低”算半对。

题 71　三换协议：偏差藏在哪里

(1) 三换各查什么

- 换初值：查“学到的是向量场还是那一条曲线”。若 f_θ 学对了，任何初值的轨迹都该贴合；若只记住了曲线，换初值立刻崩。
- 换步长：查“离散误差有没有被吸收进参数”（表 6 就是它的读数）。
- 换区间：查“能不能外推”。内插段通过之后，向两端各推 20%，看误差涨多少。

发现不了的东西也要说清楚：换初值对“乘性偏差”是盲的（见 (2)），换步长对“参数本身就错但形式对”是能发现的，换区间对“训练点附近的系统性偏差”不敏感。

(2) 换初值：三行全是 $+53.1\%$

模型的解析解 $u_0e^{-\hat{\theta}t}$ 与真解 u_0e^{-t} 之比恒为 $e^{(\theta^\star-\hat{\theta})t}=e^{0.213\times2}=1.531$，与 u_0 无关。

u_0	0.02	0.05	0.2
模型的解析解 $u_0e^{-\hat{\theta}t}$	0.004143	0.010361	0.041443
真解 u_0e^{-t}	0.002707	0.006767	0.027067
相对偏差	+53.1%	+53.1%	+53.1%
用训练装置 Euler($h=0.5$) 复算的相对偏差	−0.02%	−0.02%	−0.02%

这一步发现不了 53% 的偏差。 更糟的是：若验证时沿用训练那套 Euler($h=0.5$)，三条轨迹的偏差都是 -0.02%——看上去“换初值完美通过”，实际上参数错了 21%。

原因：对线性右端，初值只是乘一个因子，误差也是乘性的；换初值只检验“线性/结构是否对”，检验不了“衰减率对不对”。这一步的盲区正是 §1.4 的病灶——病灶在 $\hat{\theta}$，而 $\hat{\theta}$ 的错误与 u_0 无关。

(3) 换步长：这一步才看得见

由表 6，Euler 从 $h=0.5$ 减到 0.25，终点值从 0.135 跳到 0.173，漂移 0.038（相对 28%）；再减半 0.0174、再减半 0.0084。

一个“真解不随步长变”的问题，数值解漂了 28%，只能说明参数把离散误差背走了。阈值可以定得很朴素：漂移超过数据噪声（这里 1%）一个量级，就不该相信这个模型。

(4) 换区间：外推段翻倍

$\hat{\theta}$ 的偏差是随时间累积的：相对偏差 $=e^{(\theta^*-\hat{\theta})t}-1$。

$$t=2.4:\quad e^{0.213\times 2.4}-1=+66.7\%;\qquad t=4:\quad e^{0.213\times 4}-1=+134.4\%.$$

外推 20%（$t=2\to 2.4$），偏差从 53% 涨到 67%；外推到 $t=4$，涨到 134%；到 $t=6$ 是 259%。偏差按 $e^{(\theta^*-\hat{\theta})t}-1$ 增长，外推段的相对误差随距离成倍放大（本题 $t=4$ 处已是内插段终点的 2.5 倍），这正是正文说的“Neural ODE 的固有局限”。所以报告里应当把外推段单独列出，不能混进内插误差里平均掉。

(5) 陷阱：三换全过就是学到了向量场

错，而且这个错误很危险——(2) 已经给出反例：换初值“全过”（偏差甚至为零），参数却错了 21%。

根源是三类偏差的形状不同：

• 结构错（学到的是曲线不是场）→ 换初值崩； • 尺度错（参数把离散误差吸收）→ 换步长崩，换初值不崩； • 区间错（外推能力差）→ 换区间崩。

所以三换必须同时做，且每换一项都要报告“变了多少”，而不是“通过/不通过”。正文用“过拟合排行榜”作比：公开榜刷得再高，私有榜一换就露馅——换的正是“装置”。

常见错误： 只换初值就宣称“学到了向量场”；或把“训练点误差为 0”当合格证。

题 72　快速题：离散误差界

(1) 见题面公式。它的来路与推论 1 同一条链：单步的局部截断误差是 $\tau=\frac{h}{2}|u''(\xi)|\le\frac{h}{2}\max|u''|$；$[0,T]$ 上共 T/h 步，每步的误差被后续放大 $e^{L(T-t)}$ 倍，累加后就是

$$\sum_j \tau\, e^{L(T-t_j)}\approx\frac{T}{h}\cdot\frac{h}{2}\max|u''|\cdot\frac{e^{LT}-1}{LT}=\frac{h}{2}\max|u''|\cdot\frac{e^{LT}-1}{L}.$$

换句话说：把推论 1 里的“残差”换成“每步的截断误差”，同一个 Gronwall 引理就给出了离散误差的界：h 与 $(e^{LT}-1)/L$ 一个来自“步数”，一个来自“放大”。

(2) $u(t)=e^{-t}$，$\max|u''|=1$，$L=1$，$T=2$：

h	Euler 的 $u(2)$	实际误差	界 $\frac{h}{2}(e^2-1)$	界 / 实际
0.5	0.06250	0.07284	1.5973	21.9
0.25	0.10011	0.03522	0.7986	22.7

界比实际大 22 倍左右。为什么“松但不废”：界的阶是对的——h 减半，误差减半（$0.0728 \to 0.0352$），界也减半（$1.60 \to 0.80$），倍数稳定在 22；所以界可以用来定预算与判趋势（例如“要 10^{-4} 需要多大的 h”），只是不能当精确预测。松的原因与 3.1 节题 3(3) 同源：Gronwall 只用了 $|f'| \le L$ 这个标量上界，没有用到 $f' < 0$（衰减）的符号信息。

(3) $L = 50$、$T = 1$、$h = 0.01$、$\max|u''| = 1$：

$$\frac{0.01}{2} \cdot 1 \cdot \frac{e^{50}-1}{50} = 0.005 \times \frac{5.1847 \times 10^{21}}{50} \approx 5.2 \times 10^{17}.$$

界大到毫无意义——它说的是“误差不超过 10^{17}”，而真解本身只有 $O(1)$。问题不在界算错了，而在 e^{LT} 这个因子：$L = 50$、$T = 1$ 时扰动可以被放大 e^{50} 倍。要让这个界回到 10^{-3} 量级，h 得小到 10^{-23} 量级——这就是刚性方程“贵”的根子，也是正文 3.1 节“L 大、T 长时步长必须小得离谱”的算式版。

补充一句：实际误差远小于界（真正解按 e^{-50t} 衰减，扰动也跟着衰减），所以这里的界几乎全是“符号信息丢失”的代价。工程上的对策不是把 h 缩到 10^{-20}，而是换隐式格式（3.1 节的稳定性讨论），或换用可证明更紧的界。

题 73　找错题：两个写坏的验证脚本

(1) 版本 A：`arange` 的末点不是 T

`np.arange(0, T, h)` 不含右端点（上界开区间）：

h	长度	末点
0.5	4	1.5
0.25	8	1.75

按 $\hat\theta = 0.787$、Euler 复算：$h = 0.5$ 时 $u(1.5) = 0.6065^3 = 0.22310$，脚本报

$$|0.22310 - 0.13534| = 0.0878 \quad (\text{相对 } 64.8\%);$$

$h = 0.25$ 时 $u(1.75) = 0.21575$，脚本报

$$|0.21575 - 0.13534| = 0.0804 \quad (\text{相对 } 59.4\%).$$

于是脚本会打印“换步长后误差从 65% 降到 59%，怎么都下不去”——错误结论：模型没学到向量场。真相是它拿 $t = 1.5$（或 1.75）的值去和 $t = 2$ 的真值比，两次比的是不同时刻。

顺带一提，h 不能整除 T 时更糟：`arange(0,2,0.3)` 的长度是 7、末点 1.8，与别的 h 下数组长度都不同，后续逐元素比较会直接形状不匹配或静默广播。

(2) 版本 B：`true()` 用的是另一个初值

$u_0=2$、$t=2$：

• 模型：$2e^{-1.574}=0.41443$；　• 正确的真解（同初值 2）：$2e^{-2}=0.270671$；正确偏差 $+53.1\%$；　• 脚本用的真解（初值 1）：$e^{-2}=0.135335$；报出偏差

$$\frac{0.41443}{0.135335}-1=+206\%.$$

也就是说，脚本把一个 53% 的偏差报成了 206%，然后得出“换初值彻底失败、模型只记住了训练那条曲线”的结论——与事实相反。

(3) 最小改动与“静默”原因

版本 A：

```
ts = np.linspace(0.0, T, int(round(T / h)) + 1)
assert abs(ts[-1] - T) < 1e-12
```

版本 B：

```
err = max(abs(u2 - true(ts, u0=2.0)))   # 基准必须与验证对象同初值
```

为什么“程序不报错、结论全错”：两处都是语义错误而不是语法/运行时错误。数组长度一致（或比较的是标量）时，`max` 与 `abs` 都正常返回一个数；脚本没有越界、没有除零，只是把“时刻没对齐”和“基准没对齐”算进了模型误差。这类错误只能靠断言（末点必须等于 T、基准必须同初值）来防，不能靠看曲线发现。

评分要点：（1）要算出 0.0878 并指出末点是 1.5；（2）要同时给出 206% 与正确的 53.1%；（3）要说出“断言”这一层，而不只是“改代码”。

第十一部分　3.4.4 趣味编程（题 74–77）

题 74　造数据与灵敏度

(1) 验证式 (11) 满足 Verhulst 方程

记 $C=\dfrac{K}{u_0}-1=19$，则 $u=\dfrac{K}{1+Ce^{-rt}}$，求导

$$u'=\frac{KrCe^{-rt}}{\left(1+Ce^{-rt}\right)^2}=ru\cdot\frac{Ce^{-rt}}{1+Ce^{-rt}}=ru\left(1-\frac{u}{K}\right).$$

数值核对（差商用 10^{-6}）：

t	$u(t)$	差商给出的 u'	$ru(1-u/K)$
0.5	0.07280	0.054001	0.054001
1.5	0.14875	0.101299	0.101299

两者到 6 位小数一致，式 (11) 确实是 Verhulst 方程的解。

(2) 采样表

表 7　logistic 数据（$K=1$、$u_0=0.05$、$r=0.8$）与 1% 噪声的标准差

t	0	0.5	1.0	1.5	2.0
$u(t)$	0.05000	0.07280	0.10485	0.14875	0.20678
$\sigma=0.01u$	0.00050	0.00073	0.00105	0.00149	0.00207

21 个点从 0.05 涨到 0.207，只用了 K 的 21%：曲线在这段上“刚起步”，还看不出饱和。

(3) 三个灵敏度

在 $[0,2]$ 上的最大值（均出现在右端点 $t=2$）：

$$\max\left|\frac{\partial u}{\partial r}\right|=0.328,\qquad \max\left|\frac{\partial u}{\partial K}\right|=0.0341,\qquad \max\left|\frac{\partial u}{\partial u_0}\right|=3.45.$$

比值

$$\frac{\max|\partial u/\partial r|}{\max|\partial u/\partial K|}=\frac{0.328}{0.0341}\approx 9.6.$$

这与正文图 14 的“差近十倍”一致。$\partial u/\partial u_0=3.45$ 最大：因为初值在 $t=0$ 处直接影响全部轨迹（$u\approx u_0e^{rt}$），前段几乎只由它决定。

(4) 同样的噪声，为什么 K 更受伤

参数的误差大致按“噪声 / 灵敏度”放大。取 $\sigma_u\approx 0.002$（$t=2$ 处的 1% 噪声）：

$$\sigma_r\approx\frac{0.002}{0.328}=0.006\quad(r\text{ 的 }0.8\%),\qquad \sigma_K\approx\frac{0.002}{0.0341}=0.059\quad(K\text{ 的 }5.9\%).$$

同一个 1% 的观测噪声，两个参数差了近十倍——与灵敏度的比值 9.6 同一量级，也与正文实测的 1% vs 8% 同一量级。

更本质的原因：在 $[0,2]$ 上 u 还很小，$u\approx u_0e^{rt}$，K 几乎不出现；K 的信息藏在“曲线什么时候弯”里，而这段数据还没弯。所以这不是优化问题，是数据里没有信息。

(5) 陷阱：21 个点就能把三个参数都定到 1%

错。点数只影响“平均噪声”，不改变“灵敏度”这个乘子。$\partial u/\partial K$ 只有 $\partial u/\partial r$ 的 $1/9.6$、$\partial u/\partial u_0$ 的 $1/100$，误差就不可能都落在 1%。

换个说法：21 个点让“每个时刻的观测”更准，但没有让“K 在哪个时刻起作用”这件事发生改变。要改善 K，只能换窗口（题 75）或换测量量（例如直接测饱和值）。

常见错误：　把“参数误差大”归因于优化不收敛或数据太少；忽略灵敏度这一层。

题 75　反演、可辨识性与窗口

(1) 三个参数的相对误差

$$\hat r:\ \frac{0.810-0.8}{0.8}=+1.25\%,\qquad \hat K:\ \frac{0.922-1}{1}=-7.8\%,\qquad \hat u_0:\ \frac{0.0497-0.05}{0.05}=-0.6\%.$$

三个误差之比约 $1.25:7.8:0.6\approx 2:13:1$，而灵敏度之比为 $\partial u/\partial r:\partial u/\partial K:\partial u/\partial u_0 = 0.328:0.0341:3.45\approx 10:1:100$——大体上，灵敏度越小的参数，误差越大，与题 74 的机理对得上（r 与 u_0 的误差都比 K 小一个量级）。

所以“$\hat K$ 偏 8%”不是优化没做好：即使优化到全局最优，只要数据窗口是 $[0,2]$、噪声是 1%，K 就不可辨识到这个精度。

(2) 拟合曲线与真曲线

表 8　三个参数拟合后的预测（真值 $K=1$、$u_0=0.05$、$r=0.8$）

t	0.5	1	2	4	6	8
拟合曲线 $(\hat r,\hat K,\hat u_0)$	0.07256	0.10468	0.20611	0.54641	0.81160	0.89783
真曲线	0.07280	0.10485	0.20678	0.56354	0.86478	0.96940
相对偏差	-0.3%	-0.2%	-0.3%	-3.0%	-6.1%	-7.4%

在训练区间 $[0,2]$ 内偏差都小于 0.3%——完全被 1% 噪声吞掉，看不出来。但一旦进入饱和段（$t\ge 4$），偏差长到 $3\%\to 6\%\to 7\%$。

这就是可辨识性问题的典型症状：参数错得很，训练集上却“完美”。而 $\hat K$ 偏小 7.8%，正是 $t\to\infty$ 时预测偏低 7.4% 的来源——终值 $u(\infty)=K$，所以外推误差的下限就是 K 的误差。

(3) 把窗口扩到 $[0,6]$

在 $[0,6]$ 上重算灵敏度：

$$\max\left|\frac{\partial u}{\partial r}\right| = 1.014,\qquad \max\left|\frac{\partial u}{\partial K}\right| = 0.742,\qquad \frac{\max|\partial u/\partial r|}{\max|\partial u/\partial K|}\approx 1.37.$$

比值从 9.6 降到 1.37，改善约 7 倍。原因是窗口一过 $t^*=3.68$（半饱和时间），曲线开始弯，K 终于“露面”：$u\to K$ 时 $\partial u/\partial K\to 1$，而 $\partial u/\partial r$ 在饱和后趋于 0。到窗口右端 $t=6$，$\partial u/\partial K=0.742$ 已经超过 $\partial u/\partial r=0.702$——$K$ 的信息从“几乎为零”变成了“与 r 相当”。

(4) 只改一个条件，改哪个

改窗口。理由：误差 $\approx$ 噪声 / 灵敏度，噪声降 10 倍只能改善 10 倍，采样密度只改善 $\sqrt{N}$，而窗口能把灵敏度提升 20 倍以上（$0.0341\to 0.742$），且这是乘法性的改善。

这也回答了 4.1 节的“主动学习”：最值钱的一次测量不是“再测一个已测过的时刻”，而是“在曲线开始弯的地方补一个点”。

(5) 陷阱：$\hat K$ 偏了，多训几轮

错。训练集上的残差已经在噪声水平（表 8 前三列偏差 $<0.3\%$），继续训练只会把噪声也拟合进去，$\hat K$ 可能更偏。

判据很简单：训练误差不再下降、而换窗口/换区间的误差仍在变化，说明瓶颈在数据（可辨识性），不在优化。此时该改的是实验设计，不是训练轮数。

评分要点：（1）要算出 $+1.25\%$、-7.8%、-0.6% 并与灵敏度对照；（2）表要给出 $t=4,6,8$ 三列的增大趋势；（3）要落在“灵敏度由窗口决定”这一句上。

题 76　快速题：logistic 的三件事

（1）见题 74(1) 的推导：记 $C=K/u_0-1$，则

$$u'=\frac{KrCe^{-rt}}{(1+Ce^{-rt})^2}=ru\Big(1-\frac{u}{K}\Big).$$

初始条件 $u(0)=\dfrac{K}{1+C}=u_0$ 也满足，故式 (11) 是 Verhulst 方程的解。

（2）令 $u=K/2$，即 $1+Ce^{-rt^*}=2$，得

$$t^*=\frac{\ln C}{r}=\frac{\ln 19}{0.8}=3.68.$$

而采样区间是 $[0,2]$，t^* 落在它外面；$t=2$ 处 $u/K=0.207$，只走完到半饱和的 54% 路程（按时间算）。这就是“曲线还没弯”的量化版本：饱和信息在 t^* 之后才开始出现。

（3）$f(u)=ru(1-u/K)$，$f'(u)=r(1-2u/K)$：

- $u=0$：$f'(0)=r>0$，不稳定（轻微的扰动会被放大，$u\approx u_0e^{rt}$）；
- $u=K$：$f'(K)=-r<0$，稳定（渐近稳定，$K-u\approx Ce^{-rt}$）。

这条与题 74 的灵敏度是同一件事的两面：K 是稳定平衡点，所以长期数据必然把它“钉住”；而 u 在 $[0,2]$ 上离 K 太远，钉不住。

题 77　开放题：一条真实生长曲线的完整实验设计

(1) 四个误差来源与估法

逼近误差（网络容量/结构）：用“同结构、不同容量”的模型对比；或当结构已知（如 logistic）时直接与解析解比，可判为 0。

离散误差：换步长、换积分器复算（题 70 的漂移量）。

优化误差：多组初始化、多随机种子、训练更久，看参数是否还在变（正文提到“换随机种子结果可能差一个量级”，就是这一项的读数）。

数据噪声：用残差分布估（χ^2 检验式的做法），或重复测量；本题 1% 对应 $\sigma=0.002$ 量级（表 7）。

(2) 三换的做法与阈值

- 换初值：$u(0)=0.02,\,0.05,\,0.2$ 各积一条，与同初值的真解比；阈值取“噪声水平一个量级”，即相对偏差 $\lesssim 1\%$。
- 换步长：h 取训练步长的 $1/2$ 与 2 倍，比较终点漂移；阈值同 1%。超过就说明参数背了离散误差。
- 换区间：向两端各外推 20%，单独报告外推段误差（本题 $t=4$ 处已是内插终点的 2.5 倍），阈值可以放宽到 5%，但必须如实写出来。

(3) 不确定度怎么报

灵敏度写法（一阶）：由题 74(3)，参数误差 $\approx$ 观测噪声 / 灵敏度，故

$$\sigma_r \approx \frac{\sigma_u}{\max|\partial u/\partial r|}, \qquad \sigma_K \approx \frac{\sigma_u}{\max|\partial u/\partial K|}.$$

对本题（$\sigma_u \approx 0.002$）：$\sigma_r \sim 0.006$、$\sigma_K \sim 0.06$——即 r 可报到 1%，K 只能报到 6% 左右。这与实测的 1.25% 与 7.8% 同量级。

后验写法（贝叶斯/Laplace 近似）：在最小二乘解处算 Jacobian J、噪声协方差 Σ，则

$$\mathrm{Cov}(\theta) \approx \left(J^{\mathsf{T}}\Sigma^{-1}J\right)^{-1},$$

给出的是 (r, K, u_0) 的联合分布（椭圆），而不是三个孤立的数。它能直接显示“K 与 u_0 强相关”——这正是本题 $\hat{K}$ 偏小时 $\hat{u}_0$ 也在补偿的原因。

(4) 窗口扩到 $[0, 6]$

预期 $\hat{K}$ 的误差从 8% 降到 1% 量级。依据：$\max|\partial u/\partial K|$ 从 0.0341 涨到 0.742（22 倍），而 $\max|\partial u/\partial r|$ 只从 0.328 涨到 1.014（3 倍）；灵敏度比的改善约 7 倍，与题 75(3) 一致。

但还有一个前提：扩窗之后曲线进入饱和段，K 与 u_0 的相关性也被打断（u_0 的信息集中在前段、K 的信息在后段），因此不只是“灵敏度变大”，而是“两个参数终于分开可辨识”。

(5) 一句话结论

本节能保证的是：给定了 f_θ 与积分器，解存在唯一、连续依赖于初值与参数，总误差可以拆成逼近/离散/优化三笔、并在独立装置上逐笔量出来；不能保证的是：f_θ 就是真实的 f、外推段仍然准、以及一个可证明的误差界（对非线性、高维、带学习参数的系统，目前大多只有经验结果）。

换句话说：本章能让你说清楚误差有多大、来自哪里，不能让你保证误差一定小。这两件事在工程上的差别，就是“能签字”与“能上线”的差别。

第04章 矩阵的解剖

> 编写组：A1（G14）；来源版本：`230873df074c`，main已合并稿。本文件按原稿收录正文、独立习题与参考解答，保留原稿待完成项。

先看一幕：给矩阵做一次体检

求解器递来一份漂亮的报告：“残差几乎为零。”矩阵医生却拿起放大镜：如果输入轻轻动一下，解会不会跳很远？

先猜，再验证： 设 $A = \mathrm{diag}(1, 10^{-10})$。仅把右端第二个分量增加 10^{-10}，解的第二个分量会改变多少？这与求解器报告的小残差是否矛盾？

把故事翻译成数学： 在这个对角例子中，解的第二分量增加1，二范数条件数为 10^{10}。体检对应对矩阵结构与扰动敏感性的分析；小残差和小解误差是不同主张，还要说明残差针对哪个右端计算。

知识主题： 线性方程组、特征值与SVD

本章核心问题： Ax=b如何高效求解？矩阵的本质结构是什么？

只聚焦三个核心概念： LU、CG、SVD

先修知识： 矩阵运算、内积、正定性与向量/矩阵范数

课时： 4节 × 2学时 = 8学时；全书八章共64学时。

本章四节安排

节	内容	学时
4.1	直接法：LU与Cholesky	2
4.2	迭代法：CG与预条件	2
4.3	特征值与SVD	2
4.4	AI关联：可微求解器、PCA、LoRA	2

4.1 直接法：LU与Cholesky（2学时）

本节目标： 能按矩阵条件选择分解，并解释主元和三角求解。

概念讲解：把问题说明白

我们考虑线性方程组

$$Ax = b, \qquad A \in \mathbb{R}^{n\times n}, \quad x, b \in \mathbb{R}^{n}.$$

需要唯一解时，假设A非奇异。“直接法”指在精确算术中经过有限步骤得到解；使用浮点数时仍存在舍入误差，因此“计算结束”不等于“精度已经得到保证”。本节的基本思路是把系数矩阵分解为三角矩阵，再通过前代、回代求解，通常不显式形成A^{-1}。

引入问题： 在结构计算中，同一刚度矩阵会对应不同载荷。如果A不变而b不断变化，哪些计算可以保留？如果主对角元为零，是否必须判定方程无唯一解？如果求解器给出很小的残差，解是否一定准确？

4.1.1 高斯消元与带主元LU分解

高斯消元通过可逆的初等行变换，逐列消去主对角线下方的元素，把原方程组变成上三角方程组。消去第k列第i个元素所用的倍数称为乘子；将乘子保存下来，就得到三角因子。

当原顺序的主元都非零时，可以得到$A = LU$，其中L为单位下三角矩阵，U为上三角矩阵。一般非奇异矩阵并不一定能按原顺序消元，因此实际常采用列主元策略：在当前列尚未处理的行中选绝对值最大的元素，并交换相应两行。

本节统一约定

$$PA = LU, \qquad P, L, U \in \mathbb{R}^{n\times n}.$$

P表示累计的行置换，L的对角元均为1。求解时先解$Ly = Pb$，再解$Ux = y$，右端必须执行与A相同的换行。这里的“列主元”是指在当前列中选主元，执行的操作是交换行。

例如$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$非奇异，但第一步主元为零，必须先换行。即使主元非零，过小的主元也可能使乘子及中间量过大，放大舍入误差。选主元改善的是消元过程，不能消除原方程组本身的病态性。

4.1.2 对称正定矩阵与Cholesky分解

实矩阵A满足

$$A = A^T, \qquad z^T A z > 0 \quad \text{对所有 } z \in \mathbb{R}^n \setminus \{0\},$$

就称为对称正定矩阵，简称SPD矩阵。它一定非奇异，且存在唯一的正对角下三角矩阵L，使

$$A = LL^T.$$

这称为Cholesky分解，也称平方根分解。此处L的对角元严格为正，通常不等于1，应与LU分解中的单位下三角因子区分。求解分为$Ly = b$和$L^T x = y$两步。

对称正定结构使我们只需计算并保存一个三角因子。对稠密矩阵，按乘法和加减各计一次的口径，Cholesky分解的主要运算量约为$n^3/3$，一般LU约为$2n^3/3$。

“对称且对角元全正”还不够。例如$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$在向量$(1, -1)^T$方向上的二次型为负，不适用本节的Cholesky分解。

4.1.3 回代、多右端求解与误差核验

下三角方程组从第一行向下求解，称为前代；上三角方程组从最后一行向上求解，称为回代。每一步只使用已经求出的分量。

多个右端写为

$$AX = B, \qquad B = [b^{(1)}, \ldots, b^{(s)}] \in \mathbb{R}^{n\times s}, \quad X = [x^{(1)}, \ldots, x^{(s)}] \in \mathbb{R}^{n\times s}.$$

只要A不变，就可以分解一次，反复进行三角求解。对于带主元LU，计算$LY=PB$、$UX=Y$；对于Cholesky，计算$LY=B$、$L^TX=Y$。中间量Y也为$n\times s$矩阵。

核验结果时，需区分三个问题：分解关系是否成立，计算解是否满足原方程，以及计算解是否接近真解。它们分别对应分解误差、残差和前向误差。小残差能说明方程满足得好，但还需结合条件数，才能解释解的准确程度。

关键推导：让结论有依据

推导一：高斯消元、LU与置换

一、从消去一个未知量开始

设第k步开始时的系数和右端分别为$a_{ij}^{(k)}$、$b_i^{(k)}$，并且前$k-1$列的对角线下方已经为零。初始时$A^{(1)}=A$、$b^{(1)}=b$。

如果当前主元$a_{kk}^{(k)}\neq 0$，为了消去第i行中的x_k，取乘子

$$\ell_{ik}=\frac{a_{ik}^{(k)}}{a_{kk}^{(k)}},\qquad i=k+1,\dots,n,$$

执行“第i行减去ℓ_{ik}倍的第k行”，得到

$$a_{ij}^{(k+1)}=a_{ij}^{(k)}-\ell_{ik}a_{kj}^{(k)},\qquad j=k+1,\dots,n,$$

$$b_i^{(k+1)}=b_i^{(k)}-\ell_{ik}b_k^{(k)}.$$

由于$a_{ik}^{(k)}-\ell_{ik}a_{kk}^{(k)}=0$，第$k$列需要消去的元素恰好变成零。不断重复，便把原方程组化为上三角方程组$Ux=y$。

初等行变换可逆，因此它保持方程组的解集。消元改变了方程的表达形式，没有改变所求的解。

二、为什么消元可以写成LU分解

先考虑不交换行且消元能够完成的情况。令L为对角元全为1的下三角矩阵，并把各步乘子ℓ_{ik}存入相应位置；令U为消元后得到的上三角矩阵，则

$$A=LU.$$

以三阶矩阵说明这一点。消元结果满足

$$U=\begin{bmatrix}a_{11} & a_{12} & a_{13}\\ 0 & a_{22}-\ell_{21}a_{12} & a_{23}-\ell_{21}a_{13}\\ 0 & 0 & u_{33}\end{bmatrix},\qquad L=\begin{bmatrix}1 & 0 & 0\\ \ell_{21} & 1 & 0\\ \ell_{31} & \ell_{32} & 1\end{bmatrix}.$$

其中

$$\ell_{21}=\frac{a_{21}}{a_{11}},\quad \ell_{31}=\frac{a_{31}}{a_{11}},\quad \ell_{32}=\frac{a_{32}-\ell_{31}a_{12}}{u_{22}},$$

$$u_{33}=a_{33}-\ell_{31}a_{13}-\ell_{32}u_{23}.$$

直接相乘可见，LU的第二行为ℓ_{21}倍的U第一行加上U第二行，恢复了A第二行；第三行为ℓ_{31}倍的第一行加上ℓ_{32}倍的第二行再加上第三行，恢复了A第三行。L记录了消元的乘子，乘回L相当于逐步恢复被消去的部分。

一般的逐项计算公式为

$$u_{kj} = a_{kj} - \sum_{r=1}^{k-1} \ell_{kr} u_{rj}, \qquad j = k, \dots, n,$$

$$\ell_{ik} = \frac{a_{ik} - \sum_{r=1}^{k-1} \ell_{ir} u_{rk}}{u_{kk}}, \qquad i = k+1, \dots, n,$$

并规定$\ell_{kk} = 1$。这一约定称为Doolittle形式。

从消元矩阵看一般的分解关系。 记e_k为第k个标准单位向量，把第k步的乘子放入列向量

$$v_k = (0, \dots, 0, \ell_{k+1,k}, \dots, \ell_{nk})^{T} \in \mathbb{R}^{n},$$

其中前k个分量为零。定义消元矩阵

$$E_k = I - v_k e_k^{T}.$$

对任意矩阵$W \in \mathbb{R}^{n \times n}$，有$E_k W = W - v_k (e_k^{T} W)$。$e_k^{T} W$取出$W$的第$k$行，左乘$v_k$后，第$i$行就是$\ell_{ik}$倍的第$k$行，所以这一乘法正好执行本步所有行消元。

由于$e_k^{T} v_k = 0$，有

$$(v_k e_k^{T})^2 = v_k (e_k^{T} v_k) e_k^{T} = 0,$$

因而

$$(I - v_k e_k^{T})(I + v_k e_k^{T}) = I, \qquad E_k^{-1} = I + v_k e_k^{T}.$$

消元按从右到左的乘法顺序作用于A，故

$$U = E_{n-1} \cdots E_2 E_1 A,$$

$$A = E_1^{-1} E_2^{-1} \cdots E_{n-1}^{-1} U = L U.$$

当$i < j$时，v_j的第i个分量为零，所以

$$(v_i e_i^{T})(v_j e_j^{T}) = v_i (e_i^{T} v_j) e_j^{T} = 0.$$

展开上述按下标递增排列的逆矩阵乘积，所有含两个及以上非恒等项的乘积均为零，得到

$$L = I + \sum_{k=1}^{n-1} v_k e_k^{T}.$$

因此L确实是单位下三角矩阵，其第k列对角线下方就是第k步的乘子。这给出任意阶情况下“存储乘子即得到LU”的依据。

无主元LU的条件。 对非奇异矩阵A，存在上述形式的无主元LU分解，当且仅当所有顺序主子式

$$\Delta_k = \det A_{1:k,1:k} \neq 0, \qquad k = 1, \dots, n.$$

这里$A_{1:k,1:k}$表示左上角的$k \times k$子矩阵。条件成立时分解唯一。

依据。 若分解存在，则左上角子块也满足$A_{1:k,1:k} = L_{1:k,1:k} U_{1:k,1:k}$，因而

$$\Delta_k = u_{11} \cdots u_{kk}.$$

因为A非奇异，所有u_{kk}均非零。反过来，令$\Delta_0 = 1$，按上述递推式构造因子：第一个主元为Δ_1；已完成前$k-1$步时，对前k阶子块作相同消元，有$\Delta_k = \Delta_{k-1} u_{kk}$，故$u_{kk} = \Delta_k / \Delta_{k-1} \neq 0$，递推可继续。递推式中的每一项均由此前项唯一决定，故分解唯一。

此条件用于理解理论，实际程序不需要预先计算一系列行列式。

三、为什么需要选择主元

零主元不等于矩阵奇异。 例如

$$A_0 = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \det A_0 = -1.$$

第一步不能除以$a_{11} = 0$，但交换两行即可继续。因此，“非奇异矩阵都能直接按原顺序消元”是错误的。

很小的主元也可能造成困难。 考虑

$$A_\delta = \begin{bmatrix} \delta & 1 \\ 1 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 1+\delta \\ 2 \end{bmatrix}, \quad 0 < \delta \ll 1.$$

精确解为$x = (1,1)^T$。不换行时，乘子为$1/\delta$，第二行系数和右端都出现很大的中间量，回代还可能发生严重相消。换行后，第一个主元为1，乘子为δ，可以避免这个问题。4.1.3节给出实际浮点实验。

四、列主元策略与置换约定

本节采用列主元高斯消元，即部分选主元：第k步在当前第k列、尚未处理的行中选取绝对值最大的元素，

$$p = \arg\max_{i=k,\dots,n} |a_{ik}^{(k)}|,$$

然后交换第k行与第p行。“列主元”是指在当前列中选择主元，实际执行的是换行。

把所有换行累计为置换矩阵$P \in \mathbb{R}^{n\times n}$，本节统一使用

$$\boxed{PA = LU.}$$

P每行、每列恰有一个1，其余元素为0，且$P^{-1} = P^T$。PA表示重新排列A的行。

于是

$$Ax = b \iff PAx = Pb \iff LUx = Pb.$$

先解$Ly = Pb$，再解$Ux = y$即可。右端必须按相同顺序换行。 部分文献或软件采用$A = PLU$的记号，其中置换矩阵与这里相差一个转置，使用前应核对定义。

存在性说明。 精确算术中，非奇异方阵的列主元消元不会因整列候选主元全为零而中断：每步消元后矩阵具有块上三角结构，已经确定的主元非零；若剩余子块的第一列全为零，则剩余子块奇异，从而整个矩阵也奇异，产生矛盾。故可逐步得到$PA = LU$。

列主元使每步的乘子满足$|\ell_{ik}| \le 1$，有助于控制误差。但它并不保证所有矩阵的中间元素都不增长，也不改变原问题的病态程度。

为什么换行时只交换L中已经完成的列？ 在第k步开始前，设部分分解满足$PA = LU$，其中L只有前$k-1$列存有乘子，其余尚未处理的主块为单位矩阵。令R交换尚未处理的两行，则

$$(RP)A = R(LU) = (RLR^T)(RU).$$

将L分块为

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix}, \qquad R = \begin{bmatrix} I & 0 \\ 0 & R_2 \end{bmatrix},$$

可得

$$RLR^T = \begin{bmatrix} L_{11} & 0 \\ R_2 L_{21} & I \end{bmatrix}.$$

换行只需要作用于此前存储乘子的L_{21}，未处理的单位主块保持不变。这说明直接交换L的整行会把它的单位对角结构也破坏；算法应只交换前$k-1$列的相应元素。

五、手算例题：第二步换行时，L也要调整

求解

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix}.$$

第一步。 第一列绝对值最大元素为2，交换第1、3行：

$$\left[\begin{array}{ccc|c} 2 & 0 & 1 & 1 \\ 1 & 1 & 0 & 3 \\ 0 & 2 & 1 & 3 \end{array}\right].$$

乘子为$\ell_{21} = 1/2$、$\ell_{31} = 0$。第二行减去第一行的一半，得到

$$\left[\begin{array}{ccc|c} 2 & 0 & 1 & 1 \\ 0 & 1 & -1/2 & 5/2 \\ 0 & 2 & 1 & 3 \end{array}\right].$$

第二步。 在第二列剩余元素中选2为主元，交换第2、3行：

$$\left[\begin{array}{ccc|c} 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & 3 \\ 0 & 1 & -1/2 & 5/2 \end{array}\right].$$

这一换行同时改变了此前乘子的归属：L第一列的两个已存乘子要从$(1/2, 0)^T$交换为$(0, 1/2)^T$。新乘子为$\ell_{32} = 1/2$。继续消元得到

$$\left[\begin{array}{ccc|c} 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & 3 \\ 0 & 0 & -1 & 1 \end{array}\right].$$

因此

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & 1/2 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & -1 \end{bmatrix}.$$

相乘核验：

$$LU = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 0 \end{bmatrix} = PA.$$

由$Ly = Pb$得到$y = (1, 3, 1)^T$，回代得到

$$x_3 = -1, \qquad x_2 = \frac{3 - x_3}{2} = 2, \qquad x_1 = \frac{1 - x_3}{2} = 1.$$

最终$x = (1, 2, -1)^T$，代回原方程有$Ax = b$。

六、算法与实现

算法4.1：列主元LU分解。 数学下标从1开始。

```
输入：非奇异方阵 A ∈ R^(n×n)
初始化：U ← A，L ← I，P ← I
对 k = 1，...，n：
    p ← 在 i = k，...，n 中使 |U[i,k]| 最大的行号
    若 U[p,k] = 0：报告分解失败
    交换 U 的第 k、p 行；交换 P 的第 k、p 行
    若 k > 1：交换 L[k,1:k-1] 与 L[p,1:k-1]
    对 i = k+1，...，n：
        L[i,k] ← U[i,k] / U[k,k]
        U[i,k+1:n] ← U[i,k+1:n] - L[i,k] U[k,k+1:n]
        U[i,k] ← 0
输出：P、L、U，满足 PA = LU
```

“实验设计”栏目给出实数矩阵的教学实现。用排列向量p代替显式存储P：A[p，:]就是PA，B[p，:]就是PB。函数只在当前候选主元恰为零时停止；它没有把“小于某个固定绝对阈值”直接等同于奇异。近奇异问题还须结合尺度与误差指标判断。

复杂度。 对稠密$n \times n$矩阵，把一次乘法和一次加减各计为一次浮点运算，LU分解的主项约为$2n^3/3$；选主元比较需要$O(n^2)$次操作，不改变三次主项。实际库常将L、U压缩存入同一个数组，本教学实现则分开存储，便于阅读。三次主项来自各步剩余子矩阵的更新：第k步约需$2(n-k)^2$次乘加操作，因此

$$2\sum_{k=1}^{n-1}(n-k)^2 = 2\sum_{r=1}^{n-1} r^2 = \frac{(n-1)n(2n-1)}{3} = \frac{2}{3}n^3 + O(n^2).$$

这里采用乘法和加减各计一次的口径；教材中若仅统计乘除法，则LU主项约为$n^3/3$，与这里并不矛盾。

推导二：Cholesky的条件、证明与计算

一、对称正定是什么意思

实矩阵$A \in \mathbb{R}^{n\times n}$满足

$$A = A^T, \qquad z^T A z > 0 \quad \text{对所有 } z \neq 0,$$

就称为对称正定矩阵，简称SPD矩阵。

正定性意味着二次型沿任意非零方向均为正。若$Az = 0$且$z \neq 0$，就有$z^T A z = 0$，矛盾，因此SPD矩阵必非奇异。取标准单位向量e_i，还可得到$a_{ii} = e_i^T A e_i > 0$。

但对角元全正并不足以保证正定。例如

$$C = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \qquad z = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \qquad z^T C z = -2 < 0.$$

对任意实矩阵$M \in \mathbb{R}^{m\times n}$及$\alpha > 0$，矩阵$M^T M + \alpha I$是SPD，因为

$$z^T(M^T M + \alpha I)z = \|Mz\|_2^2 + \alpha\|z\|_2^2 > 0.$$

这说明正定矩阵有自然的来源；并不意味着求任意最小二乘问题时都应先形成$M^T M$。

二、Cholesky分解的存在性与唯一性

定理4.1。 实对称矩阵A正定，当且仅当存在对角元严格为正的下三角矩阵L，使

$$\boxed{A = LL^T.}$$

满足这一约定的L唯一。这里的L不要求对角元等于1，应与LU分解中的单位下三角因子区分。

证明：正定性保证分解存在。 对阶数使用归纳法。$n=1$时，$a_{11}>0$，取$L=[\sqrt{a_{11}}]$即可。

对$n>1$，写成分块形式

$$A=\begin{bmatrix} a & r^T \\ r & C \end{bmatrix},\quad a\in\mathbb{R},\quad r\in\mathbb{R}^{n-1},\quad C\in\mathbb{R}^{(n-1)\times(n-1)}.$$

正定性给出$a>0$。设

$$\ell=\sqrt{a},\qquad w=\frac{r}{\ell},\qquad S=C-ww^T=C-\frac{rr^T}{a}.$$

S称为这一步的Schur补。为证明它正定，任取$z\in\mathbb{R}^{n-1}$、$z\neq 0$，构造

$$v=\begin{bmatrix} -r^Tz/a \\ z \end{bmatrix}\neq 0.$$

逐项展开可得

$$\begin{aligned} v^TAv &= a\left(-\frac{r^Tz}{a}\right)^2+2\left(-\frac{r^Tz}{a}\right)r^Tz+z^TCz \\ &= \frac{(r^Tz)^2}{a}-2\frac{(r^Tz)^2}{a}+z^TCz \\ &= z^T\left(C-\frac{rr^T}{a}\right)z=z^TSz. \end{aligned}$$

因为A正定，左端大于零，所以S也对称正定。由归纳假设，$S=L_SL_S^T$，其中L_S为对角元正的下三角矩阵。于是

$$L=\begin{bmatrix} \ell & 0 \\ w & L_S \end{bmatrix}$$

满足

$$LL^T=\begin{bmatrix} a & r^T \\ r & ww^T+L_SL_S^T \end{bmatrix}=A.$$

证明：分解保证正定性。 若$A=LL^T$且L对角元均正，则L可逆。对$z\neq 0$，有$L^Tz\neq 0$，从而

$$z^TAz=\|L^Tz\|_2^2>0.$$

证明：唯一性。 若$L_1L_1^T=L_2L_2^T$，则$Q=L_2^{-1}L_1$为下三角矩阵，且$QQ^T=I$。因此$Q^{-1}=Q^T$既为下三角矩阵又为上三角矩阵，只能是对角矩阵。它的对角元满足$q_{ii}^2=1$；又因两因子的对角元都正，$q_{ii}>0$，故$Q=I$，即$L_1=L_2$。证毕。

这一证明说明：精确算术中，每步剩余的Schur补仍然正定，因此可以不选主元地进行Cholesky分解。

三、逐元素公式与手算例题

比较$A=LL^T$的第j个对角元，

$$a_{jj}=\sum_{k=1}^{j-1}\ell_{jk}^2+\ell_{jj}^2,$$

取正平方根得到

$$\ell_{jj}=\sqrt{a_{jj}-\sum_{k=1}^{j-1}\ell_{jk}^2}.$$

比较第i,j个元素，其中$i>j$，

$$a_{ij} = \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk} + \ell_{ij}\ell_{jj},$$

所以

$$\ell_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk}}{\ell_{jj}}.$$

例题。 对矩阵

$$H = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{bmatrix}$$

依次计算：

$$\begin{aligned} &\ell_{11} = \sqrt{4} = 2, && \ell_{21} = 2/2 = 1, && \ell_{31} = 2/2 = 1, \\ &\ell_{22} = \sqrt{5 - 1^2} = 2, && \ell_{32} = (1 - 1\cdot 1)/2 = 0, \\ &\ell_{33} = \sqrt{3 - 1^2 - 0^2} = \sqrt{2}. \end{aligned}$$

因此

$$L_H = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \sqrt{2} \end{bmatrix}, \qquad H = L_H L_H^{T}.$$

L_H对角元均正，所以分解也证明了H正定。若右端为$g = (6, 11, 1)^{T}$，先解$L_H y = g$：

$$y_1 = 3, \qquad y_2 = (11 - 3)/2 = 4, \qquad y_3 = (1 - 3)/\sqrt{2} = -\sqrt{2}.$$

再解$L_H^{T} x = y$：

$$x_3 = -1, \qquad x_2 = 2, \qquad x_1 = (3 - 2 + 1)/2 = 1.$$

所以$x = (1, 2, -1)^{T}$，可核验$Hx = g$。

四、算法、实现与适用边界

算法4.2：Cholesky分解。 数学下标从1开始。

```
输入：实对称正定矩阵 A ∈ R^(n×n)
初始化：L ← 零矩阵
对 j = 1, ..., n：
    d ← A[j,j] - Σ(k=1,...,j-1) L[j,k]^2
    若 d ≤ 0：报告失败，检查正定性及舍入误差
    L[j,j] ← sqrt(d)
    对 i = j+1, ..., n：
        L[i,j] ← (A[i,j] - Σ(k=1,...,j-1) L[i,k]L[j,k]) / L[j,j]
输出：L，满足 A = L L^T
```

若实际数据只是近似对称，应先分析非对称部分的来源与大小，再决定是否采用对称模型；不能在没有说明的情况下直接把输入改成$(A + A^{T})/2$。一些库函数只读取指定的一个三角部分，因此“函数成功返回”也不能代替对原始输入对称性的核查。

Cholesky分解的主项约为$n^3/3$次浮点运算，约为一般LU的一半；只需保存一个三角因子，共$n(n+1)/2$个数。实验栏中的程序为便于展示使用完整数组，并未实现压缩存储。

矩阵条件	首选方法	需要注意
一般非奇异实方阵	带主元LU	保存置换，对右端执行相同换行
实对称正定矩阵	Cholesky	使用$A=LL^T$，不需要主元交换
对称但不正定	不能直接套用本节Cholesky	可用带主元LU；对称不定分解属于扩展内容
奇异矩阵	本节唯一解流程不适用	需另行讨论相容性、秩和解的结构

正半定矩阵也可能出现零平方根项，例如$\left[\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right]$。此外，浮点运算中的极度病态SPD矩阵可能因舍入而使计算出的平方根项非正；失败不总能直接等同于原始精确矩阵不正定。

五、与LU、顺序主子式及改进平方根法的联系（选读）

正定性怎样保证顺序主子式为正？ 设A正定。对任意k及$z\in\mathbb{R}^k\setminus\{0\}$，把$z$补零为$\tilde z=(z^T,0,\dots,0)^T\in\mathbb{R}^n$，则

$$z^TA_{1:k,1:k}z=\tilde z^TA\tilde z>0.$$

故每个顺序主子矩阵都正定，均有正对角Cholesky分解，从而

$$\Delta_k=\det A_{1:k,1:k}=\prod_{j=1}^{k}\ell_{jj}^2>0.$$

因此SPD矩阵也满足无主元LU的存在条件。注意：“所有顺序主子式非零”保证无主元LU可做；“实对称且所有顺序主子式为正”才是正定性判据。

由LU得到LDL^T。 对实对称矩阵，假设所有顺序主子式非零，已有唯一分解$A=L_0U$，其中L_0为单位下三角矩阵。取

$$D=\operatorname{diag}(u_{11},\dots,u_{nn}),\qquad R=D^{-1}U,$$

则R为单位上三角矩阵，且

$$A=L_0DR=A^T=R^TDL_0^T.$$

左式与右式都是“单位下三角矩阵乘以上三角矩阵”的分解。由唯一性，$L_0=R^T$，于是

$$A=L_0DL_0^T,\qquad d_j=u_{jj}=\frac{\Delta_j}{\Delta_{j-1}},\quad \Delta_0=1.$$

若所有$\Delta_j>0$，则所有$d_j>0$。对任意非零z，向量$w=L_0^Tz$非零，故

$$z^TAz=w^TDw=\sum_j d_jw_j^2>0.$$

这也证明了上述正定性判据的反方向，称为Sylvester判据。

当A正定时，可以取

$$D^{1/2}=\operatorname{diag}(\sqrt{d_1},\dots,\sqrt{d_n}),\qquad L=L_0D^{1/2},$$

于是$LL^T=L_0DL_0^T=A$，这就把LU、LDL^T与Cholesky联系起来。L_0与L的对角约定不同，不能在算法中混用。

比较$A=L_0DL_0^T$各元素，可得不需要开平方的递推式。写L_0的元素为q_{ij}，规定$q_{jj}=1$，则

$$d_j = a_{jj} - \sum_{r=1}^{j-1} q_{jr}^2 d_r,$$

$$q_{ij} = \frac{a_{ij} - \sum_{r=1}^{j-1} q_{ir} d_r q_{jr}}{d_j}, \qquad i > j.$$

这就是改进平方根法的基本形式。求解顺序为$L_0 y = b$、$Dz = y$、$L_0^T x = z$。本节主线实现Cholesky；这里给出联系，便于阅读教材的相应推导。对称不定矩阵的可靠求解通常需要带主元、允许块对角D的专门算法，不能直接套用SPD结论。

推导三：前代、回代与多右端

一、前代和回代：为什么只需要二次运算量

前代。 对一般非奇异下三角矩阵L，第i行是

$$\sum_{j=1}^{i-1} \ell_{ij} y_j + \ell_{ii} y_i = c_i.$$

前面的$y_1, \dots, y_{i-1}$已知，因此按$i = 1, \dots, n$计算

$$y_i = \frac{c_i - \sum_{j=1}^{i-1} \ell_{ij} y_j}{\ell_{ii}}.$$

对于LU中的单位下三角因子，$\ell_{ii} = 1$，可省去除法；对于Cholesky因子则必须保留。

回代。 对上三角矩阵U，从最后一行开始，按$i = n, \dots, 1$计算

$$x_i = \frac{y_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}}.$$

因为每行只使用已经算出的未知量，所以不需要重新求解完整方程组。单次前代或回代的主项均约为n^2次运算；两次三角求解合计约$2n^2$次。

二、多右端：一次分解，反复使用

对$AX = B$，由$PA = LU$得到

$$LY = PB, \qquad UX = Y, \qquad Y \in \mathbb{R}^{n \times s}.$$

第i行的前代和回代公式分别为

$$Y_{i,:} = (PB)_{i,:} - \sum_{j=1}^{i-1} \ell_{ij} Y_{j,:},$$

$$X_{i,:} = \frac{Y_{i,:} - \sum_{j=i+1}^{n} u_{ij} X_{j,:}}{u_{ii}}.$$

这里前代公式使用LU的单位对角约定。Cholesky则求解

$$L_H Y = B, \qquad L_H^T X = Y,$$

前代需要除以对应的$(L_H)_{ii}$。

求解安排	稠密LU主要运算量
对每个右端重新分解	$s(2n^3/3 + 2n^2)$
分解一次，求解全部右端	$2n^3/3 + 2sn^2$

因子只取决于A。只要A不变，就可以复用；当A发生变化，不能直接把旧因子当成新矩阵的精确分解。把多个右端作为矩阵一起处理，还有助于数值库使用高效的矩阵运算。

延续LU例题，取

$$X_* = \begin{bmatrix} 1 & 2 \\ 2 & 0 \\ -1 & 1 \end{bmatrix}, \qquad B = AX_* = \begin{bmatrix} 3 & 1 \\ 3 & 2 \\ 1 & 5 \end{bmatrix}.$$

用此前的P, L, U即可同时求出两列解，不必再次消元。具体地，

$$PB = \begin{bmatrix} 1 & 5 \\ 3 & 1 \\ 3 & 2 \end{bmatrix}.$$

由$LY = PB$，逐行计算

$$Y_{1,:} = (1, 5), \quad Y_{2,:} = (3, 1), \quad Y_{3,:} = (3, 2) - \tfrac{1}{2}(1, 5) - \tfrac{1}{2}(3, 1) = (1, -1).$$

再由$UX = Y$，从最后一行向上计算

$$X_{3,:} = (-1, 1), \qquad X_{2,:} = \frac{(3, 1) - (-1, 1)}{2} = (2, 0),$$
$$X_{1,:} = \frac{(1, 5) - (-1, 1)}{2} = (1, 2).$$

两列解正好组成制造解矩阵X_*。公式中的行向量均为1×2，前代与回代每一步的维数保持一致。

Cholesky的回代要读对下标。 若$A = LL^T$，第二次三角求解为$L^T x = y$，所以第i行满足

$$\ell_{ii} x_i + \sum_{j=i+1}^{n} \ell_{ji} x_j = y_i,$$

$$\boxed{x_i = \frac{y_i - \sum_{j=i+1}^{n} \ell_{ji} x_j}{\ell_{ii}}, \qquad i = n, \ldots, 1.}$$

分子必须使用前代后得到的y_i，且转置后的上三角元素为ℓ_{ji}。不能把原始b_i直接放回分子，也不能把ℓ_{ji}误写为位于L上三角零区的ℓ_{ij}。

推导四：分解误差、残差与解误差

本节主要使用无穷范数：

$$\|v\|_\infty = \max_i |v_i|, \qquad \|A\|_\infty = \max_i \sum_j |a_{ij}|.$$

后一个公式是矩阵的最大绝对行和，不能误写为矩阵中最大的元素绝对值。它满足$\|Av\|_\infty \le \|A\|_\infty \|v\|_\infty$。

1. 分解关系是否成立。 对非零A，分别计算

$$\varepsilon_{\mathrm{LU}} = \frac{\|PA - LU\|_\infty}{\|A\|_\infty}, \qquad \varepsilon_{\mathrm{Chol}} = \frac{\|A - LL^T\|_\infty}{\|A\|_\infty}.$$

这一检查能够发现置换方向错误、漏换L中乘子或转置使用错误等实现问题。

2. 计算解是否满足原方程。 用原始A, b重新计算残差

$$r = b - A\widehat{x}.$$

当$b \neq 0$时，相对残差为

$$\rho = \frac{\|r\|_\infty}{\|b\|_\infty}.$$

还可报告归一化残差指标

$$\eta = \frac{\|r\|_\infty}{\|A\|_\infty \|\widehat{x}\|_\infty + \|b\|_\infty}.$$

它具有后向误差的解释：若分母非零，可以找到相对大小均不超过η的扰动$\Delta A, \Delta b$，使

$$(A + \Delta A)\widehat{x} = b + \Delta b.$$

这里扰动满足$\|\Delta A\|_\infty \le \eta \|A\|_\infty$、$\|\Delta b\|_\infty \le \eta \|b\|_\infty$；不限制扰动保持正定性等额外结构。主线只需理解：$\eta$小，表示计算解精确满足一个与原问题接近的方程组。

▶ 选读证明：归一化残差为何等于这一扰动模型下的最小后向误差？

若$b = 0$，不报告以$\|b\|$为分母的相对残差，改报绝对残差和适用的归一化指标。对多个右端，应逐列报告指标，避免较大的右端掩盖较小右端的问题。

3. 计算解是否接近真解。 若已知$x_* \neq 0$，计算相对前向误差

$$\varepsilon_x = \frac{\|\widehat{x} - x_*\|_\infty}{\|x_*\|_\infty}.$$

实际问题中常不知道x_*，实验可以先指定x_*，再构造$b = Ax_*$，称为制造解。构造b本身也有舍入误差，因此应选择简单可核验的数据，或使用更高精度参考值。本节两个三阶主实验的A、X_*和B均由可精确表示的小整数构成。

为什么小残差不必带来小解误差？ 若A非奇异且$Ax_* = b \neq 0$，则

$$A(x_* - \widehat{x}) = r, \qquad x_* - \widehat{x} = A^{-1} r.$$

于是

$$\|x_* - \widehat{x}\|_\infty \le \|A^{-1}\|_\infty \|r\|_\infty.$$

又有$\|b\|_\infty \le \|A\|_\infty \|x_*\|_\infty$，所以

$$\boxed{\frac{\|x_* - \widehat{x}\|_\infty}{\|x_*\|_\infty} \le \kappa_\infty(A) \frac{\|r\|_\infty}{\|b\|_\infty}, \qquad \kappa_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty.}$$

$\kappa_\infty(A)$称为无穷范数条件数。它大时，小残差仍可能对应较大的解误差；这是问题对扰动敏感，不能只归咎于算法。此处的A^{-1}用于推导，不要求求解时显式形成逆矩阵。条件数的进一步解释见4.3节。

例如

$$D = \mathrm{diag}(1, 10^{-10}), \quad x_* = (1, 1)^T, \quad b = Dx_* = (1, 10^{-10})^T.$$

取$\widehat{x} = (1, 2)^T$，则

$$r = (0, -10^{-10})^T, \quad \rho = 10^{-10}, \quad \varepsilon_x = 1, \quad \kappa_\infty(D) = 10^{10}.$$

该残差针对原始右端b计算；它很小，但第二个解分量已经相差1。SPD矩阵也可能病态。

实验设计：先预测，再运行

实验任务：用小型一般矩阵及正定矩阵求解，与可信库函数比较；加入需交换主元的例子。

运行前预测： 一般矩阵例题需要两次换行，分解应满足$PA = LU$；正定例题应得到正对角Cholesky因子；两个右端复用同一组因子后，均应恢复已知真解。小主元无主元消元可能失准，病态矩阵的候选解可能同时具有小残差和大解误差。

要填写的项目	本组内容
要回答的问题与运行前预测	验证列主元LU、Cholesky和多右端三角求解；预测两个良态主实验均能在给定容差内恢复制造解，小主元实验与病态反例分别揭示算法稳定性和问题敏感性的限制
输入、参数、单位、数据来源与划分	使用本节手算的三阶一般矩阵A与正定矩阵H，给定$X_* = \begin{bmatrix} 1 & 2 \\ 2 & 0 \\ -1 & 1 \end{bmatrix}$并构造$B = AX_*$或$HX_*$；补充$\delta = 10^{-20}$小主元矩阵、对称不定及正半定矩阵、$D = \mathrm{diag}(1, 10^{-10})$；均为自构造无量纲数据，无外部数据集或训练/测试划分
独立参考解/基线及选择理由	以手算制造解X_*为主要独立参考，以`np.linalg.solve`为库函数对照，以`np.linalg.cholesky`核对Cholesky因子；小主元实验另用`Fraction`求存储问题的精确有理解，避免把输入舍入与算法误差混淆
误差指标、容差及其依据	无穷范数分解相对误差；逐右端相对残差ρ、归一化残差η、前向误差及库解相对差异。两个良态三阶主实验用$\tau = 500n\,\mathrm{eps} \approx 3.331 \times 10^{-13}$；这是留有裕量的教学验收阈值，不是任意矩阵的统一精度保证。反例另用明确断言核验预期现象
环境、种子、设备与运行预算	本次复跑打印Python、NumPy版本及处理器架构；使用`float64`，无随机数据，无需随机种子；普通CPU上的小矩阵计算，不设大型任务或计时比较；不根据Python循环性能推断底层数值库性能

结果解释与本节结论

运行环境： Python 3.12.14，NumPy 2.3.5，x86_64，`float64`。本次运行日期：2026-10-20。主实验容差为3.331×10^{-13}。下表中的数字保留四位有效数字，不同平台最后几位可能略有差异。

方法	右端	分解相对误差	相对残差ρ	归一化残差η	前向误差	与库解的相对差异
LU	1	0	0	0	0	0
LU	2	0	0	0	0	0
Cholesky	1	5.551×10^{-17}	4.037×10^{-17}	1.645×10^{-17}	5.551×10^{-17}	5.551×10^{-17}
Cholesky	2	5.551×10^{-17}	0	0	5.551×10^{-17}	5.551×10^{-17}

两种方法都恢复了X_*，所有主实验指标均小于给定容差。LU例题仅出现可精确表示的整数与二进制分数，因此这次运行的相关误差为零；不能据此断言LU一般没有舍入误差。

补充实验	实际结果	解释
$\delta = 10^{-20}$的小主元	无主元解为$(0, 1)^T$，带主元解为$(1, 1)^T$	无主元消元在此发生严重相消，带主元结果与存储问题的精确参照一致到所列容差

补充实验	实际结果	解释
对称不定矩阵	按预期报告Cholesky失败	第二个平方根项为负
秩一正半定矩阵	按预期报告Cholesky失败	第二个平方根项为零
病态对角矩阵的候选解	$\rho = 10^{-10}$，$\varepsilon_x = 1$	小残差不足以保证小前向误差

另行复核了遗漏乘子交换这一故意错误：遗漏L乘子交换时，分解相对误差确为0.5。奇异LU输入与非对称Cholesky输入也分别按设计报告失败。

这些实验分别提供了不同证据：LU的$PA - LU$检查验证了两次换行及L乘子的同步调整；制造解检查了求解结果；库函数提供另一个实现的对照；小主元实验说明了选主元的作用；病态反例说明了残差的解释边界。仅与一个库函数吻合，不能单独证明所有输入上的正确性。

结果适用范围。 三阶主实验支持本节给出的具体实现与例题，不代表对所有矩阵都能达到同一误差水平。没有对大型稀疏矩阵做性能实验，也没有根据这组小矩阵作运行时间比较。浮点残差本身也包含矩阵向量乘法与相减的舍入误差，不能把打印出的零残差解释为无限精度的证明。

本节结论。 同一个矩阵对应多个右端时，分解可以保留，三角求解需要重新执行。一般非奇异矩阵使用列主元LU，SPD矩阵可利用Cholesky节省运算和因子存储。零对角元可能只意味着需要换行；能否准确求解，还要结合分解核验、原方程残差和条件数判断。

易错点与讨论

常见主张	判断与依据
“只要矩阵非奇异，就能不换行完成LU。”	错。A_0非奇异但第一个主元为零；无主元LU需要额外条件。
“列主元消元就是交换列。”	错。本节在当前列中选主元，交换的是行，得到$PA = LU$。
“第二步换行只影响U，不影响L。”	错。此前存储的乘子随行归属改变，只需同步交换L此前已完成的列。
“对称且对角元为正，就能做Cholesky。”	错。矩阵C提供反例，必须检查正定性。
“LU和Cholesky中的L都具有单位对角线。”	错。本节LU采用单位对角约定；Cholesky采用正对角约定。
“右端改变就必须重新分解A。”	错。A不变时复用因子，仅进行三角求解。
“选了主元就不会产生大解误差。”	错。选主元不能消除病态性，也不能无条件排除元素增长。
“小残差一定对应小解误差。”	错。还需考虑条件数与计算残差所用的原始数据。

随堂提问： 设

$$A = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 5 \end{bmatrix}.$$

请按$PA = LU$约定手算分解和方程组的解，并判断能否使用Cholesky。

同学可能的回答一： “首个主元是零，所以A奇异。”
核验： $\det A = -2 \neq 0$。零首主元意味着原顺序消元不能直接进行，应先交换两行。

同学可能的回答二： “只要可逆，就能使用Cholesky。”
核验： 本题$A \neq A^T$，不满足本节Cholesky的适用条件。

最终解释与手算结果：

$$P=\begin{bmatrix}0&1\\1&0\end{bmatrix},\qquad L=I,\qquad U=\begin{bmatrix}2&3\\0&1\end{bmatrix},\qquad Pb=\begin{bmatrix}5\\1\end{bmatrix}.$$

前代得到$y=Pb$，回代得到$x_2=1$、$x_1=(5-3)/2=1$。所以$x=(1,1)^T$，且$Ax=b$。若程序在第二次换行后出现异常结果，还应优先检查$PA-LU$，确认此前保存的L乘子是否同步交换。

4.2 迭代法：CG与预条件（2学时）

本节目标： 能说明CG的适用条件和预条件改善的对象。

想像你站在一座碗形的山谷里，山谷最低点埋着一口井，井的位置就是 $Ax=b$ 的解。直接法像一个拿着完整等高线图的测绘师，一口气算出井的精确坐标；迭代法则像一个蒙眼下山的人，只凭脚下这一处的坡度，一小步一小步往下挪，每走一步都离井口更近一点。普通梯度下降只凭当前坡度挪步，容易在狭长山谷里来回折返，而共轭梯度法，则像一个记住走过路径的聪明人，把“蒙眼下山”走得又快又稳。

概念讲解：把问题说明白

直接法对稠密小矩阵有效，但对大规模稀疏矩阵（如有限元离散化），LU 分解的填充会破坏稀疏性，存储和计算量不可接受。这时迭代法登场，它从初始猜测 $x^{(0)}$ 出发，每步只改进一点，直到残差满足停止准则。

共轭梯度法（Conjugate Gradient, CG）是求解对称正定线性方程组 $Ax=b$ 的一种迭代方法。它将求解线性方程组 $Ax=b$ 等价于求二次函数 $f(x)=\frac{1}{2}x^TAx-b^Tx$ 的极小值点问题。把这个二次函数想象成一个光滑的碗——A 对称正定时，碗底向上、处处光滑，有且只有一个最低点，这个最低点恰好就是解。CG利用 A-共轭方向（$d_i^TAd_j=0,\ i\neq j$）依次沿互不干扰的方向下降，这些方向彼此"井水不犯河水"，走完一个再走下一个，各走各的，绝不互相抵消。当 A 对称正定时，$f(x)$ 有唯一极小点，且该极小点恰好是 $Ax=b$ 的解。它属于迭代法，但理论上对于 n 维问题最多 n 步即可得到精确解，因此具有直接法的特征。

符号约定：设 $A=(a_{ij})\in\mathbb{R}^{n\times n}$ 是对称正定矩阵，$b=(b_1,b_2,\cdots,b_n)^T$，求解线性方程组：

$$Ax=b.$$

考虑如下定义的二次函数 $\varphi:\mathbb{R}^n\to\mathbb{R}$：

$$\varphi(x)=\frac{1}{2}(Ax,x)-(b,x)=\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}a_{ij}x_ix_j-\sum_{j=1}^{n}b_jx_j.$$

该函数梯度为 $\nabla\varphi(x)=Ax-b$；且若 $x^*=A^{-1}b$ 是方程组解，则 $\varphi(x^*)$ 取得极小值。

残量 $r^{(k)}=b-Ax^{(k)}$；搜索方向 $p^{(k)}$；步长 α_k；方向更新系数 β_k。

适用条件：CG 方法严格要求系数矩阵 A 对称正定。

停止准则：实用中通常以相对残量 $\|r^{(k)}\|_2\le\varepsilon\|b\|_2$ 或最大迭代次数为终止条件。理论上 CG 最多 n 步即可求得精确解，但在舍入误差影响下很难保证正交性，实际计算步数常 $k\ll n$ 即可达到精度要求。

简单对角预条件：当 A 为病态矩阵（条件数 $K\gg1$）时，碗被拉成了又长又窄的山谷：沿长轴走一步，垂直方向只挪动一点点，登山者只好来回折返，收敛极慢。预条件就像给山谷做一次"整形"，把它压回接近圆形，让每一小步都朝谷底直去。最简单的方式是取 $M=\mathrm{diag}(A)$。将原方程组转化为 $M^{-1}Ax=M^{-1}b$，

此时新系数矩阵 $M^{-1}A$ 的条件数通常比 A 小，从而加速收敛。但需注意，$M^{-1}A$ 往往不再对称，因此实际应用需要采用特定的预条件CG算法（如PCG）以保持对称性。

例：取 $A=\begin{pmatrix}3 & 1\\ 1 & 2\end{pmatrix}$，$b=\begin{pmatrix}5\\ 5\end{pmatrix}$，$x^{(0)}=(0,0)^T$。矩阵 A 显然对称且正定。如果直接使用CG法，从 $x^{(0)}=(0,0)^T$ 出发，理论上经过 2 步迭代即可得到精确解 $x^*=(1,2)^T$。如果我们使用对角预条件 $M=\begin{pmatrix}3 & 0\\ 0 & 2\end{pmatrix}$，则可以将新系统的条件数从原来的约 2.618 降低，从而可能减少迭代步数或提高数值稳定性。

关键推导：让结论有依据

已知条件：$A\in\mathbb{R}^{n\times n}$ 对称正定，$b\in\mathbb{R}^n$，目标求解 $Ax=b$，即求二次函数

$$\varphi(x)=\frac{1}{2}x^TAx-b^Tx$$

的极小点。

关键步骤：

（1）最速下降法与共轭方向的引入

设从 $x^{(k)}$ 出发，沿方向 $p^{(k)}$ 作一维搜索：$x^{(k+1)}=x^{(k)}+\alpha_kp^{(k)}$。为使 $\varphi(x^{(k+1)})$ 极小化，将 $x^{(k+1)}$ 代入二次函数并令其关于 α_k 的导数为 0，即

$$\frac{d}{d\alpha_k}\varphi(x^{(k)}+\alpha_kp^{(k)})=(A(x^{(k)}+\alpha_kp^{(k)})-b,p^{(k)})=0.$$

解此方程得最优步长

$$\alpha_k=\frac{(b-Ax^{(k)},p^{(k)})}{(Ap^{(k)},p^{(k)})}=\frac{(r^{(k)},p^{(k)})}{(Ap^{(k)},p^{(k)})}.$$

其中 $r^{(k)}=b-Ax^{(k)}$ 为残差。若每次取 $p^{(k)}=r^{(k)}$，则为最速下降法，即"闭着眼睛直直往下滚"，但收敛极慢，因为它会在拉长的山谷里来回震荡、兜圈子。CG 法的核心是寻找一组 A-共轭 的搜索方向 $\{p^{(k)}\}$，即满足：$(Ap^{(i)},p^{(j)})=0,\quad i\neq j$。

（2）CG 搜索方向的构造 取初始方向 $p^{(0)}=r^{(0)}$。对于 $k\geq1$，新的搜索方向不是单纯沿着当前残差（负梯度）$r^{(k)}$ 走，而是在残差方向基础上，叠加一部分上一步的搜索方向，令 $p^{(k)}=r^{(k)}+\beta_{k-1}p^{(k-1)}$。 系数 β_{k-1} 的作用是微调旧方向的权重，保证新方向 $p^{(k)}$ 和上一个方向 $p^{(k-1)}$ 关于矩阵 A 共轭，也就是满足共轭条件 $(p^{(k)},Ap^{(k-1)})=0$，将 $p^{(k)}=r^{(k)}+\beta_{k-1}p^{(k-1)}$ 带入得

$$(r^{(k)}+\beta_{k-1}p^{(k-1)},Ap^{(k-1)})=0.$$

求解得

$$\beta_{k-1}=-\frac{(r^{(k)},Ap^{(k-1)})}{(p^{(k-1)},Ap^{(k-1)})}.$$

$r^{(k)}=r^{(k-1)}-\alpha_{k-1}Ap^{(k-1)}$ 移项得到 $Ap^{(k-1)}=\frac{1}{\alpha_{k-1}}\left(r^{(k-1)}-r^{(k)}\right)$。将其代入分子，再利用残差正交性 $(r^{(k)},r^{(k-1)})=0$ 得

$$\begin{aligned}(r^{(k)},Ap^{(k-1)})&=\frac{1}{\alpha_{k-1}}\left[(r^{(k)},r^{(k-1)})-(r^{(k)},r^{(k)})\right]\\&=-\frac{1}{\alpha_{k-1}}(r^{(k)},r^{(k)}).\end{aligned}$$

$\alpha_{k-1}=\frac{(\boldsymbol{r}^{(k-1)},\boldsymbol{r}^{(k-1)})}{(\boldsymbol{p}^{(k-1)},A\boldsymbol{p}^{(k-1)})}$ 移项得

$$(\boldsymbol{p}^{(k-1)},A\boldsymbol{p}^{(k-1)})=\frac{1}{\alpha_{k-1}}(\boldsymbol{r}^{(k-1)},\boldsymbol{r}^{(k-1)}).$$

进而得到更常用的简化计算公式

$$\beta_{k-1}=\frac{(\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)})}{(\boldsymbol{r}^{(k-1)},\boldsymbol{r}^{(k-1)})}.$$

（3）CG 算法标准迭代格式

1. 任取 $\boldsymbol{x}^{(0)}\in\mathbb{R}^n$，计算 $\boldsymbol{r}^{(0)}=\boldsymbol{b}-A\boldsymbol{x}^{(0)}$，取 $\boldsymbol{p}^{(0)}=\boldsymbol{r}^{(0)}$。
2. 对 $k=0,1,\cdots$，计算

$$\alpha_k=\frac{(\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)})}{(\boldsymbol{p}^{(k)},A\boldsymbol{p}^{(k)})}$$

$$\boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)}+\alpha_k\boldsymbol{p}^{(k)}$$

$$\boldsymbol{r}^{(k+1)}=\boldsymbol{r}^{(k)}-\alpha_kA\boldsymbol{p}^{(k)}$$

$$\beta_k=\frac{(\boldsymbol{r}^{(k+1)},\boldsymbol{r}^{(k+1)})}{(\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)})}$$

$$p^{(k+1)}=\boldsymbol{r}^{(k+1)}+\beta_k\boldsymbol{p}^{(k)}.$$

3. 若 $\boldsymbol{r}^{(k)}=0$ 或 $(\boldsymbol{p}^{(k)},A\boldsymbol{p}^{(k)})=0$，计算停止，$\boldsymbol{x}^{(k)}=\boldsymbol{x}^*$。

（4）收敛性定理与预条件必要性

CG 算法满足误差估计式：

$$\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^*\|_A\le 2\left(\frac{\sqrt{K}-1}{\sqrt{K}+1}\right)^k\|\boldsymbol{x}^{(0)}-\boldsymbol{x}^*\|_A,$$

其中 $\|\boldsymbol{x}\|_A=(A\boldsymbol{x},\boldsymbol{x})^{1/2}$，$K=\mathrm{cond}(A)_2$ 为条件数。当 $K\gg 1$（即 A 为病态矩阵）时，CG法收敛极慢。预条件的目的正是降低有效条件数，改善收敛速度。

（5）预条件所需条件与 PCG 迭代

预条件的核心是寻找一个非奇异且对称正定的矩阵 M（称为预条件子），将原方程组转化为：$M^{-1}A\boldsymbol{x}=M^{-1}\boldsymbol{b}$。为了保证转化后的系统仍能使用 CG 法，必须满足 $M^{-1}A$ 在 M 内积下是对称正定的，这等价于要求 M 本身必须是对称正定矩阵。最简单常用的预条件是取 $M=\mathrm{diag}(A)$（对角预条件），此时求解 $M\boldsymbol{z}=\boldsymbol{r}$ 只需 $O(n)$ 的除法运算，成本极低，但能显著改善条件数。

条件不满足时的限制：若 A 不对称或不正定，由于二次函数不成立且无法保证共轭方向构成基，标准 CG 不能保证收敛。此时需采用 GMRES 等适用于非对称矩阵的方法。若预条件子 M 不对称正定，PCG 的理论基础同样被破坏。

实验设计：先预测，再运行

实验任务：对同一正定系统比较CG与对角预条件CG，在相同停止标准下记录迭代次数和实际误差。

要填写的项目	本组内容
要回答的问题与运行前预测	问题：对角预条件能否减少迭代次数？计入预条件应用成本后，总时间是否一定更短？ 预测：对于条件数较大的正定矩阵，PCG 迭代次数应明显少于 CG；但由于每步多一次对角求解，在问题规模较小时，总时间可能相差不大。
输入、参数、单位、数据来源与划分	构造对称正定矩阵 $A = Q\Lambda Q^T$，其中 Q 为正交矩阵，$\Lambda = \mathrm{diag}(\lambda_i)$，取 λ_i 跨度较大（如 1 到 10^4）以产生病态条件数。右端 $b = Ax_{\text{true}}$，x_{true} 取全1向量。无单位。
独立参考解/基线及选择理由	用 `scipy.linalg.solve`（底层为 LU 分解）计算 x_{ref} 作为“真解”参考。基线选择理由：对于中等规模问题，直接法在浮点精度下可认为给出精确解，用于计算真实误差。
误差指标、容差及其依据	停止准则：$\|r^{(k)}\|_2 \le 10^{-8}\|b\|_2$。报告指标：迭代次数 k、最终相对残量、最终真实误差 $\|x^{(k)} - x_{\text{ref}}\|_2/\|x_{\text{ref}}\|_2$、总计算时间。容差 10^{-8} 依据：该精度下舍入误差影响已不可忽略，但足以展示两种方法的差异。
环境、种子、设备与运行预算	Python 3 + NumPy + SciPy + Matplotlib；固定随机种子 `seed=42` 生成正交矩阵 Q；问题规模 $n = 500$（可调）；运行预算：单次 CG/PCG 应在秒级完成。

结果解释与本节结论

实际结果

根据实验输出，对于规模 $n = 500$、条件数 $\mathrm{cond}(A) \approx 1.00 \times 10^4$ 的对称正定系统：

- 参考解精度：直接法（LU分解）得到的参考解相对误差极小（2.74×10^{-13}），可作为真解基线。
- 迭代次数：标准 CG 迭代 552 次，对角 PCG 迭代 545 次。两者的迭代次数非常接近，PCG 仅减少了 7 次迭代。
- 总耗时：标准 CG 耗时 0.0816 秒，对角 PCG 耗时 0.0753 秒。PCG 略快于 CG。
- 最终真实误差：在达到相对残量容差 10^{-8} 时，标准 CG 的相对真实误差为 2.18×10^{-6}，对角 PCG 为 2.27×10^{-6}，两者精度处于同一量级。
- 收敛历史图：如图中所示（蓝色实线为 CG，橙色实线为 PCG，红色虚线为容差 10^{-8}），两条残差下降曲线几乎重合。残差在迭代初期（0~50次）下降极快，随后在对数坐标下呈现近乎直线的下降趋势，但在迭代中后期（约 200 次之后）出现了明显的锯齿状震荡。

与预测或理论是否一致

- 关于迭代次数：预测认为“对于条件数较大的正定矩阵，PCG 迭代次数应明显少于 CG”。实验结果与预测不一致。理论上，对角预条件通过缩放矩阵对角线来降低条件数，但本实验中的矩阵 $A = Q\Lambda Q^T$ 的生成方式随机，其对角元素并不悬殊（或未主导整个矩阵的谱分布），因此对角预条件未能有效改善矩阵的条件数，导致迭代次数几乎没有减少。
- 关于总时间：预测认为“在问题规模较小时，总时间可能相差不大”。实验结果与预测一致。PCG 每步需要额外进行一次对角求解，但迭代次数没有显著减少，所以总时间几乎持平（PCG 略快可能是由于随机波动或向量化操作带来的细微差异）。

不一致的原因

本实验构造的矩阵 $A = Q\Lambda Q^T$ 是一个“旋转”后的对角阵。对于这种矩阵，其对角线上的元素通常是相近的。对角预条件 $M = \text{diag}(A)$ 实际上近似于一个单位阵的倍数，因此预条件后的系统 $M^{-1}A$ 与 A 的条件数几乎相同。预条件子失效是导致迭代次数没有明显减少的根本原因。在实际应用中，对角预条件更适用于对角线元素量级差异极大的矩阵（如偏微分方程离散化后的矩阵）。

尚未验证的情况

1. 更有效的预条件子：如不完全 Cholesky 分解或不完全 LU 分解，在保持稀疏性的同时能显著降低条件数，预期能大幅减少迭代次数。
2. 更大规模的问题：当 n 增大到 10^4 或 10^5 时，预条件成本与迭代次数的权衡关系可能会发生变化。
3. 真实误差与残差的关系：图中显示残量已降至 10^{-8}，但真实误差仅为 10^{-6} 量级。这是由于矩阵条件数较大（10^4），误差放大因子（$\kappa(A)$）导致残差小并不等价于误差小。可以进一步验证当容差设定为 10^{-10} 或更小时，真实误差是否继续下降。

结论与证据：

- 证据：图中 CG 与 PCG 的残差曲线几乎完全重叠，且最终收敛到同一容差线。
- 结论：对于本实验随机生成的正交相似对角化矩阵，对角预条件未能有效改善条件数，因此没有显著减少迭代次数。在总耗时方面，由于迭代次数相近，计入预条件应用成本后，总时间并未显著缩短。这警示我们在选择预条件子时，必须针对矩阵的具体结构进行设计，盲目使用对角预条件对某些病态矩阵可能无效。

易错点与讨论

待核验的风险：残差小不一定解误差小；迭代次数少不一定总时间短。

用推导回应第一条风险：由 $r = b - Ax$ 和 $x^* = A^{-1}b$ 可得 $x^* - x = A^{-1}r$，因此：

$$\|x^* - x\| = \|A^{-1}r\| \le \|A^{-1}\| \cdot \|r\|$$

当 A 病态时，$\|A^{-1}\|$ 很大，即使残量 $\|r\|$ 很小，解误差仍可能很大。反之，若 $\|A\|$ 很大，也可能出现残量大而解误差小的情况。残量和解误差之间的定量联系依赖于条件数。

用实验回应第二条风险：对角预条件每步成本为 $O(n)$（仅对角缩放），远低于矩阵向量乘。但在某些问题中，迭代次数减少不够显著（如条件数本来就不大），总时间可能因每步额外操作而略增。对于更复杂的预条件（如SSOR），每步需解三角方程组，在问题规模不够大或并行度不足时，预条件成本可能抵消迭代次数减少带来的收益。

随堂提问： 给定对称正定矩阵 A（条件数约 10^3），右端 b，停止准则 $\|r\|_2 \le 10^{-6}\|b\|_2$。要求分别记录CG和PCG的（a）迭代次数，（b）矩阵向量乘次数，（c）预条件应用次数，（d）总浮点运算量估计或实际运行时间。

同学可能的回答：

- “PCG迭代次数少，所以一定更快。”——需追问：每步多一次 $M^{-1}r$ 的计算，若 M 是稠密矩阵，单步成本可能超过CG一步。
- “迭代次数是唯一评价标准。”——需指出：在并行计算中，迭代次数减少可降低同步开销；但在串行环境下，单步成本同样关键。

本组最终解释：评价预条件优劣需综合迭代次数与单步成本。对于对角预条件，单步成本增量小，收益通常为正；对于IC/SSOR等，需具体分析问题规模、稀疏模式和硬件环境。理论指导是：好的预条件应使 $M^{-1}A$ 的条件数接近1，且 $M^{-1}z$ 的求解成本远低于 $A^{-1}z$。

4.3 特征值与SVD（2学时）

本节目标： 能区分两种分解，并用奇异值解释敏感性和低秩近似。

本节目标： 能严格区分特征值与奇异值两种分解；能证明 SVD 存在性、奇异值与 A^TA 特征值的关系、条件数扰动上界和 Eckart – Young 定理；能用图形直观理解敏感性、低秩近似及其边界。

三个关键词： 特征方向、奇异值谱、低秩近似。

先修知识： 矩阵乘法、转置、内积、特征多项式、向量与矩阵范数、正交矩阵、秩-零化度定理、紧集上连续函数取极值。

本节一句话： 特征值回答“方阵在哪些方向上只伸缩”，SVD 回答“任意矩阵把哪些正交方向拉伸成哪些正交方向”；奇异值谱是矩阵的“体检报告”，最大与最小奇异值之比给出条件数，截断奇异值给出低秩压缩的误差界。

本节结构：

小节	内容	核心问题
4.3.1	概念讲解	特征值、SVD、秩、条件数、伪逆、截断 SVD、幂迭代是什么？
4.3.2	关键推导（含证明）	为什么 SVD 存在？为什么 $\sigma_i = \sqrt{\lambda_i(A^TA)}$？为什么条件数放大扰动？为什么 Eckart – Young 成立？
4.3.3	实验设计	如何用可复核的小矩阵验证上述结论？
4.3.4	结果解释与结论	实验结果与理论是否一致？边界在哪里？
4.3.5	易错点与讨论	特征值当奇异值、低秩当预测、PCA 不中心化、残差当误差等

4.3.1 概念讲解：把问题说明白

从两个问题开始

在进入定义之前，先看两个问题。

问题一： 一个方阵作用在向量上，哪些方向只被伸缩、不被转向？这是特征值要回答的。

问题二： 一个任意形状的矩阵把单位球变成了什么形状？最大拉伸和最小拉伸差多少？这是奇异值要回答的。

这两个问题看起来相近，答案却常常不同。下面的概念讲解会反复回到这两个问题，并用图形和小例子说明：什么时候特征值和奇异值一致，什么时候不能混用。

特征值与特征向量

先看方阵。设 $A \in \mathbb{R}^{n\times n}$。如果存在非零向量 v 和数 λ，使

$$Av = \lambda v,$$

就说 λ 是 A 的特征值，v 是对应特征向量。几何上，A 作用在 v 上不改变方向（$\lambda>0$）或反转方向（$\lambda<0$），只把长度伸缩 $|\lambda|$ 倍；$\lambda=0$ 时 v 被压到零，说明 A 奇异。

等价地，λ 是特征多项式 $p_A(\lambda)=\det(A-\lambda I)$ 的根，v 是 $(\lambda I - A)$ 的零空间中的非零向量。特征值满足

$$\sum_{i=1}^{n}\lambda_i = \operatorname{tr}(A), \qquad \prod_{i=1}^{n}\lambda_i = \det(A),$$

其中特征值按代数重数计。这两个等式在实验和手算中很好用。

一个容易忽略的限制： 特征值只对方阵定义；特征向量一般不一定正交，甚至可能不够 n 个线性无关。例如

$$A=\begin{bmatrix}1 & 1\\ 0 & 1\end{bmatrix}$$

的特征值只有 1（二重），但 $A-I=\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}$ 的零空间只有一维 $\operatorname{span}\{(1,0)^{\mathsf T}\}$，所以只有一维特征空间，不能对角化。这种矩阵称为亏损矩阵。

图 4.3-1 直观对比了对称矩阵和非对称矩阵：对称矩阵的特征方向正交，可以拿来当“正交坐标轴”；非对称矩阵的特征方向不一定正交，甚至可能几乎平行，不能直接当正交基用。SVD 的奇异向量则总是正交的。

图 4.3-1：特征向量与奇异向量的几何对比

```
\begin{tikzpicture}
  % 左：对称矩阵
  \begin{scope}
    \clip (-2.2,-2.2) rectangle (2.2,2.2);
    \draw[grid] (-2.2,-2.2) grid[step=0.5] (2.2,2.2);
    \draw[axis] (-2.2,0) -- (2.2,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,-2.2) -- (0,2.2) node[above,lbl]{\(x_2\)};

    \draw[ell] (0,0) ellipse (1.6 and 0.8);
    \draw[vec] (0,0) -- (1.6,0) node[below right,lbl]{\(v_1\)};
    \draw[vec] (0,0) -- (0,0.8) node[above left,lbl]{\(v_2\)};

    \node[sublbl] at (0,-2.6) {对称矩阵：\(v_1 \perp v_2\)};
  \end{scope}

  % 右：非对称矩阵
  \begin{scope}[xshift=5.5cm]
    \clip (-2.2,-2.2) rectangle (2.2,2.2);
    \draw[grid] (-2.2,-2.2) grid[step=0.5] (2.2,2.2);
    \draw[axis] (-2.2,0) -- (2.2,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,-2.2) -- (0,2.2) node[above,lbl]{\(x_2\)};

    \begin{scope}[rotate=25]
      \draw[ell] (0,0) ellipse (1.8 and 0.5);
    \end{scope}
    \draw[svec] (0,0) -- (1.63,0.76) node[above right,lbl]{\(u_1\)};
    \draw[svec] (0,0) -- (-0.21,0.45) node[above left,lbl]{\(u_2\)};
```

```
    \draw[vec] (0,0) -- (1.6,0) node[below right,lbl]{\(v_1\)};
    \draw[vec] (0,0) -- (0.6,1.0) node[above right,lbl]{\(v_2\)};

    \node[sublbl] at (0,-2.6) {非对称：\(v_1 \not\perp v_2\)，\(u_1 \perp u_2\)};
  \end{scope}
\end{tikzpicture}
```

图注：对称矩阵的特征方向就是椭圆的主轴；非对称矩阵的特征方向（红实线）一般不与椭圆主轴（蓝虚线，即奇异方向）重合，也不一定正交。

可对角化与谱定理

如果存在可逆矩阵 P 和对角矩阵 Λ 使 $A = P\Lambda P^{-1}$，就说 A 可对角化。但对一类特殊矩阵，情形非常好：实对称矩阵。

定理 4.3.1（实对称矩阵谱定理） 设 $A \in \mathbb{R}^{n\times n}$ 对称。则：

1. A 的所有特征值为实数；
2. 属于不同特征值的特征向量互相正交；
3. 存在正交矩阵 Q 和对角矩阵 Λ 使 $A = Q\Lambda Q^{\mathsf{T}}$，即 A 可正交对角化。

证明见 4.3.2 推导 2。谱定理是 SVD 的“对称版本”：对称矩阵已经能用正交基对角化；SVD 要把这件事推广到任意矩阵。

SVD：任意矩阵的正交拉伸

任意矩阵 $A \in \mathbb{R}^{m\times n}$ 不一定是方阵、不一定对称、不一定满秩。但 SVD 仍然存在。

定义 4.3.3（奇异值分解） 设 $A \in \mathbb{R}^{m\times n}$。若存在正交矩阵 $U \in \mathbb{R}^{m\times m}$、$V \in \mathbb{R}^{n\times n}$ 和对角形矩阵 $\Sigma \in \mathbb{R}^{m\times n}$，使

$$A = U\Sigma V^{\mathsf{T}},$$

则称其为 A 的奇异值分解。Σ 的非负对角元

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$$

称为奇异值。若 $r = \operatorname{rank}(A)$，则恰有 r 个正奇异值。

定理 4.3.2（SVD 存在性） 任意实矩阵 $A \in \mathbb{R}^{m\times n}$ 都存在奇异值分解。构造性证明见 4.3.2 推导 3。

图 4.3-2 把 SVD 的几何意义画成 2×2 流程：左上输入单位圆，右上经 V^{T} 旋转，左下经 Σ 拉伸，右下经 U 旋转输出。单位圆最终变成椭圆，半轴长度就是奇异值。

图 4.3-2：SVD 的 2×2 流程

```
\begin{tikzpicture}[>=Stealth]
  % 左上：输入
  \begin{scope}
    \draw[highlightbox] (-2,-2) rectangle (2,2);
    \draw[grid] (-1.8,-1.8) grid[step=0.5] (1.8,1.8);
    \draw[axis] (-1.8,0) -- (1.8,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,-1.8) -- (0,1.8) node[above,lbl]{\(x_2\)};
    \draw[ell] (0,0) circle (1);
```

```
    \draw[vec] (0,0) -- (0.707,0.707) node[above right,lbl]{\(v_1\)};
    \draw[vec] (0,0) -- (-0.707,0.707) node[above left,lbl]{\(v_2\)};
    \node[sublbl, below] at (0,-2.2) {输入：单位圆};
  \end{scope}

  % 箭头 V^T
  \draw[->, thick, mygray!80] (2.5,0) -- (4.0,0) node[midway,above,lbl]{\(V^{\mathsf T}\)};

  % 右上：旋转
  \begin{scope}[xshift=6.5cm]
    \draw[highlightbox] (-2,-2) rectangle (2,2);
    \draw[grid] (-1.8,-1.8) grid[step=0.5] (1.8,1.8);
    \draw[axis] (-1.8,0) -- (1.8,0) node[right,lbl]{\(z_1\)};
    \draw[axis] (0,-1.8) -- (0,1.8) node[above,lbl]{\(z_2\)};
    \draw[ell] (0,0) circle (1);
    \draw[vec] (0,0) -- (1,0) node[below right,lbl]{\(e_1\)};
    \draw[vec] (0,0) -- (0,1) node[above left,lbl]{\(e_2\)};
    \node[sublbl, below] at (0,-2.2) {旋转：坐标对齐};
  \end{scope}

  % 箭头 Sigma（向下）
  \draw[->, thick, mygray!80] (6.5,-2.5) -- (6.5,-4.0) node[midway,left,lbl]{\(\Sigma\)};

  % 左下：拉伸
  \begin{scope}[yshift=-9cm]
    \draw[highlightbox] (-3,-2) rectangle (3,2);
    \draw[grid] (-2.8,-1.8) grid[step=0.5] (2.8,1.8);
    \draw[axis] (-2.8,0) -- (2.8,0) node[right,lbl]{\(w_1\)};
    \draw[axis] (0,-1.8) -- (0,1.8) node[above,lbl]{\(w_2\)};
    \draw[ell] (0,0) ellipse (2.2 and 0.6);
    \draw[vec] (0,0) -- (2.2,0) node[below right,lbl]{\(\sigma_1 e_1\)};
    \draw[vec] (0,0) -- (0,0.6) node[above left,lbl]{\(\sigma_2 e_2\)};
    \node[sublbl, below] at (0,-2.2) {拉伸：\(\sigma_1,\sigma_2\)};
  \end{scope}

  % 箭头 U（向右）
  \draw[->, thick, mygray!80] (3.5,-9) -- (5.0,-9) node[midway,above,lbl]{\(U\)};

  % 右下：输出
  \begin{scope}[xshift=8.5cm,yshift=-9cm]
    \draw[highlightbox] (-3,-2) rectangle (3,2);
    \draw[grid] (-2.8,-1.8) grid[step=0.5] (2.8,1.8);
    \draw[axis] (-2.8,0) -- (2.8,0) node[right,lbl]{\(y_1\)};
    \draw[axis] (0,-1.8) -- (0,1.8) node[above,lbl]{\(y_2\)};
    \begin{scope}[rotate=30]
      \draw[ell] (0,0) ellipse (2.2 and 0.6);
    \end{scope}
    \draw[vec] (0,0) -- (1.9,1.1) node[above right,lbl]{\(\sigma_1 u_1\)};
    \draw[vec] (0,0) -- (-0.3,0.52) node[above left,lbl]{\(\sigma_2 u_2\)};
    \node[sublbl, below] at (0,-2.2) {输出：椭圆};
  \end{scope}
\end{tikzpicture}
```

图注：$V^{\mathsf T}$ 旋转输入坐标，Σ 沿坐标轴拉伸，U 旋转输出坐标。单位圆最终变成椭圆，半轴长度为奇异值 σ_1, σ_2。

条件数：矩阵的“扁平程度”

有了奇异值，条件数可以看成“椭圆有多扁”。图 4.3-3 对比了良态矩阵和病态矩阵：良态矩阵把单位圆变成接近圆的椭圆，$\sigma_{\max}/\sigma_{\min} \approx 1$；病态矩阵把单位圆压成很扁的椭圆，$\sigma_{\max}/\sigma_{\min}$ 很大。求逆时，扁的方向被放大 $1/\sigma_{\min}$ 倍，误差主要来自这个方向。

图 4.3-3：条件数的几何意义

```
\begin{tikzpicture}
  % 良态
  \begin{scope}
    \draw[highlightbox] (-2.5,-2.5) rectangle (2.5,2.5);
    \draw[grid] (-2,-2) grid[step=0.5] (2,2);
    \draw[axis] (-2,0) -- (2,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,-2) -- (0,2) node[above,lbl]{\(x_2\)};
    \draw[ell] (0,0) ellipse (1.5 and 1.2);
    % 标注长短轴
    \draw[<->, myred, thick] (-1.5,0.1) -- (1.5,0.1);
    \draw[<->, myred, thick] (0.1,-1.2) -- (0.1,1.2);
    \node[sublbl, align=center, below] at (0,-2.5) {良态：椭圆接近圆\\\(\kappa_2 \approx 1.25\)};
  \end{scope}
  % 病态
  \begin{scope}[xshift=6cm]
    \draw[highlightbox] (-3,-2.5) rectangle (3,2.5);
    \draw[grid] (-2.5,-2) grid[step=0.5] (2.5,2);
    \draw[axis] (-2.5,0) -- (2.5,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,-2) -- (0,2) node[above,lbl]{\(x_2\)};
    \draw[ell] (0,0) ellipse (2.5 and 0.2);
    % 标注长短轴
    \draw[<->, myred, thick] (-2.5,-0.3) -- (2.5,-0.3);
    \draw[<->, myred, thick] (2.7,-0.2) -- (2.7,0.2);
    \node[sublbl, align=center, below] at (0,-2.5) {病态：椭圆极扁\\\(\kappa_2 \approx 12.5\)};
  \end{scope}
\end{tikzpicture}
```

图注：单位圆被 A 映射成椭圆，椭圆越扁，条件数越大。扁的方向对应最小奇异值，求逆时误差被放大最多。

奇异值与 $A^{\mathsf{T}}A$、AA^{T} 的关系

SVD 和对称矩阵的特征值之间有一座桥。

定理 4.3.3 设 $A \in \mathbb{R}^{m\times n}$，$r = \operatorname{rank}(A)$。则：

1. $A^{\mathsf{T}}A$ 和 AA^{T} 都是对称半正定矩阵；
2. $A^{\mathsf{T}}A$ 的非零特征值恰为 $\sigma_1^2, \dots, \sigma_r^2$；
3. $\sigma_i(A) = \sqrt{\lambda_i(A^{\mathsf{T}}A)}$，$\lambda_i(A^{\mathsf{T}}A)$ 按降序排列；
4. $\operatorname{rank}(A) = \operatorname{rank}(A^{\mathsf{T}}A) = \operatorname{rank}(AA^{\mathsf{T}}) =$ 正奇异值个数；
5. $\|A\|_2 = \sigma_{\max}$，$\|A\|_F = \sqrt{\sum_i \sigma_i^2}$。

证明见 4.3.2 推导 4。注意：$\sigma_i(A) = \sqrt{\lambda_i(A^{\mathsf{T}}A)}$ 中的 $\lambda_i(A^{\mathsf{T}}A)$ 必须按降序排列且非负，不能写成“A 的特征值绝对值”。例 4.3.2 会给出最小反例。

截断 SVD 与 Eckart–Young

把 SVD 写成外积和：

$$A = \sum_{i=1}^{r} \sigma_i u_i v_i^{\mathsf{T}}.$$

如果只保留前 k 项，得到

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^{\mathsf{T}}.$$

则 $\operatorname{rank}(A_k) \le k$。图 4.3-4 把奇异值按大小画成柱状图：保留的柱子是蓝色，丢弃的柱子是灰色；截断 SVD 只保留前 k 根柱子，丢掉的部分就是误差来源。

图 4.3-4：奇异值谱与截断 SVD

```
\begin{tikzpicture}
  \begin{axis}[
    ybar,
    bar width=14pt,
    axis lines=left,
    xmin=0.5, xmax=9.5,
    ymin=0, ymax=10,
    xtick={1,2,3,4,5,6,7,8},
    xticklabels={1,2,3,4,5,6,7,8},
    xlabel={奇异值序号 \(i\)},
    ylabel={奇异值 \(\sigma_i\)},
    ymajorgrids=true,
    grid style={dashed, mygray!30},
    tick label style={font=\small\sffamily, mygray!80!black},
    label style={font=\small\sffamily, mygray!80!black},
    enlarge x limits=0.05,
    nodes near coords,
    every node near coord/.append style={font=\footnotesize\sffamily, /pgf/number format/fixed, /pgf/number
format/precision=2}
  ]
    % 保留的奇异值（蓝色）
    \addplot[fill=myblue!80, draw=myblue!80] coordinates {
      (1,9.5) (2,7.0) (3,4.5) (4,2.8)
    };
    % 丢弃的奇异值（灰色）
    \addplot[fill=mygray!40, draw=mygray!40] coordinates {
      (5,1.6) (6,0.8) (7,0.35) (8,0.12)
    };

    % 截断线
    \draw[myred, thick, dashed] (axis cs:4.5,0) -- (axis cs:4.5,10);
    \node[myred, font=\small\sffamily\bfseries, above] at (axis cs:4.5,9.8) {截断 \(k=4\)};

    % 底部标注
    \draw[decorate,decoration={brace,amplitude=8pt,mirror},thick,mygray!70]
      (axis cs:4.8,-1.5) -- (axis cs:8.2,-1.5) node[midway,below=8pt,font=\small\sffamily,mygray!80!black]{被丢弃的
奇异值};
  \end{axis}
\end{tikzpicture}
```

图注：保留的柱子为蓝色，丢弃的柱子为灰色；谱范数误差为 σ_{k+1}，F 范数误差为剩余奇异值平方和开方。

定理 4.3.4（Eckart–Young） 设 $A \in \mathbb{R}^{m\times n}$，$r=\operatorname{rank}(A)$，$0 \le k \le r$。则

$$\|A-A_k\|_2=\sigma_{k+1}, \qquad \|A-A_k\|_F=\sqrt{\sum_{i=k+1}^{r}\sigma_i^2},$$

且对任意 $\operatorname{rank}(B)\le k$ 的 B，有 $\|A-B\|_2 \ge \sigma_{k+1}$，$\|A-B\|_F \ge \sqrt{\sum_{i=k+1}^{r}\sigma_i^2}$。

Eckart–Young 只保证重构范数最优，不保证下游任务（分类、回归、推荐）指标最优。

幂迭代

给定方阵 A 和非零初值 $v^{(0)}$，迭代

$$w^{(t)}=Av^{(t)}, \qquad v^{(t+1)}=\frac{w^{(t)}}{\|w^{(t)}\|_2},$$

并用 Rayleigh 商 $\rho^{(t)}=\frac{(v^{(t)})^{\mathsf{T}}Av^{(t)}}{(v^{(t)})^{\mathsf{T}}v^{(t)}}$ 估计主特征值。

若 A 可对角化且主特征值在绝对值意义下严格占优，则 $v^{(t)}$ 收敛到主特征向量（差一个符号），$\rho^{(t)}\to\lambda_1$，收敛速率由 $|\lambda_2/\lambda_1|^t$ 控制。证明见 4.3.2 推导 8。

三个例子

例 4.3.1（对称正定矩阵） 取 $D=\operatorname{diag}(3,1)$。特征值为 $3,1$，奇异值也是 $3,1$。单位圆被 D 映射成半轴为 $3,1$ 的椭圆。

例 4.3.2（非对称矩阵，特征值与奇异值不同） 取

$$A=\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}.$$

特征值 $0,0$；$A^{\mathsf{T}}A=\operatorname{diag}(0,1)$，特征值 $1,0$，故奇异值 $1,0$。特征值绝对值 $0,0$ 与奇异值 $1,0$ 不同。该矩阵把 e_2 映到 e_1，再把 e_1 映到零：存在一次非零拉伸，秩为 1；但作为迭代算子，它两次作用后归零。这是“特征值不能当奇异值用”的最小反例。

例 4.3.3（非正规矩阵，特征值对扰动极敏感） 取

$$A=\begin{bmatrix}1 & 1/\varepsilon\\ 0 & 1\end{bmatrix}, \qquad E=\begin{bmatrix}0 & 0\\ \varepsilon & 0\end{bmatrix}, \qquad \|E\|_2=\varepsilon.$$

A 的特征值为 $1,1$（二重，亏损）。$A+E$ 的特征多项式为 $(1-\lambda)^2-1=\lambda(\lambda-2)$，特征值为 $0,2$。扰动 $\|E\|_2=\varepsilon$ 使特征值移动了 $O(1)$，与 ε 无关。在复平面上可以清楚地看到这个现象：A 的两个特征值都在 1；加小扰动后一个跑到 0、一个跑到 2。而奇异值满足 Weyl 不等式 $|\sigma_i(A+E)-\sigma_i(A)|\le\|E\|_2$，不会这样跳。

4.3.2 关键推导：让结论有依据

这一节给出八个推导，顺序对应 4.3.1 的结论。每个推导先说“为什么要证”，再给证明，最后说“条件不满足时会怎样”。

推导 1：特征值的基本性质

为什么要证： 迹和行列式是核对特征值的最快工具；实验里算完特征值，用它们检查有没有算错。

命题 4.3.1 设 $A\in\mathbb{C}^{n\times n}$，特征值（按代数重数）为 $\lambda_1,\dots,\lambda_n$。则 $\sum_{i=1}^{n}\lambda_i=\operatorname{tr}(A)$，$\prod_{i=1}^{n}\lambda_i=\det(A)$。

证明： 迹： 特征多项式 $p_A(\lambda)=\det(A-\lambda I)=(-1)^n\prod_{i=1}^n(\lambda-\lambda_i)$。按行列式定义展开，$\lambda^{n-1}$ 的系数为 $(-1)^{n-1}\operatorname{tr}(A)$。比较系数得 $\sum_i\lambda_i=\operatorname{tr}(A)$。 行列式： 令 $\lambda=0$，得 $p_A(0)=\det(A)=(-1)^n\prod_i(-\lambda_i)=\prod_i\lambda_i$。□

推导 2：实对称矩阵谱定理

为什么要证： 谱定理是 SVD 的“对称版本”。理解了对称矩阵为什么能正交对角化，就更容易理解 SVD 为什么对任意矩阵存在。

定理 4.3.5（谱定理） 设 $A\in\mathbb{R}^{n\times n}$ 对称。则 A 的特征值全为实数，属于不同特征值的特征向量正交，且存在正交矩阵 Q 使 $A=Q\Lambda Q^{\mathsf T}$，Λ 为实对角矩阵。

证明分三步。 第一步：特征值为实数。 设 $Av=\lambda v$，$v\neq0$，$v\in\mathbb{C}^n$。取共轭转置：$v^*A^*=\bar\lambda v^*$。因 A 实对称，$A^*=A$，故 $v^*A=\bar\lambda v^*$。两边右乘 v：$v^*Av=\bar\lambda v^*v$。另一方面，由 $Av=\lambda v$ 左乘 v^*：$v^*Av=\lambda v^*v$。两式相减：$(\lambda-\bar\lambda)v^*v=0$。因 $v\neq0$，$v^*v=\|v\|_2^2>0$，故 $\lambda=\bar\lambda$，即 $\lambda\in\mathbb{R}$。 第二步：不同特征值的特征向量正交。 设 $Av_1=\lambda_1v_1$，$Av_2=\lambda_2v_2$，$\lambda_1\neq\lambda_2$。因 A 对称且 λ_i 实，

$$\lambda_1v_1^{\mathsf T}v_2=(Av_1)^{\mathsf T}v_2=v_1^{\mathsf T}A^{\mathsf T}v_2=v_1^{\mathsf T}Av_2=\lambda_2v_1^{\mathsf T}v_2.$$

故 $(\lambda_1-\lambda_2)v_1^{\mathsf T}v_2=0$。因 $\lambda_1\neq\lambda_2$，得 $v_1^{\mathsf T}v_2=0$。 第三步：存在 n 个正交特征向量（对 n 归纳）。$n=1$ 显然。设对 $n-1$ 成立。取 A 的一个单位特征向量 q_1。令 $Q_1=[q_1,W]$ 为正交矩阵（W 的列张成 $q_1^{\perp}$）。则

$$Q_1^{\mathsf T}AQ_1=\begin{bmatrix}\lambda_1&0\\0&B\end{bmatrix},$$

其中 $B=W^{\mathsf T}AW\in\mathbb{R}^{(n-1)\times(n-1)}$ 对称。由归纳假设，B 有 $n-1$ 个正交特征向量。则 q_1 和 $Q_1[0,w_i^{\mathsf T}]^{\mathsf T}$ 构成 A 的 n 个正交特征向量。归一化后组成正交矩阵 Q，得 $A=Q\Lambda Q^{\mathsf T}$。□

推论 4.3.1 若 A 对称半正定，则所有特征值 ≥0。 证明： 设 $Av=\lambda v$，$v\neq0$，则 $\lambda\|v\|^2=v^{\mathsf T}Av\geq0$，故 $\lambda\geq0$。因此 $A^{\mathsf T}A$ 的特征值非负，开方才有意义。□

推导 3：SVD 存在性

为什么要证： SVD 的定义只说了“若存在”，没有说“一定存在”。这个证明是构造性的：它告诉你最大奇异方向怎么找，然后降维递归。

定理 4.3.6（SVD 存在性） 任意 $A\in\mathbb{R}^{m\times n}$ 都存在 SVD。

证明（对 $m+n$ 归纳）： 第一步：最大拉伸方向存在。 定义 $f(x)=\|Ax\|_2$，定义域为单位球面 S^{n-1}。S^{n-1} 是紧集，f 连续，故取最大值。记

$$\sigma_1=\max_{\|x\|_2=1}\|Ax\|_2=\|A\|_2\geq0.$$

取 $v_1\in S^{n-1}$ 使 $\|Av_1\|_2=\sigma_1$。若 $\sigma_1=0$，则 $A=0$，取任意正交 U,V 和 $\Sigma=0$ 即可。以下设 $\sigma_1>0$。 第二步：构造 u_1。 令 $u_1=Av_1/\sigma_1$。由 Lagrange 乘子法，存在 μ 使 $A^{\mathsf T}Av_1=\mu v_1$。左乘 $v_1^{\mathsf T}$ 得 $\mu=\|Av_1\|^2=\sigma_1^2$。故 $A^{\mathsf T}Av_1=\sigma_1^2v_1$，于是

$$A^{\mathsf T}u_1=\frac{1}{\sigma_1}A^{\mathsf T}Av_1=\frac{\sigma_1^2}{\sigma_1}v_1=\sigma_1v_1.$$

第三步：正交补上的降维。 取 $U_1=[u_1,\tilde U]$ 和 $V_1=[v_1,\tilde V]$ 为正交矩阵。则

$$U_1^{\mathsf{T}} A V_1 = \begin{bmatrix} \sigma_1 & 0 \\ 0 & A_1 \end{bmatrix},$$

其中 $A_1 = \tilde{U}^{\mathsf{T}} A \tilde{V}$。逐项理由：$u_1^{\mathsf{T}} A v_1 = \sigma_1$；$u_1^{\mathsf{T}} A \tilde{V} = 0$ 因为 $A^{\mathsf{T}} u_1 = \sigma_1 v_1 \perp v_1^{\perp}$；$\tilde{U}^{\mathsf{T}} A v_1 = 0$ 因为 $A v_1 = \sigma_1 u_1 \perp u_1^{\perp}$。 第四步：归纳。 对 A_1 用归纳假设，存在 SVD $A_1 = U_2 \Sigma_2 V_2^{\mathsf{T}}$。令

$$U = U_1 \begin{bmatrix} 1 & 0 \\ 0 & U_2 \end{bmatrix}, \quad V = V_1 \begin{bmatrix} 1 & 0 \\ 0 & V_2 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix},$$

则 $A = U \Sigma V^{\mathsf{T}}$。归纳完成。□

推导 4：$\sigma_i(A) = \sqrt{\lambda_i(A^{\mathsf{T}} A)}$

为什么要证： 这个等式把 SVD 和对称特征值问题连起来，是计算和核对奇异值的主要工具。但它也最容易被误读成“A 的特征值绝对值开方”。

定理 4.3.7 设 $A = U \Sigma V^{\mathsf{T}}$ 是 SVD。则 $A^{\mathsf{T}} A = V \Sigma^{\mathsf{T}} \Sigma V^{\mathsf{T}}$，且 $\sigma_i(A) = \sqrt{\lambda_i(A^{\mathsf{T}} A)}$（$\lambda_i$ 按降序，非负）。

证明： 由 $A = U \Sigma V^{\mathsf{T}}$ 和 $U^{\mathsf{T}} U = I$，

$$A^{\mathsf{T}} A = V \Sigma^{\mathsf{T}} U^{\mathsf{T}} U \Sigma V^{\mathsf{T}} = V (\Sigma^{\mathsf{T}} \Sigma) V^{\mathsf{T}}.$$

$\Sigma^{\mathsf{T}} \Sigma$ 是对角矩阵，对角元为 $\sigma_1^2, \dots, \sigma_{\min(m,n)}^2, 0, \dots, 0$。因 V 正交，$A^{\mathsf{T}} A$ 与 $\Sigma^{\mathsf{T}} \Sigma$ 正交相似，特征值相同。开方得 $\sigma_i = \sqrt{\lambda_i(A^{\mathsf{T}} A)}$。□

最容易误读的地方： 定理 4.3.7 不能反过来读成“A 的特征值绝对值开方”。$A^{\mathsf{T}} A$ 的特征值是 σ_i^2，与 A 的特征值 $\lambda_i(A)$ 一般无关。只有正规矩阵才有 $|\lambda_i(A)| = \sigma_i(A)$。例 4.3.2 就是反例。

推导 5：条件数与右端扰动

为什么要证： 本章开场说“小残差、大解误差”，这个定理给出定量上界。图 4.3-5 把几何直观画出来：A 把单位圆压成很扁的椭圆；右端 b 在椭圆内，小扰动 δb 在“扁方向”上，对应的解扰动 $\delta x = A^{-1} \delta b$ 被放大 $1/\sigma_{\min}$ 倍。

图 4.3-5：小残差、大解误差的几何直观

```
\begin{tikzpicture}[>=Stealth]
  % 左：右端空间
  \begin{scope}
    \draw[highlightbox] (-3,-2) rectangle (3,2);
    \draw[grid] (-2.5,-1.5) grid[step=0.5] (2.5,1.5);
    \draw[axis] (-2.5,0) -- (2.5,0) node[right,lbl]{\(y_1\)};
    \draw[axis] (0,-1.5) -- (0,1.5) node[above,lbl]{\(y_2\)};
    \draw[ell] (0,0) ellipse (2.2 and 0.3);

    \fill[myred] (1.6,0) circle (3pt) node[below right=2pt,lbl]{\(b\)};
    \fill[myred!40] (1.6,0.35) circle (3pt);
    \draw[->, thick, myred] (1.6,0.05) -- (1.6,0.30) node[midway,right=2pt,lbl]{\(\delta b\)};

    \node[sublbl, below] at (0,-2.3) {右端空间：\(\delta b\) 很小};
  \end{scope}

  % 箭头 A^{-1}
  \draw[->, ultra thick, mygray!60] (3.3,0) -- (4.7,0) node[midway,above=2pt,lbl]{\(A^{-1}\)};

  % 右：解空间
  \begin{scope}[xshift=8cm]
```

```
    \draw[highlightbox] (-3,-2) rectangle (3,2);
    \draw[grid] (-2.5,-1.5) grid[step=0.5] (2.5,1.5);
    \draw[axis] (-2.5,0) -- (2.5,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,-1.5) -- (0,1.5) node[above,lbl]{\(x_2\)};
    \draw[ell] (0,0) circle (1.5);

    \fill[myred] (1.5,0) circle (3pt) node[below right=2pt,lbl]{\(x\)};
    \fill[myred!40] (1.5,1.5) circle (3pt);
    \draw[->, thick, myred] (1.5,0.05) -- (1.5,1.45) node[midway,right=2pt,lbl]{\(\delta x = A^{-1}\delta b\)};

    \node[sublbl, below] at (0,-2.3) {解空间：\(\delta x\) 被放大};
  \end{scope}
\end{tikzpicture}
```

图注：A 把单位圆压成很扁的椭圆；右端小扰动 δb 在扁方向（对应 $\sigma_{\min}$）上，经 A^{-1} 放大 $1/\sigma_{\min}$ 倍，解扰动 δx 很大。

定理 4.3.8（右端扰动上界） 设 $A \in \mathbb{R}^{n\times n}$ 可逆，$Ax = b$，$A(x+\delta x) = b + \delta b$。则

$$\frac{\|\delta x\|_2}{\|x\|_2} \le \kappa_2(A)\frac{\|\delta b\|_2}{\|b\|_2}.$$

证明： 两式相减得 $\delta x = A^{-1}\delta b$。由 $\|A^{-1}\|_2 = 1/\sigma_{\min}$，

$$\|\delta x\|_2 \le \frac{\|\delta b\|_2}{\sigma_{\min}}.$$

又 $\|b\|_2 \le \sigma_{\max}\|x\|_2$，即 $1/\|x\|_2 \le \sigma_{\max}/\|b\|_2$。两式相乘：

$$\frac{\|\delta x\|_2}{\|x\|_2} \le \frac{\|\delta b\|_2}{\sigma_{\min}}\cdot\frac{\sigma_{\max}}{\|b\|_2} = \kappa_2(A)\frac{\|\delta b\|_2}{\|b\|_2}. \square$$

推导 6：矩阵扰动界

为什么要证： 推导 5 只讨论右端扰动。如果矩阵本身有扰动呢？奇异值和特征值对扰动的反应很不一样，这决定了数值分析该用哪个量讨论敏感性。

定理 4.3.9（Weyl 不等式） 设 $A, E \in \mathbb{R}^{m\times n}$。则对每个 $i = 1,\dots,\min(m,n)$，
$|\sigma_i(A+E) - \sigma_i(A)| \le \|E\|_2$。

证明思路： 由 Ky Fan 变分刻画 $\sigma_i(A) = \max_{\dim S = i}\min_{x\in S,\|x\|=1}\|Ax\|$。对任意 x，
$\|(A+E)x\| \le \|Ax\| + \|E\|_2$，且 $\|(A+E)x\| \ge \|Ax\| - \|E\|_2$。代入变分刻画，内层 $\min$ 的差不超过 $\|E\|_2$，外层 $\max$ 保持不等式。$\square$

定理 4.3.10（逆扰动界） 设 A 可逆，$\|A^{-1}E\|_2 < 1$。则 $A+E$ 可逆，且

$$\frac{\|(A+E)^{-1} - A^{-1}\|_2}{\|A^{-1}\|_2} \le \frac{\kappa_2(A)\|E\|_2/\|A\|_2}{1 - \kappa_2(A)\|E\|_2/\|A\|_2}.$$

证明思路：$(A+E)^{-1} - A^{-1} = -(A+E)^{-1}EA^{-1}$，用 Neumann 级数估计 $\|(A+E)^{-1}\|$。完整证明见 Golub & Van Loan, Matrix Computations, 4th ed., §2.6.2。$\square$

定理 4.3.11（非正规矩阵特征值无 Lipschitz 界） 存在 A, E 使 $|\lambda_i(A+E) - \lambda_i(A)|$ 与 $\|E\|_2$ 之比任意大。

反例（例 4.3.3 的严格化）： $A = \begin{bmatrix} 1 & 1/\varepsilon \\ 0 & 1 \end{bmatrix}$，$\lambda(A) = \{1,1\}$。$E = \begin{bmatrix} 0 & 0 \\ \varepsilon & 0 \end{bmatrix}$，$\|E\|_2 = \varepsilon$。
$A + E = \begin{bmatrix} 1 & 1/\varepsilon \\ \varepsilon & 1 \end{bmatrix}$，

$$\det((A+E)-\lambda I) = (1-\lambda)^2 - 1 = \lambda(\lambda-2),$$

$\lambda(A+E)=\{0,2\}$。特征值移动 $1=O(1)$，而 $\|E\|_2=\varepsilon\to 0$。比值 $1/\varepsilon\to\infty$。□

结论： 奇异值对扰动是 Lipschitz 的（Weyl），特征值对扰动一般不是。这是数值分析中用奇异值讨论敏感性的核心理由。

推导 7：Eckart–Young 定理

为什么要证： “截断 SVD 是最优低秩近似”不能只靠直觉。这个定理给出误差公式，并证明任何秩不超过 k 的矩阵都不能比 A_k 更好。图 4.3-4 已经给出误差来源的直观：丢掉的是后几根灰色柱子。

定理 4.3.12（Eckart–Young） 设 $A=U\Sigma V^{\mathsf T}$，$r=\operatorname{rank}(A)$，$0\le k\le r$，$A_k=\sum_{i=1}^{k}\sigma_i u_i v_i^{\mathsf T}$。则对任意 $\operatorname{rank}(B)\le k$ 的 B，

$$\|A-B\|_2\ge\sigma_{k+1},\qquad \|A-B\|_F\ge\sqrt{\sum_{i=k+1}^{r}\sigma_i^2},$$

且 A_k 取到等号。

证明： 谱范数。 令 $V_{k+1}=\operatorname{span}\{v_1,\dots,v_{k+1}\}$，维数 $k+1$。对任意 $z\in V_{k+1}$，$z=\sum_{i=1}^{k+1}c_i v_i$，则

$$\|Az\|_2^2=\sum_{i=1}^{k+1}c_i^2\sigma_i^2\ge\sigma_{k+1}^2\sum_{i=1}^{k+1}c_i^2=\sigma_{k+1}^2\|z\|_2^2.$$

设 $\|A-B\|_2<\sigma_{k+1}$。则对任意非零 $z\in V_{k+1}$，

$$\|Bz\|_2\ge\|Az\|_2-\|(A-B)z\|_2\ge\sigma_{k+1}\|z\|_2-\|A-B\|_2\|z\|_2>0.$$

故 B 在 V_{k+1} 上单射，$\operatorname{rank}(B)\ge k+1$，与 $\operatorname{rank}(B)\le k$ 矛盾。因此 $\|A-B\|_2\ge\sigma_{k+1}$。 Frobenius 范数。 令 $C=U^{\mathsf T}BV$，则 $\operatorname{rank}(C)\le k$，且 $\|A-B\|_F=\|\Sigma-C\|_F$。展开

$$\|\Sigma-C\|_F^2=\sum_{i}(\sigma_i-C_{ii})^2+\sum_{i\ne j}C_{ij}^2.$$

最优 C 为对角，且保留 k 个最大的 σ_i，其余置零。此时 $\|\Sigma-C\|_F^2=\sum_{i=k+1}^{r}\sigma_i^2$，对应 $B=A_k$ 取到等号。□

推导 8：幂迭代收敛性

为什么要证： 幂迭代看起来只是“反复乘矩阵”，但它为什么能收敛、收敛多快，需要证明。

定理 4.3.13（幂迭代） 设 $A\in\mathbb{C}^{n\times n}$ 可对角化，特征值满足 $|\lambda_1|>|\lambda_2|\ge\cdots\ge|\lambda_n|$，对应特征向量 $v_1,\dots,v_n$ 线性无关。设初值 $x^{(0)}=\sum_{i=1}^{n}c_i v_i$ 满足 $c_1\ne 0$。则幂迭代 $x^{(t)}=A^t x^{(0)}$ 满足

$$\frac{x^{(t)}}{\lambda_1^t}\to c_1 v_1\quad(t\to\infty),$$

归一化后 $x^{(t)}/\|x^{(t)}\|\to\pm v_1/\|v_1\|$，Rayleigh 商 $\rho^{(t)}\to\lambda_1$，误差以 $|\lambda_2/\lambda_1|^t$ 速率衰减。

证明： 因 $Av_i=\lambda_i v_i$，

$$x^{(t)}=\sum_{i=1}^{n}c_i\lambda_i^t v_i=\lambda_1^t\left(c_1v_1+\sum_{i=2}^{n}c_i\left(\frac{\lambda_i}{\lambda_1}\right)^t v_i\right).$$

因 $|\lambda_i/\lambda_1|<1$（$i\geq 2$），当 $t\to\infty$ 时 $(\lambda_i/\lambda_1)^t\to 0$，故括号内趋于 c_1v_1。归一化后方向趋于 v_1。Rayleigh 商 $\rho^{(t)}=\frac{\sum_i c_i^2\lambda_i^{2t+1}}{\sum_i c_i^2\lambda_i^{2t}}\to\lambda_1$。误差项主导为 $(|\lambda_2/\lambda_1|)^t$。□

限制： 需要 $|\lambda_1|>|\lambda_2|$；需要 $c_1\neq 0$；收敛速率由 $|\lambda_2/\lambda_1|$ 控制，接近 1 时很慢。

4.3.3 实验设计（概述）

完整的实验设计、代码和验证值见 A1组-实验.ipynb。实验共 6 个，覆盖以下结论：

1. 条件数与右端扰动（验证推导 5）：$A=\mathrm{diag}(1,10^{-10})$，右端小扰动导致解大误差而残差仍小。
2. 非对称矩阵特征值不等于奇异值（验证推导 4）：$A_1=\begin{bmatrix}0&1\\0&0\end{bmatrix}$ 与 $A_2=\begin{bmatrix}2&1\\0&3\end{bmatrix}$ 对比。
3. 奇异值谱与截断 SVD 误差（验证推导 7）：$M\in\mathbb{R}^{3\times 2}$ 与 $D=\mathrm{diag}(3,1)$ 在不同秩下的谱误差和 F 误差。
4. PCA 与中心化数据的 SVD：4 个二维样本，中心化后第一主成分与重构误差。
5. Weyl 不等式数值验证（验证推导 6）：$A=\begin{bmatrix}2&1\\0&3\end{bmatrix}$，$E=\varepsilon I$。
6. 非正规矩阵特征值的敏感性（验证推导 6 与例 4.3.3）：$A=\begin{bmatrix}1&1/\varepsilon\\0&1\end{bmatrix}$，$E=\begin{bmatrix}0&0\\\varepsilon&0\end{bmatrix}$。

4.3.4 结果解释与本节结论

实验的具体结果、解析验证值和实测输出对照见实验文件的“4.3.4 结果解释”与“实验运行记录”。这里只给结论层面的解释。

小残差不等于小解误差

实验 1 中，右端相对扰动约 10^{-10}，解相对误差约 0.7071，而对原右端的相对残差只有约 10^{-10}。条件数 10^{10} 给出的上界约 1，实际 $0.7071\leq 1$。图 4.3-5 已经说明几何原因：A 把单位圆压成很扁的椭圆，右端在扁方向上的小扰动被 A^{-1} 放大。小残差只说明新解对原右端几乎满足方程，不说明它接近原解。

特征值不等于奇异值

实验 2 中，$A_1=\begin{bmatrix}0&1\\0&0\end{bmatrix}$ 的特征值为 $0,0$，奇异值为 $1,0$，秩为 1。$A_2=\begin{bmatrix}2&1\\0&3\end{bmatrix}$ 的特征值为 $2,3$，奇异值约 $3.2566,1.8424$。非对称矩阵的特征值绝对值不能普遍当奇异值用（图 4.3-1）。

截断 SVD 误差等于剩余奇异值

实验 3 中，M 的 $k=1$ 截断谱误差与 F 误差都等于 $\sigma_2=0.51430$；D 的 $k=1$ 截断谱误差与 F 误差都等于 1。与 Eckart – Young 公式一致（图 4.3-4）。谱衰减越快，低秩压缩潜力越大。

PCA 是中心化数据的 SVD

实验 4 中，中心化矩阵 X_c 的奇异值为 $3,1$，第一主成分为 $(1,1)/\sqrt{2}$，$k=1$ 重构相对 F 误差为 0.31623。图 4.3-6 对比了中心化前后：中心化后第一主成分沿数据变化最大的方向；不中心化时第一主成分被均值方向主导。

图 4.3-6：PCA 必须中心化的几何直观

```
\begin{tikzpicture}[>=Stealth]
  % 左：未中心化
  \begin{scope}
    \draw[highlightbox] (-1,-1) rectangle (7,7);
    \draw[grid] (0,0) grid[step=1] (6.5,6.5);
    \draw[axis] (0,0) -- (7,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,0) -- (0,7) node[above,lbl]{\(x_2\)};

    % 数据点（扩充至12个，呈现真实点云感）
    \foreach \x/\y in {
      1.2/2.1, 1.8/1.5, 2.0/3.2, 2.5/2.8, 3.0/4.1, 3.2/3.5,
      3.5/5.2, 4.0/4.5, 4.2/5.8, 4.8/5.5, 5.0/6.2, 5.5/6.5
    } {
      \fill[myblue!80, opacity=0.85] (\x,\y) circle (3.5pt);
    }

    % 真实变化方向（未中心化时被均值拖拽偏离）
    \draw[thick, mygray!80, dashed] (0,0) -- (5.5,5.5) node[above right, lbl]{第一主成分};
    % 均值向量
    \draw[->, ultra thick, myred] (0,0) -- (3.5,4.3) node[midway, above left, lbl]{均值方向};

    \node[sublbl, align=center, below] at (3.25,-1.0) {未中心化：原点拖拽了主成分方向\\偏离数据真实分布};
  \end{scope}

  % 右：中心化
  \begin{scope}[xshift=9cm]
    \draw[highlightbox] (-4,-4) rectangle (4,4);
    \draw[grid] (-3.5,-3.5) grid[step=1] (3.5,3.5);
    \draw[axis] (-4,0) -- (4,0) node[right,lbl]{\(x_1\)};
    \draw[axis] (0,-4) -- (0,4) node[above,lbl]{\(x_2\)};

    % 数据点（平移到原点周围）
    \foreach \x/\y in {
      -2.3/-2.2, -1.7/-2.8, -1.5/-1.1, -1.0/-1.5, -0.5/-0.2, -0.3/-0.8,
      0.0/0.9, 0.5/0.2, 0.7/1.5, 1.3/1.2, 1.5/1.9, 2.0/2.2
    } {
      \fill[myblue!80, opacity=0.85] (\x,\y) circle (3.5pt);
    }

    % 方差椭圆（置信椭圆）
    \begin{scope}[rotate=45]
      \draw[thick, myblue!60, fill=mylightblue, fill opacity=0.4] (0,0) ellipse (3.2 and 0.8);
    \end{scope}

    % 第一主成分（完美切合点云）
    \draw[very thick, myblue, drop shadow={opacity=0.2}] (-3,-3) -- (3,3) node[above right, lbl]{第一主成分};
    % 第二主成分
    \draw[thick, mygray, dashed, rotate=45] (-1.5,-1.5) -- (1.5,1.5) node[above left, lbl]{第二主成分};

    % 投影虚线（展示降维）
    \foreach \x/\y in {
      -2.3/-2.2, -1.7/-2.8, -1.5/-1.1, -1.0/-1.5, -0.5/-0.2, -0.3/-0.8,
      0.0/0.9, 0.5/0.2, 0.7/1.5, 1.3/1.2, 1.5/1.9, 2.0/2.2
    } {
      \pgfmathsetmacro{\proj}{\x} % 因为主成分在 y=x 上，投影点就是坐标互换/平均
      \draw[very thin, mygray!50, dashed] (\x,\y) -- (\y,\x);
    }

    \node[sublbl, align=center, below] at (0,-4.5) {中心化后：主成分完美捕捉数据\\的真实变化方向（方差最大）};
```

```
  \end{scope}
\end{tikzpicture}
```

图注：左：未中心化时，第一主成分（灰虚线）受到原点（$0,0$）的强烈拖拽，完全偏离数据的长轴；右：中心化后，数据平移到原点周围，第一主成分（蓝实线）完美贴合点云的长轴（方差最大方向），且与第二主成分（灰虚线）正交。

Weyl 不等式是上界

实验 5 中，$|\sigma_i(A+\varepsilon I)-\sigma_i(A)|\le\varepsilon$ 对所有测试的 ε 成立，且实际差值通常远小于 ε。Weyl 是上界，不保证取等号。

非正规矩阵特征值无 Lipschitz 界

实验 6 中，$\varepsilon=10^{-4}$ 时 $\|E\|_2=10^{-4}$，特征值移动 1，比值 10^4。例 4.3.3 已经给出这个现象的复平面直观：特征值会“跳”，奇异值不会。

本节结论

1. 特征值描述方阵“不转向方向”，SVD 描述任意矩阵“正交拉伸方向”。非对称矩阵特征值绝对值不能普遍当奇异值；非正规矩阵特征值对扰动无 Lipschitz 界，而奇异值有 Weyl 界。
2. SVD 对任意矩阵存在，且 $\sigma_i=\sqrt{\lambda_i(A^{\mathsf{T}}A)}$，$\|A\|_2=\sigma_{\max}$，$\|A\|_F=\sqrt{\sum\sigma_i^2}$。
3. 条件数解释敏感性：$\kappa_2=\sigma_{\max}/\sigma_{\min}$ 给出右端扰动放大上界；小残差不等于小解误差。
4. 截断 SVD 有可证明的最优性：Eckart – Young 定理给出谱范数误差 σ_{k+1}、F 范数误差 $\sqrt{\sum_{i>k}\sigma_i^2}$。
5. 低秩重构不等于预测。PCA 必须中心化，LoRA 不是对预训练权重做截断 SVD。

4.3.5 易错点与讨论

风险一：把非对称矩阵的特征值绝对值普遍当作奇异值

反例：$A=\begin{bmatrix}0&1\\0&0\end{bmatrix}$ 特征值 $0,0$，奇异值 $1,0$。正确说法是 $\sigma_i(A)=\sqrt{\lambda_i(A^{\mathsf{T}}A)}$；只有正规矩阵才有 $|\lambda_i(A)|=\sigma_i(A)$。

风险二：把低秩重构等同于预测

Eckart – Young 只说明 A_k 在谱范数/F 范数下最优重构 A。若 A 是评分矩阵，重构误差小不保证推荐命中率高。低秩是压缩/去噪工具，不是预测效果的证明。

风险三：PCA 未中心化会改变解释

图 4.3-6 已给出直观：未中心化时第一主成分被均值方向主导。正确流程：先减列均值，再对中心化矩阵做 SVD。

风险四：残差小不等于解误差小

图 4.3-5 已给出几何原因：扁椭圆方向上右端小扰动被放大。讨论残差时必须同时说明针对哪个右端、用什么范数、矩阵条件数多大。

风险五：条件数依赖范数

$\kappa_2(A)=\sigma_{\max}/\sigma_{\min}$ 是二范数条件数；换用其他范数数值会变。

风险六：把 LoRA 当作“对预训练权重做截断 SVD”

LoRA 通常把权重更新写成 $W=W_0+BA$，只训练低秩增量 BA，参数量约 $r(m+n)$。它不是对已训练好的 W_0 做 SVD 截断。

风险七：把幂迭代当成“总能收敛”

幂迭代要求主特征值严格占优且初值在主特征方向有分量。若 $|\lambda_1|=|\lambda_2|$ 会振荡；若初值恰好正交于 v_1 会失败。

随堂提问与参考回答

提问 1： 取 $A=\begin{bmatrix}0&1\\0&0\end{bmatrix}$，特征值和奇异值分别是什么？为什么不能把特征值绝对值直接当奇异值？ 参考回答： 特征值 $0,0$；$A^{\mathsf{T}}A=\mathrm{diag}(0,1)$，特征值 $0,1$，奇异值 $1,0$。该矩阵把 e_2 映到 e_1，再把 e_1 映到零，存在一次非零拉伸，秩为 1。非对称时特征值与奇异值不等同（图 4.3-1）。

提问 2： 对 $A=\mathrm{diag}(1,10^{-10})$，$b=(1,10^{-10})^{\mathsf{T}}$，只把第二个右端分量增加 10^{-10}。预测解的第二分量变化多少？残差会不会也很小？ 参考回答： 解的第二分量增加 1，相对解误差 $1/\sqrt{2}\approx0.7071$；对原右端的相对残差约 10^{-10}。条件数 10^{10} 把右端相对扰动放大，上界约 1（图 4.3-5）。

提问 3： 对 $M=\begin{bmatrix}1&2\\3&4\\5&6\end{bmatrix}$，取 $k=1$ 做截断 SVD。谱范数误差和 F 范数误差各是多少？如果 M 是评分矩阵，能否说“重构误差小，所以推荐一定好”？ 参考回答： $\|M-M_1\|_2=\sigma_2\approx0.5143$，$\|M-M_1\|_F=\sigma_2\approx0.5143$（图 4.3-4）。Eckart－Young 只保证重构范数最优；推荐效果还要看排序指标等。

提问 4： LoRA 的低秩增量是不是“对预训练权重直接做截断 SVD”？ 参考回答： 不是。LoRA 只训练低秩增量 BA，参数量约 $r(m+n)$；不是对已训练好的 W_0 做 SVD 截断。

提问 5： 对 $A=\begin{bmatrix}1&1/\varepsilon\\0&1\end{bmatrix}$，$E=\begin{bmatrix}0&0\\\varepsilon&0\end{bmatrix}$，特征值移动多少？奇异值移动多少？ 参考回答： 特征值从 $\{1,1\}$ 移动到 $\{0,2\}$，移动 $1=O(1)$，与 $\|E\|_2=\varepsilon$ 无关。奇异值满足 Weyl 不等式。

提问 6： PCA 为什么必须中心化？不中心化会怎样？ 参考回答： 不中心化时第一主成分被均值方向主导（图 4.3-6）。中心化后最多 $n-1$ 个非零奇异值，这是 PCA 的固有性质。

4.4 AI关联：可微求解器、PCA、LoRA（2学时）

本节目标： 能说明矩阵方法与学习模型的连接及其边界。

一个“会倒着算”的求解器、一个“给高维数据减肥”的方法、一个“给大模型打补丁”的技巧——它们之间有什么关系？答案藏在一个词里：结构。可微求解器靠的是求解过程的代数结构（LU 分解），PCA 靠的是数据的谱结构（SVD），LoRA 靠的是权重的低秩结构。三个故事殊途同归：结构在哪里，就在哪里省力。下面把它们一个个拆开。

概念讲解：把问题说明白

先修概念： 矩阵求逆、雅可比矩阵、链式法则、SVD、协方差矩阵、特征值分解、矩阵秩、低秩近似。

先给三个主角各配一张“名片”：可微求解器 ——“会倒着算的求解器”，不满足于算出 $\boldsymbol{x}$，还要知道“如果 $\boldsymbol{A}$ 或 $\boldsymbol{b}$ 动了一点，$\boldsymbol{x}$ 会怎么动”，这个“怎么动”正是梯度，是反向传播最想要的东西。PCA ——“给数据瘦身的放大镜”，拥有成百上千个特征的数据像一个“臃肿的房间”，PCA 找到房间里真正决定形状的那几根“承重方向”（真正决定方差的方向），把高维数据压成低维而尽量不失真。LoRA ——“给大模型打的小补丁”，不动庞大的预训练权重，只在旁边加一块小巧的低秩增量，用极少的参数撬动大模型的适配。

线性求解器处理的是 $\boldsymbol{Ax}=\boldsymbol{b}$，其中 $\boldsymbol{A}\in\mathbb{R}^{n\times n}$ 可逆，$\boldsymbol{x}$ 是待求解向量。所谓“可微求解器”，指把 $\boldsymbol{x}=\boldsymbol{A}^{-1}\boldsymbol{b}$ 视为关于 $\boldsymbol{A}$ 和 $\boldsymbol{b}$ 的可微函数，从而在反向传播中计算损失对 $\boldsymbol{A}$、$\boldsymbol{b}$（或产生它们的参数）的梯度。适用于$\boldsymbol{A}$ 在每次前向中确实被求解（或至少被分解），且反向时能够访问求解器内部的分解信息，$\boldsymbol{A}$ 需可逆。

PCA 处理的是中心化数据矩阵 $\boldsymbol{X_c}\in\mathbb{R}^{n\times d}$（$n$ 个样本，d 个特征，每列均值为零）。对 $\boldsymbol{X_c}$ 做 SVD：$\boldsymbol{X_c}=\boldsymbol{U\Sigma V}^{\top}$。右奇异向量 $\boldsymbol{V}$ 的列就是主成分方向，奇异值平方除以 $n-1$ 给出各主成分的方差。适用于数据已中心化；若未中心化，SVD 的右奇异向量不再等于协方差矩阵的特征向量。

LoRA 处理的是微调中的权重增量 $\boldsymbol{\Delta W}$。对一个预训练权重 $\boldsymbol{W_0}\in\mathbb{R}^{d\times k}$，LoRA 不直接学习 $\boldsymbol{\Delta W}$，而是令 $\boldsymbol{\Delta W}=\boldsymbol{BA}$，其中 $\boldsymbol{B}\in\mathbb{R}^{d\times r}$，$\boldsymbol{A}\in\mathbb{R}^{r\times k}$，秩 $r\ll\min(d,k)$。可训练参数量从 dk 降为 $r(d+k)$。适用于微调任务的内在维度远低于全参数空间，因此低秩增量足以表达任务适配。

假设有一个 1000×1000 的权重矩阵需要微调，全参数是 10^6 个。取 $r=8$，LoRA 只训练 $8\times(1000+1000)=16000$ 个参数，压缩约 62 倍。打个比方：你不必把整栋楼重新装修，只需给关键承重墙贴一块薄而精的加固板。这并不保证效果无损，但给出了一个可调的容量—成本权衡旋钮。

三个对象的连接在于：可微求解器利用矩阵分解的代数结构来避免显式求逆；PCA 利用 SVD 的谱结构来压缩数据；LoRA 利用低秩结构来压缩可训练参数。 共同前提是：结构确实存在，且被正确利用。

关键推导：让结论有依据

推导一：由 $\boldsymbol{Ax}=\boldsymbol{b}$ 出发的微扰关系（$\boldsymbol{A}$ 可逆）。

设有线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$，其中 $\boldsymbol{A}$ 可逆，$\boldsymbol{x}$ 是线性方程组的准确解。

设 $\boldsymbol{A}$ 是精确的，$\boldsymbol{b}$ 有误差 $\boldsymbol{\delta b}$，解为 $\boldsymbol{x}+\boldsymbol{\delta x}$，则

$$\boldsymbol{A}(\boldsymbol{x}+\boldsymbol{\delta x})=\boldsymbol{b}+\boldsymbol{\delta b}.$$

减去 $\boldsymbol{Ax}=\boldsymbol{b}$，得 $\boldsymbol{A\delta x}=\boldsymbol{\delta b}$，因此

$$\boldsymbol{\delta x}=\boldsymbol{A}^{-1}\boldsymbol{\delta b},\quad \|\boldsymbol{\delta x}\|\leqslant\|\boldsymbol{A}^{-1}\|\|\boldsymbol{\delta b}\|.$$

由线性方程组有

$$\|\boldsymbol{b}\|\leqslant\|\boldsymbol{A}\|\|\boldsymbol{x}\|,\quad \frac{1}{\|\boldsymbol{x}\|}\leqslant\frac{\|\boldsymbol{A}\|}{\|\boldsymbol{b}\|}\quad(\text{设 }\boldsymbol{b}\neq0).$$

则

$$\frac{\|\boldsymbol{\delta x}\|}{\|\boldsymbol{x}\|}\leqslant\|\boldsymbol{A}^{-1}\|\|\boldsymbol{A}\|\frac{\|\boldsymbol{\delta b}\|}{\|\boldsymbol{b}\|}.$$

上式给出了解的相对误差的上界，常数项 $\boldsymbol{b}$ 的相对误差在解中可能放大 $\|A^{-1}\|\|A\|$ 倍。

现设 $\boldsymbol{b}$ 是精确的，$\boldsymbol{A}$ 有微小误差(扰动)$\delta \boldsymbol{A}$，解为 $\boldsymbol{x}+\delta \boldsymbol{x}$，则

$$\begin{cases}(A+\delta A)(\boldsymbol{x}+\delta \boldsymbol{x})=\boldsymbol{b}, \\ (A+\delta A)\delta \boldsymbol{x}=-(\delta A)\boldsymbol{x}.\end{cases}$$

如果 $\delta \boldsymbol{A}$ 不受限制的话，$\boldsymbol{A}+\delta \boldsymbol{A}$ 可能奇异，而

$$(A+\delta A)=A(I+A^{-1}\delta A),$$

当 $\|A^{-1}\delta A\|<1$ 时，$(I+A^{-1}\delta A)^{-1}$ 存在，有

$$\delta \boldsymbol{x}=-(I+A^{-1}\delta A)^{-1}A^{-1}(\delta A)\boldsymbol{x},$$

因此

$$\|\delta \boldsymbol{x}\| \leqslant \frac{\|A^{-1}\|\|\delta A\|\|\boldsymbol{x}\|}{1-\|A^{-1}(\delta A)\|}.$$

设 $\|A^{-1}\|\|\delta A\|<1$，即得

$$\frac{\|\delta \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leqslant \frac{\|A^{-1}\|\|A\|\frac{\|\delta A\|}{\|A\|}}{1-\|A^{-1}\|\|A\|\frac{\|\delta A\|}{\|A\|}}.$$

若 $\boldsymbol{A}$ 接近奇异，$\boldsymbol{A^{-1}}$ 的范数很大，微扰关系给出的线性化不再可靠；若 $\boldsymbol{A}$ 奇异，推导本身失效。此外，若前向求解只用了迭代法且未收敛到机器精度，反向的伴随求解可能引入累积误差，此时“可微”在数学上成立但在数值上不可靠。

推导二：中心化数据的 SVD 与主成分对应。

设 $\boldsymbol{X_c} \in \mathbb{R}^{n\times d}$ 已中心化（列均值为零）。样本协方差矩阵（忽略 $1/(n-1)$ 因子）为

$$C=\frac{1}{n-1}X_c^{\top}X_c.$$

对 $\boldsymbol{X_c}$ 做 SVD：$\boldsymbol{X_c}=U\Sigma V^{\top}$，其中 $U^{\top}U=I$，$V^{\top}V=I$，Σ 对角。代入：

$$X_c^{\top}X_c=V\Sigma^{\top}U^{\top}U\Sigma V^{\top}=V\Sigma^2V^{\top}.$$

因此

$$C=V\left(\frac{\Sigma^2}{n-1}\right)V^{\top}.$$

这意味着 $\boldsymbol{V}$ 的列是 $\boldsymbol{C}$ 的特征向量，即主成分方向；第 $\boldsymbol{i}$ 个奇异值的平方除以 $\boldsymbol{n-1}$ 是第 $\boldsymbol{i}$ 个主成分的方差。若只保留前 $\boldsymbol{k}$ 个主成分，投影为 $\boldsymbol{Z=X_cV_k}$，重构为 $\hat{\boldsymbol{X}}_c=ZV_k^{\top}$。

若 $\boldsymbol{X}$ 未中心化，则 $\boldsymbol{X^{\top}X}$ 不等于协方差矩阵（相差一个与均值有关的秩一修正），SVD 的右奇异向量一般不再对应主成分方向。此外，若数据矩阵的列数远大于行数，显式构造 $\boldsymbol{d\times d}$ 的协方差矩阵可能不可行，此时直接对 $\boldsymbol{X_c}$ 做 SVD 或使用截断 SVD 更合适。

实验设计：先预测，再运行

要填写的项目	本组内容
要回答的问题与运行前预测	问题1：可微求解器对 b 的梯度是否与有限差分一致？预测：一致，但步长过小会因舍入误差偏离，步长过大会因非线性项偏离。问题2：PCA 重构误差随主成分数增加是否单调下降？预测：是，但未中心化时下降曲线不同，且首主成分方向偏移。问题3：LoRA 在 $d=64, k=64, r=4$ 时的可训练参数量是多少？预测：$4\times(64+64)=512$。
输入、参数、单位、数据来源与划分	实验1：随机生成可逆 $A\in\mathbb{R}^{5\times5}$（条件数控制在 10^2 量级），固定种子；b 随机生成；有限差分步长取 $10^{-2}, 10^{-4}, 10^{-6}$。实验2：用 sklearn 的 iris 数据集或随机生成 $n=100, d=5$ 的带相关结构数据；分别做中心化与未中心化两组，计算 $k=1,\dots,4$ 的重构误差（Frobenius 范数平方）。实验3：纯维度计算，无需数据。
独立参考解/基线及选择理由	实验1的参考解：解析梯度 $\partial x/\partial b = A^{-1}$（对单位扰动逐列求解）。选择理由：这是精确的线性代数结果，不依赖自动微分实现。实验2的基线：用协方差矩阵特征分解（中心化后）与 SVD 结果比较主成分方向的内积绝对值，接近 1 说明一致。实验3的基线：全参数微调参数量 $d\times k$。
误差指标、容差及其依据	实验1：相对误差 $\|\delta x_{\mathrm{AD}}-\delta x_{\mathrm{FD}}\|/\|\delta x_{\mathrm{FD}}\|$，容差 10^{-5}（float64 下中心差分期望精度约 $h^2+\epsilon/h$，取 $h\approx10^{-4}$ 时理论误差量级约 10^{-8}，留两个数量级余量）。实验2：重构相对误差，不设硬容差，观察趋势；PCA 方向一致性用 $\lvert v_i^{\top}\hat{v}_i\rvert$，中心化时应 >0.999。实验3：精确等式，无容差。
环境、种子、设备与运行预算	Python 3.10+，NumPy/SciPy，PyTorch（实验1可选）。种子固定为 42。CPU 即可，运行预算：实验1 < 1 秒，实验2 < 2 秒，实验3 手算或一行代码。

结果解释与本节结论

实验1

- 结果：在 h 从 10^{-2} 缩小到 10^{-7} 的过程中，有限差分雅可比与解析雅可比的方向余弦恒为 1.000000；相对误差从 2.36×10^{-14} 单调上升至 2.26×10^{-9}。
- 对比分析：与预测完全一致。线性映射下中心差分的截断误差恒为零，观测到的误差完全由舍入主导，因此 h 越小误差越大。最大相对误差 2.26×10^{-9} 远小于容差 10^{-5}，验证通过。

实验2

- 结果：
  - 中心化重构误差：$k=1$ 至 4 分别为 $0.4466, 0.1479, 0.0567, 0.0000$。
  - 未中心化重构误差：$k=1$ 至 4 分别为 $0.2570, 0.1051, 0.0225, 0.0000$。
  - 中心化 SVD 与协方差特征向量各列 $\lvert\langle\cdot,\cdot\rangle\rvert$ 均为 1.0。
  - 未中心化时，第一主成分与均值方向 $\lvert\cos\rvert=0.9997$。

- 对比分析：重构误差随主成分数增加单调下降，符合预测。中心化时方向一致性完美符合理论。未中心化误差数值虽小，但因其第一主成分几乎完全等于均值方向（0.9997），说明它把均值吸收进了低秩近似，而非纯粹提取方差结构，证实了“未中心化不可用于 PCA”的边界。

实验3

- 结果：
  - $(d=64, k=64, r=4)$：全参数 4096，LoRA 512（比例 $8.00\times$），与预测完全一致。
  - $(d=512, k=512, r=16)$：全参数 262144，LoRA 16384（比例 $16.00\times$）。
- 对比分析：精确等式成立，证明低秩增量在参数效率上的代数基础。

尚未验证的情况

- 由于环境缺失 PyTorch，实验1的自动微分（autograd）梯度尚未验证，可微求解器的实际反向传播精度需后续补齐。
- 实验2未使用 sklearn 的 iris 数据集，而是使用了随机生成的带相关结构数据，后续可在真实数据上复现。
- LoRA 仅验证了维度与参数量，其学习层面的“低秩增量能否有效逼近全参数微调效果”未涉及，需后续训练实验补足。

易错点与讨论

风险：LoRA 不等于对预训练权重直接做截断 SVD。

一个常见的误解是LoRA 的 BA 就是对 W_0 做 SVD 后取前 r 个奇异值重构的结果。LoRA 学习的是增量 $\Delta W = BA$，它叠加在冻结的 W_0 上：$W' = W_0 + \frac{\alpha}{r}BA$。$BA$ 的低秩性是对更新量的约束，不是对原始权重的压缩。用反例说明：若 W_0 本身满秩且任务需要改变所有奇异方向上的成分，LoRA 的低秩增量无法精确表示一个满秩的 ΔW。但“无法精确表示”不等于“无法有效微调”，因为微调任务的内在维度通常远低于参数维度。另一个区别是：截断 SVD 的误差由奇异值衰减速度控制，而 LoRA 的 BA 由梯度下降优化得到，没有闭式最优性保证。

风险：PCA 未中心化会改变解释。

推导已经表明中心化是 SVD 右奇异向量等于协方差特征向量的前提。若不中心化，第一主成分往往会被均值方向“吸走”，反映的是数据的绝对位置而非相对变异。用一个小反例回应：取三个一维样本 $\{1, 2, 6\}$。中心化后为 $\{-2, -1, 3\}$，PCA 方向（一维情况下就是符号）反映的是围绕均值的离散。若不中心化直接做 SVD，第一奇异向量会被整体均值（3）主导。在二维情形中，未中心化的“第一主成分”可能几乎平行于均值向量，而不是最大方差方向。因此，任何声称“对数据矩阵直接 SVD 就是 PCA”的表述，都必须补充中心化条件。

随堂提问： 设有一个 $d=512$、$k=512$ 的权重矩阵，用 LoRA 取 $r=16$。同学 A 说：“LoRA 的参数量是 512×512 的 16/512，也就是 8192 个。”同学 B 说：“不对，是 $2\times512\times16=16384$ 个。”谁对？依据是什么？

同学可能的回答： A 可能把秩的比例直接乘到全参数上；B 使用了 $r(d+k)$ 的公式。需要追问：d 和 k 是否相等？LoRA 是否对每一层都加？是否包含偏置？

本组最终解释： B 的数值正确（在 $d = k = 512$、单层、无偏置、只分解一个权重矩阵的条件下）。LoRA 的可训练参数来自 $B \in \mathbb{R}^{d\times r}$ 和 $A \in \mathbb{R}^{r\times k}$，合计 $dr + rk = r(d + k)$。这里 $r(d + k) = 16 \times 1024 = 16384$。A 的错误在于把“秩的比例”直接当作参数压缩比例：全参数 $dk = 262144$，压缩比为 $dk/[r(d + k)] = 512^2/(16 \times 1024) = 16$，不是 $512/16 = 32$。维度依据是矩阵乘法的形状匹配：B 的列数必须等于 A 的行数，即 r。

回到开场：用证据回答

本章核心问题： Ax=b如何高效求解？矩阵的本质结构是什么？

本章开场问：残差几乎为零，解会不会跳很远？4.3 节的回答是：会，只要矩阵条件数大。SVD 把矩阵拆成一组正交拉伸方向，最大与最小奇异值之比给出条件数；右端在“被压扁方向”上的小扰动，会在解中被放大（图 4.3-5）。具体到 $A = \mathrm{diag}(1, 10^{-10})$：相对右端扰动约 10^{-10}，相对解误差约 0.7071，对原右端的相对残差约 10^{-10}。小残差和小解误差是两个不同主张。

连接到本章核心问题“$Ax = b$ 如何高效求解？矩阵的本质结构是什么？”：

- 求解： LU/Cholesky（4.1）给直接法，CG（4.2）给迭代法；它们都受条件数影响。SVD 不直接用来解 $Ax = b$（太贵），但给出诊断语言；伪逆 A^+ 给出最小二乘或秩亏系统的规范解。
- 结构： SVD 把矩阵拆成“旋转—拉伸—旋转”（图 4.3-2），奇异值谱是矩阵的本质结构；截断 SVD 给出低秩近似的最优误差（图 4.3-4）。
- 边界： 大规模问题通常不显式做完整 SVD，而用随机 SVD、Lanczos/Arnoldi 等方法。

比喻的局限： 漫画用“体检”“放大镜”比喻矩阵解剖，直观但不可照搬。其一，“残差几乎为零”的体检报告只说明计算解精确满足一个与原问题接近的方程组（后向误差小），并不保证前向误差小；比喻没有体现条件数这个放大因子，容易让人误读为“指标正常即健康”。其二，图中的对角矩阵是最简单的病态例子，不能据此推断所有小残差都危险，也不能推断选主元或迭代法能消除病态性——选主元改善的是消元过程的稳定性，CG 的收敛速度才受条件数控制。4.4 节把 LoRA 比作“给大模型打小补丁”同样需加注：LoRA 学习的是叠加在冻结权重上的加性低秩增量 BA，而非对预训练权重 W_0 做截断 SVD；补丁是“增量”，不是“压缩版原权重”，这一代数区别不能由比喻替代。

本章小结与课后习题

LU 方法解决的问题是：对同一个 A 反复求解 $Ax = b$ 时，避免每次重新消元。适用条件是 A 可逆且需要多次求解或需要求行列式；最容易混淆的是把 LU 分解本身当成“求解”，实际上分解只是把问题转化为两个三角系统的前代和回代。与可微求解器的联系：前向 LU 分解若被保留，反向伴随求解可以复用 L 和 U，这是可微求解器高效的关键。

CG（共轭梯度法） 解决的问题是在 A 对称正定时，用迭代方式求解大型稀疏系统，不需要显式存储或分解 A。适用条件是 A 对称正定；最容易混淆的是把 CG 用于非对称或不定矩阵，此时可能不收敛。与可微求解器的联系：CG 的迭代过程若被展开并反向传播，内存开销大；更常用的做法是在收敛解处做隐式微分，即用伴随方程代替展开迭代。

SVD 解决的问题是把任意矩阵分解为旋转—拉伸—旋转三个部分，同时给出秩、条件数和最优低秩近似的完整信息。适用条件没有可逆性要求；最容易混淆的是把 SVD 直接当作 PCA 而不做中心化。与 LoRA 的联系：LoRA 的低秩增量在形式上与截断 SVD 的秩约束相似，但 LoRA 的 BA 是学出来的增量，不是对 W_0

的谱截断，这是概念上的关键区分。

其他方法说明：QR 分解与 LU 同属直接法，但正交性更好，适合最小二乘；Cholesky 是 SPD 矩阵的专用 LU 变体，速度约为 LU 的两倍。它们与上述三个概念的关系是：都是“利用矩阵结构降低计算成本”这一原则的不同实现

课后题按四节对应填写到A1组-习题.md，参考解答单独放在**A1组-习题参考解答.md**。

选读边界

GMRES、QR迭代和稀疏存储细节移出必修主线；后向误差仅在残差讨论中给出必要定义，完整分析选读。

选读不计入本章8学时和最低完成要求。若添加附录，只在此处填写已有材料的名称和链接，不为了占位增加空文档。

参考资料、图片来源与AI使用

支撑的主张/公式/图片	作者、标题、版本/年份	页码/章节/链接	本组人工核对
LoRA的低秩参数增量（4.4）	Hu等，LoRA: Low-Rank Adaptation of Large Language Models，2021	原论文	已核对
4.1.1：无主元LU条件、列主元置换及三角求解	李庆扬、王能超、易大义，《数值分析》第5版，清华大学出版社，2008年12月	5.2节、5.3.1节；重点核对书内第146、148、152页	已据定义推导并运行程序核验
4.1.2：Cholesky存在唯一性、平方根法与LDL^T联系	李庆扬、王能超、易大义，《数值分析》第5版，清华大学出版社，2008年12月	5.3.2节，书内第156—158页	已整理证明并核验实例
4.1.3：条件数与误差放大	李庆扬、王能超、易大义，《数值分析》第5版，清华大学出版社，2008年12月	5.5.1节，书内第167—169页	已给出残差到解误差的推导及反例
4.1：主元、LU、范数与病态性的辅助核对	张威、杨月婷，《数值分析（第5版）习题解答》，清华大学出版社，2010年	第5章，书内第69—72页	已核对相关页并自编例题
4.1：求解和Cholesky库函数对照	NumPy官方API及本次安装版本的函数文档字符串	`solve`、`cholesky`	教学程序与库函数已对照运行
4.1：误差表、反例与程序	本稿自构造数据与Python实现，2026-10-20复跑	本文4.1节“实验设计”和“结果解释”栏目	程序已实际运行
4.2迭代法：CG与预条件	李庆扬、王能超、易大义，《数值分析》第5版，清华大学出版社，2008年12月	第六章，第202-208页	已人工核对

支撑的主张/公式/图片	作者、标题、版本/年份	页码/章节/链接	本组人工核对
SVD 存在性与构造性证明	Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins Studies in the Mathematical Sciences. Baltimore, MD: The Johns Hopkins University Press. ISBN 978-1-4214-0794-4; DOI: 10.1137/1.9781421407944		
奇异值与 A^TA 特征值关系	Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins University Press. ISBN 978-1-4214-0794-4; DOI: 10.1137/1.9781421407944. 另参：Trefethen, L. N., & Bau, D., III. (1997). Numerical Linear Algebra. Philadelphia: SIAM. ISBN 978-0-89871-361-9; DOI: 10.1137/1.9780898719574		
条件数与扰动分析	Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins University Press. ISBN 978-1-4214-0794-4; DOI: 10.1137/1.9781421407944. 另参：Trefethen, L. N., & Bau, D., III. (1997). Numerical Linear Algebra. SIAM. ISBN 978-0-89871-361-9; DOI: 10.1137/1.9780898719574		
Weyl 不等式、逆扰动界	Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins University Press. ISBN 978-1-4214-0794-4; DOI: 10.1137/1.9781421407944		
非正规矩阵特征值敏感性	Trefethen, L. N., & Bau, D., III. (1997). Numerical Linear Algebra. Philadelphia: SIAM. ISBN 978-0-89871-361-9; DOI: 10.1137/1.9780898719574		

支撑的主张/公式/图片	作者、标题、版本/年份	页码/章节/链接	本组人工核对
Eckart–Young 截断 SVD 最优性	Eckart, C., & Young, G. (1936). The Approximation of One Matrix by Another of Lower Rank. Psychometrika, 1(3), 211–218. https://doi.org/10.1007/BF02288367		
幂迭代收敛性	Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins University Press. ISBN 978-1-4214-0794-4; DOI: 10.1137/1.9781421407944. 另参：Trefethen, L. N., & Bau, D., III. (1997). Numerical Linear Algebra. SIAM. ISBN 978-0-89871-361-9; DOI: 10.1137/1.9780898719574		
PCA 与中心化	Jolliffe, I. T. (2002). Principal Component Analysis (2nd ed.). Springer Series in Statistics. New York: Springer. ISBN 978-0-387-95442-4; DOI: 10.1007/b98835		
LoRA 低秩参数增量	Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv preprint arXiv:2106.09685. https://doi.org/10.48550/arXiv.2106.09685		
谱定理	Horn, R. A., & Johnson, C. R. (2013). Matrix Analysis (2nd ed.). Cambridge: Cambridge University Press. ISBN 978-0-521-83940-2 (hardback); DOI: 10.1017/CBO9781139020411		
4.4在A可逆的条件下，由Ax=b推导微扰关系	李庆扬、王能超、易大义，《数值分析》第5版，清华大学出版社，2008年12月	5.5节，第167——169页	已人工核对

图片来源说明： 图 4.3-1 至 4.3-6均由本组用 TikZ 和 pgfplots 绘制，属于概念示意图，不作为实测结果。图注已写明坐标、参数和适用条件。

漫画为仓库初始化助手绘制的概念示意，A组需结合最终正文核对并记录修改。AI建议与人工结论分开，本章对话分别记录到A1组-AI对话记录.md、模板-B组-AI对话记录.md、模板-C组-AI对话记录.md；A/B/C按轮换角色填写。

本章习题与参考解答

第04章 矩阵的解剖：课后习题

状态：已完善题面和解答，待B/C核验。

题1：对应4.1 直接法：LU与Cholesky

考虑以下实矩阵与右端向量，完成下列推导、编程和辨析任务。

$$A=\begin{bmatrix}0 & 2 & 1\\ 1 & -1 & 0\\ 2 & 1 & 3\end{bmatrix},\qquad b=\begin{bmatrix}0\\2\\7\end{bmatrix},$$

$$H=\begin{bmatrix}4 & 2 & 0\\ 2 & 5 & 2\\ 0 & 2 & 5\end{bmatrix},\qquad g=\begin{bmatrix}2\\1\\8\end{bmatrix}.$$

本题统一采用$PA=LU$约定：P表示对A的行置换，L为单位下三角矩阵，U为上三角矩阵。列主元策略是在当前列尚未处理的行中选择绝对值最大的元素，再交换相应两行。对于Cholesky，采用$H=CC^{T}$，其中C为对角元严格为正的下三角矩阵。这里使用C，以区别于LU中的L。

多个右端的制造解取为

$$X_{*}=\begin{bmatrix}1 & -1\\ 2 & 0\\ -1 & 2\end{bmatrix}\in\mathbb{R}^{3\times 2},\qquad B_{A}=AX_{*},\qquad B_{H}=HX_{*}.$$

X_{*}仅用于多右端实验的独立核验；单右端手算任务仍需从$Ax=b$、$Hz=g$出发给出完整求解步骤。所有输入均为自构造、无量纲数据，无外部数据集，无需训练/测试划分。

- 推导或概念小问： 完成以下四项，写出计算依据，不能只列最终矩阵或向量。

1. 高斯消元与列主元LU（对应4.1.1）。 先计算$\det A$，判断A是否非奇异，并解释为什么不能直接以原始a_{11}为除数。按列主元规则逐步消元，记录每一步的候选主元、换行和乘子，写出P,L,U并直接相乘核验$PA=LU$。特别说明：第二步换行时，应如何调整此前已经存入L的乘子？随后先解$Ly=Pb$，再回代解$Ux=y$，求出$Ax=b$的解。判断A能否使用本节的实Cholesky分解，并写明理由。
2. 正定性与Cholesky（对应4.1.2）。 核查H的对称性，给出它正定的依据。可以使用顺序主子式，也可以构造正对角因子C并由$w^{T}Hw=\|C^{T}w\|_{2}^{2}$证明。按逐元素公式手算$H=CC^{T}$，列出全部非零因子的计算过程；再依次求解$Cq=g$、$C^{T}z=q$，得到$Hz=g$的解。说明这里的$C$为什么不能当作单位下三角矩阵处理，以及回代分子为什么使用q_{i}而不是原始g_{i}。

3. 多右端与计算量（对应4.1.3）。 写出用已有P, L, U求解$AX = B_A$的两个三角方程组，以及用已有C求解$HZ = B_H$的两个三角方程组。标注每个矩阵的维数，解释为什么系数矩阵不变时可以复用分解。对稠密$n \times n$矩阵、s个右端，按乘法和加减各计一次的口径，比较“每个右端重新分解”和“分解一次、多次三角求解”的主要运算量。说明当A发生变化时，旧因子能否直接当作新矩阵的精确分解。

4. 残差与前向误差。 对非奇异矩阵$K \in \mathbb{R}^{n \times n}$，设$Kx_* = f \neq 0$，近似解为$\widehat{x}$，残差为$r = f - K\widehat{x}$。从$K(x_* - \widehat{x}) = r$出发，逐步推导

$$\frac{\|\widehat{x} - x_*\|_\infty}{\|x_*\|_\infty} \le \kappa_\infty(K) \frac{\|r\|_\infty}{\|f\|_\infty}, \qquad \kappa_\infty(K) = \|K\|_\infty \|K^{-1}\|_\infty.$$

指出该式中所用范数必须相容；当$f = 0$时，解释为什么不能直接使用以$\|f\|_\infty$为分母的相对残差。推导中可以使用K^{-1}，编程求解时不要求显式形成逆矩阵。

- 代码或结果分析小问： 使用Python与NumPy实现并运行以下实验，可参考正文中的教学程序。NumPy的`solve`、`cholesky`只用于独立对照，不能替代本题要求展示的消元、分解和三角求解。全部计算使用`float64`。

1. 实现与多右端复用。 实现列主元LU、下三角前代、上三角回代和Cholesky。LU必须明确保存最终排列向量或置换矩阵的含义，并在后续换行时同步调整L中已经完成的列。分别只分解A、H各一次，计算B_A, B_H，同时求解两列右端，得到$\widehat{X}_A, \widehat{X}_H$。另用原有因子求解单右端$b, g$，与手算结果核对。允许用标准库和NumPy编写教学实现，不得以显式求逆代替三角求解。

2. 分解与解的误差表。 使用原始输入计算

$$\varepsilon_{\mathrm{LU}} = \frac{\|PA - LU\|_\infty}{\|A\|_\infty}, \qquad \varepsilon_{\mathrm{Chol}} = \frac{\|H - CC^T\|_\infty}{\|H\|_\infty}.$$

对于$K = A, H$的每个右端f_j及计算解$\widehat{x}_j$，逐列计算

$$r_j = f_j - K\widehat{x}_j, \qquad \rho_j = \frac{\|r_j\|_\infty}{\|f_j\|_\infty},$$

$$\eta_j = \frac{\|r_j\|_\infty}{\|K\|_\infty \|\widehat{x}_j\|_\infty + \|f_j\|_\infty}, \qquad e_j = \frac{\|\widehat{x}_j - x_{*,j}\|_\infty}{\|x_{*,j}\|_\infty}.$$

再以`np.linalg.solve(K, B_K)`为对照解X_{ref}，逐列报告

$$d_j = \frac{\|\widehat{x}_j - x_{\mathrm{ref},j}\|_\infty}{\|x_{\mathrm{ref},j}\|_\infty}.$$

这里B_K分别为B_A、B_H；$x_{*,j}$为X_*的第j列。使用无穷范数时，向量取最大绝对分量，矩阵取最大绝对行和。提交包含“方法、右端编号、分解误差、ρ_j、η_j、e_j、d_j”的表格，并解释每个指标回答什么问题。本题给定多右端的分母均非零；若自行增加零右端测试，应另行处理相对指标的定义。禁止用已消元后的矩阵和已变换右端冒充原方程的残差。

3. 故障注入：遗漏交换已存乘子。 复制正确的LU实现，仅删除“换行时交换L此前已完成列”的语句，其余代码保持一致。保留正确实现，先预测本题哪一步会暴露问题，再运行错误实现。报告错误因子$\widetilde{L}$及

$$\varepsilon_{\mathrm{bad}} = \frac{\|PA - \widetilde{L}U\|_\infty}{\|A\|_\infty}.$$

对原始右端b计算错误求解结果的残差，并与正确实现比较。用断言证明此处错误确实被检测到，再恢复该语句复测。不能只展示“程序没有报错”，也不能用删除失败输出的方式宣称实现正确。

4. 适用条件测试。 对实对称矩阵

$$S = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$$

先构造一个非零向量v证明$v^T S v < 0$，再尝试本节正对角Cholesky算法。用明确的异常捕获验证算法按预期报告失败，指出出问题的平方根项。解释为什么失败与“对角元都是正数”并不矛盾。不允许通过把非正平方根项取绝对值来让程序继续运行。

5. 小残差与大解误差。 令

$$D = \mathrm{diag}(1, 10^{-12}), \qquad x_*^{(D)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad f_D = D x_*^{(D)}, \quad \widehat{x}^{(D)} = \begin{bmatrix} 1 \\ 3/2 \end{bmatrix}.$$

先解析计算，再用程序核对残差、相对残差、相对前向误差、η及$\kappa_\infty(D)$，检查前述误差上界是否成立。解释：D已经对称正定，为什么仍不能只根据小残差判断解准确？这里$\widehat{x}^{(D)}$是人为给定的候选解，不把它描述成某个求解器的实际输出。

结果提交要求： 保存手算步骤、完整可运行代码、从头运行后的输出、误差表及简要解释；记录Python、NumPy版本、浮点类型和运行设备。所有输入均确定，无随机种子。本题不要求绘图或计时；若自行比较时间，应说明是否包含分解、预热和重复次数，不能根据单次小矩阵耗时下结论。

- 陷阱辨析小问： 逐条判断以下主张是否正确；错误主张须给出本题中的反例或计算证据，正确主张须补齐成立条件。以下语句均为教学构造，不是任何AI系统的真实原话。

1. “只要A非奇异，就一定可以按原始行顺序完成所有无主元消元步骤。”
2. “列主元消元实际交换的是列，因此只需保存列置换，不用交换右端。”
3. “对称矩阵只要对角元都是正数，就可以无条件做Cholesky。”
4. “当系数矩阵不变、只改变右端时，可以复用原来的LU或Cholesky因子。”
5. “只要相对残差小于10^{-10}，相对解误差就一定也小于10^{-10}。”

- 验收依据： 数学条件、实际程序输出和解释三者应一致，具体按下表核验。以下阈值只用于本题的确定性小矩阵，不作为所有矩阵的统一误差保证。

核验项目	应满足的条件或提交证据
列主元LU手算	解释零首主元与非奇异性的区别；逐步记录本题发生的两次换行；给出符合$PA = LU$的因子和右端置换；正确调整已存乘子
Cholesky手算	给出对称正定的明确依据；C下三角且对角元严格为正；相乘满足$H = CC^T$；前代与转置回代的方向、右端及下标正确
单右端求解	手算解代入原始$Ax = b$与$Hz = g$成立；程序结果与手算一致，报告实际残差
多右端与复杂度	$B_A, B_H, \widehat{X}_A, \widehat{X}_H$均为$3 \times 2$；对同一系数矩阵只分解一次；写明三角求解、分解复用与计数口径
误差界推导	写明K非奇异、$f \neq 0$及相容范数；从残差关系逐步得到条件数放大界，不能把残差直接等同于前向误差

核验项目	应满足的条件或提交证据
正确实现的数值验收	令$\mathrm{eps}=\mathrm{np.finfo(np.float64).eps}$、$n=3$，取$\tau=500n\,\mathrm{eps}\approx 3.331\times 10^{-13}$。两个主实验的分解误差及每一列的$\rho_j,\eta_j,e_j,d_j$均应小于$\tau$；先报告原始数值，再作是否通过的判断
漏换乘子的错误检测	错误实现应给出明显不满足分解关系的因子；本题应检测到$\varepsilon_{\mathrm{bad}}>10^{-2}$，且用原始$b$计算的绝对残差大于$10^{-2}$；恢复正确语句后重新通过主实验
非正定输入	手算给出$v^TSv<0$，程序明确捕获预期的Cholesky失败；不返回或验收虚构的实正对角因子
病态反例	解析值与程序结果在舍入误差内一致；应观察到相对残差小于10^{-10}，相对前向误差大于10^{-1}；结合条件数解释，不把给定候选解说成求解器输出
陷阱辨析与复现记录	五条主张逐条给出判断和依据；保存输入、环境、代码与实际输出；“未运行”必须如实注明，不能以库函数吻合或单次运行代替全部论证

- 难度、预计课后用时与来源： 综合题，中等难度；预计60—90分钟（手算与推导约20—30分钟，程序实现与核验约30—40分钟，辨析与整理约10—20分钟），不增加课堂学时。

题2：对应4.2 迭代法：CG与预条件

考虑一维泊松边值问题

$$-u''(x)=f(x),\quad x\in(0,1),\qquad u(0)=u(1)=0,$$

均匀网格 $x_i=ih$，$h=1/(n+1)$，$i=1,\dots,n$，取 $n=8,16,32$。中心差分离散得

$$A_h=\frac{1}{h^2}\mathrm{tridiag}(-1,\,2,\,-1)\in\mathbb{R}^{n\times n},$$

这是对称正定三对角矩阵。右端取制造解 $u_*(x)=\sin(\pi x)$ 的离散值，令 $b_h=A_hu_*$，从而已知真解，可独立核验误差。所有输入为自构造、无量纲数据。

比较三种方法：

1. 无预条件CG；
2. Jacobi-PCG，预条件子 $M_J=\mathrm{diag}(A_h)=\frac{2}{h^2}I$；
3. 直接法（Thomas算法），作为成本参照。

全部使用 `float64`，`np.linalg.solve` 仅用于小规模对照。

推导或概念小问

1. CG适用条件。 说明 A_h 对称正定，写出CG第 k 步解所在Krylov子空间 $\mathcal{K}_k(A_h,b_h)$，并说明CG在 A_h-范数下最小化误差。

2. Jacobi预条件是否有效。 本题 $\mathrm{diag}(A_h)=\frac{2}{h^2}I$ 是标量矩阵。写出 $M_J^{-1}A_h$ 的表达式，说明它与 A_h 只差常数倍，因此

$$\kappa_2(M_J^{-1}A_h)=\kappa_2(A_h)=O(h^{-2}),$$

并据此解释：Jacobi预条件在本题不能改变迭代次数的量级，只能改变常数因子。

3. 条件数与迭代次数。 一维 A_h 的特征值为

$$\lambda_k = \frac{4}{h^2}\sin^2\frac{k\pi h}{2}, \qquad k = 1, \dots, n,$$

故 $\kappa_2(A_h) = \dfrac{\lambda_{\max}}{\lambda_{\min}} \approx \dfrac{4}{h^2\pi^2} = O(h^{-2})$。引用CG收敛界

$$\frac{\|u_* - u_k\|_{A_h}}{\|u_* - u_0\|_{A_h}} \le 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

定性说明无预条件CG达到固定精度约需 $k = O(h^{-1}) = O(\sqrt{n})$ 步。

4. 成本口径。 定义总成本 = 迭代时间（Jacobi含每步对角缩放，成本 $O(n)$）。说明比较时必须统一容差 $\|r_k\|_2/\|b_h\|_2 \le 10^{-10}$、$u_0 = 0$、$N_{\max} = 10^4$。解释为什么本题中Jacobi-PCG虽迭代次数与CG同量级，但每步多一次 $O(n)$ 缩放，总时间可能不占优。

代码或结果分析小问

使用Python与NumPy实现并运行，全部 `float64`。

1. 实现与正确性。 实现三对角装配、CG、Jacobi-PCG、Thomas直接法。对 $n = 8$ 用 `np.linalg.solve` 对照，报告相对无穷范数误差，应 $< 10^{-10}$。统一容差、初值、最大迭代数如上。
2. 迭代表。 对 $n = 8, 16, 32$ 提交表格：方法、n、迭代次数、最终相对残差、相对误差 $\|u_k - u_*\|_2/\|u_*\|_2$、迭代时间。解释每个指标。
3. 残差曲线。 对 $n = 16$ 绘制CG与Jacobi-PCG相对残差随迭代次数变化的半对数曲线，标注容差线。说明两条曲线为何接近平行（收敛因子相近）。
4. 成本对照。 报告 $n = 8, 16, 32$ 下CG、Jacobi-PCG、Thomas直接法的总时间排序。说明在小规模下直接法可能最快，并解释原因（CG迭代次数虽少但每步有向量运算开销，直接法 $O(n)$ 且常数小）。

陷阱辨析小问

1. “CG对任意非奇异矩阵都收敛。”
2. “本题Jacobi预条件能显著减少CG迭代次数。”
3. “迭代次数少的方法，总运行时间一定短。”
4. “残差 $\|b_h - A_h u_k\|_2$ 很小，解误差就一定很小。”

验收依据

核验项目	应满足的条件或提交证据
CG适用条件	写出 A_h 对称正定；写出Krylov子空间与 A_h-范数最优性
Jacobi分析	指出 $\mathrm{diag}(A_h)$ 为标量矩阵；说明 $\kappa(M_J^{-1}A_h) = \kappa(A_h) = O(h^{-2})$
条件数与迭代	写出一维特征值公式与 $\kappa = O(h^{-2})$；定性给出 $k = O(h^{-1})$
正确性对照	$n = 8$ 三种方法与 `solve` 相对误差 $< 10^{-10}$

核验项目	应满足的条件或提交证据
迭代表	$n=8,16,32$ 均达相对残差 $\le 10^{-10}$；报告迭代次数、误差、时间
残差曲线	半对数图正确，两曲线接近平行，标注容差线
成本对照	给出三种方法总时间排序；说明小规模下直接法可能最快及原因
陷阱辨析	四条逐条判断并给本题证据；不以单次运行代替论证
复现记录	保存输入、环境、代码与实际输出；“未运行”如实注明

难度、预计课后用时与来源

基础综合题，中等难度；预计40—60分钟（推导约10—15分钟，实现与核验约20—30分钟，辨析与整理约10—15分钟），不增加课堂学时。

题3：对应4.3 特征值与SVD

一、基础题（概念辨析）

题 1（基础） 判断下列说法是否正确，并说明理由：

1. 任意方阵都可以正交对角化。
2. 任意矩阵都存在 SVD。
3. 非对称矩阵的特征值绝对值就是它的奇异值。
4. 条件数越大，矩阵越病态。
5. 截断 SVD 的重构误差小，说明下游预测一定好。

题 2（基础） 用自己的语言回答：

1. 特征值和奇异值分别描述矩阵的什么性质？为什么说 SVD 是“任意矩阵的正交拉伸”？
2. 什么是亏损矩阵？举一个例子。
3. 二范数条件数的定义是什么？它依赖范数吗？

题 3（基础） 填空：

1. 若 A 可逆，则 $\kappa_2(A)=$ $\dfrac{\sigma_{\max}(A)}{\qquad}$。
2. 对任意矩阵 A，$\|A\|_2=$ _____，$\|A\|_F=$ _____。
3. 若 $A=U\Sigma V^{\mathsf{T}}$，则 $A^{\mathsf{T}}A$ 的特征值为 _____。
4. 截断 SVD A_k 的谱范数误差为 _____，F 范数误差为 _____。

二、计算题

题 4（计算） 设

$$A=\begin{bmatrix}3 & 0\\ 0 & -2\end{bmatrix}.$$

求 A 的特征值和奇异值，并计算 $\kappa_2(A)$。

题 5（计算） 设

$$A=\begin{bmatrix}0 & 2\\ 0 & 0\end{bmatrix}.$$

求 A 的特征值、奇异值、秩和 $\kappa_2(A)$。若把 A 的第二个元素改为 $2+\varepsilon$，奇异值如何变化？特征值如何变化？

题 6（计算） 设

$$A=\begin{bmatrix}2 & 1\\ 1 & 2\end{bmatrix}.$$

1. 验证 A 对称正定。
2. 求 A 的特征值和奇异值。
3. 计算 $\kappa_2(A)$。
4. 计算 $A^{\mathsf T}A$ 的特征值，验证 $\sigma_i=\sqrt{\lambda_i(A^{\mathsf T}A)}$。

题 7（计算） 设

$$M=\begin{bmatrix}3 & 0\\ 0 & 1\end{bmatrix}.$$

对 $k=0,1,2$，计算截断 SVD M_k、谱范数误差 $\|M-M_k\|_2$、F 范数误差 $\|M-M_k\|_F$，并验证 Eckart–Young 公式。

题 8（计算） 设

$$A=\begin{bmatrix}1 & 10^{4}\\ 0 & 1\end{bmatrix}.$$

1. 求 A 的特征值。
2. 求 A 的奇异值（可用 $A^{\mathsf T}A$ 的特征值）。
3. 计算 $\kappa_2(A)$。
4. 与例 4.3.3 对比，说明非正规矩阵特征值对扰动敏感的现象。

三、证明题

题 9（证明） 设 $A\in\mathbb{R}^{m\times n}$。证明 $A^{\mathsf T}A$ 是对称半正定矩阵，并由此说明它的特征值非负。

题 10（证明） 设 $A=U\Sigma V^{\mathsf T}$ 是 SVD。证明 $A^{\mathsf T}A=V\Sigma^{\mathsf T}\Sigma V^{\mathsf T}$，并由此推出 $\sigma_i(A)=\sqrt{\lambda_i(A^{\mathsf T}A)}$。

题 11（证明） 设 $A\in\mathbb{R}^{n\times n}$ 可逆，$Ax=b$，$A(x+\delta x)=b+\delta b$。证明

$$\frac{\|\delta x\|_2}{\|x\|_2}\le\kappa_2(A)\frac{\|\delta b\|_2}{\|b\|_2}.$$

题 12（证明） 证明：对任意 $A,E\in\mathbb{R}^{m\times n}$，有

$$|\sigma_i(A+E)-\sigma_i(A)|\le\|E\|_2,\qquad i=1,\dots,\min(m,n).$$

（可引用 Ky Fan 变分刻画；若不用变分刻画，给出你能写出的证明或说明证明思路。）

题 13（证明） 证明：若 $A\in\mathbb{R}^{n\times n}$ 正规（$A^{\mathsf T}A=AA^{\mathsf T}$），则 $|\lambda_i(A)|=\sigma_i(A)$（按适当排序）。举一个非正规矩阵的反例说明一般情形不成立。

四、编程题

题 14（编程） 用 Python/NumPy 完成以下任务，并写出运行前预测：

1. 构造 $A=\mathrm{diag}(1,10^{-8})$，$b=(1,10^{-8})^{\mathsf{T}}$，$\delta b=(0,10^{-8})^{\mathsf{T}}$。计算 $\kappa_2(A)$、$\|\delta x\|/\|x\|$、新解对原右端的相对残差。
2. 构造 $A=\begin{bmatrix}2&1\\0&3\end{bmatrix}$，分别用 `np.linalg.eigvals` 和 `np.linalg.svd` 计算特征值和奇异值，比较结果。
3. 构造 $M=\begin{bmatrix}1&2\\3&4\\5&6\end{bmatrix}$，对 $k=0,1,2$ 做截断 SVD，打印谱误差和相对 F 误差，与 Eckart–Young 公式核对。

题 15（编程） 用 Python/NumPy 验证 PCA 必须中心化：

1. 取 4 个二维样本 $(1,1),(2,3),(3,2),(4,4)$，中心化后做 SVD，打印奇异值和第一主成分。
2. 不对数据做中心化，直接做 SVD，打印奇异值和第一主成分。
3. 比较两种情况，解释为什么 PCA 必须先中心化。

题 16（编程） 用 Python/NumPy 验证 Weyl 不等式：

1. 取 $A=\begin{bmatrix}2&1\\0&3\end{bmatrix}$，$E=\varepsilon I$，$\varepsilon=10^{-3},10^{-2},10^{-1}$。
2. 对每个 ε，计算 $|\sigma_i(A+\varepsilon I)-\sigma_i(A)|$，检查是否 $\le\varepsilon$。
3. 取 $A=\begin{bmatrix}1&1/\varepsilon\\0&1\end{bmatrix}$，$E=\begin{bmatrix}0&0\\\varepsilon&0\end{bmatrix}$，对特征值做同样的检查，观察是否仍满足 Lipschitz 界。

五、讨论题

题 17（讨论） 有人说：“只要残差小，解就一定准。”请用本节的例子反驳这句话，并说明讨论残差时必须同时说明哪些信息。

题 18（讨论） LoRA 的低秩增量 BA 与“对预训练权重 W_0 做截断 SVD”有什么不同？两者都叫“低秩”，但数学对象和训练流程分别是什么？

题 19（讨论） 低秩重构误差小，是否意味着推荐系统效果好？请从 Eckart–Young 定理的适用范围和推荐系统的评价指标两方面回答。

题 20（讨论） 与 4.1 的 LU、4.2 的 CG 相比，SVD 在“求解 $Ax=b$”这件事上扮演什么角色？为什么大规模问题通常不直接做完整 SVD？

题4：对应4.4 AI关联：可微求解器、PCA、LoRA

样本矩阵 $X\in\mathbb{R}^{m\times d}$，$m=6$，$d=4$，取

$$X=\begin{bmatrix}1&1&1&1\\2&3&2&3\\3&2&4&2\\4&4&3&5\\5&6&5&6\\6&5&7&5\end{bmatrix}.$$

- 预训练权重 $W_0\in\mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$，取 $d_{\text{out}}=4$，$d_{\text{in}}=6$，W_0 由题给确定矩阵（见代码小问）。
- LoRA 增量 BA，其中 $B\in\mathbb{R}^{d_{\text{out}}\times r}$，$A\in\mathbb{R}^{r\times d_{\text{in}}}$，秩 $r=2$。

- 可微求解器：$K(\theta)\in\mathbb{R}^{3\times 3}$ 对称正定，右端 $f\in\mathbb{R}^3$，解 $x=K(\theta)^{-1}f$，损失 $\ell=\frac{1}{2}\|x-x_*\|_2^2$。

需要回答的具体问题：

1. PCA：对 X 中心化后做SVD，求奇异值、第一主成分方向、$k=2$ 的截断重构与相对F范数误差；并说明不中心化时第一主成分如何改变。
2. LoRA：给定 W_0,B,A，计算 $W=W_0+BA$ 的秩、BA 的秩、$\|W-W_0\|_F$；说明 BA 与“对 W_0 做截断SVD”在数学对象上的区别；计算将 BA 合并进 W_0 前后的推理输出差异（取一个输入向量）。
3. 可微求解器：对 $K(\theta)$ 写出 x 对 θ 的梯度公式（隐函数微分），并用有限差分核验一个分量；说明为什么不能对迭代求解器简单“截断反传”。

推导或概念小问

完成以下四项，写出依据。

1. PCA的中心化与SVD。 写出中心化矩阵 $\tilde{X}=X-\mathbf{1}\mu^{\mathsf{T}}$（$\mu$ 为列均值），说明PCA等价于对 $\tilde{X}$ 做SVD $\tilde{X}=U\Sigma V^{\mathsf{T}}$，第一主成分为 V 的第一列。解释为什么必须中心化：不中心化时SVD分解的是 X，第一主成分倾向于指向样本均值方向，而非最大方差方向。给出 k 截断重构 $\tilde{X}_k=U_k\Sigma_kV_k^{\mathsf{T}}$ 及Eckart–Young误差 $\|\tilde{X}-\tilde{X}_k\|_F^2=\sum_{i>k}\sigma_i^2$。
2. LoRA的数学对象与维度。 设 $W_0\in\mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$，$B\in\mathbb{R}^{d_{\text{out}}\times r}$，$A\in\mathbb{R}^{r\times d_{\text{in}}}$，$r\ll\min(d_{\text{out}},d_{\text{in}})$。写出前向 $y=W_0x+BAx$，说明 BA 的秩不超过 r，可训练参数数为 $r(d_{\text{out}}+d_{\text{in}})$，远小于 $d_{\text{out}}d_{\text{in}}$。说明 W_0 在训练中冻结，B,A 随机初始化（如 $B=0$）保证初始增量为零。解释 BA 不是对 W_0 的截断SVD：BA 是训练得到的增量，不要求逼近 W_0 的主子空间。
3. 可微求解器的隐函数梯度。 设 $K(\theta)x=f$，K 对称正定，$x=K^{-1}f$。对 θ 的分量 θ_j 求导：$K\frac{\partial x}{\partial\theta_j}+\frac{\partial K}{\partial\theta_j}x=0$，故

$$\frac{\partial x}{\partial\theta_j}=-K^{-1}\frac{\partial K}{\partial\theta_j}x.$$

若损失 $\ell=\frac{1}{2}\|x-x_*\|_2^2$，则 $\frac{\partial\ell}{\partial\theta_j}=-(x-x_*)^{\mathsf{T}}K^{-1}\frac{\partial K}{\partial\theta_j}x$。说明实际计算只需解一个以 K 为系数、右端为 $x-x_*$ 的线性系统（伴随求解），不必显式形成 K^{-1}。解释为什么对迭代求解器直接截断反传会得到有偏梯度。

代码或结果分析小问

使用Python与NumPy实现并运行以下实验，全部计算使用 `float64`。`np.linalg.svd`、`np.linalg.solve` 可用于对照，但PCA中心化、截断重构、LoRA前向与梯度核验须展示计算过程。

1. PCA中心化对照。 对给定 X 分别做中心化SVD与不中心化SVD，打印奇异值、第一主成分方向、$k=2$ 截断重构的相对F范数误差。用Eckart–Young公式核对误差。解释两种情况下第一主成分的差异。
2. LoRA维度与输出。 取

$$W_0=\begin{bmatrix}1&0&2&0&1&0\\0&1&0&2&0&1\\1&1&0&0&1&1\\0&0&1&1&0&0\end{bmatrix},\quad B=\begin{bmatrix}1&0\\0&1\\1&1\\0&1\end{bmatrix},\quad A=\begin{bmatrix}1&0&1&0&1&0\\0&1&0&1&0&1\end{bmatrix}.$$

计算 $\mathrm{rank}(W_0)$、$\mathrm{rank}(BA)$、$\mathrm{rank}(W_0+BA)$、$\|W-W_0\|_F$、可训练参数数。取输入 $x=(1,0,1,0,1,0)^{\mathsf{T}}$，比较 W_0x 与 $(W_0+BA)x$。说明 BA 与对 W_0 做 $r=2$ 截断SVD得到的 $W_0^{(2)}$ 在数值和含义上的区别（可计算 $\|W_0-W_0^{(2)}\|_F$ 与 $\|BA\|_F$ 对比）。

3. 可微求解器梯度核验。 取

$$K(\theta)=\begin{bmatrix}2+\theta_1 & 1 & 0\\ 1 & 3 & 1\\ 0 & 1 & 2+\theta_2\end{bmatrix},\quad f=\begin{bmatrix}1\\2\\3\end{bmatrix},\quad x_*=\begin{bmatrix}0\\0\\0\end{bmatrix},$$

在 $\theta=(0.1,0.2)$ 处用隐函数公式计算 $\partial\ell/\partial\theta_1,\partial\ell/\partial\theta_2$，并用中心差分（步长 10^{-6}）核验，报告相对误差。说明伴随求解只需一次线性求解。

4. 故障注入：PCA未中心化。 复制中心化PCA实现，仅删除减均值语句，其余不变。保留正确实现，先预测第一主成分和重构误差会如何变化，再运行错误实现，报告第一主成分与正确值的夹角、$k=2$ 相对F范数误差，并与正确实现比较。用断言检测“未中心化”与“中心化”的第一主成分不同。不能只展示“程序没有报错”。

5. 适用条件测试。 对矩阵

$$S=\begin{bmatrix}1 & 2\\ 2 & 1\end{bmatrix}$$

检查其是否正定（求特征值），并说明若在可微求解器中把 K 换成 S 且 S 不正定，隐函数梯度公式是否仍成立、求解是否稳定。给出明确判断，不要求实现完整反传。

陷阱辨析小问

逐条判断以下主张是否正确；错误主张须给出本题中的反例或计算证据，正确主张须补齐成立条件。以下语句均为教学构造，不是任何AI系统的真实原话。

1. “LoRA就是对预训练权重 W_0 做截断SVD，取前 r 个奇异值。”
2. “PCA不中心化也能得到最大方差方向，只是数值略有不同。”
3. “可微求解器可以对迭代求解器的每次迭代直接反传，和隐函数梯度等价。”
4. “LoRA的秩 r 越小，可训练参数越少，所以 r 越小效果一定越好。”

验收依据

数学条件、实际程序输出和解释三者应一致，具体按下表核验。阈值只用于本题确定性小矩阵，不作为所有问题的统一误差保证。

核验项目	应满足的条件或提交证据
PCA中心化推导	写出中心化矩阵与SVD；说明不中心化第一主成分偏向均值方向；给出Eckart – Young误差公式
PCA数值	中心化与非中心化奇异值、第一主成分、$k=2$ 相对F误差均报告；与Eckart – Young公式核对一致
LoRA维度与秩	正确写出 BA 秩 $\le r$、参数数 $r(d_{\text{out}}+d_{\text{in}})$；给出本题 $\mathrm{rank}(BA)$、$\mathrm{rank}(W_0+BA)$、$\|W-W_0\|_F$
LoRA与截断SVD辨析	明确二者数学对象不同：BA 为训练增量，截断SVD为 W_0 的最优低秩逼近；给出数值对比

核验项目	应满足的条件或提交证据
隐函数梯度	写出 $\partial x/\partial\theta_j = -K^{-1}(\partial K/\partial\theta_j)x$ 与伴随形式；说明只需一次线性求解、不显式求逆
梯度核验	隐函数梯度与中心差分相对误差小于 10^{-5}
故障注入	未中心化实现的第一主成分与正确值夹角明显非零；相对F误差大于中心化实现；断言检测到差异
非正定测试	对 S 给出特征值与正定性判断；说明不正定时隐函数梯度与求解的适用性问题
陷阱辨析	五条主张逐条判断并给出本题证据；不以库函数吻合或单次运行代替论证
复现记录	保存输入、环境、代码与实际输出；“未运行”须如实注明

难度、预计课后用时与来源

综合题，中等偏上难度；预计75—105分钟（推导约20—25分钟，实现与核验约40—55分钟，辨析与整理约15—25分钟），不增加课堂学时。

第04章 矩阵的解剖：习题参考解答

状态：已完成，题号与A1组-习题.md一一对应。

题1：对应4.1 直接法：LU与Cholesky

- 条件与符号： 对应A1组-习题.md中的题1；本题的矩阵、右端及误差指标均按题面取值。

$$A=\begin{bmatrix}0&2&1\\1&-1&0\\2&1&3\end{bmatrix},\quad b=\begin{bmatrix}0\\2\\7\end{bmatrix},\qquad H=\begin{bmatrix}4&2&0\\2&5&2\\0&2&5\end{bmatrix},\quad g=\begin{bmatrix}2\\1\\8\end{bmatrix}.$$

采用$PA=LU$，其中L为单位下三角矩阵；采用$H=CC^T$，其中C为正对角下三角矩阵。
$P,L,U,C,A,H\in\mathbb{R}^{3\times3}$，单右端向量均属于$\mathbb{R}^3$。多个右端的制造解为

$$X_*=\begin{bmatrix}1&-1\\2&0\\-1&2\end{bmatrix}\in\mathbb{R}^{3\times2},\qquad B_A=AX_*,\quad B_H=HX_*.$$

向量无穷范数为最大绝对分量，矩阵无穷范数为最大绝对行和。全部程序使用`float64`。分解误差、相对残差ρ_j、归一化残差η_j、前向误差e_j和库解差异d_j严格使用题面定义，不能互相替代。

- 手动推导或概念解释： 以下依次对应题面的四个推导小问和五个数值实验。

一、高斯消元与列主元LU

1. 非奇异性与零首主元

沿第一行展开行列式：

$$\begin{aligned}\det A&=-2\det\begin{bmatrix}1&0\\2&3\end{bmatrix}+\det\begin{bmatrix}1&-1\\2&1\end{bmatrix}\\&=-2\cdot3+(1+2)=-3\neq0.\end{aligned}$$

因此方程组有唯一解。但$a_{11}=0$，原顺序消元的第一步会出现除以零，必须先换行。零首主元与矩阵奇异是不同的判断。

2. 第一步：选第一列主元并消元

第一列候选元素为$0,1,2$，绝对值最大的是第三行的2。交换第1、3行，同时交换右端：

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 7 \\ 1 & -1 & 0 & 2 \\ 0 & 2 & 1 & 0 \end{array}\right].$$

第一步乘子为

$$\ell_{21}=\frac{1}{2},\qquad \ell_{31}=0.$$

第2行减去第1行的一半；第3行保持不变，得到

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 7 \\ 0 & -3/2 & -3/2 & -3/2 \\ 0 & 2 & 1 & 0 \end{array}\right].$$

3. 第二步：换行时同步调整L中的已存乘子

第二列尚未处理的候选元素为$-3/2$与2，故交换当前第2、3行：

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 7 \\ 0 & 2 & 1 & 0 \\ 0 & -3/2 & -3/2 & -3/2 \end{array}\right].$$

此前保存的第一列乘子也随行归属改变：当前L中的(ℓ_{21},ℓ_{31})应从$(1/2,0)$交换为$(0,1/2)$。只交换已经完成的第一列，不交换L尚未处理的单位对角块。

新的第二列乘子为

$$\ell_{32}=\frac{-3/2}{2}=-\frac{3}{4}.$$

第3行减去$-3/4$倍的第2行，即加上$3/4$倍的第2行，得到

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 7 \\ 0 & 2 & 1 & 0 \\ 0 & 0 & -3/4 & -3/2 \end{array}\right].$$

4. 写出并核验因子

最终行顺序为原第3、1、2行，故

$$P=\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix},\quad L=\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & -3/4 & 1 \end{bmatrix},\quad U=\begin{bmatrix} 2 & 1 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & -3/4 \end{bmatrix}.$$

直接相乘：

$$LU=\begin{bmatrix} 2 & 1 & 3 \\ 0 & 2 & 1 \\ 1 & -1 & 0 \end{bmatrix}=PA.$$

例如乘积第三行是$\frac{1}{2}U_{1,:}-\frac{3}{4}U_{2,:}+U_{3,:}=(1,-1,0)$，正好恢复置换后的第三行。

5. 前代与回代

先计算$Pb=(7,0,2)^T$。由$Ly=Pb$，

$$y_1=7,\qquad y_2=0,\qquad y_3=2-\frac{1}{2}\cdot 7+\frac{3}{4}\cdot 0=-\frac{3}{2}.$$

由$Ux=y$，从最后一行向上：

$$x_3=\frac{-3/2}{-3/4}=2,$$

$$x_2=\frac{0-x_3}{2}=-1,$$

$$x_1=\frac{7-x_2-3x_3}{2}=\frac{7+1-6}{2}=1.$$

因此

$$\boxed{x=(1,-1,2)^T},\qquad Ax=\begin{bmatrix}-2+2\\1+1\\2-1+6\end{bmatrix}=\begin{bmatrix}0\\2\\7\end{bmatrix}=b.$$

A不对称，例如$a_{12}=2\neq 1=a_{21}$，所以它不能写成本节所要求的实CC^T形式，不适用本节Cholesky算法。非奇异性并不足以保证Cholesky适用。

二、对称正定矩阵与Cholesky

1. 正定性依据

$H=H^T$。它的三个顺序主子式为

$$\Delta_1=4>0,\qquad \Delta_2=4\cdot 5-2\cdot 2=16>0,$$

$$\Delta_3=4(25-4)-2(10)=84-20=64>0.$$

由实对称矩阵的Sylvester判据，H正定。下面的正对角分解还会给出一个不依赖该判据的直接验证。

2. 逐元素计算因子

设$H=CC^T$，C为下三角矩阵且$c_{ii}>0$。按公式

$$c_{jj}=\sqrt{h_{jj}-\sum_{k=1}^{j-1}c_{jk}^2},\qquad c_{ij}=\frac{h_{ij}-\sum_{k=1}^{j-1}c_{ik}c_{jk}}{c_{jj}}\quad (i>j),$$

逐项计算：

$$c_{11}=\sqrt{4}=2,\qquad c_{21}=2/2=1,\qquad c_{31}=0/2=0,$$

$$c_{22}=\sqrt{5-1^2}=2,\qquad c_{32}=\frac{2-0\cdot 1}{2}=1,$$

$$c_{33}=\sqrt{5-0^2-1^2}=2.$$

所以

$$\boxed{C=\begin{bmatrix}2&0&0\\1&2&0\\0&1&2\end{bmatrix}}.$$

核验

$$CC^T=\begin{bmatrix}2&0&0\\1&2&0\\0&1&2\end{bmatrix}\begin{bmatrix}2&1&0\\0&2&1\\0&0&2\end{bmatrix}=\begin{bmatrix}4&2&0\\2&5&2\\0&2&5\end{bmatrix}=H.$$

C的对角元均为2，故可逆。对任意非零w，$C^Tw\neq 0$，于是

$$w^THw=\|C^Tw\|_2^2>0,$$

也直接证明了H正定。

3. 前代与转置回代

由$Cq=g$，

$$q_1=\frac{2}{2}=1,\qquad q_2=\frac{1-q_1}{2}=0,\qquad q_3=\frac{8-q_2}{2}=4.$$

再由$C^Tz=q$，

$$z_3=\frac{q_3}{2}=2,\qquad z_2=\frac{q_2-z_3}{2}=-1,\qquad z_1=\frac{q_1-z_2}{2}=1.$$

故

$$\boxed{z=(1,-1,2)^T},\qquad Hz=\begin{bmatrix}4-2\\2-5+4\\-2+10\end{bmatrix}=\begin{bmatrix}2\\1\\8\end{bmatrix}=g.$$

这里$c_{ii}=2$，前代必须除以c_{ii}，不能按单位下三角矩阵处理。第二次求解的方程是$C^Tz=q$，它的第i行为

$$c_{ii}z_i+\sum_{j=i+1}^{n}c_{ji}z_j=q_i,$$

所以回代分子使用q_i，而不是g_i；上三角因子的元素是c_{ji}，而不是c_{ij}。

三、多右端与因子复用

1. 右端与三角求解的维数

由制造解直接计算

$$B_A=\begin{bmatrix}3&2\\-1&-1\\1&4\end{bmatrix},\qquad B_H=\begin{bmatrix}8&-4\\10&2\\-1&10\end{bmatrix}.$$

LU求解流程为

$$LY_A=PB_A,\qquad U\widehat{X}_A=Y_A.$$

其中$L,U,P\in\mathbb{R}^{3\times3}$，$B_A,Y_A,\widehat{X}_A\in\mathbb{R}^{3\times2}$。因此每个乘积的形状都是$(3\times3)(3\times2)=3\times2$。

Cholesky求解流程为

$$CY_H=B_H,\qquad C^T\widehat{X}_H=Y_H,$$

其中$C\in\mathbb{R}^{3\times3}$，$B_H,Y_H,\widehat{X}_H\in\mathbb{R}^{3\times2}$。

2. LU多右端的逐行计算

$$PB_A=\begin{bmatrix}1&4\\3&2\\-1&-1\end{bmatrix}.$$

前代得到

$$(Y_A)_{1,:}=(1,4),\qquad (Y_A)_{2,:}=(3,2),$$

$$
\begin{aligned}
(Y_A)_{3,:} &= (-1,-1) - \tfrac{1}{2}(1,4) + \tfrac{3}{4}(3,2) \\
&= (3/4, -3/2).
\end{aligned}
$$

回代得到

$$
(\widehat{X}_A)_{3,:} = (-1,2), \qquad (\widehat{X}_A)_{2,:} = \frac{(3,2)-(-1,2)}{2} = (2,0),
$$

$$
(\widehat{X}_A)_{1,:} = \frac{(1,4)-(2,0)-3(-1,2)}{2} = (1,-1).
$$

所以$\widehat{X}_A = X_*$。

3. Cholesky多右端的逐行计算

前代得到

$$
(Y_H)_{1,:} = \frac{(8,-4)}{2} = (4,-2),
$$

$$
(Y_H)_{2,:} = \frac{(10,2)-(4,-2)}{2} = (3,2), \qquad (Y_H)_{3,:} = \frac{(-1,10)-(3,2)}{2} = (-2,4).
$$

回代得到

$$
(\widehat{X}_H)_{3,:} = (-1,2), \qquad (\widehat{X}_H)_{2,:} = \frac{(3,2)-(-1,2)}{2} = (2,0),
$$

$$
(\widehat{X}_H)_{1,:} = \frac{(4,-2)-(2,0)}{2} = (1,-1).
$$

所以$\widehat{X}_H = X_*$。这里每个行向量均有两个分量，一次行更新同时处理两个右端。

4. 计算量与复用条件

按乘法和加减各计一次的口径，一次前代或一次回代的主项约为n^2次运算，二者合计约$2n^2$。忽略低阶项：

方法	每个右端重新分解	分解一次、处理s个右端
LU	$s(2n^3/3 + 2n^2)$	$2n^3/3 + 2sn^2$
Cholesky	$s(n^3/3 + 2n^2)$	$n^3/3 + 2sn^2$

LU分解主项$2n^3/3$来自消元更新的累计乘加；Cholesky利用对称性将其减半。因子只取决于系数矩阵，因此A或H不变时可以复用；右端的变化只引起三角求解的变化。若系数矩阵改变，旧因子通常不再是新矩阵的精确分解，不能直接冒用。

四、残差到前向误差的推导

设K非奇异、$Kx_* = f \neq 0$，则$x_* \neq 0$。记$r = f - K\widehat{x}$，有

$$
K(x_* - \widehat{x}) = r, \qquad x_* - \widehat{x} = K^{-1}r.
$$

对任意矩阵M及向量v，由逐行三角不等式，

$$
\begin{aligned}
\|Mv\|_\infty &= \max_i \left| \sum_j m_{ij} v_j \right| \\
&\le \max_i \sum_j |m_{ij}||v_j| \\
&\le \left(\max_i \sum_j |m_{ij}| \right) \|v\|_\infty = \|M\|_\infty \|v\|_\infty.
\end{aligned}
$$

这就是所用范数的相容性。因此

$$\|x_*-\widehat{x}\|_\infty \le \|K^{-1}\|_\infty \|r\|_\infty .$$

再由$f=Kx_*$，

$$\|f\|_\infty \le \|K\|_\infty \|x_*\|_\infty, \qquad \frac{1}{\|x_*\|_\infty} \le \frac{\|K\|_\infty}{\|f\|_\infty}.$$

合并得到

$$\boxed{\frac{\|\widehat{x}-x_*\|_\infty}{\|x_*\|_\infty} \le \|K^{-1}\|_\infty \|K\|_\infty \frac{\|r\|_\infty}{\|f\|_\infty} = \kappa_\infty(K)\rho.}$$

当$f=0$时，ρ的分母为零，不能继续使用该相对指标；同时真解$x_*=0$，以$\|x_*\|$为分母的相对前向误差也不定义。此时应报告绝对残差与绝对误差，并可用

$$\|\widehat{x}\|_\infty \le \|K^{-1}\|_\infty \|r\|_\infty$$

作绝对误差估计。这里K^{-1}用于数学推导，不要求程序显式形成逆矩阵。

五、遗漏交换乘子的错误为何必然被本题检测到

第一步保存的乘子为$1/2, 0$，若第二步换行时没有交换它们，返回的错误因子是

$$\widetilde{L}=\begin{bmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ 0 & -3/4 & 1 \end{bmatrix}.$$

P和U仍与正确实现相同。计算

$$\widetilde{L}U=\begin{bmatrix} 2 & 1 & 3 \\ 1 & 5/2 & 5/2 \\ 0 & -3/2 & -3/2 \end{bmatrix},$$

$$PA-\widetilde{L}U=\begin{bmatrix} 0 & 0 & 0 \\ -1 & -1/2 & -3/2 \\ 1 & 1/2 & 3/2 \end{bmatrix}.$$

故

$$\|PA-\widetilde{L}U\|_\infty = 3, \qquad \|A\|_\infty = \max(3,2,6)=6,$$

$$\boxed{\varepsilon_{\text{bad}} = \frac{3}{6}=\frac{1}{2}.}$$

如果仍使用$\widetilde{L}y_{\text{bad}}=Pb$、$Ux_{\text{bad}}=y_{\text{bad}}$求解，前代得到

$$y_{\text{bad}}=\begin{bmatrix} 7 \\ -7/2 \\ -5/8 \end{bmatrix},$$

回代依次得到

$$(x_{\text{bad}})_3=\frac{5}{6}, \qquad (x_{\text{bad}})_2=-\frac{13}{6}, \qquad (x_{\text{bad}})_1=\frac{10}{3}.$$

代回原方程，

$$Ax_{\text{bad}}=\begin{bmatrix} -7/2 \\ 11/2 \\ 7 \end{bmatrix}, \qquad r_{\text{bad}}=b-Ax_{\text{bad}}=\begin{bmatrix} 7/2 \\ -7/2 \\ 0 \end{bmatrix}.$$

所以

$$\boxed{\|r_{\text{bad}}\|_\infty = \frac{7}{2}}, \qquad \frac{\|r_{\text{bad}}\|_\infty}{\|b\|_\infty} = \frac{1}{2}.$$

错误实现仍可能正常返回一个向量，但它解的是错误因子定义的方程。必须用原始A, b计算残差，才能确认它是否解对原问题。

六、非正定输入与病态反例

1. 对称且对角元全正不保证正定

对$S = \left[\begin{smallmatrix} 1 & 2 \\ 2 & 1 \end{smallmatrix}\right]$，取$v = (1, -1)^T$，则

$$Sv = (-1, 1)^T, \qquad v^T S v = -2 < 0.$$

尝试Cholesky时，第一列为$c_{11} = 1$、$c_{21} = 2$，第二个平方根项是

$$s_{22} - c_{21}^2 = 1 - 4 = -3 < 0.$$

所以实正对角Cholesky算法应明确报告失败。若擅自把-3改为$|-3|$，计算出的乘积便不再等于原始S，这不是合法修正。

2. SPD矩阵也可能病态

本题

$$D = \operatorname{diag}(1, 10^{-12}), \quad x_*^{(D)} = (1, 1)^T, \quad f_D = (1, 10^{-12})^T, \quad \widehat{x}^{(D)} = (1, 3/2)^T.$$

所以

$$r_D = f_D - D\widehat{x}^{(D)} = (0, -\tfrac{1}{2} \cdot 10^{-12})^T,$$

$$\boxed{\rho_D = 5 \times 10^{-13}}, \qquad \boxed{e_D = \frac{1}{2}}.$$

又有$\|D\|_\infty = 1$、$\|\widehat{x}^{(D)}\|_\infty = 3/2$、$\|f_D\|_\infty = 1$，因此

$$\boxed{\eta_D = \frac{5 \times 10^{-13}}{3/2 + 1} = 2 \times 10^{-13}}.$$

由于$D^{-1} = \operatorname{diag}(1, 10^{12})$，

$$\boxed{\kappa_\infty(D) = 10^{12}}, \qquad \kappa_\infty(D)\rho_D = 10^{12} \cdot 5 \times 10^{-13} = \frac{1}{2} = e_D.$$

因此误差界在精确算术中恰好取等号。D虽然对称正定，但其第二个对角元很小，对应方向上的解误差会被压缩为极小残差。这个例子反驳了“小残差必然保证小前向误差”，不说明某个实际求解器在此返回了$\widehat{x}^{(D)}$，因为该候选解是题面人为指定的。

- 实现位置与独立参考： 完整实现位于本文件下列四个Python代码块，按出现顺序运行即可；仅依赖NumPy及Python标准库，无隐藏变量或外部数据文件。

迁入本章Notebook 4.1节时的建议单元	内容	独立参考与核验
题1-单元1	正确LU与故障LU	上文手算P, L, U；故障版仅省略已存乘子交换

迁入本章Notebook 4.1节时的建议单元	内容	独立参考与核验
题1-单元2	Cholesky与三角求解	上文手算C、单右端前代回代
题1-单元3	制造解、多右端求解与误差表	已知X_*为主要参考；`np.linalg.solve`及`cholesky`为库函数对照
题1-单元4	漏换乘子、非正定输入与病态反例	精确手算$\varepsilon_{\text{bad}} = 1/2$、$\|r_{\text{bad}}\|_\infty = 7/2$及上述解析反例

上述单元名称是迁入Notebook后的建议安排；本次实际执行的是本Markdown中提取的代码，未把尚未生成的Notebook单元当作已完成成果。

代码块1：正确LU与故障版本

```
import sys
import platform
import numpy as np


def as_square_real(A):
    """检查实数、有限、非空方阵，并复制为float64。"""
    if np.iscomplexobj(A):
        raise ValueError("本节实现只处理实数矩阵")
    A = np.array(A, dtype=np.float64, copy=True)
    if A.ndim != 2 or A.shape[0] != A.shape[1] or A.shape[0] == 0:
        raise ValueError("A必须为非空方阵")
    if not np.all(np.isfinite(A)):
        raise ValueError("A含有非有限数值")
    return A


def lu_pp(A):
    """部分选主元LU：返回p, L, U，使A[p, :] = L @ U。"""
    U = as_square_real(A)
    n = U.shape[0]
    L = np.eye(n)
    p = np.arange(n)
    for k in range(n):
        q = k + np.argmax(np.abs(U[k:, k]))
        if U[q, k] == 0.0:
            raise np.linalg.LinAlgError("当前候选主元全为零，无法继续")
        if q != k:
            U[[k, q], :] = U[[q, k], :]
            p[[k, q]] = p[[q, k]]
            # 必须同步交换此前已存储的乘子，只交换前k列。
            L[[k, q], :k] = L[[q, k], :k]
        for i in range(k + 1, n):
            L[i, k] = U[i, k] / U[k, k]
            U[i, k + 1:] -= L[i, k] * U[k, k + 1:]
            U[i, k] = 0.0
    return p, L, U

def lu_pp_bad(A):
    """故意错误：仅省略换行时对已存乘子的交换，用于故障实验。"""
    U = as_square_real(A)
    n = U.shape[0]
    L = np.eye(n)
```

```
    p = np.arange(n)
    for k in range(n):
        q = k + np.argmax(np.abs(U[k:, k]))
        if U[q, k] == 0.0:
            raise np.linalg.LinAlgError("当前候选主元全为零，无法继续")
        if q != k:
            U[[k, q], :] = U[[q, k], :]
            p[[k, q]] = p[[q, k]]
            # 必须同步交换此前已存储的乘子，只交换前k列。
            # 故意删去L已存乘子的交换语句。
        for i in range(k + 1, n):
            L[i, k] = U[i, k] / U[k, k]
            U[i, k + 1:] -= L[i, k] * U[k, k + 1:]
            U[i, k] = 0.0
    return p, L, U
```

故障函数与正确函数的数值操作完全相同，唯一故意差异是缺少交换L已存乘子的语句。它独立保留，正确函数未被覆盖。

代码块2：Cholesky、前代与回代

```
def chol_lower(A):
    """实对称正定矩阵的Cholesky教学实现，返回下三角因子。"""
    A = as_square_real(A)
    # 本节实验输入精确对称，因此使用严格检查。
    if not np.array_equal(A, A.T):
        raise ValueError("输入矩阵不对称")
    n = A.shape[0]
    L = np.zeros_like(A)
    for j in range(n):
        d = A[j, j] - L[j, :j] @ L[j, :j]
        if d <= 0.0:
            raise np.linalg.LinAlgError("出现非正平方根项：检查输入与舍入误差")
        L[j, j] = np.sqrt(d)
        for i in range(j + 1, n):
            L[i, j] = (A[i, j] - L[i, :j] @ L[j, :j]) / L[j, j]
    return L

def rhs_matrix(B, n):
    """内部统一为二维数组；记录原输入是否为向量。"""
    if np.iscomplexobj(B):
        raise ValueError("本节实现只处理实数右端")
    C = np.array(B, dtype=np.float64, copy=True)
    is_vector = (C.ndim == 1)
    if is_vector:
        C = C[:, None]
    if C.ndim != 2 or C.shape[0] != n or C.shape[1] == 0:
        raise ValueError("右端形状应为(n,)或(n,s)，且s至少为1")
    if not np.all(np.isfinite(C)):
        raise ValueError("右端含有非有限数值")
    return C, is_vector


def forward_substitution(L, B, unit_diagonal=False):
    L = as_square_real(L)
    C, is_vector = rhs_matrix(B, L.shape[0])
    Y = np.zeros_like(C)
    for i in range(L.shape[0]):
        row = C[i, :] - L[i, :i] @ Y[:i, :]
        if unit_diagonal:
```

```
            Y[i, :] = row
        else:
            if L[i, i] == 0.0:
                raise np.linalg.LinAlgError("下三角因子存在零对角元")
            Y[i, :] = row / L[i, i]
    return Y[:, 0] if is_vector else Y


def backward_substitution(U, B):
    U = as_square_real(U)
    C, is_vector = rhs_matrix(B, U.shape[0])
    X = np.zeros_like(C)
    for i in range(U.shape[0] - 1, -1, -1):
        if U[i, i] == 0.0:
            raise np.linalg.LinAlgError("上三角因子存在零对角元")
        X[i, :] = (C[i, :] - U[i, i + 1:] @ X[i + 1:, :]) / U[i, i]
    return X[:, 0] if is_vector else X


def lu_solve_factored(p, L, U, B):
    C, is_vector = rhs_matrix(B, L.shape[0])
    Y = forward_substitution(L, C[p, :], unit_diagonal=True)
    X = backward_substitution(U, Y)
    return X[:, 0] if is_vector else X


def chol_solve_factored(L, B):
    Y = forward_substitution(L, B)
    return backward_substitution(L.T, Y)
```

p为最终排列向量，A[p, :]对应PA，B[p, :]对应PB；不是逐步主元交换记录。unit_diagonal=True只用于LU中的单位下三角因子，不能用于本题C。

代码块3：主实验与误差表

```
print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__, "架构:", platform.machine())

A = np.array([[0., 2., 1.], [1., -1., 0.], [2., 1., 3.]])
b = np.array([0., 2., 7.])
H = np.array([[4., 2., 0.], [2., 5., 2.], [0., 2., 5.]])
g = np.array([2., 1., 8.])
Xstar = np.array([[1., -1.], [2., 0.], [-1., 2.]])
BA, BH = A @ Xstar, H @ Xstar
single_true = np.array([1., -1., 2.])
eps = np.finfo(np.float64).eps
tau = 500 * A.shape[0] * eps


def ni(x):
    return np.linalg.norm(x, ord=np.inf)


def compare_columns(K, B, X, true_X, reference_X):
    rows = []
    for j in range(B.shape[1]):
        f, x = B[:, j], X[:, j]
        residual = f - K @ x
        rho = ni(residual) / ni(f)
        eta = ni(residual) / (ni(K) * ni(x) + ni(f))
```

```
        err = ni(x - true_X[:, j]) / ni(true_X[:, j])
        diff = ni(x - reference_X[:, j]) / ni(reference_X[:, j])
        rows.append((rho, eta, err, diff))
    return np.array(rows)


## 主求解流程：A和H各分解一次；保留原始输入以计算残差。
p, L, U = lu_pp(A)
C = chol_lower(H)
P = np.eye(3)[p, :]
assert np.array_equal(p, [2, 0, 1])
assert np.array_equal(L, [[1, 0, 0], [0, 1, 0], [.5, -.75, 1]])
assert np.array_equal(U, [[2, 1, 3], [0, 2, 1], [0, 0, -.75]])
assert np.array_equal(C, [[2, 0, 0], [1, 2, 0], [0, 1, 2]])
assert np.allclose(C, np.linalg.cholesky(H), rtol=0, atol=tau)

XA = lu_solve_factored(p, L, U, BA)
XH = chol_solve_factored(C, BH)
xa = lu_solve_factored(p, L, U, b)
zh = chol_solve_factored(C, g)
assert XA.shape == XH.shape == (3, 2)
assert xa.shape == zh.shape == (3,)
assert ni(xa - single_true) < tau
assert ni(zh - single_true) < tau

factor_A = ni(P @ A - L @ U) / ni(A)
factor_H = ni(H - C @ C.T) / ni(H)
metrics_A = compare_columns(A, BA, XA, Xstar, np.linalg.solve(A, BA))
metrics_H = compare_columns(H, BH, XH, Xstar, np.linalg.solve(H, BH))
assert factor_A < tau and factor_H < tau
assert np.all(metrics_A < tau) and np.all(metrics_H < tau)
print("tau=%.16e" % tau)
print("A的单右端解:", xa, "原方程绝对残差=%.3e" % ni(b - A @ xa))
print("H的单右端解:", zh, "原方程绝对残差=%.3e" % ni(g - H @ zh))
print("多右端LU解=\n", XA)
print("多右端Cholesky解=\n", XH)
for name, factor, metrics in [("LU", factor_A, metrics_A),
                              ("Cholesky", factor_H, metrics_H)]:
    for j, row in enumerate(metrics, start=1):
        print("%s 右端%d: factor=%.3e, rho=%.3e, eta=%.3e, e=%.3e, d=%.3e"
              % (name, j, factor, *row))
```

主求解流程对A, H各分解一次，既求多右端，又复用因子求单右端。库函数基线与故障恢复复测是额外验证，不计作“对每个右端重复分解”的主求解流程。

代码块4：故障检测、适用条件与病态反例

```
## 故障实验：保留正确函数，调用仅漏掉乘子交换的副本。
p_bad, L_bad, U_bad = lu_pp_bad(A)
x_bad = lu_solve_factored(p_bad, L_bad, U_bad, b)
bad_factor = ni(A[p_bad, :] - L_bad @ U_bad) / ni(A)
bad_residual = b - A @ x_bad
assert bad_factor > 1e-2 and ni(bad_residual) > 1e-2
assert np.isclose(bad_factor, .5, rtol=0, atol=1e-14)
assert np.allclose(x_bad, [10/3, -13/6, 5/6], rtol=0, atol=1e-14)
assert np.allclose(bad_residual, [3.5, -3.5, 0.], rtol=0, atol=1e-14)
print("错误L=\n", L_bad)
print("错误解:", x_bad)
print("错误分解相对误差=%.16e" % bad_factor)
print("错误解原方程残差:", bad_residual)
```

```
print("错误解绝对残差=%.16e" % ni(bad_residual))

## 恢复复测：重新调用含有交换语句的正确函数，额外分解属于验证。
p_ok, L_ok, U_ok = lu_pp(A)
X_ok = lu_solve_factored(p_ok, L_ok, U_ok, BA)
assert ni(A[p_ok, :] - L_ok @ U_ok) / ni(A) < tau
assert np.all(compare_columns(A, BA, X_ok, Xstar, np.linalg.solve(A, BA)) < tau)
assert ni(b - A @ lu_solve_factored(p_ok, L_ok, U_ok, b)) < tau
print("恢复正确交换后：分解、单右端及多右端复测通过")

## 预期失败：捕获明确的异常，同时断言确实发生。
S = np.array([[1., 2.], [2., 1.]])
v = np.array([1., -1.])
assert v @ S @ v == -2.0
failed = False
try:
    chol_lower(S)
except np.linalg.LinAlgError as error:
    failed = True
    print("Cholesky按预期失败:", error)
assert failed
print("S的第二个平方根项:", 1. - 2.**2)

## 病态反例：候选解由题面给定；不将其冒充求解器输出。
D = np.diag([1., 1e-12])
x_true_D = np.ones(2)
fD = D @ x_true_D
x_hat_D = np.array([1., 1.5])
rD = fD - D @ x_hat_D
rhoD = ni(rD) / ni(fD)
errD = ni(x_hat_D - x_true_D) / ni(x_true_D)
etaD = ni(rD) / (ni(D) * ni(x_hat_D) + ni(fD))
## 对角矩阵的无穷范数条件数直接由对角元素求得，无需形成逆矩阵。
absolute_diagonal = np.abs(np.diag(D))
kappaD = np.max(absolute_diagonal) / np.min(absolute_diagonal)
assert np.isclose(rhoD, 5e-13, rtol=1e-14, atol=0)
assert np.isclose(etaD, 2e-13, rtol=1e-14, atol=0)
assert errD == .5 and kappaD == 1e12
assert rhoD < 1e-10 and errD > 1e-1
assert errD <= kappaD * rhoD * (1 + 10 * eps)
print("D的残差:", rD)
print("D: rho=%.16e, e=%.16e, eta=%.16e" % (rhoD, errD, etaD))
print("D: kappa=%.16e, kappa*rho=%.16e" % (kappaD, kappaD*rhoD))
```

- 实际输出、误差和容差理由： 以下为2026-10-24在本次执行环境中的实际运行结果。没有进行计时性能比较，也未以输出相同代替数学推导。

环境与精度： Python 3.12.14，NumPy 2.3.5，x86_64，`float64`；无随机输入、无需随机种子。主实验容差$\tau = 3.3306690738754696 \times 10^{-13}$。

方法	右端	分解相对误差	ρ_j	η_j	e_j	d_j
LU	1	0.000e+00	0.000e+00	0.000e+00	0.000e+00	0.000e+00
LU	2	0.000e+00	0.000e+00	0.000e+00	0.000e+00	0.000e+00
Cholesky	1	0.000e+00	0.000e+00	0.000e+00	0.000e+00	0.000e+00
Cholesky	2	0.000e+00	0.000e+00	0.000e+00	0.000e+00	0.000e+00

单右端数值解均为$(1,-1,2)^T$，两组多右端解均与X_*一致；本次单右端原方程绝对残差均为0。

补充核验	实际输出	结论
漏换乘子	分解相对误差5.0000000000000000e-01；原方程绝对残差3.5000000000000000e+00	超过故障检测阈值，错误被识别
恢复正确语句	分解、单右端与多右端断言均通过	修复后复测通过
对称不定矩阵S	捕获LinAlgError；第二个平方根项为-3	按预期失败
病态矩阵D	ρ_D = 5.0000000000000009e-13；e_D = 5.0000000000000000e-01	小残差与较大解误差同时出现
D的归一化指标与误差界	η_D = 2.0000000000000003e-13；$\kappa_\infty(D)$ = 1.0000000000000000e+12；$\kappa_\infty(D)\rho_D$ = 5.0000000000000011e-01	与手算值相符，浮点误差范围内满足上界

浮点输出与解析值的最后几位可能略有差异；例如程序中的$5.0000000000000009\times10^{-13}$对应解析值$5\times10^{-13}$。验收以题目阈值和解析核验为依据。

误差指标的解释。 分解误差检查保存的因子是否对应原系数矩阵；ρ_j检查解对原方程的满足程度；η_j加入了$\|K\|\|\widehat{x}_j\|+\|f_j\|$的尺度归一化；$e_j$与制造解比较，衡量实际前向误差；$d_j$与库函数输出比较，只能作为另一实现的交叉核验。本题制造解与三阶主实验数据均为可精确表示的小整数；因子只含整数和有限二进制分数，故本次部分计算恰好无舍入误差。打印零不代表任意矩阵、任意平台或无限精度下都没有误差。

容差理由。 使用$\tau=500n\,\mathrm{eps}$、$n=3$，其中eps是float64在1附近的间隔；留出三角求解、矩阵乘法及不同底层实现的舍入裕量。该阈值用于本题良态、尺度温和的三阶主实验，不是一般矩阵的误差定理。舍入到最近数时常用的单位舍入误差为$u=\mathrm{eps}/2$，不要将两者混用。

验收结论。 正确实现的两个主实验均通过题面阈值；故障实现的分解误差和原方程绝对残差均明显大于10^{-2}；非正定输入按预期失败；病态反例满足$\rho_D<10^{-10}$且$e_D>10^{-1}$。程序中的断言同时验证数值、已知因子、形状及预期失败；故障恢复后重新通过主实验。没有验证大型稀疏矩阵上的性能，也没有声称一次实验覆盖所有输入。

- 陷阱的正确解释及适用范围： 五条主张的判断与题目顺序一致。

序号	判断	正确解释与证据
1	错误	本题$\det A=-3\neq0$，但$a_{11}=0$，原顺序第一步不能消元。非奇异矩阵可做带主元LU；无主元LU需要额外条件。对非奇异矩阵，所有顺序主子式非零是本节单位下三角LU存在的条件。
2	错误	本节“列主元”是在当前列选主元，实际交换行。采用$PA=LU$时右端必须变成Pb，多个右端变成PB_A。列交换属于另一类变量重排，不能混用。

序号	判断	正确解释与证据
3	错误	S对称且对角元均为1，但 $v^TSv=-2<0$，第二个平方根项为-3。本节要求实对称正定性，不能只检查正对角元。
4	正确，有条件	系数矩阵保持不变、既有分解有效且维数相容时，因子可以复用，只需重新做三角求解。系数矩阵改变后，旧因子通常不再是新矩阵的精确分解。
5	错误	本题D的候选解有 $\rho_D=5\times10^{-13}<10^{-10}$，但$e_D=1/2$。还须结合$\kappa_\infty(D)=10^{12}$解释前向误差；正定性不能保证条件数小。

题2：对应4.2 迭代法：CG与预条件

一、推导或概念小问

1. CG适用条件与Krylov子空间

$A_h=\frac{1}{h^2}\mathrm{tridiag}(-1,2,-1)$ 对称（$A_h^{\mathsf T}=A_h$），且对任意 $v\neq0$，

$$v^{\mathsf T}A_hv=\frac{1}{h^2}\sum_{i=1}^{n}\left(v_i-v_{i-1}\right)^2>0,\qquad v_0=v_{n+1}=0,$$

故对称正定。CG第 k 步解满足

$$u_k\in u_0+\mathcal{K}_k(A_h,r_0),\qquad \mathcal{K}_k(A_h,r_0)=\mathrm{span}\{r_0,A_hr_0,\dots,A_h^{k-1}r_0\},\quad r_0=b_h-A_hu_0,$$

并在该空间中最小化 A_h-范数误差 $\|u_*-u\|_{A_h}=\sqrt{(u_*-u)^{\mathsf T}A_h(u_*-u)}$。对称正定保证三项递推中 $p_k^{\mathsf T}A_hp_k>0$，短递推成立。

2. Jacobi预条件是否有效

$\mathrm{diag}(A_h)=\frac{2}{h^2}I$ 是标量矩阵，故

$$M_J^{-1}A_h=\frac{h^2}{2}A_h,$$

与 A_h 只差正常数倍，特征值同比缩放，因此

$$\kappa_2(M_J^{-1}A_h)=\kappa_2(A_h)=O(h^{-2}).$$

CG收敛速度由 $\kappa=\kappa_2(M^{-1/2}A_hM^{-1/2})$ 决定；条件数量级不变，收敛因子的量级不变，迭代次数仍为 $O(h^{-1})=O(\sqrt{n})$，Jacobi只改变常数因子（甚至本题中因标量缩放对CG迭代序列无实质改变）。结论：本题Jacobi预条件不能改变迭代次数量级。

3. 条件数与迭代次数

一维 A_h 特征值为

$$\lambda_k=\frac{4}{h^2}\sin^2\frac{k\pi h}{2},\quad k=1,\dots,n.$$

$\lambda_{\min}=\lambda_1=\frac{4}{h^2}\sin^2\frac{\pi h}{2}\approx\pi^2$（$h\to0$），$\lambda_{\max}=\lambda_n=\frac{4}{h^2}\cos^2\frac{\pi h}{2}\approx\frac{4}{h^2}$，故

$$\kappa_2(A_h) \approx \frac{4}{\pi^2 h^2} = O(h^{-2}).$$

CG收敛界

$$\frac{\|u_* - u_k\|_{A_h}}{\|u_* - u_0\|_{A_h}} \le 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \approx 2\left(1 - \frac{2}{\sqrt{\kappa}}\right)^k.$$

要达固定精度，需 $k \sim \frac{\sqrt{\kappa}}{2}\ln\frac{2}{\varepsilon} = O(\sqrt{\kappa}) = O(h^{-1}) = O(\sqrt{n})$ 步。

4. 成本口径

总成本 = 迭代时间。统一容差 $\|r_k\|_2/\|b_h\|_2 \le 10^{-10}$、$u_0 = 0$、$N_{\max} = 10^4$。

- 无预条件CG每步一次稀疏矩阵-向量乘（三对角 $O(n)$）加若干 $O(n)$ 向量运算。
- Jacobi-PCG每步额外一次对角缩放 $z = M_J^{-1} r = \frac{h^2}{2} r$，也是 $O(n)$，但多一次向量乘常数。
- 本题 M_J 为标量，PCG与CG的迭代序列几乎相同（数学上等价于对 $\frac{h^2}{2}A_h$ 跑CG再缩放），迭代次数相同，但PCG每步多一次缩放运算，总时间不占优。这就是“迭代次数同量级时，每步成本决定总时间”的体现。

二、代码实现

```
import numpy as np
import time
import matplotlib.pyplot as plt

def assemble_A(n):
    h = 1.0/(n+1)
    off = -np.ones(n-1, dtype=np.float64)
    dia = 2.0 * np.ones(n, dtype=np.float64)
    A = np.diag(dia) + np.diag(off, 1) + np.diag(off, -1)
    A /= h**2
    return A, h

def thomas(a, b, c, rhs):
    """
    Thomas算法求解：b主对角，a下对角，c上对角
    a[0] unused, c[-1] unused
    A = diag(a[1:],-1)+diag(b)+diag(c[:-1],+1)
    """
    n = len(rhs)
    c_prime = np.zeros(n, dtype=np.float64)
    d_prime = np.zeros(n, dtype=np.float64)
    x = np.zeros(n, dtype=np.float64)
    c_prime[0] = c[0]/b[0]
    d_prime[0] = rhs[0]/b[0]
    for i in range(1, n):
        denom = b[i] - a[i]*c_prime[i-1]
        c_prime[i] = c[i]/denom
        d_prime[i] = (rhs[i] - a[i]*d_prime[i-1])/denom
    x[-1] = d_prime[-1]
    for i in range(n-2, -1, -1):
        x[i] = d_prime[i] - c_prime[i]*x[i+1]
    return x

def solve_thomas_wrapper(n, b):
    h = 1.0/(n+1)
    a = np.full(n, -1.0/h**2, dtype=np.float64)
    bdia = np.full(n, 2.0/h**2, dtype=np.float64)
```

```
    c = np.full(n, -1.0/h**2, dtype=np.float64)
    return thomas(a, bdia, c, b)

def cg(A, b, u0, tol=1e-10, maxit=10000):
    u = u0.copy()
    r = b - A @ u
    p = r.copy()
    res_hist = [np.linalg.norm(r)/np.linalg.norm(b)]
    for k in range(maxit):
        Ap = A @ p
        alpha = (r.T @ r)/(p.T @ Ap)
        u = u + alpha * p
        r_new = r - alpha * Ap
        beta = (r_new.T @ r_new)/(r.T @ r)
        p = r_new + beta * p
        r = r_new
        rel_res = np.linalg.norm(r)/np.linalg.norm(b)
        res_hist.append(rel_res)
        if rel_res <= tol:
            return u, k+1, rel_res, res_hist
    return u, maxit, rel_res, res_hist

def jacobi_pcg(A, b, u0, tol=1e-10, maxit=10000):
    # M_J = diag(A) = 2/h^2 I, M^{-1} = h^2/2 I
    n = len(b)
    h = 1.0/(n+1)
    Minv = (h**2 / 2.0)
    u = u0.copy()
    r = b - A @ u
    z = Minv * r
    p = z.copy()
    res_hist = [np.linalg.norm(r)/np.linalg.norm(b)]
    for k in range(maxit):
        Ap = A @ p
        alpha = (r.T @ z)/(p.T @ Ap)
        u = u + alpha * p
        r_new = r - alpha * Ap
        z_new = Minv * r_new
        beta = (r_new.T @ z_new)/(r.T @ z)
        p = z_new + beta * p
        r = r_new
        z = z_new
        rel_res = np.linalg.norm(r)/np.linalg.norm(b)
        res_hist.append(rel_res)
        if rel_res <= tol:
            return u, k+1, rel_res, res_hist
    return u, maxit, rel_res, res_hist

def run_case(n):
    A, h = assemble_A(n)
    xi = np.linspace(h, 1-h, n)
    ustar = np.sin(np.pi * xi)
    b = A @ ustar
    u0 = np.zeros_like(b)
    results = {}

    # Thomas
    t0 = time.time()
    u_th = solve_thomas_wrapper(n, b)
    t1 = time.time()
    err_th = np.linalg.norm(u_th - ustar, 2)/np.linalg.norm(ustar,2)
```

```
    results["Thomas"] = {"u":u_th, "time":t1-t0, "err":err_th, "it":1}

    # CG
    t0 = time.time()
    u_cg, it_cg, rr_cg, hist_cg = cg(A,b,u0)
    t1 = time.time()
    err_cg = np.linalg.norm(u_cg - ustar,2)/np.linalg.norm(ustar,2)
    results["CG"] = {"u":u_cg, "time":t1-t0, "err":err_cg, "it":it_cg, "res_hist":hist_cg, "rr":rr_cg}

    # Jacobi PCG
    t0 = time.time()
    u_pcg, it_pcg, rr_pcg, hist_pcg = jacobi_pcg(A,b,u0)
    t1 = time.time()
    err_pcg = np.linalg.norm(u_pcg - ustar,2)/np.linalg.norm(ustar,2)
    results["JacobiPCG"] = {"u":u_pcg, "time":t1-t0, "err":err_pcg, "it":it_pcg, "res_hist":hist_pcg, "rr":rr_pcg}
    return results, ustar

if __name__=="__main__":
    np.set_printoptions(precision=4)
    ns = [8,16,32]
    table = []
    res_n16 = None
    for n in ns:
        res, ustar = run_case(n)
        if n == 16:
            res_n16 = res
        for name in ["Thomas","CG","JacobiPCG"]:
            d = res[name]
            table.append({
                "n":n,
                "method":name,
                "iter":d["it"],
                "rel_res":d.get("rr",0.0),
                "rel_err":d["err"],
                "time":d["time"]
            })
    print(f"{'method':<10}{'n':<4}{'iter':<6}{'rel_res':<14}{'rel_err':<14}{'time(s)'}")
    for row in table:
        print(f"{row['method']:<10}{row['n']:<4}{row['iter']:<6}{row['rel_res']:<14.3e}{row['rel_err']:<14.3e}
{row['time']:.3e}")

    # n=16残差半对数图
    plt.figure(figsize=(7,4.5))
    h_cg = res_n16["CG"]["res_hist"]
    h_pcg = res_n16["JacobiPCG"]["res_hist"]
    plt.semilogy(np.arange(len(h_cg)), h_cg, '-o', label="CG", markersize=4)
    plt.semilogy(np.arange(len(h_pcg)), h_pcg, '-s', label="Jacobi-PCG", markersize=4)
    plt.axhline(y=1e-10, color='k', linestyle='--', label="tol $10^{-10}$")
    plt.xlabel("iteration k")
    plt.ylabel(r"$\|r_k\|_2/\|b\|_2$")
    plt.title("n=16: relative residual history")
    plt.legend()
    plt.grid(True, which="both", ls=":")
    plt.tight_layout()
    plt.show()

    # 正确性校验 n=8 和 np.linalg.solve
    n=8
    A,h = assemble_A(n)
    xi = np.linspace(h,1-h,n)
    ustar = np.sin(np.pi*xi)
```

```
    b = A @ ustar
    u_direct = np.linalg.solve(A,b)
    err_inf = np.linalg.norm(u_direct - ustar, np.inf)
    print(f"\nn=8, np.linalg.solve vs true u, inf norm error = {err_inf:.3e}")
```

三、陷阱辨析

陷阱 1：CG 的适用条件

错。 CG 要求系数矩阵对称正定；非对称或非正定矩阵不保证收敛。本题 A_h 满足对称正定，但不能推广为“任意非奇异矩阵都可用 CG”。

陷阱 2：Jacobi 预处理不改变条件数

错。 本题 $\mathbf{diag}(A_h)$ 为标量矩阵，故

$$M_J^{-1}A_h$$

与 A_h 仅差一个常数倍，条件数 κ 的量级不变，迭代次数量级也不变（实际几乎相同）。反例即本题计算：CG 与 Jacobi-PCG 的迭代次数相同。

陷阱 3：Jacobi-PCG 一定比 CG 快

错。 反例见成本对照：本题 Jacobi-PCG 迭代次数与 CG 相同甚至更少（若实现差异），但每步多一次 $O(n)$ 缩放，总时间可能更长；且小规模下 Thomas 一次求解更快。该说法成立需固定“总成本”口径并计入每步开销。

陷阱 4：残差小 ⇒ 误差小

错。 误差受条件数放大：

$$\|u-u_k\| \le \|A_h^{-1}\|\,\|r_k\|,$$

本题 $\kappa = O(h^{-2})$，h 小时小残差仍可对应不可忽略的误差。须结合 κ 与右端范数判断，不能只看残差。

题3：对应4.3 特征值与SVD

一、基础题

题 1 解答

1. 错误。 只有实对称矩阵（更一般地，正规矩阵）才能正交对角化；一般方阵不一定可对角化，例如 $\begin{bmatrix}1 & 1\\ 0 & 1\end{bmatrix}$ 是亏损矩阵。
2. 正确。 任意实矩阵 $A\in\mathbb{R}^{m\times n}$ 都存在 SVD，见正文推导 3。
3. 错误。 非对称矩阵的特征值绝对值一般不等于奇异值。反例 $A=\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}$：特征值 $0,0$，奇异值 $1,0$。
4. 正确。 条件数 $\kappa_2=\sigma_{\max}/\sigma_{\min}$ 越大，矩阵越接近奇异，求逆时误差放大越严重。
5. 错误。 Eckart–Young 只保证重构范数最优，不保证下游任务（分类、回归、推荐）指标最优。低秩重构误差小不等于预测好。

题 2 解答

1. 特征值描述方阵在“不转向方向”上的伸缩倍数；奇异值描述任意矩阵在“正交拉伸方向”上的伸缩倍数。SVD 把任意矩阵拆成“正交变换—伸缩—正交变换”，所以叫“任意矩阵的正交拉伸”。
2. 亏损矩阵是指特征向量不够 n 个线性无关的方阵，即不可对角化。例子：$A=\begin{bmatrix}1 & 1\\ 0 & 1\end{bmatrix}$，特征值 1（二重），但特征空间只有一维。
3. 二范数条件数 $\kappa_2(A)=\|A\|_2\|A^{-1}\|_2=\sigma_{\max}/\sigma_{\min}$（$A$ 可逆）。它依赖范数；换用其他范数数值会变，正文默认二范数。

题 3 解答

1. $\sigma_{\min}(A)$。
2. $\|A\|_2=\sigma_{\max}$，$\|A\|_F=\sqrt{\sum_i \sigma_i^2}$。
3. $A^{\mathsf{T}}A$ 的特征值为 $\sigma_1^2,\dots,\sigma_{\min(m,n)}^2$ 和零（按重数计）。
4. 谱范数误差为 σ_{k+1}，F 范数误差为 $\sqrt{\sum_{i=k+1}^{r}\sigma_i^2}$。

二、计算题

题 4 解答

$A=\operatorname{diag}(3,-2)$。

- 特征值：$3,-2$。
- 奇异值：$|3|=3$，$|-2|=2$，即 $3,2$。
- $\kappa_2(A)=3/2=1.5$。

注意：特征值有负数，奇异值取绝对值；这是对角矩阵的简单情形。

题 5 解答

$A=\begin{bmatrix}0 & 2\\ 0 & 0\end{bmatrix}$。

- 特征多项式 $\det(A-\lambda I)=\lambda^2$，特征值 $0,0$。
- $A^{\mathsf{T}}A=\begin{bmatrix}0 & 0\\ 0 & 4\end{bmatrix}$，特征值 $4,0$，奇异值 $2,0$。
- 秩为 1（一个正奇异值）。
- $\kappa_2(A)=\infty$（A 奇异）。

若把 A 的第二个元素改为 $2+\varepsilon$：

- 特征值仍为 $0,0$（上三角，对角元不变）。
- 奇异值为 $|2+\varepsilon|,0$，变化 ε。
- 特征值不变，奇异值变化 ε。这说明特征值和奇异值对同一扰动的反应可以不同；本例中特征值恰好不变，但不能据此认为特征值总是不敏感（见题 8）。

题 6 解答

$A=\begin{bmatrix}2 & 1\\ 1 & 2\end{bmatrix}$。

1. A 对称；顺序主子式 $2>0$，$\det A=4-1=3>0$，故正定。
2. 特征多项式 $(2-\lambda)^2-1=(\lambda-1)(\lambda-3)$，特征值 $3,1$。因 A 对称正定，奇异值也是 $3,1$。
3. $\kappa_2(A)=3/1=3$。
4. $A^{\mathsf{T}}A=A^2=\begin{bmatrix}5&4\\4&5\end{bmatrix}$，特征值 $9,1$。$\sqrt{9}=3$，$\sqrt{1}=1$，与奇异值一致。

题 7 解答

$M=\operatorname{diag}(3,1)$，$\sigma_1=3,\sigma_2=1$，$\|M\|_F=\sqrt{10}$。

- $k=0$：$M_0=0$，$\|M-M_0\|_2=3$，$\|M-M_0\|_F=\sqrt{10}$。
- $k=1$：$M_1=\operatorname{diag}(3,0)$，$\|M-M_1\|_2=1$，$\|M-M_1\|_F=1$。
- $k=2$：$M_2=M$，误差 0。

验证：$k=1$ 时谱误差 $=\sigma_2=1$，F 误差 $=\sqrt{\sigma_2^2}=1$，符合 Eckart–Young 公式。

题 8 解答

$A=\begin{bmatrix}1&10^4\\0&1\end{bmatrix}$。

1. 上三角，特征值 $1,1$（二重，亏损）。
2. $A^{\mathsf{T}}A=\begin{bmatrix}1&10^4\\10^4&10^8+1\end{bmatrix}$。迹 $=10^8+2$，行列式 $=1$。特征值 $\lambda_{1,2}=\frac{10^8+2\pm\sqrt{(10^8+2)^2-4}}{2}$。数值上 $\lambda_1\approx 10^8+2$，$\lambda_2\approx 10^{-8}$。故 $\sigma_1\approx 10^4$，$\sigma_2\approx 10^{-4}$。
3. $\kappa_2(A)\approx 10^8$。
4. 与例 4.3.3 对比：这里 A 的特征值都是 1，但奇异值相差 10^8 倍，条件数 10^8。非正规矩阵的特征值对扰动敏感，且特征值不能反映矩阵的“拉伸范围”；奇异值才能反映条件数。若对 A 加一个小扰动 $E=\begin{bmatrix}0&0\\\varepsilon&0\end{bmatrix}$，特征值会移动 $O(\sqrt{\varepsilon})$ 甚至 $O(1)$（取决于扰动位置），而奇异值满足 Weyl 界 $|\sigma_i(A+E)-\sigma_i(A)|\le\|E\|_2$。

三、证明题

题 9 解答

对称性：$(A^{\mathsf{T}}A)^{\mathsf{T}}=A^{\mathsf{T}}(A^{\mathsf{T}})^{\mathsf{T}}=A^{\mathsf{T}}A$。

半正定性：对任意 $x\in\mathbb{R}^n$，

$$x^{\mathsf{T}}A^{\mathsf{T}}Ax=(Ax)^{\mathsf{T}}(Ax)=\|Ax\|_2^2\ge 0.$$

故 $A^{\mathsf{T}}A$ 对称半正定。由正文推论 4.3.1，对称半正定矩阵的特征值非负。□

题 10 解答

由 $A=U\Sigma V^{\mathsf{T}}$ 和 $U^{\mathsf{T}}U=I$，

$$A^{\mathsf{T}}A=V\Sigma^{\mathsf{T}}U^{\mathsf{T}}U\Sigma V^{\mathsf{T}}=V(\Sigma^{\mathsf{T}}\Sigma)V^{\mathsf{T}}.$$

$\Sigma^{\mathsf{T}}\Sigma$ 是对角矩阵，对角元为 $\sigma_1^2,\dots,\sigma_{\min(m,n)}^2,0,\dots,0$。因 V 正交，$A^{\mathsf{T}}A$ 与 $\Sigma^{\mathsf{T}}\Sigma$ 正交相似，特征值相同。故 $A^{\mathsf{T}}A$ 的特征值为 σ_i^2（$i\le\min(m,n)$）和零。开方得 $\sigma_i=\sqrt{\lambda_i(A^{\mathsf{T}}A)}$。□

题 11 解答

两式相减得 $A\,\delta x = \delta b$，即 $\delta x = A^{-1}\delta b$。由 $\|A^{-1}\|_2 = 1/\sigma_{\min}$，

$$\|\delta x\|_2 \le \|A^{-1}\|_2 \|\delta b\|_2 = \frac{\|\delta b\|_2}{\sigma_{\min}}.$$

又 $b = Ax$，$\|b\|_2 \le \|A\|_2 \|x\|_2 = \sigma_{\max}\|x\|_2$，即 $1/\|x\|_2 \le \sigma_{\max}/\|b\|_2$。两式相乘：

$$\frac{\|\delta x\|_2}{\|x\|_2} \le \frac{\|\delta b\|_2}{\sigma_{\min}} \cdot \frac{\sigma_{\max}}{\|b\|_2} = \kappa_2(A)\frac{\|\delta b\|_2}{\|b\|_2}.$$

□

题 12 解答

用 Ky Fan 变分刻画：

$$\sigma_i(A) = \max_{\dim S = i} \min_{x \in S, \|x\|=1} \|Ax\|_2.$$

对任意 x，由三角不等式，

$$\|(A+E)x\|_2 \le \|Ax\|_2 + \|E\|_2, \qquad \|(A+E)x\|_2 \ge \|Ax\|_2 - \|E\|_2.$$

对固定 i 维子空间 S，内层 $\min$ 满足

$$\min_{x\in S,\|x\|=1} \|(A+E)x\|_2 \le \min_{x\in S,\|x\|=1} \|Ax\|_2 + \|E\|_2,$$

$$\min_{x\in S,\|x\|=1} \|(A+E)x\|_2 \ge \min_{x\in S,\|x\|=1} \|Ax\|_2 - \|E\|_2.$$

对 S 取外层 $\max$，得

$$\sigma_i(A+E) \le \sigma_i(A) + \|E\|_2, \qquad \sigma_i(A+E) \ge \sigma_i(A) - \|E\|_2.$$

即 $|\sigma_i(A+E) - \sigma_i(A)| \le \|E\|_2$。□

题 13 解答

若 A 正规，则 A 可酉对角化：$A = Q\Lambda Q^*$，Q 酉，Λ 对角。则

$$A^*A = Q\bar{\Lambda}\Lambda Q^* = Q|\Lambda|^2 Q^*.$$

故 A^*A 的特征值为 $|\lambda_i|^2$，奇异值为 $|\lambda_i|$（按适当排序）。□

反例：$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ 非正规。特征值 $0, 0$，奇异值 $1, 0$，$|\lambda_i(A)| \ne \sigma_i(A)$。

四、编程题

题 14 解答

1. 条件数扰动。 预测：$\kappa_2 = 10^8$，$\|\delta x\|/\|x\| = 1/\sqrt{2} \approx 0.7071$，相对残差约 10^{-8}。

```
import numpy as np
A = np.diag([1.0, 1e-8])
b = np.array([1.0, 1e-8])
db = np.array([0.0, 1e-8])
x = np.linalg.solve(A, b)
dx = np.linalg.solve(A, db)
print(np.linalg.cond(A, 2))
```

```
print(np.linalg.norm(dx)/np.linalg.norm(x))
print(np.linalg.norm(b - A@(x+dx))/np.linalg.norm(b))
```

预期输出：`1e8`、`0.7071`、约 `1e-8`。

2. 特征值与奇异值对比。 $A=\begin{bmatrix}2 & 1\\ 0 & 3\end{bmatrix}$：特征值 $2,3$；奇异值约 $3.2566, 1.8424$。`np.linalg.eigvals` 与 `np.linalg.svd(..., compute_uv=False)` 的输出与上述一致（特征值可能有 10^{-16} 量级数值误差）。

3. 截断 SVD。 M 的奇异值约 $9.5255, 0.5143$。$k=1$ 时谱误差与 F 误差都约 0.5143，相对 F 误差约 0.0539；$k=2$ 时误差为 0。与 Eckart–Young 公式一致。

题 15 解答

```
import numpy as np
X = np.array([[1,1],[2,3],[3,2],[4,4]], dtype=float)
Xc = X - X.mean(axis=0)
for M, name in [(Xc, "centered"), (X, "raw")]:
    U, s, Vt = np.linalg.svd(M, full_matrices=False)
    print(name, s, Vt[0])
```

预期：中心化后奇异值 $3,1$，第一主成分 $(1,1)/\sqrt{2}$（符号可反）；未中心化时第一主成分被均值方向主导。PCA 必须先中心化，否则第一主成分解释的是“均值方向”而不是“变化最大的方向”。

题 16 解答

```
import numpy as np
A = np.array([[2,1],[0,3]], dtype=float)
sA = np.linalg.svd(A, compute_uv=False)
for eps in [1e-3, 1e-2, 1e-1]:
    sAe = np.linalg.svd(A + eps*np.eye(2), compute_uv=False)
    print(eps, np.abs(sAe - sA), np.all(np.abs(sAe-sA) <= eps))
```

预期：全部为 `True`，Weyl 界成立。

```
for eps in [1e-1, 1e-2, 1e-4]:
    A = np.array([[1, 1/eps],[0,1]], dtype=float)
    E = np.array([[0,0],[eps,0]], dtype=float)
    print(eps, np.linalg.eigvals(A+E) - np.linalg.eigvals(A), np.linalg.norm(E,2))
```

预期：特征值移动约 1，而 $\|E\|_2=\varepsilon$，比值 $1/\varepsilon$ 随 ε 减小而增大，不满足 Lipschitz 界。

五、讨论题

题 17 解答

反例：$A=\mathrm{diag}(1,10^{-10})$，$b=(1,10^{-10})^{\mathsf{T}}$，只把第二个右端分量增加 10^{-10}。新解对原右端的相对残差约 10^{-10}，但相对解误差约 0.7071。残差小只说明新解对原右端几乎满足方程，不说明接近原解。

讨论残差时必须同时说明：残差针对哪个右端、用什么范数、矩阵条件数多大。后向误差（残差）和前向误差（解误差）是两个不同概念，不能混为一谈。

题 18 解答

LoRA 通常把权重更新写成 $W = W_0 + BA$，其中 $B \in \mathbb{R}^{m\times r}$，$A \in \mathbb{R}^{r\times n}$，只训练低秩增量 BA，参数量约 $r(m+n)$。它不是对已训练好的 W_0 做 SVD 截断；W_0 保持不变，BA 从零开始训练。

“对 W_0 做截断 SVD”是另一种操作：先算出 W_0 的 SVD，再保留前 k 个奇异值，得到低秩近似。两者都利用低秩结构，但数学对象（训练增量 vs 近似已训练权重）和训练流程（优化 vs 一次性分解）不同。

题 19 解答

Eckart－Young 只保证 A_k 在谱范数/F 范数下最优重构 A，不保证推荐指标（如命中率、NDCG、覆盖率）最优。推荐系统还要考虑排序、冷启动、多样性、公平性等。重构误差小可能只是把热门物品拟合得好，不代表推荐质量高。因此不能从“重构误差小”推出“推荐一定好”。

题 20 解答

LU/Cholesky 和 CG 是“求解算法”：它们关心怎样解 $Ax = b$。SVD 是“结构分析工具”：它关心矩阵本身有多敏感、秩和条件数是什么、怎样做低秩近似。SVD 不直接用来解 $Ax = b$，因为完整 SVD 的代价高于 LU（$O(mn\min(m,n))$ 量级，且常数更大）。但 SVD 给出诊断语言：条件数大时，直接法和迭代法都可能遇到困难；伪逆 A^+ 给出最小二乘或秩亏系统的规范解。大规模问题通常用随机 SVD、Lanczos/Arnoldi 等方法近似，而不是显式做完整 SVD。

- B核验、C复核及版本： 待核验。

题4：对应4.4 AI关联：可微求解器、PCA、LoRA

一、PCA

1. 中心化与 SVD

样本矩阵

$$X = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 3 & 2 & 3 \\ 3 & 2 & 4 & 2 \\ 4 & 4 & 3 & 5 \\ 5 & 6 & 5 & 6 \\ 6 & 5 & 7 & 5 \end{bmatrix}, \quad m = 6,\ d = 4.$$

列均值

$$\mu = (3.5,\ 3.5,\ 3.6667,\ 3.6667).$$

中心化矩阵 $\tilde{X} = X - \mathbf{1}\mu^{\mathsf{T}}$：

$$\tilde{X} = \begin{bmatrix} -2.5 & -2.5 & -2.6667 & -2.6667 \\ -1.5 & -0.5 & -1.6667 & -0.6667 \\ -0.5 & -1.5 & 0.3333 & -1.6667 \\ 0.5 & 0.5 & -0.6667 & 1.3333 \\ 1.5 & 2.5 & 1.3333 & 2.3333 \\ 2.5 & 1.5 & 3.3333 & 1.3333 \end{bmatrix}.$$

对 $\tilde{X}$ 做 SVD：$\tilde{X} = U\Sigma V^{\mathsf{T}}$。奇异值为

$$\sigma = (8.2706,\ 2.9410,\ 0.7730,\ 0.1303).$$

第一主成分方向（V 的第一列，符号约定取首分量正）为

$$v_1 \approx (0.4981,\ 0.4812,\ 0.5268,\ 0.4928)^{\mathsf{T}}.$$

它近似指向“各维同向增加”的方向，因为四个特征高度正相关。

$k=2$ 截断重构：$\tilde{X}_2 = U_2\Sigma_2 V_2^{\mathsf{T}}$，

$$\tilde{X}_2 = \begin{bmatrix} -2.5499 & -2.5365 & -2.6340 & -2.6156 \\ -1.2612 & -0.6988 & -1.7832 & -0.5893 \\ -0.4902 & -1.4457 & 0.3219 & -1.7173 \\ 0.1476 & 0.9099 & -0.5072 & 1.1187 \\ 1.6913 & 2.2590 & 1.2487 & 2.4657 \\ 2.4623 & 1.5121 & 3.3538 & 1.3377 \end{bmatrix}.$$

相对 F 范数误差（相对于中心化矩阵）：

$$\frac{\|\tilde{X} - \tilde{X}_2\|_F}{\|\tilde{X}\|_F} = \frac{\sqrt{\sigma_3^2 + \sigma_4^2}}{\sqrt{\sum_i \sigma_i^2}} = \frac{\sqrt{0.7730^2 + 0.1303^2}}{8.8129} \approx 0.0889.$$

Eckart – Young 核对：$\sigma_3^2 + \sigma_4^2 = 0.6145$，$\|\tilde{X}\|_F^2 = 77.6667$，比值 $\sqrt{0.6145/77.6667} = 0.0889$，与直接计算一致。

2. 不中心化时的第一主成分

直接对 X 做 SVD，奇异值为

$$\sigma = (19.4087,\ 2.9467,\ 0.7730,\ 0.1431),$$

第一主成分方向

$$v_1^{(0)} \approx (-0.4900,\ -0.4872,\ -0.5139,\ -0.5084)^{\mathsf{T}}.$$

与中心化结果夹角约

$$\arccos |v_1^{\mathsf{T}} v_1^{(0)}| \approx 1.30^\circ.$$

本题两者接近，是因为 X 的列均值本身近似沿“全 1 方向”，而数据方差也主要沿该方向。关键点在于：不中心化 SVD 分解的是 X，第一主成分最大化的是 $\|Xv\|^2 = \|\mathbf{1}\mu^{\mathsf{T}}v + \tilde{X}v\|^2$，其中均值项 $\sqrt{m}|\mu^{\mathsf{T}}v|$ 会被优先放大。当均值远大于方差（例如把 X 每列加一个大常数 C）时，第一主成分会趋于 $\mu/\|\mu\|$（样本均值方向），而不是最大方差方向。结论：不中心化时第一主成分被样本均值方向“污染”，在均值主导的数据中会明显偏离最大方差方向；本题因均值与主方差方向接近而数值差异小，但机理不同。

不中心化 $k=2$ 相对 F 误差（相对于 X 本身）：

$$\frac{\sqrt{\sigma_3^2 + \sigma_4^2}}{\|X\|_F} = 0.0400.$$

注意这个数不能和中心化误差直接比较（分母、对象都不同），它只说明“用前两个方向近似原始未中心化矩阵”的误差。

二、LoRA

给定

$$W_0=\begin{bmatrix}1&0&2&0&1&0\\0&1&0&2&0&1\\1&1&0&0&1&1\\0&0&1&1&0&0\end{bmatrix}\in\mathbb{R}^{4\times6},\quad B=\begin{bmatrix}1&0\\0&1\\1&1\\0&1\end{bmatrix}\in\mathbb{R}^{4\times2},\quad A=\begin{bmatrix}1&0&1&0&1&0\\0&1&0&1&0&1\end{bmatrix}\in\mathbb{R}^{2\times6}.$$

1. 秩、范数、参数数

$$BA=\begin{bmatrix}1&0&1&0&1&0\\0&1&0&1&0&1\\1&1&1&1&1&1\\0&1&0&1&0&1\end{bmatrix},\quad W=W_0+BA=\begin{bmatrix}2&0&3&0&2&0\\0&2&0&3&0&2\\2&2&1&1&2&2\\0&1&1&2&0&1\end{bmatrix}.$$

数值结果：

- $\mathrm{rank}(W_0)=3$。
- $\mathrm{rank}(BA)=2$（B 列满秩、A 行满秩，乘积秩 $=r=2$）。
- $\mathrm{rank}(W)=4$。
- $\|W-W_0\|_F=\|BA\|_F=\sqrt{15}=3.8730$。
- 可训练参数数 $=r(d_{\text{out}}+d_{\text{in}})=2(4+6)=20$，而全量 W 有 $4\times6=24$ 个参数（本题规模小，优势不明显；一般 d 大时 $r(d_{\text{out}}+d_{\text{in}})\ll d_{\text{out}}d_{\text{in}}$）。

2. 推理输出对比

取 $x=(1,0,1,0,1,0)^{\mathsf{T}}$：

$$W_0x=(4,\ 0,\ 2,\ 1)^{\mathsf{T}},\qquad Wx=(7,\ 0,\ 5,\ 1)^{\mathsf{T}},$$

差异

$$BAx=Wx-W_0x=(3,\ 0,\ 3,\ 0)^{\mathsf{T}}.$$

合并前后输出确实不同；合并（把 BA 加进 W_0 得到 W）不改变推理计算图，只改变权重矩阵，前向结果就是 Wx。

3. BA 与对 W_0 做截断 SVD 的区别

对 W_0 做 $r=2$ 截断 SVD：$W_0^{(2)}=U_2\Sigma_2V_2^{\mathsf{T}}$，数值上

$$\|W_0-W_0^{(2)}\|_F\approx1.7809,\qquad \|BA\|_F=3.8730.$$

两者数值不同，更重要的是数学对象不同：

- $W_0^{(2)}$ 是 W_0 在 Frobenius 范数下的最优秩 2 逼近（Eckart–Young），是 W_0 的主子空间投影，依赖 W_0，与训练数据/任务无关。
- BA 是训练得到的增量，B,A 是优化变量，初始化 $B=0$ 时 $BA=0$（初始不动原模型），训练目标是下游损失，不要求 BA 逼近 W_0 的任何子空间，也不要求 W_0+BA 低秩（本题 $\mathrm{rank}(W)=4$）。
- 因此“LoRA = 对 W_0 做截断 SVD”是错误类比：截断 SVD 是压缩/近似 W_0，LoRA 是在冻结 W_0 上学习低秩修正。

三、可微求解器

1. 隐函数梯度公式

$K(\theta)x=f$，K 对称正定。对 θ_j 求导：

$$K\frac{\partial x}{\partial \theta_j}+\frac{\partial K}{\partial \theta_j}x=0 \implies \frac{\partial x}{\partial \theta_j}=-K^{-1}\frac{\partial K}{\partial \theta_j}x.$$

损失 $\ell=\frac{1}{2}\|x-x_*\|^2$，则

$$\frac{\partial \ell}{\partial \theta_j}=(x-x_*)^{\mathsf{T}}\frac{\partial x}{\partial \theta_j}=-(x-x_*)^{\mathsf{T}}K^{-1}\frac{\partial K}{\partial \theta_j}x.$$

伴随求解：令 $K\lambda=x-x_*$（一次线性求解，K 对称故无需转置），则

$$\frac{\partial \ell}{\partial \theta_j}=-\lambda^{\mathsf{T}}\frac{\partial K}{\partial \theta_j}x.$$

只需一次以 K 为系数的线性求解，不必显式形成 K^{-1}，也不必对每个 θ_j 单独解系统。

2. 数值核验

取

$$K(\theta)=\begin{bmatrix}2+\theta_1 & 1 & 0\\ 1 & 3 & 1\\ 0 & 1 & 2+\theta_2\end{bmatrix},\quad f=(1,2,3)^{\mathsf{T}},\quad x_*=0,\quad \theta=(0.1,0.2).$$

在 θ 处解 $Kx=f$ 得

$$x\approx(0.4393,\ 0.0774,\ 1.3285)^{\mathsf{T}}.$$

$\partial K/\partial\theta_1=E_{11}$，$\partial K/\partial\theta_2=E_{33}$。隐函数梯度：

$$\frac{\partial \ell}{\partial \theta_1}=-(x-x_*)^{\mathsf{T}}K^{-1}E_{11}x,\qquad \frac{\partial \ell}{\partial \theta_2}=-(x-x_*)^{\mathsf{T}}K^{-1}E_{33}x.$$

实际计算（伴随）：解 $K\lambda=x$ 得 $\lambda\approx(-0.3785,\ 0.3555,\ -0.7654)^{\mathsf{T}}$，于是

$$\frac{\partial \ell}{\partial \theta_1}=-\lambda_1x_1\approx-0.1663,\qquad \frac{\partial \ell}{\partial \theta_2}=-\lambda_3x_3\approx-1.0168.$$

中心差分（步长 10^{-6}）核验：

$$\frac{\partial \ell}{\partial \theta_1}\Big|_{\mathrm{FD}}\approx-0.1663,\qquad \frac{\partial \ell}{\partial \theta_2}\Big|_{\mathrm{FD}}\approx-1.0168,$$

相对误差均约 10^{-9} 量级，远小于 10^{-5}。

3. 为什么不能对迭代求解器简单“截断反传”

迭代求解器（如共轭梯度、不动点迭代）实际执行的是有限步近似 $x_T\approx K^{-1}f$。若只对最后 T 步计算图做反传（截断反传），得到的是有限步迭代映射的导数，它一般不等于隐函数导数 $-K^{-1}(\partial K/\partial\theta)x$：

- 截断反传忽略了“未展开的迭代步”对 θ 的依赖，梯度是有偏的；
- 偏差随 T 减小而增大，且依赖于迭代初值、步数、停止准则；
- 隐函数梯度对应 $T\to\infty$（收敛到精确解）的极限，只依赖 K 与 x，与迭代路径无关；
- 若 K 病态或迭代未收敛，截断反传的梯度可能严重偏离真实梯度，甚至符号错误。

因此正确做法是：前向用迭代求解得到 x，反向用隐函数/伴随公式（一次线性求解）计算梯度，而不是对迭代过程逐层反传。

四、推导/概念小问

1. PCA 中心化与 SVD

中心化矩阵 $\tilde{X} = X - \mathbf{1}\mu^{\mathsf{T}}$，$\mu = \frac{1}{m}X^{\mathsf{T}}\mathbf{1}$。PCA 等价于对 $\tilde{X}$ 做 SVD $\tilde{X} = U\Sigma V^{\mathsf{T}}$，第一主成分为 V 第一列，方差为 $\sigma_1^2/(m-1)$。

必须中心化：不中心化时 SVD 分解 X，第一主成分最大化 $\|Xv\|^2 = \|\mathbf{1}\mu^{\mathsf{T}}v + \tilde{X}v\|^2$。均值项 $\sqrt{m}|\mu^{\mathsf{T}}v|$ 可能远大于方差项，导致 v_1 趋于 $\mu/\|\mu\|$（样本均值方向），而非最大方差方向。中心化消除均值项，使 $\|Xv\|^2 = \|\tilde{X}v\|^2$ 只反映方差。

k 截断重构 $\tilde{X}_k = U_k\Sigma_k V_k^{\mathsf{T}}$，Eckart – Young：

$$\|\tilde{X} - \tilde{X}_k\|_F^2 = \sum_{i>k}\sigma_i^2.$$

2. LoRA 数学对象与维度

前向 $y = W_0x + BAx = (W_0 + BA)x$。$B \in \mathbb{R}^{d_{\text{out}}\times r}$，$A \in \mathbb{R}^{r\times d_{\text{in}}}$，$\operatorname{rank}(BA) \le r$。可训练参数 $r(d_{\text{out}} + d_{\text{in}})$，全量 $d_{\text{out}}d_{\text{in}}$。$W_0$ 冻结，B, A 随机初始化（常取 $B = 0$ 使初始 $BA = 0$）。BA 是训练增量，不要求逼近 W_0 的主子空间；截断 SVD 是最优低秩逼近 W_0，二者数学对象不同。

3. 隐函数梯度

如上：$\partial x/\partial\theta_j = -K^{-1}(\partial K/\partial\theta_j)x$，伴随形式 $\partial\ell/\partial\theta_j = -\lambda^{\mathsf{T}}(\partial K/\partial\theta_j)x$，$K\lambda = x - x_*$，一次线性求解。截断反传得到的是有限步迭代映射的导数，一般有偏，不等于隐函数梯度。

五、代码/结果分析小问

1. PCA 中心化对照

结果见第一部分。中心化奇异值 $(8.2706, 2.9410, 0.7730, 0.1303)$，第一主成分 $(0.4981, 0.4812, 0.5268, 0.4928)$，$k = 2$ 相对 F 误差 0.0889（与 Eckart – Young 一致）。不中心化奇异值 $(16.8859, 3.0190, 1.4106, 0.6672)$，第一主成分 $(0.5230, 0.4889, 0.5224, 0.4628)$，$k = 2$ 相对 F 误差（对 X）0.0889。本题两者第一主成分夹角约 $1.30°$，因为均值方向与主方差方向接近；但机理上不中心化第一主成分偏向均值方向，均值主导数据时差异会显著。

2. LoRA 维度与输出

$\operatorname{rank}(W_0) = 3$，$\operatorname{rank}(BA) = 2$，$\operatorname{rank}(W) = 4$，$\|W - W_0\|_F = 3.8729$，可训练参数 20。$W_0x = (4, 0, 2, 1)^{\mathsf{T}}$，$Wx = (7, 0, 5, 1)^{\mathsf{T}}$，差异 $(3, 0, 3, 0)^{\mathsf{T}}$。$\|W_0 - W_0^{(2)}\|_F \approx 1.7809$ 与 $\|BA\|_F = 3.8730$ 数值不同；含义上 $W_0^{(2)}$ 是 W_0 最优秩 2 逼近，BA 是训练增量。

3. 可微求解器梯度核验

$\partial\ell/\partial\theta_1 \approx -0.1663$，$\partial\ell/\partial\theta_2 \approx -1.0168$，中心差分相对误差约 10^{-9}，小于 10^{-5}。伴随只需一次解 $K\lambda = x - x_*$。

4. 故障注入：PCA 未中心化

预测：删除减均值语句后，SVD 作用于 X 而非 $\tilde{X}$，第一主成分被均值方向拉偏；$k = 2$ 重构误差（对 X）与中心化误差不可直接比，但若强行用同一“相对 F 误差”定义（对各自输入矩阵），未中心化实现会报告不同的第一主成分和误差。

运行结果：未中心化第一主成分 $(-0.4900, -0.4872, -0.5139, -0.5084)$，与正确值 $(0.4981, 0.4812, 0.5258, 0.4928)$ 夹角约 $1.30°$（非零，断言通过）；未中心化 $k=2$ 相对 F 误差（对 X）0.0889，中心化（对 $\tilde{X}$）0.8946。注意：这里误差数值大小关系不能简单说“未中心化更好”，因为分母和对象不同；正确解读是两者在分解不同矩阵，未中心化误差小是因为 X 的均值项主导了 F 范数分母，不代表 PCA 更优。断言 `assert not np.allclose(v1_centered, v1_uncentered)` 通过，检测到差异。

5. 适用条件测试

$$S = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \quad \lambda_1 = 3,\ \lambda_2 = -1.$$

S 不正定（有负特征值）。若把可微求解器中的 K 换成 S：S 非奇异时 $x = S^{-1}f$ 仍有定义，隐函数微分公式 $S\,\partial x/\partial\theta_j = -(\partial S/\partial\theta_j)x$ 在代数上仍成立（只要 S 可逆），但：

- S 不正定，$x = S^{-1}f$ 不是能量极小解，迭代求解（如 CG）可能不收敛或数值不稳定；
- 若 S 奇异（本题不是），S^{-1} 不存在，隐函数定理条件不满足，梯度公式失效；
- 正定性保证 K^{-1} 存在、条件数可控、伴随系统稳定可解。因此本题 S 可逆但非正定：梯度公式代数上可用，但求解稳定性与“可微求解器”的凸/稳定前提不满足，实际不应使用。

六、陷阱辨析小问

1. “LoRA 就是对 W_0 做截断 SVD，取前 r 个奇异值。”——错误。 本题 $\|BA\|_F = 3.8730$，而 $\|W_0 - W_0^{(2)}\|_F = 1.7321$，$BA$ 不是 W_0 的最优秩 2 逼近；BA 是训练增量，初始化 $B=0$ 时为 0，截断 SVD 依赖 W_0 且与训练无关。反例即本题数值。

2. “PCA 不中心化也能得到最大方差方向，只是数值略有不同。”——错误。 不中心化 SVD 最大化 $\|Xv\|^2$，包含均值项 $\sqrt{m}|\mu^{\mathsf{T}}v|$。若给本题 X 每列加 $C=100$，第一主成分会趋于全 1 方向（均值方向），而非中心化后的最大方差方向；本题均值与主方差方向接近只是巧合，夹角约 $1°$ 不能证明一般成立。

3. “可微求解器可以对迭代求解器每次迭代直接反传，和隐函数梯度等价。”——错误。 截断反传是有限步迭代映射的导数，忽略未展开步对 θ 的依赖，一般有偏；隐函数梯度是 $T \to \infty$ 的精确解导数。本题中若用很少迭代步截断反传，梯度会偏离 $-0.02204, -0.34030$，且偏差随步数变化。

4. “LoRA 的秩 r 越小，参数越少，效果一定越好。”——错误。 参数少不等于效果好。r 太小表达能力不足（欠拟合），r 太大过参数化/过拟合且省参数优势消失。本题 $r=2$ 时 BA 秩为 2、参数 20；若 $r=1$ 参数更少但 BA 秩至多 1，可能无法表达需要秩 2 的修正。效果需下游验证，不能由 r 单调决定。

七、代码

```
import numpy as np
np.set_printoptions(precision=10, suppress=True)

## ============================================================
## 公共数据
## ============================================================
X = np.array([
    [1, 1, 1, 1],
    [2, 3, 2, 3],
    [3, 2, 4, 2],
```

```
    [4, 4, 3, 5],
    [5, 6, 5, 6],
    [6, 5, 7, 5],
], dtype=np.float64)

W0 = np.array([
    [1, 0, 2, 0, 1, 0],
    [0, 1, 0, 2, 0, 1],
    [1, 1, 0, 0, 1, 1],
    [0, 0, 1, 1, 0, 0],
], dtype=np.float64)

B = np.array([
    [1, 0],
    [0, 1],
    [1, 1],
    [0, 1],
], dtype=np.float64)

A = np.array([
    [1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1],
], dtype=np.float64)

x_in = np.array([1, 0, 1, 0, 1, 0], dtype=np.float64)


def rel_fro_error(X_true, X_approx):
    return np.linalg.norm(X_true - X_approx, ord='fro') / np.linalg.norm(X_true, ord='fro')


def principal_angle_deg(v, w):
    v = v / np.linalg.norm(v)
    w = w / np.linalg.norm(w)
    c = np.clip(abs(np.dot(v, w)), 0.0, 1.0)
    return np.degrees(np.arccos(c))


## ============================================================
## 1. PCA 中心化对照
## ============================================================
print("=" * 70)
print("1. PCA 中心化对照")
print("=" * 70)

m, d = X.shape
mu = X.mean(axis=0)
Xc = X - np.ones((m, 1)) @ mu.reshape(1, d)  # 中心化矩阵

## 中心化 SVD
Uc, sc, Vtc = np.linalg.svd(Xc, full_matrices=False)
Vc = Vtc.T
first_pc_centered = Vc[:, 0]

## k=2 截断重构
k = 2
Xc_k = Uc[:, :k] @ np.diag(sc[:k]) @ Vtc[:k, :]
err_centered = rel_fro_error(Xc, Xc_k)
err_centered_sq = np.linalg.norm(Xc - Xc_k, ord='fro') ** 2
eckart_young_sq = np.sum(sc[k:] ** 2)
```

```
## 不中心化 SVD
Uu, su, Vtu = np.linalg.svd(X, full_matrices=False)
Vu = Vtu.T
first_pc_uncentered = Vu[:, 0]
X_k_uncentered = Uu[:, :k] @ np.diag(su[:k]) @ Vtu[:k, :]
err_uncentered = rel_fro_error(X, X_k_uncentered)

print("列均值 mu =", mu)
print("中心化矩阵 Xc =\n", Xc)
print("中心化奇异值 sc =", sc)
print("中心化第一主成分方向 =", first_pc_centered)
print("中心化 k=2 截断重构 Xc_k =\n", Xc_k)
print("中心化 k=2 相对F范数误差 =", err_centered)
print("中心化误差平方 ||Xc-Xc_k||_F^2 =", err_centered_sq)
print("Eckart-Young sum_{i>k} sigma_i^2 =", eckart_young_sq)
print("Eckart-Young 核对差 =", err_centered_sq - eckart_young_sq)
print()
print("不中心化奇异值 su =", su)
print("不中心化第一主成分方向 =", first_pc_uncentered)
print("不中心化 k=2 相对F范数误差 =", err_uncentered)
print("两种第一主成分夹角(度) =", principal_angle_deg(first_pc_centered, first_pc_uncentered))
print("说明：不中心化 SVD 最大化 ||Xv||^2，含均值项 m|mu^T v|^2，")
print("      第一主成分会被均值方向牵引；中心化后才是最大方差方向。")
print()


## ============================================================
## 2. LoRA 维度与输出
## ============================================================
print("=" * 70)
print("2. LoRA 维度与输出")
print("=" * 70)

BA = B @ A
W = W0 + BA

rank_W0 = np.linalg.matrix_rank(W0)
rank_BA = np.linalg.matrix_rank(BA)
rank_W = np.linalg.matrix_rank(W)

delta_fro = np.linalg.norm(W - W0, ord='fro')
BA_fro = np.linalg.norm(BA, ord='fro')

d_out, d_in = W0.shape
r = B.shape[1]
trainable_params = r * (d_out + d_in)

y0 = W0 @ x_in
y_lora = W @ x_in
y_diff = y_lora - y0

## 对 W0 做 r=2 截断 SVD
Uw, sw, Vtw = np.linalg.svd(W0, full_matrices=False)
W0_2 = Uw[:, :r] @ np.diag(sw[:r]) @ Vtw[:r, :]
W0_2_fro_err = np.linalg.norm(W0 - W0_2, ord='fro')

print("B @ A =\n", BA)
print("W = W0 + BA =\n", W)
print("rank(W0) =", rank_W0)
print("rank(BA) =", rank_BA)
print("rank(W0 + BA) =", rank_W)
```

```
print("||W - W0||_F =", delta_fro)
print("||BA||_F =", BA_fro)
print("可训练参数数 r(d_out + d_in) =", trainable_params)
print("输入 x =", x_in)
print("W0 x =", y0)
print("(W0 + BA) x =", y_lora)
print("输出差异 (W0+BA)x - W0x =", y_diff)
print()
print("W0 的奇异值 =", sw)
print("W0 的 r=2 截断 SVD 重构 W0^(2) =\n", W0_2)
print("||W0 - W0^(2)||_F =", W0_2_fro_err)
print("对比: ||BA||_F =", BA_fro)
print("说明: BA 是训练增量，不是对 W0 的最优秩 r 逼近; ")
print("      截断 SVD 依赖 W0 本身，与训练无关。")
print()


## ============================================================
## 3. 可微求解器梯度核验
## ============================================================
print("=" * 70)
print("3. 可微求解器梯度核验")
print("=" * 70)

f_vec = np.array([1.0, 2.0, 3.0], dtype=np.float64)
x_star = np.zeros(3, dtype=np.float64)
theta0 = np.array([0.1, 0.2], dtype=np.float64)


def K_func(theta):
    t1, t2 = theta
    return np.array([
        [2.0 + t1, 1.0, 0.0],
        [1.0, 3.0, 1.0],
        [0.0, 1.0, 2.0 + t2],
    ], dtype=np.float64)


def loss_func(theta):
    K = K_func(theta)
    x = np.linalg.solve(K, f_vec)
    return 0.5 * np.linalg.norm(x - x_star) ** 2, x


## dK/dtheta_1, dK/dtheta_2
dK_dt1 = np.array([
    [1.0, 0.0, 0.0],
    [0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0],
], dtype=np.float64)

dK_dt2 = np.array([
    [0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0],
    [0.0, 0.0, 1.0],
], dtype=np.float64)

K0 = K_func(theta0)
x0 = np.linalg.solve(K0, f_vec)
ell0 = 0.5 * np.linalg.norm(x0 - x_star) ** 2
```

```
## 隐函数梯度：dl/dtheta_j = -(x-x*)^T K^{-1} (dK/dtheta_j) x
## 伴随求解：解 K^T lambda = -(x - x*)，K 对称故 K lambda = -(x - x*)
residual = x0 - x_star
lambda_adj = np.linalg.solve(K0, -residual)  # K lambda = -(x-x*)

grad_hidden = np.array([
    lambda_adj @ (dK_dt1 @ x0),
    lambda_adj @ (dK_dt2 @ x0),
], dtype=np.float64)

## 有限差分
eps = 1e-6
grad_fd = np.zeros(2, dtype=np.float64)
for j in range(2):
    theta_plus = theta0.copy()
    theta_minus = theta0.copy()
    theta_plus[j] += eps
    theta_minus[j] -= eps
    ell_plus, _ = loss_func(theta_plus)
    ell_minus, _ = loss_func(theta_minus)
    grad_fd[j] = (ell_plus - ell_minus) / (2.0 * eps)

rel_err = np.abs(grad_hidden - grad_fd) / (np.abs(grad_fd) + 1e-15)

print("theta0 =", theta0)
print("K(theta0) =\n", K0)
print("x = K^{-1} f =", x0)
print("loss =", ell0)
print("伴随变量 lambda = K^{-1}(-(x-x*)) =", lambda_adj)
print("隐函数梯度 [dl/dtheta1, dl/dtheta2] =", grad_hidden)
print("中心差分梯度 =", grad_fd)
print("相对误差 =", rel_err)
print("说明：伴随求解只需一次以 K 为系数的线性求解；")
print("      若直接对迭代求解器截断反传，只得到有限步迭代映射的导数，")
print("      忽略未展开迭代对 theta 的依赖，一般有偏。")
print()


## ============================================================
## 4. 故障注入：PCA 未中心化
## ============================================================
print("=" * 70)
print("4. 故障注入：PCA 未中心化")
print("=" * 70)

## 正确实现（中心化）
Uc, sc, Vtc = np.linalg.svd(Xc, full_matrices=False)
first_pc_correct = Vtc[0, :]  # 第一主成分方向
Xc_k_correct = Uc[:, :k] @ np.diag(sc[:k]) @ Vtc[:k, :]
err_correct = rel_fro_error(Xc, Xc_k_correct)

## 错误实现：仅删除减均值语句，其余不变
X_wrong = X.copy()  # 未中心化
Uw2, sw2, Vtw2 = np.linalg.svd(X_wrong, full_matrices=False)
first_pc_wrong = Vtw2[0, :]
X_k_wrong = Uw2[:, :k] @ np.diag(sw2[:k]) @ Vtw2[:k, :]

## 注意：错误实现若仍以 Xc 为“真值”算误差，会显得更大；
## 这里同时报告“相对于 X”的重构误差，便于比较。
err_wrong_rel_X = rel_fro_error(X, X_k_wrong)
```

```
angle = principal_angle_deg(first_pc_correct, first_pc_wrong)

print("正确（中心化）第一主成分 =", first_pc_correct)
print("错误（未中心化）第一主成分 =", first_pc_wrong)
print("第一主成分夹角(度) =", angle)
print("正确实现 k=2 相对误差（相对 Xc） =", err_correct)
print("错误实现 k=2 相对误差（相对 X） =", err_wrong_rel_X)
print("正确实现 k=2 相对误差（相对 X） =", rel_fro_error(X, Xc_k_correct))
print("错误实现 k=2 相对误差（相对 Xc） =", rel_fro_error(Xc, X_k_wrong))

## 断言检测
assert not np.allclose(first_pc_correct, first_pc_wrong, atol=1e-8), \
    "未中心化与中心化第一主成分应当不同"
print("断言通过：未中心化与中心化第一主成分不同。")
print()


## ============================================================
## 5. 适用条件测试：S 的正定性
## ============================================================
print("=" * 70)
print("5. 适用条件测试：S 的正定性")
print("=" * 70)

S = np.array([
    [1.0, 2.0],
    [2.0, 1.0],
], dtype=np.float64)

eigvals_S = np.linalg.eigvalsh(S)  # 对称矩阵用 eigvalsh
print("S =\n", S)
print("S 的特征值 =", eigvals_S)
print("是否正定 =", np.all(eigvals_S > 0))
print("是否半正定 =", np.all(eigvals_S >= 0))

if np.all(eigvals_S > 0):
    print("结论：S 正定，隐函数梯度公式成立，线性系统求解稳定。")
else:
    print("结论：S 不正定（有负特征值），不能直接套用“K 对称正定”下的隐函数梯度结论；")
    print("      若 S 奇异，K^{-1} 不存在；若 S 可逆但不定，x=K^{-1}f 仍可定义，")
    print("      但迭代求解可能不稳定，且伴随系统 K^T lambda = -(x-x*) 的条件数可能很差。")
```